GIÁO TRÌNH ĐÀO TẠO KỸ SƯ FRONTEND CHUYÊN SÂU

HỌC LẬP TRÌNH WEB

CHUYÊN SÂU FRONTEND

Từ Nền Tảng Đến Kiến Tạo Ứng Dụng Web Hiện Đại

■ HTML5 • ■ CSS3 Nâng Cao • ■ Modern JS ES6+ • ■ Web APIs

TÀI LIỆU HỌC TẬP & THỰC HÀNH CHUYÊN SÂU

Hệ thống hóa toàn bộ kiến thức Frontend chuẩn mực • Minh họa trực quan • Kiến thức thực chiến

ĐỘI NGŨ BIÊN SOẠN WEB Tài liệu lưu hành nội bộ

NĂM 2026 Phiên bản XeLaTeX Unicode

XÂY DỰNG NỀN TẢNG VỮNG CHẮC — CHINH PHỤC CÔNG NGHỆ FRONTEND HIỆN ĐẠI

ĐỘI NGŨ BIÊN SOẠN WEB

TÀI LIỆU LƯU HÀNH NỘI BỘ & THỰC HÀNH CHUYÊN SÂU

HỌC LẬP TRÌNH WEB

CHUYÊN SÂU FRONTEND

Từ Nền Tảng Đến Kiến Tạo Ứng Dụng Web Hiện Đại

GIÁO TRÌNH ĐÀO TẠO KỸ SƯ FRONTEND

Biên soạn chi tiết • Minh họa trực quan • Kiến thức thực chiến

NĂM 2026

Phiên bản XeLaTeX Unicode

MỤC LỤC TÀI LIỆU

I Nền Tảng Lập Trình Web & Tổng Quan Kiến Trúc **1**

1 Kiến Trúc Web & Các Khái Niệm Cốt Lõi **2**

1.1 Lịch Sử và Nguồn Gốc của Lập Trình Web 2

1.1.1 Những năm đầu của World Wide Web (1990 – 1995) 2

1.1.2 Sự phát triển của Web động (1995 – 2005) 3

1.1.3 Web 2.0 – Mạng xã hội và Web động (2005 – 2015) 4

1.1.4 Web 3.0 – Trí tuệ nhân tạo và Blockchain (2015 – Nay) 5

1.1.5 Tương lai của lập trình web 5

1.2 Mô Hình Client – Server (Client – Server Architecture) 6

1.2.1 Cách hoạt động của Mô hình Client – Server 6

1.2.2 Ví dụ thực tế: Truy cập Website Google 7

1.2.3 Tìm hiểu sâu về Server (Máy chủ) 7

1.2.4 Tìm hiểu sâu về Client (Máy khách) 8

1.3 Giao Thức Mạng (Network Protocol) 8

1.3.1 Khái niệm & Vai trò của Giao thức Mạng 8

1.3.2 Giao thức hoạt động như thế nào? 9

1.3.3 Bộ giao thức TCP/IP 9

1.3.4 Các Giao Thức Mạng Phổ Biến 10

1.3.5 Bảng so sánh tổng hợp các giao thức mạng 13

1.4 Địa Chỉ IP (IP Address) 13

1.5 Tên Miền (Domain Name) 14

1.5.1 Khái niệm Tên Miền 14

1.5.2 Tại sao cần tên miền? 14

1.5.3 Tên miền hoạt động như thế nào? 15

1.5.4 Cấu trúc của tên miền 15

1.5.5 Các cấp của tên miền 16

1.5.6 Phân loại tên miền 17

1.5.7 Quy trình đăng ký tên miền 17

1.5.8 Vai trò của hệ thống DNS đối với tên miền 17

1.5.9 Ví dụ tổng hợp phân tích tên miền 18

1.5.10 Mối quan hệ tổng hòa giữa Domain, DNS và IP 18

1.6 Cổng Dịch Vụ (Port) 19

1.7 URL (Uniform Resource Locator) . . . 19
1.7.1 Khái niệm URL . . . 19
1.7.2 Vai trò của URL . . . 20
1.7.3 Cấu trúc tổng quát của URL . . . 20
1.7.4 URL hoạt động như thế nào? . . . 23
1.7.5 Phân biệt URL Tuyệt Đối và URL Tương Đối . . . 23
1.7.6 Ví dụ tổng hợp phân tích cấu trúc URL . . . 24
1.8 Trang Web, Website & Dịch Vụ Hosting . . . 24
1.8.1 Phân biệt Web Page và Website . . . 24
1.8.2 Dịch vụ Hosting & Các loại Hosting . . . 24
1.9 Cách Trình Duyệt Render Trang (Critical Rendering Path) . . . 25
1.10 Lập Trình Front-End (Frontend Development) . . . 26
1.10.1 Các công nghệ chính trong lập trình Front-end . . . 26
1.10.2 Các thư viện và framework phổ biến . . . 26
1.10.3 Công việc của lập trình viên Front-end . . . 27
1.10.4 Sự khác biệt giữa Front-end và Back-end . . . 27
1.10.5 Học lập trình Front-end bắt đầu từ đâu? . . . 27
1.11 Khái Niệm UI và UX: Sự Khác Biệt & Mối Quan Hệ . . . 28
1.11.1 UI (User Interface) – Giao diện người dùng . . . 28
1.11.2 UX (User Experience) – Trải nghiệm người dùng . . . 28
1.11.3 Sự khác biệt giữa UI và UX . . . 29
1.11.4 Mối quan hệ giữa UI và UX . . . 29
1.11.5 Làm thế nào để cải thiện UI/UX? . . . 29
1.12 Tóm Tắt Chương 1 . . . 30

2 Môi Trường & Công Cụ Lập Trình Frontend Masterclass 31
2.1 Trình Soạn Thảo Mã Nguồn: Visual Studio Code (VS Code) . . . 31
2.1.1 Top Extension Quan Trọng Cho Frontend Developer . . . 31
2.1.2 Cấu Hình settings.json Chuẩn Cho VS Code & Prettier . . . 32
2.1.3 Bảng Tra Cứu Phím Tắt Nâng Cao Năng Suất Lập Trình . . . 32
2.1.4 Cơ Chế Chú Thích Mã Nguồn Trong VS Code (VS Code Commenting) . . . 32
2.2 Làm Chủ Chrome DevTools (Developer Tools Masterclass) . . . 34
2.2.1 5 Tab DevTools Cốt Lõi Không Thể Sống Thiếu . . . 34
2.3 Quản Lý Mã Nguồn Với Git & GitHub (Git Version Control System) . . . 34
2.3.1 Mô Hình Hoạt Động 4 Trạng Thái Của Git . . . 35
2.3.2 Quy Trình Làm Việc Chuẩn Với Git & GitHub Hàng Ngày . . . 35
2.3.3 Quy Chuẩn Viết Commit Message (Conventional Commits Standard) . . . 35
2.4 Tóm Tắt Chương 2 . . . 36

II HTML5 – Cấu Trúc & Định Dạng Siêu Văn Bản **37**

3 Tổng Quan HTML5, Cấu Trúc Tài Liệu & Thẻ <meta> Masterclass **38**

3.1 Tổng Quan Về HTML . . . 38
3.1.1 Cấu Trúc Giải Phẫu Một Phần Tử HTML (HTML Element Anatomy) . . . 39
3.2 Cấu Trúc Cơ Bản Của Một Tài Liệu HTML . . . 44
3.2.1 Giải Thích Chi Tiết 10 Thành Phần Trong Cấu Trúc HTML . . . 45
3.3 Chi Tiết Chuyên Sâu Về Thẻ <meta> Trong HTML (Meta Masterclass) . . . 46
3.3.1 Thẻ <meta> là gì? . . . 46
3.3.2 Các loại thẻ <meta> phổ biến & ứng dụng . . . 46
3.3.3 Ví dụ mã nguồn tệp HTML chứa bộ thẻ <meta> đầy đủ tiêu chuẩn . . . 49
3.4 Các Thẻ HTML Phổ Biến Trong Phát Triển Web . . . 50
3.4.1 Thẻ Tiêu Đề Trong HTML (Heading Tags: <h1> - <h6>) . . . 50
3.4.2 Thẻ Văn Bản Và Định Dạng Trong HTML (Text Formatting Tags) . . . 55
3.4.3 c. Thẻ liên kết (Anchor - <a>) . . . 66
3.4.4 d. Thẻ hình ảnh (Image - <img>) . . . 78
3.4.5 e. Lý Thuyết Về Đường Dẫn (Path) Trong HTML . . . 96
3.4.6 f. Thẻ Video Trong HTML (Video - <video>) . . . 106
3.4.7 h. Thẻ Danh Sách (<ul> và <ol>) Trong HTML . . . 113
3.4.8 i. Thẻ bảng (Table) . . . 121
3.5 Tổng Quan Sơ Lược Về Biểu Mẫu (HTML Forms Overview) . . . 122
3.6 HTML5 Và Các Tính Năng Mới Hiện Đại . . . 123
3.7 HTML Kết Hợp Với CSS và JavaScript . . . 123
3.8 Phân Loại Phần Tử HTML: Block, Inline & Inline-Block Masterclass . . . 124
3.8.1 Block Elements (Phần Tử Khối) . . . 124
3.8.2 Inline Elements (Phần Tử Nội Tuyến) . . . 126
3.8.3 Sự Khác Biệt Giữa Block Elements Và Inline Elements . . . 129
3.8.4 Chuyên Sâu Thẻ Và : Quy Tắc Sử Dụng & Anti-Patterns . . . 130
3.8.5 Inline-Block: Trung Gian Giữa Block Và Inline . . . 133
3.8.6 Quy Tắc Lồng Thẻ Chuẩn HTML5: Ranh Giới Block – Inline & Các Ngoại Lệ . . . 134
3.8.7 Kỹ Thuật Chuyển Đổi Kiểu Hiển Thị Bằng CSS (display Property) . . . 136
3.8.8 Chuyên Sâu Layout Engines: Block Formatting Context (BFC) & IFC . . . 137
3.8.9 Hiện Tượng Margin Collapsing (Gộp Lề Đứng) & Kỹ Thuật Xử Lý . . . 138
3.8.10 Giải Mã Khoảng Trắng 4px (Whitespace Gap) Của Inline-Block & Fix Triệt Để . . . 139
3.8.11 Bảng Tra Cứu So Sánh 5 Giá Trị display Cốt Lõi Trong CSS3 . . . 140
3.9 Thuộc Tính id Và class Trong HTML Masterclass . . . 140
3.9.1 Thuộc Tính id — Định Danh Duy Nhất Cho Một Phần Tử . . . 140
3.9.2 Thuộc Tính class — Nhóm Các Phần Tử Có Chung Thuộc Tính . . . 142
3.9.3 Bảng So Sánh Chi Tiết Giữa id Và class . . . 144
3.9.4 Khi Nào Nên Dùng id Và Khi Nào Nên Dùng class? . . . 144

3.9.5 Ví Dụ Thực Tế Kết Hợp id Và class Trong Mẫu Trang HTML5 145
3.9.6 Quy Tắc Đặt Tên & Lưu Ý Quan Trọng Tránh Lỗi Thực Tế 146
3.9.7 Tổng Kết Ma Trận Ra Quyết Định Khi Nào Dùng id Và class 148
3.10 Thực Thể HTML (HTML Entities) & Kỹ Thuật Escaping 148
3.11 Bảng (Table) Trong HTML5 & Định Dạng CSS Masterclass 149
3.11.1 1. Cấu Trúc Cơ Bản Của Thẻ <table> & Sơ Đồ Cấu Trúc 149
3.11.2 2. Bộ Thẻ Mở Rộng Trong Bảng (Semantics & Structure) 151
3.11.3 3. Thuộc Tính HTML Kế Thừa & Kỹ Thuật Gộp Ô (Colspan/Rowspan) 153
3.11.4 4. Định Dạng CSS Masterclass & Phân Tích Thuộc Tính Trọng Tâm 157
3.11.5 Phân Tích Chi Tiết Các Thuộc Tính Trọng Tâm 161
3.11.6 Đọc Thêm: Structural Semantics, Responsive Matrix & Accessibility 165
3.12 Thẻ Nhúng <iframe> (Inline Frame) Masterclass 173
3.12.1 Cú Pháp Khai Báo & Nguyên Lý Hoạt Động Của <iframe> 174
3.12.2 Bảng Tra Cứu Toàn Bộ Thuộc Tính Quan Trọng Của <iframe> 175
3.12.3 4 Trường Hợp Sử Dụng <iframe> Phổ Biến Trong Thực Tế Dự Án 175
3.12.4 Hướng Dẫn Chi Tiết Quy Trình Lấy Đường Dẫn Nhúng (Embed Link) 179
3.12.5 Cơ Chế Bảo Mật Tấn Công Clickjacking & Thuộc Tính sandbox 180
3.12.6 Kỹ Thuật Loại Bỏ Viền & Định Kiểu CSS Hiện Đại 181
3.12.7 Bảng Đánh Giá Ưu Điểm & Nhược Điểm Của <iframe> 182
3.12.8 Chuyên Sâu Thuộc Tính name & Kỹ Thuật Điều Hướng target 182
3.12.9 Phân Biệt Chi Tiết Thuộc Tính name Và id Trong <iframe> 184
3.13 Chi Tiết Về Các Thẻ Semantic Trong HTML 184
3.13.1 Thẻ Semantic Là Gì? 184
3.13.2 Lợi Ích Của Thẻ Semantic 185
3.13.3 Danh Sách Các Thẻ Semantic Và Cách Sử Dụng 185
3.13.4 Ví Dụ Sử Dụng Thực Tế Trong Một Trang Web 186
3.13.5 Tổng Kết: Vì Sao Nên Dùng Thẻ Semantic? 187
3.13.6 Phân Tích Chi Tiết Danh Sách Một Số Thẻ Semantic Phổ Biến 188
3.13.7 Phân Tích Chuyên Sâu 4 Lý Do Vì Sao Nên Dùng Thẻ Semantic 190
3.14 Đọc Thêm: Mở Rộng Kiến Thức Chuyên Sâu Về Semantic HTML5 192
3.14.1 Sự Tương Quan Giữa Thẻ Semantic HTML5 Và WAI-ARIA Landmark Roles . 192
3.14.2 Accessibility Tree (Cây Trợ Năng) Và Trình Đọc Màn Hình (Screen Readers) . . 193
3.14.3 Thuật Toán Phân Mạch Nội Dung (HTML5 Outlining Algorithm) 194
3.14.4 Những Lỗi Phổ Biến Và Anti-Patterns Cần Tránh Khi Dùng Thẻ Semantic . . . 194
3.14.5 Góc Phỏng Vấn Frontend: Các Câu Hỏi Thường Gặp Về Semantic HTML . . . 195
3.14.6 Nhóm Thẻ Semantic Mức Văn Bản (Text-Level / Inline Semantics) 195
3.14.7 3 Mô Hình Kiến Trúc Bố Cụ Bằng Semantic Tags Trong Thực Tế 197
3.14.8 Tích Hợp Semantic HTML Với Dữ Liệu Có Cấu Trúc SEO (Schema.org & Microdata) 200
3.14.9 Hướng Dẫn Debug Cây Trợ Năng (Accessibility Panel Inspection) 200

3.14.10 Bộ Thẻ Semantic HTML5 Hiện Đại Mới Nhất (<search>, <hgroup>, <meter> vs <progress>) . . . 201
3.14.11 Kỹ Thuật Responsive Image Semantics Với Thẻ <picture> . . . 202
3.14.12 Cấu Trúc Hộp Thoại Modal Native Với Thẻ <dialog> . . . 203
3.15 Tóm Tắt Chương 3 . . . 204

4 Thẻ Bố Cục Ngữ Nghĩa (Layout Semantics) & Thẻ Văn Bản 205
4.1 Thẻ Thân Tài Liệu <body> . . . 205
4.1.1 1. Lý Thuyết & Thuộc Tính Sự Kiện Window . . . 205
4.1.2 2. Ví Dụ Mã Nguồn Thực Tế . . . 205
4.2 Thẻ Đỉnh Trang / Đỉnh Phân Đoạn <header> . . . 205
4.2.1 1. Lý Thuyết & Quy Tắc Sử Dụng . . . 205
4.2.2 2. Ví Dụ Mã Nguồn Thực Tế . . . 206
4.3 Thẻ Thanh Điều Hướng <nav> . . . 206
4.3.1 1. Lý Thuyết & Accessibility (a11y) . . . 206
4.3.2 2. Ví Dụ Mã Nguồn Thực Tế . . . 206
4.4 Thẻ Nội Dung Chính <main> . . . 207
4.4.1 1. Lý Thuyết & Quy Tắc Độc Nhất . . . 207
4.4.2 2. Ví Dụ Mã Nguồn Thực Tế . . . 207
4.5 Thẻ Bài Viết Độc Lập <article> . . . 207
4.5.1 1. Lý Thuyết & Tiêu Chuẩn Tách Tải . . . 207
4.5.2 2. Ví Dụ Mã Nguồn Thực Tế . . . 208
4.6 Thẻ Phân Đoạn Chủ Đề <section> . . . 208
4.6.1 1. Lý Thuyết & So Sánh <section> vs <article> . . . 208
4.6.2 2. Ví Dụ Mã Nguồn Thực Tế . . . 208
4.7 Thẻ Thanh Bên Phụ <aside> . . . 209
4.7.1 1. Lý Thuyết & Mục Đích Sử Dụng . . . 209
4.7.2 2. Ví Dụ Mã Nguồn Thực Tế . . . 209
4.8 Thẻ Chân Trang <footer> . . . 210
4.8.1 1. Lý Thuyết & Thành Phần Bên Trong . . . 210
4.8.2 2. Ví Dụ Mã Nguồn Thực Tế . . . 210
4.9 Thẻ Thông Tin Liên Hệ <address> . . . 210
4.9.1 1. Lý Thuyết & Định Dạng Chuẩn . . . 210
4.9.2 2. Ví Dụ Mã Nguồn Thực Tế . . . 211
4.10 Nhóm Thẻ Tiêu Đề <h1> đến <h6> & <hgroup> . . . 211
4.10.1 1. Lý Thuyết & Cây Phân Tầng Tiêu Đề SEO . . . 211
4.10.2 2. Ví Dụ Mã Nguồn Thực Tế . . . 211
4.11 Thẻ Khối Chứa Tìm Kiếm <search> . . . 211
4.11.1 1. Lý Thuyết & Chuẩn WHATWG Mới . . . 212
4.11.2 2. Ví Dụ Mã Nguồn Thực Tế . . . 212

4.11.3 3. Đọc Thêm: Hệ Thống ARIA Landmarks & Accessibility Tree 212
4.12 Thẻ Đoạn Văn <p> & Khối Văn Bản Giữ Nguyên <pre> 213
4.12.1 1. Lý Thuyết & Khác Biệt Cốt Lõi 213
4.12.2 2. Ví Dụ Mã Nguồn Thực Tế 213
4.13 Thẻ Trích Dẫn Khối <blockquote> & Trích Dẫn Nội Dòng <q> 214
4.13.1 1. Lý Thuyết & Thuộc Tính cite 214
4.13.2 2. Ví Dụ Mã Nguồn Thực Tế 214
4.14 Các Thẻ Danh Sách <ol>, <ul>, <li>, <dl>, <dt>, <dd> 215
4.14.1 1. Lý Thuyết & Các Thuộc Tính Danh Sách 215
4.14.2 2. Ví Dụ Mã Nguồn Thực Tế 215
4.15 Thẻ Đường Kẻ Phân Cách <hr> 216
4.15.1 1. Lý Thuyết & Ý Nghĩa Ngữ Nghĩa 216
4.15.2 2. Ví Dụ Mã Nguồn Thực Tế 216
4.16 Thẻ Liên Kết Siêu Văn Bản <a> Masterclass 216
4.16.1 1. Lý Thuyết & Mục Đích Sử Dụng 216
4.16.2 2. Cú Pháp Cơ Bản & Phân Loại Các Loại Đường Dẫn (href) 217
4.16.3 3. Phân Tích Chuyên Sâu Về href="#" (Masterclass href="#") 218
4.16.4 4. Bảng Tra Cứu Các Thuộc Tính Quan Trọng Của Thẻ <a> 219
4.16.5 5. Thẻ <a> Với Nội Dung Khối (Clickable UI Cards) 220
4.16.6 6. Tạo Kiểu Thẻ <a> Bằng CSS & Pseudo-Classes (Quy Tắc LVHA) 220
4.17 Nhóm Thẻ Nhấn Mạnh Văn Bản , , <mark>, <small>, <s>, <del>, <ins> 221
4.17.1 1. Lý Thuyết & Mục Đích Ngữ Nghĩa 221
4.17.2 2. Ví Dụ Mã Nguồn Thực Tế 222
4.18 Nhóm Thẻ Kỹ Thuật <code>, <kbd>, <var>, <samp> 222
4.18.1 1. Lý Thuyết & Mục Đích Sử Dụng 222
4.18.2 2. Ví Dụ Mã Nguồn Thực Tế 222
4.19 Nhóm Thẻ Thời Gian & Mã Số <time> & <data> 222
4.19.1 1. Lý Thuyết & Thuộc Tính datetime/value 223
4.19.2 2. Ví Dụ Mã Nguồn Thực Tế 223
4.20 Thẻ Từ Viết Tắt <abbr> & Định Nghĩa <dfn> 223
4.20.1 1. Lý Thuyết & Thuộc Tính title 223
4.20.2 2. Ví Dụ Mã Nguồn Thực Tế 223
4.21 Nhóm Thẻ Đa Ngôn Ngữ & Ruby Annotations <bdi>, <bdo>, <ruby>, <rt>, <rp> . . . 223
4.21.1 1. Lý Thuyết & Chú Thích Phát Âm 224
4.21.2 2. Ví Dụ Mã Nguồn Thực Tế 224
4.22 Thẻ Chia Khối & Thẻ Inline 224
4.22.1 1. Lý Thuyết & Cảnh Báo Lạm Dụng Div/Span 224
4.22.2 2. Ví Dụ Mã Nguồn Thực Tế 224
4.23 Thẻ Khối Chú Thích Hình Ảnh & Sơ Đồ <figure> & <figcaption> 225

4.23.1 1. Lý Thuyết & Accessibility (a11y) . . . 225
4.23.2 2. Ví Dụ Mã Nguồn Thực Tế . . . 225
4.24 Thẻ Chỉ Số & Trích Dẫn Nguồn <sub>, <sup>, <cite> . . . 225
4.24.1 1. Lý Thuyết & Ứng Dụng Trong Khoa Học & Văn Bản . . . 225
4.24.2 2. Ví Dụ Mã Nguồn Thực Tế . . . 226
4.25 Thẻ Ngắt Dòng & Ngắt Từ Linh Hoạt
 & <wbr> . . . 226
4.25.1 1. Lý Thuyết & So Sánh Behavior . . . 226
4.25.2 2. Ví Dụ Mã Nguồn Thực Tế . . . 226
4.26 Thẻ Định Dạng Giao Diện Phân Biệt Ngữ Nghĩa <b>, <i>, <u> . . . 227
4.26.1 1. Lý Thuyết & Phân Biệt Với Strong / Em . . . 227
4.26.2 2. Ví Dụ Mã Nguồn Thực Tế . . . 227
4.26.3 3. Đọc Thêm: Phân Biệt Ngữ Nghĩa Trợ Năng (Screen Reader Sound Modulation) 228
4.27 Thẻ Danh Sách Nút Hành Động <menu> . . . 228
4.27.1 1. Lý Thuyết & Ngữ Nghĩa Toolbar . . . 228
4.27.2 2. Ví Dụ Mã Nguồn Thực Tế . . . 228
4.28 Tóm Tắt Chương 4 . . . 228

5 Biểu Mẫu Web (HTML5 Forms) & Các Ô Nhập Liệu 229
5.1 PHẦN 1: Nền Tảng Biểu Mẫu Web & Ô Nhập Liệu Cốt Lõi . . . 229
5.1.1 1. Cấu Trúc Cơ Bản & Thuộc Tính Thẻ <form> . . . 229
5.1.2 2. Phân Tích Chuyên Sâu Phương Thức Gửi Dữ Liệu GET vs POST (method) . 240
5.1.3 3. Chuyên Sâu Thẻ <input> & 22+ Loại Input Types Trong HTML5 . . . 249
5.1.4 4. Chuyên Sâu Thẻ <label>: Quy Tắc 2 Cách Dùng & Chuẩn Trợ Năng A11y . 256
5.1.5 5. Chuyên Sâu Thuộc Tính value & name Trong HTML Forms . . . 259
5.1.6 6. Phân Tích Chuyên Sâu Thuộc Tính placeholder, required & autocomplete . . 265
5.1.7 7. Các Thuộc Tính Kiểm Soát Nhập Liệu Advanced Trong HTML5 . . . 276
5.1.8 8. Gom Nhóm Biểu Mẫu Chuẩn Ngữ Nghĩa Với <fieldset> & <legend> . . . 277
5.1.9 9. So Sánh Chuyên Sâu Bộ Chọn <select> vs Bộ Gợi Ý Autocomplete <datalist> 284
5.1.10 10. Nhập Văn Bản Đa Dòng Với Thẻ <textarea> . . . 285
5.1.11 11. Thẻ Hiển Thị Kết Quả Tính Toán <output> . . . 285
5.1.12 12. Cơ Chế HTML5 Form Validation & Thuộc Tính Ràng Buộc . . . 286
5.1.13 13. Mã Nguồn Minh Họa Biểu Mẫu Đăng Ký Tổng Hợp . . . 288
5.2 PHẦN 2: Chuyên Sâu Từng Ô Nhập Liệu, Định Dạng CSS & Trợ Năng A11y . . . 291
5.3 Thẻ Ô Chọn Checkbox (<input type="checkbox">) . . . 291
5.3.1 Cú Pháp Cơ Bản & Thuộc Tính Thường Gặp . . . 291
5.3.2 Ví Dụ Đầy Đủ & Cơ Chế Gửi Dữ Liệu Lên Server . . . 292
5.3.3 So Sánh Checkbox name Giống Nhau (Gom Nhóm) vs name Khác Nhau (Độc Lập) . . . 293
5.3.4 Tối Ưu Trải Nghiệm Người Dùng Với Thẻ <label> . . . 296
5.3.5 Các Kỹ Thuật Sử Dụng Checkbox Nâng Cao . . . 297

5.3.6 Phân Biệt HTML Attribute checked vs DOM Property . . . 299
5.3.7 Bắt Buộc Chọn Ít Nhất 1 Checkbox Trong Nhóm . . . 299
5.3.8 Thu Thập Dữ Liệu Trong JavaScript Modern & React . . . 300
5.3.9 Cảnh Báo Bảo Mật (Security Notes) . . . 301
5.3.10 Minh Họa Sơ Đồ Luồng Xử Lý Dữ Liệu Checkbox Khi Submit . . . 302
5.3.11 Bảng Tổng Kết & Ghi Nhớ Cốt Lõi . . . 302
5.3.12 Đọc Thêm (Mở Rộng Kiến Thức Chuyên Sâu) . . . 303
5.3.13 Bộ Câu Hỏi Phỏng Vấn Frontend Thực Chiến (Interview Q&A) . . . 306
5.3.14 Bảng Độ Tương Thích Trình Duyệt & Best Practices Checklist . . . 307
5.4 Thẻ Nút Chọn Đơn Radio Button (<input type="radio">) . . . 308
5.4.1 Cú Pháp Cơ Bản & Bảng Thuộc Tính Thường Gặp . . . 308
5.4.2 Ví Dụ Đầy Đủ & Cơ Chế Gửi Dữ Liệu Lên Server . . . 309
5.4.3 Tối Ưu Trải Nghiệm Tương Tác Với Thẻ <label> . . . 310
5.4.4 Các Lưu Ý Quan Trọng & Lỗi Thường Gặp . . . 311
5.4.5 So Sánh Chi Tiết Checkbox vs Radio Button . . . 312
5.4.6 Minh Họa Sơ Đồ Luồng Xử Lý Radio Button . . . 313
5.4.7 Đọc Thêm (Mở Rộng Kiến Thức Chuyên Sâu) . . . 314
5.4.8 Bộ Câu Hỏi Phỏng Vấn Frontend Thực Chiến (Interview Q&A) . . . 315
5.4.9 Bảng Độ Tương Thích Trình Duyệt & Best Practices Checklist . . . 316
5.5 Các Thẻ Nút Bấm Trong Form (<input type="submit">, <input type="button">, <button>) . . . 317
5.5.1 Thẻ Nút Gửi Form (<input type="submit">) . . . 317
5.5.2 Thẻ Nút Bấm JavaScript (<input type="button">) . . . 319
5.5.3 Thẻ Nút Bấm Đa Năng Linh Hoạt (<button>) . . . 321
5.5.4 Thẻ Nút Khôi Phục Dữ Liệu Reset Form (<input type="reset"> & <button type="reset">) . . . 327
5.5.5 So Sánh Chi Tiết <input type="submit"> vs <button type="submit"> . . . 329
5.5.6 Minh Họa Sơ Đồ Luồng Xử Lý Submit Form Khi Nhấp Nút . . . 330
5.5.7 Đọc Thêm (Mở Rộng Kiến Thức Chuyên Sâu) . . . 330
5.5.8 Bộ Câu Hỏi Phỏng Vấn Frontend Thực Chiến (Interview Q&A) . . . 332
5.5.9 Bảng Độ Tương Thích Trình Duyệt & Best Practices Checklist . . . 333
5.6 Thẻ Ô Nhập Số (<input type="number">) . . . 334
5.6.1 Cú Pháp Cơ Bản & Bảng Thuộc Tính Thường Gặp . . . 334
5.6.2 Ví Dụ Minh Họa Chi Tiết . . . 335
5.6.3 Các Lưu Ý Quan Trọng & Bẫy Thường Gặp . . . 337
5.6.4 Bảng Tổng Kết & Ghi Nhớ Cốt Lõi . . . 338
5.6.5 Bộ Câu Hỏi Phỏng Vấn Frontend Thực Chiến (Interview Q&A) . . . 338
5.6.6 Đọc Thêm (Mở Rộng Kiến Thức Chuyên Sâu) . . . 339
5.7 Thẻ Ô Chọn Khoảng Giá Trị (<input type="range">) . . . 340
5.7.1 Cú Pháp Cơ Bản & Bảng Thuộc Tính Thường Gặp . . . 340

5.7.2 Ví Dụ Minh Họa Chi Tiết . . . 341
5.7.3 Các Lưu Ý Quan Trọng & Bẫy Thường Gặp . . . 343
5.7.4 Bảng Tổng Kết & Ghi Nhớ Cốt Lõi . . . 343
5.7.5 Bộ Câu Hỏi Phỏng Vấn Frontend Thực Chiến (Interview Q&A) . . . 343
5.7.6 Đọc Thêm (Mở Rộng Kiến Thức Chuyên Sâu) . . . 344
5.8 Thẻ Ô Nhập Văn Bản Nhiều Dòng Textarea (<textarea>) . . . 345
5.8.1 Cú Pháp Cơ Bản & Cấu Trúc Khai Báo . . . 345
5.8.2 Các Thuộc Tính Quan Trọng Của Thẻ <textarea> . . . 345
5.8.3 Các Giá Trị Mặc Định Của <textarea> Trong HTML . . . 346
5.8.4 So Sánh Textarea Mặc Định vs Textarea Cấu Hình Thuộc Tính . . . 347
5.8.5 Các Ví Dụ Minh Họa Chi Tiết Thực Tế . . . 348
5.8.6 Các Lưu Ý Quan Trọng Cần Ghi Nhớ Nằm Lòng . . . 351
5.8.7 Đọc Thêm: Kỹ Thuật Nâng Cao & Tối Ưu Trải Nghiệm Lập Trình Textarea . . . 352
5.9 Thẻ Danh Sách Thả Xuống Select (<select>, <option>, <optgroup>) . . . 353
5.9.1 Cú Pháp Cơ Bản & Cấu Trúc Khai Báo . . . 353
5.9.2 Các Thuộc Tính Quan Trọng Của Thẻ <select> . . . 353
5.9.3 Các Thuộc Tính Quan Trọng Của Thẻ <option> Bên Trong . . . 354
5.9.4 Cơ Chế Hoạt Động Của name và value Khi Submit Form . . . 355
5.9.5 Thiết Lập Tùy Chọn Mặc Định & Tùy Chọn Trống (Placeholder Option) . . . 357
5.9.6 Các Ví Dụ Minh Họa Chi Tiết Thực Tế . . . 358
5.9.7 Chi Tiết Về Thẻ Phân Nhóm Tùy Chọn <optgroup> . . . 359
5.9.8 Các Lưu Ý Quan Trọng Cần Ghi Nhớ Nằm Lòng . . . 363
5.9.9 Đọc Thêm: Kỹ Thuật Nâng Cao & Tối Ưu Lập Trình Dropdown . . . 363
5.10 Thay Đổi Font Chữ Mặc Định Trong Form HTML . . . 365
5.10.1 1. Phương Pháp Thay Đổi Font Đồng Bộ Cực Kỳ Đơn Giản . . . 365
5.10.2 2. Bảng Các Thuộc Tính Liên Quan Đến Font Chữ Trong Form . . . 366
5.10.3 3. Ví Dụ Mã Nguồn HTML & CSS Đầy Đủ . . . 367
5.11 Giả Lớp CSS :placeholder-shown & Kỹ Thuật Floating Label . . . 368
5.11.1 1. Cú Pháp Cơ Bản & Ví Dụ Minh Họa . . . 369
5.12 Giả Lớp CSS :not(:placeholder-shown) & Ứng Dụng Floating Label . . . 370
5.12.1 1. Mã Nguồn Form Validation Thuần CSS với :not(:placeholder-shown) . . . 371
5.12.2 2. Các Ứng Dụng Thực Tế Đỉnh Cao . . . 372
5.12.3 3. Đọc Thêm: Phân Biệt Phân Cấp Pseudo-class :placeholder-shown vs Pseudo-element ::placeholder . . . 373
5.13 Giả Lớp CSS :focus-within & Hiệu Ứng Focus Nhóm . . . 374
5.13.1 1. Cú Pháp Cơ Bản . . . 374
5.13.2 2. Ví Dụ Minh Họa Chi Tiết . . . 375
5.13.3 3. Ví Dụ 2: Khung Tìm Kiếm Tự Động Hiển Thị Menu Gợi Ý (Search Box Dropdown) . . . 377
5.13.4 4. Ví Dụ 3: Floating Label Màu Tím Neon Tối Ưu Tương Tác . . . 379

5.13.5 5. Ví Dụ 4: Giao Diện Multi-Card Form – Nổi Bật Thẻ Card Đang Thao Tác . . 381
5.14 Truy Cập Web Cho Biểu Mẫu (Form Accessibility - a11y) . . . 382
5.14.1 1. Các Quy Tắc Vàng Giúp Form Đạt Chuẩn WCAG . . . 382
5.14.2 2. Thuộc Tính Điều Hướng Bàn Phím tabindex Trong HTML . . . 383
5.14.3 3. Ví Dụ Minh Họa Chi Tiết Về Thứ Tự Tab . . . 384

6 Thẻ Đa Phương Tiện (Multimedia), Nhúng & Graphics 386
6.1 Thẻ Hình Ảnh `<img>` . . . 386
6.1.1 1. Lý Thuyết & Bảng Toàn Bộ Thuộc Tính Tối Ưu Hiệu Năng . . . 386
6.1.2 2. Ví Dụ Mã Nguồn Thực Tế . . . 386
6.2 Thẻ Nguồn Ảnh Thích Ứng `<picture>` & `<source>` . . . 387
6.2.1 1. Lý Thuyết & Kỹ Thuật Art Direction . . . 387
6.2.2 2. Ví Dụ Mã Nguồn Thực Tế . . . 387
6.3 Thẻ Bản Đồ Ảnh `<map>` & `<area>` . . . 387
6.3.1 1. Lý Thuyết & Thuộc Tính Tọa Độ . . . 387
6.3.2 2. Ví Dụ Mã Nguồn Thực Tế . . . 387
6.4 Thẻ Phát Video `<video>` . . . 388
6.4.1 1. Lý Thuyết & Bảng Thuộc Tính . . . 388
6.4.2 2. Ví Dụ Mã Nguồn Thực Tế . . . 388
6.5 Thẻ Phát Âm Thanh `<audio>` . . . 388
6.5.1 1. Lý Thuyết & Định Dạng Hỗ Trợ . . . 388
6.5.2 2. Ví Dụ Mã Nguồn Thực Tế . . . 388
6.6 Thẻ Nhúng Phụ Đề `<track>` . . . 388
6.6.1 1. Lý Thuyết & Chuẩn Phụ Đề WebVTT . . . 388
6.6.2 2. Ví Dụ Mã Nguồn Thực Tế . . . 388
6.7 Thẻ Khung Nhúng Trang Web `<iframe>` . . . 389
6.7.1 1. Lý Thuyết & Bảo Mật Sandbox . . . 389
6.7.2 2. Ví Dụ Mã Nguồn Thực Tế . . . 389
6.8 Thẻ Đồ Họa Vector Inline `<svg>` & Các Thẻ Con . . . 389
6.8.1 1. Lý Thuyết & Đồ Họa Độc Lập Độ Phân Giải . . . 389
6.8.2 2. Ví Dụ Mã Nguồn Thực Tế . . . 389
6.9 Thẻ Khung Vẽ Điểm Ảnh `<canvas>` . . . 390
6.9.1 1. Lý Thuyết & 2D Context API . . . 390
6.9.2 2. Ví Dụ Mã Nguồn Thực Tế . . . 390
6.10 Thẻ Công Thức Toán Học `<math>` (MathML) . . . 390
6.10.1 1. Lý Thuyết & Các Phần Tử MathML . . . 390
6.10.2 2. Ví Dụ Mã Nguồn Thực Tế . . . 390
6.11 Thẻ Nhúng Đối Tượng Khác `<object>`, `<embed>`, `<param>` . . . 390
6.11.1 1. Lý Thuyết & Nhúng File PDF . . . 391
6.11.2 2. Ví Dụ Mã Nguồn Thực Tế . . . 391

6.12 Tóm Tắt Chương 6 . . . 391

7 Thẻ Tương Tác Hiện Đại, Web Components, Global Attributes & SEO 392
7.1 Thẻ Accordion <details> & <summary> . . . 392
7.2 Thẻ Cửa Sổ Modal <dialog> . . . 393
7.3 Thẻ Popover API (popover, popovertarget) . . . 394
7.4 Thẻ Thước Đo <meter> & Tiến Trình <progress> . . . 395
7.5 Thẻ Mẫu Thụ Động <template> & Thẻ Điểm Chèn <slot> . . . 395
7.5.1 1. Lý Thuyết & Web Components Architecture . . . 395
7.5.2 2. Ví Dụ Mã Nguồn Thực Tế . . . 396
7.6 Tất Cả Thuộc Tính Toàn Cục (Global Attributes Masterclass) . . . 396
7.6.1 1. Lý Thuyết & Bảng Tra Cứu Toàn Bộ Thuộc Tính Toàn Cục . . . 396
7.6.2 2. Ví Dụ Mã Nguồn Thực Tế . . . 396
7.7 Bộ Thẻ Social Media Metadata (Open Graph & Twitter Cards) . . . 397
7.7.1 1. Lý Thuyết & Thẻ Hiển Thị Xem Trước Khi Chia Sẻ . . . 397
7.7.2 2. Ví Dụ Mã Nguồn Thực Tế . . . 397
7.8 Dữ Liệu Cấu Trúc Schema.org JSON-LD . . . 397
7.8.1 1. Lý Thuyết & Cú Pháp <script type="application/ld+json"> . . . 397
7.8.2 2. Ví Dụ Mã Nguồn Thực Tế . . . 397
7.9 Kỹ Thuật Bảo Mật Với HTML Header Tags . . . 398
7.9.1 1. Lý Thuyết & CSP, SRI . . . 398
7.9.2 2. Ví Dụ Mã Nguồn Thực Tế . . . 398
7.10 Tóm Tắt Chương 7 . . . 398

8 Phụ Lục A: Bách Khoa Tra Cứu Toàn Bộ 110+ Thẻ HTML & Thuộc Tính 399
8.1 1. Các Thẻ Khởi Tạo & Siêu Dữ Liệu (<head>) . . . 399
8.2 2. Các Thẻ Bố Cục & Ngữ Nghĩa Khối (Sectioning & Structural Semantics) . . . 400
8.3 3. Thẻ Định Dạng Văn Bản Nội Dòng (Inline Text Semantics) . . . 401
8.4 4. Thẻ Form & Điều Khiển Đầu Vào (Forms & Controls) . . . 402
8.5 5. Đa Phương Tiện, Graphics & Web Components . . . 403
8.6 6. Tất Cả Thuộc Tính Toàn Cục (Global Attributes) . . . 403

III CSS3 – Thiết Kế Giao Diện & Bố Cục Trang Web 405

9 Nền Tảng CSS: Cascading Style Sheets, Box Model & Thuộc Tính Outline 406
9.1 Sơ Lược & Khái Niệm Về CSS . . . 406
9.1.1 Mục Đích Cốt Lõi Của Ngôn Ngữ CSS . . . 406
9.2 Cú Pháp CSS (CSS Syntax) . . . 406
9.2.1 Cấu Trúc Cơ Bản Của Cú Pháp CSS . . . 406
9.2.2 Ví Dụ Cụ Thể & Giải Thích Cú Pháp . . . 407

9.3 Các Cách Thêm Style Vào Trang Web . . . 407
9.3.1 Inline CSS (CSS Trong Thẻ HTML) . . . 408
9.3.2 Internal CSS (CSS Nội Bộ) . . . 408
9.3.3 External CSS (CSS Bên Ngoài) . . . 409
9.4 Một Số Thuộc Tính CSS Cơ Bản & Đơn Vị Đo Kích Thước . . . 409
9.4.1 Bảng Tra Cứu Một Số Thuộc Tính CSS Cơ Bản . . . 410
9.4.2 Chuyên Đề Hệ Thống Màu Sắc Trong CSS (CSS Colors Masterclass Từ A đến Z) 411
9.4.3 Chuyên Đề Phông Chữ & Typography Trong CSS (Generic Font Families Masterclass) . . . 448
9.5 Các Loại Selectors Trong CSS (CSS Selectors Masterclass) . . . 457
9.5.1 Phân Loại Hệ Thống: 5 Nhóm CSS Selectors Cốt Lõi . . . 457
9.5.2 Bảng Tổng Quan Chi Tiết Các Bộ Chọn CSS Quan Trọng . . . 459
9.5.3 2. Chi Tiết Các Bộ Chọn Cơ Bản (Basic Selectors) . . . 460
9.5.4 3. Bộ Chọn Kết Hợp (Combinator Selectors) Chi Tiết & Ví Dụ Thực Tế . . . 469
9.5.5 4. Chuyên Đề Phân Biệt Chi Tiết: ”Phần Tử Cùng Cấp” & ”Phần Tử Con Trực Tiếp” (Masterclass Structural Combinators) . . . 498
9.5.6 5. So Sánh Tổng Hợp Chi Tiết Và Dễ Hiểu Nhất Về Tất Cả Các Bộ Chọn Kết Hợp Trong CSS . . . 507
9.5.7 6. Chuyên Đề Phân Tích Chi Tiết & Thực Nghiệm Về Bộ Chọn Thuộc Tính (Attribute Selectors Masterclass) . . . 511
9.5.8 7. Chuyên Đề Phân Tích Chi Tiết & Thực Nghiệm Về Bộ Chọn Lớp Giả (Pseudo-class Selectors Masterclass) . . . 518
9.5.9 8. Chuyên Đề Phân Tích Chi Tiết & Thực Nghiệm Về Bộ Chọn Phần Tử Giả (Pseudo-element Selectors Masterclass) . . . 603
9.6 Thứ Tự Ưu Tiên Trong CSS (CSS Specificity Masterclass) . . . 625
9.6.1 1. Bảng Xếp Hạng Chi Tiết Độ Ưu Tiên Specificity (6 Bậc Giá Trị) . . . 627
9.6.2 2. Quy Trình 4 Bước Duyệt DOM & Công Thức (A, B, C, D) / (a, b, c, d) . . . 627
9.6.3 3. Bảng Tra Cứu Specificity Của Các Bộ Chọn Thường Gặp (15 Ví Dụ) . . . 630
9.6.4 4. Những Quy Tắc Đặc Biệt Trong Specificity . . . 630
9.6.5 5. Phân Tích Chi Tiết: Khi Nào Quy Tắc ”Viết Sau Đè Viết Trước” Đúng Hoặc Sai? . . . 634
9.6.6 6. Bài Tập & Ví Dụ Phân Tích Chuyên Sâu Độ Ưu Tiên . . . 637
9.6.7 7. Pseudo-class :not(), :is() Và :where() Trong CSS Modern . . . 641
9.6.8 8. Sơ Đồ Tính Điểm Specificity Từng Bước (Step-by-Step) . . . 642
9.6.9 9. Kinh Nghiệm Thực Tế & Tối Ưu Kiến Trúc CSS . . . 642
9.7 Thuộc Tính Opacity (Độ Mờ & Độ Trong Suốt Trong CSS) . . . 643

10 Bố Cục Modern CSS: Flexbox & CSS Grid 650
10.1 Bố Cục Một Chiều Với CSS Flexbox . . . 650
10.2 Bố Cục Hai Chiều Với CSS Grid . . . 650

11 Responsive Web Design Nâng Cao **651**
11.1 Tư Duy Mobile-First . . . 651
11.2 Fluid Typography & Container Queries . . . 651

12 CSS Variables & BEM Architecture **652**
12.1 CSS Custom Properties (CSS Variables) . . . 652
12.2 Phương Pháp Đặt Tên BEM (Block - Element - Modifier) . . . 652

13 CSS Animations & Transitions **653**
13.1 Hiệu Ứng Chuyển Động Với CSS Transitions . . . 653
13.2 Keyframe Animations . . . 653

IV JavaScript Modern – Ngôn Ngữ Xương Sống Frontend **654**

14 Cơ Cơ Hoạt Động Của JavaScript Engine **655**
14.1 Execution Context & Call Stack . . . 655
14.2 Scope, Closure & Event Loop . . . 655

15 Modern JavaScript ES6+ **656**
15.1 Khai Báo Biến: const, let vs var . . . 656
15.2 Tính Năng ES6+ Đáng Giá . . . 656

16 Thao Tác DOM & Xử Lý Sự Kiện **657**
16.1 Tổng Quan Về DOM & Cấu Trúc Cây DOM Tree . . . 657
16.1.1 Ví Dụ Mã Nguồn Cấu Trúc DOM . . . 657
16.2 DOM Manipulation API Masterclass . . . 658
16.2.1 1. Các Phương Thức Truy Vấn Phần Tử (DOM Selection) . . . 658
16.2.2 2. Ví Dụ Mã Nguồn Truy Vấn & Thao Tác Nội Dung . . . 659
16.2.3 3. Tạo, Chèn Và Xóa Phần Tử Động (Dynamic DOM Node Building) . . . 659
16.3 Cơ Chế Sự Kiện: Bubbling, Capturing & Delegation . . . 660
16.3.1 1. Luồng Truyền Sự Kiện (Event Propagation Flow) . . . 660
16.3.2 2. Kỹ Thuật Uỷ Quyền Sự Kiện (Event Delegation Masterclass) . . . 661
16.4 Tóm Tắt & Bộ Câu Hỏi Phỏng Vấn DOM & Events . . . 662

17 Lập Trình Bất Đồng Bộ: Promises & Async/Await **663**
17.1 Từ Callback Hell Đến Promises . . . 663
17.2 Cú Pháp Hiện Đại Async/Await & Fetch API . . . 663

18 Lưu Trữ Trình Duyệt & Web APIs **664**
18.1 Web Storage API: LocalStorage & SessionStorage . . . 664
18.2 Cookies & IndexDB Overview . . . 664

V Dự Án Thực Chiến & Hệ Sinh Thái Frontend **665**

19 Build Tools & Package Managers **666**
19.1 Package Managers: npm, yarn & pnpm 666
19.2 Modern Build Tool: Vite 666

20 Kiến Trúc Frontend Modern & Frameworks Overview **667**
20.1 Single Page Application (SPA) vs Multi Page Application (MPA) 667
20.2 Kiến Trúc Dựa Trên Component (Component-Based Architecture) 667

21 Dự Án Thực Chiến Frontend Từ A-Z **668**
21.1 Phân Tích Yêu Cầu & Thiết Kế Bố Cục Dự Án 668
21.1.1 1. Cấu Trúc Thư Mục Dự Án Chuẩn Thực Tế 668
21.1.2 2. Bố Cục Giao Diện Kiến Trúc Grid & Flexbox 668
21.2 Triển Khai Mã Nguồn HTML, CSS & JavaScript 668
21.2.1 1. Mã Nguồn Cấu Trúc HTML5 Masterclass 669
21.2.2 2. Triển Khai Logic JavaScript Tương Tác (Search & Dark Mode) 671
21.3 Tóm Tắt & Checklist Đóng Gói Đưa Lên Production 672

PHẦN I

Nền Tảng Lập Trình Web & Tổng Quan Kiến Trúc

CHƯƠNG 1

Kiến Trúc Web & Các Khái Niệm Cốt Lõi

1.1 Lịch Sử và Nguồn Gốc của Lập Trình Web

Lập trình web đã phát triển vượt bậc trong hơn 30 năm qua, từ những trang web tĩnh đầu tiên cho đến các ứng dụng web động mạnh mẽ ngày nay. Dưới đây là cái nhìn tổng quan về lịch sử lập trình web.

1.1.1 Những năm đầu của World Wide Web (1990 – 1995)

[Bảng] Các Mốc Lịch Sử World Wide Web (1990 – 1995)

Năm	Sự kiện quan trọng
1989	Tim Berners-Lee đề xuất ý tưởng về World Wide Web (WWW) tại CERN.
1990	Berners-Lee phát triển trình duyệt web đầu tiên: WorldWideWeb (NeXT).
1991	Trang web đầu tiên ra đời tại CERN, chỉ có văn bản và các liên kết.
1993	Trình duyệt đồ họa đầu tiên Mosaic được phát triển, giúp web phổ biến hơn.
1994	Netscape Navigator ra mắt, thống trị thị trường trình duyệt.

Đặc điểm:

- Trang web hoàn toàn tĩnh, chỉ hiển thị văn bản và hình ảnh.
- Sử dụng HTML cơ bản để tạo các trang web đơn giản.

Ví dụ trang web đầu tiên:

Mã Nguồn HTML Trang Web Đầu Tiên

```html
<html>
  <head><title>Trang web đầu tiên</title></head>
  <body>
    <h1>Chào mừng đến với World Wide Web!</h1>
    <p>Đây là trang web đầu tiên trên thế giới.</p>
  </body>
</html>
```

[Ghi Chú] Bổ sung kiến thức: Sự ra đời của HTML

HTML (HyperText Markup Language) ban đầu do Tim Berners-Lee thiết kế năm 1991 chỉ bao gồm 18 thẻ cơ bản. Mục đích chính ban đầu của HTML chỉ là chia sẻ các tài liệu nghiên cứu khoa học có chứa liên kết siêu văn bản (*hyperlinks*) giữa các nhà vật lý tại CERN.

1.1.2 Sự phát triển của Web động (1995 – 2005)

[Bảng] Công Nghệ Động & Hệ CSDL Đầu Tiên (1995 – 2005)

Năm	Công nghệ quan trọng
1995	JavaScript ra đời bởi Netscape, giúp tạo hiệu ứng động.
1995	PHP (Personal Home Page) xuất hiện, hỗ trợ tạo web động.
1995	MySQL ra mắt, giúp quản lý cơ sở dữ liệu cho web.
1996	CSS1 ra đời, giúp định dạng giao diện web dễ dàng hơn.
2000	XHTML ra mắt, giúp chuẩn hóa HTML.
2003	WordPress được phát triển, giúp tạo blog dễ dàng.

Đặc điểm:

- Web bắt đầu có tính tương tác nhờ JavaScript.
- Sự xuất hiện của PHP + MySQL giúp xây dựng website động.
- CSS giúp tùy chỉnh giao diện dễ dàng hơn.

Ví dụ trang web động với PHP:

Mã Nguồn PHP Minh Họa Trang Web Động

```php
<?php
echo "Chào mừng! Hôm nay là " . date("Y-m-d");
?>
```

[Mẹo] Kỷ nguyên LAMP Stack

Trong giai đoạn này, bộ tứ công nghệ **LAMP** (Linux, Apache, MySQL, PHP) đã trở thành tiêu chuẩn vàng thống trị ngành phát triển web động, là nền tảng khởi đầu cho các hệ quản trị nội dung (CMS) khổng lồ như WordPress, Joomla và Drupal.

1.1.3 Web 2.0 – Mạng xã hội và Web động (2005 – 2015)

[Bảng] Kỷ Nguyên Web 2.0 & Mạng Xã Hội (2005 – 2015)

Năm	Công nghệ quan trọng
2005	AJAX giúp web tải dữ liệu mà không cần tải lại trang.
2006	Facebook API ra đời, mở ra kỷ nguyên mạng xã hội.
2008	Google Chrome ra mắt, hỗ trợ tốt JavaScript.
2009	Node.js xuất hiện, giúp viết backend bằng JavaScript.
2010	HTML5 và CSS3 ra mắt, cải tiến mạnh về đồ họa và video.

Đặc điểm:

- Web 2.0 xuất hiện với mạng xã hội như Facebook, Twitter.
- AJAX giúp web trở nên nhanh và mượt hơn.
- HTML5 & CSS3 giúp phát triển web mạnh mẽ hơn.

Ví dụ AJAX gửi yêu cầu mà không tải lại trang:

Yêu Cầu AJAX Sử Dụng Fetch API

```javascript
fetch("server.php")
  .then(response => response.text())
  .then(data => console.log(data));
```

[Khái Niệm] Bước ngoặt AJAX & Node.js

AJAX (Asynchronous JavaScript and XML) xóa bỏ trải nghiệm "trắng màn hình" mỗi khi người dùng click chuột. Trong khi đó, **Node.js** (2009) dựa trên V8 Engine của Chrome đã biến JavaScript từ ngôn ngữ kịch bản chạy ở trình duyệt thành ngôn ngữ lập trình đa nền tảng, cho phép lập trình viên dùng duy nhất JS cho cả Frontend và Backend (Full-stack JS).

1.1.4 Web 3.0 – Trí tuệ nhân tạo và Blockchain (2015 – Nay)

[Bảng] Kỷ Nguyên Web 3.0, AI & Decentralized Apps (2015 – Nay)

Năm	Công nghệ quan trọng
2015	React.js và Vue.js xuất hiện, giúp phát triển SPA (Single Page Application).
2018	WebAssembly cho phép chạy mã máy trên trình duyệt.
2020	AI, Chatbot, Blockchain tích hợp vào web.
2022	Web 3.0 với ứng dụng phi tập trung (DApps) phát triển mạnh.

Đặc điểm:

- SPA (Single Page Application): Web hoạt động mượt mà như ứng dụng di động.
- AI & Machine Learning giúp cá nhân hóa trải nghiệm web.
- Web 3.0: Dữ liệu phân tán, bảo mật tốt hơn nhờ blockchain.

Ví dụ React.js tạo trang web hiện đại:

Component React.js Hiện Đại

```
function App() {
  return <h1>Chào mừng đến với Web 3.0!</h1>;
}
```

1.1.5 Tương lai của lập trình web

- **AI và Trí tuệ nhân tạo**: Các website sẽ trở nên thông minh hơn.
- **Web 3.0 & Blockchain**: Mạng phi tập trung giúp bảo mật tốt hơn.
- **VR/AR Web**: Web sẽ hỗ trợ thực tế ảo (VR) và thực tế tăng cường (AR).
- **No-code & Low-code**: Xây dựng web mà không cần code nhiều.

[Ghi Chú] Kết luận

Lập trình web đã phát triển mạnh mẽ trong hơn 30 năm và sẽ còn tiến xa hơn trong tương lai.

1.2 Mô Hình Client – Server (Client – Server Architecture)

Mô hình **Client – Server** là một mô hình kiến trúc mạng nền tảng, trong đó các máy tính được phân chia chức năng thành hai nhóm chính:

- **Server (Máy chủ)**: Là máy tính chuyên nghiệp cung cấp tài nguyên, dữ liệu hoặc dịch vụ.
- **Client (Máy khách)**: Là máy tính hoặc thiết bị gửi yêu cầu (*Request*) đến Server để sử dụng tài nguyên/dịch vụ đó.

Thay vì mỗi máy tính phải tự lưu trữ toàn bộ dữ liệu, Client chỉ cần gửi yêu cầu (*Request*), còn Server đảm nhận việc xử lý yêu cầu và trả về kết quả (*Response*).



Hình 1.1: Mô hình Kiến trúc Client – Server

1.2.1 Cách hoạt động của Mô hình Client – Server

Quá trình giao tiếp giữa Client và Server diễn ra theo chu kỳ 5 bước chính:

[Khái Niệm] Quy Trình 5 Bước Giao Tiếp Client – Server

Bước 1: **Client gửi yêu cầu đến Server**: Ví dụ như mở website, đăng nhập tài khoản, xem sản phẩm hoặc gửi tin nhắn.

Bước 2: **Server tiếp nhận yêu cầu**: Máy chủ lắng nghe cổng mạng và nhận gói tin yêu cầu.

Bước 3: **Server xử lý dữ liệu**: Thực hiện truy vấn CSDL, kiểm tra quyền tài khoản, tính toán logic hoặc đọc file.

Bước 4: **Server gửi kết quả về Client**: Đóng gói dữ liệu kết quả và trả về qua phản hồi (*Response*).

Bước 5: **Client hiển thị kết quả**: Trình duyệt hoặc ứng dụng hiển thị dữ liệu phản hồi cho người dùng.

1.2.2 Ví dụ thực tế: Truy cập Website Google

Khi người dùng nhập địa chỉ `https://google.com` vào trình duyệt:

1. Trình duyệt (Client) gửi yêu cầu (*Request*) đến máy chủ Google.
2. Máy chủ Google nhận yêu cầu.
3. Google xử lý yêu cầu tìm kiếm.
4. Máy chủ trả về trang HTML.
5. Trình duyệt hiển thị giao diện Google cho người dùng xem.

Người dùng chỉ nhìn thấy kết quả cuối cùng mà không cần biết Server đã xử lý bên dưới phức tạp ra sao.

1.2.3 Tìm hiểu sâu về Server (Máy chủ)

Khái niệm:

Server là máy tính hoặc hệ thống máy tính có nhiệm vụ **cung cấp dịch vụ, dữ liệu hoặc tài nguyên** cho nhiều Client cùng lúc thông qua kết nối mạng.

Đặc điểm của Server:

- Hoạt động liên tục 24/7/365 không ngừng nghỉ.
- Cấu hình phần cứng chuyên dụng (CPU nhiều nhân, RAM lớn, ổ SSD NVMe tốc độ cao).
- Kết nối mạng băng thông rộng cao và ổn định.
- Chạy hệ điều hành tối ưu cho máy chủ (*Linux Server, Windows Server*).

[Bảng] Các Loại Server Phổ Biến Trong Ngành Web

Loại Server	Nhiệm vụ chính	Phần mềm / Công nghệ phổ biến
Web Server	**Lưu trữ & trả về HTML, CSS, JS**	Apache, Nginx, IIS, LiteSpeed
Database Server	**Lưu trữ & quản lý CSDL**	MySQL, PostgreSQL, SQL Server, Oracle
File Server	**Chia sẻ tập tin trong mạng**	NAS, Samba, FTP Server
Mail Server	**Xử lý gửi & nhận Email**	Microsoft Exchange, Postfix, Exim
Application Server	**Chạy ứng dụng doanh nghiệp**	Tomcat, WildFly, GlassFish, Node.js

[Mẹo] Một Máy Chủ Có Thể Đảm Nhận Nhiều Nhiệm Vụ

Một máy chủ hoàn toàn có thể chạy đồng thời Web Server, Database Server và Mail Server. Tuy nhiên, trong các hệ thống lớn, người ta thường tách riêng từng dịch vụ ra các Server độc lập để tối ưu hiệu năng và dễ dàng mở rộng (*Scaling*).

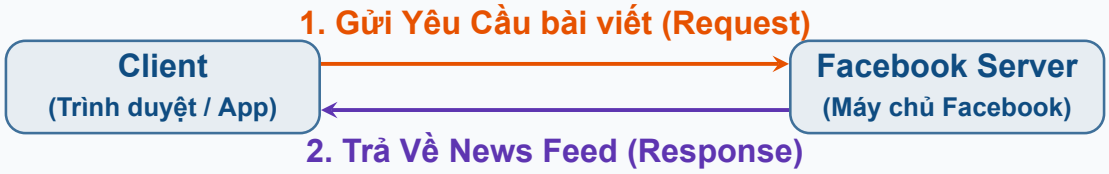
1.2.4 Tìm hiểu sâu về Client (Máy khách)

Khái niệm:

Client là thiết bị hoặc ứng dụng tiêu thụ dịch vụ do Server cung cấp. Client bao gồm: Máy tính bàn, Laptop, Điện thoại thông minh, Máy tính bảng, Smart TV, thiết bị IoT.

Nhiệm vụ của Client:

- Gửi yêu cầu (*Request*) đến Server.
- Nhận dữ liệu phản hồi (*Response*) từ Server.
- Render và hiển thị giao diện cho người dùng tương tác.



Hình 1.2: Luồng Truy Vấn Facebook giữa Client và Server

[Ghi Chú] Một Máy Tính Vừa Là Client Vừa Là Server

Khi lập trình web local trên máy cá nhân, máy tính của bạn vừa đóng vai trò **Client** (mở trình duyệt xem web) vừa đóng vai trò **Server** (chạy môi trường XAMPP, Node.js hoặc Python local server).

1.3 Giao Thức Mạng (Network Protocol)

1.3.1 Khái niệm & Vai trò của Giao thức Mạng

Giao thức mạng (Network Protocol) là tập hợp các quy tắc, tiêu chuẩn và quy ước được thiết lập để các thiết bị trong mạng có thể giao tiếp, trao đổi dữ liệu với nhau một cách chính xác và thống nhất.

Nói cách khác, giao thức mạng giống như **ngôn ngữ chung** mà các thiết bị phải tuân theo để có thể hiểu nhau. Nếu mỗi thiết bị sử dụng một cách giao tiếp khác nhau, dữ liệu sẽ không thể được truyền nhận hoặc sẽ bị hiểu sai.

Ví dụ, khi một người Việt nói chuyện với một người Anh, cả hai cần sử dụng một ngôn ngữ chung (chẳng hạn tiếng Anh) để hiểu nhau. Trong mạng máy tính cũng vậy, các thiết bị phải sử dụng cùng một giao thức để truyền và nhận dữ liệu.

Giao thức mạng đóng vai trò nền tảng trong mọi hoạt động của Internet và mạng máy tính. Nó quy định toàn bộ quá trình truyền thông giữa các thiết bị, bao gồm:

- **Thiết lập connection**: Xác định cách thiết lập kết nối giữa các máy tính.
- **Đóng gói & Phân đoạn**: Quy định cách đóng gói dữ liệu và chia thành các gói tin (*Packets*).
- **Định tuyến địa chỉ**: Xác định chính xác địa chỉ gửi (*Source IP*) và địa chỉ nhận (*Destination IP*).

- **Kiểm soát lỗi & Tốc độ**: Kiểm tra tính toàn vẹn dữ liệu, phát hiện/xử lý lỗi và điều khiển tốc độ truyền nhằm tránh tắc nghẽn mạng.
- **Giải phóng kết nối**: Quy định quy trình kết thúc kết nối sau khi hoàn tất truyền nhận.

Nhờ các giao thức mạng, dữ liệu có thể được truyền đi một cách chính xác, an toàn và hiệu quả ngay cả khi phải đi qua nhiều thiết bị trung gian như router, switch hoặc firewall.

1.3.2 Giao thức hoạt động như thế nào?

Giả sử người dùng mở trình duyệt và truy cập địa chỉ `https://www.google.com`. Quá trình truyền dữ liệu diễn ra qua 10 bước phối hợp nhịp nhàng giữa các tầng giao thức:

[Khái Niệm] Quy Trình 10 Bước Phối Hợp Giữa Các Giao Thức Mạng

Bước 1: **Nhập URL**: Người dùng nhập đường dẫn `https://www.google.com` trên trình duyệt.

Bước 2: **Gửi Yêu Cầu HTTPS**: Trình duyệt khởi tạo giao thức HTTPS để chuẩn bị gửi yêu cầu bảo mật.

Bước 3: **Thiết Lập Kết Nối TCP**: HTTPS sử dụng tầng giao vận TCP để mở kết nối 3 bước (*3-way handshake*) với máy chủ.

Bước 4: **Chia Gói Tin (Packets)**: TCP cắt nhỏ dữ liệu thành nhiều gói tin và đánh số thứ tự từng gói.

Bước 5: **Định Tuyến IP**: Giao thức IP gán địa chỉ nguồn/đích và định tuyến các gói tin qua mạng Internet.

Bước 6: **Máy Chủ Nhận Gói Tin**: Máy chủ Google tiếp nhận các gói tin truyền tới từ mạng.

Bước 7: **Máy Chủ Xử Lý**: Máy chủ Google phân tích và xử lý yêu cầu tìm kiếm của người dùng.

Bước 8: **Máy Chủ Trả Dữ Liệu**: Máy chủ đóng gói dữ liệu HTML, CSS, JavaScript gửi phản hồi lại cho Client.

Bước 9: **Tái Hợp Gói Tin**: Tầng TCP ở phía Client kiểm tra lỗi và ghép các gói tin lại theo đúng thứ tự.

Bước 10: **Hiển Thị Trang Web**: Trình duyệt đọc mã HTML/CSS/JS và vẽ ra giao diện web hoàn chỉnh.

Có thể thấy, chỉ một thao tác đơn giản là mở một trang web nhưng đã có rất nhiều giao thức phối hợp chặt chẽ với nhau để hoàn thành quá trình truyền dữ liệu.

1.3.3 Bộ giao thức TCP/IP

Internet hiện nay hoạt động dựa trên **bộ giao thức TCP/IP (Transmission Control Protocol / Internet Protocol)**. TCP/IP không phải là một giao thức duy nhất mà là một **tập hợp nhiều giao thức khác nhau**, mỗi giao thức đảm nhận một chức năng riêng.

Mô hình TCP/IP tiêu chuẩn được chia thành bốn tầng chính:

[Bảng] Mô Hình 4 Tầng Của Bộ Giao Thức TCP/IP

Tầng (Layer)	Chức năng & Giao thức tiêu biểu
Application Layer (Ứng dụng)	Cung cấp giao diện dịch vụ cho người dùng (HTTP, HTTPS, FTP, SMTP, DNS).
Transport Layer (Giao vận)	Quản lý kết nối truyền dữ liệu đầu-cuối giữa hai thiết bị (TCP, UDP).
Internet Layer (Mạng)	Định tuyến và chuyển tiếp các gói tin qua mạng Internet (IP, ICMP).
Network Access (Truy nhập)	Truyền dữ liệu tín hiệu trên môi trường vật lý (Ethernet, Wi-Fi).

Các tầng này phối hợp với nhau để đảm bảo dữ liệu được truyền từ thiết bị gửi đến thiết bị nhận một cách chính xác.

1.3.4 Các Giao Thức Mạng Phổ Biến

Giao thức TCP (Transmission Control Protocol)

TCP là giao thức truyền dữ liệu đáng tin cậy nhất trên Internet với các đặc điểm cốt lõi:

- Thiết lập kết nối trước khi truyền dữ liệu (*Connection-oriented*).
- Chia dữ liệu thành nhiều gói tin và đánh số thứ tự từng gói.
- Kiểm tra lỗi, tự động yêu cầu truyền lại gói tin bị mất và ghép dữ liệu đúng thứ tự tại máy nhận.

Ứng dụng tiêu biểu: Được dùng trong Website, Email, Ngân hàng điện tử, Mạng xã hội và các Hệ thống quản lý dữ liệu nơi tính chính xác của dữ liệu là ưu tiên hàng đầu.

[Mẹo] Ví dụ thực tế về TCP

Khi tải một tệp PDF, nếu một gói tin bị mất trên đường truyền, TCP sẽ tự động yêu cầu gửi lại gói tin đó thay vì tiếp tục với dữ liệu bị thiếu, đảm bảo file PDF sau khi tải về không bị hỏng.

Giao thức UDP (User Datagram Protocol)

UDP là giao thức truyền dữ liệu không yêu cầu thiết lập kết nối trước và không kiểm tra việc dữ liệu có đến nơi hay không.

- Không thiết lập kết nối trước khi truyền, không kiểm tra lỗi hay truyền lại gói tin bị mất.
- Tốc độ truyền cực nhanh và độ trễ (*latency*) rất thấp.

Ứng dụng tiêu biểu: Phù hợp cho Xem video trực tuyến, Livestream, Họp trực tuyến, Chơi game online và Cuộc gọi VoIP – những ứng dụng ưu tiên tốc độ thời gian thực hơn độ chính xác tuyệt đối.

[Mẹo] Ví dụ thực tế về UDP

Khi xem một trận bóng đá trực tiếp, nếu mất một vài khung hình thì người xem vẫn có thể tiếp tục theo dõi trận đấu. Tuy nhiên, nếu phải chờ truyền lại từng gói tin bị mất, hình ảnh sẽ bị giật và trải nghiệm sẽ kém hơn. Vì vậy UDP được ưu tiên trong các ứng dụng thời gian thực.

Giao thức IP (Internet Protocol)

IP là giao thức chịu trách nhiệm xác định địa chỉ và định tuyến các gói tin trên Internet. Nhiệm vụ của IP bao gồm:

- Gán địa chỉ IP cho thiết bị, xác định địa chỉ nguồn và địa chỉ đích.
- Chọn đường đi tối ưu cho gói tin qua các router trung gian và chuyển tiếp dữ liệu đến đúng máy nhận.

Lưu ý: IP không đảm bảo dữ liệu sẽ đến nơi, nhiệm vụ kiểm soát tin cậy này do tầng TCP đảm nhận.

[Ghi Chú] Ví dụ định tuyến IP

Một gói tin từ Việt Nam đến máy chủ ở Hoa Kỳ có thể phải đi qua nhiều quốc gia và nhiều router khác nhau. Giao thức IP chịu trách nhiệm tìm đường đi phù hợp cho gói tin.

Giao thức HTTP (HyperText Transfer Protocol)

HTTP là giao thức dùng để truyền tải tài liệu siêu văn bản (HyperText) dùng cho các trang web, hoạt động theo mô hình **Request – Response**:

1. Trình duyệt gửi yêu cầu (*HTTP Request*).
2. Máy chủ xử lý yêu cầu và phản hồi (*HTTP Response*).
3. Trình duyệt nhận và hiển thị nội dung.

Ví Dụ Một Cặp HTTP Request & Response

```
GET /index.html HTTP/1.1
Host: example.com

HTTP/1.1 200 OK
Content-Type: text/html
```

[Cảnh Báo] Cảnh Báo An Toàn Của HTTP

HTTP không mã hóa dữ liệu, vì vậy thông tin truyền trên mạng có thể bị đọc hoặc chỉnh sửa nếu bị kẻ xấu chặn bắt.

Giao thức HTTPS (HyperText Transfer Protocol Secure)

HTTPS là phiên bản bảo mật của HTTP, kết hợp giữa **HTTP** và chứng chỉ **SSL/TLS** (*Secure Sockets Layer / Transport Layer Security*).

Ưu điểm vượt trội:

- Mã hóa toàn bộ dữ liệu truyền giữa trình duyệt và máy chủ, ngăn chặn nghe lén.
- Bảo vệ thông tin tài khoản, mật khẩu và giao dịch ngân hàng.
- Chống giả mạo máy chủ và đảm bảo tính toàn vẹn của dữ liệu.

Khi truy cập website sử dụng HTTPS (ví dụ: `https://google.com`), trình duyệt hiển thị biểu tượng ổ khóa an toàn. Hầu hết các website hiện đại ngày nay đều bắt buộc sử dụng HTTPS.

Giao thức FTP (File Transfer Protocol)

FTP là giao thức dùng để truyền tập tin giữa máy tính và máy chủ, thường dùng để: Upload mã nguồn website lên Hosting, tải dữ liệu hoặc quản lý tệp trên máy chủ từ xa qua các phần mềm như FileZilla, WinSCP, Cyberduck.

> **[Mẹo] Ví dụ thực tế sử dụng FTP**
>
> Một lập trình viên sau khi hoàn thành website sẽ sử dụng FileZilla để tải toàn bộ mã nguồn từ máy tính cá nhân lên Hosting thông qua giao thức FTP.

Giao thức SMTP (Simple Mail Transfer Protocol)

SMTP là giao thức chuyên dụng chịu trách nhiệm gửi thư điện tử (Email) từ máy tính người gửi đến máy chủ thư và chuyển tiếp Email giữa các máy chủ thư với nhau.

> **[Ghi Chú] Ví dụ về SMTP**
>
> Khi gửi Email bằng Gmail, SMTP sẽ chuyển nội dung thư từ máy tính của bạn đến máy chủ Gmail và sau đó đến máy chủ của người nhận.

Giao thức POP3 & IMAP (Nhận Email)

POP3 và IMAP là hai giao thức chịu trách nhiệm nhận Email từ máy chủ về Client:

- **POP3 (Post Office Protocol v3)**: Tải Email từ máy chủ về máy tính cá nhân và có thể xóa thư trên máy chủ sau khi tải. Phù hợp khi chỉ sử dụng một thiết bị duy nhất.
- **IMAP (Internet Message Access Protocol)**: Đồng bộ Email liên tục trên nhiều thiết bị (điện thoại, máy tính) và giữ nguyên thư trên máy chủ.

Hiện nay, hầu hết các dịch vụ Email hiện đại đều mặc định sử dụng giao thức IMAP.

Hệ Thống Phân Giải Tên Miền DNS (Domain Name System)

DNS là hệ thống dịch chuyển tên miền dễ nhớ (ví dụ: `www.google.com`) thành địa chỉ IP số (ví dụ: `142.250.xxx.xxx`) để máy tính có thể định tuyến đến đúng máy chủ. Quá trình phân giải này diễn ra tự động và tức thì mỗi khi bạn gõ tên miền trên trình duyệt.

1.3.5 Bảng so sánh tổng hợp các giao thức mạng

[Bảng] Bảng So Sánh Chi Tiết 10 Giao Thức Mạng Phổ Biến

Giao thức	Chức năng	Có mã hóa	Ví dụ sử dụng
TCP	Truyền dữ liệu tin cậy	Không	Website, Email, Ngân hàng
UDP	Truyền dữ liệu nhanh	Không	Game, Livestream, VoIP
IP	Định tuyến gói tin	Không	Mọi kết nối Internet
HTTP	Truyền trang web	Không	Website cũ
HTTPS	Truyền trang web an toàn	Có	Website hiện đại, thương mại điện tử
FTP	Truyền tập tin	Không (FTP truyền thống)	Upload website
SMTP	Gửi Email	Có thể kết hợp TLS	Gửi thư điện tử
POP3	Tải Email về thiết bị	Có thể kết hợp TLS	Nhận Email trên một thiết bị
IMAP	Đồng bộ Email	Có thể kết hợp TLS	Đồng bộ Email trên nhiều thiết bị
DNS	Phân giải tên miền thành địa chỉ IP	Không (DNS truyền thống, có DoH/DoT)	Truy cập website

[Ghi Chú] Tổng Kết Giao Thức Mạng

Giao thức mạng là nền tảng của mọi hoạt động trên Internet. Khi người dùng truy cập một website, nhiều giao thức sẽ phối hợp với nhau: **DNS** phân giải tên miền thành địa chỉ IP, **TCP** thiết lập kết nối đáng tin cậy, **IP** định tuyến các gói tin, **HTTPS** mã hóa và truyền dữ liệu giữa trình duyệt và máy chủ. Nhờ sự kết hợp này, dữ liệu được truyền đi chính xác, an toàn và hiệu quả. Đây cũng là nền tảng để các ứng dụng web, ứng dụng di động và hầu hết các dịch vụ Internet hiện đại hoạt động.

1.4 Địa Chỉ IP (IP Address)

Địa chỉ IP là địa chỉ định danh duy nhất của mỗi thiết bị trong mạng (tương tự như số nhà trong đời thực).

- **IPv4**: Chuỗi 4 nhóm số phân cách bởi dấu chấm (từ `0` đến `255`), ví dụ: `192.168.1.100`, `8.8.8.8` (Google DNS).
- **Localhost (`127.0.0.1`)**: Địa chỉ IP đặc biệt trỏ về chính máy tính bạn đang sử dụng. Khi lập trình web local, ta truy cập qua `http://localhost`.

1.5 Tên Miền (Domain Name)

1.5.1 Khái niệm Tên Miền

Tên miền (Domain Name) là một chuỗi ký tự có ý nghĩa, được sử dụng để **định danh và truy cập một website hoặc một dịch vụ trên Internet**. Mỗi tên miền được liên kết với một hoặc nhiều **địa chỉ IP** thông qua hệ thống **DNS (Domain Name System)**.

Nói một cách đơn giản, tên miền đóng vai trò như **"địa chỉ dễ nhớ"** của một website. Thay vì phải ghi nhớ các dãy số IP dài và khó nhớ, người dùng chỉ cần nhập tên miền vào trình duyệt để truy cập website.

[Bảng] Ví Dụ Ánh Xạ Giữa Tên Miền Và Địa Chỉ IP

Tên miền (Domain)	Địa chỉ IP (Ví dụ thực tế)
google.com	142.250.xxx.xxx
facebook.com	157.240.xxx.xxx
ut.edu.vn	35.197.136.140

Khi người dùng nhập địa chỉ `https://www.google.com`, trình duyệt sẽ không kết nối trực tiếp đến tên miền mà sẽ yêu cầu hệ thống DNS chuyển đổi tên miền này thành địa chỉ IP của máy chủ Google trước khi thiết lập kết nối.

1.5.2 Tại sao cần tên miền?

Máy tính trên Internet chỉ có thể giao tiếp với nhau thông qua **địa chỉ IP**, trong khi con người lại dễ ghi nhớ tên hơn các dãy số. Ví dụ: Thay vì phải nhớ dãy số phức tạp `142.250.190.78`, chúng ta chỉ cần nhớ tên gọi ngắn gọn `google.com`.

Các lợi ích cốt lõi của Tên miền:

- Dễ nhớ hơn địa chỉ IP rất nhiều.
- Tạo sự chuyên nghiệp và uy tín cho website.
- Xây dựng và bảo vệ thương hiệu trên môi trường Internet.
- Giúp người dùng truy cập nhanh chóng và chính xác.
- Cho phép linh hoạt thay đổi máy chủ mà không cần thay đổi địa chỉ website (chỉ cần cập nhật bản ghi DNS).

[Mẹo] Tính Linh Hoạt Của Tên Miền

Nếu Google thay đổi toàn bộ hệ thống máy chủ và địa chỉ IP mới, người dùng vẫn chỉ cần truy cập `google.com`, hệ thống DNS sẽ tự động chuyển hướng đến địa chỉ IP mới mà không ảnh hưởng tới người dùng.

1.5.3 Tên miền hoạt động như thế nào?

Giả sử người dùng nhập địa chỉ `https://www.ut.edu.vn`. Quá trình phân giải tên miền diễn ra qua 8 bước:

[Khái Niệm] Quy Trình 8 Bước Hoạt Động Phân Giải Tên Miền (DNS Resolution)

- **Bước 1:** **Nhập Tên Miền**: Người dùng nhập tên miền vào trình duyệt.
- **Bước 2:** **Kiểm Tra DNS Cache**: Trình duyệt kiểm tra xem IP của tên miền đã lưu trong bộ nhớ đệm (*DNS Cache*) chưa. Nếu có → sử dụng ngay; nếu chưa → chuyển sang bước 3.
- **Bước 3:** **Gửi Yêu Cầu DNS**: Trình duyệt gửi yêu cầu đến **DNS Server** hỏi: *"Địa chỉ IP của* `www.ut.edu.vn` *là gì?"*.
- **Bước 4:** **Tìm Kiếm Database**: DNS Server thực hiện tìm kiếm trong cơ sở dữ liệu phân giải của mình.
- **Bước 5:** **Trả Về IP**: DNS Server trả về địa chỉ IP tương ứng (ví dụ: `35.197.136.140`).
- **Bước 6:** **Kết Nối Server**: Trình duyệt sử dụng địa chỉ IP đó để thiết lập kết nối đến máy chủ web.
- **Bước 7:** **Xử Lý Yêu Cầu**: Máy chủ web xử lý yêu cầu và trả về nội dung trang web.
- **Bước 8:** **Hiển Thị Website**: Trình duyệt kết xuất và hiển thị trang web cho người dùng.

Lưu ý: Toàn bộ quá trình phân giải tên miền (DNS Resolution) thường chỉ mất vài mili giây và diễn ra hoàn toàn tự động.

1.5.4 Cấu trúc của tên miền

Tên miền được tổ chức theo dạng phân cấp (*Hierarchy*), các cấp được phân tách bằng dấu chấm (`.`).

Xét ví dụ tên miền `www.ut.edu.vn`:

`www`	`ut`	`edu`	`vn`
↑	↑	↑	↑
Subdomain	**Tên tổ chức**	**Cấp 2 (SLD)**	**Cấp cao nhất (TLD)**

Xét ví dụ khác `mail.google.com`:

[Bảng] Phân Tích Cấu Trúc Tên Miền mail.google.com

Thành phần	Ý nghĩa chức năng
mail	Subdomain (máy chủ thư điện tử chuyên dụng).
google	Tên doanh nghiệp / tổ chức sở hữu.
com	Tên miền cấp cao nhất (Top-Level Domain – TLD).

1.5.5 Các cấp của tên miền

1. Tên miền cấp cao nhất (Top-Level Domain – TLD)

Đây là phần nằm ngoài cùng bên phải của tên miền. Ví dụ: `.com`, `.net`, `.org`, `.edu`, `.gov`, `.vn`.

[Bảng] Các Tên Miền Dùng Chung (gTLD) Phổ Biến Phân Theo Lĩnh Vực

Đuôi TLD	Ý nghĩa định danh lĩnh vực
`.com`	Commercial – Thương mại, doanh nghiệp
`.org`	Organization – Tổ chức phi lợi nhuận
`.net`	Network – Hạ tầng mạng và dịch vụ Internet
`.edu`	Education – Giáo dục, trường học, viện nghiên cứu
`.gov`	Government – Cơ quan chính phủ
`.mil`	Military – Quân đội Hoa Kỳ
`.info`	Information – Trang thông tin đại chúng
`.biz`	Business – Hoạt động kinh doanh thương mại

Ngoài ra còn có các tên miền quốc gia (*Country Code Top-Level Domain – ccTLD*):

[Bảng] Một Số Tên Miền Quốc Gia (ccTLD) Tiêu Biểu

Đuôi ccTLD	Quốc gia sở hữu
`.vn`	Việt Nam
`.jp`	Nhật Bản
`.kr`	Hàn Quốc
`.uk`	Vương quốc Anh
`.us`	Hoa Kỳ
`.au`	Úc

2. Tên miền cấp hai (Second-Level Domain – SLD)

Đây là phần đứng ngay trước TLD. Ví dụ trong `google.com`, thì `google` là tên miền cấp hai.

3. Tên miền cấp ba (Subdomain)

Subdomain là tên miền con được tạo ra từ tên miền chính để phân chia dịch vụ.

[Bảng] Các Subdomain Phổ Biến Trong Phát Triển Web

Subdomain	Chức năng định danh dịch vụ
`www`	Website chính (*World Wide Web*)
`mail`	Hệ thống quản trị Email
`blog`	Trang nhật ký bài viết / tin tức
`api`	Cổng giao tiếp API cho ứng dụng
`docs`	Hệ thống tài liệu hướng dẫn
`admin`	Trang quản trị hệ thống nội bộ
`shop`	Cửa hàng thương mại điện tử

Ví dụ thực tế: `api.facebook.com`, `docs.python.org`, `mail.google.com`.

1.5.6 Phân loại tên miền

1. Tên miền quốc tế:

Là các tên miền được quản lý trên phạm vi toàn cầu (ví dụ: `google.com`, `facebook.com`, `github.com`, `microsoft.com`). Tổ chức chịu trách nhiệm quản lý toàn cầu là **ICANN** (*Internet Corporation for Assigned Names and Numbers*).

2. Tên miền quốc gia:

Là tên miền dành riêng cho từng quốc gia (ví dụ: `.gov.vn`, `.edu.vn`, `.com.vn`, `.org.vn`). Tại Việt Nam, các tên miền đuôi `.vn` được quản lý trực tiếp bởi Trung tâm Internet Việt Nam **VNNIC** (*Vietnam Internet Network Information Center*). Ví dụ: `moet.gov.vn`, `ut.edu.vn`.

1.5.7 Quy trình đăng ký tên miền

Tên miền không tự động thuộc sở hữu của bất kỳ cá nhân hay tổ chức nào. Muốn sở hữu một tên miền, người dùng phải thực hiện 4 bước:

1. Kiểm tra xem tên miền mong muốn đã có người đăng ký hay chưa.
2. Đăng ký thông qua nhà đăng ký tên miền hợp pháp (*Domain Registrar*).
3. Thanh toán phí đăng ký quyền sử dụng.
4. Gia hạn quyền sử dụng định kỳ (thường tính theo từng năm).

Ví dụ: Bạn muốn đăng ký tên miền `myshop.vn`. Nếu chưa ai sở hữu, bạn có thể đăng ký sử dụng ngay. Nếu đã có người đăng ký, bạn phải chọn tên miền khác hoặc thương lượng mua lại.

1.5.8 Vai trò của hệ thống DNS đối với tên miền

Tên miền chỉ đơn thuần là một chuỗi chữ cái đại diện. Để truy cập được website, nhất thiết phải có **DNS (Domain Name System)** làm nhiệm vụ chuyển đổi tên miền thành địa chỉ IP số:

`www.google.com` → DNS → `142.250.xxx.xxx` → Google Server

[Cảnh Báo] Hậu Quả Khi Hệ Thống DNS Sự Cố

Nếu hệ thống DNS gặp sự cố hoặc bị hỏng, trình duyệt sẽ không thể biết địa chỉ IP của máy chủ, dẫn đến website không thể truy cập mặc dù máy chủ web vẫn đang hoạt động bình thường.

1.5.9 Ví dụ tổng hợp phân tích tên miền

Xét tên miền dịch vụ Email `https://mail.google.com`:

[Bảng] Phân Tích Tên Miền mail.google.com

Thành phần	Giá trị	Ý nghĩa chức năng
Giao thức	`https`	Giao thức bảo mật mã hóa SSL/TLS.
Subdomain	`mail`	Máy chủ cung cấp dịch vụ Email Gmail.
Domain chính	`google`	Tên doanh nghiệp / tổ chức sở hữu.
TLD	`com`	Tên miền cấp cao nhất thương mại.

Xét ví dụ tên miền trước. Một URL đầy đủ tiêu chuẩn có cấu trúc cú pháp như sau:

```
protocol://host_name:port/path/file?query#fragment
```

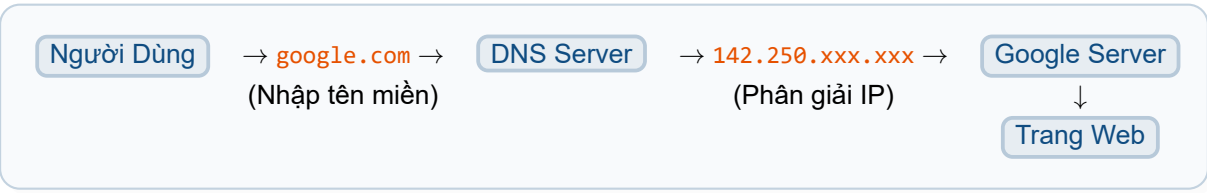
Để phân biệt rõ ràng giữa tên miền và địa chỉ IP, ta có bảng so sánh chi tiết dưới đây:

[Bảng] Bảng So Sánh Tên Miền Và Địa Chỉ IP

Tiêu chí	Tên Miền (Domain Name)	Địa Chỉ IP (IP Address)
Bản chất	Chuỗi ký tự chữ cái có ý nghĩa dễ nhớ.	Dãy số định danh thiết bị mạng.
Mục đích	Giúp người dùng truy cập website dễ dàng.	Giúp máy tính định vị và giao tiếp với nhau.
Ví dụ minh họa	`google.com`	`142.250.190.78`
Đối tượng sử dụng	Con người trực tiếp tương tác.	Máy tính, router và thiết bị mạng.
Khả năng thay đổi	Thường giữ ổn định định danh thương hiệu.	Thay đổi linh hoạt khi đổi hạ tầng máy chủ.

1.5.10 Mối quan hệ tổng hòa giữa Domain, DNS và IP

Ba khái niệm Domain, DNS và IP luôn phối hợp khép kín theo mô hình sau:



Hình 1.3: Sơ Đồ Mối Quan Hệ Giữa Domain, DNS Server Và Máy Chủ IP

[Ghi Chú] Tổng Kết Mối Quan Hệ Domain - DNS - IP

Tên miền là cầu nối giữa con người và hệ thống mạng Internet. Con người sử dụng **Domain Name** vì dễ nhớ, trong khi máy tính sử dụng **IP Address** để giao tiếp. Hệ thống **DNS** đóng vai trò trung gian, tự động phân giải tên miền thành địa chỉ IP. Nhờ cơ chế này, người dùng có thể truy cập các website bằng những tên gọi thân thiện như `google.com`, `facebook.com` hay `ut.edu.vn` mà không cần ghi nhớ các dãy số IP phức tạp.

1.6 Cổng Dịch Vụ (Port)

Port là số hiệu định danh xác định dịch vụ hoặc chương trình cụ thể đang chạy trên máy chủ (phạm vi từ `0` đến `65535`).

[Bảng] Các Cổng Dịch Vụ (Port) Phổ Biến Theo Tiêu Chuẩn

Số Cổng (Port)	Dịch Vụ	Mô Tả Chức Năng
20, 21	FTP	Truyền tập tin dữ liệu tập trung
22	SSH	Truy cập điều khiển máy chủ từ xa an toàn
25	SMTP	Gửi thư điện tử (Email)
53	DNS	Phân giải tên miền thành địa chỉ IP
80	HTTP	Truyền tải trang web không mã hóa
110 / 143	POP3 / IMAP	Nhận và đồng bộ thư điện tử
443	HTTPS	Truyền tải trang web bảo mật mã hóa SSL/TLS
3306	MySQL	Hệ quản trị cơ sở dữ liệu MySQL
5432	PostgreSQL	Hệ quản trị cơ sở dữ liệu PostgreSQL

1.7 URL (Uniform Resource Locator)

1.7.1 Khái niệm URL

URL (Uniform Resource Locator) là địa chỉ dùng để xác định **vị trí của một tài nguyên trên Internet hoặc trong mạng nội bộ**, đồng thời chỉ ra **cách thức truy cập** đến tài nguyên đó.

Tài nguyên (*Resource*) trên Internet có thể bao gồm:

- Một trang web (*Web Page*).
- Một hình ảnh, video, tệp âm thanh hoặc tệp PDF.
- Một điểm cuối giao diện lập trình ứng dụng (*API Endpoint*).
- Một tài liệu trực tuyến hoặc một tệp tin tải xuống.

> **[Mẹo] Ví Dụ Thực Tế Về Địa Chỉ Nhà**
>
> Nói một cách đơn giản, URL giống như **địa chỉ nhà** trong đời thực. Nếu muốn gửi thư cho ai đó, bạn cần biết địa chỉ nhà của họ; tương tự, nếu muốn truy cập một tài nguyên trên Internet, trình duyệt cần biết URL chính xác của tài nguyên đó.

Ví dụ về đường dẫn URL:

- `https://www.google.com`: URL trang chủ của Google.
- `https://ut.edu.vn/articles/triet-ly-giao-duc-119.html`: URL bài viết trên website Trường Đại học Giao thông Vận tải TP.HCM.

1.7.2 Vai trò của URL

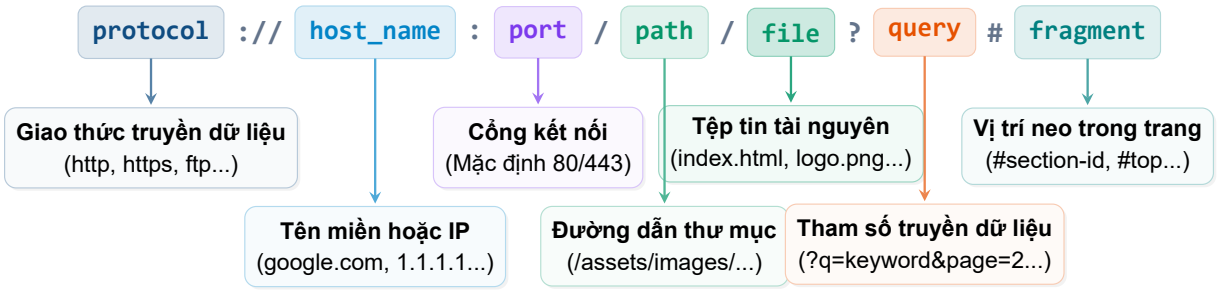
URL đóng vai trò vô cùng quan trọng trong hệ thống Web vì nó giúp:

- Xác định chính xác vị trí của tài nguyên trên mạng toàn cầu.
- Cho biết giao thức truyền dữ liệu cần sử dụng (HTTP, HTTPS, FTP...).
- Chỉ định máy chủ lưu trữ tài nguyên và cổng giao tiếp.
- Xác định đường dẫn chính xác đến tệp tin trên máy chủ.
- Cho phép người dùng chia sẻ hoặc truy cập trực tiếp một nội dung cụ thể.

Nếu không có URL, trình duyệt sẽ không thể biết phải kết nối đến máy chủ nào và lấy tài nguyên gì.

1.7.3 Cấu trúc tổng quát của URL

Một URL đầy đủ tiêu chuẩn có cấu trúc cú pháp chi tiết như sau:



[Bảng] Bảng Phân Tích Ý Nghĩa 7 Thành Phần Căn Bản Của URL

Thành phần	Ý nghĩa & Chức năng
protocol	Giao thức truyền dữ liệu (HTTP, HTTPS, FTP, SMTP...).
host_name	Tên miền hoặc địa chỉ IP định danh máy chủ.
port	Cổng dịch vụ kết nối của máy chủ (mặc định 80 cho HTTP, 443 cho HTTPS).
path	Đường dẫn đến thư mục chứa tài nguyên trên máy chủ.
file	Tên tệp tin tài nguyên cụ thể cần truy cập.
query	Các tham số truyền dữ liệu cho máy chủ (*Query String*).
fragment	Vị trí mỏ neo cụ thể bên trong tài liệu (*Anchor*).

Giao thức (Protocol)

Thành phần `https` chỉ định giao thức mà trình duyệt sử dụng để giao tiếp với máy chủ.

[Bảng] Các Giao Thức Thường Gặp Trong URL

Giao thức	Mục đích sử dụng
HTTP	Truyền trang web không mã hóa văn bản thuần (*plain text*).
HTTPS	Truyền trang web bảo mật được mã hóa SSL/TLS.
FTP	Truyền tập tin dữ liệu lớn giữa Client và Server.
SMTP	Gửi thư điện tử Email.
WebSocket (ws, wss)	Giao tiếp hai chiều thời gian thực giữa trình duyệt và máy chủ.

Ví dụ: `http://example.com` không mã hóa dữ liệu, trong khi `https://example.com` mã hóa toàn bộ dữ liệu giữa trình duyệt và máy chủ.

Tên máy chủ (Host Name)

Thành phần `ut.edu.vn` là tên miền đại diện cho máy chủ. Trình duyệt gửi yêu cầu đến địa chỉ IP tương ứng thông qua hệ thống DNS:

`google.com` → DNS → `142.250.xxx.xxx` → Kết nối máy chủ Google

Host cũng có thể là địa chỉ IP trực tiếp, ví dụ: `http://192.168.1.100` hoặc `http://127.0.0.1` (Localhost).

Cổng dịch vụ (Port)

Thành phần `:443` xác định cổng dịch vụ tiếp nhận yêu cầu trên máy chủ. Một máy chủ chạy nhiều dịch vụ cùng lúc nên cần Port để phân biệt.

[Bảng] Các Cổng Dịch Vụ Mặc Định Phổ Biến

Số Port	Dịch vụ	Ghi chú mặc định
80	HTTP	Trình duyệt tự động ẩn trên URL
443	HTTPS	Trình duyệt tự động ẩn trên URL
21	FTP	Cổng truyền file
22	SSH	Cổng điều khiển máy chủ an toàn
3306	MySQL	Cổng cơ sở dữ liệu

Thông thường, `https://google.com` thực chất là `https://google.com:443`, nhưng trình duyệt tự động ẩn cổng 443 vì đây là chuẩn mặc định của HTTPS.

Đường dẫn thư mục (Path)

Thành phần `/articles/` chỉ ra đường dẫn thư mục lưu trữ tài nguyên trên ổ cứng máy chủ. Ví dụ trong `https://example.com/images/logo.png`, thì `/images/` là thư mục chứa hình ảnh.

Tên tài nguyên (File)

Thành phần `triet-ly-giao-duc-119.html` là tên tệp tài nguyên cụ thể (HTML, PDF, PNG, JPG, MP4, CSS, JS), ví dụ: `logo.png` hoặc `index.html`.

[Ghi Chú] Lưu Ý Về Định Tuyến Routing Hiện Đại

Ngày nay, nhiều website sử dụng cơ chế định tuyến (*Routing*), vì vậy URL có thể không hiển thị tên tệp thực tế. Ví dụ: `https://facebook.com/profile` dù không có đuôi `.html`, máy chủ vẫn xử lý và trả về giao diện trang cá nhân phù hợp.

Tham số truy vấn (Query String)

Thành phần `?id=10&page=2` là tập hợp các tham số gửi dữ liệu lên máy chủ theo cặp `key=value`, các tham số phân cách nhau bằng dấu `&`.

Ví dụ:

- `https://shop.com/products?id=15`: Server biết người dùng muốn xem sản phẩm có mã số 15.
- `https://google.com/search?q=frontend`: Tham số `q` có giá trị `frontend`, máy chủ Google sẽ tìm kiếm từ khóa ”frontend”.

Mỏ neo nội dung (Fragment)

Thành phần `#section3` được dùng làm vị trí mỏ neo. **Fragment không được gửi lên máy chủ**, nó chỉ giúp trình duyệt tự động cuộn màn hình xuống vị trí thẻ có ID tương ứng.

Ví dụ: Trong `https://example.com/tutorial.html#chapter5`, trình duyệt cuộn ngay xuống phần `chapter5`. Fragment thường dùng trong mục lục, tài liệu hướng dẫn, ebook và website dài.

1.7.4 URL hoạt động như thế nào?

Giả sử người dùng nhập URL:

```
https://www.example.com/products/laptop?id=15
```

Quá trình xử lý URL diễn ra qua 10 bước chuẩn hóa:

[Khái Niệm] Quy Trình 10 Bước Trình Duyệt Xử Lý Và Tải URL

Bước 1: **Nhập URL**: Người dùng nhập URL vào thanh địa chỉ trình duyệt.

Bước 2: **Phân Tích URL**: Trình duyệt kiểm tra giao thức HTTPS và tên miền `www.example.com`.

Bước 3: **Tra Cứu DNS**: Trình duyệt gửi yêu cầu đến hệ thống DNS để chuyển tên miền `www.example.com` thành IP `203.113.xx.xx`.

Bước 4: **Mở Kết Nối**: Trình duyệt thiết lập kết nối TCP và SSL/TLS (đối với HTTPS) đến máy chủ.

Bước 5: **Gửi Request**: Trình duyệt gửi yêu cầu: `GET /products/laptop?id=15 HTTP/1.1` (`Host: www.example.com`).

Bước 6: **Tiếp Nhận Request**: Máy chủ tiếp nhận yêu cầu gửi tới.

Bước 7: **Xử Lý Server**: Máy chủ xử lý: truy vấn CSDL tìm sản phẩm ID=15 và sinh trang HTML.

Bước 8: **Trả Phản Hồi**: Máy chủ gửi trả về kết quả phản hồi cho Client.

Bước 9: **Tải Tài Nguyên Phụ**: Trình duyệt tải các tệp HTML, CSS, JavaScript, hình ảnh, font chữ.

Bước 10: **Render Hiển Thị**: Trình duyệt kết xuất (Render) và hiển thị trang web cho người dùng.

1.7.5 Phân biệt URL Tuyệt Đối và URL Tương Đối

Trong phát triển web, có hai loại URL chính:

[Bảng] So Sánh URL Tuyệt Đối Và URL Tương Đối

Đặc điểm	URL Tuyệt Đối (Absolute URL)	URL Tương Đối (Relative URL)
Cấu trúc	Chứa đầy đủ thông tin từ giao thức đến tài nguyên.	Chỉ chứa đường dẫn tương đối so với trang hiện tại.
Ví dụ	`https://example.com/images/logo`	`images/logo.png` hoặc `../css/style.css`
Phạm vi sử dụng	Truy cập từ bất kỳ đâu trên Internet.	Liên kết giữa các tài nguyên trong cùng một website.
Ưu điểm	Không phụ thuộc vào vị trí file hiện tại.	Linh hoạt hơn khi thay đổi tên miền hoặc môi trường triển khai.

1.7.6 Ví dụ tổng hợp phân tích cấu trúc URL

Xét URL phức hợp thực tế dưới đây:

```
https://shop.example.com:443/products/electronics/laptop.html?id=20&color=black#review
```

[Bảng] Bảng Tổng Hợp Phân Tích Cấu Trúc URL Phức Hợp

Thành phần	Giá trị mẫu	Ý nghĩa chức năng
Protocol	`https`	Giao thức truyền dữ liệu an toàn.
Host	`shop.example.com`	Tên miền của máy chủ.
Port	`443`	Cổng HTTPS.
Path	`/products/electronics/`	Thư mục chứa tài nguyên.
File	`laptop.html`	Trang hoặc tài nguyên được yêu cầu.
Query	`id=20&color=black`	Tham số gửi đến máy chủ.
Fragment	`review`	Vị trí trong trang mà trình duyệt sẽ cuộn tới.

[Ghi Chú] Ứng Dụng URL Trong Các Framework Web Hiện Đại (Routing)

Trong các ứng dụng web hiện đại (React, Angular, Vue, Next.js...), URL không chỉ dùng để xác định vị trí tài nguyên vật lý mà còn được sử dụng để **định tuyến (Routing)** giữa các trang hoặc trạng thái của ứng dụng. Ví dụ, URL `https://shop.example.com/products/20` có thể được Router ánh xạ đến trang chi tiết sản phẩm có mã số 20 mà không cần tồn tại tệp `products/20.html` trên máy chủ. Điều này giúp tạo ra các URL thân thiện, dễ đọc và tối ưu cho SEO.

1.8 Trang Web, Website & Dịch Vụ Hosting

1.8.1 Phân biệt Web Page và Website

- **Web Page (Trang web)**: Là một tài liệu HTML đơn lẻ hiển thị trên trình duyệt, tạo thành từ HTML, CSS và JavaScript.
- **Website**: Là một tập hợp nhiều Web Page có liên quan với nhau, cùng chia sẻ chung một tên miền (ví dụ: `wikipedia.org` gồm trang chủ, trang bài viết, trang đăng nhập...).

1.8.2 Dịch vụ Hosting & Các loại Hosting

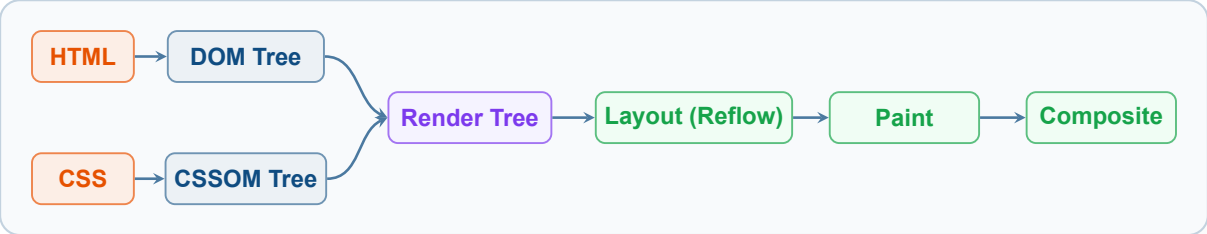
Hosting là dịch vụ cho thuê không gian lưu trữ và tài nguyên tính toán trên máy chủ để đưa website lên mạng Internet.

[Bảng] Phân Loại Các Dạng Dịch Vụ Hosting

Loại Hosting	Ưu điểm	Nhược điểm
Shared Hosting	Chi phí rất rẻ, dễ cài đặt quản lý	Chia sẻ tài nguyên, dễ bị ảnh hưởng bởi web khác
VPS (Máy chủ ảo)	Hiệu năng cao, toàn quyền quản trị	Cần có kiến thức quản trị hệ thống Linux/Windows
Dedicated Server	Hiệu năng tối đa, bảo mật tuyệt đối	Chi phí đầu tư & vận hành rất cao
Cloud Hosting	Mở rộng linh hoạt, độ ổn định cực cao	Chi phí tính theo lưu lượng sử dụng thực tế

1.9 Cách Trình Duyệt Render Trang (Critical Rendering Path)

Để trở thành một kĩ sư Frontend chuyên nghiệp, việc hiểu cách trình duyệt chuyển đổi mã HTML/CSS thô thành các điểm ảnh (pixels) trên màn hình là cực kỳ quan trọng. Quá trình này được gọi là **Critical Rendering Path (CRP)**.



Hình 1.4: Cơ chế Critical Rendering Path của Trình duyệt Web

Các giai đoạn cụ thể:

1. **Xây dựng DOM (Document Object Model)**: Trình duyệt đọc dữ liệu nhị phân (bytes) của HTML, chuyển thành các ký tự, tokens, nodes và tạo nên cây DOM.
2. **Xây dựng CSSOM (CSS Object Model)**: Tương tự DOM, trình duyệt phân tích các quy tắc CSS để dựng cây quy tắc kiểu dáng CSSOM.
3. **Tạo Render Tree**: Kết hợp DOM và CSSOM để lọc ra các phần tử thực sự hiển thị trên màn hình (loại bỏ các thẻ có `display: none`).
4. **Bố cục (Layout / Reflow)**: Tính toán tọa độ chính xác (x, y) và kích thước $(width, height)$ của từng phần tử trong màn hình (*viewport*).
5. **Vẽ (Paint)**: Tô màu, kẻ viền, vẽ văn bản và ảnh lên các lớp (*layers*).
6. **Hợp nhất Lớp (Composite)**: Ghép các lớp lại theo đúng thứ tự hiển thị (z-index, 3D transforms) và đẩy ra màn hình.

[Cảnh Báo] Tránh Hiện Tượng Reflow & Repaint Cắm Đầu

Khi bạn thay đổi kích thước phần tử bằng JavaScript hoặc CSS Animation như `width`, `height`, `margin`, trình duyệt bắt buộc phải chạy lại từ bước **Layout (Reflow)**, gây hiện tượng giật lag (jank).
Mẹo tốt nhất: Ưu tiên sử dụng `transform` (scale, translate) và `opacity` vì chúng chỉ kích hoạt bước **Composite**, giúp animation mượt mà ở tần số 60fps.

1.10 Lập Trình Front-End (Frontend Development)

Lập trình Front-end là quá trình phát triển giao diện người dùng của một ứng dụng hoặc trang web, bao gồm tất cả những gì mà người dùng có thể nhìn thấy và tương tác trực tiếp. Lập trình Front-end tập trung vào việc hiển thị nội dung, bố cục, hình ảnh, hiệu ứng động và các yếu tố tương tác như nút bấm, biểu mẫu nhập liệu, menu điều hướng,...

1.10.1 Các công nghệ chính trong lập trình Front-end

Nền tảng cốt lõi của lập trình Front-end dựa trên bộ ba công nghệ tiêu chuẩn web:

[Bảng] Bộ Ba Công Nghệ Cốt Lõi Của Lập Trình Front-End

Công nghệ	Tên đầy đủ	Vai trò & Chức năng chính
`HTML`	**HyperText Markup Language**	Ngôn ngữ đánh dấu giúp xây dựng cấu trúc và định hình nội dung của trang web.
`CSS`	**Cascading Style Sheets**	Ngôn ngữ dùng để thiết kế, định dạng giao diện (màu sắc, bố cục, font chữ, hiệu ứng...).
`JavaScript (JS)`	**JavaScript Programming Language**	Ngôn ngữ lập trình giúp trang web có các chức năng động và xử lý tương tác người dùng.

1.10.2 Các thư viện và framework phổ biến

Để tối ưu tốc độ phát triển và xây dựng các ứng dụng lớn, lập trình viên Front-end thường sử dụng các công cụ hiện đại:

- **React.js**: Thư viện JavaScript mạnh mẽ do Meta phát triển, giúp xây dựng giao diện người dùng theo dạng thành phần tái sử dụng (*Component-based*).
- **Vue.js**: Framework nhẹ, linh hoạt, dễ tiếp cận và học tập, phù hợp cho nhiều quy mô dự án.
- **Angular**: Framework toàn diện và mạnh mẽ của Google, sử dụng TypeScript để xây dựng các ứng dụng web phức tạp cấp doanh nghiệp (*Enterprise Apps*).
- **Tailwind CSS, Bootstrap**: Thư viện và framework CSS giúp thiết kế giao diện nhanh chóng, chuẩn hóa và nhất quán trên các thiết bị.

1.10.3 Công việc của lập trình viên Front-end

Một kĩ sư lập trình Front-end đảm nhận các trách nhiệm chính bao gồm:

1. Thiết kế và phát triển giao diện trang web hoặc ứng dụng theo bản vẽ UI/UX.
2. Tối ưu hiệu suất hiển thị và tốc độ tải trang (*Page Speed Optimization*).
3. Đảm bảo tính tương thích trên nhiều thiết bị và kích thước màn hình khác nhau (*Responsive Web Design*).
4. Xử lý các thao tác tương tác của người dùng và giao tiếp dữ liệu với máy chủ Backend thông qua API.

1.10.4 Sự khác biệt giữa Front-end và Back-end

[Bảng] Bảng So Sánh Sự Khác Biệt Giữa Front-End Và Back-End

Yếu tố	Front-end	Back-end
Vai trò	Hiển thị giao diện, trực tiếp tương tác với người dùng.	Xử lý logic nghiệp vụ, lưu trữ và quản lý dữ liệu.
Công nghệ chính	HTML, CSS, JavaScript, React, Vue, Angular.	Node.js, Python, Java, C#, SQL, NoSQL.
Giao tiếp	Làm việc với API, nhận dữ liệu từ Backend để hiển thị.	Xây dựng API, xử lý dữ liệu, xác thực và bảo mật người dùng.

1.10.5 Học lập trình Front-end bắt đầu từ đâu?

Để trở thành một lập trình viên Front-end chuyên nghiệp, bạn có thể tham khảo lộ trình 5 bước nền tảng:

[Khái Niệm] Lộ Trình 5 Bước Học Lập Trình Front-End Từ Cơ Bản Đến Chuyên Nghiệp

Bước 1: **Nắm Vững HTML & CSS**: Học cấu trúc trang web cơ bản, dàn trang, phối màu và làm chủ *Responsive Design*.

Bước 2: **Thành Thạo JavaScript**: Học cú pháp cơ bản, xử lý sự kiện, thao tác DOM và JavaScript ES6+.

Bước 3: **Làm Quen Framework Modern UI**: Chọn và thành thạo một trong các thư viện/framework phổ biến như React.js, Vue.js hoặc Angular.

Bước 4: **Quản Lý Mã Nguồn Git/GitHub**: Học các lệnh Git cơ bản để quản lý phiên bản mã nguồn và làm việc nhóm hiệu quả.

Bước 5: **Kết Nối Backend Qua API**: Tìm hiểu về REST API, giao thức HTTP và dữ liệu JSON để kết nối ứng dụng với máy chủ Backend.

1.11 Khái Niệm UI và UX: Sự Khác Biệt & Mối Quan Hệ

UI (User Interface) và **UX (User Experience)** là hai khái niệm quan trọng trong thiết kế và phát triển phần mềm, đặc biệt là trong lĩnh vực lập trình Front-end. Mặc dù liên quan mật thiết với nhau, nhưng chúng có vai trò và mục đích hoàn toàn khác nhau.

1.11.1 UI (User Interface) – Giao diện người dùng

UI (Giao diện người dùng) là tất cả những yếu tố trên màn hình mà người dùng có thể nhìn thấy và tương tác trực tiếp, bao gồm:

- Bố cục (*layout*) của trang web hoặc ứng dụng.
- Màu sắc, kiểu chữ (*typography*), biểu tượng (*icons*), nút bấm, hình ảnh.
- Hiệu ứng chuyển động (*animations & transitions*).
- Các thành phần tương tác như nút bấm, biểu mẫu nhập liệu, menu điều hướng, v.v.

> **[Mẹo] Mục Tiêu Của UI**
>
> Mục tiêu cốt lõi của UI là làm cho giao diện ứng dụng trở nên **trực quan, thẩm mỹ, hài hòa và dễ sử dụng**.

Ví dụ về UI tốt:

- Một trang web có thiết kế đẹp mắt, màu sắc hài hòa, phông chữ dễ đọc và cấu trúc bố cục giúp tìm kiếm thông tin nhanh chóng.
- Một ứng dụng di động có các nút bấm rõ ràng, kích thước hợp lý, dễ thao tác vuốt chạm trên cả điện thoại và máy tính.

1.11.2 UX (User Experience) – Trải nghiệm người dùng

UX (Trải nghiệm người dùng) là tổng thể cảm nhận và phản ứng của người dùng khi sử dụng sản phẩm. UX không chỉ liên quan đến giao diện hiển thị mà còn bao gồm:

- Tính dễ sử dụng (*usability*) của sản phẩm.
- Mức độ tiện lợi, nhanh chóng khi người dùng hoàn thành một tác vụ.
- Cảm xúc của người dùng trong suốt quá trình trải nghiệm (thoải mái, hài lòng hay bực bội, khó chịu...).
- Khả năng giải quyết đúng nhu cầu và mong muốn thực tế của người dùng.

> **[Mẹo] Mục Tiêu Của UX**
>
> Mục tiêu cốt lõi của UX là **tạo ra trải nghiệm tốt nhất**, giúp người dùng đạt được mục tiêu của họ một cách dễ dàng, nhanh chóng và thoải mái nhất.

Ví dụ về UX tốt:

- Một ứng dụng đặt hàng online có quy trình thanh toán nhanh gọn 1-click, không bắt người dùng nhập lại các thông tin không cần thiết.

- Một trang web có hệ thống tìm kiếm thông minh với gợi ý tự động, giúp người dùng dễ dàng tìm thấy chính xác sản phẩm họ cần.

1.11.3 Sự khác biệt giữa UI và UX

[Bảng] Bảng So Sánh Sự Khác Biệt Chi Tiết Giữa UI Và UX

Tiêu chí	UI (Giao diện người dùng)	UX (Trải nghiệm người dùng)
Định nghĩa	Giao diện trực quan mà người dùng nhìn thấy và tương tác.	Cảm giác và tổng thể trải nghiệm của người dùng khi sử dụng sản phẩm.
Mục tiêu	Tạo ra giao diện đẹp, bắt mắt, hấp dẫn và dễ nhìn.	Giúp người dùng có trải nghiệm thoải mái, dễ dàng và đạt hiệu quả cao.
Thành phần	Màu sắc, hình ảnh, bố cục, typography, hiệu ứng động.	Luồng điều hướng, tốc độ tải trang, sự thuận tiện trong thao tác.
Liên quan đến	Thiết kế đồ họa, phối màu, lập trình Front-end.	Tâm lý học người dùng, nghiên cứu hành vi và tối ưu luồng.

1.11.4 Mối quan hệ giữa UI và UX

UI và UX có sự liên kết hữu cơ và chặt chẽ với nhau:

- **UI đẹp nhưng UX tệ**: Sản phẩm có giao diện cực kỳ bắt mắt nhưng tốc độ tải trang quá chậm, luồng thanh toán rắc rối hoặc khó tìm kiếm thông tin → Người dùng sẽ nhanh chóng rời bỏ sản phẩm.
- **UX tốt nhưng UI xấu**: Sản phẩm rất tiện lợi và dễ sử dụng nhưng thiết kế sơ sài, màu sắc nhếch nhác, thiếu chuyên nghiệp → Giảm niềm tin của người dùng và làm giảm trải nghiệm chung.

[Ghi Chú] Sự Kết Hợp Hoàn Hảo Giữa UI Và UX

Một sản phẩm công nghệ thành công **bắt buộc phải có cả UI đẹp mắt lẫn UX mượt mà**. Hai yếu tố này bổ trợ cho nhau để mang lại sản phẩm toàn diện cho người dùng.

1.11.5 Làm thế nào để cải thiện UI/UX?

Về mặt UI (Giao diện):

- Sử dụng màu sắc hợp lý, chuẩn hóa bảng màu và đảm bảo độ tương phản tốt giữa chữ và nền.
- Chọn kiểu chữ dễ đọc, kích thước phông và khoảng cách dòng phù hợp.
- Giữ thiết kế đơn giản, tối giản (*Clean Design*), tránh đưa quá nhiều chi tiết gây rối mắt.
- Đảm bảo tính đồng nhất (*Consistency*) trong toàn bộ thiết kế (như kiểu dáng nút bấm, khoảng cách các lề).

Về mặt UX (Trải nghiệm):

- Tìm hiểu nhu cầu và hành vi thực tế của người dùng thông qua khảo sát và thử nghiệm kiểm thử (*User Testing*).
- Cắt giảm các bước thao tác trung gian không cần thiết để tối ưu hóa quy trình.

- Cải thiện tốc độ tải trang và phản hồi giao diện tức thì khi người dùng tương tác.
- Áp dụng triết lý ”Mobile-First”, đảm bảo sản phẩm hoạt động mượt mà và mĩ quan trên mọi kích thước màn hình.

[Cảnh Báo] Kết Luận Về UI/UX Trong Phát Triển Web

- **UI** giúp sản phẩm trở nên đẹp mắt, ấn tượng và thu hút người dùng ngay từ cái nhìn đầu tiên.
- **UX** giúp sản phẩm trở nên tiện lợi, dễ dùng và giải quyết triệt để nhu cầu người dùng.
- Một ứng dụng web thành công phải kết hợp hoàn hảo cả UI xuất sắc và UX tối ưu để giữ chân người dùng lâu dài.

1.12 Tóm Tắt Chương 1

[Ghi Chú] Tổng Kết Cốt Lõi Chương 1

Mô hình **Client–Server** chính là nền tảng khởi đầu của hầu hết mọi ứng dụng web hiện đại. Để một website vận hành hoàn chỉnh trên Internet, cần sự phối hợp nhịp nhàng của nhiều thành phần:

Client gửi yêu cầu → **DNS** phân giải **Domain** thành **IP** → Kết nối đến **Server** qua **Protocol** & **Port** → Server xử lý và phản hồi **Web Page** qua **URL**.

Website được triển khai trên hạ tầng **Hosting**, trình duyệt kết xuất giao diện qua quy trình **Critical Rendering Path (CRP)**. Từ đó, kĩ sư **Front-end** vận dụng bộ ba công nghệ **HTML, CSS, JavaScript** và các framework hiện đại kết hợp nguyên lý **UI/UX** để tạo ra các ứng dụng web chất lượng cao, tối ưu trải nghiệm người dùng.

CHƯƠNG 2

Môi Trường & Công Cụ Lập Trình Frontend Masterclass

Một môi trường phát triển (**Development Environment**) được chuẩn hóa, kết hợp với bộ công cụ hiện đại và quy trình quản lý mã nguồn chuyên nghiệp là yếu tố quyết định tốc độ làm việc, chất lượng sản phẩm và khả năng làm việc nhóm của một kĩ sư Frontend chuyên nghiệp.

2.1 Trình Soạn Thảo Mã Nguồn: Visual Studio Code (VS Code)

Visual Studio Code (VS Code) là trình biên soạn mã nguồn (*Code Editor*) phổ biến nhất thế giới hiện nay cho các lập trình viên Web. VS Code sở hữu tốc độ xử lý nhanh, tiêu tốn ít tài nguyên hệ thống, tích hợp sẵn Terminal, Git client và có một chợ ứng dụng mở rộng (*Extension Marketplace*) cực kỳ phong phú.

2.1.1 Top Extension Quan Trọng Cho Frontend Developer

Để biến VS Code thành một môi trường lập trình Frontend mạnh mẽ, các kĩ sư web cần cài đặt các Extension cốt lõi sau:

[Bảng] Bảng Tổng Hợp Các Extension Hàng Đầu Cho VS Code

Tên Extension	Từ khóa Search	Chức năng & Ý nghĩa kỹ thuật cốt lõi
Live Server	`Ritwick Dey`	Tự động tạo local server và làm mới trình duyệt (*Hot Reloading*) ngay khi lưu file HTML/CSS/JS.
Prettier	`Prettier`	Tự động định dạng mã nguồn chuẩn mực, thẳng hàng, căn lề tự động theo chuẩn công nghiệp.
ESLint	`Microsoft`	Kiểm tra lỗi cú pháp, cảnh báo mã xấu (*code smell*) và bắt buộc tuân thủ chuẩn JavaScript.
Auto Rename Tag	`Jun Han`	Tự động đổi tên thẻ đóng tương ứng khi bạn chỉnh sửa thẻ mở trong HTML/JSX.
GitLens	`GitKraken`	Hiển thị chi tiết lịch sử commit, tác giả và thời gian chỉnh sửa trực tiếp trên từng dòng code.
Path Intellisense	`Christian Kohler`	Tự động gợi ý đường dẫn tệp tin (*file path*) khi nhúng hình ảnh, CSS hoặc JS.

2.1.2 Cấu Hình settings.json Chuẩn Cho VS Code & Prettier

Để đảm bảo mã nguồn luôn được tự động định dạng chuẩn mực mỗi khi bấm lưu tệp (`Ctrl+S`), bạn hãy thiết lập tệp cấu hình `.vscode/settings.json` trong thư mục gốc của dự án:

Cấu Hình tệp .vscode/settings.json Chuẩn Cho Prettier

```
{
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "prettier.singleQuote": true,
  "prettier.semi": true,
  "prettier.tabWidth": 2,
  "prettier.trailingComma": "es5",
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": "explicit"
  }
}
```

2.1.3 Bảng Tra Cứu Phím Tắt Nâng Cao Năng Suất Lập Trình

[Bảng] Bảng Tra Cứu Phím Tắt VS Code Thường Dùng Cho Frontend

Phím tắt (Windows)	Phím tắt (macOS)	Chức năng & Tác dụng nhanh
`Ctrl + S`	Cmd + S	Lưu tệp tin và kích hoạt tự động định dạng Prettier.
`Ctrl + /`	Cmd + /	Bật/Tắt ghi chú (*Comment*) cho dòng hoặc khối code.
`Alt + Shift + F`	Option + Shift + F	Định dạng lại toàn bộ tệp tin mã nguồn ngay lập tức.
`Alt + Up / Down`	Option + Up / Down	Di chuyển dòng code hiện tại lên hoặc xuống.
`Shift + Alt + Down`	Shift + Option + Down	Nhân bản (*Duplicate*) dòng code hiện tại xuống dưới.
`Ctrl + D`	Cmd + D	Chọn các từ giống nhau tiếp theo để sửa đồng loạt.
`Ctrl +`	Cmd +	Ẩn/Hiện cửa sổ dòng lệnh Terminal tích hợp.

2.1.4 Cơ Chế Chú Thích Mã Nguồn Trong VS Code (VS Code Commenting)

Trong Visual Studio Code (VS Code), bạn có thể sử dụng các tổ hợp phím tắt nhanh để tạo hoặc gỡ chú thích (*comment*) một cách hiệu quả, giúp lập hồ sơ ghi chú tài liệu hoặc vô hiệu hóa tạm thời các đoạn mã nguồn trong quá trình phát triển.

[Bảng] Tổ Hợp Phím Tắt Chú Thích Trong VS Code

Loại chú thích	Windows/Linux	macOS	Nguyên lý hoạt động
Dòng đơn (Single-line)	`Ctrl + /`	`Cmd + /`	Tự động chèn ký hiệu chú thích ở đầu dòng hiện tại.
Nhiều dòng (Multi-line)	`Shift + Alt + A`	`Shift + Option + A`	Bọc đoạn mã nguồn được bôi đen trong khối chú thích.

Dưới đây là cú pháp và ví dụ minh họa cách viết chú thích trong các ngôn ngữ lập trình phổ biến:

Ví dụ cú pháp chú thích trong các ngôn ngữ lập trình

```
<!-- 1. HTML: Sử dụng cặp thẻ mở/đóng đặc trưng -->
<!-- Đây là một chú thích dòng đơn trong HTML -->
<!--
  Đây là một chú thích trên nhiều dòng trong HTML
-->
```

```
/* 2. CSS: Chỉ hỗ trợ kiểu chú thích khối */
/* Đây là một chú thích trong CSS */
/*
  Đây là một chú thích nhiều dòng trong CSS
*/

// 3. JS / C++ / C# / Java: Hỗ trợ cả chú thích dòng đơn và khối
// Đây là chú thích dòng đơn (dùng hai dấu gạch chéo)
/*
  Đây là chú thích nhiều dòng (dùng cặp ký hiệu mở/đóng)
*/
```

```
# 4. Python: Sử dụng ký tự thăng (#) làm chú thích dòng đơn
# Đây là một chú thích dòng đơn trong Python
# Python không hỗ trợ chú thích nhiều dòng chính thức, nhưng có thể dùng
    Docstring thay thế:
'''
Đây là một chú thích nhiều dòng sử dụng Docstring trong Python
'''
```

[Mẹo] Mẹo Sử Dụng Phím Tắt Chú Thích Hiệu Quả

- **Hủy chú thích nhanh**: Để bỏ chú thích dòng hoặc khối code, chỉ cần đặt con trỏ tại dòng đó hoặc bôi đen khối đó, rồi nhấn lại chính tổ hợp phím tắt tương ứng (`Ctrl + /` hoặc `Shift + Alt + A`).
- **Chú thích đồng loạt**: Nếu bôi đen nhiều dòng và nhấn `Ctrl + /`, VS Code sẽ tự động chèn ký hiệu chú thích dòng đơn vào đầu mỗi dòng được chọn cùng lúc.

2.2 Làm Chủ Chrome DevTools (Developer Tools Masterclass)

Chrome Developer Tools (Chrome DevTools) là bộ công cụ kiểm tra và gỡ lỗi (*Debugging*) bá đạo được tích hợp sẵn trong trình duyệt Google Chrome. Nhấn phím `F12` hoặc `Ctrl+Shift+I` (trên macOS: `Cmd+Option+I`) để mở DevTools.

2.2.1 5 Tab DevTools Cốt Lõi Không Thể Sống Thiếu

[Bảng] Bảng Phân Tích 5 Tab DevTools Quan Trọng Nhất

Tên Tab DevTools	Phím tắt mở	Tác dụng & Ứng dụng gỡ lỗi kỹ thuật cốt lõi
Elements Tab	`Ctrl+Shift+C`	Kiểm tra và chỉnh sửa cấu trúc DOM, hiển thị sơ đồ Box Model, Flexbox/Grid và thử nghiệm CSS thời gian thực.
Console Tab	`Esc` / `Ctrl+``	Xem nhật ký lỗi JavaScript, chạy thử các đoạn mã JS trực tiếp và kiểm tra dữ liệu qua `console.log()`, `console.table()`.
Network Tab	`F12` → `Network`	Theo dõi tất cả yêu cầu HTTP/HTTPS (Requests), thời gian phản hồi, dung lượng tài nguyên và dữ liệu API (JSON/Payloads).
Application Tab	`F12` → `App`	Kiểm tra bộ nhớ đệm trình duyệt: LocalStorage, SessionStorage, Cookies, Cache Storage và Service Workers.
Lighthouse Tab	`F12` → `Audit`	Đánh giá điểm số tự động về Hiệu năng (Performance), Độ truy cập (Accessibility), SEO và Best Practices.

[Mẹo] Mẹo Kiểm Tra Responsive Trên Nhiều Thiết Bị Với Toggle Device Toolbar

Nhấn tổ hợp phím `Ctrl + Shift + M` (hoặc click vào biểu tượng Điện thoại/Máy tính bảng ở góc trên bên trái cửa sổ DevTools) để bật chế độ giả lập màn hình di động. Bạn có thể chọn thử nghiệm trên iPhone, iPad, Samsung Galaxy hoặc tùy chỉnh độ phân giải màn hình linh hoạt.

2.3 Quản Lý Mã Nguồn Với Git & GitHub (Git Version Control System)

Git là Hệ thống quản lý phiên bản phân tán (*Distributed Version Control System*) giúp theo dõi mọi sự thay đổi trong mã nguồn, lưu lại các mốc lịch sử và hỗ trợ nhiều lập trình viên cùng làm việc trên một dự án mà không bị ghi đè code của nhau.

GitHub là nền tảng điện toán đám mây cung cấp dịch vụ lưu trữ kho mã nguồn (*Repository*) sử dụng Git, giúp chia sẻ code, quản lý dự án và làm việc nhóm trực tuyến.

2.3.1 Mô Hình Hoạt Động 4 Trạng Thái Của Git

[Bảng] Bảng Phân Tích 4 Trạng Thái Làm Việc Của Git

Trạng Thái / Khu Vực	Lệnh di chuyển	Ý nghĩa & Bản chất kỹ thuật
Working Directory	`Tệp vừa sửa`	Thư mục làm việc thực tế chứa các tệp tin mã nguồn đang được chỉnh sửa.
Staging Area	`git add .`	Trạng thái đệm chuẩn bị. Đánh dấu các tệp tin sẵn sàng để đóng gói commit.
Local Repository	`git commit`	Kho lưu trữ nội bộ trên máy tính cá nhân, lưu giữ các bản đóng gói history.
Remote Repository	`git push`	Kho lưu trữ trực tuyến trên GitHub/GitLab giúp đồng bộ mã nguồn với nhóm.

2.3.2 Quy Trình Làm Việc Chuẩn Với Git & GitHub Hàng Ngày

Quy Trình Làm Việc Chuẩn Với Git & GitHub (Terminal Commands)

```
# 1. Khai bao thong tin ca nhan (Luu y: Chi thuc hien 1 lan dau tien)
git config --global user.name "Nguyen Van A"
git config --global user.email "nguyenvana@example.com"

# 2. Khoi tao kho luu tru local moi trong thu muc du an
git init

# 3. Kiem tra trang thai cac tep tin (Tracked / Untracked)
git status

# 4. Dua toan bo tep tin thay doi vao Staging Area
git add .

# 5. Dong goi phien ban thay doi voi thong diep commit ro rang
git commit -m "feat: Khoi tao giao dien trang chu HTML5"

# 6. Ket noi kho local voi GitHub va truyen tai code len Remote
git remote add origin https://github.com/user/my-project.git
git branch -M main
git push -u origin main
```

2.3.3 Quy Chuẩn Viết Commit Message (Conventional Commits Standard)

Một kĩ sư Frontend chuyên nghiệp luôn tuân thủ quy chuẩn viết thông điệp Commit (**Conventional Commits**) để lịch sử dự án rõ ràng, dễ tra cứu:

[Bảng] Bảng Tiêu Chuẩn Tiền Đề Commit (Conventional Commits)

Tiền đề Prefix	Ý nghĩa thay đổi	Ví dụ thông điệp Commit chuẩn
`feat:`	Tính năng mới	`feat: add responsive navigation bar`
`fix:`	Sửa lỗi bug	`fix: resolve broken image path on mobile`
`docs:`	Cập nhật tài liệu	`docs: update README with installation steps`
`style:`	Định dạng code	`style: reformat HTML with prettier`
`refactor:`	Tái cấu trúc code	`refactor: simplify header component layout`
`chore:`	Cập nhật cấu hình	`chore: update package dependencies`

2.4 Tóm Tắt Chương 2

[Ghi Chú] Tổng Kết Cốt Lõi Chương 2 & Checklist Môi Trường Mẫu

Một môi trường phát triển chuyên nghiệp với VS Code được cấu hình chuẩn hóa, cài đặt đầy đủ các Extension hỗ trợ, thành thạo bộ công cụ Chrome DevTools để kiểm tra DOM/Network, cùng quy trình quản lý mã nguồn Git/GitHub chuẩn mực sẽ nâng cao hiệu suất làm việc và chất lượng sản phẩm của kĩ sư Web lên gấp nhiều lần.

PHẦN II

HTML5 – Cấu Trúc & Định Dạng Siêu Văn Bản

CHƯƠNG 3

Tổng Quan HTML5, Cấu Trúc Tài Liệu & Thẻ <meta> Masterclass

[MỤC LỤC TRỌNG TÂM CHƯƠNG 3] Các Chủ Đề Cốt Lõi Cần Nắm Vững

Mục 1: Tổng Quan Về HTML & Giải Phẫu Element Nền tảng ngôn ngữ đánh dấu, phân biệt Tag và Element, 4 thành phần giải phẫu cấu trúc HTML.

Mục 2: Cấu Trúc Chuẩn Của Tài Liệu HTML5 Thẻ `<!DOCTYPE html>`, thẻ `<html>`, `<head>` và `<body>` theo chuẩn W3C.

Mục 3: Thẻ <meta> & Cấu Hình SEO Masterclass Tối ưu UTF-8, Responsive Viewport, SEO Meta Description, Open Graph và Twitter Cards.

Mục 4: Bộ Thẻ HTML Phổ Biến Trong Web Development Định dạng văn bản, liên kết (`<a>`), danh sách (`<ul>`, `<ol>`), nhúng ảnh (`<img>`).

Mục 5: Mô Hình Hiển Thị Block, Inline & Inline-Block Phân biệt thuộc tính `display`, dòng chảy tài liệu (*Normal Flow*) và quy tắc lồng thẻ.

Mục 6: Bảng Biểu (Tables) & Định Dạng CSS Masterclass Cấu trúc `<table>`, thuộc tính gộp ô `colspan`/`rowspan` và định dạng viền CSS.

Mục 7: Thẻ Nhúng <iframe> & An Toàn Bảo Mật Web Nhúng trang ngoài, kiểm soát quyền truy cập qua thuộc tính `sandbox` và `allow`.

Mục 8: Semantic HTML5 Masterclass & Cấu Trúc Ngữ Nghĩa Các thẻ Landmark (`<header>`, `<nav>`, `<main>`, `<article>`), SEO & Accessibility Tree.

Mục 9: Tóm Tắt & Bộ Câu Hỏi Phỏng Vấn Frontend Tổng kết kiến thức trọng tâm và bộ câu hỏi thường gặp khi đi phỏng vấn tuyển dụng.

HTML (HyperText Markup Language) là ngôn ngữ đánh dấu siêu văn bản dùng để xây dựng và trình bày nội dung trên các trang web. Đây là ngôn ngữ cơ bản nhất trong phát triển web, đóng vai trò tạo dựng cấu trúc khung xương cho toàn bộ ứng dụng web.

3.1 Tổng Quan Về HTML

Để hiểu đúng bản chất của HTML, kĩ sư Web cần nắm vững các đặc điểm cốt lõi sau:

- **Không phải ngôn ngữ lập trình**: HTML không phải là ngôn ngữ lập trình (không xử lý logic nghiệp vụ, không có biến hay vòng lặp) mà là một **ngôn ngữ đánh dấu (Markup Language)**.
- **Sử dụng thẻ (Tags)**: HTML sử dụng các thẻ (*tags*) đặt trong cặp dấu ngoặc nhọn (ví dụ: `<html>`, `<body>`, `<p>`) để đánh dấu và định dạng các phần tử nội dung.
- **Trình duyệt giải mã HTML**: Các trình duyệt web hiện đại (Google Chrome, Mozilla Firefox, Mi-

crosoft Edge, Apple Safari...) đọc mã nguồn HTML và kết xuất (*render*) thành giao diện người dùng trực quan.

3.1.1 Cấu Trúc Giải Phẫu Một Phần Tử HTML (HTML Element Anatomy)

Để xây dựng một trang web chuẩn mực, nhà phát triển cần hiểu rõ giải phẫu cấu trúc của một phần tử HTML. Về bản chất, một phần tử HTML hoàn chỉnh (**HTML Element**) được hợp thành từ 4 thành phần kỹ thuật cốt lõi: **Thẻ mở (Opening Tag)**, **Thuộc tính (Attribute)**, **Nội dung (Content)** và **Thẻ đóng (Closing Tag)**.

Sơ đồ trực quan dưới đây mô tả giải phẫu chi tiết của phần tử `<p class="highlight">Chào mừng đến HTML5</p>`:



Bảng dưới đây phân tích chi tiết chức năng và quy tắc kỹ thuật của từng thành phần trong giải phẫu phần tử HTML:

[Bảng] Bảng Phân Tích Chi Tiết Các Thành Phần Trong Giải Phẫu HTML Element

Thành phần giải phẫu	Cú pháp đại diện	Ý nghĩa và Quy tắc kỹ thuật
Thẻ mở (*Opening Tag*)	`<p ...>`	Đánh dấu điểm bắt đầu của phần tử HTML. Bao gồm tên thẻ nằm giữa cặp dấu `<` và `>`.
Thuộc tính (*Attribute*)	`class="highlight"`	Cung cấp thêm thông tin cấu hình hoặc định danh cho phần tử. Luôn khai báo bên trong thẻ mở theo dạng cặp `tên="giá_trị"`.
Nội dung (*Content*)	`Chào mừng...`	Phần văn bản, hình ảnh hoặc các thẻ con lồng nhau hiển thị cho người dùng tương tác.
Thẻ đóng (*Closing Tag*)	`</p>`	Đánh dấu điểm kết thúc của phần tử HTML. Có cú pháp tương tự thẻ mở nhưng bổ sung dấu gạch chéo `/` trước tên thẻ.

[Khái Niệm] Phân Biệt Cốt Lõi: Thẻ (Tag) và Phần Tử (Element)

Trong lập trình Web, thuật ngữ **Thẻ (Tag)** và **Phần tử (Element)** thường bị nhầm lẫn:

- **Tag (Thẻ)**: Chỉ bao gồm đoạn mã khai báo mở `<p>` hoặc đóng `</p>`.
- **Element (Phần tử)**: Bao gồm toàn bộ cấu trúc từ thẻ mở, các thuộc tính, phần nội dung bên trong cho đến hết thẻ đóng (ví dụ: `<p class="highlight">Nội dung</p>`).

1. HTML Elements (Phần tử HTML)

Element (phần tử) trong HTML là một thành phần cơ bản của trang web, được sử dụng để tạo và hiển thị nội dung. Một phần tử HTML bao gồm một thẻ mở, nội dung và một thẻ đóng (đối với thẻ có đóng).

- **Cấu trúc cơ bản của một phần tử HTML**: `<tagname> Nội dung </tagname>`. Ví dụ: `<p>Đây là một đoạn văn bản.</p>`.
- **Thẻ tự đóng (Self-closing tags)**: Một số phần tử có thể không có thẻ đóng, được gọi là các thẻ tự đóng. Ví dụ: `<img src="image.jpg" alt="Mô tả hình ảnh">`, `<br>`, `<input type="text">`. Trong các thẻ tự đóng này, dấu `/>` có thể được sử dụng (ví dụ: `<img />`) nhưng hoàn toàn không bắt buộc trong chuẩn HTML5.

2. Các loại Elements trong HTML

Các phần tử HTML được phân thành nhiều nhóm chức năng dựa trên cách hiển thị và hoạt động của chúng:

- **Block-level Elements (Phần tử khối)**: Luôn bắt đầu trên một dòng mới và chiếm toàn bộ chiều rộng của phần tử cha (trải dài hết bề ngang màn hình). Ví dụ: `<div>`, `<p>`, `<h1>` - `<h6>`, `<ul>`, `<ol>`, `<table>`.

- **Inline Elements (Phần tử nội dòng)**: Chỉ chiếm không gian vừa đủ chứa nội dung của nó và không bắt đầu trên một dòng mới. Ví dụ: `<a>`, `<span>`, `<b>`, `<i>`, `<img>`.
- **Self-closing Elements (Phần tử tự đóng)**: Không có thẻ đóng, chỉ cần một thẻ mở duy nhất và dấu `/` tùy chọn. Ví dụ: `<img>`, `<input>`, `<br>`, `<hr>`.

Ví dụ về các loại phần tử HTML

```html
<!-- 1. Phần tử khối (Block-level): Luôn chiếm toàn bộ chiều rộng --><div>
  <h1>Đây là tiêu đề</h1>
  <p>Đây là đoạn văn.</p>
</div>

<!-- 2. Phần tử nội dòng (Inline): Chỉ chiếm không gian của nội dung --><p>Đây
    là<span style="color: red;">một phần của đoạn văn</span>được tô màu đỏ.</p>

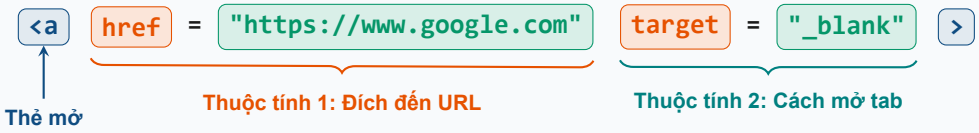
<!-- 3. Phần tử tự đóng (Self-closing): Không cần thẻ đóng đóng
    lại --><img src="logo.png" alt="Logo của trang web">
<input type="text" placeholder="Nhập tên của bạn">
```

3. HTML Attributes (Thuộc tính HTML)

[Khái Niệm] Khái Niệm Cốt Lõi: Thuộc Tính HTML (HTML Attributes)

Attribute (thuộc tính) là các thông số mở rộng được khai báo **bên trong thẻ mở**, có vai trò cung cấp thông tin bổ sung, định danh hoặc cấu hình hành vi hiển thị của phần tử HTML.

- **Vị trí bắt buộc**: Luôn nằm ngay sau tên thẻ trong thẻ mở (không bao giờ nằm trong thẻ đóng).
- **Quy tắc cú pháp chuẩn**: Luôn có dạng cặp khóa – giá trị `name="value"`.
- **Đa thuộc tính**: Một thẻ mở có thể chứa đồng thời nhiều thuộc tính, mỗi thuộc tính phân tách nhau bằng ít nhất một khoảng trắng (*space*).



[Bảng] Bảng Giải Phẫu Chi Tiết Các Thành Phần Cú Pháp Thuộc Tính

Thành phần	Cú pháp đại diện	Ý nghĩa và Quy tắc kỹ thuật
Tên thuộc tính (*Name*)	`href, target, class`	Viết thường toàn bộ theo chuẩn W3C, xác định loại cấu hình cần áp dụng cho thẻ.
Dấu gán (*Equal sign*)	`=`	Phân cách giữa tên thuộc tính và giá trị, viết liền không khoảng cách.
Giá trị (*Value*)	`"https://...", "_blank"`	Luôn đặt trong cặp dấu nháy kép `""`, chứa tham số cấu hình tương ứng.
Khoảng cách (*Space*)	`dấu cách giữa các thuộc tính`	Dùng để phân tách khi một thẻ mở khai báo đồng thời từ hai thuộc tính trở lên.

4. Các loại thuộc tính trong HTML

Trong tài liệu HTML5, các thuộc tính được phân chia thành 5 nhóm kỹ thuật chính:

[Bảng] Phân Loại 5 Nhóm Thuộc Tính Quan Trọng Trong HTML5

Nhóm thuộc tính	Các thuộc tính tiêu biểu	Mục đích sử dụng & Lưu ý kỹ thuật
Toàn cục (*Global*)	`id`, `class`, `style`, `title`, `hidden`	Áp dụng được cho mọi thẻ HTML. Dùng để định danh, style CSS và gán hành vi JavaScript.
Liên kết (*Links*)	`href`, `target`, `rel`, `download`	Cấu hình địa chỉ đích, phương thức mở tab và quan hệ bảo mật giữa trang hiện tại và URL đích.
Đa phương tiện (*Media*)	`src`, `alt`, `width`, `height`, `loading`	Chỉ định nguồn tài nguyên ảnh/video, văn bản thay thế (*alt text*) và tối ưu hiệu suất tải.
Biểu mẫu (*Forms*)	`type`, `name`, `value`, `placeholder`	Kiểm soát định dạng nhập liệu (*text, email, password...*) và truyền dữ liệu lên máy chủ.
Logic (*Boolean*)	`required`, `disabled`, `readonly`, `checked`	Thuộc tính nhị phân (chỉ cần khai báo tên thuộc tính là kích hoạt, không cần truyền giá trị).

Ví dụ minh họa cách khai báo các loại thuộc tính HTML

```
<!-- 1. Thuộc tính toàn cục (Global Attributes) --><p id="intro" class="highlight
    " style="color: blue;" title="Đây là một đoạn văn
    bản">
  Đoạn văn bản có thuộc tính
</p>

<!-- 2. Thuộc tính liên kết (<a>) và hình ảnh
    (<img>) --><a href="https://example.com" target="_blank">Nhấn vào
    đây</a><img src="image.jpg" alt="Ảnh đẹp" width="300">

<!-- 3. Thuộc tính biểu mẫu (<input>) và thuộc tính
    Boolean --><input type="text" name="username" placeholder="Nhập tên của
    bạn" required>
<input type="checkbox" checked> Tôi đồng ý
<input type="text" disabled placeholder="Bạn không thể nhập vào đây">
```

5. Phân biệt Elements và Attributes

[Bảng] Bảng So Sánh Giữa HTML Elements và HTML Attributes

Đặc điểm	HTML Elements (Phần tử)	HTML Attributes (Thuộc tính)
Chức năng	Tạo cấu trúc và nội dung hiển thị cho trang web.	Cung cấp thông tin bổ sung hoặc cấu hình cho các phần tử.
Vị trí	Bao quanh phần nội dung, có thể có cả thẻ mở và thẻ đóng.	Luôn nằm bên trong thẻ mở của phần tử.
Ví dụ	`<p>Đoạn văn</p>`	`<p style="color: red;">Đoạn văn</p>`

6. Tổng kết

[Ghi Chú] Tổng Kết Cốt Lõi Về Elements & Attributes

- **Elements (Phần tử)** là khối xây dựng chính của tài liệu HTML, có thể chứa nội dung hoặc lồng chứa các phần tử con khác.
- **Attributes (Thuộc tính)** cung cấp thông tin cấu hình bổ sung về phần tử, luôn nằm trong thẻ mở dưới dạng `key="value"`.
- Các phần tử HTML được phân loại dựa trên phạm vi hiển thị: phần tử khối (*block-level*), phần tử nội dòng (*inline*) và phần tử tự đóng (*self-closing*).
- Các thuộc tính quan trọng như `id`, `class`, `style`, `src`, `href`, `alt`, `placeholder`, cũng như các thuộc tính Boolean như `checked`, `disabled` rất hữu ích và phổ biến trong lập trình web.

3.2 Cấu Trúc Cơ Bản Của Một Tài Liệu HTML

Một trang web HTML chuẩn mực có cấu trúc cú pháp cơ bản như sau:

Mã Nguồn HTML5 Cơ Bản Tiêu Chuẩn

```
<!DOCTYPE html>
<html lang="vi">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Tiêu đề Trang</title>
</head>
<body>
    <h1>Chào mừng đến với HTML</h1>
    <p>Đây là một đoạn văn bản.</p>
</body>
</html>
```

3.2.1 Giải Thích Chi Tiết 10 Thành Phần Trong Cấu Trúc HTML

[Bảng] Bảng Phân Tích Chi Tiết 10 Thành Phần Trong Cấu Trúc HTML5

Mã nguồn / Dòng	Thành phần	Giải thích tác dụng & Ý nghĩa kỹ thuật cốt lõi
`<!DOCTYPE html>`	**Khai báo DOCTYPE**	Khai báo chuẩn HTML5. Giúp trình duyệt chạy ở chế độ chuẩn (*Standards Mode*), tránh rơi vào *Quirks Mode* gây vỡ khung Box Model.
`<html lang="vi">`	**Thẻ gốc HTML**	Thẻ gốc bao bọc toàn bộ trang web. Thuộc tính `lang="vi"` giúp Googlebot phân loại tiếng Việt và Trình đọc màn hình phát âm chuẩn.
`<head>`	**Đầu trang Metadata**	Container chứa các thông tin siêu dữ liệu (Metadata), liên kết CSS, Font chữ. Không hiển thị trực tiếp lên giao diện.
`<meta charset="UTF-8">`	**Mã hóa UTF-8**	Khai báo bộ mã ký tự Unicode UTF-8. Bảo đảm hiển thị chính xác tiếng Việt có dấu (*á, ớ, ũ, đ...*), chống lỗi vỡ phông ??.
`<meta name="viewport"...>`	**Viewport Responsive**	Điều chỉnh trang web co giãn vừa vặn trên điện thoại di động (`width=device-width`) với mức zoom ban đầu `initial-scale=1.0`.
`<title>... </title>`	**Tiêu đề trang web**	Tiêu đề hiển thị trên Tab trình duyệt và dòng tiêu đề kết quả tìm kiếm Google (SERP). Hỗ trợ SEO và UX.
`<body>`	**Thân trang Web**	Container chứa toàn bộ nội dung hiển thị cho người dùng tương tác (văn bản, ảnh, video, biểu mẫu, nút bấm...).
`<h1>...</h1>`	**Tiêu đề chính cấp 1**	Thẻ tiêu đề lớn nhất, quan trọng nhất trang. Định hình chủ đề chính cho bài viết và giúp Googlebot đánh giá từ khóa.
`<p>...</p>`	**Đoạn văn bản**	Thẻ tạo đoạn văn bản. Trình duyệt tự động chèn khoảng trống lề trên/dưới giúp bố cục nội dung mạch lạc, dễ đọc.
`</body></html>`	**Các thẻ đóng**	Đánh dấu kết thúc phần thân `</body>` và kết thúc toàn bộ tài liệu HTML `</html>`.

[Ghi Chú] Tóm Tắt Nhanh 10 Thành Phần Khai Báo HTML Chuẩn

- `<!DOCTYPE html>` → Khai báo chuẩn HTML5.
- `<html lang="vi">` → Thẻ gốc và ngôn ngữ tiếng Việt.
- `<head>` → Chứa thông tin siêu dữ liệu (Metadata).
- `<meta charset="UTF-8">` → Mã hóa phông chữ tiếng Việt có dấu.
- `<meta name="viewport">` → Tối ưu hiển thị Responsive trên di động.
- `<title>` → Tiêu đề hiển thị trên Tab trình duyệt & Google SERP.
- `<body>` → Chứa nội dung hiển thị của trang.
- `<h1>` → Tiêu đề chính quan trọng nhất.
- `<p>` → Đoạn văn bản.
- `</body></html>` → Đóng kết thúc tài liệu HTML.

3.3 Chi Tiết Chuyên Sâu Về Thẻ <meta> Trong HTML (Meta Masterclass)

3.3.1 Thẻ <meta> là gì?

Thẻ <meta> được đặt bên trong phần `<head>` của tài liệu HTML. Nó cung cấp các thông tin siêu dữ liệu (**Metadata**) – tức là các dữ liệu mô tả thông tin về trang web nhưng **không hiển thị trực tiếp** trên giao diện màn hình.

Các công cụ tìm kiếm (Googlebot, Bingbot), trình duyệt web và các nền tảng mạng xã hội (Facebook, Twitter, Zalo) sử dụng thẻ `<meta>` để phân tích nội dung, xác định cách thức hiển thị và xử lý trang web.

Cấu trúc tổng quát của thẻ `<meta>`:

```
<meta name="tên-thuộc-tính" content="giá-trị">
```

3.3.2 Các loại thẻ <meta> phổ biến & ứng dụng

1. Mã hóa ký tự (charset)

```
<meta charset="UTF-8">
```

Tác dụng: Xác định bộ mã ký tự của trang web là UTF-8. Giúp trình duyệt hiển thị đúng các ký tự tiếng Việt có dấu (*á, ó, đ, ũ...*). Nếu thiếu thẻ này, trang web có nguy cơ vỡ phông và xuất hiện ký tự lỗi hoặc `??`.

2. Thiết lập Viewport (Tương thích thiết bị di động)

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Tác dụng: Đảm bảo trang web tự động co giãn vừa vặn trên thiết bị di động.

- `width=device-width`: Đặt chiều rộng trang bằng đúng chiều rộng màn hình thiết bị.
- `initial-scale=1.0`: Giữ nguyên tỷ lệ thu phóng mặc định ban đầu.

3. Mô tả trang web (description)

```
<meta name="description" content="Đây là một trang web học HTML cơ bản.">
```

Tác dụng: Cung cấp đoạn văn ngắn (từ 150 đến 160 ký tự) mô tả tóm tắt nội dung trang web cho các công cụ tìm kiếm.

MINH HỌA KẾT QUẢ TÌM KIẾM GOOGLE (SERP PREVIEW)

HTML Cơ Bản – Học HTML Từ A Đến Z

https://example.com/html-co-ban

Đây là một trang web học HTML cơ bản từ nền tảng đến nâng cao. Cung cấp đầy đủ kiến thức về cấu trúc HTML5, các thẻ meta chuẩn SEO và nguyên lý thiết kế web hiện đại.

Lưu ý: Nếu không khai báo thẻ `description`, Google sẽ tự động trích xuất một đoạn văn ngẫu nhiên bất kỳ trên trang web để hiển thị, gây mất tính chuyên nghiệp.

4. Từ khóa SEO (keywords)

```
<meta name="keywords" content="HTML, học lập trình, thiết kế web">
```

Tác dụng & Lưu ý: Trích dẫn các từ khóa chính của trang web. Trước đây thẻ này hỗ trợ SEO rất lớn, tuy nhiên hiện nay thuật toán tìm kiếm của Google đã ngừng sử dụng thẻ này để xếp hạng bài viết nhằm chống gian lận spam từ khóa.

5. Khai báo tác giả (author)

```
<meta name="author" content="Nguyễn Văn A">
```

Tác dụng: Xác định tên cá nhân hoặc tổ chức biên soạn mã nguồn trang web. Hữu ích cho việc quản trị hệ thống và xác minh bản quyền.

6. Điều hướng tự động sau khoảng thời gian (refresh)

```
<meta http-equiv="refresh" content="5; url=https://example.com">
```

Tác dụng: Tự động chuyển hướng (*Redirect*) người dùng sang trang `https://example.com` sau 5 giây.

[Cảnh Báo] Lưu ý về SEO với thẻ Refresh

Hạn chế sử dụng thẻ này cho mục đích chuyển hướng SEO, nên ưu tiên sử dụng mã chuyển hướng HTTP 301 trên máy chủ hoặc chuyển hướng bằng JavaScript.

7. Kiểm soát con bọ tìm kiếm (robots)

```
<meta name="robots" content="index, follow">
```

Tác dụng: Chỉ dẫn cho các con bọ tìm kiếm (*Search Engine Crawlers*) cách thức lập chỉ mục trang web.

- `index`: Cho phép Google lưu và hiển thị trang web trên kết quả tìm kiếm.

- `noindex`: Cấm Google lập chỉ mục trang này.
- `follow`: Cho phép con bọ đi theo các đường liên kết (*links*) có trên trang.
- `nofollow`: Cấm con bọ đi theo các đường liên kết trên trang.

Ví dụ chặn Google lập chỉ mục trang quản trị nội bộ hoặc trang thử nghiệm:

```
<meta name="robots" content="noindex, nofollow">
```

8. Kiểm soát bộ nhớ đệm Trình duyệt (cache-control)

```
<meta http-equiv="cache-control" content="no-cache">
```

Tác dụng: Ngăn trình duyệt lưu lại bộ nhớ đệm (*cache*) của trang web. Rất hữu ích khi bạn cập nhật phiên bản ứng dụng liên tục và muốn người dùng luôn tải về dữ liệu mới nhất.

9. Mạng xã hội: Open Graph (Facebook) & Twitter Cards

Khi chia sẻ đường link trang web lên các mạng xã hội như Facebook, Zalo, LinkedIn, Twitter... các thẻ `<meta>` chuẩn hóa sẽ quyết định giao diện thẻ hiển thị (*Rich Cards*):

Các Thẻ Meta Facebook Open Graph Tiêu Chuẩn (og:...)

```
<meta property="og:title" content="Học HTML từ A đến Z">
<meta property="og:description" content="Trang web dạy HTML miễn phí từ cơ bản
    đến nâng cao.">
<meta property="og:image" content="https://example.com/image.jpg">
<meta property="og:url" content="https://example.com">
```

Các Thẻ Meta Twitter Cards Tiêu Chuẩn (twitter:...)

```
<meta name="twitter:card" content="summary_large_image">
<meta name="twitter:title" content="Học HTML từ A đến Z">
<meta name="twitter:description" content="Trang web dạy HTML miễn phí từ cơ bản
    đến nâng cao.">
<meta name="twitter:image" content="https://example.com/image.jpg">
```

3.3.3 Ví dụ mã nguồn tệp HTML chứa bộ thẻ <meta> đầy đủ tiêu chuẩn

Tệp Mã Nguồn HTML Mẫu Hoàn Chỉnh Bộ Thẻ Meta Tiêu Chuẩn

```
<!DOCTYPE html>
<html lang="vi">
<head>
    <!-- 1. Mã hóa ký tự tiếng Việt -->    <meta charset="UTF-8">

    <!-- 2. Cấu hình Responsive di động -->    <meta name="viewport" content="
    width=device-width, initial-scale=1.0">

    <!-- 3. SEO Metadata cơ bản -->    <meta name="description" content="Trang
    web dạy HTML miễn phí từ cơ bản đến nâng cao.">
    <meta name="keywords" content="HTML, CSS, JavaScript, học lập trình, thiết kế
    web">
    <meta name="author" content="Nguyễn Văn A">
    <meta name="robots" content="index, follow">

    <!-- 4. Facebook Open Graph Tags -->
    <meta property="og:title" content="Học HTML từ A đến Z">
    <meta property="og:description" content="Trang web dạy HTML miễn phí từ cơ
    bản đến nâng cao.">
    <meta property="og:image" content="https://example.com/image.jpg">
    <meta property="og:url" content="https://example.com">

    <!-- 5. Tiêu đề trang tab trình duyệt -->    <title>Học HTML Cơ Bản</title>
</head>
<body>
    <h1>Chào mừng đến với HTML!</h1>
    <p>Trang web học lập trình web chuẩn mực dành cho kĩ sư Frontend.</p>
</body>
</html>
```

[Bảng] Tóm Tắt Bốn Thẻ <meta> Bắt Buộc Phải Có Cho Mọi Website

Cú pháp Thẻ Meta	Tác dụng cốt lõi không thể thiếu
`<meta charset="UTF-8">`	Bảo đảm hiển thị đúng tiếng Việt có dấu, không vỡ phông.
`<meta name="viewport" content="...">`	Bảo đảm hiển thị chuẩn mực trên mọi màn hình di động.
`<meta name="description" content="...">`	Hiển thị đoạn văn tóm tắt chuyên nghiệp trên Google SERP.
`<meta property="og:title" content="...">`	Hiển thị ảnh đại diện, tiêu đề đẹp mắt khi chia sẻ lên MXH.

3.4 Các Thẻ HTML Phổ Biến Trong Phát Triển Web

3.4.1 Thẻ Tiêu Đề Trong HTML (Heading Tags: <h1> - <h6>)

Heading Tag (Thẻ tiêu đề) trong HTML (gồm 6 cấp độ từ `<h1>` đến `<h6>`) là nhóm thẻ được sử dụng để xác định tiêu đề và định hình cấu trúc phân cấp nội dung cho trang web.

- **Định vị chức năng**: Tiêu đề giúp phân chia nội dung rõ ràng, mạch lạc, giúp cả người đọc lẫn các con bọ tìm kiếm (*Search Engine Crawlers*) dễ dàng thấu hiểu mô hình cấu trúc của trang web.
- **Phân cấp 6 cấp độ**: Cấu trúc bao gồm 6 cấp độ từ `<h1>` (quan trọng nhất) đến `<h6>` (ít quan trọng nhất).
- **Kích thước mặc định**: Theo thiết lập mặc định của các trình duyệt, kích thước phông chữ (*font-size*) sẽ giảm dần từ `<h1>` xuống `<h6>`.

1. Cấu trúc và cú pháp khai báo thẻ tiêu đề

Cú Pháp Khai Báo 6 Cấp Độ Thẻ Tiêu Đề Heading

```
<h1>Tiêu đề cấp 1</h1>
<h2>Tiêu đề cấp 2</h2>
<h3>Tiêu đề cấp 3</h3>
<h4>Tiêu đề cấp 4</h4>
<h5>Tiêu đề cấp 5</h5>
<h6>Tiêu đề cấp 6</h6>
```

Dưới đây là tệp mã nguồn HTML minh họa cách tổ chức các thẻ tiêu đề trong một tài liệu web thực tế:

Ví Dụ Thực Tế Tổ Chức Thẻ Tiêu Đề Trong Tài Liệu HTML

```html
<!DOCTYPE html>
<html lang="vi">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Ví dụ thẻ tiêu đề</title>
</head>
<body>
    <h1>Chào mừng đến với trang web của tôi</h1>
    <h2>Giới thiệu về HTML</h2>
    <h3>Lịch sử của HTML</h3>
    <h4>HTML5 có gì mới?</h4>
    <h5>Những cải tiến về API</h5>
    <h6>Kết luận</h6>
</body>
</html>
```

KẾT QUẢ HIỂN THỊ KÍCH THƯỚC TRÊN TRÌNH DUYỆT WEB (BROWSER PREVIEW)

Chào mừng đến với trang web của tôi

<h1> **Font-size**: 32px (2.00em) | **Vai trò**: Tiêu đề chính duy nhất

Giới thiệu về HTML

<h2> **Font-size**: 24px (1.50em) | **Vai trò**: Tiêu đề phụ mục lớn

Lịch sử của HTML

<h3> **Font-size**: 18.72px (1.17em) | **Vai trò**: Tiêu đề con trong h2

HTML5 có gì mới?

<h4> **Font-size**: 16px (1.00em) | **Vai trò**: Phân nhỏ mục h3

Những cải tiến về API

<h5> **Font-size**: 13.28px (0.83em) | **Vai trò**: Chi tiết nhỏ trong h4

Kết luận

<h6> **Font-size**: 10.72px (0.67em) | **Vai trò**: Phân cấp sâu nhất

[Ghi Chú] Kết Quả Hiển Thị & Ý Nghĩa Quản Lý

- `<h1>` có kích thước lớn nhất và nổi bật nhất, dùng làm tiêu đề chính của toàn bộ trang.
- `<h2>` đến `<h6>` đóng vai trò làm các tiêu đề phụ và tiêu đề con, giúp tổ chức các mục nhỏ một cách hợp lý và chuẩn mực.

2. Kích thước phông chữ mặc định và tùy chỉnh bằng CSS

Mặc định, các trình duyệt web tự động áp dụng thông số kích thước phông chữ (*font-size*) cho các thẻ tiêu đề dựa trên đơn vị `px` (hoặc tương đương `em`):

[Bảng] Bảng Thông Số Kích Thước Phông Chữ Mặc Định Của Các Thẻ Tiêu Đề

Thẻ tiêu đề	Cỡ chữ mặc định (px)	Quy đổi tương đương (em / rem)
`<h1>`	`32px`	2.00em (lớn nhất)
`<h2>`	`24px`	1.50em
`<h3>`	`18.72px`	1.17em
`<h4>`	`16px`	1.00em (bằng kích thước chữ cơ sở)
`<h5>`	`13.28px`	0.83em
`<h6>`	`10.72px`	0.67em (nhỏ nhất)

Trong thực tế lập trình giao diện, bạn có thể dễ dàng can thiệp và ghi đè kích thước hoặc màu sắc của các thẻ tiêu đề bằng mã CSS:

Mã Nguồn CSS Tùy Chỉnh Kích Thước Và Màu Sắc Cho Thẻ Tiêu Đề

```css
/* Tùy chỉnh kích thước và màu sắc cho thẻ h1 */
h1 {
    font-size: 40px; /* Tăng kích thước chữ cho h1 */
    color: blue; /* Màu xanh dương */
}

/* Tùy chỉnh kích thước và màu sắc cho thẻ h2 */
h2 {
    font-size: 30px;
    color: red;
}
```

3. Hướng dẫn ngữ cảnh sử dụng chuẩn xác từng cấp độ thẻ tiêu đề

[Bảng] Bảng Hướng Dẫn Chi Tiết Ngữ Cảnh Sử Dụng Thẻ Tiêu Đề

Thẻ	Khi nào nên sử dụng? Quy tắc ngữ cảnh kỹ thuật
`<h1>`	Dùng cho tiêu đề chính của trang, chỉ nên có một thẻ `<h1>` trên mỗi trang.
`<h2>`	Dùng cho tiêu đề phụ, thường là các phần lớn trong nội dung.
`<h3>`	Dùng cho tiêu đề con của `<h2>`, chia nhỏ nội dung hơn.
`<h4>`	Dùng khi cần phân nhỏ nội dung trong `<h3>`.
`<h5>`	Hiếm khi dùng, chỉ dùng khi cần chia nhỏ hơn trong `<h4>`.
`<h6>`	Ít được sử dụng, chỉ khi cần phân chia nhỏ hơn nữa.

4. Tầm quan trọng của thẻ tiêu đề đối với SEO (Search Engine Optimization)

Trong kỹ thuật tối ưu hóa công cụ tìm kiếm (SEO), các thẻ tiêu đề đóng vai trò là ”bản đồ chỉ đường” định hình chủ đề cho các con bọ thu thập dữ liệu của Google:

- **Thẻ `<h1>` là quan trọng nhất đối với SEO**: Thẻ này giúp Google hiểu nội dung chính của trang.
- **Duy nhất một `<h1>`**: Chỉ nên có một `<h1>` trên mỗi trang để tránh nhầm lẫn.
- **Phân chia logic `<h2>` - `<h6>`**: Dùng `<h2>` - `<h6>` để phân chia nội dung hợp lý, giúp bài viết dễ đọc hơn.
- **Tránh lạm dụng**: Không lạm dụng quá nhiều tiêu đề trong một trang, nên có cấu trúc logic.

Dưới đây là ví dụ minh họa cách tối ưu tiêu đề cho SEO:

Mã Nguồn Mẫu Phân Cấp Thẻ Tiêu Đề Chuẩn SEO

```
<h1>Hướng dẫn học HTML cơ bản</h1>
<h2>1. HTML là gì?</h2>
<h3>1.1. Lịch sử của HTML</h3>
<h3>1.2. Các phiên bản HTML</h3>
<h2>2. Cách viết mã HTML</h2>
<h3>2.1. Cấu trúc cơ bản của một trang HTML</h3>
<h3>2.2. Các thẻ quan trọng trong HTML</h3>
```

[Mẹo] Lợi Ích Cốt Lõi Khi Phân Cấp Chuẩn SEO

- **Dành cho người đọc**: Giúp người đọc dễ theo dõi nội dung bài viết.
- **Dành cho công cụ tìm kiếm**: Công cụ tìm kiếm hiểu rõ cấu trúc bài viết hơn.

5. Các lưu ý quan trọng và lỗi sai phổ biến khi dùng thẻ tiêu đề

[Cảnh Báo] Các Lỗi Sai Thường Gặp & Quy Tắc Sử Dụng Thẻ Tiêu Đề

- **[SAI] Không sử dụng `<h1>` nhiều lần trong một trang**.
- **[SAI] Không bỏ qua cấp độ tiêu đề** (không nhảy từ `<h1>` xuống `<h4>` mà không có `<h2>`, `<h3>`).
- **[ĐÚNG] Sử dụng tiêu đề theo thứ tự hợp lý** để đảm bảo cấu trúc rõ ràng.
- **[SAI] Không dùng tiêu đề chỉ để làm chữ to**. Nếu chỉ muốn thay đổi kích thước chữ, hãy dùng CSS (`font-size`) thay vì dùng thẻ tiêu đề.

Ví dụ sai cách dùng thẻ tiêu đề:

Mã Nguồn Ví Dụ Sai Cách Dùng Thẻ Tiêu Đề (Nhảy Cấp)

```
<!-- [SAI] SAI: Nhảy từ h1 xuống h4 mà không có h2, h3 -->
<h1>Chào mừng đến với trang web</h1>
<h4>Giới thiệu về HTML</h4>
<h2>Lịch sử HTML</h2>
```

Ví dụ đúng cách dùng thẻ tiêu đề:

Mã Nguồn Ví Dụ Đúng Cách Dùng Thẻ Tiêu Đề (Phân Cấp Tuần Tự)

```
<!-- [ĐÚNG] ĐÚNG: Phân cấp tuần tự h1 -> h2 -> h3 -->
<h1>Chào mừng đến với trang web</h1>
<h2>Giới thiệu về HTML</h2>
<h3>Lịch sử HTML</h3>
```

6. Tổng kết bài học

[Ghi Chú] Tổng Kết Các Điểm Cốt Lõi Về Thẻ Tiêu Đề (Heading Tags)

- Thẻ tiêu đề `<h1>` - `<h6>` giúp tổ chức nội dung trang web rõ ràng.
- Chỉ nên có một `<h1>` trên mỗi trang web.
- Dùng `<h2>` - `<h6>` theo thứ tự logic để phân chia nội dung.
- Có ảnh hưởng lớn đến SEO, giúp công cụ tìm kiếm hiểu cấu trúc trang web.
- Không nên dùng tiêu đề chỉ để làm chữ to, hãy dùng CSS để điều chỉnh kích thước chữ.

7. Đọc thêm: Chuyên sâu về Heading trong Frontend & HTML5 Modern Standard

[Khái Niệm] Mở Rộng Kiến Thức: Kỹ Thuật Heading Chuyên Sâu Cho Kĩ Sư Frontend

- **Khả năng truy cập (Accessibility - A11y)**: Trình đọc màn hình (*Screen Readers* như NVDA, JAWS) cho phép người dùng khiếm thị bấm phím `H` để duyệt nhanh danh sách các tiêu đề trên trang. Việc nhảy cấp tiêu đề (ví dụ từ `<h1>` xuống `<h4>`) sẽ khiến trình đọc màn hình báo lỗi cấu trúc bài viết.
- **Chuẩn HTML5 Sectioning Roots**: Theo tiêu chuẩn HTML5, khi tiêu đề nằm trong các thẻ ngữ nghĩa độc lập như `<article>` hoặc `<section>`, mỗi thẻ ngữ nghĩa được tính là một phạm vi (*sectioning context*) có thể chứa thẻ `<h1>` riêng biệt mà vẫn tối ưu SEO.
- **CSS Reset Headings**: Trong các dự án thực tế dùng Framework như Tailwind CSS hay Bootstrap, kích thước phông chữ của các thẻ tiêu đề thường được reset về mặc định và yêu cầu lập trình viên định kiểu bằng class CSS để tăng tính đồng bộ UI.

3.4.2 Thẻ Văn Bản Và Định Dạng Trong HTML (Text Formatting Tags)

Trong HTML, các thẻ văn bản được sử dụng để định dạng, trình bày cấu trúc và truyền tải ý nghĩa ngữ nghĩa (*semantics*) cho nội dung văn bản trên trang web. Dưới đây là danh sách tổng hợp chi tiết tất cả các thẻ văn bản quan trọng cùng ví dụ thực thi thực tế.

1. Thẻ đoạn văn (`<p>` - Paragraph)

Thẻ `<p>` (viết tắt của **Paragraph**) được dùng để tạo một đoạn văn bản trên trang web. Theo mặc định, trình duyệt sẽ tự động chèn một khoảng trống phía trên/dưới và xuống dòng sau mỗi thẻ `<p>`.

Ví Dụ Khai Báo Thẻ Đoạn Văn

```
<p>Đây là một đoạn văn bản trong HTML.</p>
<p>HTML rất quan trọng trong phát triển web.</p>
```

KẾT QUẢ HIỂN THỊ TRÊN TRÌNH DUYỆT (BROWSER PREVIEW)

Đây là một đoạn văn bản trong HTML.

HTML rất quan trọng trong phát triển web.

Quy tắc thu gọn khoảng trắng (Whitespace Collapsing) của thẻ <p>:

Mặc định, trình duyệt HTML sẽ tự động bỏ qua và thu gọn các khoảng trắng liên tiếp trong nội dung văn bản của thẻ `<p>`:

- Nhiều khoảng trắng liên tiếp sẽ bị thu gọn thành một khoảng trắng duy nhất.
- Dấu xuống dòng trong tệp mã nguồn cũng không được tính là khoảng xuống dòng trên giao diện.

Mã Nguồn Minh Họa Khoảng Trắng Bị Thu Gọn (Nhiều Khoảng Trắng và Xuống Dòng)

```
<p>Đây       là       một       đoạn       văn       bản.</p>
```

KẾT QUẢ HIỂN THỊ TRÊN TRÌNH DUYỆT (BROWSER PREVIEW)

Đây là một đoạn văn bản. (Các khoảng trắng thừa đã bị trình duyệt thu gọn)

Bảng so sánh các phương pháp xử lý và giữ khoảng trắng trong HTML:

[Bảng] Bảng So Sánh Các Giải Pháp Giữ Khoảng Trắng Vẫn Xuống Dòng

Phương pháp / Cú pháp	Hiệu quả kỹ thuật và đánh giá
` `	Giữ khoảng trắng nhưng phải thêm thủ công từng chỗ.
`<pre>`	Giữ nguyên cả khoảng trắng và xuống dòng, nhưng không thể dùng CSS để định dạng dễ dàng.
`white-space: pre;`	Giữ nguyên khoảng trắng và xuống dòng, có thể kết hợp với CSS.
`white-space: pre-wrap;`	Giữ khoảng trắng nhưng vẫn cho phép tự động xuống dòng khi chạm mép màn hình.

[Ghi Chú] Kết Luận Phương Pháp Xuống Dòng & Giữ Khoảng Trắng

- Nếu cần hiển thị văn bản đúng với định dạng gốc → Dùng `<pre>` hoặc CSS `white-space: pre;`.
- Nếu chỉ muốn thêm vài khoảng trắng → Dùng ký tự thực thể ` `.

2. Thẻ xuống dòng (
 - Line Break)

Thẻ `<br>` (viết tắt của **Line Break**) dùng để xuống dòng trực tiếp trong một đoạn văn mà không cần tạo thành một đoạn văn mới. Đây là thẻ tự đóng (*self-closing tag*), không cần thẻ đóng `</br>`.

Ví Dụ Ngắt Dòng Bằng Thẻ br

```
<p>Đây là dòng đầu tiên.<br>Đây là dòng thứ hai.</p>
```

KẾT QUẢ HIỂN THỊ TRÊN TRÌNH DUYỆT (BROWSER PREVIEW)

Đây là dòng đầu tiên.
Đây là dòng thứ hai.

3. Thẻ ngắt dòng theo chủ đề (<hr> - Horizontal Rule)

Thẻ <hr> (viết tắt của **Horizontal Rule**) dùng để tạo một đường kẻ ngang phân tách các nội dung theo chủ đề khác nhau. Đây cũng là một thẻ tự đóng.

Ví Dụ Phân Tách Phân Đoạn Bằng Thẻ hr

```
<p>Đây là phần đầu tiên.</p>
<hr>
<p>Đây là phần tiếp theo.</p>
```

KẾT QUẢ HIỂN THỊ TRÊN TRÌNH DUYỆT (BROWSER PREVIEW)

Đây là phần đầu tiên.

Đây là phần tiếp theo.

4. Thẻ in đậm (<b> và)

- <b> (**Bold**): In đậm văn bản thuần túy về mặt thị giác, không mang ý nghĩa ngữ nghĩa đặc biệt.
- (**Strong Importance**): In đậm văn bản và khẳng định tầm quan trọng của nội dung, mang lại giá trị SEO tốt hơn.

Mã Nguồn Khai Báo Thẻ In Đậm b Và strong

```
<p>Đây là một<b>đoạn văn bản in đậm</b>.</p>
<p>Đây là một<strong>đoạn văn bản quan trọng</strong>.</p>
```

KẾT QUẢ HIỂN THỊ TRÊN TRÌNH DUYỆT (BROWSER PREVIEW)

Đây là một **đoạn văn bản in đậm**.

Đây là một **đoạn văn bản quan trọng**.

5. Thẻ in nghiêng (<i> và)

- <i> (**Italic**): In nghiêng văn bản thuần túy về thị giác, không mang ý nghĩa nhấn mạnh.
- (**Emphasized Text**): In nghiêng văn bản và nhấn mạnh nội dung, hỗ trợ tối ưu SEO và trình đọc màn hình.

Mã Nguồn Khai Báo Thẻ In Nghiêng i Và em

```
<p>Đây là một<i>đoạn văn bản in nghiêng</i>.</p>
<p>Đây là một<em>đoạn văn bản cần nhấn mạnh</em>.</p>
```

KẾT QUẢ HIỂN THỊ TRÊN TRÌNH DUYỆT (BROWSER PREVIEW)

Đây là một *đoạn văn bản in nghiêng*.

Đây là một *đoạn văn bản cần nhấn mạnh*.

6. Thẻ gạch chân (<u> - Underline)

Thẻ <u> (**Underline**) dùng để gạch chân văn bản.

Mã Nguồn Khai Báo Thẻ Gạch Chân u

```
<p>Đây là một<u>đoạn văn bản gạch chân</u>.</p>
```

KẾT QUẢ HIỂN THỊ TRÊN TRÌNH DUYỆT (BROWSER PREVIEW)

Đây là một đoạn văn bản gạch chân.

7. Thẻ gạch ngang (<s>, <del> và <strike>)

- <s> (**Strikethrough**): Gạch ngang văn bản nhưng không có ý nghĩa nội dung bị xóa.
- <del> (**Deleted Text**): Gạch ngang văn bản với ý nghĩa ngữ nghĩa là nội dung đã bị xóa bỏ.
- <strike> (**Strikeout**): Tương tự <s>, nhưng đã bị coi là lỗi thời (*deprecated*) trong chuẩn HTML5.

Mã Nguồn Khai Báo Các Thẻ Gạch Ngang Văn Bản

```
<p>Giá cũ:<s>500.000đ</s>Giá mới: 450.000đ</p>
<p>Giá cũ:<del>600.000đ</del>Giá mới: 550.000đ</p>
```

KẾT QUẢ HIỂN THỊ TRÊN TRÌNH DUYỆT (BROWSER PREVIEW)

Giá cũ: ~~500.000đ~~ Giá mới: 450.000đ

Giá cũ: ~~600.000đ~~ Giá mới: 550.000đ

8. Thẻ chỉ số trên (<sup>) và chỉ số dưới (<sub>)

- <sup> (**Superscript**): Chỉ số trên, thường dùng trong công thức số mũ hoặc ký hiệu chú thích.
- <sub> (**Subscript**): Chỉ số dưới, thường dùng trong công thức hóa học hoặc các biến số.

Mã Nguồn Khai Báo Chỉ Số Trên sup Và Chỉ Số Dưới sub

```
<p>Công thức toán học: x<sup>2</sup> + y<sup>2</sup> = z<sup>2</sup></p>
<p>Công thức hóa học: H<sub>2</sub>O</p>
```

KẾT QUẢ HIỂN THỊ TRÊN TRÌNH DUYỆT (BROWSER PREVIEW)

Công thức toán học: $x^2 + y^2 = z^2$

Công thức hóa học: H_2O

9. Thẻ đánh dấu nội dung (<mark> - Highlight)

Thẻ <mark> dùng để tô sáng (*highlight*) một đoạn văn bản nhằm gây sự chú ý đặc biệt.

Mã Nguồn Khai Báo Thẻ Tô Sáng mark

```
<p>Đây là một đoạn văn bản có từ<mark>quan trọng</mark>được tô sáng.</p>
```

KẾT QUẢ HIỂN THỊ TRÊN TRÌNH DUYỆT (BROWSER PREVIEW)

Đây là một đoạn văn bản có từ quan trọng được tô sáng.

10. Thẻ trích dẫn (<blockquote>, <q>, <cite>)

- <blockquote> (**Block Quotation**): Dùng để trích dẫn một khối văn bản dài.
- <q> (**Short Quotation**): Dùng để trích dẫn một câu ngắn trong dòng văn bản.
- <cite> (**Citation**): Dùng để ghi nhận nguồn gốc tham chiếu hoặc tên tác phẩm.

Mã Nguồn Khai Báo Các Thẻ Trích Dẫn

```
<blockquote>
    "Thành công không phải là chìa khóa của hạnh phúc. Hạnh phúc là chìa khóa của
    thành công." - Albert Schweitzer
</blockquote>
<p>Steve Jobs đã nói:<q>Stay hungry, stay foolish.</q></p>
<p><cite>HTML Guide</cite>là một tài liệu hướng dẫn HTML phổ biến.</p>
```

KẾT QUẢ HIỂN THỊ TRÊN TRÌNH DUYỆT (BROWSER PREVIEW)

> *"Thành công không phải là chìa khóa của hạnh phúc. Hạnh phúc là chìa khóa của thành công." – Albert Schweitzer*

Steve Jobs đã nói: *Stay hungry, stay foolish.*"

HTML Guide là một tài liệu hướng dẫn HTML phổ biến.

11. Thẻ định dạng trước (<pre> - Preformatted Text)

Thẻ <pre> (**Preformatted Text**) dùng để hiển thị văn bản giữ nguyên chính xác tất cả định dạng khoảng trắng và dấu xuống dòng từ tệp mã nguồn.

Mã Nguồn Căn Chỉnh Cột Bằng Thẻ Preformatted pre

```
<pre>
Họ và tên: Nguyễn Văn A
Nghề nghiệp: Kỹ sư Frontend
Kỹ năng: HTML5, CSS3, JavaScript
</pre>
```

KẾT QUẢ HIỂN THỊ TRÊN TRÌNH DUYỆT (MONOSPACE ALIGNMENT PREVIEW)

```
Họ và tên:      Nguyễn Văn A
Nghề nghiệp:     Kỹ sư Frontend
Kỹ năng:       HTML5, CSS3, JavaScript
```

12. Thẻ mã nguồn (<code>, <kbd>, <samp>, <var>)

- <code> (**Code**): Hiển thị một đoạn mã lập trình ngắn.
- <kbd> (**Keyboard Input**): Hiển thị phím bấm hoặc tổ hợp phím nhập từ bàn phím.
- <samp> (**Sample Output**): Hiển thị kết quả đầu ra mẫu của chương trình.
- <var> (**Variable**): Hiển thị một biến số trong toán học hoặc mã lập trình.

Mã Nguồn Khai Báo Nhóm Thẻ Mã Nguồn Chi Tiết (code, kbd, samp, var)

```
<p>Lệnh JavaScript:<code>console.log("Hello World");</code></p>
<p>Tổ hợp phím tắt:<kbd>Ctrl</kbd> + <kbd>Shift</kbd> + <kbd>R</kbd></p>
<p>Thông báo hệ thống:<samp>200 OK - Request Successful</samp></p>
<p>Biến toán học:<var>y</var> = <var>f</var>(<var>x</var>)</p>
```

KẾT QUẢ HIỂN THỊ TRÊN TRÌNH DUYỆT (PROGRAMMING ELEMENTS PREVIEW)

Lệnh JavaScript: `console.log("Hello World");`

Tổ hợp phím tắt: Ctrl + Shift + R

Thông báo hệ thống: `200 OK - Request Successful`

Biến toán học: $y = f(x)$

13. Thẻ văn bản nhỏ (<small> - Small Text)

1. Giới thiệu về thẻ <small>:

Thẻ <small> được dùng để hiển thị văn bản với kích thước nhỏ hơn văn bản thông thường. Chủ yếu được dùng để hiển thị ghi chú, chú thích, hoặc thông tin pháp lý/bản quyền. Kích thước chữ của thẻ <small> thường nhỏ hơn một cấp so với văn bản xung quanh nó và được hỗ trợ mặc định bởi tất cả các trình duyệt.

2. Cú pháp và ví dụ sử dụng:

Mã Nguồn Mẫu Sử Dụng Thẻ small

```
<p>Đây là văn bản bình thường.<small>Đây là văn bản nhỏ.</small></p>
```

KẾT QUẢ HIỂN THỊ TRÊN TRÌNH DUYỆT (BROWSER PREVIEW)

Đây là văn bản bình thường. Đây là văn bản nhỏ.

3. Ứng dụng thực tế của thẻ <small>:

- Hiển thị chú thích hoặc thông tin pháp lý.
- Dùng để hiển thị cảnh báo nhỏ hoặc ghi chú về bản quyền.
- Thường xuất hiện trong khu vực chân trang (*footer*) của trang web.

Mã Nguồn Mẫu Sử Dụng Thẻ small Trong Chân Trang (Footer)

```
<footer>
    <p>&copy; 2025 Công ty ABC.<small>Mọi quyền được bảo lưu.</small></p>
</footer>
```

KẾT QUẢ HIỂN THỊ TRÊN TRÌNH DUYỆT (BROWSER PREVIEW)

© 2025 Công ty ABC. Mọi quyền được bảo lưu.

4. Kết hợp thẻ <small> với CSS:

Mặc định, thẻ `<small>` giảm kích thước văn bản theo tỉ lệ trình duyệt, nhưng ta có thể dễ dàng tùy chỉnh bằng mã CSS:

Mã Nguồn Tùy Chỉnh Kích Thước Thẻ small Bằng CSS

```
<style>
    small {
        font-size: 12px;
        color: gray;
    }
</style>
<p>Chữ thường<small>và chữ nhỏ với CSS.</small></p>
```

KẾT QUẢ HIỂN THỊ TRÊN TRÌNH DUYỆT (BROWSER PREVIEW)

Chữ thường và chữ nhỏ với CSS.

5. So sánh thẻ <small> với các phương pháp giảm kích thước chữ khác:

[Bảng] Bảng So Sánh Các Phương Pháp Giảm Kích Thước Chữ Trong HTML

Cách thực hiện	Cú pháp đại diện	Kết quả hiển thị kỹ thuật
Dùng <small>	Chữ nhỏ	Kích thước giảm tự động (mặc định trình duyệt).
Dùng CSS (font-size)		Kiểm soát chính xác tuyệt đối kích thước phông chữ.
Dùng <sub> / <sup>	_{Chữ nhỏ dưới} / ^{...}	Kích thước nhỏ hơn nhưng đồng thời thay đổi vị trí tọa độ dòng.

14. Thẻ chèn nội dung mới (<ins> - Inserted Text)

Thẻ <ins> (**Inserted Text**) được sử dụng song hành với thẻ <del> để biểu thị đoạn văn bản mới được thêm vào trong quá trình chỉnh sửa tài liệu. Trình duyệt mặc định gạch chân dưới văn bản trong thẻ <ins>.

Mã Nguồn Sử Dụng Thẻ ins Kết Hợp Với del

```
<p>Giá cũ:<del>500.000đ</del>Giá mới:<ins>400.000đ</ins></p>
```

KẾT QUẢ HIỂN THỊ TRÊN TRÌNH DUYỆT (BROWSER PREVIEW)

Giá cũ: ~~500.000đ~~ Giá mới: 400.000đ

15. Thẻ từ viết tắt (<abbr> - Abbreviation)

Thẻ <abbr> (**Abbreviation**) dùng để định nghĩa các từ viết tắt. Khi kết hợp với thuộc tính title, trình duyệt sẽ hiển thị tooltip giải nghĩa đầy đủ khi người dùng rê chuột vào.

Mã Nguồn Sử Dụng Thẻ Abbreviation abbr

```
<p>Học lập trình web với<abbr title="HyperText Markup Language">HTML</abbr>và<
    abbr title="Cascading Style Sheets">CSS</abbr>.</p>
```

KẾT QUẢ HIỂN THỊ TRÊN TRÌNH DUYỆT (BROWSER PREVIEW)

Học lập trình web với HTML (Rê chuột hiện tooltip: *HyperText Markup Language*) và CSS.

16. Thẻ thời gian chuẩn hóa SEO (`<time>` - Time Element)

Thẻ `<time>` giúp máy móc và công cụ tìm kiếm Googlebot nhận diện chính xác dữ liệu ngày tháng thời gian thông qua thuộc tính chuẩn hóa `datetime`.

Mã Nguồn Khai Báo Thẻ Thời Gian time

```
<p>Bài viết xuất bản ngày<time datetime="2026-07-22">22/07/2026</time>.</p>
<p>Sự kiện diễn ra lúc<time datetime="2026-07-22T20:00">20:00 tối nay</time>.</p>
```

KẾT QUẢ HIỂN THỊ TRÊN TRÌNH DUYỆT (BROWSER PREVIEW)

Bài viết xuất bản ngày 22/07/2026. (Dữ liệu chuẩn hóa ISO: 2026-07-22)

Sự kiện diễn ra lúc 20:00 tối nay.

17. Thẻ thông tin liên hệ (`<address>` - Contact Info)

Thẻ `<address>` dùng để xác định thông tin liên lạc của tác giả hoặc chủ sở hữu bài viết/doanh nghiệp.

Mã Nguồn Khai Báo Thẻ Liên Hệ address

```
<address>
    Viết bởi Nguyễn Văn A<br>
    Email: contact@example.com<br>
    Địa chỉ: TP. Hồ Chí Minh, Việt Nam
</address>
```

KẾT QUẢ HIỂN THỊ TRÊN TRÌNH DUYỆT (BROWSER PREVIEW)

Viết bởi Nguyễn Văn A
Email: contact@example.com
Địa chỉ: TP. Hồ Chí Minh, Việt Nam

18. Thẻ định nghĩa thuật ngữ (`<dfn>` - Definition)

Thẻ `<dfn>` (**Definition Element**) dùng để đánh dấu lần đầu tiên một thuật ngữ kỹ thuật mới được giới thiệu và định nghĩa trong văn bản.

Mã Nguồn Khai Báo Thẻ Định Nghĩa dfn

```
<p><dfn>HTML</dfn>là ngôn ngữ đánh dấu siêu văn bản dùng để tạo cấu trúc trang
    web.</p>
```

KẾT QUẢ HIỂN THỊ TRÊN TRÌNH DUYỆT (BROWSER PREVIEW)

HTML là ngôn ngữ đánh dấu siêu văn bản dùng để tạo cấu trúc trang web.

19. Thẻ gom nhóm nội dòng trung tính (- Inline Container)

Thẻ là thẻ nội dòng (*inline*) trung tính, không mang ý nghĩa ngữ nghĩa mặc định, được dùng làm thùng chứa để áp dụng CSS hoặc JavaScript cho một đoạn văn bản cụ thể.

Mã Nguồn Khai Báo Thẻ span Kết Hợp Với CSS

```
<p>Chào mừng đến với trang web<span style="color: red; font-weight: bold;">nổi bật</span>.</p>
```

KẾT QUẢ HIỂN THỊ TRÊN TRÌNH DUYỆT (BROWSER PREVIEW)

Chào mừng đến với trang web **nổi bật**.

20. Tổng kết danh sách các thẻ định dạng văn bản HTML

[Bảng] Bảng Tổng Kết Công Dụng Tất Cả Các Thẻ Định Dạng Văn Bản HTML

Thẻ HTML	Công dụng kỹ thuật cốt lõi
`<p>`	Tạo đoạn văn bản độc lập với khoảng cách dòng chuẩn.
` `	Ngắt dòng trực tiếp không tạo đoạn mới (thẻ tự đóng).
`<hr>`	Tạo đường kẻ ngang phân tách chủ đề (thẻ tự đóng).
`<b>` / `<strong>`	In đậm văn bản (giao diện / khẳng định mức độ quan trọng ngữ nghĩa).
`<i>` / `<em>`	In nghiêng văn bản (giao diện / nhấn mạnh ngữ nghĩa).
`<u>`	Gạch chân đoạn văn bản.
`<s>` / `<del>`	Gạch ngang văn bản (giao diện / văn bản bị xóa ngữ nghĩa).
`<ins>`	Chèn đoạn văn bản mới thêm vào khi chỉnh sửa tài liệu.
`<sup>` / `<sub>`	Định dạng chỉ số trên (số mũ) / chỉ số dưới (hóa học).
`<mark>`	Tô sáng (highlight) đoạn văn bản.
`<blockquote>` / `<q>`	Trích dẫn khối đoạn văn dài / trích dẫn dòng ngắn.
`<pre>`	Hiển thị văn bản giữ nguyên định dạng khoảng trắng gốc.
`<code>` / `<kbd>` / `<samp>`	Hiển thị mã nguồn / phím bấm bàn phím / đầu ra mẫu.
`<var>`	Hiển thị biến số toán học hoặc mã lập trình.
`<small>`	Hiển thị văn bản cỡ nhỏ cho ghi chú, bản quyền, footer.
`<abbr>`	Từ viết tắt kèm thuộc tính giải nghĩa tooltip `title`.
`<time>`	Chuẩn hóa dữ liệu ngày tháng thời gian tối ưu SEO (`datetime`).
`<address>`	Xác định thông tin liên hệ của tác giả hoặc doanh nghiệp.
`<dfn>`	Đánh dấu thuật ngữ mới lần đầu được định nghĩa.
`<span>`	Thẻ gom nhóm nội dòng trung tính để định kiểu bằng CSS.

15. Khung minh họa kết quả hiển thị trên trình duyệt (Browser Output Preview)

MINH HỌA GIAO DIỆN KẾT QUẢ CÁC THẺ VĂN BẢN TRÊN TRÌNH DUYỆT (BROWSER PREVIEW)

Thẻ In đậm: Đây là **đoạn in đậm b** và đây là **đoạn quan trọng strong**.

Thẻ In nghiêng: Đây là *đoạn in nghiêng i* và đây là *đoạn nhấn mạnh em*.

Thẻ Gạch chân & Gạch ngang: Gạch chân u, ~~Gạch ngang s~~, và ~~Đã xóa del~~.

Thẻ Chỉ số: Phương trình toán học $x^2 + y^2 = z^2$ và công thức hóa học H_2O.

Thẻ Tô sáng & Nhỏ: Văn bản có từ quan trọng mark và văn bản chú thích nhỏ small.

Thẻ Trích dẫn ngắn: Steve Jobs đã nói: *Stay hungry, stay foolish.*”

Thẻ Phím bấm & Mã: Nhấn `Ctrl + S` để chạy đoạn mã `console.log()`.

16. Đọc thêm: Ngữ nghĩa (Semantics) và Truy cập (Accessibility) của Thẻ Văn Bản

[Khái Niệm] Mở Rộng Kiến Thức: Ý Nghĩa Ngữ Nghĩa Vẫn Chuẩn A11y Trong Frontend Modern

- **Khác biệt cốt lõi giữa `<b>` vs `<strong>` và `<i>` vs `<em>`**: Các thẻ `<b>` và `<i>` chỉ đơn thuần là chỉ thị giao diện (*presentational elements*) cho trình duyệt vẽ phông chữ. Ngược lại, `<strong>` và `<em>` mang ý nghĩa ngữ nghĩa (*semantic elements*). Khi trình đọc màn hình (*Screen Reader*) đọc đến `<strong>`, nó sẽ nhấn giọng hoặc tăng âm lượng để báo hiệu từ quan trọng cho người khiếm thị.
- **Thẻ `<del>` và `<ins>` trong kiểm soát phiên bản**: Trong các ứng dụng Web quản lý tài liệu (như Google Docs, Wiki, Git Diff View), thẻ `<del>` biểu thị đoạn bị gạch bỏ và `<ins>` biểu thị đoạn được chèn mới giúp công cụ tìm kiếm hiểu lịch sử thay đổi tài liệu.
- **Tránh lạm dụng `<u>` (Underline)**: Trong giao diện Web hiện đại, người dùng mặc định coi văn bản gạch chân là một đường liên kết (`<a>`). Sử dụng thẻ `<u>` quá nhiều có thể gây hiểu nhầm cho người dùng khi họ bấm vào mà không có phản hồi chuyển trang.

3.4.3 c. Thẻ liên kết (Anchor - `<a>`)

Thẻ `<a>` (viết tắt của *”anchor”* - điểm neo) là xương sống của không gian mạng World Wide Web (WWW). Thẻ này được sử dụng để tạo ra các liên kết siêu văn bản (*hyperlinks*) giúp kết nối các trang web, điều hướng đến một vị trí cụ thể trong cùng trang, hoặc kết nối đến các tài nguyên đa dạng như tệp tin tải xuống, địa chỉ email, số điện thoại và kích hoạt các hành động JavaScript. Khi người dùng nhấp vào một liên kết, trình duyệt sẽ tự động điều hướng đến đích của liên kết đó.

1. Cấu trúc cú pháp cơ bản của thẻ `<a>`

Cú pháp của thẻ `<a>` rất đơn giản nhưng chứa đựng thuộc tính điều hướng quan trọng nhất:

Cú Pháp Khai Báo Thẻ Liên Kết a

```
<a href="URL_dich">Nội dung hiển thị liên kết</a>
```

[Khái Niệm] Phân Tích Thành Phần Trong Cú Pháp Thẻ <a>

- `<a ...>`: Thẻ mở bắt đầu một liên kết siêu văn bản.
- `href="URL_dich"` (**Hyperlink Reference**): Thuộc tính bắt buộc và quan trọng nhất, chỉ định địa chỉ URL đích hoặc tài nguyên mà liên kết sẽ dẫn đến. Nếu thiếu thuộc tính `href`, thẻ `<a>` sẽ không được trình duyệt xem là một liên kết điều hướng thực thụ.
- **Nội dung hiển thị liên kết**: Văn bản, hình ảnh hoặc các phần tử khối HTML mà người dùng nhìn thấy và nhấp chuột vào.
- `</a>`: Thẻ đóng hoàn tất phần tử liên kết.

Mã Nguồn Khai Báo Liên Kết Cơ Bản Minh Họa

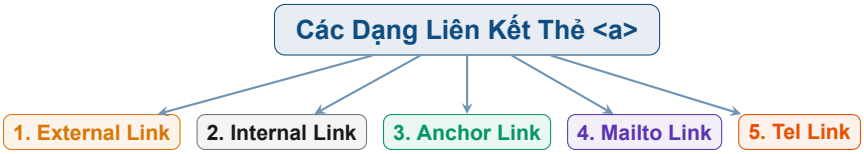
```
<a href="https://www.google.com">Tìm kiếm trên Google</a>
```

KẾT QUẢ HIỂN THỊ TRÊN TRÌNH DUYỆT (BROWSER PREVIEW)

Hiển thị liên kết: Tìm kiếm trên Google (Bấm vào sẽ chuyển hướng đến https://www.google.com)

2. Phân loại các dạng liên kết thường gặp với thẻ <a>

Trong phát triển Web thực tế, thuộc tính `href` có thể chứa các dạng đường dẫn URL khác nhau tùy theo mục đích sử dụng:



2.1. Đường dẫn tuyệt đối (Absolute URL - External Link):

Chỉ định một địa chỉ web đầy đủ bao gồm giao thức (`http://`, `https://`), tên miền và đường dẫn chi tiết. Dùng khi muốn điều hướng người dùng sang một trang web ngoài hệ thống.

Mã Nguồn Liên Kết Đường Dẫn Tuyệt Đối (External Link)

```
<a href="https://www.google.com">Truy cập Google</a>
<a href="https://www.facebook.com">Mở Facebook</a>
<a href="https://www.example.com/pages/about.html">Về chúng tôi</a>
```

2.2. Đường dẫn tương đối (Relative URL - Internal Link):

Chỉ định vị trí tệp tương đối so với vị trí của trang hiện tại. Rất hữu ích khi liên kết giữa các trang con trong cùng hệ thống website:

- **Trong cùng thư mục**: `<a href="contact.html">Liên hệ</a>`
- **Trong thư mục con**: `<a href="pages/about.html">Về chúng tôi</a>`
- **Trong thư mục cha (quay ra ngoài 1 cấp)**: `<a href="../index.html">Trang chủ</a>`
- **Từ thư mục gốc website**: `<a href="/about.html">Giới thiệu</a>`

Mã Nguồn Liên Kết Đường Dẫn Tương Đối (Internal Link)

```
<!-- 1. Cùng thư mục -->
<a href="contact.html">Liên hệ</a>

<!-- 2. Thư mục con pages -->
<a href="pages/about.html">Về chúng tôi</a>

<!-- 3. Thư mục cha cấp trên -->
<a href="../index.html">Trang chủ</a>

<!-- 4. Thư mục gốc website -->
<a href="/about.html">Giới thiệu</a>
```

2.3. Liên kết neo nội bộ trong cùng trang (Anchors / Internal Page Links):

Cho phép điều hướng hoặc cuộn tự động đến một vị trí phân đoạn cụ thể trong cùng một trang web qua 2 bước:

1. **Bước 1: Đặt ID cho phần tử đích**: Gán thuộc tính `id` duy nhất cho phần tử cần nhảy tới.
2. **Bước 2: Tạo liên kết với dấu `#`**: Sử dụng ký tự `#` theo sau là tên `id` của phần tử đích trong thuộc tính `href`.

Mã Nguồn Tạo Liên Kết Cuộn Đến Vị Trí ID Trong Trang

```
<!-- Ví dụ 1: Nhảy đến Phần 2 -->
<a href="#section2">Đến phần 2</a>
<!-- ... các đoạn văn bản ... -->
<h2 id="section2">Phần 2</h2>

<!-- Ví dụ 2: Nhảy đến phần giới thiệu sản phẩm -->
<a href="#section1">Đi đến phần giới thiệu</a>
<h2 id="section1">Giới thiệu về sản phẩm</h2>

<!-- Ví dụ 3: Nhảy xuống chân trang -->
<a href="#footer">Đi đến cuối trang</a>
<footer id="footer">Đây là chân trang</footer>
```

2.4. Liên kết kích hoạt ứng dụng Email (mailto:):

Mở ứng dụng thư điện tử mặc định của thiết bị (Outlook, Mail, Gmail) và tự động điền sẵn địa chỉ người nhận. Bạn cũng có thể thiết lập trước tiêu đề (*subject*) và nội dung (*body*) bằng cách mã hóa ký tự khoảng trắng thành `%20`:

Mã Nguồn Tạo Liên Kết Gửi Email Mặc Định

```
<!-- 1. Gửi email cơ bản -->
<a href="mailto:abc@example.com">Gửi email</a>
<a href="mailto:info@example.com">Gửi email cho chúng tôi</a>

<!-- 2. Gửi email có sẵn Tiêu đề và Nội dung (<a
     href="mailto:info@example.com?subject=Ho
```

2.5. Liên kết gọi điện thoại (tel:):

Cho phép người dùng nhấp vào để thực hiện cuộc gọi trực tiếp (trên các thiết bị di động như Smartphone hoặc ứng dụng nghe gọi):

Mã Nguồn Tạo Liên Kết Cuộc Gọi Điện Thoại

```
<a href="tel:+84123456789">Gọi ngay</a>
<a href="tel:+84123456789">Gọi ngay: 0123.456.789</a>
```

2.6. Liên kết gợi ý tải xuống tệp tin (Download):

Trình duyệt thường mở xem trực tiếp các tệp tin như hình ảnh, PDF hay MP4. Sử dụng thuộc tính `download` ra lệnh cho trình duyệt tải tệp về máy, đồng thời có thể chỉ định tên tệp tải về mới đề xuất:

Mã Nguồn Tạo Liên Kết Tải File Về Máy (Download)

```
<!-- 1. Tải file PDF cơ bản -->
<a href="file.pdf" download>Download PDF</a>

<!-- 2. Tải file PDF và đổi tên tệp đề xuất thành bao_cao_quy_III.pdf -->
<a href="document.pdf" download="bao_cao_quy_III.pdf">Tải báo cáo PDF</a>

<!-- 3. Tải file ảnh và đổi tên tệp đề xuất thành beautiful_sunset.jpg -->
<a href="my-image.jpg" download="beautiful_sunset.jpg">Tải ảnh hoàng hôn</a>
```

2.7. Thuộc tính title - Cung cấp văn bản gợi ý (Tooltip):

Thuộc tính `title` cung cấp văn bản thông tin bổ sung dạng khung gợi ý (*tooltip*) xuất hiện khi người dùng di con trỏ chuột dừng trên liên kết:

Mã Nguồn Sử Dụng Thuộc Tính title Tạo Tooltip

```
<a href="/download/report.zip" title="Tải xuống báo cáo tài chính năm 2023">Tải
    báo cáo</a>
```

2.8. Liên kết thực thi câu lệnh JavaScript:

Nhúng trực tiếp lệnh JavaScript vào `href` để chạy hàm xử lý tương tác:

Mã Nguồn Chạy JavaScript Trực Tiếp Trong href

```
<a href="javascript:alert('Hello!')">Nhấp để hiện thông báo</a>
```

3. Tổng hợp các thuộc tính quan trọng của thẻ <a>

[Bảng] Bảng Danh Sách Thuộc Tính Cốt Lõi Của Thẻ a Trong HTML

Thuộc tính	Ý nghĩa và Công dụng	Mô tả chi tiết và Cú pháp mẫu
`href`	Đường dẫn URL liên kết đến (bắt buộc để tạo link)	Chỉ định trang web, tệp tin, ID, email hoặc SĐT. Ví dụ: `href="https://site.com"`
`target`	Nơi hiển thị trang mới khi người dùng nhấp vào	Quy định mở ở tab hiện tại (`_self`), tab mới (`_blank`), khung cha (`_parent`), hay top khung (`_top`).
`download`	Gợi ý tải tệp tin về máy thay vì mở xem	Có thể gán tên tệp đề xuất khi tải về. Ví dụ: `download="bao_cao_quy_III.pdf"`
`rel`	Mối quan hệ giữa trang hiện tại và trang liên kết	Đóng vai trò nâng cao bảo mật (`noopener`, `noreferrer`) và tối ưu SEO (`nofollow`, `external`).
`title`	Văn bản gợi ý (Tooltip) hiển thị khi rê chuột	Cung cấp thông tin bổ sung khi con trỏ chuột dừng trên liên kết. Ví dụ: `title="Tải báo cáo năm 2023"`
`type`	Chỉ định kiểu MIME của tài nguyên đích	Gợi ý định dạng tệp tin cho trình duyệt (ít dùng trong thực tế). Ví dụ: `type="application/pdf"`
`name`	Tạo điểm neo cũ (Legacy Anchor HTML4)	Thuộc tính cũ trong HTML4 để làm điểm neo, hiện tại chuẩn HTML5 đã thay thế hoàn toàn bằng thuộc tính `id`.

4. Phân tích chi tiết các thuộc tính nâng cao

4.1. Thuộc tính target - Điều hướng cửa sổ hiển thị:

[Bảng] Bảng Chi Tiết Giá Trị Của Thuộc Tính target

Giá trị target	Ý nghĩa chính	Chi tiết hành vi trình duyệt
`_self`	(Mặc định) Tab hiện tại	Mở trang liên kết ngay trong khung cửa sổ/tab đang xem.
`_blank`	Tab / Cửa sổ mới	Mở trang liên kết trong một tab mới hoàn toàn. Phổ biến khi liên kết ra ngoài website.
`_parent`	Frame cha	Mở liên kết trong khung cửa sổ cha chứa nó (thường dùng trong `<iframe>`).
`_top`	Top-level Frame	Mở liên kết ở cấp cửa sổ cao nhất (thoát khỏi tất cả các khung lồng nhau).
`framename`	Khung có tên	Mở liên kết trong một khung cụ thể có thuộc tính `name="framename"`.

Mã Nguồn Mở Trang Web Trong Tab Mới Bằng target="_blank"

```
<a href="https://example.com" target="_blank">Mở tab mới</a>
<a href="https://www.w3schools.com" target="_blank">Tìm hiểu thêm tại W3Schools
    (Mở tab mới)</a>
```

4.2. Thuộc tính rel - Bảo mật chống tấn công Tabnabbing và tối ưu SEO:

Khi mở liên kết ngoài với `target="_blank"`, nếu không cấu hình an toàn, trang web của bạn đứng trước nguy cơ bị tấn công *Reverse Tabnabbing* (trang mới mở có thể dùng lệnh `window.opener.location` để đổi trang web gốc của bạn thành trang phishing giả mạo). Do đó, việc kết hợp thuộc tính `rel` là quy tắc bảo mật bắt buộc:

- `noopener`: Ngăn trang mới mở truy cập đối tượng `window.opener` của trang hiện tại, triệt hạ hoàn toàn lỗ hổng Tabnabbing.
- `noreferrer`: Tương tự `noopener`, đồng thời chặn trình duyệt gửi tiêu đề HTTP Referer Header sang trang mới (giấu thông tin nguồn xuất phát).
- `nofollow`: Báo cho công cụ tìm kiếm (Googlebot) không "theo dõi" liên kết này (không truyền điểm uy tín PageRank / Link Juice), dùng cho liên kết quảng cáo hoặc nội dung do người dùng đăng tải tránh spam.
- `external`: Báo hiệu ngữ nghĩa liên kết dẫn đến một tài liệu bên ngoài hệ thống tên miền hiện tại.

Mã Nguồn Khai Báo Liên Kết An Toàn Chuẩn Bảo Mật Và SEO

```
<!-- Cấu hình chuẩn bắt buộc cho mọi liên kết dẫn sang trang ngoài -->
<a href="https://example.com" target="_blank" rel="noopener noreferrer">Liên kết ngoài</a>
<a href="https://www.example.org" target="_blank" rel="noopener noreferrer">Trang web bên ngoài</a>
```

5. Đặt các phần tử khác làm nội dung bên trong thẻ `<a>`

Nội dung của thẻ `<a>` không bị giới hạn ở văn bản thuần túy. Trong chuẩn HTML5, bạn có thể lồng hình ảnh hoặc thậm chí các phần tử cấp khối (*Block-level elements*) như `<div>`, `<h1>`, `<p>`, `<img>` vào trong thẻ `<a>` để biến toàn bộ khối giao diện (*UI Card*) thành vùng nhấp chuột:

5.1. Liên kết bằng hình ảnh (`<a><img ...></a>`):

Mã Nguồn Tạo Liên Kết Bằng Hình Ảnh

```
<a href="product-details.html">
    <img src="product.jpg" alt="Ảnh sản phẩm">
</a>
```

5.2. Biến toàn bộ UI Card thành liên kết (Block-level wrapping):

Mã Nguồn Bọc Cả Khối UI Card Bằng Thẻ a

```
<a href="/blog/my-post" class="blog-card-link">
    <div class="blog-card">
        <h3>Tiêu đề bài viết Blog</h3>
        <p>Mô tả ngắn gọn về bài viết.</p>
        <img src="blog-thumbnail.jpg" alt="Ảnh thumbnail blog">
    </div>
</a>
```

6. Styling thẻ `<a>` với CSS - Quản lý 5 trạng thái tương tác và tạo nút bấm

Thẻ `<a>` sở hữu 5 trạng thái tương tác được định kiểu qua các lớp giả (*pseudo-classes*) trong CSS. Cần tuân theo thứ tự ưu tiên khai báo chuẩn **LVHA+F** (*Link* → *Visited* → *Focus* → *Hover* → *Active*):

[Bảng] Bảng So Sánh Các Trạng Thái Tương Tác Của Thẻ a Trong CSS

Trạng thái CSS	Tên Pseudo-class	Thời điểm và hành vi kích hoạt
Link chưa ghé	`a:link`	Định kiểu cho các liên kết chưa từng được người dùng bấm vào.
Đã truy cập	`a:visited`	Định kiểu cho các liên kết đã từng được mở trong lịch sử duyệt web.
Tiêu điểm phím	`a:focus`	Kích hoạt khi liên kết nhận tiêu điểm chọn bằng bàn phím (Tab).
Rê chuột qua	`a:hover`	Kích hoạt khi con trỏ chuột di chuyển đè lên vùng liên kết.
Đang nhấp chuột	`a:active`	Kích hoạt tại khoảnh khắc người dùng nhấn giữ phím chuột trái.

Mã Nguồn CSS Định Kiểu Đầy Đủ Các Trạng Thái Thẻ a (Chuẩn LVHA+F)

```css
/* 1. a:link - Màu chữ và định kiểu mặc định */
a {
    color: #007bff; /* Màu chữ mặc định */
    text-decoration: none; /* Bỏ gạch chân mặc định */
    transition: color 0.3s ease; /* Hiệu ứng chuyển đổi màu mượt mà */
}

/* 2. a:visited - Kiểu dáng cho liên kết đã được truy cập */
a:visited {
    color: #6610f2; /* Màu chữ cho liên kết đã ghé thăm */
}

/* 3. a:focus - Viền dễ nhận biết khi dùng bàn phím (A11y) */
a:focus {
    outline: 2px solid #007bff; /* Viền để dễ nhận biết khi dùng bàn phím */
    outline-offset: 2px;
}

/* 4. a:hover - Kiểu dáng khi con trỏ chuột di chuyển qua liên kết */
a:hover {
    color: #0056b3; /* Đậm hơn khi di chuột */
    text-decoration: underline; /* Gạch chân khi di chuột */
}

/* 5. a:active - Kiểu dáng khi liên kết đang được nhấp (giữ chuột) */
a:active {
    color: #004085; /* Thêm nữa khi đang click */
}
```

Tạo nút bấm (Button) đẹp mắt từ thẻ <a>:

Trong thực tế giao diện Web, thẻ <a> thường được biến tấu thành các nút kêu gọi hành động (*Call to Action - CTA*) bằng CSS:

Mã Nguồn CSS Chuyển Thẻ a Thành Nút Bấm (Button)

```
<!-- HTML Nút Bấm -->
<a href="#" class="btn">Bấm vào đây</a>

<!-- CSS Làm Đẹp Nút -->
<style>
  .btn {
    display: inline-block;
    background: #007BFF;
    color: white;
    padding: 10px 20px;
    text-decoration: none;
    border-radius: 5px;
  }
  .btn:hover {
    background: #0056b3;
  }
</style>
```

7. Kết hợp thẻ <a> với JavaScript và Chuyên sâu về <a href="#">

7.1. Chạy hành động JavaScript từ thẻ <a>:

Có 2 cách chính để kết hợp thẻ <a> với mã JavaScript:

Mã Nguồn Kết Hợp Thẻ a Với JavaScript

```
<!-- Cách 1: Dùng thuộc tính onclick inline (kèm return false để chặn chuyển
    trang) -->
<a href="#" onclick="alert('Click!'); return false;">Click tôi</a>

<!-- Cách 2 (Khuyến nghị chuẩn Modern): Dùng event listener trong JS -->
<a href="#" id="myLink">Bấm em đi</a>

<script>
  document.getElementById("myLink").addEventListener("click", function(e) {
    e.preventDefault(); // Ngăn cuộn lên đầu trang hoặc nhảy link
    alert("Đã bấm!");
  });
</script>
```

7.2. Phân tích chuyên sâu về <a href="#">:

Khi bạn khai báo href="#", dấu thăng (#) đóng vai trò là một định danh mảnh (*fragment identifier*) rỗng:

- **Cách hoạt động**: Trình duyệt cố gắng tìm một phần tử có id="" rỗng. Do không có ID nào rỗng, trình

duyệt không gửi yêu cầu HTTP mới mà mặc định cuộn màn hình thẳng lên đầu trang (`<body>`/`<html>`), đồng thời có thể thêm ký tự `#` vào thanh địa chỉ URL (ví dụ `https://yourwebsite.com/yourpage.html#`).

- **Các trường hợp sử dụng phổ biến**:
  1. *Chỗ đặt tạm thời (Placeholder)*: Khi đang phát triển UI và cần liên kết tạm thời chưa có link thật: `<a href="#">Thêm vào giỏ hàng</a>`, `<a href="#">Đăng nhập</a>`.
  2. *Liên kết không làm mới trang (Non-Navigational Links)*: Kết hợp JS mở modal, bật menu hay gửi AJAX.
  3. *Nút quay lại đầu trang*: Dùng cuộn nhanh lên đỉnh trang (`<a href="#">Quay lại đầu trang</a>`).
- **Ví dụ gây lỗi nếu không ngăn mặc định**: Với mã `<a href="#" onclick="alert('Click!')">Click tôi</a>`, trình duyệt hiện alert nhưng đồng thời cuộn giật về đầu trang do thiếu lệnh ngăn mặc định.
- **Cách xử lý đúng**: Luôn dùng `event.preventDefault()` trong JS hoặc `return false;` trong thuộc tính `onclick`.

Mã Nguồn Các Mẫu Liên Kết Placeholder Và Chặn Cuộn Mặc Định

```html
<!-- 1. Placeholder chỗ đặt tạm thời -->
<a href="#">Thêm vào giỏ hàng</a>
<a href="#">Đăng nhập</a>

<!-- 2. Nút quay lại đầu trang -->
<a href="#">Quay lại đầu trang</a>

<!-- 3. Liên kết tải lại trang hiện tại -->
<a href="" title="Tải lại trang">Làm mới trang</a>

<!-- 4. Dùng javascript:void(0) để không làm gì -->
<a href="javascript:void(0)">Liên kết tạm dừng</a>
```

[Bảng] Bảng So Sánh Các Giải Pháp Thay Thế Cho href="#"

Mục đích nhu cầu	Giải pháp khuyến nghị	Đánh giá kỹ thuật và Ngữ nghĩa HTML
Thực hiện hành động JS không điều hướng	`<button type="button">`	**Chuẩn ngữ nghĩa tốt nhất (Best Practice)**. Thẻ `<button>` sinh ra cho hành động JS, không bị giật trang, tối ưu khả năng truy cập (A11y).
Tạo liên kết nhảy tới phần tử thực	`<a href="#some-id">`	Chuẩn xác. Trỏ chính xác đến `id` thực tế tồn tại trên trang (ví dụ `<footer id="footer">Đây là chân trang</footer>`).
Link không reload trang	`href="#"` + `e.preventDefault()`	Phổ biến khi làm nút giả bằng thẻ `<a>`, nhưng bắt buộc phải có lệnh ngăn hành vi cuộn mặc định.
Tạm dừng không tải lại trang	`href="javascript:vo`	Ít khuyến nghị hơn vì không thân thiện với SEO và các công cụ đọc màn hình.
Để href trống (`href=""`)	Tải lại trang hiện tại	Làm mới trình duyệt (tương tự reload), thường không phải hành vi mong muốn cho nút JS.

8. Bảng tổng hợp những lưu ý quan trọng và lỗi thường gặp

[Bảng] Bảng Tổng Hợp Cảnh Báo Và Lưu Ý Quan Trọng Khi Sử Dụng Thẻ a

Trường hợp / Lưu ý	Giải thích chi tiết và Hướng xử lý
Thiếu thuộc tính `href`	Thẻ `<a>` không có `href` sẽ không phải liên kết (không có hành vi click mặc định, không có con trỏ bàn tay `cursor: pointer`). Có thể dùng để làm nút giả lập hoặc dùng CSS `pointer-events: none` để vô hiệu hóa sự kiện click.
Dùng `href="#"` thiếu JS	Khiến trang bị giật cuộn lên đầu màn hình. Phải bổ sung `e.preventDefault()` hoặc `return false;`.
Mở tab mới thiếu `rel`	Dễ bị tấn công Reverse Tabnabbing. Luôn luôn kèm `rel="noopener noreferrer"` khi có `target="_blank"`.
Dùng `<a>` làm nút bấm JS	Về mặt ngữ nghĩa HTML5, nếu chỉ chạy JS mà không điều hướng URL, hãy ưu tiên dùng thẻ `<button type="button">` thay vì `<a>`.

9. Bảng tổng kết nhanh công dụng các thuộc tính thẻ `<a>`

[Bảng] Bảng Tóm Tắt Nhanh Công Dụng Các Thuộc Tính Của Thẻ a

Thuộc tính / Cấu hình	Công dụng chính cốt lõi
`href`	Tạo liên kết (Bắt buộc nếu muốn link).
`target="_blank"`	Mở trong tab mới.
`download`	Bắt trình duyệt tải file về máy thay vì mở xem.
`rel="noopener"`	Tăng bảo mật khi dùng với `target="_blank"`.
`title`	Tooltip hiển thị khi di chuột vào liên kết.

3.4.4 d. Thẻ hình ảnh (Image - `<img>`)

Thẻ `<img>` trong HTML được sử dụng để hiển thị hình ảnh trên trang web. Đây là một thẻ **tự đóng** (*self-closing tag*), nghĩa là không cần thẻ đóng `</img>`.

1. Cấu Trúc Cơ Bản của Thẻ `<img>`

Cú pháp chung của thẻ `<img>`:

Cú Pháp Khai Báo Thẻ Hình Ảnh img

```
<img src="duong-dan-hinh-anh" alt="Mo-ta-hinh-anh">
```

[Bảng] Bảng Hai Thuộc Tính Cốt Lõi Của Thẻ img

Thuộc tính	Ý nghĩa
`src`	Đường dẫn đến hình ảnh.
`alt`	Văn bản mô tả hình ảnh (hiển thị khi hình ảnh lỗi hoặc hỗ trợ SEO).

Mã Nguồn Ví Dụ Hiển Thị Hình Ảnh Cơ Bản

```
<img src="cat.jpg" alt="Hình ảnh con mèo">
```

[Ghi Chú] Kết Quả Hiển Thị Trình Duyệt

Nếu `cat.jpg` tồn tại, trình duyệt sẽ hiển thị ảnh. Nếu không, chữ *Hình ảnh con mèo"* sẽ hiện ra thay thế.

2. Đường Dẫn (src) Của Ảnh

2.1. Ảnh từ thư mục nội bộ:

Mã Nguồn Hiển Thị Ảnh Từ Thư Mục Nội Bộ

```
<img src="images/logo.png" alt="Logo website">
```

Hình ảnh `logo.png` nằm trong thư mục `/images` cùng cấp với file HTML.

2.2. Ảnh từ URL bên ngoài (Ảnh Online):

Mã Nguồn Hiển Thị Ảnh Từ URL Bên Ngoài

```
<img src="https://example.com/image.jpg" alt="Ảnh từ web">
```

Ảnh sẽ được tải trực tiếp từ máy chủ của `example.com`.

3. Các Thuộc Tính Quan Trọng của `<img>`

3.1. width và height - Định kích thước ảnh:

Dùng để điều chỉnh kích thước hiển thị của ảnh.

Mã Nguồn Thiết Lập Kích Thước Ảnh Bằng width Và height

```
<img src="dog.jpg" alt="Chó dễ thương" width="200" height="150">
```

Ảnh sẽ có chiều rộng `200px` và chiều cao `150px`.

[Mẹo] Khai Báo width Và height Để Tránh CLS

Luôn cố gắng cung cấp cả `width` và `height` để giúp trình duyệt dành không gian cho hình ảnh **trước khi** nó tải xong. Điều này giúp tránh hiện tượng **CLS** (Cumulative Layout Shift) — nội dung bị nhảy dịch khi ảnh tải, cải thiện điểm Core Web Vitals và SEO. Nếu bạn chỉ chỉ định một giá trị, trình duyệt sẽ tự tính giá trị còn lại để giữ tỷ lệ gốc của ảnh.

3.2. alt - Văn bản thay thế:

- Dùng khi ảnh bị lỗi hoặc giúp SEO tốt hơn.
- Quan trọng để hỗ trợ người khiếm thị (Accessibility).

Mã Nguồn Văn Bản Alt Thay Thế Khi Ảnh Lỗi

```
<img src="not-found.jpg" alt="Ảnh không tìm thấy">
```

Nếu ảnh không tải được, văn bản *Ảnh không tìm thấy"* sẽ hiển thị thay thế.

[Cảnh Báo] Không Bao Giờ Để Trống Thuộc Tính alt

- **Không bao giờ bỏ qua** thuộc tính `alt`. Đây là yêu cầu của chuẩn WCAG về Accessibility.
- Nếu ảnh hoàn toàn là trang trí (không mang ngữ nghĩa), hãy dùng `alt=""` (chuỗi rỗng) để trình đọc màn hình bỏ qua nó. Không nên bỏ hẳn thuộc tính.
- **Cấm** dùng tên file làm alt (ví dụ: `alt="img123.jpg"`) — không có ý nghĩa gì với người dùng và gây hại SEO.

3.3. title - Hiển thị mô tả khi rê chuột:

Mã Nguồn Thuộc Tính title Tạo Tooltip Trên Ảnh

```
<img src="bird.jpg" alt="Chim đẹp" title="Chim cánh cụt dễ thương">
```

Khi rê chuột vào ảnh, sẽ hiển thị tooltip *Chim cánh cụt dễ thương"*.

[Ghi Chú] title Không Thay Thế Được alt

Không nên dùng `title` để thay thế `alt`. Thuộc tính `alt` là bắt buộc và quan trọng hơn cho khả năng truy cập (Accessibility) — trình đọc màn hình đọc `alt`, không đọc `title`. Thuộc tính `title` chỉ là thông tin bổ sung hiển thị khi di chuột.

3.4. loading="lazy" - Tối ưu tải trang:

- Dùng để trì hoãn tải ảnh cho đến khi người dùng cuộn đến ảnh đó.
- Giúp trang tải nhanh hơn, cải thiện điểm Core Web Vitals.

Mã Nguồn Sử Dụng Lazy Loading Tối Ưu Tốc Độ Tải Trang

```
<img src="large-image.jpg" alt="Ảnh lớn" loading="lazy">
```

[Mẹo] Kết Hợp loading="lazy" Và decoding="async" Để Tối Ưu Tối Đa

Kết hợp cả hai thuộc tính cho ảnh không quan trọng ngay lập tức (ảnh nằm dưới màn hình):

- `loading="lazy"`: Trì hoãn tải ảnh cho đến khi người dùng cuộn gần đến.
- `decoding="async"`: Giải mã ảnh bất đồng bộ — không chặn render trang.

Ví dụ lý tưởng: `<img src="photo.jpg" alt="..." loading="lazy" decoding="async">`

[Cảnh Báo] Không Dùng loading="lazy" Cho Ảnh Quan Trọng Đầu Trang

Không áp dụng `loading="lazy"` cho ảnh **hero banner**, logo, hay bất kỳ hình ảnh nào hiển thị ngay khi trang mở (above the fold"). Điều này sẽ làm chậm thời gian hiển thị nội dung đầu tiên (LCP — Largest Contentful Paint), ảnh hưởng tiêu cực đến Core Web Vitals.

4. Định Dạng Hình Ảnh Phổ Biến

[Bảng] Bảng So Sánh Các Định Dạng Hình Ảnh Phổ Biến Trong Web

Định dạng	Ưu điểm	Nhược điểm
JPG/JPEG	Kích thước nhỏ, hiển thị màu sắc đẹp.	Không hỗ trợ nền trong suốt.
PNG	Hỗ trợ nền trong suốt, hình ảnh sắc nét.	Dung lượng lớn hơn JPG.
GIF	Hỗ trợ ảnh động.	Chất lượng thấp, không nhiều màu sắc.
SVG	Hình ảnh vector, không mất chất lượng khi phóng to.	Không phù hợp với ảnh thực tế.
WEBP	Kích thước nhỏ hơn JPG và PNG, chất lượng cao.	Không phải trình duyệt nào cũng hỗ trợ.

Mã Nguồn Ví Dụ Sử Dụng Ảnh SVG Không Bị Mờ Khi Zoom

```
<img src="vector.svg" alt="Ảnh SVG">
```

[Mẹo] Ưu Tiên WebP Và AVIF Cho Web Hiện Đại

Trong năm 2024+, hầu hết trình duyệt đã hỗ trợ tốt `WebP` và `AVIF`. Hãy ưu tiên dùng chúng thay vì JPEG/PNG để giảm dung lượng tải 25–50% mà vẫn giữ chất lượng tốt. Luôn dùng thẻ `<picture>` kèm fallback JPEG để đảm bảo tương thích với trình duyệt cũ.

5. Hiển Thị Ảnh Nền Bằng CSS (Thay Vì `<img>`)

Dùng `background-image` trong CSS để làm ảnh nền trang web hoặc phần tử giao diện:

Mã Nguồn CSS Thiết Lập Ảnh Nền Toàn Trang

```css
body {
    background-image: url("background.jpg");
    background-size: cover;
    background-position: center;
}
```

[Bảng] Bảng Phân Biệt Khi Nào Dùng img vs background-image

Trường hợp	Nên dùng
Ảnh có ý nghĩa nội dung (ảnh sản phẩm, ảnh blog, hình minh họa)	`<img>`
Ảnh chỉ để trang trí (nền trang, hoa văn, pattern)	`background-image`

6. Dùng Thẻ `<figure>` và `<figcaption>` Để Mô Tả Ảnh

Thẻ `<figure>` kết hợp với `<figcaption>` cho phép gắn chú thích mô tả ngữ nghĩa cho hình ảnh, cải thiện cả SEO lẫn Accessibility:

Mã Nguồn Sử Dụng figure Và figcaption Mô Tả Ảnh

```html
<figure>
    <img src="mountain.jpg" alt="Ngọn núi cao">
    <figcaption>Hình ảnh một ngọn núi vào mùa đông.</figcaption>
</figure>
```

[Mẹo] Lợi Ích Của figure Và figcaption

- Cải thiện SEO và Accessibility — trình đọc màn hình nhận biết mối quan hệ ảnh–chú thích.
- Văn bản mô tả ảnh được hiển thị đẹp hơn, chuẩn ngữ nghĩa HTML5.

[Ghi Chú] Đọc Thêm — figure Và figcaption Trong HTML5

Thẻ `<figure>` không chỉ dùng cho ảnh — nó có thể bao bọc bất kỳ nội dung tự đứng độc lập nào như biểu đồ, đoạn code, video, thậm chí bảng biểu. `<figcaption>` phải là phần tử con đầu tiên hoặc cuối cùng bên trong `<figure>`.

7. Tạo Ảnh Click Được (<a> + <img>)

Kết hợp thẻ `<a>` và `<img>` để tạo một liên kết bằng hình ảnh. Khi người dùng bấm vào ảnh, họ sẽ được chuyển đến một trang web hoặc một phần khác trong trang.

7.1. Liên kết đến một trang web khác bằng hình ảnh:

Mã Nguồn Tạo Liên Kết Đến Trang Web Khác Bằng Hình Ảnh

```
<a href="https://www.google.com">
    <img src="google-logo.png" alt="Google" width="150">
</a>
```

Khi bấm vào ảnh, người dùng sẽ được chuyển đến Google.

7.2. Liên kết đến một phần khác trong cùng trang:

Mã Nguồn Tạo Liên Kết Ảnh Cuộn Đến ID Trong Trang

```
<a href="#section2">
    <img src="scroll-down.png" alt="Cuộn xuống" width="50">
</a>
<h2 id="section2">Phần 2: Nội dung quan trọng</h2>
<p>Đây là phần nội dung mà người dùng sẽ nhảy đến khi nhấp vào hình ảnh.</p>
```

Khi bấm vào ảnh, trang sẽ tự động cuộn xuống phần có `id="section2"`.

7.3. Mở liên kết trong tab mới (target=”_blank”):

Mã Nguồn Tạo Liên Kết Ảnh Mở Tab Mới

```
<a href="https://www.example.com" target="_blank">
    <img src="example-logo.png" alt="Example" width="120">
</a>
```

Liên kết sẽ mở trong một tab mới thay vì thay thế trang hiện tại.

7.4. Liên kết ảnh với hiệu ứng hover (CSS):

Mã Nguồn Tạo Hiệu Ứng Hover Làm Mờ Ảnh Khi Di Chuột

```
<a href="https://www.example.com">
    <img src="normal.jpg" alt="Hình ảnh" id="hover-img" width="200">
</a>
<style>
    #hover-img:hover {
        opacity: 0.7; /* Làm mờ ảnh khi hover */
    }
</style>
```

Khi di chuột vào ảnh, nó sẽ trở nên mờ hơn.

7.5. Liên kết đến số điện thoại hoặc email bằng ảnh:

Mã Nguồn Liên Kết Gọi Điện Và Gửi Email Bằng Hình Ảnh

```
<!-- Liên kết gọi điện bằng ảnh icon -->
<a href="tel:+84901234567">
    <img src="call-icon.png" alt="Gọi ngay" width="50">
</a>

<!-- Liên kết gửi email bằng ảnh icon -->
<a href="mailto:example@email.com">
    <img src="email-icon.png" alt="Gửi email" width="50">
</a>
```

Khi bấm vào ảnh, người dùng sẽ có thể gọi điện hoặc mở ứng dụng gửi email.

8. Hiển Thị Ảnh Tròn hoặc Bo Góc Bằng CSS

Mã Nguồn CSS Bo Góc Và Làm Tròn Hình Ảnh

```
/* Bo góc ảnh */
img {
    border-radius: 10px;
}

/* Làm ảnh tròn */
img {
    border-radius: 50%;
}
```

Mã Nguồn HTML Ví Dụ Ảnh Tròn Dạng Avatar

```
<img src="avatar.jpg" alt="Ảnh đại
    diện" width="100" height="100" style="border-radius: 50%;">
```

Ảnh sẽ tròn như avatar Facebook.

9. Hình Ảnh Đáp Ứng (Responsive) Cho Mọi Màn Hình

Dùng CSS để ảnh tự điều chỉnh theo kích thước màn hình:

Mã Nguồn CSS Làm Ảnh Tự Co Giãn Đáp Ứng Mọi Màn Hình

```
img {
    max-width: 100%;
    height: auto;
}
```

[Mẹo] Lợi Ích Của max-width: 100%

- Ảnh không bị méo trên điện thoại.
- Không cần đặt kích thước cố định cho từng thiết bị.

10. Thẻ `<picture>` - Hiển Thị Ảnh Theo Kích Thước Màn Hình

Thẻ `<picture>` giúp hiển thị ảnh khác nhau tùy vào thiết bị (Media Queries). Trình duyệt sẽ tự chọn nguồn ảnh phù hợp nhất:

Mã Nguồn Thẻ picture Hiển Thị Ảnh Theo Kích Thước Màn Hình

```
<picture>
    <source srcset="small.jpg" media="(max-width: 600px)">
    <source srcset="medium.jpg" media="(max-width: 1200px)">
    <img src="large.jpg" alt="Ảnh tự động đổi kích thước">
</picture>
```

[Mẹo] Lợi Ích Của Thẻ picture

Tải ảnh nhỏ hơn trên điện thoại, giúp trang nhanh hơn và tiết kiệm băng thông cho người dùng di động.

13. Bảng Đầy Đủ Tất Cả Các Thuộc Tính HTML Của Thẻ img

[Bảng] Bảng Đầy Đủ Tất Cả Các Thuộc Tính Của Thẻ img

Thuộc tính	Mô tả chi tiết
`src`	Đường dẫn hình ảnh (local hoặc URL). Bắt buộc.
`alt`	Mô tả ảnh dùng khi ảnh lỗi hoặc cho trình đọc màn hình. Bắt buộc về mặt truy cập (Accessibility).
`width`	Chiều rộng ảnh (px hoặc %). Nên khai báo để tránh CLS (Cumulative Layout Shift).
`height`	Chiều cao ảnh. Kết hợp với `width` giúp trình duyệt đặt trước không gian trước khi ảnh tải xong.
`title`	Tooltip khi rê chuột vào ảnh. Không nên dùng thay thế `alt`.
`loading`	Điều khiển hành vi tải ảnh: `lazy` (trì hoãn đến khi cuộn tới) hoặc `eager` (tải ngay lập tức).
`decoding`	Gợi ý cách giải mã ảnh: `sync` (đồng bộ), `async` (bất đồng bộ — ưu tiên), `auto`.
`referrerpolicy`	Kiểm soát header referrer gửi khi tải ảnh. Quan trọng cho bảo mật và quyền riêng tư.
`crossorigin`	Kiểm soát CORS khi ảnh từ domain khác: `anonymous` hoặc `use-credentials`.
`srcset`	Danh sách các URL ảnh kèm mô tả độ rộng (`480w`) hoặc mật độ pixel (`2x`) cho responsive image.
`sizes`	Mô tả kích thước hiển thị ảnh tại các điều kiện viewport khác nhau (kết hợp với `srcset`).

14. Cách Hoạt Động Của Thẻ img Trong Trình Duyệt

[Khái Niệm] Cơ Chế Xử Lý Thẻ img Bởi Trình Duyệt

- Trình duyệt gặp thẻ `<img>` sẽ gửi request HTTP đến địa chỉ `src`, tải ảnh về rồi hiển thị vào vị trí tương ứng trên trang.
- Nếu không tìm thấy ảnh → hiện văn bản `alt` thay thế.
- Ảnh không phải là nội dung thẻ chứa" → không có thẻ đóng (self-closing).

[Ghi Chú] Đọc Thêm — Ảnh Và Hiệu Suất Trang

Mỗi thẻ `<img>` tạo ra một yêu cầu HTTP riêng biệt đến máy chủ. Trên trang có nhiều ảnh, số lượng request này tăng lên đáng kể, ảnh hưởng đến tốc độ tải. Các chiến lược giảm thiểu:

- Dùng **CSS Sprites** (ghép nhiều icon nhỏ thành 1 ảnh lớn).
- Dùng **Base64 inline** cho ảnh rất nhỏ.
- Dùng **CDN** để phân phối ảnh từ máy chủ gần người dùng nhất.
- Dùng `loading="lazy"` và `decoding="async"` cho ảnh không quan trọng.

15. Các Thuộc Tính CSS Quan Trọng Áp Dụng Cho `<img>`

[Bảng] Bảng Các Thuộc Tính CSS Phổ Biến Cho Thẻ img

Thuộc tính CSS	Tác dụng
`width`, `height`	Kích thước hiển thị.
`border-radius`	Bo góc (tạo avatar tròn với `border-radius: 50%`).
`box-shadow`	Đổ bóng ảnh.
`filter`	Làm mờ (`blur`), grayscale, sepia, độ tương phản, v.v.
`object-fit`	Điều chỉnh cách ảnh lấp đầy vùng chứa mà không bị méo.
`display: block`	Loại bỏ khoảng trắng bên dưới ảnh (ảnh là inline element mặc định).
`pointer-events`	Cho phép (`auto`) hoặc vô hiệu hóa (`none`) sự kiện click ảnh.

object-fit – Điều chỉnh ảnh nằm gọn trong khung:

Thuộc tính `object-fit` cực kỳ hữu ích khi bạn muốn ảnh lấp đầy một khung cố định mà không bị méo:

Mã Nguồn CSS object-fit Kiểm Soát Ảnh Trong Khung Cố Định

```
<div style="width:200px; height:200px; overflow:hidden;">
    <img src="img.jpg" style="width:100%; height:100%; object-fit:cover;">
</div>
```

[Mẹo] Chọn Giá Trị object-fit Phù Hợp Với Từng Trường Hợp

- Dùng `cover` cho thumbnail, card ảnh, gallery — đảm bảo khung luôn đầy.
- Dùng `contain` cho logo, icon — không bao giờ cắt xén hình ảnh.
- Tránh dùng `fill` trừ khi bạn chắc chắn tỷ lệ ảnh luôn khớp với khung.

[Bảng] Bảng So Sánh Các Giá Trị Của Thuộc Tính object-fit

Giá trị	Kết quả
`cover`	Cắt ảnh sao cho khung đầy — ảnh luôn lấp kín, có thể bị cắt 2 cạnh.
`contain`	Ảnh vừa khít bên trong, không bị cắt — có thể xuất hiện viền trống 2 cạnh.
`fill`	Ảnh bị bóp méo để lấp toàn bộ khung.
`none`	Ảnh hiển thị kích thước gốc, không co giãn.
`scale-down`	Chọn cái nhỏ hơn giữa `none` và `contain`.

16. Các Lỗi Thường Gặp Với Thẻ `<img>`

[Bảng] Bảng Lỗi Thường Gặp Và Cách Khắc Phục Khi Dùng Thẻ img

Vấn đề	Nguyên nhân	Cách khắc phục
Ảnh không hiện	Sai `src`, thiếu ảnh, ảnh bị lỗi	Kiểm tra đường dẫn, đảm bảo file tồn tại.
Ảnh quá to hoặc quá nhỏ	Không đặt `width`/`height` hoặc thiếu CSS	Thêm `max-width: 100%; height: auto` hoặc khai báo kích thước.
Có viền xanh khi dùng `<a>`	Trình duyệt tự thêm viền outline cho ảnh link	Dùng `border: 0` hoặc `outline: none` cho `img` trong CSS.
Có khoảng trắng bên dưới ảnh	Ảnh là inline element, có baseline descender	Dùng `display: block` hoặc `vertical-align: bottom` trong CSS.
Ảnh bị méo trong khung cố định	Không dùng `object-fit`	Thêm `object-fit: cover` hoặc `contain`.

17. Responsive Images Nâng Cao — srcset Và sizes

Đây là cách hiện đại để cung cấp nhiều phiên bản của cùng một hình ảnh, cho phép trình duyệt chọn phiên bản phù hợp nhất dựa trên kích thước màn hình, độ phân giải pixel và băng thông mạng. Giúp tối ưu hóa hiệu suất đáng kể:

Mã Nguồn srcset Và sizes Cho Responsive Images Nâng Cao

```
<img
    srcset="small.jpg 480w,
            medium.jpg 800w,
            large.jpg 1200w"
    sizes="(max-width: 600px) 480px,
           (max-width: 900px) 800px,
           1200px"
    src="medium.jpg"
    alt="Một phong cảnh tuyệt đẹp"
>
```

- `srcset`: Danh sách các URL hình ảnh và "mô tả" của chúng (ví dụ: `480w` cho chiều rộng pixel, hoặc `1x`, `2x` cho mật độ pixel màn hình Retina).
- `sizes`: Kết hợp với `w` trong `srcset`, mô tả kích thước mà hình ảnh sẽ được hiển thị tại các điều kiện viewport khác nhau.
- `src`: Là hình ảnh dự phòng (*fallback*) cho các trình duyệt không hỗ trợ `srcset`.

[Mẹo] srcset Và sizes — Thực Hành Tốt Nhất

Khi dùng `srcset` với descriptor chiều rộng (`w`), luôn kết hợp với `sizes` để trình duyệt tính toán chính xác ảnh cần tải. Không cần thiết khai báo `sizes` khi dùng descriptor mật độ pixel (`1x`, `2x`).

Thẻ <picture> với srcset định dạng hiện đại (WebP + AVIF):

Mã Nguồn picture Kết Hợp WebP Và AVIF Fallback

```
<picture>
    <source srcset="image.avif" type="image/avif">
    <source srcset="image.webp" type="image/webp">
    <img src="image.jpg" alt="Mô tả hình ảnh">
</picture>
```

Trình duyệt sẽ chọn thẻ `<source>` đầu tiên mà nó hỗ trợ và khớp với điều kiện, sau đó sử dụng `src` trong thẻ `<img>` làm dự phòng cuối cùng.

[Ghi Chú] Đọc Thêm — Thứ Tự Ưu Tiên Của Trình Duyệt Trong picture

Trình duyệt đọc các thẻ `<source>` theo thứ tự từ trên xuống và chọn cái đầu tiên phù hợp. Do đó hãy đặt định dạng **hiện đại nhất** (AVIF) lên đầu, WebP ở giữa, và JPEG/PNG làm fallback cuối cùng trong thẻ `<img>`.

18. Thuộc Tính decoding, referrerpolicy Và crossorigin

decoding:

Gợi ý cách trình duyệt nên giải mã hình ảnh:

- `async`: Giải mã bất đồng bộ — không chặn render trang (ưu tiên cho ảnh không quan trọng ngay).
- `sync`: Giải mã đồng bộ — có thể chặn render.
- `auto`: Để trình duyệt tự quyết định.

Mã Nguồn Sử Dụng decoding async Cho Ảnh Không Ưu Tiên

```
<img src="heavy-image.jpg" alt="Ảnh nặng" decoding="async" loading="lazy">
```

referrerpolicy:

Kiểm soát thông tin Referer HTTP được gửi khi trình duyệt yêu cầu tải ảnh. Quan trọng cho bảo mật và quyền riêng tư. Các giá trị phổ biến: `no-referrer`, `origin`, `unsafe-url`.

crossorigin:

Điều khiển cách trình duyệt xử lý yêu cầu CORS khi tải ảnh từ một miền khác. Cần thiết khi muốn thao tác với ảnh trên `<canvas>` hoặc dùng hiệu ứng CSS phức tạp:

[Cảnh Báo] Cảnh Báo CORS Khi Dùng crossorigin

Nếu máy chủ cung cấp ảnh không trả về header `Access-Control-Allow-Origin` đúng cách, việc khai báo `crossorigin` sẽ khiến trình duyệt từ chối tải ảnh hoàn toàn (thay vì hiển thị bình thường). Chỉ dùng `crossorigin` khi bạn kiểm soát được cả phía máy chủ ảnh hoặc máy chủ đó đã hỗ trợ CORS.

- `anonymous`: Yêu cầu hình ảnh mà không gửi thông tin định danh người dùng.
- `use-credentials`: Yêu cầu hình ảnh và gửi cookie, chứng chỉ TLS.

Mã Nguồn Dùng crossorigin Khi Tải Ảnh Từ Domain Khác

```
<img src="https://cdn.example.com/image.jpg" alt="CDN Ảnh" crossorigin="anonymous
    ">
```

19. Lưu Ý Quan Trọng Về Tối Ưu Hóa Hình Ảnh

Hình ảnh thường là tài nguyên nặng nhất trên trang web, ảnh hưởng đáng kể đến tốc độ tải. Các nguyên tắc quan trọng cần tuân thủ:

- **Nén hình ảnh**: Sử dụng các công cụ nén ảnh (TinyPNG, Squoosh) để giảm kích thước tệp mà vẫn giữ được chất lượng chấp nhận được.
- **Sử dụng định dạng phù hợp**:

  - *JPEG/JPG*: Tốt cho ảnh chụp, ảnh có nhiều màu sắc và chi tiết phức tạp.
  - *PNG*: Tốt cho đồ họa, logo, ảnh có nền trong suốt.
  - *SVG*: Tốt cho biểu tượng, logo, đồ họa vector (không bị vỡ khi phóng to).
  - *WebP/AVIF*: Định dạng hiện đại, nén tốt hơn JPEG/PNG — nên dùng với thẻ `<picture>` kèm fallback.
- **Kích thước ảnh phù hợp**: Không tải ảnh 4000px để hiển thị 200px. Dùng `srcset` và `sizes` để phục vụ ảnh theo kích thước màn hình.
- **Sử dụng CDN**: Phân phát hình ảnh từ Mạng phân phối nội dung (CDN) để tải nhanh hơn cho người dùng toàn cầu.
- **Luôn khai báo width & height**: Giúp trình duyệt tránh hiện tượng CLS (Cumulative Layout Shift) — nội dung bị dịch chuyển sau khi tải.

[Mẹo] Bộ Công Cụ Tối Ưu Ảnh Được Khuyến Nghị

- **TinyPNG / TinyJPG**: Nén JPEG và PNG trực tuyến, giảm 60–80% dung lượng.
- **Squoosh** (squoosh.app): Công cụ của Google, hỗ trợ chuyển đổi sang WebP và AVIF.
- **ImageMagick**: Công cụ dòng lệnh mạnh mẽ cho xử lý ảnh hàng loạt.
- **Cloudinary / Imgix**: CDN ảnh thông minh, tự động tối ưu và phục vụ đúng định dạng.

20. Hỗ Trợ Trình Duyệt

[Ghi Chú] Hỗ Trợ Trình Duyệt Của Thẻ img Và Các Thuộc Tính Mới

- Thẻ `<img>` hỗ trợ tất cả các trình duyệt hiện đại.
- `loading="lazy"` chỉ được hỗ trợ từ Chrome 76+, Firefox 75+, Edge Chromium. Không hỗ trợ Safari cũ.

21. Bảng Tóm Tắt Nhanh — Có Nên Dùng Thuộc Tính?

[Bảng] Bảng Đánh Giá Mức Độ Ưu Tiên Các Thuộc Tính Thẻ img

Thuộc tính	Có nên dùng?	Vì sao?	Mô tả
`src`	Bắt buộc	Đường dẫn ảnh	Không có src thì không có ảnh.
`alt`	Quan trọng	Trợ năng và SEO	Giúp người khiếm thị và Google hiểu ảnh.
`width`, `height`	Nên có	Kiểm soát bố cục	Tránh CLS, trình duyệt đặt trước không gian.
`loading="lazy"`	Nên dùng	Giảm tải trang	Lazy loading tăng tốc đáng kể trang dài.
`title`	Tùy chọn	Tooltip phụ	Không bắt buộc, chỉ là thông tin bổ sung.
`decoding="asyn`	Nên dùng	Không chặn render	Cải thiện hiệu suất tổng thể trang.

22. Chi Tiết Các Loại Định Dạng Ảnh Phổ Biến

22.1. JPEG / JPG (.jpg, .jpeg):

[Bảng] Bảng Chi Tiết Định Dạng JPEG / JPG

Tiêu chí	Mô tả
Loại nén	Nén mất dữ liệu (lossy).
Dung lượng	Nhỏ — thích hợp cho web.
Chất lượng	Tốt ở mức trung bình đến cao.
Hỗ trợ nền trong suốt	Không hỗ trợ.
Hỗ trợ trình duyệt	Tất cả trình duyệt.
Ứng dụng	Ảnh chụp, ảnh nền, blog, sản phẩm.

Ví Dụ Sử Dụng Ảnh JPEG

```
<img src="photo.jpg" alt="Ảnh chụp tự nhiên">
```

22.2. PNG (.png):

[Bảng] Bảng Chi Tiết Định Dạng PNG

Tiêu chí	Mô tả
Loại nén	Nén không mất dữ liệu (lossless).
Dung lượng	Trung bình đến lớn.
Chất lượng	Rất cao, sắc nét.
Hỗ trợ nền trong suốt	Có (Alpha channel).
Hỗ trợ trình duyệt	Tất cả trình duyệt.
Ứng dụng	Logo, icon, ảnh có nền trong suốt, ảnh đồ họa.

Ví Dụ Sử Dụng Ảnh PNG

```
<img src="logo.png" alt="Logo công ty trong suốt">
```

22.3. GIF (.gif):

[Bảng] Bảng Chi Tiết Định Dạng GIF

Tiêu chí	Mô tả
Loại nén	Không mất dữ liệu.
Dung lượng	Nhỏ đến vừa.
Hỗ trợ nền trong suốt	Có (1-bit alpha, không mềm mại).
Hỗ trợ chuyển động	Có (ảnh động nhiều frame).
Màu sắc	Chỉ tối đa 256 màu.
Ứng dụng	Ảnh động đơn giản, biểu cảm, hiệu ứng nhỏ.

Ví Dụ Sử Dụng Ảnh GIF Động

```
<img src="funny.gif" alt="Ảnh động hài hước">
```

22.4. SVG (.svg):

[Bảng] Bảng Chi Tiết Định Dạng SVG

Tiêu chí	Mô tả
Loại ảnh	Vector (dựa trên công thức toán học, không phải pixel).
Dung lượng	Rất nhỏ (file text XML).
Có thể phóng to không vỡ	Có — không bao giờ bị vỡ pixel.
Hỗ trợ trình duyệt	Rất tốt (tất cả trình duyệt hiện đại).
Có thể CSS/JS can thiệp	Có thể chỉnh màu, animate bằng CSS và JavaScript.
Ứng dụng	Icon, logo, đồ họa đơn giản, biểu đồ.

Ví Dụ Sử Dụng Ảnh SVG Vector

```
<img src="icon.svg" alt="Biểu tượng vector">
```

22.5. WebP (.webp):

[Bảng] Bảng Chi Tiết Định Dạng WebP

Tiêu chí	Mô tả
Loại nén	Cả lossy và lossless.
Dung lượng	Rất nhỏ (~25–30% nhỏ hơn JPEG/PNG cùng chất lượng).
Chất lượng	Rất tốt.
Hỗ trợ nền trong suốt	Có.
Hỗ trợ ảnh động	Có (thay thế GIF).
Hỗ trợ trình duyệt	Chrome, Firefox, Edge, Safari 14+ (tốt).
Ứng dụng	Tối ưu tốc độ website, thay thế JPEG/PNG.

Ví Dụ Sử Dụng Ảnh WebP Tối Ưu

```
<img src="banner.webp" alt="Ảnh tối ưu WebP">
```

22.6. AVIF (.avif) — Định Dạng Hiện Đại:

[Bảng] Bảng Chi Tiết Định Dạng AVIF

Tiêu chí	Mô tả
Loại nén	Lossy và Lossless.
Dung lượng	Nhỏ hơn cả WebP (thế hệ mới nhất).
Chất lượng	Rất cao.
Hỗ trợ nền trong suốt	Có.
Hỗ trợ trình duyệt	Chrome, Firefox, Edge, Safari 16+.
Ứng dụng	Web tốc độ cao, ảnh chất lượng tốt, tiết kiệm băng thông.

22.7. HEIC / HEIF — Định Dạng Apple:

[Bảng] Bảng Chi Tiết Định Dạng HEIC / HEIF

Tiêu chí	Mô tả
Dung lượng	Nhỏ hơn JPEG cùng chất lượng.
Chất lượng	Cao.
Hỗ trợ nền trong suốt	Có.
Hỗ trợ trình duyệt	Không đầy đủ — Safari/Apple tốt hơn, Chrome/Firefox hạn chế.
Ứng dụng	Ảnh chụp từ iPhone, iPad, macOS.

23. Bảng So Sánh Nhanh Tất Cả Định Dạng Ảnh

[Bảng] Bảng So Sánh Nhanh Tất Cả Định Dạng Ảnh Phổ Biến

Định dạng	Nền trong suốt	Ảnh động	Nén mất dữ liệu	Vector	Phù hợp dùng
JPG	Không	Không	Có (lossy)	Không	Ảnh chụp, gallery.
PNG	Có	Không	Không (lossless)	Không	Logo, icon, nền trong suốt.
GIF	Có	Có	Không	Không	Ảnh động đơn giản.
SVG	Có	Không	Không áp dụng	Có	Icon, logo vector.
WebP	Có	Có	Cả hai	Không	Tối ưu ảnh web.
AVIF	Có	Có	Cả hai	Không	Web tốc độ cao.
HEIC	Có	Không	Lossy	Không	Ảnh iPhone/macOS.

24. Gợi Ý Sử Dụng Định Dạng Ảnh Thực Tế

[Bảng] Bảng Gợi Ý Chọn Định Dạng Ảnh Theo Mục Đích Thực Tế

Mục đích sử dụng	Định dạng nên dùng
Ảnh chụp, gallery, sản phẩm	JPG, WebP, AVIF
Logo, icon	SVG, PNG
Ảnh có nền trong suốt	PNG, WebP, SVG
Ảnh động nhỏ (sticker, reaction)	GIF, WebP
Tối ưu tốc độ web	WebP, AVIF (kèm fallback JPG/PNG qua `<picture>`)
Đồ họa vector, biểu đồ	SVG

[Khái Niệm] Kết Luận Về Thẻ a Kết Hợp Thẻ img

- Thẻ `<a>` kết hợp với `<img>` giúp tạo liên kết trực quan hơn.
- Có thể sử dụng để dẫn đến một trang khác, một phần trong trang, hoặc mở ứng dụng (điện thoại, email).
- Có thể thêm hiệu ứng hover bằng CSS để tăng trải nghiệm người dùng.

3.4.5 e. Lý Thuyết Về Đường Dẫn (Path) Trong HTML

Trong HTML, **đường dẫn** (path) là cách để chỉ định vị trí của một tệp (hình ảnh, tệp CSS, JavaScript, tài liệu, v.v.) trên máy chủ hoặc máy tính cá nhân. Đường dẫn có thể là **tuyệt đối** hoặc **tương đối**, tùy thuộc vào cách bạn muốn truy cập tệp.

[Khái Niệm] Định Nghĩa Đường Dẫn (Path) Trong HTML

Đường dẫn là cách chỉ định vị trí của một tệp tài nguyên (ảnh, CSS, JavaScript, video, tài liệu) để trình duyệt có thể tìm và hiển thị nó. Đường dẫn được sử dụng phổ biến trong các thuộc tính:

- `<a href="...">`: Đường dẫn cho liên kết.
- `<img src="...">`: Đường dẫn cho hình ảnh.
- `<link href="...">`: Đường dẫn cho tệp CSS.
- `<script src="...">`: Đường dẫn cho tệp JavaScript.

1. Các Loại Đường Dẫn Trong HTML

1.1. Đường Dẫn Tuyệt Đối (Absolute Path)

Là đường dẫn đầy đủ tính từ gốc của máy chủ hoặc hệ thống. Dùng khi tệp nằm trên một máy chủ khác hoặc website khác.

Ví Dụ Đường Dẫn Tuyệt Đối

```
<img src="https://example.com/images/photo.jpg" alt="Ảnh trên server">
<a href="https://www.google.com">Truy cập Google</a>
```

- **Ưu điểm:** Luôn chính xác, không bị ảnh hưởng bởi vị trí tệp HTML.
- **Nhược điểm:** Nếu đường dẫn thay đổi, liên kết sẽ bị lỗi. Tốc độ tải có thể chậm nếu máy chủ xa.

[Mẹo] Khi Nào Nên Dùng Đường Dẫn Tuyệt Đối?

- Liên kết đến trang web **bên ngoài** (Google, Facebook, CDN).
- Nhúng ảnh từ **dịch vụ lưu trữ ảnh bên ngoài** (Cloudinary, Imgix, S3).
- Tham chiếu đến tài nguyên API hoặc font từ Google Fonts.
- Khi website được nhúng vào **email HTML** — email client không hiểu đường dẫn tương đối.

1.2. Đường Dẫn Tương Đối (Relative Path)

Là đường dẫn dựa trên vị trí của tệp HTML đang chứa nó. Dùng khi tệp nằm trên cùng một trang web.

Ví Dụ Đường Dẫn Tương Đối

```
<img src="images/photo.jpg" alt="Ảnh nằm trong thư mục images">
<a href="contact.html">Trang Liên hệ</a>
```

- **Ưu điểm:** Linh hoạt, dễ thay đổi cấu trúc thư mục mà không bị lỗi. Không cần dùng tên miền đầy đủ.
- **Nhược điểm:** Nếu tệp HTML di chuyển sang thư mục khác, đường dẫn có thể bị lỗi.

[Ghi Chú] Đọc Thêm — URL Encoding Trong Đường Dẫn

Nếu tên file hoặc thư mục chứa ký tự đặc biệt (khoảng trắng, dấu tiếng Việt, ký hiệu), trình duyệt sẽ tự động mã hóa chúng theo chuẩn URL Encoding. Ví dụ: khoảng trắng thành `%20`, dấu `&` thành `%26`. Để tránh rắc rối, **đặt tên file và thư mục chỉ dùng chữ thường, số, dấu gạch ngang** `-` hoặc gạch dưới `_`.

2. Các Dạng Đường Dẫn Tương Đối

2.1. Đường Dẫn Cùng Thư Mục:

Khi tệp HTML và tài nguyên nằm cùng một thư mục.

Ảnh Cùng Thư Mục Với Tệp HTML

```
<img src="photo.jpg" alt="Ảnh cùng thư mục">
```

2.2. Đường Dẫn Vào Thư Mục Con:

Khi tệp HTML nằm ngoài, còn tài nguyên nằm trong thư mục con.

Ảnh Nằm Trong Thư Mục Con images/

```
<img src="images/photo.jpg" alt="Ảnh trong thư mục images">
```

2.3. Đường Dẫn Ra Thư Mục Cha (../):

Khi tệp HTML nằm trong thư mục con, cần truy cập tệp ở thư mục cha. `../` có nghĩa là lùi lại một cấp thư mục. Nếu cần lùi hai cấp, dùng `../../`.

Lùi Ra Thư Mục Cha Bằng ../

```
<img src="../photo.jpg" alt="Ảnh trong thư mục cha">
<img src="../../images/logo.png" alt="Ảnh lùi 2 cấp">
```

2.4. Đường Dẫn Từ Gốc Trang Web (/):

Bắt đầu từ thư mục gốc của trang web.

Đường Dẫn Từ Gốc Website Dùng /

```
<img src="/images/photo.jpg" alt="Ảnh từ gốc trang web">
```

[Ghi Chú] Lưu Ý Khi Dùng Đường Dẫn Gốc /

Đường dẫn bắt đầu bằng / hoạt động tốt trên máy chủ web, nhưng trên máy tính cá nhân (mở file trực tiếp qua trình duyệt) có thể bị lỗi vì trình duyệt sẽ trỏ về gốc ổ đĩa thay vì thư mục dự án.

[Cảnh Báo] Phân Biệt Chữ Hoa/Thường Trong Đường Dẫn

Trên hệ thống **Linux/macOS** (phần lớn server web dùng Linux), đường dẫn **phân biệt chữ hoa và thường**:

- `Images/photo.jpg` và `images/photo.jpg` là **hai đường dẫn khác nhau**.
- Code chạy đúng trên Windows (không phân biệt hoa/thường) nhưng **bị lỗi khi deploy lên Linux server**.

Giải pháp: Luôn dùng **chữ thường nhất quán** cho tất cả tên file và thư mục.

[Bảng] Bảng Tóm Tắt Các Dạng Đường Dẫn Tương Đối Phổ Biến

Đường dẫn	Ý nghĩa
`image.jpg`	Cùng thư mục với tệp HTML hiện tại.
`images/image.jpg`	Thư mục con `images/` của thư mục hiện tại.
`../image.jpg`	Thư mục cha (lùi 1 cấp).
`../../image.jpg`	Hai cấp thư mục cha (lùi 2 cấp).
`/images/image.jpg`	Từ thư mục gốc của website (server).

3. Ví Dụ Minh Họa Từng Trường Hợp

3.1. Tệp HTML và hình ảnh cùng thư mục:

Cấu Trúc Thư Mục — HTML Và Ảnh Cùng Cấp

```
project/├──
  index.html└──
  logo.png
```

Mã HTML Trỏ Đến Ảnh Cùng Thư Mục

```html
<img src="logo.png" alt="Logo">
<img src="./logo.png" alt="Logo">      <!-- Tương đương nhau -->
```

[Ghi Chú] Hai Cách Viết Tương Đương

`./baby.jpg` và `baby.jpg` hoàn toàn tương đương — cả hai đều trỏ đến thư mục hiện tại. Dấu `./` có thể bỏ qua.

[Mẹo] Đặt Tên File Và Thư Mục Theo Chuẩn

- Dùng **chữ thường** và dấu **gạch ngang** - để phân cách: `my-photo.jpg`, `hero-image.webp`.
- Không dùng khoảng trắng, dấu tiếng Việt, hay ký tự đặc biệt.
- Tổ chức thư mục theo quy ước: `images/`, `css/`, `js/`, `assets/`.

3.2. Tệp HTML ở ngoài, hình ảnh trong thư mục con:

Cấu Trúc — HTML Ngoài Thư Mục Gốc, Ảnh Trong images/

```
project/├──
 index.html└──
 images/
    └── photo.jpg
```

Mã HTML Trỏ Vào Thư Mục Con images/

```
<img src="images/photo.jpg" alt="Hình ảnh trong thư mục images">
```

3.3. Tệp HTML trong thư mục con, cần lùi ra thư mục cha:

Cấu Trúc — HTML Trong about/, Ảnh Trong images/

```
project/├──
 index.html├──
 images/│
   └── photo.jpg└──
 about/
    └── about.html          <!-- Đang ở đây -->
```

Mã HTML Trong about.html Trỏ Ra Thư Mục Cha

```
<!-- Trong about/about.html, lùi 1 cấp để vào images/ -->
<img src="../images/photo.jpg" alt="Hình ảnh trong thư mục cha">
```

3.4. Dùng / để chỉ thư mục gốc trên máy chủ:

Cấu Trúc Máy Chủ — Đường Dẫn Tuyệt Đối Từ Gốc

```
/var/www/html/├──
 index.html└──
 assets/
    └── logo.png
```

Mã HTML Dùng Đường Dẫn Từ Gốc Server

```
<img src="/assets/logo.png" alt="Logo từ thư mục gốc">
```

4. So Sánh Chi Tiết 3 Cách Viết Đường Dẫn

Xét ví dụ minh họa với 3 cách viết khác nhau cho cùng một hình ảnh:

Ba Cách Viết Đường Dẫn Cho Cùng Một Hình Ảnh

```
<img src="img/baby.jpg" alt="Baby Image">     <!-- Cách 1 -->
<img src="/project/img/baby.jpg" alt="Baby Image"> <!-- Cách 2 -->
<img src="./img/baby.jpg" alt="Baby Image">  <!-- Cách 3 -->
```

Cách 1 — img/baby.jpg (Đường dẫn tương đối, không có ./ hay /):

- Đây là đường dẫn **tương đối không có tiền tố**. Trình duyệt tìm thư mục `img` cùng cấp với tệp HTML đang chạy.
- Nếu HTML ở `/project/index.html`, ảnh phải nằm ở `/project/img/baby.jpg`.
- **Ưu điểm:** Hoạt động linh hoạt khi di chuyển HTML và thư mục ảnh cùng nhau.
- **Nhược điểm:** Nếu HTML ở thư mục con, đường dẫn này có thể sai.

Cách 2 — /project/img/baby.jpg (Đường dẫn tuyệt đối từ gốc website):

- Đây là đường dẫn **tuyệt đối từ gốc website**. Trình duyệt luôn tìm từ thư mục gốc của domain, không phụ thuộc vị trí file HTML.
- **Ưu điểm:** Dễ quản lý trên máy chủ với cấu trúc thư mục cố định.
- **Nhược điểm:** Nếu trang web không được deploy trong thư mục `/project/`, đường dẫn sẽ bị lỗi.

[Cảnh Báo] Cẩn Thận Với Đường Dẫn Tuyệt Đối Từ Gốc

Trình duyệt sẽ tìm `/project/img/baby.jpg` từ gốc domain (ví dụ: `https://yourdomain.com/project/img/baby.jpg`). Chỉ dùng loại đường dẫn này khi chắc chắn cấu trúc thư mục trên server khớp hoàn toàn. Trên môi trường localhost không qua server (mở file trực tiếp), cách này thường bị lỗi.

Cách 3 — ./img/baby.jpg (Đường dẫn tương đối có ./):

- Dấu `./` có nghĩa là **thư mục hiện tại** — chính là thư mục chứa tệp HTML đang chạy.
- Hoàn toàn **tương đương** với `img/baby.jpg`, vì `./` có thể được bỏ qua.
- Nếu HTML ở `/project/index.html`, ảnh phải nằm ở `/project/img/baby.jpg`.

[Bảng] Bảng Tổng Kết So Sánh 3 Cách Viết Đường Dẫn

Cách viết	Loại đường dẫn	Tìm ảnh ở đâu?	Lưu ý
`img/baby.jpg`	Tương đối	Cùng cấp với file HTML	Hoạt động tốt nếu HTML cùng cấp với `img/`.
`/img/baby.jpg`	Tuyệt đối từ gốc web	`/img/baby.jpg` từ thư mục gốc web	Chỉ dùng nếu chắc ảnh ở thư mục gốc website.
`./img/baby.jpg`	Tương đối (có ./)	Cùng cấp với file HTML	Tương đương `img/baby.jpg`.

[Mẹo] Mẹo Kiểm Tra Đường Dẫn Nhanh Bằng DevTools

Mở **DevTools** (F12) → tab **Network** → tải lại trang. Nếu có tài nguyên nào bị lỗi (status `404 Not Found`), đường dẫn đang bị sai. Nhấp vào request đó để xem đường dẫn đầy đủ mà trình duyệt đang thử tải.

[Mẹo] Khuyến Nghị Thực Tế

Nếu không chắc chắn, hãy sử dụng đường dẫn tương đối (`img/baby.jpg`) để tránh lỗi khi chạy local lẫn khi deploy lên server. Chỉ dùng `/img/baby.jpg` khi bạn kiểm soát được cấu trúc thư mục gốc của server.

5. Mô Phỏng Cấu Trúc Thư Mục Thực Tế

Cấu Trúc Thư Mục Dự Án Thực Tế Nhiều Bài

```
khoa-front-end-t2-2023/├──
 bai-01/└──
 bai-02/
    ├── images/
    │   ├── image-2.jpg
    │   └── image.jpg
    ├── index.html
    ├── audio.mp3
    ├── block-and-inline.html
    ├── class-id.html
    ├── list.jpg
    ├── table-1.jpg
    ├── table-2.jpg
    ├── table.html
    ├── ul-ol.html
    └── video.mp4
```

- Thư mục `bai-02/` chứa tất cả các tệp HTML, hình ảnh, âm thanh và video.
- Thư mục `images/` bên trong `bai-02/` chứa các hình ảnh như `image-2.jpg` và `image.jpg`.
- Tệp `index.html` nằm trực tiếp trong `bai-02/`, không nằm trong `images/`.

Trường hợp 1 — Đường dẫn tương đối (không dùng /):

Khi không dùng dấu `/` ở đầu, đang trỏ đến thư mục `bai-02/`. Vì `images/` đồng cấp với `index.html` trong thư mục này:

Truy Cập Ảnh Từ index.html Bằng Đường Dẫn Tương Đối

```html
<img src="images/image-2.jpg" alt="Image 2">
```

Vì `index.html` và thư mục `images/` nằm cùng cấp trong `bai-02/`, chỉ cần `images/image-2.jpg` để trỏ đúng đến ảnh.

Trường hợp 2 — Đường dẫn tuyệt đối từ gốc (dùng /):

Khi dùng dấu `/` ở đầu, trỏ về thư mục gốc `khoa-front-end-t2-2023/` và từ thư mục này đi vào `bai-02/` rồi mới vào `images/`:

Truy Cập Ảnh Từ index.html Bằng Đường Dẫn Gốc

```html
<img src="/bai-02/images/image-2.jpg" alt="Image 2">
```

6. Đường Dẫn Trong CSS & JavaScript

6.1. Đường Dẫn Trong CSS:

Sử Dụng url() Trong CSS Để Chỉ Đường Dẫn Ảnh Nền

```
body {
    background-image: url("images/background.jpg");
}
```

[Ghi Chú] Lưu Ý Đường Dẫn CSS Tương Đối Theo Vị Trí File .css

Đường dẫn `url()` trong CSS được tính **tương đối theo vị trí của file CSS**, không phải theo file HTML. Nếu file CSS nằm trong thư mục `css/`, và ảnh nằm trong `images/`, thì trong CSS phải viết `url("../images/background.jpg")` — lùi 1 cấp ra khỏi `css/` rồi vào `images/`.

[Mẹo] Dùng Đường Dẫn Tuyệt Đối Từ Gốc Trong CSS Để Tránh Nhầm Lẫn

Nếu bạn dùng framework (React, Vue, Webpack), hãy ưu tiên đường dẫn tuyệt đối từ gốc `/images/background.jpg` trong CSS để tránh nhầm lẫn khi file CSS được đặt ở nhiều cấp thư mục khác nhau.

6.2. Đường Dẫn Trong JavaScript:

Liên Kết File JavaScript Bằng Thẻ script

```
<script src="js/script.js"></script>
```

[Ghi Chú] Đọc Thêm — Thẻ base Trong HTML

Thẻ `<base href="...">` trong `<head>` cho phép bạn đặt **đường dẫn gốc mặc định** cho tất cả các liên kết tương đối trên trang. Ví dụ:

- `<base href="https://example.com/project/">`
- Sau đó `<img src="images/photo.jpg">` sẽ được hiểu là: `https://example.com/project/images/photo.jpg`

Rất hữu ích khi làm Single Page Application (SPA) hoặc khi cần thay đổi base URL trong một lần duy nhất.

7. So Sánh Đường Dẫn Tuyệt Đối & Tương Đối

[Bảng] Bảng So Sánh Đường Dẫn Tuyệt Đối Và Tương Đối

Loại đường dẫn	Cách viết ví dụ	Ưu & Nhược điểm
Tuyệt đối	`https://example.com/im`	Luôn chính xác, không bị ảnh hưởng vị trí tệp HTML. Khó bảo trì, phụ thuộc máy chủ.
Tương đối	`images/photo.jpg`	Linh hoạt, dễ thay đổi cấu trúc thư mục. Có thể bị lỗi nếu tệp HTML thay đổi vị trí.

8. Khi Nào Dùng Đường Dẫn Tuyệt Đối & Tương Đối?

[Khái Niệm] Quy Tắc Chọn Loại Đường Dẫn Phù Hợp

Dùng đường dẫn tuyệt đối khi:

- Liên kết đến một trang web khác (domain khác).
- Cần đảm bảo đường dẫn luôn chính xác bất kể vị trí file HTML.
- Tham chiếu đến tài nguyên CDN hoặc API bên ngoài.

Dùng đường dẫn tương đối khi:

- Làm việc trong cùng một trang web / dự án.
- Muốn dễ dàng di chuyển file hoặc deploy lên nhiều môi trường khác nhau.
- Phát triển local và cần code hoạt động cả local lẫn production.

[Bảng] Bảng So Sánh Tổng Quan Đường Dẫn Tuyệt Đối Và Tương Đối

Tiêu chí	Đường dẫn tuyệt đối	Đường dẫn tương đối
Dễ bảo trì	Khó bảo trì nếu thay đổi domain.	Dễ bảo trì khi di chuyển website.
Tốc độ tải trang	Có thể chậm nếu trỏ đến nguồn ngoài.	Nhanh hơn do tải từ cùng máy chủ.
Linh hoạt khi di chuyển file	Không bị ảnh hưởng.	Có thể bị lỗi nếu thay đổi vị trí tệp.
Khả năng hoạt động offline	Không hoạt động nếu trỏ tài nguyên ngoài.	Hoạt động tốt nếu dùng cho nội bộ.
Dùng với tài nguyên ngoài website	Dùng tốt cho liên kết bên ngoài.	Không thể trỏ đến tài nguyên bên ngoài.
Tính đơn giản khi viết mã	Dễ hiểu nếu URL cố định.	Dễ nhầm lẫn với đường dẫn lùi `../`.

9. Lưu Ý Quan Trọng Khi Dùng Đường Dẫn

[Mẹo] Tối Ưu Hiệu Suất Khi Dùng Đường Dẫn Ảnh

- Dùng **WebP** thay cho JPG/PNG để giảm dung lượng 25–50%.
- Kết hợp `loading="lazy"` cho tất cả ảnh không hiển thị ngay:

Ảnh Tối Ưu Kết Hợp WebP Và Lazy Loading

```
<img src="photo.webp" loading="lazy" alt="Ảnh tối ưu">
```

[Khái Niệm] Kết Luận — Đường Dẫn Trong HTML

- Đường dẫn giúp trình duyệt tìm đến tệp cần hiển thị.
- Có **đường dẫn tuyệt đối** (URL đầy đủ) và **đường dẫn tương đối** (dựa trên vị trí tệp HTML).
- `img/baby.jpg` và `./img/baby.jpg` hoàn toàn tương đương — khuyến nghị dùng dạng này cho dự án nội bộ.
- Chọn đường dẫn phù hợp giúp tăng hiệu suất và tránh lỗi khi xây dựng trang web.

3.4.6 f. Thẻ Video Trong HTML (Video - <video>)

Thẻ `<video>` trong HTML được sử dụng để nhúng video trực tiếp vào trang web mà không cần sử dụng trình phát bên thứ ba như YouTube hay Vimeo.

[Khái Niệm] Định Nghĩa Thẻ <video> Trong HTML

Thẻ `<video>` là phần tử HTML5 cho phép phát video trực tiếp trên trình duyệt web. Nó hỗ trợ điều khiển phát lại, tự động phát, phát lặp, ảnh đại diện và tích hợp nhiều nguồn video khác nhau qua thẻ con `<source>`.

1. Cú Pháp Cơ Bản Của Thẻ <video>

Cú Pháp Cơ Bản Nhúng Video Với Thẻ <video>

```
<video controls>
    <source src="video.mp4" type="video/mp4">
    <source src="video.webm" type="video/webm">
    Trình duyệt của bạn không hỗ trợ thẻ video.
</video>
```

- `controls`: Hiển thị các nút điều khiển mặc định của trình duyệt (phát, tạm dừng, âm lượng, toàn màn

hình).

- `<source>`: Xác định các tệp video với định dạng khác nhau để trình duyệt tự động chọn định dạng phù hợp nhất.
- **Dòng văn bản dự phòng (text fallback):** Hiển thị thông báo nếu trình duyệt của người dùng quá cũ không hỗ trợ thẻ `<video>`.

Phân Tích 2 Cách Sử Dụng Thẻ <video> & Ý Nghĩa Của Từng Cách:

Trong thực tế lập trình web, thẻ `<video>` có 2 cách khai báo chính với cú pháp và ngữ cảnh sử dụng hoàn toàn khác nhau:

Cách 1 — Khai Báo src Trực Tiếp Trong Thẻ Thẻ Mở video

```
<video src="video.mp4" controls width="640" height="360">
    Trình duyệt không hỗ trợ video.
</video>
```

Cách 2 — Dùng Các Thẻ Con source Bên Trong Khung video

```
<video controls width="640" height="360">
    <source src="video.mp4" type="video/mp4">
    <source src="video.webm" type="video/webm">
    Trình duyệt không hỗ trợ video.
</video>
```

[Bảng] Bảng So Sánh 2 Cách Sử Dụng Thẻ <video>

Tiêu chí	Cách 1: Khai báo src trực tiếp	Cách 2: Sử dụng thẻ con <source>
Cú pháp	Ngắn gọn, khai báo `src="..."` ngay trong thẻ mở `<video>`.	Khai báo nhiều phần tử con `<source>` bên trong.
Số lượng tệp	Chỉ khai báo được **duy nhất 1 tệp video**.	Khai báo được **nhiều tệp dự phòng** ở các định dạng/codec khác nhau.
Khả năng tương thích	Kém linh hoạt. Nếu trình duyệt không hỗ trợ định dạng này, video sẽ bị lỗi hoàn toàn.	Cao nhất. Trình duyệt tự đọc lần lượt từ trên xuống và chọn tệp đầu tiên nó hỗ trợ.
Tối ưu băng thông	Không tối ưu nếu file có dung lượng lớn mà trình duyệt không phát được.	Trình duyệt đọc thuộc tính `type` để quyết định tải file mà không lãng phí băng thông.
Ngữ cảnh sử dụng	Dùng cho các dự án nhỏ, chạy thử nghiệm nhanh với duy nhất tệp MP4 chuẩn.	**Chuẩn hóa Production**: Bắt buộc dùng cho dự án thực tế để đảm bảo tương thích mọi thiết bị.

2. Các Thuộc Tính Của Thẻ `<video>`

[Bảng] Bảng Thuộc Tính Của Thẻ <video>

Thuộc tính	Ý nghĩa	Cú pháp mẫu
`controls`	Hiển thị thanh điều khiển video (play, pause, volume...)	`<video controls>`
`autoplay`	Tự động phát video ngay khi tải xong trang	`<video autoplay>`
`loop`	Tự động phát lại video liên tục khi kết thúc	`<video loop>`
`muted`	Tắt âm thanh của video mặc định khi khởi tạo	`<video muted>`
`poster`	Hình ảnh hiển thị làm đại diện (thumbnail) trước khi video được phát	`<video poster="thumbnail.jpg">`
`preload`	Kiểm soát cơ chế tải trước tệp video (`auto`, `metadata`, `none`)	`<video preload="metadata">`
`width` & `height`	Định kích thước chiều rộng và chiều cao cho khung hiển thị video	`<video width="640" height="360">`

3. Hỗ Trợ Định Dạng Video Trong HTML

Không phải trình duyệt nào cũng hỗ trợ tất cả các định dạng video. Việc cung cấp nhiều nguồn video qua thuộc tính `type` giúp tối ưu khả năng tương thích.

[Bảng] Bảng Hỗ Trợ Định Dạng Video Trên Các Trình Duyệt

Định dạng	MIME Type	Chrome	Firefox	Safari	Edge	Opera
MP4 (H.264)	`video/mp4`	Có	Có	Có	Có	Có
WebM (VP8/VP9)	`video/webm`	Có	Có	Không	Có	Có
Ogg (Theora)	`video/ogg`	Có	Có	Không	Không	Có

[Mẹo] Khuyến Nghị Định Dạng Video

Nên luôn sử dụng định dạng **MP4 (h.264)** làm định dạng chính vì nó được hỗ trợ 100% trên tất cả các trình duyệt hiện đại cũng như các thiết bị di động (iOS/Android).

4. Các Ví Dụ Chi Tiết

4.1. Video với điều khiển, tự động phát và lặp lại:

Mẫu Video Có Controls, Autoplay, Loop Và Muted

```
<video controls autoplay loop muted>
    <source src="video.mp4" type="video/mp4">
    <source src="video.webm" type="video/webm">
    Trình duyệt của bạn không hỗ trợ thẻ video.
</video>
```

[Cảnh Báo] Chính Sách Autoplay Trên Các Trình Duyệt Hiện Đại

Thuộc tính `autoplay` **chỉ hoạt động khi có kèm thuộc tính** `muted`. Các trình duyệt hiện đại (Chrome, Safari, Edge) mặc định chặn video tự động phát có âm thanh để tránh làm phiền trải nghiệm người dùng. Các thuộc tính mong muốn cần được khai báo trong thẻ mở `<video>`, cách nhau bởi một hoặc nhiều khoảng trắng.

4.2. Video có ảnh đại diện (poster / thumbnail):

Nhúng Video Có Ảnh Đại Diện Khi Chưa Phát

```
<video controls poster="thumbnail.jpg">
    <source src="video.mp4" type="video/mp4">
</video>
```

Ảnh đại diện (`poster`) sẽ hiển thị làm khung xem trước cho đến khi người dùng nhấp vào nút ”Play”.

4.3. Video cung cấp nhiều định dạng dự phòng:

Nhúng Video Đa Định Dạng Tối Ưu Tương Thích Trình Duyệt

```
<video controls>
    <source src="video.mp4" type="video/mp4">
    <source src="video.webm" type="video/webm">
    <source src="video.ogv" type="video/ogg">
    Trình duyệt của bạn không hỗ trợ video.
</video>
```

Trình duyệt sẽ duyệt lần lượt từ trên xuống dưới và phát định dạng đầu tiên mà nó hỗ trợ.

4.4. Điều chỉnh kích thước video (Thuộc tính HTML vs. CSS):

Cách 1: Dùng thuộc tính `width` & `height` của HTML

Đặt Kích Thước Trực Tiếp Bằng Thuộc Tính HTML

```
<video controls width="640" height="360">
    <source src="video.mp4" type="video/mp4">
</video>
```

Cách 2: Dùng CSS để tạo video co giãn linh hoạt (Responsive Video)

Định Tỷ Lệ Co Giãn Cho Video Bằng CSS

```
<video controls class="video-custom">
    <source src="video.mp4" type="video/mp4">
</video>

<style>
    .video-custom {
        width: 100%;
        max-width: 800px;
        height: auto;
    }
</style>
```

[Mẹo] Lời Khuyên Khi Khai Báo Thẻ <source>

- **Luôn đặt đúng `type`:** Giúp trình duyệt nhận biết chính xác bộ giải mã (codec) mà không mất công tải thử file.
- **Cung cấp nhiều định dạng dự phòng:** Luôn khai báo MP3 kèm Ogg để tối ưu trên mọi nền tảng.
- **Kiểm thử đa trình duyệt:** Kiểm tra phát nhạc trên cả Safari (iOS/macOS) và Chrome/Firefox (Android/Windows).

3. Các Thuộc Tính Của Thẻ `<audio>`

[Bảng] Bảng Thuộc Tính Của Thẻ <audio>

Thuộc tính	Ý nghĩa	Cú pháp mẫu
`controls`	Hiển thị thanh điều khiển âm thanh mặc định	`<audio controls>`
`autoplay`	Tự động phát âm thanh ngay khi tải xong trang	`<audio autoplay>`
`loop`	Tự động phát lặp lại âm thanh liên tục	`<audio loop>`
`muted`	Tắt tiếng âm thanh mặc định khi khởi tạo	`<audio muted>`
`preload`	Điều khiển cơ chế tải trước tệp âm thanh	`<audio preload="auto">`

[Cảnh Báo] Hành Vi Autoplay Âm Thanh Trên Trình Duyệt Hiện Đại

Nếu chọn thuộc tính tự động phát âm thanh (`autoplay`), trên phần lớn các trình duyệt hiện đại (Chrome, Safari, Firefox), **âm thanh sẽ bị chặn hoặc tự động chuyển sang chế độ tắt tiếng (muted)**. Trình duyệt yêu cầu phải có tương tác của người dùng (click, tap) trước khi phát âm thanh có tiếng.

4. Chi Tiết Về Thuộc Tính preload

Thuộc tính `preload` quyết định cách trình duyệt tải trước tệp âm thanh trước khi người dùng nhấn phát.

[Bảng] Bảng Các Giá Trị Của Thuộc Tính preload

Giá trị	Ý nghĩa & Cơ chế tải
`auto`	Trình duyệt tự động tải trước toàn bộ file âm thanh (có thể gây tốn băng thông người dùng).
`metadata`	Trình duyệt chỉ tải trước thông tin dữ liệu tả (kích thước file, độ dài thời lượng).
`none`	Không tải trước bất kỳ dữ liệu nào cho tới khi người dùng bấm nút ”Play” (tối ưu băng thông tối đa).

Ví Dụ Tối Ưu Tải Âm Thanh Bằng preload=”metadata”

```
<audio controls preload="metadata">
    <source src="audio.mp3" type="audio/mpeg">
</audio>
```

5. Điều Khiển Thẻ `<audio>` Bằng JavaScript

Thẻ `<audio>` cung cấp các phương thức và thuộc tính API mạnh mẽ trong JavaScript như `play()` và `pause()` để tạo trình phát nhạc tùy chỉnh.

Mẫu Điều Khiển Phát / Dừng Âm Thanh Bằng JavaScript

```
<!-- Thẻ audio ẩn giao diện mặc định -->
<audio id="myAudio">
    <source src="audio.mp3" type="audio/mpeg">
</audio>

<!-- Các nút bấm tùy chỉnh giao diện -->
<button onclick="playAudio()">Phát Nhạc</button>
<button onclick="pauseAudio()">Tạm Dừng</button>

<script>
    let audio = document.getElementById("myAudio");

    function playAudio() {
        audio.play();
    }

    function pauseAudio() {
        audio.pause();
    }
</script>
```

6. Các Lưu Ý Quan Trọng Khi Sử Dụng Thẻ `<audio>`

[Cảnh Báo] 5 Quy Tắc Vàng Khi Nhúng Âm Thanh Vào Website

- **Đa dạng định dạng:** Tải nhiều định dạng âm thanh (MP3 + Ogg) để đảm bảo hoạt động 100% trên mọi trình duyệt.
- **Tránh lạm dụng Autoplay:** Hạn chế dùng `autoplay` vì gây khó chịu cho người dùng và bị trình duyệt chặn.
- **Tiết kiệm băng thông:** Sử dụng `preload="none"` hoặc `preload="metadata"` nếu trang web có nhiều tệp âm thanh.
- **Nén dung lượng tệp:** Nén tệp âm thanh ở bitrate phù hợp (128kbps–192kbps) để tối ưu tốc độ tải trang mà vẫn giữ chất lượng nghe tốt.
- **Tùy biến với JavaScript:** Sử dụng JavaScript API nếu cần xây dựng trình phát nhạc riêng theo giao diện thiết kế.

[Khái Niệm] Kết Luận — Thẻ Audio Trong HTML

- Thẻ `<audio>` là công cụ mạnh mẽ giúp nhúng âm thanh vào trang web một cách dễ dàng và chuẩn hóa.
- Khi sử dụng, cần lưu ý lựa chọn định dạng tệp phù hợp, kiểm soát cơ chế tải âm thanh (`preload`), và tối ưu hóa tốc độ tải trang để đem lại trải nghiệm người dùng tốt nhất.

3.4.7 h. Thẻ Danh Sách (`<ul>` và `<ol>`) Trong HTML

Danh sách (**Lists**) là một trong những phần tử cấu trúc quan trọng và được sử dụng phổ biến nhất trong HTML để nhóm các mục thông tin có liên quan lại với nhau một cách hệ thống, từ các mục menu điều hướng (`<nav>`) cho đến các danh sách bài viết hay quy trình thực hiện công việc.

1. Lý Thuyết Chuyên Sâu & Quy Tắc Ngữ Nghĩa Của Thẻ Danh Sách

Bản chất Phần Tử Khối (Block-level Element):

Cả hai thẻ `<ul>` và `<ol>` đều là phần tử khối (**Block-level elements**). Chúng chiếm 100% chiều rộng của phần tử cha và luôn bắt đầu trên một dòng mới trong luồng hiển thị HTML.

Quy tắc Cấu Trúc Thẻ Con Hợp Lệ (Strict HTML Validation):

[Cảnh Báo] Quy Tắc Vàng Về Cấu Trúc Thẻ Con Của ul Và ol

Theo tiêu chuẩn W3C/WHATWG HTML5:

- Thẻ con trực tiếp duy nhất hợp lệ của `<ul>` và `<ol>` bắt buộc phải là thẻ `<li>` (**List Item**) (hoặc các thẻ môi trường đặc biệt như `<template>`, `<script>`).
- Tuyệt đối **KHÔNG** được đặt thẻ `<div>`, `<p>`, `<a>`, hay `<span>` trực tiếp làm con của `<ul>` hoặc `<ol>`. Tất cả các thẻ này bắt buộc phải nằm **bên trong** thẻ `<li>`.

2. Thẻ `<ul>` (Unordered List – Danh Sách Không Có Thứ Tự)

Mô tả:

- Dùng để tạo danh sách không có thứ tự (mỗi mục trong danh sách có dấu đầu dòng thay vì số thứ tự).
- Mặc định, dấu đầu dòng là chấm tròn (•).
- Có thể thay đổi kiểu dấu đầu dòng bằng thuộc tính `list-style-type` trong CSS.

Cú pháp cơ bản:

Mã Nguồn Cú Pháp Cơ Bản Với Thẻ ul

```
<ul>
    <li>Mục 1</li>
    <li>Mục 2</li>
    <li>Mục 3</li>
</ul>
```

BROWSER PREVIEW — **ul – Unordered List Mặc Định**

- **Mục 1** – Phần tử danh sách thứ nhất
- **Mục 2** – Phần tử danh sách thứ hai
- **Mục 3** – Phần tử danh sách thứ ba

Thay đổi kiểu dấu đầu dòng bằng CSS:

Tùy Chỉnh Ký Hiệu Đầu Dòng Với CSS Inline list-style-type

```
<ul style="list-style-type: square;">
    <li>Mục 1</li>
    <li>Mục 2</li>
    <li>Mục 3</li>
</ul>
```

BROWSER PREVIEW — **ul – Kiểu Dấu Đầu Dòng Ô Vuông square**

- **Mục 1** – Phần tử danh sách dạng ô vuông thứ nhất.
- **Mục 2** – Phần tử danh sách dạng ô vuông thứ hai.
- **Mục 3** – Phần tử danh sách dạng ô vuông thứ ba.

[Ghi Chú] Lưu Ý Quan Trọng Về Thuộc Tính reversed

- Thuộc tính `reversed` chỉ thay đổi số thứ tự hiển thị (đếm ngược từ số lượng phần tử về 1) chứ **không làm đảo ngược vị trí cấu trúc của các thẻ `<li>` trong DOM HTML**.
- Vì thế, phần tử đầu tiên trong mã HTML `<li>Mục 1</li>` vẫn đứng đầu danh sách nhưng mang chỉ số thứ tự cao nhất là **3. Mục 1**.
- Nếu muốn thay đổi cả vị trí xuất hiện thực tế của phần tử, cần sắp xếp lại thứ tự thẻ trong HTML hoặc dùng JavaScript/CSS Flexbox (`flex-direction: column-reverse`).

5. Danh Sách Lồng Nhau (Nested Lists)

Danh sách có thể lồng nhau để tạo cấp bậc phân nhánh dữ liệu phức tạp.

Ví dụ lồng danh sách không thứ tự:

Cấu Trúc Danh Sách Không Thứ Tự Lồng Nhau Nhiều Cấp

```
<ul>
    <li>Mục chính 1</li>
    <li>Mục chính 2
        <ul>
            <li>Mục phụ 2.1</li>
            <li>Mục phụ 2.2</li>
        </ul>
    </li>
    <li>Mục chính 3</li>
</ul>
```

BROWSER PREVIEW — **Danh Sách Không Thứ Tự Lồng Nhau 2 Cấp**

- **Mục chính 1**
- **Mục chính 2**
  - Mục phụ 2.1
  - Mục phụ 2.2
- **Mục chính 3**

Ví dụ danh sách có thứ tự lồng trong danh sách không có thứ tự:

Kết Hợp Danh Sách Có Thứ Tự (ol) Bên Trong Danh Sách Không Thứ Tự (ul)

```
<ul>
    <li>Chủ đề 1</li>
    <li>Chủ đề 2
        <ol>
            <li>Mục 2.1</li>
            <li>Mục 2.2</li>
        </ol>
    </li>
</ul>
```

BROWSER PREVIEW — **Danh Sách Kết Hợp ul Và ol**

- **Chủ đề 1**
- **Chủ đề 2**
  1. Mục 2.1
  2. Mục 2.2

6. Kỹ Thuật Định Kiểu Danh Sách Bằng CSS Hiện Đại

Tùy chỉnh khoảng cách vị trí dấu đầu dòng với `list-style-position`:

Thuộc tính `list-style-position` kiểm soát dấu đầu dòng nằm bên ngoài (`outside`) hay bên trong (`inside`) khối nội dung của phần tử `<li>`.

[Bảng] So Sánh list-style-position inside Và outside

Giá trị	Đặc điểm & Trạng thái thụt lề khi xuống dòng
`outside`	Dấu đầu dòng nằm ngoài khối hộp `<li>`. Khi nội dung xuống dòng, văn bản sẽ tự động thụt lề thẳng hàng với dòng trên (Mặc định).
`inside`	Dấu đầu dòng nằm trong khối hộp `<li>`. Khi nội dung xuống dòng, dòng thứ 2 sẽ thò ra ngoài thẳng hàng ngay dưới dấu bullet.

Trang trí dấu đầu dòng bằng CSS Pseudo-element `::marker`:

CSS3 cung cấp con trỏ `::marker` giúp tùy chỉnh màu sắc, phông chữ, biểu tượng trực tiếp cho dấu đầu dòng mà không cần tạo thẻ phụ.

Định Kiểu Biểu Tượng Bullet Bằng CSS ::marker

```
<style>
  ul.custom-bullet li::marker {
    color: #e65100;
    font-size: 1.2em;
    content: ">> ";
  }
</style>

<ul class="custom-bullet">
  <li>Khởi động dự án Frontend</li>
  <li>Xây dựng cấu trúc HTML5 chuẩn SEO</li>
</ul>
```

BROWSER PREVIEW — **CSS ::marker – Custom Bullet Marker**

» **Khởi động dự án Frontend** HTML5

» **Xây dựng cấu trúc chuẩn SEO** Semantic

7. Các Ví Dụ Vận Dụng Thực Chiến Trong Thiết Kế Web

Ví dụ 1: Xây dựng Thanh Điều Hướng (Navigation Menu) đa cấp với `<ul>` & CSS Flexbox

Trong thực tế phát triển Web, thẻ `<ul>` được bọc trong thẻ `<nav>` là tiêu chuẩn vàng để xây dựng thanh Menu điều hướng.

Ứng Dụng ul Tạo Thanh Điều Hướng Header Nav

```
<nav class="navbar">
  <ul class="nav-menu">
    <li><a href="#home">Trang Chủ</a></li>
    <li><a href="#about">Giới Thiệu</a></li>
    <li class="dropdown">
      <a href="#services">Dịch Vụ ▼</a>
      <ul class="dropdown-menu">
        <li><a href="#web-dev">Thiết Kế Web</a></li>
        <li><a href="#seo">Tối Ưu SEO</a></li>
      </ul>
    </li>
    <li><a href="#contact">Liên Hệ</a></li>
  </ul>
</nav>
```

BROWSER PREVIEW — **Thực Chiến – Header Navigation Bar**



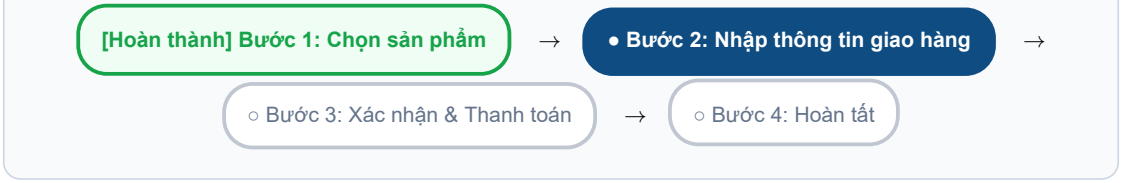
Ví dụ 2: Tạo Quy Trình Thanh Toán (Checkout Step Wizard) bằng thẻ `<ol>`

Thẻ `<ol>` thể hiện xuất sắc các quy trình nghiệp vụ có tính tuần tự bắt buộc như các bước mua hàng/thanh toán trực tuyến.

Ứng Dụng ol Làm Thanh Tiến Trình Các Bước Mua Hàng

```
<ol class="checkout-steps">
  <li class="completed">Bước 1: Chọn sản phẩm vào giỏ hàng</li>
  <li class="active">Bước 2: Nhập thông tin giao hàng</li>
  <li class="pending">Bước 3: Xác nhận & Thanh toán</li>
  <li class="pending">Bước 4: Hoàn tất đơn hàng</li>
</ol>
```

BROWSER PREVIEW — **Thực Chiến – Checkout Step Progress Wizard**



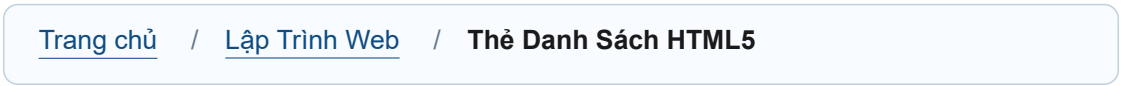
Ví dụ 3: Xây dựng Đường Dẫn Điều Hướng Trang (Breadcrumbs) chuẩn SEO & ARIA

Thẻ `<ol>` kết hợp với thuộc tính ARIA giúp công cụ tìm kiếm Google và các thiết bị đọc màn hình Screen Reader hiểu rõ cấu trúc vị trí phân cấp của trang web.

Xây Dựng Breadcrumbs Chuẩn Semantic & Accessibility

```
<nav aria-label="Breadcrumb">
  <ol class="breadcrumb">
    <li><a href="/">Trang chủ</a></li>
    <li><a href="/lap-trinh-web">Lập Trình Web</a></li>
    <li aria-current="page">Thẻ Danh Sách HTML5</li>
  </ol>
</nav>
```

BROWSER PREVIEW — **Thực Chiến – Breadcrumb Navigation SEO**



8. So Sánh Chi Tiết Giữa Thẻ <ul> và

**[Bảng] Bảng So Sánh Chi Tiết Thẻ <ul> và **

Tiêu chí	Thẻ (Unordered List)	Thẻ (Ordered List)
Dấu đầu dòng	Chấm tròn (•), ô vuông (■), hình tròn (○), hoặc không có.	Số (1, 2, 3...), chữ cái (A, B, C...), số La Mã (I, II, III...).
Mục đích sử dụng	Danh sách không cần thứ tự, ví dụ: danh sách mua sắm, danh sách kỹ năng.	Danh sách có thứ tự logic/bắt buộc, ví dụ: hướng dẫn các bước làm bài, bảng xếp hạng.
Thay đổi kiểu dáng	Thông qua CSS (list-style-type).	Thông qua thuộc tính HTML type (1, A, I...) hoặc CSS.
Vận dụng thực tế	Menu điều hướng (<nav>), danh sách bài viết, danh sách tag/icon.	Các bước thanh toán (Checkout Wizard), Breadcrumbs, bảng xếp hạng Top 10.

[Mẹo] Mẹo Khắc Phục Lỗi Mất Accessibility Trên Trình Duyệt Safari

Khi bạn sử dụng CSS `list-style: none;` để xóa dấu đầu dòng (ví dụ khi làm Menu nav), một số phiên bản trình duyệt VoiceOver / Safari trên iOS sẽ loại bỏ luôn ngữ nghĩa ”danh sách” của phần tử. Để khắc phục triệt để lỗi này và đảm bảo đạt chuẩn truy cập (**Web Accessibility**):

```
<ul class="nav-menu" role="list">
  <li><a href="#">Trang chủ</a></li>
</ul>
```

Thêm thuộc tính `role="list"` sẽ ép buộc Screen Reader đọc chính xác: *”Danh sách gồm N mục”*.

[Khái Niệm] Kết Luận — Thẻ <ul> Và <ol> Trong HTML

- Thẻ <ul> được sử dụng cho danh sách không có thứ tự, dấu đầu dòng có thể thay đổi bằng CSS.
- Thẻ <ol> được sử dụng cho danh sách có thứ tự, có thể thay đổi kiểu đánh số bằng thuộc tính type.
- Thẻ con trực tiếp duy nhất hợp lệ của <ul> và <ol> bắt buộc phải là thẻ <li>.
- Có thể lồng nhiều cấp danh sách để tạo bố cục có hệ thống hơn.
- Nên sử dụng CSS (list-style-type, ::marker) để tùy chỉnh danh sách thay vì thuộc tính HTML cũ.

3.4.8 i. Thẻ bảng (Table)

Dùng để trình bày dữ liệu dạng hàng và cột:

[Bảng] Bảng Dữ Liệu HTML Mẫu Khởi Tạo

Cột Tiêu Đề 1	Cột Tiêu Đề 2
Dữ liệu ô số 1	Dữ liệu ô số 2
Dữ liệu ô số 3	Dữ liệu ô số 4

3.5 Tổng Quan Sơ Lược Về Biểu Mẫu (HTML Forms Overview)

Biểu mẫu HTML (**HTML Form**) là thành phần cốt lõi trên trang web cho phép người dùng nhập và gửi dữ liệu lên máy chủ (đăng nhập, đăng ký, tìm kiếm, phản hồi, tải lên tệp tin...). Một biểu mẫu cơ bản được định nghĩa bởi cặp thẻ `<form>` đi kèm các ô điều khiển nhập liệu (`<input>`) và nút gửi dữ liệu (`<button>` hoặc `<input type="submit">`).

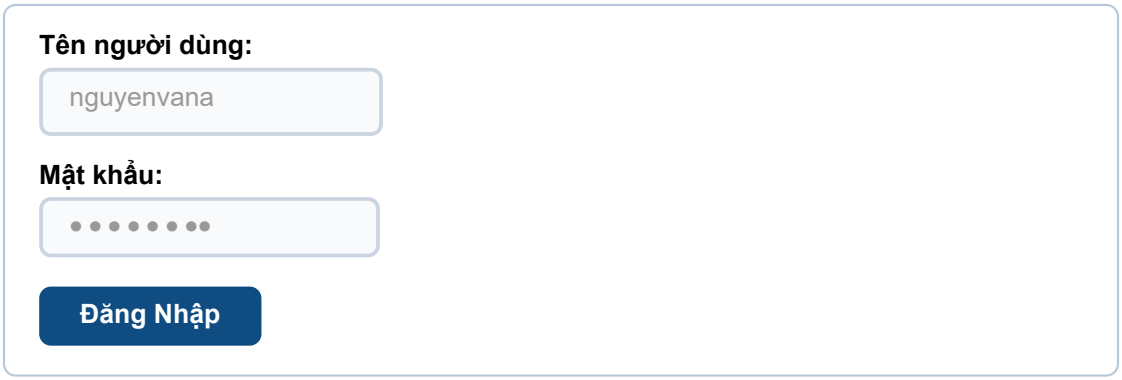
Mã Nguồn Khai Báo Biểu Mẫu HTML Đăng Nhập Cơ Bản

```
<form action="/login" method="POST">
  <label for="username">êTn ườngi ùdng:</label>
  <input type="text" id="username" name="username" required>

  <label for="password">ậMt ẩkhu:</label>
  <input type="password" id="password" name="password" required>

  <button type="submit">Đăng ậNhp</button>
</form>
```

BROWSER PREVIEW — **Kết Quả Hiển Thị: Biểu Mẫu Đăng Nhập Cơ Bản**



[Ghi Chú] Điều Hướng Chuyên Đề: Hệ Thống Biểu Mẫu Web Toàn Diện (HTML Forms Masterclass)

Để đảm bảo tính liên tục và chiều sâu sư phạm cao nhất, toàn bộ hệ sinh thái kiến thức về Biểu mẫu Web HTML5 được tổ chức trọn vẹn trong một chuyên đề lớn tại **Chương 5: Biểu Mẫu Web (HTML5 Forms) & Các Ô Nhập Liệu**, bao gồm hai phần lớn:

- **Phần 1: Nền Tảng Biểu Mẫu & Ô Nhập Liệu Cốt Lõi**: Giải phẫu toàn bộ 22+ loại `<input>`, cơ chế truyền nhận dữ liệu HTTP qua `GET` vs `POST`, chuyên sâu thẻ `<label>`, thuộc tính `name` và `value`, gom nhóm `<fieldset>` & `<legend>`, bộ chọn gợi ý `<datalist>`, thẻ `<textarea>`.
- **Phần 2: Chuyên Sâu Từng Ô Nhập Liệu, Định Dạng CSS & Trợ Năng A11y**: Chi tiết kỹ thuật từng ô điều khiển chuyên biệt, kỹ thuật tạo kiểu CSS thời thượng (`:placeholder-shown`, Floating Label, `:focus-within`) và tiêu chuẩn Trợ năng Web (Form Accessibility theo chuẩn W3C/WAI-ARIA).

3.6 HTML5 Và Các Tính Năng Mới Hiện Đại

HTML5 là phiên bản tiêu chuẩn mới nhất và mạnh mẽ nhất của HTML, bổ sung nhiều thẻ ngữ nghĩa và giao diện lập trình ứng dụng (API) hiện đại:

- **Thẻ ngữ nghĩa mới**: `<header>`, `<footer>`, `<section>`, `<article>`, `<nav>` giúp phân chia cấu trúc trang web rõ ràng, logic hơn.
- **Thẻ đa phương tiện**: `<audio>` và `<video>` cho phép nhúng trực tiếp âm thanh và video vào trang web mà không cần các plugin ngoài như Flash.
- **Hỗ trợ API hiện đại**: Web Storage (LocalStorage/SessionStorage), Geolocation (Xác định vị trí địa lý), Canvas & SVG (Vẽ đồ họa 2D/3D).

Ví dụ nhúng tệp video trực tiếp bằng HTML5:

Mã Nguồn Mẫu Nhúng Video Trực Tiếp Trong HTML5

```
<video controls width="600">
    <source src="video.mp4" type="video/mp4">
    Trình duyệt của bạn không hỗ trợ thẻ video.
</video>
```

3.7 HTML Kết Hợp Với CSS và JavaScript

Trong mô hình phát triển web hiện đại, HTML giữ vai trò cấu trúc khung xương. Để tạo nên một trang web hoàn chỉnh, hấp dẫn và tương tác cao, HTML phối hợp chặt chẽ với:

- **CSS**: Dùng để tạo kiểu dáng, màu sắc, phông chữ, khoảng cách và bố cục giao diện.
- **JavaScript**: Dùng để tạo tính năng động, xử lý logic và tương tác thời gian thực với người dùng.

Ví dụ nhúng trực tiếp mã CSS vào HTML bằng thẻ `<style>`:

Mã Nguồn Mẫu Nhúng CSS Nội Bộ (style)

```
<style>
    body { background-color: lightblue; }
    h1 { color: red; }
</style>
```

Ví dụ nhúng trực tiếp mã JavaScript vào HTML bằng thẻ `<script>`:

Mã Nguồn Mẫu Nhúng JavaScript Nội Bộ (script)

```
<script>
    alert("Chào mừng đến với học Lập Trình HTML!");
</script>
```

3.8 Phân Loại Phần Tử HTML: Block, Inline & Inline-Block Masterclass

Trong HTML, các phần tử (elements) khi hiển thị trên trình duyệt được phân chia thành hai loại mặc định chính:

1. **Block Elements (Phần tử khối)**: Tạo dựng bộ khung bố cục (Layout Structure).
2. **Inline Elements (Phần tử nội tuyến)**: Định dạng nội dung chi tiết bên trong văn bản (Text Formatting).
3. **Inline-Block Elements (Phần tử trung gian)**: Kết hợp tính chất hiển thị chung dòng của Inline và khả năng chỉnh kích thước của Block.

3.8.1 Block Elements (Phần Tử Khối)

Mô tả chi tiết:

- Luôn luôn bắt đầu trên một dòng mới trong luồng hiển thị (Document Flow) và độc chiếm trọn vẹn 100% chiều rộng một hàng.
- Chiếm toàn bộ chiều rộng (100% width) của phần tử cha, ngay cả khi nội dung bên trong rất ngắn (dù chỉ có một từ hay một ký tự).
- Cho phép thiết lập tự do kích thước chiều rộng (`width`), chiều cao (`height`), khoảng cách lề (`margin`) và đệm (`padding`) cả 4 hướng bằng CSS.
- Thường được sử dụng để phân chia khu vực, xây dựng bố cục trang web.
- Có thể chứa cả các phần tử khối (**Block Elements**) khác và các phần tử nội tuyến (**Inline Elements**).

Ví dụ mã nguồn phần tử khối:

Mã Nguồn Minh Họa Các Thẻ Block Elements

```
<div style="background: lightblue;">Thẻ div</div>
<p style="background: lightcoral;">Thẻ p</p>
<h1 style="background: lightgreen;">Thẻ h1</h1>
```

BROWSER PREVIEW — **Hiển Thị Thực Tế Thẻ Block trên Trình Duyệt**

Thẻ div – Chiếm 100% chiều rộng dòng 1

Thẻ p – Tự động xuống dòng mới, chiếm 100% chiều rộng dòng 2

Thẻ h1 – Tự động xuống dòng mới, chiếm 100% chiều rộng dòng 3

Ví dụ phần tử khối lồng nhau (Nested Block Elements):

Cấu Trúc Các Thẻ Block Lồng Nhau

```
<div>
    <h2>Tiêu đề bài viết</h2>
    <p>Đây là một đoạn văn bản nằm bên trong khối div.</p>
</div>
```

BROWSER PREVIEW — **Khối Block Lồng Nhau Chiếm Toàn Bộ Chiều Rộng Chứa**

Tiêu đề bài viết

Đây là một đoạn văn bản nằm bên trong khối div.

Danh sách tổng hợp các thẻ Block Elements phổ biến trong HTML5:

[Bảng] Bảng Tra Cứu Thẻ Block Elements Trong HTML5 (Grid 2 Cột Tiết Kiệm Trang)

Thẻ	Chức Năng & Ứng Dụng	Thẻ	Chức Năng & Ứng Dụng
`<div>`	Nhóm phần tử, phân chia vùng	`<p>`	Đoạn văn bản chính
`<h1>-<h6>`	Tiêu đề từ lớn đến nhỏ	`<ul>`	Danh sách không thứ tự
`<ol>`	Danh sách có thứ tự	`<li>`	Mục trong danh sách
`<table>`	Bảng dữ liệu	`<thead>`	Phần đầu tiêu đề bảng
`<tbody>`	Phần thân dữ liệu bảng	`<tfoot>`	Phần chân tổng kết bảng
`<tr>`	Hàng trong bảng	`<th>`	Ô tiêu đề cột
`<td>`	Ô dữ liệu bảng	`<form>`	Biểu mẫu thu thập nhập liệu
`<fieldset>`	Nhóm thành phần form	`<legend>`	Tiêu đề nhãn fieldset
`<section>`	Phân vùng theo chủ đề	`<article>`	Nội dung độc lập (blog)
`<aside>`	Thanh bên (sidebar)	`<header>`	Phần đầu trang/bài viết
`<footer>`	Phần chân trang/bài viết	`<nav>`	Thanh điều hướng Menu
`<main>`	Nội dung chính của trang	`<figure>`	Chứa ảnh/đồ thị độc lập
`<figcaption>`	Chú thích cho figure	`<blockquote>`	Khối trích dẫn dài
`<pre>`	Giữ nguyên định dạng	`<hr>`	Đường kẻ ngang phân cách
`<address>`	Địa chỉ tác giả/liên hệ		

3.8.2 Inline Elements (Phần Tử Nội Tuyến)

Mô tả chi tiết:

- **Không** bắt đầu trên một dòng mới; các phần tử hiển thị nối tiếp nhau trên cùng một hàng và **chỉ xuống dòng khi hết chiều ngang** (hết độ rộng của khung chứa) hoặc khi gặp thẻ ngắt dòng `<br>`.
- Chiếm kích thước chiều rộng (`width`) và chiều cao (`height`) vừa khít với nội dung bên trong nó; không thể thiết lập `width` và `height` bằng CSS (nếu đặt, trình duyệt sẽ tự động bỏ qua).
- Khoảng cách `margin` và `padding` chỉ có tác dụng theo phương ngang (trái – phải); khoảng cách theo phương đứng (trên – dưới) không đẩy các dòng văn bản xung quanh ra.
- Chủ yếu được sử dụng để định dạng, nhấn mạnh hoặc tạo liên kết cho một phần từ/cụm từ bên trong khối văn bản.
- **Không thể** chứa các phần tử khối (**Block Elements**); chỉ được chứa các phần tử nội tuyến khác hoặc văn bản thô (ngoại trừ ngoại lệ đặc biệt của thẻ `<a>` trong chuẩn HTML5).

Ví dụ mã nguồn phần tử nội tuyến:

Mã Nguồn Minh Họa Các Thẻ Inline Elements

```
<p>Đây là <span style="color: red;">văn bản màu đỏ</span> trong một đoạn văn.</p>
<b>Đây là một đoạn văn bản.</b> <b>Đây là một đoạn văn bản.</b>
```

BROWSER PREVIEW **Hiển Thị Các Phần Tử Inline Nằm Nối Tiếp Trên Cùng Một Dòng**

Đây là **văn bản màu đỏ** trong một đoạn văn. **Đây là một đoạn văn bản. Đây là một đoạn văn bản.**

Ví dụ phần tử nội tuyến trong dòng văn bản:

Định Kiểu Inline Nhấn Mạnh Từ Trong Văn Bản

```
<p>Học lập trình với <strong>HTML</strong> và <em>CSS</em> là rất quan trọng.</p>
```

BROWSER PREVIEW **Văn Bản Định Kiểu Bằng Thẻ strong Và em**

Học lập trình với **HTML** và *CSS* là rất quan trọng.

Danh sách tổng hợp các thẻ Inline Elements phổ biến trong HTML5:

[Bảng] Bảng Tra Cứu Thẻ Inline Elements Trong HTML5 (Grid 2 Cột Tiết Kiệm Trang)

Thẻ	Chức Năng & Ứng Dụng	Thẻ	Chức Năng & Ứng Dụng
`<span>`	Nhóm nội dung nhỏ định kiểu	`<a>`	Liên kết siêu văn bản
`<strong>`	Chữ in đậm nhấn mạnh	`<em>`	Chữ in nghiêng nhấn mạnh
`<b>`	Chữ in đậm (thị giác)	`<i>`	Chữ in nghiêng (thị giác)
`<u>`	Chữ gạch chân	`<small>`	Văn bản nhỏ hơn
`<mark>`	Tô sáng nền vàng	`<sub>`	In chỉ số dưới (H_2O)
`<sup>`	In chỉ số trên (x^2)	`<code>`	Mã nguồn (monospace)
`<kbd>`	Phím bàn phím (Ctrl+C)	`<samp>`	Mẫu kết quả phần mềm
`<var>`	Biến số công thức	`<time>`	Thời gian, ngày tháng
` `	Xuống dòng ngắt đoạn	`<wbr>`	Gợi ý điểm ngắt từ
`<img>`	Chèn hình ảnh	`<map>`	Bản đồ hình ảnh
`<area>`	Vùng nhấp chuột trong map	`<input>`	Trường nhập thông tin
`<select>`	Dropdown menu	`<option>`	Tùy chọn trong select
`<optgroup>`	Nhóm tùy chọn select	`<button>`	Nút bấm thao tác
`<label>`	Nhãn tiêu đề input	`<output>`	Kết quả tính toán Form
`<progress>`	Thanh tiến trình	`<meter>`	Thước đo định lượng
`<bdi>`	Cách ly hướng văn bản	`<bdo>`	Ghi đè hướng văn bản

3.8.3 Sự Khác Biệt Giữa Block Elements Và Inline Elements

[Bảng] So Sánh Chi Tiết Giữa Block Elements Và Inline Elements

Tiêu Chí So Sánh	Block Elements (Khối)	Inline Elements (Nội tuyến)
Bắt đầu dòng mới?	**Có** (Luôn luôn xuống dòng mới, độc chiếm 1 hàng)	**Không** (Nằm cùng dòng, **chỉ xuống dòng khi hết chiều ngang**)
Chiếm toàn bộ chiều rộng?	**Có** (100% chiều rộng phần tử cha)	**Không** (Chỉ vừa đúng dung lượng nội dung)
Chứa phần tử khối khác?	**Có** (Cho phép lồng khối)	**Không** (Chỉ chứa Inline hoặc văn bản)
Cấu hình Width / Height	**Tùy biến tự do** qua CSS	**Bị bỏ qua** bởi trình duyệt (ôm sát nội dung)
Tác động Margin/Padding	Đẩy khung cả 4 hướng (Top, Right, Bottom, Left)	Chỉ đẩy phương ngang (Left, Right); Top/Bottom không đẩy dòng
Ứng dụng chính	Xây dựng bố cục, phân vùng trang	Định dạng văn bản, tạo liên kết, gắn icon

Ví dụ mã nguồn so sánh trực quan:

Mã Nguồn So Sánh Sự Khác Biệt Giữa Div (Block) Và Span (Inline)

```
<!-- Block Element -->
<div style="background: lightblue;">Đây là div (Block)</div>

<!-- Inline Element -->
<p>Đây là <span style="background: yellow;">span (Inline)</span> trong một đoạn
    văn.</p>
```

BROWSER PREVIEW **Kết Quả Hiển Thị So Sánh Block Chiếm 100% Width Và Inline Ôm Sát Nội Dung**

Đây là div (Block) – Chiếm toàn bộ 100% chiều rộng chiều ngang

Đây là **span (Inline)** trong một đoạn văn (chỉ chiếm diện tích màu vàng đúng bằng chữ "span (Inline)").

3.8.4 Chuyên Sâu Thẻ Và : Quy Tắc Sử Dụng & Anti-Patterns

Thẻ và là hai phần tử container phi ngữ nghĩa (**Non-semantic Containers**) phổ biến nhất trong HTML. Việc hiểu rõ tính chất và thời điểm nên/không nên sử dụng hai thẻ này là kỹ năng cốt lõi của lập trình viên Frontend:

1. Thẻ – Khối Chứa Nội Dung (Block-Level Container)

Định nghĩa:

Thẻ (viết tắt của *"division"*) là một phần tử **Block-level**, thường dùng để nhóm các phần tử khác lại với nhau và giúp định dạng bố cục trang web dễ dàng hơn.

Ví Dụ Sử Dụng Thẻ Nhóm Các Phần Tử Khối

```
<div class="box">
    <h2>Tieu de bai viet</h2>
    <p>Day la noi dung cua bai viet.</p>
</div>
```

**[Ghi Chú] Lưu Ý Quan Trọng Về Thẻ **

- Thẻ **không có ý nghĩa ngữ nghĩa**, chủ yếu dùng để tạo nhóm phần tử và xây dựng bố cục layout.
- Thường kết hợp với CSS (class, id, Flexbox, Grid) để định dạng và căn chỉnh giao diện.
- Không nên lạm dụng thay cho các thẻ Semantic chuẩn mực như <section>, <article>, <header>, <footer>.

2. Thẻ – Nhóm Nội Dung Nhỏ Trong Dòng (Inline-Level Container)

Định nghĩa:

Thẻ là một phần tử **Inline-level**, dùng để bọc một phần nhỏ nội dung trong đoạn văn hoặc tiêu đề mà **không làm thay đổi cấu trúc dòng** của văn bản.

Ví Dụ Sử Dụng Thẻ Định Kiểu Nội Dòng

```
<p>San pham cua chung toi co mau <span style="color: red;">do</span>, <span
    style="color: blue;">xanh</span> va <span style="color: green;">luc</span>.<
    /p>
```

**[Ghi Chú] Lưu Ý Quan Trọng Về Thẻ **

- `<span>` **không làm thay đổi bố cục** dòng chảy văn bản, chỉ dùng để áp dụng CSS hoặc JavaScript vào một vị trí từ/cụm từ cụ thể.
- Không nên sử dụng `<span>` nếu không thực sự có nhu cầu định kiểu hoặc can thiệp script.

3. Bảng So Sánh Chi Tiết Giữa `<div>` Và `<span>`

**[Bảng] Bảng So Sánh Toàn Diện Thuộc Tính Kỹ Thuật Giữa Và **

Tiêu Chí	Thẻ (Khối – Block)	Thẻ (Nội tuyến – Inline)
Loại phần tử	Block-level (tự động xuống dòng, chiếm 100% chiều rộng).	Inline-level (nằm cùng dòng, chỉ chiếm vừa đúng kích thước nội dung).
Mục đích dùng	Nhóm nhiều phần tử phức tạp lại với nhau để phân chia vùng layout.	Nhóm một phần văn bản/ký tự nhỏ bên trong dòng văn bản.
Ví dụ ứng dụng	Chia cột, phân khung bố cục trang web (Container, Sidebar).	Định dạng màu sắc, phông chữ cho một từ/cụm từ cụ thể.
CSS hay dùng	`width`, `height`, `margin`, `padding`, `display: block`.	`color`, `font-size`, `font-weight`, `display: inline`.

4. Quy Tắc Ra Quyết Định Khi Nào NÊN / KHÔNG NÊN Dùng `<div>`

Khi NÊN dùng :

- Khi không có thẻ HTML Semantic phù hợp để diễn đạt nội dung.
- Khi cần tạo nhóm nhiều phần tử lại với nhau để áp dụng CSS layout hoặc JavaScript event handler.
- Khi dùng để bố cục trang web (ví dụ: tạo wrapper container, chia cột Flexbox/Grid).

**Ví Dụ Đúng Bố Cục Trang Web Dùng **

```
<!-- [DUNG]: Dung div lam container wrapper bo cuc trang web -->
<div class="container">
    <div class="header">Day la tieu de</div>
    <div class="content">Noi dung chinh</div>
    <div class="footer">Chan trang</div>
</div>
```

Khi KHÔNG NÊN dùng :

- Khi đã có thẻ Semantic chuẩn mực phù hợp để thay thế (`<header>`, `<nav>`, `<main>`, `<footer>`, `<article>`, `<section>`).

- Khi chỉ cần nhóm văn bản hoặc nội dung nhỏ nội dòng (nên dùng `<span>` thay vì `<div>`).

Ví Dụ Khắc Phục Lỗi Dùng Thay Cho Thẻ Semantic

```
<!-- [LOI SAI]: Dung div thua khi da co the Semantic chuan -->
<div class="header">
    <h1>Tieu de</h1>
</div>

<!-- [CACH TOT HON]: Dung the <header> truc tiep chuan ngu nghia -->
<header>
    <h1>Tieu de</h1>
</header>
```

5. Quy Tắc Ra Quyết Định Khi Nào NÊN / KHÔNG NÊN Dùng `<span>`

Khi NÊN dùng thay vì :

- Khi chỉ cần bọc một phần nhỏ bên trong dòng văn bản.
- Khi không muốn thay đổi cấu trúc dòng chảy bố cục (vì `<div>` là Block-level sẽ ép xuống dòng mới, còn `<span>` là Inline-level giữ nguyên dòng).

Ví Dụ Đúng Sử Dụng Thẻ Bọc Văn Bản Nội Dòng

```
<p>Chao mung ban den voi <span class="highlight">HTML5</span>!</p>
<p>Hom nay troi <span style="color: red;">rat dep</span>!</p>
```

Khi NÊN dùng :

- Khi cần bọc một từ/cụm từ nhỏ trong văn bản để áp dụng CSS style hoặc xử lý JavaScript.
- Khi không muốn ảnh hưởng đến bố cục chung của trang web.
- Khi không có thẻ HTML ngữ nghĩa nội dòng nào khác phù hợp hơn.

Khi KHÔNG NÊN dùng :

- Khi có thẻ HTML ngữ nghĩa khác phù hợp hơn (như `<strong>`, `<em>`, `<mark>`, `<code>`, `<time>`).
- Khi cần bọc nhiều phần tử khối hoặc thay đổi bố cục trang → Phải sử dụng `<div>` thay vì `<span>`.

**Ví Dụ Khắc Phục Lỗi Sai Đặt Block Elements Vào Trong Thẻ **

```
<!-- [LOI SAI]: <span> la the Inline, khong duoc chua Block -->
<span>
    <h1>Tieu de lon</h1>
    <p>Noi dung trang web</p>
</span>

<!-- [CACH TOT HON]: Dung <div> lam khoi boc Block Elements -->
<div>
    <h1>Tieu de lon</h1>
    <p>Noi dung trang web</p>
</div>
```

**[Khái Niệm] Tổng Kết Nguyên Tắc Vàng Sử Dụng Thẻ Và **

Đối với thẻ :

- Nếu có thẻ Semantic phù hợp (`<header>`, `<nav>`, `<main>`, `<article>`), hãy dùng thẻ đó thay vì `<div>`.
- Nếu cần phân chia bố cục trang web hoặc gom nhóm các phần tử để tạo khung CSS, dùng `<div>`.
- Nếu chỉ cần bọc nội dung nhỏ trong dòng, hãy dùng `<span>`.
- **Cảnh báo**: Tuyệt đối không lạm dụng `<div>` quá mức để tránh hiện tượng **"Div Soup"** (trang web chứa quá nhiều `<div>` không cần thiết, khiến mã nguồn khó hiểu, rối rắm và không tối ưu SEO).

Đối với thẻ :

- Dùng `<span>` khi cần định kiểu hoặc can thiệp nội dung nhỏ trong dòng (không ảnh hưởng tới dòng chảy bố cục).
- Dùng `<div>` khi cần nhóm nhiều phần tử hoặc thay đổi bố cục khối của trang web.
- Tuyệt đối không dùng `<span>` để chứa các thẻ Block-level như `<h1>-<h6>`, `<p>`, `<div>`.
- Không lạm dụng `<span>`; nếu có thẻ ngữ nghĩa phù hợp hơn (như `<strong>`, `<em>`, `<mark>`), hãy ưu tiên sử dụng thẻ đó để mã nguồn chuẩn SEO, dễ đọc và dễ bảo trì hơn.

3.8.5 Inline-Block: Trung Gian Giữa Block Và Inline

Mô tả chi tiết:

Trong thiết kế giao diện hiện đại, thuộc tính CSS `display: inline-block;` tạo ra một phần tử mang đặc tính lai hoàn hảo:

- **Giống Inline**: Không tự động xuống dòng mới; các phần tử đứng xếp hàng ngang cạnh nhau trên cùng một dòng và **chỉ xuống dòng khi hết chiều ngang** của phần tử cha.
- **Giống Block**: Tự do thiết lập kích thước chiều rộng (`width`), chiều cao (`height`), khoảng cách lề (`margin`) và đệm (`padding`) cả 4 hướng bằng CSS.

Ví dụ mã nguồn sử dụng `display: inline-block;`:

Mã Nguồn Định Kiểu Khối Xếp Hàng Ngang Với display: inline-block

```
<style>
    .box {
        display: inline-block;
        width: 100px;
        height: 50px;
        background-color: lightcoral;
        margin: 5px;
        text-align: center;
        line-height: 50px;
        color: white;
        font-weight: bold;
    }
</style>

<div class="box">A</div>
<div class="box">B</div>
<div class="box">C</div>
```

BROWSER PREVIEW Hiển Thị Các Khối Box Inline-Block Có Kích Thước Custom Nằm Cùng Dòng



3.8.6 Quy Tắc Lồng Thẻ Chuẩn HTML5: Ranh Giới Block – Inline & Các Ngoại Lệ

[Cảnh Báo] Các Quy Tắc Bắt Buộc Khi Sử Dụng Block Và Inline Elements

- **Luôn tôn trọng quy tắc lồng phần tử**: Không bao giờ đặt thẻ Block Element vào bên trong thẻ Inline Element (ngoại trừ ngoại lệ đặc biệt của thẻ `<a>` trong chuẩn HTML5).
- **Thẻ không chứa Block**: Các thẻ như `<p>`, `<h1>-<h6>`, `<li>` là Block Elements nhưng **không được chứa** phần tử khối khác bên trong.
- **Cảnh báo lỗi hiển thị**: Nếu đặt `<div>` bên trong `<span>`, trình duyệt sẽ cố gắng tự động sửa lỗi bằng cách đóng thẻ `<span>` sớm, làm hỏng toàn bộ cấu trúc cây DOM và gây lỗi vỡ giao diện CSS.

[MINH HỌA TRỰC QUAN] BỐ CỤC 4 QUY TẮC VÀNG LỒNG GHÉP: BLOCK VÀ INLINE

```
<!-- Sai -->
<span>
  <p>Nội dung...</p>
</span>
```

Phần tử kiểu inline không được chứa phần tử kiểu block.

```
<!-- Đúng -->
<span>
  <b>Nội dung...</b>
</span>
```

Phần tử kiểu inline chỉ chứa các phần tử kiểu inline, dữ liệu.

```
<!-- Đúng -->
<p>
  <b>Nội dung...</b>
</p>
```

Phần tử kiểu block thì được chứa các phần tử kiểu block, kiểu inline, dữ liệu.

```
<!-- Đúng (Ngoại Lệ) -->
<a href="">
  <p>Nội dung...</p>
</a>
```

Ngoại lệ: Thẻ `<a>` (kiểu inline) có thể chứa phần tử kiểu block.

Ví dụ minh họa lỗi sai phổ biến đặt Block vào Inline:

Lỗi Sai Cấu Trúc Đặt div (Block) Bên Trong span (Inline)

```
<!-- [Lỗi Sai]: Không thể đặt <div> (Block) bên trong <span> (Inline) -->
<span>
    <div>Block không thể nằm trong Inline</div>
</span>

<!-- [Hoàn Thành]: Cách viết chuẩn ngữ nghĩa: -->
<div>
    <span>Đây là nội dung hợp lệ bên trong Block</span>
</div>
```

[Ghi Chú] Sự Thay Đổi Quy Tắc Của Thẻ Siêu Liên Kết <a> Từ HTML4 Sang HTML5

- **Trong chuẩn HTML4 cũ**: Thẻ `<a>` là một Inline Element nên chỉ có thể chứa các Inline Elements khác (như `<span>`, `<strong>`, `<img>`). Đặt `<div>` hay `<h2>` vào trong `<a>` bị coi là lỗi vi phạm cú pháp W3C.
- **Trong chuẩn HTML5 hiện đại**: Tiêu chuẩn W3C đã chính thức **cho phép thẻ `<a>` bọc bên ngoài các phần tử khối** (**Block Elements**) như `<div>`, `<article>`, `<section>`, `<h2>`, giúp các lập trình viên dễ dàng biến toàn bộ một khối Card/Bài viết thành một vùng có thể nhấp chuột (Clickable Card).
- **Điều kiện duy nhất**: Không được lồng thẻ `<a>` này bên trong một thẻ `<a>` khác (Interactive Content Nesting Violation).

Ví dụ mã nguồn bọc khối Card bằng thẻ `<a>` chuẩn HTML5:

Ứng Dụng HTML5 Cho Phép Thẻ a Bọc Khối Block Card

```
<!-- [Lỗi Sai]: Lỗi sai cũ HTML4 (Tránh lồng nhầm thẻ inline thuần): -->
<a href="#">
    <div>Không nên dùng sai cách cũ</div>
</a>

<!-- [Hoàn Thành]: Cách bọc chuẩn Card Clickable trong HTML5: -->
<a href="/bai-viet/1" class="card-link">
    <div class="product-card">
        <h2>Sản phẩm Laptop ASUS ROG</h2>
        <p>Giá bán: 35.000.000đ -- Nhấp để xem chi tiết</p>
    </div>
</a>
```

3.8.7 Kỹ Thuật Chuyển Đổi Kiểu Hiển Thị Bằng CSS (display Property)

Ngoài kiểu hiển thị mặc định của trình duyệt, kỹ sư Web có thể chủ động thay đổi hoàn toàn hành vi hiển thị của bất kỳ phần tử nào thông qua thuộc tính CSS `display`:

1. Chuyển Block thành Inline (`display: inline;`):

Chuyển Thẻ div Thành Inline Bằng CSS

```
<style>
  .block-to-inline { display: inline; }
</style>

<div class="block-to-inline">Bây giờ div này sẽ hiển thị nối tiếp như
    inline</div>
```

2. Chuyển Inline thành Block (`display: block;`):

Chuyển Thẻ span Thành Block Bằng CSS

```
<style>
  .inline-to-block { display: block; }
</style>

<span class="inline-to-block">Bây giờ span này sẽ tự động xuống dòng mới như
    block</span>
```

3. Kết hợp với Inline-Block (`display: inline-block;`):

Định Kiểu Thẻ span Thành Inline-Block Có Kích Thước Kích Thước

```
<style>
  .custom-badge {
      display: inline-block;
      width: 200px;
      height: 45px;
      background-color: #0284c7;
      color: white;
      text-align: center;
      line-height: 45px;
      border-radius: 6px;
  }
</style>

<span class="custom-badge">Tôi có thể chỉnh width & height!</span>
```

3.8.8 Chuyên Sâu Layout Engines: Block Formatting Context (BFC) & IFC

Để hiểu sâu sắc cơ chế dựng hình (**Rendering**) của trình duyệt Web (như Chrome Blink, Firefox Gecko, Safari WebKit), kỹ sư Frontend cần nắm vững hai môi trường tính toán bố cục cốt lõi:

- **Block Formatting Context (BFC)**: Môi trường dựng hình độc lập dành cho các phần tử khối. Bên trong một BFC, các phần tử con được xếp chồng dọc từ trên xuống dưới. BFC có nhiệm vụ tự chứa các thẻ con có thuộc tính `float`, ngăn hiện tượng tràn lề và ngăn chặn sự ảnh hưởng của các phần tử bên ngoài.
- **Inline Formatting Context (IFC)**: Môi trường dựng hình nội tuyến. Các phần tử xếp ngang từ trái sang phải trên cùng một dòng gọi là đường cơ sở (**Baseline**). Khi hết chiều rộng container, các phần tử sẽ tự xuống dòng tạo thành các dòng chữ (**Line Boxes**).

Kỹ thuật kích hoạt BFC chuẩn hiện đại:

Trong chuẩn CSS3 hiện đại, cách chuẩn nhất để tạo một BFC mới mà không gây ra tác dụng phụ là sử dụng thuộc tính:

Kích Hoạt Block Formatting Context Bằng display: flow-root

```
<style>
  .container {
      display: flow-root; /* Cách tạo BFC chuẩn W3C hiện đại */
      background-color: #f8fafc;
      border: 1px solid #cbd5e1;
  }
</style>
```

3.8.9 Hiện Tượng Margin Collapsing (Gộp Lề Đứng) & Kỹ Thuật Xử Lý

Mô tả hiện tượng:

Khi hai phần tử khối (**Block Elements**) xếp chồng chiều dọc cạnh nhau trong luồng hiển thị thường (**Normal Flow**), khoảng cách lề trên (`margin-top`) và lề dưới (`margin-bottom`) của chúng **không cộng dồn lại với nhau**. Thay vào đó, trình duyệt thực hiện gộp lề: khoảng cách thực tế giữa hai khối sẽ bằng giá trị `margin` lớn nhất giữa hai phần tử.

Mã Nguồn Minh Họa Hiện Tượng Gộp Lề Đứng Margin Collapsing

```
<style>
  .box-top {
      margin-bottom: 30px; /* Lề dưới 30px */
      background-color: #fca5a5;
  }
  .box-bottom {
      margin-top: 20px; /* Lề trên 20px */
      background-color: #93c5fd;
  }
</style>

<div class="box-top">Khối Trên (Margin Bottom 30px)</div>
<div class="box-bottom">Khối Dưới (Margin Top 20px)</div>
<!-- Khoảng cách thực tế giữa 2 khối chỉ là 30px, KHÔNG PHẢI 50px! -->
```

BROWSER PREVIEW **Minh Họa Hiện Tượng Gộp Lề (Khoảng Cách Giữa 2 Khối = 30px Khỏi Bị Cộng Dồn)**

Khối Trên (margin-bottom: 30px)

Khối Dưới (margin-top: 20px) – *Khoảng cách giữa 2 khối thực tế chỉ là 30px (lấy max)!*

[Mẹo] 3 Giải Pháp Triệt Tiêu Gộp Lề Margin Collapsing Trong Production

1. **Sử dụng Padding / Border**: Thêm `padding: 1px;` hoặc `border: 1px solid transparent;` vào phần tử cha để ngăn sự tiếp xúc trực tiếp lề đứng giữa cha và con.
2. **Tạo BFC mới (Khuyên dùng chuẩn W3C)**: Thêm `display: flow-root;` vào phần tử chứa để tạo một Block Formatting Context hoàn toàn độc lập.
3. **Chuyển đổi kiểu hiển thị**: Chuyển đổi phần tử sang `display: inline-block;` hoặc `display: flex;` để loại bỏ luồng hiển thị thường (**Normal Flow**).

3.8.10 Giải Mã Khoảng Trắng 4px (Whitespace Gap) Của Inline-Block & Fix Triệt Để

Nguyên nhân cốt lõi:

Khi đặt các phần tử `display: inline-block;` xếp hàng ngang, trình duyệt biên dịch các ký tự xuống dòng (**Newline**) hoặc dấu cách (**Space**) giữa các thẻ HTML như một ký tự khoảng trắng văn bản. Ký tự này chiếm khoảng ≈ 4px chiều rộng (phụ thuộc vào `font-size`).

4 Kỹ thuật khắc phục trong thực tế dự án:

[Bảng] Bảng Tổng Hợp 4 Phương Pháp Khắc Phục Lỗi Khoảng Trắng Inline-Block

Phương Pháp	Cách Thực Hiện Trong Mã Nguồn	Đánh Giá Thực Tế
`1. Viết liền HTML`	`</div><div>` không xuống dòng	Khó đọc, dễ bị linter format lại.
`2. Thêm HTML Comment`	`</div><!-- --><div>`	Làm rối mã nguồn HTML.
`3. CSS font-size: 0`	Thêm `font-size: 0;` ở thẻ cha	Rất hiệu quả, nhưng cần reset lại font-size cho thẻ con.
`4. CSS Flexbox (Khuyên dùng)`	Sử dụng `display: flex;` thay thế	**Chuẩn hiện đại tốt nhất**, triệt tiêu hoàn toàn lỗi 4px.

3.8.11 Bảng Tra Cứu So Sánh 5 Giá Trị display Cốt Lõi Trong CSS3

[Bảng] So Sánh Tổng Hợp 5 Kiểu Hiển Thị display Trong CSS3

Giá Trị display	Xuống Dòng Mới?	Nhận Width/Height?	Mục Đích Sử Dụng Main
`block`	**Có** (100% width)	**Có**	Phân chia vùng bố cục lớn (div, section, p).
`inline`	**Không** (Cùng dòng)	**Không**	Định kiểu từ/cụm từ trong văn bản (span, strong).
`inline-block`	**Không** (Cùng dòng)	**Có**	Tạo các thẻ Badge, nút bấm có size custom.
`flex`	Tuỳ thuộc Flex-direction	**Có**	Xây dựng bố cục linh hoạt 1 chiều (Row/Column).
`grid`	Khung chứa Block	**Có**	Xây dựng hệ thống lưới 2 chiều (Hàng & Cột).

[Khái Niệm] Tổng Kết Cốt Lõi — Block, Inline Và Inline-Block Elements

- **Block Elements**: Luôn xuống dòng mới, chiếm 100% chiều rộng. Dùng làm khung bố cục chính. Có thể chứa Block và Inline (trừ `<p>`, `<h1>-<h6>`).
- **Inline Elements**: Không xuống dòng, ôm vừa sát kích thước nội dung. Dùng trang trí văn bản. Không được chứa phần tử khối.
- **Inline-Block**: Nằm cùng dòng như Inline nhưng cho phép thiết lập `width`, `height`, `margin`, `padding` như Block.
- **HTML5 Update**: Thẻ `<a>` được phép bọc khối Block Elements để tạo các ô Card nhấp chuột tiện lợi.
- **Layout Context**: Sử dụng `display: flow-root;` để tạo Block Formatting Context (BFC) mới, giải quyết Margin Collapsing và Float Overflow.

3.9 Thuộc Tính id Và class Trong HTML Masterclass

Trong lập trình Web, `id` và `class` là hai thuộc tính toàn cục (**Global Attributes**) quan trọng bậc nhất được sử dụng để định danh (**Identify**) và định kiểu (**Style**) các phần tử HTML bằng CSS, cũng như thao tác xử lý logic tương tác bằng JavaScript. Dù cùng dùng để gán nhãn cho phần tử, chúng sở hữu những nguyên lý hoạt động, mức độ ưu tiên và phạm vi ứng dụng hoàn toàn khác biệt.

3.9.1 Thuộc Tính id — Định Danh Duy Nhất Cho Một Phần Tử

Định nghĩa & Nguyên lý hoạt động:

Thuộc tính `id` dùng để xác định **duy nhất** một phần tử trên toàn bộ trang HTML. Về mặt ngữ nghĩa và tiêu chuẩn W3C:

- Một giá trị `id` **không được phép lặp lại** trên cùng một trang HTML (mang tính duy nhất như số Căn cước công dân / Mã số sinh viên).
- Dùng để xác định đích danh phần tử cụ thể khi viết CSS hoặc lập trình JavaScript.
- Được truy xuất với tốc độ tối ưu bằng phương thức `document.getElementById()` trong JavaScript.

1. Cách sử dụng id trong cú pháp HTML:

Khai Báo Thuộc Tính id Trong HTML

```html
<h1 id="tieude">Tiêu đề chính</h1>
<p id="mo-ta">Đây là đoạn văn có ID duy nhất.</p>
```

2. Cách sử dụng id trong CSS (Sử dụng ký tự #):

Trong CSS, để trỏ tới phần tử có thuộc tính `id`, ta sử dụng ký tự `#` phía trước tên ID:

Định Kiểu CSS Cho Phần Tử Qua id

```css
#tieude {
    color: blue;
    font-size: 24px;
}
#mo-ta {
    font-style: italic;
}
```

3. Cách truy xuất id trong JavaScript:

Truy Vấn Và Thao Tác DOM Với document.getElementById

```javascript
document.getElementById("tieude").innerHTML = "Tiêu đề đã thay đổi!";
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt Khi Định Kiểu & Thao Tác JavaScript Qua id**

Tiêu đề đã thay đổi!

Đây là đoạn văn có ID duy nhất.

[Cảnh Báo] Các Quy Tắc Bắt Buộc Khi Sử Dụng id

- **Tính duy nhất (Uniqueness)**: Mỗi `id` phải là duy nhất, tuyệt đối không trùng lặp trong cùng một tệp HTML.
- **Phạm vi áp dụng**: Chỉ nên sử dụng `id` khi cần xác định đúng một phần tử cụ thể duy nhất (như Header chính, Form đăng nhập, Navigation container).
- **Tránh lạm dụng CSS**: Không nên lạm dụng `id` để định kiểu CSS vì độ ưu tiên cao gây khó ghi đè và không thể tái sử dụng mã nguồn.

3.9.2 Thuộc Tính class — Nhóm Các Phần Tử Có Chung Thuộc Tính

Định nghĩa & Nguyên lý hoạt động:

Thuộc tính `class` được sử dụng để nhóm nhiều phần tử HTML lại với nhau nhằm chia sẻ chung kiểu hiển thị hoặc hành vi:

- Thuộc tính `class` có thể dùng chung cho **nhiều phần tử khác nhau** trên cùng một trang.
- **Một phần tử có thể chứa nhiều class cùng lúc**, ngăn cách nhau bởi dấu khoảng trắng (Space).
- Là nền tảng cốt lõi để tái sử dụng mã nguồn CSS (Reusable CSS Modules).

1. Cách sử dụng class trong HTML:

Gán Thuộc Tính class Cho Nhiều Phần Tử

```html
<p class="mau-do">Đây là một đoạn văn màu đỏ.</p>
<p class="mau-do">Đây cũng là một đoạn văn màu đỏ.</p>
```

2. Cách sử dụng class trong CSS (Sử dụng dấu chấm .):

Sau khi chuyển hướng sang CSS, ta dùng ký tự dấu chấm `.` trước giá trị của class để thay đổi style của tất cả các phần tử có cùng class đó cùng một lúc:

Định Kiểu CSS Cho Nhóm Phần Tử Qua class

```css
.mau-do {
    color: red;
    font-weight: bold;
}
```

3. Cách truy xuất class trong JavaScript:

Truy Vấn Danh Sách Phần Tử Qua getElementsByClassName

```
let elements = document.getElementsByClassName("mau-do");
for (let i = 0; i < elements.length; i++) {
    elements[i].style.textDecoration = "underline";
}
```

BROWSER PREVIEW Kết Quả Hiển Thị Trình Duyệt Khi Dùng class Cho Nhiều Phần Tử & Thao Tác JS Loop

Đây là một đoạn văn màu đỏ.

Đây cũng là một đoạn văn màu đỏ.

4. Ví dụ một phần tử sở hữu nhiều class cùng lúc:

Một phần tử HTML có thể kết hợp nhiều class cách nhau bằng dấu cách để tích hợp nhiều tập hợp style:

Kết Hợp Nhiều Class Trực Tiếp Trên Một Thẻ HTML

```
<p class="mau-do in-dam">Đoạn văn này vừa có màu đỏ vừa in đậm.</p>
```

Định Nghĩa Các Class Tách Biệt Để Tái Sử Dụng

```
.mau-do { color: red; }
.in-dam { font-weight: bold; }
```

BROWSER PREVIEW Kết Quả Hiển Thị Trình Duyệt Khi Tích Hợp Đa Class (.mau-do kết hợp .in-dam)

Đoạn văn này vừa có màu đỏ vừa in đậm.

[Mẹo] Lưu Ý Vàng Khi Sử Dụng class

- **Kết hợp đa class**: Một phần tử có thể khai báo nhiều class, ngăn cách bởi dấu cách (`class="btn btn-primary active"`).
- **Tái sử dụng tối đa**: Áp dụng cùng 1 class cho nhiều phần tử giúp mã CSS cực kỳ ngắn gọn và dễ bảo trì.

3.9.3 Bảng So Sánh Chi Tiết Giữa id Và class

[Bảng] Bảng Tra Cứu So Sánh Chi Tiết Giữa Thuộc Tính id Và class

Tiêu Chí So Sánh	Thuộc Tính id	Thuộc Tính class
Mục đích	Định danh duy nhất cho một phần tử	Dùng chung kiểu dáng cho nhiều phần tử
Số lần sử dụng	**Chỉ được dùng 1 lần** duy nhất trên trang	**Có thể dùng nhiều lần** cho vô số phần tử
Cách gọi trong CSS	Ký tự `#id` (Ví dụ: `#tieu-de`)	Ký tự `.class` (Ví dụ: `.noi-dung`)
Cách gọi trong JS	`getElementById()`	`getElementsByClassName()`
Tính tái sử dụng	Không linh hoạt, khó tái sử dụng CSS	**Cực kỳ dễ dàng tái sử dụng** cho nhiều phần tử
Độ ưu tiên CSS	**Độ ưu tiên rất cao** (Specificity Score = 100)	Độ ưu tiên thấp hơn id (Specificity Score = 10)

3.9.4 Khi Nào Nên Dùng id Và Khi Nào Nên Dùng class?

Trường Hợp Nên Dùng id:

- Cần xác định một phần tử độc nhất vô nhị trên trang (Header, Main Footer, Search Form).
- Muốn liên kết tương tác với JavaScript thông qua `getElementById()`.
- Cần tạo điểm nhảy liên kết nội bộ trang (**Anchor Link**, ví dụ: `<a href="#phan-3">Nav đến Phần 3</a>`).

Trường Hợp Nên Dùng class:

- Muốn tạo cùng một kiểu dáng giao diện cho nhiều phần tử (Button, Card, Product Item, Badge).
- Cần thiết kế giao diện theo mô hình Component, dễ dàng bảo trì và mở rộng CSS.
- Dùng JavaScript để thao tác mảng tập hợp nhiều phần tử cùng loại.

[Khái Niệm] Tóm Tắt Nhanh Quy Tắc id vs class

- **Dùng id** cho những phần tử đặc biệt, duy nhất trên trang web.
- **Dùng class** cho những phần tử có chung kiểu dáng để tái sử dụng dễ dàng.

3.9.5 Ví Dụ Thực Tế Kết Hợp id Và class Trong Mẫu Trang HTML5

Trang Web Hoàn Chỉnh Kết Hợp id Và class Trong HTML5

```
<!DOCTYPE html>
<html lang="vi">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Ví dụ về id và class</title>
    <style>
        /* CSS cho ID duy nhất */
        #tieu-de {
            font-size: 24px;
            color: blue;
            text-align: center;
        }
        /* CSS cho Class tái sử dụng */
        .noidung {
            font-size: 18px;
            color: green;
        }
        .in-dam {
            font-weight: bold;
        }
    </style>
</head>
<body>
    <!-- Dùng ID cho phần tử duy nhất trên trang -->
    <h1 id="tieu-de">Đây là tiêu đề chính</h1>

    <!-- Dùng Class cho các đoạn văn bản dùng chung style -->
    <p class="noidung">Đây là đoạn văn bản 1.</p>
    <p class="noidung in-dam">Đây là đoạn văn bản 2 (in đậm).</p>
</body>
</html>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Thực Tế Trình Duyệt Khi Kết Hợp id Và class**

Đây là tiêu đề chính

Đây là đoạn văn bản 1.

Đây là đoạn văn bản 2 (in đậm).

3.9.6 Quy Tắc Đặt Tên & Lưu Ý Quan Trọng Tránh Lỗi Thực Tế

Để tránh lỗi cú pháp và viết mã HTML/CSS/JavaScript sạch chuẩn Clean Code, kĩ sư web cần tuân thủ các quy tắc bắt buộc sau:

[Cảnh Báo] Quy Tắc Bắt Buộc Khi Đặt Tên id Và class Trong HTML

- **1. Ký tự hợp lệ**: Chỉ được chứa chữ cái không dấu (`a-z`, `A-Z`), chữ số (`0-9`), dấu gạch ngang (`-`) hoặc gạch dưới (`_`).
- **2. TUYỆT ĐỐI KHÔNG bắt đầu bằng số**: Đặt tên như `id="1tieu-de"` hoặc `class="1box"` là **SAI CÚ PHÁP**, khiến CSS và JS hoàn toàn không thể truy vấn phần tử. *(Định dạng đúng: `id="tieu-de-1"`, `class="box-1"`).*
- **3. TUYỆT ĐỐI KHÔNG chứa khoảng trắng**: Tên id/class không được chứa dấu cách. *(Lưu ý: Khoảng trắng trong thuộc tính class sẽ bị trình duyệt hiểu nhầm thành khai báo 2 class riêng biệt).*

2. Cảnh báo sai lầm trùng lặp id trong HTML & JavaScript:

SAI LẦM: Khai Báo Trùng id Trên Nhiều Thẻ HTML

```
<!-- [Lỗi Sai]: Trùng ID trên 2 phần tử khác nhau -->
<p id="mo-ta">Đoạn 1</p>
<p id="mo-ta">Đoạn 2</p> <!-- Lỗi vi phạm W3C -->
```

Hậu Quả Trong JavaScript Khi Trùng id

```
document.getElementById("mo-ta").innerHTML = "Nội dung mới";
// LỖI: JavaScript CHỈ THAY ĐỔI đoạn 1, đoạn 2 hoàn toàn BỊ BỎ BỎ QUA!
```

BROWSER PREVIEW **Kết Quả Trực Quan Lỗi Trùng ID: JavaScript Chỉ Thay Đổi Phần Tử Đầu Tiên!**

Nội dung mới (Đã được JS thay đổi)

Đoạn 2 (Giữ nguyên, BỊ BỎ QUA do trùng ID!)

Cách khắc phục chuẩn ngữ nghĩa:

Cách 1: Đặt Tên ID Khác Nhau Nếu Là 2 Phần Tử Độc Lập

```
<p id="mo-ta-1">Đoạn 1</p>
<p id="mo-ta-2">Đoạn 2</p>
```

Cách 2 (Khuyên Dùng): Chuyển Sang Dùng Class Để Áp Dụng Cho Nhiều Phần Tử

```
<p class="mo-ta">Đoạn 1</p>
<p class="mo-ta">Đoạn 2</p>
```

3. Hạn chế dùng id khi viết CSS (Lý do độ ưu tiên Specificity Score):

Trong CSS, `#id` có điểm độ ưu tiên cao tới **100**, trong khi `.class` chỉ là **10**. Nếu định kiểu bằng ID, sau này bạn sẽ rất khó ghi đè CSS nếu muốn tùy biến giao diện.

Ưu Tiên Sử Dụng Class Khi Viết CSS

```
/* [Không Nên]: Dùng ID cho CSS gây khó ghi đè */
#tieu-de { color: red; }

/* [Khuyên Dùng]: Dùng Class cho CSS linh hoạt */
.tieu-de { color: red; }
```

3.9.7 Tổng Kết Ma Trận Ra Quyết Định Khi Nào Dùng id Và class

[Bảng] Ma Trận Quyết Định Sử Dụng id Và class Trong Thực Tế Dự Án

Trường Hợp Sử Dụng	Dùng id?	Dùng class?	Ghi Chú & Khuyên Dùng
`Xác định phần tử duy nhất`	**Có**	**Không**	Dùng cho Header, Navigation, Anchor link.
`Tái sử dụng cho nhiều phần tử`	**Không**	**Có**	Dùng cho Card, Button, Input styling.
`Tạo kiểu dáng với CSS`	Hạn chế	**Khuyên dùng**	Giúp dễ ghi đè và quản lý CSS Modular.
`Tương tác với JavaScript`	**Nên dùng**	**Nên dùng**	Tùy chọn 1 phần tử hay mảng phần tử.
`Dễ bảo trì và tái sử dụng`	**Không**	**Có**	Class giúp tối ưu lại mã nguồn.

[Khái Niệm] Tổng Kết Tóm Tắt — id vs class Masterclass

- **id**: Chỉ dùng cho **1 phần tử duy nhất** trên trang. Thích hợp cho JavaScript `getElementById()` hoặc tạo Anchor link.
- **class**: Dùng cho **nhiều phần tử** có chung đặc điểm giao diện. Dễ tái sử dụng và bảo trì.
- **Độ ưu tiên CSS**: `id` có độ ưu tiên cao hơn `class`. Nên ưu tiên dùng `class` khi viết CSS.
- **Quy tắc đặt tên**: Tên `id` và `class` không được chứa khoảng trắng, không được bắt đầu bằng chữ số.

3.10 Thực Thể HTML (HTML Entities) & Kỹ Thuật Escaping

Để hiển thị các ký tự đặc biệt như `<`, `>`, `&`, `"` mà không làm vỡ cú pháp thẻ HTML, kĩ sư web sử dụng các mã thực thể **HTML Entities**:

[Bảng] Bảng Tra Cứu Thực Thể HTML Entities Thông Dụng

Ký Tự	Entity Tên	Entity Số	Tên Tiếng Anh	Ý Nghĩa & Ứng Dụng
<	`<`	`<`	Less than	Dấu nhỏ hơn (Mở thẻ).
>	`>`	`>`	Greater than	Dấu lớn hơn (Đóng thẻ).
&	`&`	`&`	Ampersand	Dấu và (Bắt đầu mã entity).
"	`"`	`"`	Double quote	Dấu ngoặc kép thuộc tính.
'	`'`	`'`	Single quote	Dấu ngoặc đơn.
(khoảng trắng)	` `	` `	Non-breaking space	Khoảng trắng không ngắt dòng.
©	`©`	`©`	Copyright	Biểu tượng bản quyền.
™	`™`	`™`	Trademark	Biểu tượng thương hiệu.
®	`®`	`®`	Registered	Biểu tượng đã đăng ký.

3.11 Bảng (Table) Trong HTML5 & Định Dạng CSS Masterclass

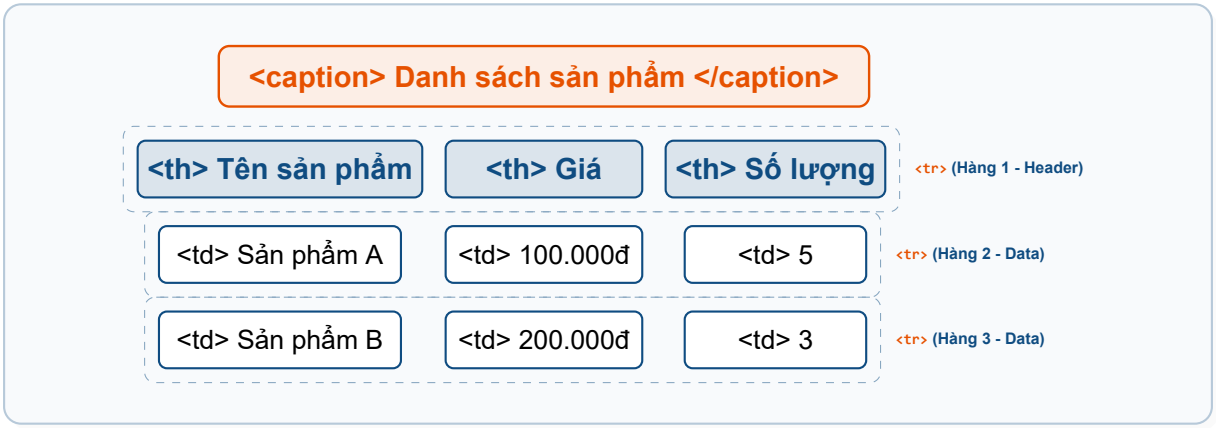
Trong thiết kế và phát triển giao diện Web, **Bảng (Table)** là cấu trúc dữ liệu hai chiều được sử dụng để hiển thị thông tin dạng ma trận hàng và cột (chẳng hạn như bảng giá, thống kê tài chính, danh sách sản phẩm, danh sách sinh viên, lịch trình...). Việc hiểu rõ cú pháp HTML5, bộ thẻ mở rộng ngữ nghĩa, thuộc tính kế thừa cũ và cách kiểm soát toàn diện bằng CSS3 là yêu cầu bắt buộc đối với mọi kĩ sư Frontend.

3.11.1 1. Cấu Trúc Cơ Bản Của Thẻ `<table>` & Sơ Đồ Cấu Trúc

Về mặt HTML, một bảng được khởi tạo bằng thẻ `<table>`. Các thẻ thành phần cơ bản gồm:

- `<table>`: Thẻ gốc khởi tạo bảng dữ liệu.
- `<caption>`: Tiêu đề mô tả nội dung của bảng.
- `<tr>` (*table row*): Xác định một hàng trong bảng.
- `<th>` (*table header*): Tiêu đề cột hoặc hàng (mặc định in đậm, căn giữa).
- `<td>` (*table data*): Ô chứa dữ liệu thông thường trong bảng.

Sơ đồ trực quan hóa cấu trúc bảng trong HTML5:



Ví dụ 1: Mã nguồn tạo bảng danh sách sản phẩm cơ bản

Mã Nguồn HTML5 Bảng Sản Phẩm Cơ Bản

```
<table>
    <caption>(*@Danh ásch ảsn ẩphm@*)</caption>
    <tr>
        <th>(*ê@Tn ảsn ẩphm@*)</th>
        <th>(*á@Gi@*)</th>
        <th>(*ố@S ượlng@*)</th>
    </tr>
    <tr>
        <td>(*ả@Sn ẩphm A@*)</td>
        <td>100.000(*đ@@*)</td>
        <td>5</td>
    </tr>
    <tr>
        <td>(*ả@Sn ẩphm B@*)</td>
        <td>200.000(*đ@@*)</td>
        <td>3</td>
    </tr>
</table>
```

Kết quả hiển thị trực quan của Ví dụ 1:

[Bảng] Kết Quả Hiển Thị Bảng HTML Mặc Định (Trình Duyệt Render Khi Chưa Có CSS)

Danh sách sản phẩm

Tên sản phẩm	Giá	Số lượng
Sản phẩm A	100.000đ	5
Sản phẩm B	200.000đ	3

Ví dụ 2: Mã nguồn tạo bảng thông tin sinh viên cơ bản

Mã Nguồn HTML5 Bảng Sinh Viên Cơ Bản

```
<table border="1">
    <tr>
        <th>(*ọ@H àv êTn@*)</th>
        <th>(*ổ@Tui@*)</th>
        <th>(*ớ@Gii ítnh@*)</th>
    </tr>
    <tr>
        <td>(*ễ@Nguyn ăVn A@*)</td>
        <td>25</td>
        <td>Nam</td>
    </tr>
    <tr>
        <td>(*ầ@Trn ịTh B@*)</td>
        <td>22</td>
        <td>(*ữ@N@*)</td>
    </tr>
</table>
```

Kết quả hiển thị trực quan của Ví dụ 2:

[Bảng] Kết Quả Hiển Thị Bảng HTML Khi Khai Báo Thuộc Tính border="1"

Họ và Tên	Tuổi	Giới tính
Nguyễn Văn A	25	Nam
Trần Thị B	22	Nữ

3.11.2 2. Bộ Thẻ Mở Rộng Trong Bảng (Semantics & Structure)

Để xây dựng bảng đúng chuẩn ngữ nghĩa HTML5, tối ưu SEO và hỗ trợ các thiết bị truy cập (Screen Readers), trình duyệt hỗ trợ bộ 8 thẻ mở rộng chuyên dụng:

[Bảng] Bảng Tra Cứu Bộ 8 Thẻ Cấu Trúc Bảng HTML5 Mở Rộng

Thẻ	Chức năng kỹ thuật
`<table>`	Thẻ gốc khởi tạo bảng dữ liệu.
`<tr>`	Xác định một hàng trong bảng (*Table Row*).
`<th>`	Tiêu đề cột/hàng (*Table Header*).
`<td>`	Ô chứa dữ liệu trong bảng (*Table Data*).
`<caption>`	Thêm tiêu đề mô tả chính cho bảng.
`<thead>`	Nhóm các hàng tiêu đề của bảng (*Table Head*).
`<tbody>`	Nhóm các hàng chứa dữ liệu chính (*Table Body*).
`<tfoot>`	Nhóm các hàng chân bảng (*Table Foot* — thường chứa tổng kết, số lượng).

Ví dụ 3: Mã nguồn sử dụng `<caption>`, `<thead>`, `<tbody>`, `<tfoot>`

Mã Nguồn Bảng Sử Dụng Đầy Đủ Thẻ Mở Rộng Ngữ Nghĩa

```
<table border="1">
    <caption>(*@Danh ásch sinh êvin@*)</caption>
    <thead>
        <tr>
            <th>(*ọ@H àv êTn@*)</th>
            <th>(*ổ@Tui@*)</th>
            <th>(*ớ@Gii ítnh@*)</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>(*ễ@Nguyn ăVn A@*)</td>
            <td>25</td>
            <td>Nam</td>
        </tr>
        <tr>
            <td>(*à@Trn ịTh B@*)</td>
            <td>22</td>
            <td>(*ữ@N@*)</td>
        </tr>
    </tbody>
    <tfoot>
        <tr>
            <td colspan="3">(*ổ@Tng ốs: 2 sinh êvin@*)</td>
        </tr>
    </tfoot>
</table>
```

Kết quả hiển thị trực quan của Ví dụ 3:

[Bảng] Kết Quả Hiển Thị Bảng Semantics Trong Trình Duyệt (border="1")

Danh sách sinh viên

Họ và Tên	Tuổi	Giới tính
Nguyễn Văn A	25	Nam
Trần Thị B	22	Nữ
Tổng số: 2 sinh viên		

3.11.3 3. Thuộc Tính HTML Kế Thừa & Kỹ Thuật Gộp Ô (Colspan/Rowspan)

1. Thuộc Tính Kế Thừa Theo HTML

Thuộc tính của bảng (`<table>`):

[Bảng] Bảng Tra Cứu Thuộc Tính HTML Của Thẻ <table>

Thuộc tính	Ý nghĩa	Ví dụ HTML
`border`	Viền bảng (lỗi thời, nên dùng CSS).	`<table border="1">`
`width`	Độ rộng bảng (pixel hoặc %).	`<table width="500px">`
`height`	Chiều cao bảng.	`<table height="300px">`
`cellpadding`	Khoảng cách nội dung với viền ô.	`<table cellpadding="10">`
`cellspacing`	Khoảng cách giữa các ô.	`<table cellspacing="5">`
`align`	Canh lề bảng (trái, phải, giữa) — lỗi thời.	`<table align="center">`
`bgcolor`	Màu nền bảng — lỗi thời.	`<table bgcolor="#f0f0f0">`

Ví dụ khai báo thuộc tính bảng bằng thuộc tính HTML:

Mã Nguồn Khai Báo Thuộc Tính Bảng HTML

```
1 <table border="1" width="500px" cellpadding="10" cellspacing="5">
```

Thuộc tính của hàng (`<tr>`):

[Bảng] Bảng Tra Cứu Thuộc Tính HTML Của Thẻ <tr>

Thuộc tính	Ý nghĩa	Ví dụ HTML
`align`	Căn lề nội dung hàng (trái, phải, giữa).	`<tr align="center">`
`valign`	Căn lề dọc nội dung (top, middle, bottom).	`<tr valign="top">`
`bgcolor`	Màu nền của hàng (lỗi thời, dùng CSS).	`<tr bgcolor="#ffcccb">`

Thuộc tính của ô (`<td>` và `<th>`):

[Bảng] Bảng Tra Cứu Thuộc Tính HTML Của <td> Và <th>

Thuộc tính	Ý nghĩa	Ví dụ HTML
`colspan`	Gộp nhiều cột thành một ô.	`<td colspan="2">`
`rowspan`	Gộp nhiều hàng thành một ô.	`<td rowspan="3">`
`align`	Căn chỉnh nội dung ngang (left, center, right).	`<td align="center">`
`valign`	Căn dọc nội dung (top, middle, bottom).	`<td valign="top">`
`bgcolor`	Màu nền ô (lỗi thời, dùng CSS).	`<td bgcolor="#ffff99">`
`width`	Độ rộng của ô.	`<td width="200px">`
`height`	Chiều cao của ô.	`<td height="50px">`

2. Ví Dụ Chi Tiết Kỹ Thuật Gộp Cột (Colspan) Và Gộp Hàng (Rowspan)

a. Kỹ thuật gộp cột bằng thuộc tính `colspan`:

Thuộc tính `colspan="N"` (viết tắt của *Column Span*) dùng để mở rộng một ô (ô `<th>` hoặc `<td>`) kéo dài sang phải chiếm độ rộng của **N cột**. Khi gộp N cột, ở các hàng bên dưới ta cần bớt đi $N - 1$ ô để tránh làm bảng bị tràn lệch sang bên phải.

Mã Nguồn Ví Dụ Gộp Cột Đơn Thuần Với colspan

```
<table border="1">
    <tr>
        <th>(*ọ@H àv êTn@*)</th>
        <th colspan="2">(*Đế@im Thi ọHc ỳK@*)</th>
    </tr>
    <tr>
        <td>(*ễ@Nguyn ăVn A@*)</td>
        <td>8.5</td>
        <td>9.0</td>
    </tr>
    <tr>
        <td>(*ầ@Trn ịTh B@*)</td>
        <td>7.0</td>
        <td>8.0</td>
    </tr>
</table>
```

Kết quả hiển thị trực quan (Gộp cột colspan=”2”):

[Bảng] Kết Quả Hiển Thị Gộp Cột Colspan (border=”1”)

Họ và Tên	Điểm Thi Học Kỳ	
Nguyễn Văn A	8.5	9.0
Trần Thị B	7.0	8.0

b. Kỹ thuật gộp hàng bằng thuộc tính `rowspan`:

Thuộc tính `rowspan="N"` (viết tắt của *Row Span*) dùng để mở rộng một ô kéo dài xuống dưới chiếm độ cao của **N hàng**. Khi một ô ở hàng trên gộp N hàng, thì ở các hàng bên dưới bị chiếm chỗ, ta bắt buộc phải **xóa bỏ ô tương ứng** ở các hàng đó để cấu trúc ô không bị đẩy lệch sang phải.

Mã Nguồn Ví Dụ Gộp Hàng Đơn Thuần Với rowspan

```
<table border="1">
    <tr>
        <th>(*@Ca ựTrc@*)</th>
        <th>(*à@Ngy@*)</th>
        <th>(*â@Nhn êVin ựTrc@*)</th>
    </tr>
    <tr>
        <td rowspan="2">(*@Ca áSng (07:00 - 12:00)@*)</td>
        <td>(*ứ@Th Hai@*)</td>
        <td>(*ễ@Nguyn ăVn A@*)</td>
    </tr>
    <tr>
        <td>(*ứ@Th Ba@*)</td>
        <td>(*ầ@Trn ịTh B@*)</td>
    </tr>
</table>
```

Kết quả hiển thị trực quan (Gộp hàng rowspan=”2”):

[Bảng] Kết Quả Hiển Thị Gộp Hàng Rowspan (border=”1”)

Ca Trực	Ngày	Nhân Viên Trực
Ca Sáng (07:00 - 12:00)	Thứ Hai	Nguyễn Văn A
	Thứ Ba	Trần Thị B

c. Kỹ thuật kết hợp đồng thời cả `colspan` và `rowspan`:

Trong các bảng Thời khóa biểu, Lịch làm việc ma trận hoặc Báo cáo doanh thu đa chiều, ta thường phải kết hợp đồng thời cả `colspan` và `rowspan` trên cùng một bảng.

Mã Nguồn Ví Dụ Kết Hợp Cả colspan Và rowspan

```
<table border="1">
    <tr>
        <th>(*ọ@H àv êTn@*)</th>
        <th colspan="2">(*ô@Thng Tin Chi ếTit@*)</th>
    </tr>
    <tr>
        <td rowspan="2">(*ễ@Nguyn ăVn A@*)</td>
        <td>(*ổ@Tui@*)</td>
        <td>25</td>
    </tr>
    <tr>
        <td>(*ớ@Gii ítnh@*)</td>
        <td>Nam</td>
    </tr>
</table>
```

Kết quả hiển thị trực quan (Kết hợp colspan=”2” và rowspan=”2”):

[Bảng] Kết Quả Hiển Thị Kết Hợp Colspan & Rowspan (border=”1”)

Họ và Tên	Thông Tin Chi Tiết	
Nguyễn Văn A	Tuổi	25
	Giới tính	Nam

3.11.4 4. Định Dạng CSS Masterclass & Phân Tích Thuộc Tính Trọng Tâm

Để định dạng giao diện bảng chuyên nghiệp, CSS cung cấp bộ thuộc tính phong phú kiểm soát viền, khoảng cách, căn lề và bố cục:

[Bảng] Bảng Tổng Hợp Các Thuộc Tính CSS Quan Trọng Dành Cho Bảng

Thuộc tính	Ý nghĩa	Giá trị phổ biến
`border`	Xác định viền của bảng hoặc ô.	`1px solid #000`
`border-collapse`	Quyết định viền bảng có gộp lại hay tách riêng.	`collapse, separate`
`border-spacing`	Khoảng cách giữa các ô (khi dùng separate).	`10px, 0`
`width`	Độ rộng của bảng.	`100%, auto, 500px`
`height`	Chiều cao của bảng hoặc ô.	`auto, 300px`
`padding`	Khoảng cách giữa nội dung và viền ô.	`10px, 5px 10px`
`text-align`	Canh lề nội dung trong ô.	`left, center, right`
`vertical-align`	Canh lề theo chiều dọc.	`top, middle, bottom`
`background-color`	Màu nền cho bảng hoặc ô.	`#f2f2f2, transparent, red`
`caption-side`	Vị trí tiêu đề bảng.	`top, bottom`
`empty-cells`	Xử lý ô trống (khi không có nội dung).	`show, hide`
`table-layout`	Xác định cách trình duyệt phân bổ độ rộng cột.	`auto, fixed`

2. Thuộc Tính Theo CSS

Thuộc tính cho `<table>`:

[Bảng] Bảng Thuộc Tính CSS Cho <table>

Thuộc tính	Ý nghĩa	Ví dụ CSS
`width`	Độ rộng của bảng	`width: 100%;`
`height`	Chiều cao của bảng	`height: 400px;`
`border`	Độ dày, kiểu, và màu của viền	`border: 1px solid #ccc;`
`border-collapse`	Gộp viền hoặc tách viền giữa các ô	`border-collapse: collapse;`
`border-spacing`	Khoảng cách giữa các ô (nếu không gộp viền)	`border-spacing: 10px;`
`background-color`	Màu nền bảng	`background-color: #f9f9f9;`
`box-shadow`	Đổ bóng cho bảng	`box-shadow: 0 2px 10px #ccc;`
`border-radius`	Bo góc bảng	`border-radius: 10px;`
`caption-side`	Vị trí của tiêu đề bảng (caption)	`caption-side: bottom;`
`empty-cells`	Kiểm soát hiển thị ô trống	`empty-cells: hide;`
`table-layout`	Kiểm soát cách bảng tính toán độ rộng	`table-layout: fixed;`

Thuộc tính cho `<th>` và `<td>` (Ô bảng):

[Bảng] Bảng Thuộc Tính CSS Cho <th> Và <td>

Thuộc tính	Ý nghĩa	Ví dụ CSS
`padding`	Khoảng cách giữa nội dung và viền	`padding: 10px;`
`text-align`	Căn lề nội dung	`text-align: center;`
`vertical-align`	Căn dọc nội dung	`vertical-align: middle;`
`border`	Viền của ô	`border: 1px solid #ddd;`
`background-color`	Màu nền ô	`background-color: #ffffff;`
`color`	Màu chữ	`color: #333;`
`font-size`	Kích thước chữ	`font-size: 16px;`
`font-weight`	Độ đậm chữ	`font-weight: bold;`
`text-transform`	Chuyển đổi chữ hoa/thường	`text-transform: uppercase;`
`white-space`	Kiểm soát xuống dòng	`white-space: nowrap;`
`word-wrap`	Tự động xuống dòng khi quá dài	`word-wrap: break-word;`

Thuộc tính cho `<tr>` (Hàng):

[Bảng] Bảng Thuộc Tính CSS Cho <tr>

Thuộc tính	Ý nghĩa	Ví dụ CSS
`background-color`	Màu nền hàng	`background-color: #f2f2f2;`
`height`	Chiều cao hàng	`height: 50px;`
`border`	Viền hàng	`border: 1px solid #ddd;`
`transition`	Hiệu ứng chuyển đổi	`transition: background 0.3s;`

Hiệu ứng đặc biệt:

[Bảng] Bảng Thuộc Tính Hiệu Ứng Đặc Biệt Pseudo-class

Hiệu ứng	Ý nghĩa	Ví dụ CSS
`:hover`	Thay đổi style khi trỏ chuột vào	`tr:hover { background: #e2e6ea; }`
`:nth-child()`	Chọn hàng/ô theo thứ tự	`tr:nth-child(odd) { color: red; }`
`:first-child`	Chọn phần tử đầu tiên	`td:first-child { font-weight: bold; }`

3.11.5 Phân Tích Chi Tiết Các Thuộc Tính Trọng Tâm

1. `border` (Viền Bảng Và Ô)

Mặc định: `none` (không có viền). Nếu không set CSS, bảng và ô sẽ không có viền.

Ví dụ mặc định không viền:

Mã Nguồn Bảng Mặc Định Không Viền

```html
<table>
    <tr>
        <th>Sản Phẩm</th>
        <th>Giá cả</th>
    </tr>
</table>
```

Ví dụ 2: Khoảng cách ngang và dọc khác nhau (Ngang 20px, Dọc 5px)

Mã Nguồn Ví Dụ 2 border-spacing Khác Nhau

```
<style>
  table {
    border: 2px solid #007bff;
    border-collapse: separate;
    border-spacing: 20px 5px; /* (*@Ngang 20px, ọdc 5px@*) */
  }
  th, td {
    border: 1px solid #ddd;
    padding: 10px;
  }
</style>
<table>
  <tr>
    <th>(*ộ@Ct 1@*)</th>
    <th>(*ộ@Ct 2@*)</th>
    <th>(*ộ@Ct 3@*)</th>
  </tr>
  <tr>
    <td>(*à@Hng 2 ộCt 1@*)</td>
    <td>(*à@Hng 2 ộCt 2@*)</td>
    <td>(*à@Hng 2 ộCt 3@*)</td>
  </tr>
</table>
```

[Bảng] Kết Quả Hiển Thị border-spacing 20px 5px

Cột 1	Cột 2	Cột 3
Hàng 2 Cột 1	Hàng 2 Cột 2	Hàng 2 Cột 3

Ví dụ 3: Ứng dụng thực tế — Bảng Card Grid Hiện Đại (Bo Góc border-radius + border-spacing)

Khi kết hợp `border-collapse: separate;` với `border-spacing: 12px;` và thuộc tính bo góc `border-radius`, ta tạo ra giao diện bảng dạng thẻ Card vô cùng bắt mắt:

Mã Nguồn Bảng Dạng Card Grid Với border-spacing VÀ border-radius

```
<style>
  .card-table {
    width: 100%;
    border-collapse: separate;
    border-spacing: 12px; /* (*ạ@To ảkhong ácch ữgia ácc ốkhi card@*) */
  }
  .card-table th {
    background-color: #007bff;
    color: white;
    padding: 12px;
    border-radius: 6px;
  }
  .card-table td {
    background-color: #ffffff;
    border: 1px solid #e0e0e0;
    border-radius: 8px; /* (*@Bo ógc ừtng ô ữđ ệliu@*) */
    padding: 12px;
  }
</style>
<table class="card-table">
  <tr><th>(*ã@M ớLp@*)</th><th>(*ê@Tn óKha ọHc@*)</th><th>(*ọ@Hc íPh@*)</th></tr>
  <tr><td>FE-01</td><td>(*ậ@Lp ìTrnh Web HTML5/CSS3@*)</td><td>2.500.000(*đ@@*)<
    /td></tr>
  <tr><td>JS-02</td><td>JavaScript ES6+ Master</td><td>3.000.000(*đ@@*)</td></tr>
</table>
```

[Bảng] Kết Quả Hiển Thị Bảng Card Grid Hiện Đại

Mã Lớp	Tên Khóa Học	Học Phí
FE-01	Lập Trình Web HTML5/CSS3	2.500.000đ
JS-02	JavaScript ES6+ Master	3.000.000đ

9. `background-color` (Màu Nền)

Mặc định: `transparent` (không màu nền).

Ví dụ:

Mã Nguồn background-color transparent

```
td {
    background-color: transparent;
}
```

Tô màu:

Mã Nguồn Tô Màu Nền Cho th

```
th {
    background-color: #007bff; /* (*à@Mu xanh@*) */
    color: #fff; /* (*ữ@Ch ắtrng@*) */
}
```

10. `caption-side` (Vị Trí Tiêu Đề Bảng)

Mặc định: `top` (tiêu đề ở trên).

Ví dụ:

Mã Nguồn caption-side top

```
caption {
    caption-side: top;
}
```

Chuyển xuống dưới:

Mã Nguồn caption-side bottom

```
caption {
    caption-side: bottom;
}
```

11. `empty-cells` (Xử Lý Ô Trống)

Mặc định: `show` (hiển thị viền dù ô trống).

Ví dụ:

Mã Nguồn empty-cells show

```
td {
    empty-cells: show;
}
```

Ẩn viền ô trống:

Mã Nguồn empty-cells hide

```
td {
    empty-cells: hide;
}
```

[Khái Niệm] Kết Luận Chung

- Trình duyệt có các giá trị mặc định giúp bảng hiển thị dễ nhìn mà không cần CSS.
- Nhưng để bảng đẹp và dễ đọc, bạn nên tinh chỉnh các thuộc tính như `border`, `padding`, `background`, và `text-align`.
- Bí quyết: Dùng `border-collapse: collapse` và `table-layout: fixed` để bảng gọn gàng, và kết hợp `nth-child` để tạo màu xen kẽ hàng!

3.11.6 Đọc Thêm: Structural Semantics, Responsive Matrix & Accessibility

1. Cấu Trúc Ngữ Nghĩa Nâng Cao: `<thead>`, `<tbody>`, `<tfoot>` & Thuộc Tính `scope`

Khi làm việc với các bảng dữ liệu phức tạp hoặc bảng phục vụ báo cáo doanh thu, tài chính, việc phân chia cấu trúc thành ba vùng ngữ nghĩa chính là quy chuẩn bắt buộc của HTML5:

- `<thead>`: Chứa hàng tiêu đề chính của bảng.
- `<tbody>`: Chứa phần thân chứa dữ liệu động.
- `<tfoot>`: Chứa phần chân bảng (thường dùng hiển thị tổng số, trung bình hoặc ghi chú).

Mã Nguồn Bảng HTML5 Semantics Đầy Đủ

```
<table>
    <caption>(*á@Bo áco doanh thu ănm 2026@*)</caption>
    <thead>
        <tr>
            <th scope="col">(*ý@Qu@*)</th>
            <th scope="col">Doanh thu (VND)</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <th scope="row">(*ý@Qu 1@*)</th>
            <td>1.500.000.000(*đ@@*)</td>
        </tr>
        <tr>
            <th scope="row">(*ý@Qu 2@*)</th>
            <td>2.100.000.000(*đ@@*)</td>
        </tr>
    </tbody>
    <tfoot>
        <tr>
            <th scope="row">(*ổ@Tng ộcng@*)</th>
            <td>3.600.000.000(*đ@@*)</td>
        </tr>
    </tfoot>
</table>
```

Thuộc tính `scope="col"` và `scope="row"` giúp thiết bị Screen Reader (dành cho người khiếm thị) phân biệt ô tiêu đề đó áp dụng cho toàn bộ cột hay toàn bộ hàng, nâng cao khả năng tiếp cận chuẩn WAI-ARIA.

2. Thẻ Định Dạng Nhóm Cột (`<colgroup>` & `<col>`)

Thẻ `<colgroup>` và `<col>` cho phép thiết lập màu nền hoặc độ rộng đồng loạt cho từng cột mà không cần lặp lại CSS trên từng ô `<td>`:

Mã Nguồn Định Dạng Nhóm Cột Bằng colgroup VÀ col

```
<table border="1">
    <colgroup>
        <col style="background-color: #f2f2f2; width: 80px;">
        <col style="background-color: #e6f7ff;">
        <col style="background-color: #fff0f6; width: 120px;">
    </colgroup>
    <tr>
        <th>(*ã@M SP@*)</th>
        <th>(*ê@Tn ảSn ẩPhm@*)</th>
        <th>(*Đơ@n áGi@*)</th>
    </tr>
    <tr>
        <td>SP01</td>
        <td>Laptop Dell XPS 15</td>
        <td>35.000.000(*đ@@*)</td>
    </tr>
    <tr>
        <td>SP02</td>
        <td>(*à@Bn íphm ơc Keychron@*)</td>
        <td>2.200.000(*đ@@*)</td>
    </tr>
</table>
```

Kết quả hiển thị trực quan (colgroup tô màu từng cột):

[Bảng] Kết Quả Hiển Thị Nhóm Cột Colgroup (border="1")

Mã SP	Tên Sản Phẩm	Đơn Giá
SP01	Laptop Dell XPS 15	35.000.000đ
SP02	Bàn phím cơ Keychron	2.200.000đ

3. Kỹ Thuật Tô Màu Xen Kẽ Hàng (Zebra Striping Masterclass)

Mã CSS tô màu dòng chẵn/lẻ bằng giả lớp `:nth-child(even)` và `:nth-child(odd)` giúp bảng dữ liệu lớn dễ đọc hơn:

Mã Nguồn CSS Bảng Màu Xen Kẽ Zebra Striping

```
<style>
    .zebra-table {
        width: 100%;
        border-collapse: collapse;
    }
    .zebra-table th, .zebra-table td {
        padding: 10px;
        border: 1px solid #ddd;
        text-align: left;
    }
    .zebra-table th {
        background-color: #007bff;
        color: white;
    }
    .zebra-table tr:nth-child(even) {
        background-color: #f8f9fa; /* (*à@Mu ềnn àhng ẫchn@*) */
    }
    .zebra-table tr:hover {
        background-color: #e2e6ea; /* (*ệ@Hiu ứng đổi àmu khi ỏtr ộchut@*) */
    }
</style>

<table class="zebra-table">
    <tr><th>STT</th><th>(*ọ@H àv êTn@*)</th><th>(*ứ@Chc ụV@*)</th></tr>
    <tr><td>1</td><td>(*ễ@Nguyn ăVn A@*)</td><td>(*ậ@Lp ìTrnh êVin Frontend@*)</td></tr>
    <tr><td>2</td><td>(*ầ@Trn ịTh B@*)</td><td>(*ê@Chuyn êVin UI/UX@*)</td></tr>
    <tr><td>3</td><td>(*ê@L ăVn C@*)</td><td>(*ỹ@K ưS Backend@*)</td></tr>
    <tr><td>4</td><td>(*ạ@Phm ịTh D@*)</td><td>(*ả@Qun ýL ựD Án@*)</td></tr>
</table>
```

Kết quả hiển thị trực quan (Zebra Striping & Hover effect):

[Bảng] Kết Quả Hiển Thị Bảng Màu Xen Kẽ Zebra Striping

STT	Họ và Tên	Chức Vụ
1	Nguyễn Văn A	Lập Trình Viên Frontend
2	Trần Thị B	Chuyên Viên UI/UX
3	Lê Văn C	Kỹ Sư Backend
4	Phạm Thị D	Quản Lý Dự Án

4. Responsive Table Cho Thiết Bị Di Động

Trên màn hình di động nhỏ, bảng nhiều cột thường bị tràn khung ngang. Giải pháp chuẩn Production bao gồm:

1. **Khung cuộn ngang (Scroll Container)**: Bọc bảng trong một thẻ `<div class="table-responsive">` và thiết lập CSS `overflow-x: auto; -webkit-overflow-scrolling: touch;`.
2. **Chuyển bảng thành Card (Stacked Matrix)**: Sử dụng CSS Media Queries để chuyển các hàng `<tr>` thành các khối Card độc lập trên màn hình nhỏ.

Mã Nguồn Khung Cuộn Ngang Cho Bảng Responsive (Phương Pháp 1)

```
<style>
    .table-responsive {
        width: 100%;
        overflow-x: auto; /* (*@Cho éphp ộcun ngang khi ảbng áqu ộrng@*) */
        -webkit-overflow-scrolling: touch;
    }
</style>

<div class="table-responsive">
    <table border="1">
        <tr>
            <th>(*ã@M Đơn@*)</th>
            <th>(*á@Khch àHng@*)</th>
            <th>(*ả@Sn ẩPhm@*)</th>
            <th>(*à@Ngy Đặt@*)</th>
            <th>(*ạ@Trng áThi@*)</th>
        </tr>
        <tr>
            <td>#1001</td>
            <td>(*ễ@Nguyn ăVn A@*)</td>
            <td>iPhone 15 Pro Max 256GB</td>
            <td>2026-07-23</td>
            <td>(*Đã@ Giao àhng@*)</td>
        </tr>
    </table>
</div>
```

Mã nguồn Kỹ thuật chuyển Bảng thành Card trên Mobile (Phương Pháp 2 — Nâng Cao):

Đây là kỹ thuật Responsive đỉnh cao trong CSS Production. Trên màn hình nhỏ (< 768px), ta ẩn hàng tiêu đề `<thead>` và biến mỗi ô `<td>` thành một hàng dạng Card, sử dụng thuộc tính `data-label` và thuộc tính CSS `content: attr(data-label)` để chèn nhãn tự động:

Mã Nguồn CSS Kỹ Thuật Responsive Stacked Card Dùng attr(data-label)

```
<style>
@media screen and (max-width: 768px) {
    /* (*ể@Chuyn ấtt ảc ầphn ửt ảbng àthnh ốkhi block@*) */
    .responsive-card-table,
    .responsive-card-table thead,
    .responsive-card-table tbody,
    .responsive-card-table th,
    .responsive-card-table td,
    .responsive-card-table tr {
        display: block;
    }

    /* (*Ẩ@n àhng êtiu đề íchnh êtrn mobile@*) */
    .responsive-card-table thead tr {
        position: absolute;
        top: -9999px;
        left: -9999px;
    }

    /* (*Đó@ng ógi ừtng àhng àthnh 1 ốkhi Card@*) */
    .responsive-card-table tr {
        margin-bottom: 15px;
        border: 1px solid #007bff;
        border-radius: 8px;
        padding: 8px;
        background-color: #ffffff;
    }

    /* (*ă@Cn ỉchnh ácc ô ạdng danh ásch label - value@*) */
    .responsive-card-table td {
        border: none;
        position: relative;
        padding-left: 45%;
        text-align: right;
    }

    /* (*è@Chn ãnhn ộct ựt động ừt data-label@*) */
    .responsive-card-table td::before {
        content: attr(data-label); /* (*ấ@Ly ági ịtr ừt ộthuc ítnh data-label@*)
    */
        position: absolute;
        left: 10px;
        font-weight: bold;
        color: #007bff;
    }
}
</style>
```

Mã Nguồn HTML Cấu Trúc Bảng Responsive Với Thuộc Tính data-label

```
<table class="responsive-card-table">
    <thead>
        <tr><th>(*ã@M Đơn@*)</th><th>(*á@Khch àHng@*)</th><th>(*ố@S ềTin@*)</th><
    /tr>
    </thead>
    <tbody>
        <tr>
            <td data-label="(*ã@M Đơn@*)">#1001</td>
            <td data-label="(*á@Khch àHng@*)">(*ễ@Nguyn ăVn A@*)</td>
            <td data-label="(*ố@S ềTin@*)">15.000.000(*đ@@*)</td>
        </tr>
    </tbody>
</table>
```

Kết quả hiển thị trực quan (Kiểu Stacked Card Responsive trên Mobile):

[Bảng] Kết Quả Hiển Thị Bảng Đã Chuyển Thành Khối Card Trên Mobile

#1001	**Mã Đơn:**
Nguyễn Văn A	**Khách Hàng:**
15.000.000đ	**Số Tiền:**

5. Best Practices & Cảnh Báo Anti-patterns Khi Thiết Kế Bảng

[Mẹo] 5 Nguyên Tắc Vàng Thiết Kế Bảng Chuẩn UX/UI Hiện Đại

- **1. Đặt tên tiêu đề cột rõ ràng:** Tiêu đề `<th>` cần ngắn gọn, chính xác và có thể kèm mũi tên sắp xếp (Sort Icon) nếu là bảng dữ liệu động.
- **2. Căn chỉnh dữ liệu theo bản chất:**
  - Dạng chữ (Text, Tên, Địa chỉ): **Căn trái** (`text-align: left`).
  - Dạng số (Giá tiền, Số lượng, Điểm số): **Căn phải** (`text-align: right`) giúp so sánh chữ số hàng đơn vị/trăm/nghìn dễ dàng.
  - Trạng thái, Ngày tháng, Mã định danh ngắn: **Căn giữa** (`text-align: center`).
- **3. Sử dụng Zebra Striping:** Tô màu xen kẽ các hàng giúp mắt người đọc không bị lệch dòng khi đọc các bảng nhiều hơn 5 cột.
- **4. Thêm hiệu ứng Hover trên hàng:** Đổi màu nền nhẹ (`tr:hover`) giúp người dùng tập trung vào hàng dữ liệu đang tương tác.
- **5. Cố định hàng tiêu đề (Sticky Header):** Với bảng dài có thanh cuộn dọc, dùng CSS `th position: sticky; top: 0;` giúp giữ hàng tiêu đề luôn hiển thị ở trên cùng.

[Cảnh Báo] Cảnh Báo Về Việc Lạm Dụng Bảng Trong Layout Website

- **Chỉ dùng bảng cho dữ liệu ma trận (Tabular Data)**: Như bảng giá, lịch làm việc, danh sách đối soát.
- **TUYỆT ĐỐI KHÔNG dùng bảng để dựng bố cục website**: Dùng `<table>` để chia cột trang web sẽ khiến cấu trúc HTML cồng kềnh, không tối ưu cho SEO và cực kỳ khó responsive trên mobile. Hãy sử dụng **Flexbox** hoặc **CSS Grid** cho bố cục toàn trang!

6. Tối Ưu Khả Năng Truy Cập WAI-ARIA (Accessibility & Screen Readers)

[Ghi Chú] Ghi Chú Accessibility: Tối Ưu Cho Người Khiếm Thị

Các thiết bị đọc màn hình (Screen Readers như NVDA, JAWS, VoiceOver) relies vào cấu trúc HTML để đọc dữ liệu bảng cho người khiếm thị:

- Luôn khai báo `<caption>` để mô tả ngắn gọn mục đích của bảng.
- Dùng thuộc tính `scope="col"` cho tiêu đề cột và `scope="row"` cho tiêu đề hàng.
- Với bảng phức tạp có nhiều mức header, sử dụng cặp thuộc tính `id="..."` trên `<th>` và `headers="..."` trên `<td>` để liên kết dữ liệu chính xác.

7. Tối Ưu Hiệu Năng Render Trong Trình Duyệt (Browser Performance)

[Mẹo] Mẹo Tối Ưu Hiệu Năng Với table-layout: fixed

Mặc định trình duyệt dùng thuật toán `table-layout: auto` để tính toán độ rộng ô dựa trên nội dung tệp. Với bảng chứa hàng ngàn dòng, trình duyệt phải tải xong 100% dữ liệu mới render được bảng (độ phức tạp tính toán $O(N \times M)$ gây hiện tượng đơ giật trang web).
Giải pháp: Khai báo CSS `table-layout: fixed; width: 100%;` giúp trình duyệt render bảng ngay lập tức dựa trên độ rộng cột đầu tiên mà không cần chờ tải hết dữ liệu!

8. Góc Phỏng Vấn Tuyển Dụng Frontend (Interview Q&A)

[Khái Niệm] Câu Hỏi Phỏng Vấn 1: Sự Khác Biệt Giữa border-collapse: collapse Và separate?

Trả lời: `collapse` gộp các viền giáp ranh giữa các ô thành 1 đường viền duy nhất, giúp bảng gọn gàng. Trong khi `separate` giữ viền riêng biệt cho từng ô và cho phép tùy chỉnh khoảng cách giữa các ô bằng thuộc tính `border-spacing`.

[Khái Niệm] Câu Hỏi Phỏng Vấn 2: Tại Sao Ngày Nay Lập Trình Viên Không Dùng Thẻ Table Để Dựng Bố Cục Trang Web?

Trả lời: 1. **Ngữ nghĩa (Semantics):** Thẻ `table` sinh ra để hiển thị dữ liệu dạng bảng, không phải bố cục web. 2. **Tốc độ tải trang:** Bảng yêu cầu trình duyệt tải toàn bộ nội dung rồi mới tính toán vẽ lại layout (Reflow/Repaint tốn kém CPU). 3. **Responsive:** Rất khó co giãn flex layout trên màn hình mobile. 4. **SEO & Accessibility:** Gây nhiễu bộ đọc của công cụ tìm kiếm và thiết bị hỗ trợ người khuyết tật.

[Khái Niệm] Câu Hỏi Phỏng Vấn 3: Làm Thế Nào Để Xử Lý Bảng Bị Tràn Khung Trên Màn Hình Điện Thoại?

Trả lời: Có 2 kỹ thuật phổ biến: 1. Bọc thẻ `<table>` trong `<div style="overflow-x: auto;">` để tạo thanh cuộn ngang mượt mà. 2. Dùng CSS Media Queries chuyển các ô `<td>` thành `display: block;` để biến từng hàng `<tr>` thành một thẻ Card đứng gọn gàng trên mobile.

3.12 Thẻ Nhúng <iframe> (Inline Frame) Masterclass

Thẻ `<iframe>` (**Inline Frame**) được sử dụng trong HTML5 để nhúng một tài liệu HTML độc lập khác trực tiếp vào bên trong khung hiển thị của trang web hiện tại. Kỹ thuật này được ứng dụng rộng rãi để tích hợp video phát trực tuyến (YouTube, Vimeo), bản đồ tương tác (Google Maps), tệp tài liệu PDF, các widget mạng xã hội hoặc ngay cả toàn bộ một ứng dụng web độc lập.

3.12.1 Cú Pháp Khai Báo & Nguyên Lý Hoạt Động Của <iframe>

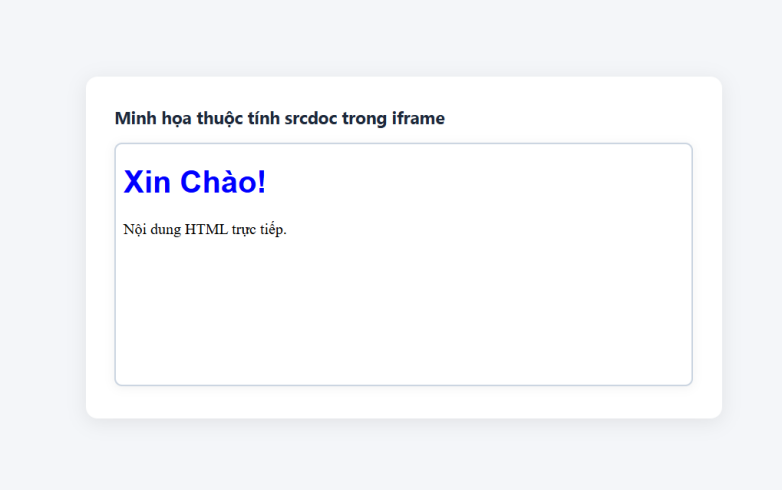
Thẻ <iframe> (**Inline Frame**) được khai báo theo cấu trúc thẻ kép với các thuộc tính điều khiển hành vi hiển thị:

Mã Nguồn Khai Báo Cú Pháp Cấu Trúc Thẻ <iframe>

```
<!-- Cú pháp cơ bản nhúng trang web bên ngoài qua URL -->
<iframe src="https://example.com" width="800" height="600"></iframe>

<!-- Cú pháp nhúng mã HTML trực tiếp bằng srcdoc (Không cần server) -->
<iframe srcdoc="<h1 style='color:blue;'>Xin Chào!</h1><p>Nội dung HTML trực
    tiếp.</p>"
        width="600" height="300">
</iframe>
```

BROWSER PREVIEW file:///project/srcdoc-demo.html



3.12.2 Bảng Tra Cứu Toàn Bộ Thuộc Tính Quan Trọng Của <iframe>

[Bảng] Bảng Tra Cứu Tất Cả Thuộc Tính Thẻ <iframe> Trong HTML5

Thuộc Tính	Ý Nghĩa & Chức Năng Cốt Lõi	Ví dụ cú pháp Minh Họa
src	Đường dẫn URL trỏ tới tài liệu cần nhúng	<iframe src="https://example.com">
srcdoc	Nhúng trực tiếp chuỗi mã HTML (HTML5)	<iframe srcdoc=" Hi ">
width	Định nghĩa chiều rộng khung (pixel/percent)	<iframe width="600">
height	Định nghĩa chiều cao khung (pixel)	<iframe height="400">
frameborder	0: Không viền, 1: Có viền (Khuyên dùng CSS)	<iframe frameborder="0">
allowfullscree	Bật quyền mở chế độ xem toàn màn hình	<iframe allowfullscreen>
sandbox	Giới hạn quyền hạn và bảo mật nội dung	<iframe sandbox="allow-scripts">
loading	lazy (tải khi cuộn), eager (tải ngay)	<iframe loading="lazy">
name	Tên khung để liên kết với thuộc tính target	<iframe name="myFrame">

3.12.3 4 Trường Hợp Sử Dụng <iframe> Phổ Biến Trong Thực Tế Dự Án

1. Nhúng một trang web mở (Ví dụ Wikipedia):

Mã Nguồn Nhúng Trang Web Wikipedia Vào Khung Iframe

```
<iframe src="https://www.wikipedia.org/" width="800" height="600"></iframe>
```

BROWSER PREVIEW https://www.wikipedia.org/



[Ghi Chú] Lưu Ý Quan Trọng Về Chính Sách X-Frame-Options

Một số trang web lớn (như Google, Facebook) thiết lập Header bảo mật `X-Frame-Options: DENY` hoặc `SAMEORIGIN` nhằm ngăn chặn trang khác nhúng vào khung iframe để chống tấn công **Clickjacking**.

2. Nhúng Video phát trực tuyến từ YouTube:

YouTube cung cấp đường dẫn nhúng chuẩn qua định dạng `/embed/VIDEO_ID`:

Mã Nguồn Nhúng Video YouTube Chuẩn Ngữ Nghĩa

```
<iframe width="560" height="315"
        src="https://www.youtube.com/embed/dQw4w9WgXcQ"
        allow="accelerometer; autoplay; clipboard-write; encrypted-media;
    gyroscope; picture-in-picture"
        allowfullscreen>
</iframe>
```

BROWSER PREVIEW https://www.youtube.com/embed/dQw4w9WgXcQ

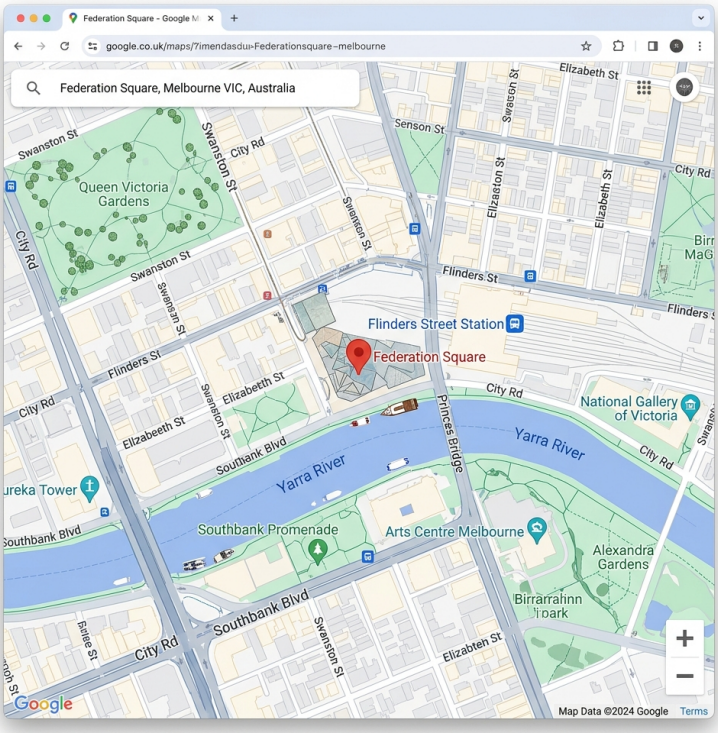


3. Nhúng Bản đồ tương tác Google Maps:

Mã Nguồn Nhúng Google Maps Vào Khung Web

```
<iframe width="600" height="450"
        src="https://www.google.com/maps/embed?pb=!1m18!1m12!1m3!1d3151
    .835434509839!2d144.9630579153167!3d-37.81410797975171!2m3!1f0!2f0!3f0!3m2!1
    i1024!2i768!4f13.1!3m3!1m2!1s0x6ad642af0f11fd81%3A0xf5770f9af14bdf8d!2
    sFederation%20Square!5e0!3m2!1sen!2s!4v1589163966416!5m2!1sen!2s"
        allowfullscreen>
</iframe>
```

BROWSER PREVIEW https://www.google.com/maps/embed

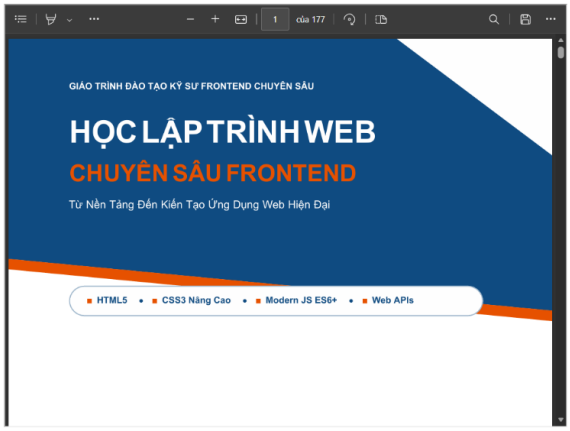


4. Nhúng Tài liệu báo cáo dạng tệp PDF:

Mã Nguồn Nhúng Trực Tiếp Tệp Tài Liệu PDF

```
<iframe src="document.pdf" width="800" height="600"></iframe>
```

BROWSER PREVIEW file:///project/document.pdf



3.12.4 Hướng Dẫn Chi Tiết Quy Trình Lấy Đường Dẫn Nhúng (Embed Link)

Một trong những sai lầm phổ biến nhất của lập trình viên mới bắt đầu là sao chép trực tiếp đường dẫn URL hiển thị trên thanh địa chỉ của trình duyệt để đưa vào thuộc tính `src` của thẻ `<iframe>`. Các nền tảng lớn như YouTube hay Google Maps đều sử dụng cơ chế bảo mật chống nhúng trực tiếp URL xem thông thường.

[Cảnh Báo] Cảnh Báo Bẫy Sai Lầm Thường Gặp Khi Lấy Link Nhúng

- **SAI**: `src="https://www.youtube.com/watch?v=VIDEO_ID"` → Trình duyệt sẽ báo lỗi **Refused to display in a frame because it set 'X-Frame-Options' to 'sameorigin'**.
- **ĐÚNG**: `src="https://www.youtube.com/embed/VIDEO_ID"` → Sử dụng giao thức nhúng chuẩn do nền tảng cung cấp.

Quy Trình 4 Bước Lấy Link Nhúng Video YouTube Chuẩn

BƯỚC 1 Mở video cần nhúng trên trang web YouTube.

BƯỚC 2 Nhấp vào nút **Chia sẻ** (*Share*) phía dưới khung phát video.

BƯỚC 3 Trong cửa sổ hiện ra, chọn biểu tượng **Nhúng** (*Embed* - biểu tượng mã `</>`).

BƯỚC 4 Nhấp nút **Sao chép** (*Copy*) để lấy toàn bộ đoạn mã `<iframe>` đã được tích hợp sẵn thuộc tính chuẩn.

[Mẹo] Mẹo Chuyển Đổi Nhanh URL YouTube Thành Embed Link

Nếu chỉ có URL dạng xem thông thường: `https://www.youtube.com/watch?v=dQw4w9WgXcQ`, bạn chỉ cần thay thế cụm `watch?v=` bằng `embed/` để tạo thành đường dẫn nhúng hợp lệ: `https://www.youtube.com/embed/dQw4w9WgXcQ`.

Quy Trình 4 Bước Lấy Link Nhúng Bản Đồ Google Maps

BƯỚC 1 Truy cập Google Maps (`maps.google.com`) và tìm kiếm địa điểm/vị trí muốn hiển thị.

BƯỚC 2 Tại bảng thông tin địa điểm ở góc trái, nhấp nút **Chia sẻ** (*Share*).

BƯỚC 3 Chuyển sang tab **Nhúng bản đồ** (*Embed a map*).

BƯỚC 4 Chọn kích thước mong muốn (Nhỏ, Vừa, Lớn) rồi nhấp nút **Sao chép thẻ HTML** (*Copy HTML*).

3. Cách nhúng tệp tài liệu PDF cục bộ hoặc trên máy chủ:

Để nhúng tệp PDF vào trang web, chỉ cần đặt tệp PDF vào cùng thư mục dự án và khai báo đường dẫn tương đối trỏ trực tiếp tới tệp:

Khai Báo Nguồn Tệp PDF Trong Cùng Thư Mục Dự Án

```
<!-- Tệp document.pdf nằm cùng cấp thư mục với tệp index.html -->
<iframe src="document.pdf" width="100%" height="600px"></iframe>

<!-- Hoặc trỏ tới thư mục chứa tài liệu tĩnh -->
<iframe src="./assets/pdf/bao-cao-2026.pdf" width="100%" height="600px"></iframe>
```

3.12.5 Cơ Chế Bảo Mật Tấn Công Clickjacking & Thuộc Tính sandbox

Rủi ro bảo mật với Iframe:

Nếu nhúng một trang web không rõ nguồn gốc, trang đó có thể chạy các kịch bản độc hại, đánh cắp Cookie hoặc đánh lừa người dùng nhấp vào các nút giả mạo (**Clickjacking Attack**).

Giải pháp thiết lập thuộc tính sandbox:

Thuộc tính `sandbox` tạo ra một môi trường cách ly (**Isolation Sandbox**) cực kỳ nghiêm ngặt. Khi khai báo `sandbox`, mặc định trình duyệt sẽ vô hiệu hóa tất cả Script, Form submit, Popup và Same-origin access:

Phân Quyền Chi Tiết Cho Iframe Bằng sandbox

```
<iframe src="https://example.com"
        sandbox="allow-scripts allow-forms">
</iframe>
```

BROWSER PREVIEW **https://example.com (sandbox="allow-scripts allow-forms")**

Môi Trường Isolation Sandbox Active

Quyền được cấp: `allow-scripts`, `allow-forms`

Bị khóa hoàn toàn: Bị cấm truy cập Cookie, LocalStorage và DOM của trang cha!

3.12.6 Kỹ Thuật Loại Bỏ Viền & Định Kiểu CSS Hiện Đại

Mặc định thẻ `<iframe>` có đường viền 3D cổ điển. Trong thực tế dự án, lập trình viên sử dụng CSS để loại bỏ viền và làm mịn giao diện:

Loại Bỏ Viền Iframe Bằng CSS3

```
<style>
  iframe {
      border: none;
      border-radius: 8px;
      box-shadow: 0 4px 12px rgba(0,0,0,0.1);
  }
</style>
```

BROWSER PREVIEW **https://example.com (style="border: none; border-radius: 8px;")**

Giao Diện Khung Iframe Đã Loại Bỏ Viền Bằng CSS

Đường viền 3D thô cứng bị xóa hoàn toàn, thay bằng góc bo 8px và bóng mờ nhẹ sang trọng.

3.12.7 Bảng Đánh Giá Ưu Điểm & Nhược Điểm Của <iframe>

[Bảng] Bảng Tổng Hợp Đánh Giá Ưu Điểm Và Nhược Điểm Của <iframe>

Ưu Điểm Nổi Bật	Nhược Điểm & Hạn Chế
Nhúng dễ dàng nội dung độc lập từ website bên ngoài	Một số trang web chặn nhúng bằng Header `X-Frame-Options`
Hỗ trợ trình chiếu Video YouTube, Bản đồ Maps, Tài liệu PDF	Làm tăng số lượng HTTP Requests, ảnh hưởng hiệu năng tải trang
Kiểm soát phân quyền bảo mật độc lập bằng `sandbox`	Tiềm ẩn rủi ro tấn công Clickjacking nếu không kiểm soát

3.12.8 Chuyên Sâu Thuộc Tính name & Kỹ Thuật Điều Hướng target

1. Định nghĩa thuộc tính name trong <iframe>:

Thuộc tính `name` được sử dụng để đặt danh xưng cho khung iframe. Mục đích chính là làm điểm nhận kết quả (**Target Destination**) cho các thẻ liên kết `<a target="...">` hoặc thẻ biểu mẫu `<form target="...">`, giúp nạp nội dung mới vào trong iframe mà **KHÔNG CẦN TẢI LẠI TRANG CHÍNH**.

2. Ví dụ 1: Điều hướng nội dung tìm kiếm Cốc Cốc vào iframe Bing:

Mã Nguồn Liên Kết Thẻ <a> Với name Của <iframe>

```
<!-- Khung Iframe hiển thị Bing mặc định, có tên name="web_bing" -->
<iframe src="https://www.bing.com/" name="web_bing" width="1000" height="600"><
    /iframe>

<!-- Thẻ a trỏ target tới name="web_bing" của iframe -->
<p>
  <a href="https://coccoc.com/search" target="web_bing">
    Tìm kiếm trên Cốc Cốc
  </a>
</p>
```

BROWSER PREVIEW https://www.bing.com (name="web_bing")

Khung Iframe nhận tín hiệu điều hướng: name="web_bing"

Ban đầu hiển thị: https://www.bing.com/

→ Khi nhấp liên kết bên dưới, nội dung tải Cốc Cốc trực tiếp vào khung này!

Tìm kiếm trên Cốc Cốc (target="web_bing")

Phân tích trình tự hoạt động chi tiết:

BƯỚC 1 Khi tải trang ban đầu, iframe nạp trang Bing (`src="https://www.bing.com/"`).

BƯỚC 2 Người dùng nhấp vào đường link *"Tìm kiếm trên Cốc Cốc"*.

BƯỚC 3 Do thẻ `<a>` có thuộc tính `target="web_bing"`, trình duyệt tải trực tiếp nội dung Cốc Cốc vào khung iframe `name="web_bing"`.

Phân biệt thuộc tính target trong thẻ <a>:

- `target="_blank"`: Mở liên kết trong một Tab / Cửa sổ mới.
- `target="_self"`: Mở liên kết ngay tại Tab hiện tại (Mặc định).
- `target="tên_iframe"`: Mở liên kết bên trong khung iframe có `name="tên_iframe"`.

3. Ví dụ 2: Điều hướng bài viết Wikipedia trong Iframe:

Mã Nguồn Liên Kết Bài Viết Wikipedia Vào Frame wiki_frame

```
<iframe src="https://www.wikipedia.org/" name="wiki_frame" width="1000" height=
    "600"></iframe>

<p>
  <a href="https://vi.wikipedia.org/wiki/Lap_trinh" target="wiki_frame">
    Xem bài viết về Lập trình trên Wikipedia
  </a>
</p>
```

BROWSER PREVIEW **https://vi.wikipedia.org/wiki/Lap_trinh (name="wiki_frame")**

Nạp Bài Viết Lập Trình Vào Khung wiki_frame

Nạp bài viết Lập trình trực tiếp vào khung mà không làm chuyển trang chính.

Xem bài viết về Lập trình trên Wikipedia (target="wiki_frame")

4. Ví dụ 3: Gửi dữ liệu Form kết quả vào khung Iframe:

Mã Nguồn Gửi Kết Quả Submit Form Trực Tiếp Vào Iframe

```
<iframe name="formResult" width="600" height="300"></iframe>

<form action="https://www.example.com/search" target="formResult">
    <input type="text" name="query" placeholder="Nhập từ khóa...">
    <button type="submit">Tìm kiếm</button>
</form>
```

BROWSER PREVIEW **https://www.example.com/search?query=HTML5 (name="formResult")**

Kết Quả Submit Form Hiển Thị Trong Khung formResult

Khi bấm Tìm kiếm, trang kết quả hiển thị trực tiếp ngay trong khung mà không làm nạp lại trang hiện tại.

3.12.9 Phân Biệt Chi Tiết Thuộc Tính name Và id Trong `<iframe>`

[Bảng] Bảng So Sánh Chi Tiết Giữa Thuộc Tính name Và id Của <iframe>

Tiêu Chí So Sánh	Thuộc Tính name	Thuộc Tính id
Mục đích chính	Định danh iframe để làm đích đến cho thuộc tính `target`	Định danh duy nhất để thao tác với CSS hoặc JavaScript
Tính trùng lặp	Có thể lặp lại tên trên nhiều phần tử	**Tuyệt đối duy nhất** trên toàn trang
Tương tác target	**Sử dụng trực tiếp** với `<a target="...">`	Không thể dùng làm giá trị cho `target`
Thao tác JavaScript	Ít phổ biến	**Rất phổ biến** (`document.getElementById`)

[Khái Niệm] Ma Trận Khi Nào Dùng name Và Khi Nào Dùng id Của Iframe

- **Dùng name**: Khi muốn điều hướng liên kết `<a target="name">` hoặc kết quả Form `<form target="name">` vào bên trong iframe.
- **Dùng id**: Khi muốn áp dụng CSS (`#myFrame { border: none; }`) hoặc lập trình JavaScript (`document.getElementById("myFrame")`).

3.13 Chi Tiết Về Các Thẻ Semantic Trong HTML

3.13.1 Thẻ Semantic Là Gì?

Thẻ Semantic (thẻ có ý nghĩa) là các thẻ HTML mô tả ý nghĩa của nội dung bên trong nó thay vì chỉ đơn thuần là một khối (`<div>`) hoặc một dòng (`<span>`).

- **Bản chất kỹ thuật**: Cung cấp thông tin ngữ nghĩa rõ ràng cho cả máy tính (trình duyệt, công cụ tìm kiếm, trình đọc màn hình) và con người (lập trình viên).
- **Mục đích cốt lõi**: Giúp trình duyệt, công cụ tìm kiếm (SEO) và lập trình viên hiểu nội dung dễ dàng

hơn.

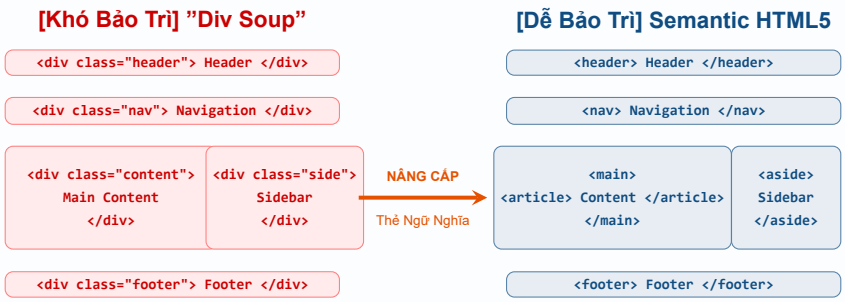
Ví Dụ So Sánh Trực Quan Giữa Thẻ Div Và Thẻ Semantic

[Bảng] So Sánh Cú Pháp: Non-Semantic (Div) vs. Semantic HTML5

Vùng Thành Phần	[Khó Bảo Trì] Cấu Trúc Div Soup	[Dễ Bảo Trì] Chuẩn Semantic HTML5
Phần đầu trang	`<div class="header">...</div>`	`<header>...</header>`
Thanh điều hướng	`<div class="nav">...</div>`	`<nav>...</nav>`
Nội dung chính	`<div class="content">...</div>`	`<main>...</main>`
Chân trang	`<div class="footer">...</div>`	`<footer>...</footer>`

Sơ Đồ Minh Họa Bố Cục Trang Web: Div Soup vs Semantic Landmarks

SƠ ĐỒ SO SÁNH KIẾN TRÚC TRANG WEB: KHÔNG SEMANTIC VS SEMANTIC HTML5



3.13.2 Lợi Ích Của Thẻ Semantic

Sử dụng thẻ Semantic mang lại 4 lợi ích lớn trong phát triển web chuyên nghiệp:

- **Tối ưu SEO**: Công cụ tìm kiếm dễ hiểu nội dung, giúp trang xếp hạng cao hơn trên kết quả tìm kiếm (Google, Bing).
- **Hỗ trợ truy cập (Accessibility - A11y)**: Người khiếm thị có thể sử dụng trình đọc màn hình (*Screen Readers*) để điều hướng trang web dễ dàng hơn.
- **Dễ bảo trì & nâng cấp**: Mã nguồn rõ ràng, dễ đọc, dễ hiểu cho lập trình viên khác khi tham gia vào dự án.
- **Giúp CSS dễ quản lý hơn**: Dùng đúng thẻ giúp giảm bớt class và ID không cần thiết trong tập tin CSS.

3.13.3 Danh Sách Các Thẻ Semantic Và Cách Sử Dụng

Dưới đây là tổng hợp đầy đủ danh sách các thẻ Semantic phổ biến trong HTML5, chức năng kỹ thuật và ví dụ cú pháp đại diện:

[Bảng] Danh Sách Các Thẻ Semantic HTML5 Và Ví Dụ Minh Họa

Thẻ Semantic	Chức năng	Ví dụ minh họa cú pháp
`<header>`	Phần đầu trang web (chứa logo, menu, tiêu đề).	`<header><h1>Header</h1> </header>`
`<nav>`	Thanh điều hướng chứa menu liên kết.	`<nav><a href="#">Home</a></nav>`
`<main>`	Nội dung chính của trang web.	`<main><h2>Nội dung</h2></main>`
`<section>`	Chia nội dung thành từng phần có chủ đề riêng.	`<section><h2>Giới thiệu</h2></section>`
`<article>`	Bài viết độc lập như blog, tin tức.	`<article><h2>Bài viết mới</h2></article>`
`<aside>`	Nội dung phụ như sidebar, quảng cáo.	`<aside><h3>Tin liên quan</h3></aside>`
`<footer>`	Chân trang, chứa bản quyền, liên hệ.	`<footer><p>© 2026</p></footer>`
`<address>`	Hiển thị thông tin liên hệ tác giả.	`<address>info@example.com </address>`
`<figure>`	Nhóm hình ảnh hoặc biểu đồ minh họa.	`<figure><img src="img.jpg"></figure>`
`<figcaption>`	Chú thích cho hình ảnh (dùng trong `<figure>`).	`<figcaption>Chú thích ảnh</figcaption>`
`<mark>`	Đánh dấu văn bản quan trọng (tô vàng).	`<mark>Nội dung quan trọng</mark>`
`<time>`	Hiển thị ngày/giờ chuẩn máy đọc.	`<time datetime="2026-07-26"> 26/07/2026</time>`
`<summary>`	Tiêu đề cho nội dung thu gọn `<details>`.	`<summary>Xem chi tiết</summary>`
`<details>`	Hiển thị nội dung thu gọn/mở rộng khi click.	`<details> <summary>Mở</summary> </details>`
`<dialog>`	Hộp thoại pop-up trong HTML5.	`<dialog open>Thông báo</dialog>`
`<cite>`	Trích dẫn nguồn tài liệu, bài báo.	`<cite>HTML Mastery</cite>`
`<code>`	Hiển thị đoạn mã lập trình.	`<code>console.log('Hi');</code>`

3.13.4 Ví Dụ Sử Dụng Thực Tế Trong Một Trang Web

Mã nguồn dưới đây minh họa cách kết hợp các thẻ Semantic để tạo dựng một cấu trúc trang web hoàn chỉnh tiêu chuẩn HTML5:

Mã Nguồn Trang Web HTML5 Hoàn Chỉnh Sử Dụng Thẻ Semantic

```
<!DOCTYPE html>
<html lang="vi">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Website với Semantic HTML</title>
</head>
<body>
    <header>
        <h1>Website của tôi</h1>
    </header>
    <nav>
        <a href="#">Trang chủ</a> | <a href="#">Giới thiệu</a> | <a href="#">Liên hệ</a>
    </nav>
    <main>
        <article>
            <h2>Bài viết đầu tiên</h2>
            <p>Đây là nội dung bài viết.</p>
        </article>
    </main>
    <aside>
        <h3>Bài viết liên quan</h3>
        <p>Thêm nội dung hữu ích.</p>
    </aside>
    <footer>
        <p>&copy; 2025 - Bản quyền thuộc về tôi.</p>
    </footer>
</body>
</html>
```

3.13.5 Tổng Kết: Vì Sao Nên Dùng Thẻ Semantic?

[Bảng] Tóm Tắt Lý Do Bắt Buộc Dùng Thẻ Semantic Trong Phát Triển Web

Lợi ích	Lý do
SEO tốt hơn	Công cụ tìm kiếm hiểu nội dung dễ dàng hơn.
Hỗ trợ người khuyết tật	Trình đọc màn hình đọc trang tốt hơn.
Dễ bảo trì code	Lập trình viên khác dễ hiểu cấu trúc trang web.
Dễ chỉnh sửa CSS	Nhắm đúng thẻ không cần nhiều class.

[Khái Niệm] Tóm Lại

Nên dùng thẻ Semantic thay vì `<div>` nếu có thể, vì nó giúp trang web rõ ràng, dễ hiểu, thân thiện với SEO & dễ bảo trì hơn!

3.13.6 Phân Tích Chi Tiết Danh Sách Một Số Thẻ Semantic Phổ Biến

1. Thẻ `<header>` – Phần Đầu Trang

Chức năng: Chứa tiêu đề, logo, menu điều hướng, hoặc thông tin đầu trang.

Mã Nguồn Mẫu Khai Báo Thẻ <header>

```
<header>
    <h1>Chào mừng đến với Website</h1>
    <nav>
        <ul>
            <li><a href="#">Trang chủ</a></li>
            <li><a href="#">Giới thiệu</a></li>
            <li><a href="#">Liên hệ</a></li>
        </ul>
    </nav>
</header>
```

2. Thẻ `<nav>` – Thanh Điều Hướng

Chức năng: Chứa danh sách liên kết để điều hướng trang web.

Mã Nguồn Mẫu Khai Báo Thẻ <nav>

```
<nav>
    <ul>
        <li><a href="#">Trang chủ</a></li>
        <li><a href="#">Sản phẩm</a></li>
        <li><a href="#">Liên hệ</a></li>
    </ul>
</nav>
```

[Ghi Chú] Lưu ý khi sử dụng thẻ <nav>

Nên dùng `<nav>` khi có nhóm liên kết chính, nếu chỉ có vài link nhỏ trong bài viết thì không cần.

3. Thẻ `<section>` – Phần Nội Dung

Chức năng: Dùng để chia nội dung thành từng phần riêng biệt.

Mã Nguồn Mẫu Khai Báo Thẻ <section>

```
<section>
    <h2>Sản phẩm nổi bật</h2>
    <p>Danh sách các sản phẩm bán chạy nhất...</p>
</section>
```

[Ghi Chú] Lưu ý khi sử dụng thẻ <section>

Dùng `<section>` khi nội dung có chủ đề riêng và có tiêu đề (`<h2>`, `<h3>`...). Nếu không có tiêu đề, có thể dùng `<div>` thay thế.

4. Thẻ `<article>` – Bài Viết Độc Lập

Chức năng: Dùng cho nội dung tự tồn tại (bài báo, blog, tin tức).

Mã Nguồn Mẫu Khai Báo Thẻ <article>

```
<article>
    <h2>Học HTML5 cơ bản</h2>
    <p>HTML5 mang lại nhiều tính năng mới giúp lập trình viên...</p>
</article>
```

[Khái Niệm] Phân biệt giữa <article> và <section>

- `<article>`: Nội dung có thể đọc độc lập (blog, tin tức).
- `<section>`: Chỉ là phần trong trang web, thường có nhiều bài viết nhỏ.

5. Thẻ `<aside>` – Nội Dung Bên Lề

Chức năng: Dùng cho quảng cáo, danh sách liên quan, thanh bên (sidebar).

Mã Nguồn Mẫu Khai Báo Thẻ <aside>

```
<aside>
    <h3>Bài viết liên quan</h3>
    <ul>
        <li><a href="#">HTML cơ bản</a></li>
        <li><a href="#">CSS nâng cao</a></li>
    </ul>
</aside>
```

[Ghi Chú] Lưu ý khi sử dụng thẻ <aside>

Nội dung không liên quan trực tiếp đến phần chính nhưng vẫn có ích.

6. Thẻ `<footer>` – Chân Trang

Chức năng: Chứa thông tin bản quyền, liên hệ, link quan trọng.

Mã Nguồn Mẫu Khai Báo Thẻ <footer>

```
<footer>
    <p>&copy; 2026 Bản quyền thuộc về Website của tôi</p>
    <p><a href="mailto:contact@example.com">Liên hệ</a></p>
</footer>
```

[Ghi Chú] Lưu ý khi sử dụng thẻ <footer>

Có thể có nhiều `<footer>` trên trang, ví dụ trong mỗi `<article>`.

7. Thẻ `<figure>` Và `<figcaption>` – Ảnh Minh Họa

Chức năng: Dùng để nhóm hình ảnh hoặc đồ họa với chú thích.

Mã Nguồn Mẫu Khai Báo Thẻ <figure> và <figcaption>

```
<figure>
    <img src="image.jpg" alt="Hình ảnh mô tả">
    <figcaption>Hình ảnh minh họa cho bài viết</figcaption>
</figure>
```

[Ghi Chú] Lưu ý khi sử dụng thẻ <figcaption>

`<figcaption>` là tùy chọn, giúp mô tả ảnh mà không cần thêm `<p>`.

3.13.7 Phân Tích Chuyên Sâu 4 Lý Do Vì Sao Nên Dùng Thẻ Semantic

Thẻ Semantic giúp trang web có cấu trúc rõ ràng hơn so với việc chỉ sử dụng `<div>`. Dưới đây là phân tích chi tiết 4 lý do chính:

1. Giúp Trình Duyệt & Công Cụ Tìm Kiếm (SEO) Hiểu Nội Dung Trang Web

- Google, Bing, Cốc Cốc và các công cụ tìm kiếm khác có thể đọc hiểu trang web dễ dàng hơn nếu sử dụng các thẻ Semantic.
- Giúp xếp hạng SEO tốt hơn vì công cụ tìm kiếm có thể phân loại nội dung chính, nội dung phụ.

[Bảng] So Sánh Giữa Khai Báo Không Rõ Ràng (Div) Và Khai Báo Semantic Rõ Ràng

Vùng Giao Diện	[Không Rõ Ràng] Dùng	[Rõ Ràng] Dùng Thẻ Semantic
Phần đầu trang	`<div class="top">...</div>`	`<header>...</header>`
Thanh menu	`<div class="nav">...</div>`	`<nav>...</nav>`
Nội dung chính	`<div class="content">...</div>`	`<main>...</main>`
Chân trang	`<div class="bottom">...</div>`	`<footer>...</footer>`

2. Cải Thiện Khả Năng Truy Cập (Accessibility - A11y) Cho Người Dùng Khuyết Tật

- Trình đọc màn hình (*screen readers*) có thể đọc hiểu nội dung dễ dàng hơn nếu dùng thẻ Semantic.
- Hỗ trợ người khiếm thị điều hướng trang web tốt hơn.

[Ghi Chú] Minh Họa Tác Động Truy Cập (A11y)

- Nếu có `<nav>`, trình đọc màn hình biết đây là thanh menu thay vì chỉ là một `<div>`.
- Nếu có `<article>`, họ biết đây là nội dung chính thay vì phải đoán.

3. Dễ Bảo Trì & Mở Rộng Code (Dễ Đọc Hiểu Hơn)

- Nếu dùng thẻ Semantic, lập trình viên mới vào dự án dễ hiểu code hơn.
- Khi sửa hoặc cập nhật nội dung, dễ dàng tìm đúng vị trí cần chỉnh sửa.

[Bảng] So Sánh Cấu Trúc Khó Bảo Trì Và Dễ Bảo Trì Trong Dự Án Web

Phần Tử Web	[Khó Bảo Trì] Dùng Div ID	[Dễ Bảo Trì] Dùng Thẻ Semantic
Khối tiêu đề	`<div id="header">...</div>`	`<header>...</header>`
Khối menu	`<div id="menu">...</div>`	`<nav>...</nav>`
Khối nội dung	`<div id="content">...</div>`	`<main>...</main>`
Khối chân trang	`<div id="footer">...</div>`	`<footer>...</footer>`

→ **Kết luận**: Dễ nhìn, dễ hiểu, dễ sửa đổi!

4. Giúp CSS Dễ Quản Lý Hơn

- CSS có thể target chính xác phần tử cần chỉnh sửa mà không cần đặt quá nhiều class hoặc ID.

[Bảng] So Sánh Cách Chọn Phần Tử CSS Với Div Class Và Thẻ Semantic Direct Target

Quy Trình Viết CSS	[Phức Tạp] Dùng Class Cho Div	[Tối Ưu] Target Thẻ Semantic
Khai báo CSS	`.top background: blue;`	`header background: blue;`
Cú pháp HTML	`<div class="top">...</div>`	`<header>...</header>`

→ **Kết luận**: Giảm bớt code CSS dư thừa!

[Ghi Chú] Tổng Kết Chung

- **SEO tốt hơn**: Công cụ tìm kiếm hiểu nội dung dễ dàng hơn.
- **Hỗ trợ người khuyết tật**: Trình đọc màn hình đọc trang tốt hơn.
- **Dễ bảo trì code**: Lập trình viên khác dễ hiểu cấu trúc trang web.
- **Dễ chỉnh sửa CSS**: Nhắm đúng thẻ không cần nhiều class.

Tóm lại: Nên dùng thẻ Semantic thay vì `<div>` nếu có thể, vì nó giúp trang web rõ ràng, dễ hiểu, thân thiện với SEO & dễ bảo trì hơn!

3.14 Đọc Thêm: Mở Rộng Kiến Thức Chuyên Sâu Về Semantic HTML5

Phần đọc thêm cung cấp các góc nhìn chuyên sâu về nguyên lý hoạt động của trình duyệt, tương quan với chuẩn WAI-ARIA, thuật toán phân cấp cây tài liệu và các câu hỏi phỏng vấn tuyển dụng Frontend thực tế liên quan đến Semantic HTML.

3.14.1 Sự Tương Quan Giữa Thẻ Semantic HTML5 Và WAI-ARIA Landmark Roles

Trước khi HTML5 ra đời, W3C đã giới thiệu bộ tiêu chuẩn WAI-ARIA (**Web Accessibility Initiative - Accessible Rich Internet Applications**) để bổ sung thuộc tính `role="..."` giúp trợ năng cho giao diện. Khi HTML5 chính thức được ban hành, các thẻ Semantic đã tự động tích hợp ngầm các ARIA Landmark Roles tương ứng:

[Bảng] Ma Trận Tương Quan Giữa Thẻ Semantic HTML5 Và WAI-ARIA Landmark Roles

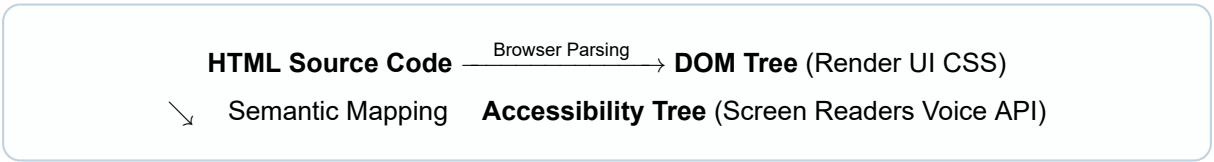
Thẻ Semantic HTML5	WAI-ARIA Role Tương Đương	Hành Vi Khả Năng Truy Cập (A11y Behavior)
`<header>`	`role="banner"`	Xác định vùng tiêu đề hàng đầu của trang web hoặc bài viết.
`<nav>`	`role="navigation"`	Đánh dấu vùng điều hướng chính chứa danh sách liên kết.
`<main>`	`role="main"`	Đánh dấu trung tâm nội dung độc nhất của toàn trang.
`<aside>`	`role="complementary`	Vùng bổ trợ thông tin, tách biệt khỏi luồng nội dung chính.
`<footer>`	`role="contentinfo"`	Chứa thông tin bản quyền, điều khoản dịch vụ và thông tin liên hệ.
`<section>`	`role="region"`	Vùng nội dung có chủ đề riêng (bắt buộc kết hợp thuộc tính aria-labelledby).
`<form>`	`role="search"` / `role="form"`	Đánh dấu vùng nhập liệu/tìm kiếm dữ liệu của người dùng.

[Cảnh Báo] Cảnh Báo Kỹ Thuật: Tránh Khai Báo Thừa ARIA Roles

Không nên khai báo trùng lặp như `<nav role="navigation">` hay `<main role="main">`. Đây là hành vi dư thừa mã (*Redundant ARIA*) vì các trình duyệt hiện đại đã tự động gắn ngầm Landmark Role mặc định cho các thẻ Semantic HTML5 này.

3.14.2 Accessibility Tree (Cây Trợ Năng) Và Trình Đọc Màn Hình (Screen Readers)

Khi trình duyệt tải tài liệu HTML, bên cạnh việc xây dựng cây **DOM (Document Object Model)**, trình duyệt còn song song tạo ra một cây phụ gọi là **Accessibility Tree (Cây Trợ Năng)**.



Các phần mềm đọc màn hình phổ biến như **NVDA**, **JAWS** (trên Windows) hoặc **VoiceOver** (trên macOS/iOS) truy cập vào Cây Trợ Năng này để cung cấp phím tắt điều hướng nhanh cho người khiếm thị:

- Nhấn phím `D` để nhảy nhanh qua các vùng Landmark (`<header>`, `<nav>`, `<main>`, `<footer>`).
- Nhấn phím `H` để nhảy tuần tự qua các thẻ tiêu đề Heading (`<h1>` đến `<h6>`).
- Nhấn phím `L` để nhảy đến danh sách liên kết (`<nav>`).

Nếu trang web viết hoàn toàn bằng `<div>`, Cây Trợ Năng sẽ hoàn toàn phẳng và người khuyết tật buộc

phải đọc lướt từng từ một từ đầu đến cuối trang web.

3.14.3 Thuật Toán Phân Mạch Nội Dung (HTML5 Outlining Algorithm)

Trong đặc tả ban đầu của HTML5, W3C định nghĩa thuật toán **HTML5 Document Outlining Algorithm**. Theo đó, mỗi khi xuất hiện các phần tử bao bọc ngữ cảnh (**Sectioning Content**) như `<article>`, `<section>`, `<aside>`, `<nav>`, một phân cấp tiêu đề mới được thiết lập.

Tuy nhiên, các nhà phát triển trình duyệt (Chrome, Firefox, Safari) đã không triển khai đầy đủ thuật toán này. Vì vậy, **Best Practice hiện đại** được khuyến nghị bởi W3C và MDN là:

- Mỗi trang web chỉ nên có duy nhất một thẻ `<h1>` dành cho tiêu đề chính đại diện trang.
- Tuân thủ chính xác thứ tự phân cấp tiêu đề giảm dần: `<h1>` → `<h2>` → `<h3>` → `<h4>`, tuyệt đối không nhảy cấp từ `<h2>` xuống `<h4>` chỉ vì lý do kích thước phông chữ CSS.

3.14.4 Những Lỗi Phổ Biến Và Anti-Patterns Cần Tránh Khi Dùng Thẻ Semantic

[Cảnh Báo] Top 4 Lỗi Sai Thường Gặp Của Lập Trình Viên Frontend Mới

1. **Dùng thẻ <main> nhiều lần trên một trang**: Thẻ `<main>` đại diện cho nội dung độc nhất của tài liệu. Chỉ được có duy nhất một thẻ `<main>` hiển thị trên một trang web (trừ trường hợp các thẻ `<main>` khác có thuộc tính `hidden`).
2. **Lạm dụng thẻ <section> để thay thế cho **: `<section>` dùng để nhóm nội dung có chung chủ đề và **phải có tiêu đề** (`<h2>`-`<h6>`). Nếu chỉ dùng để bọc khối nhằm mục đích căn chỉnh CSS layout hoặc style flexbox/grid, bắt buộc sử dụng thẻ `<div>`.
3. **Dùng thẻ <article> cho các khối giao diện nhỏ không độc lập**: Thẻ `<article>` chỉ dùng khi nội dung đó có thể tách ra đứng độc lập và phân phối lại trên hệ thống khác (RSS Feed, mạng xã hội, tin tức).
4. **Sử dụng sai thẻ <address>**: Thẻ `<address>` chỉ dùng để cung cấp thông tin liên hệ của tác giả/chủ sở hữu bài viết hoặc website, không dùng để ghi địa chỉ bưu điện thông thường không liên quan đến tác giả.

3.14.5 Góc Phỏng Vấn Frontend: Các Câu Hỏi Thường Gặp Về Semantic HTML

[Khái Niệm] Bộ Câu Hỏi Phỏng Vấn Tuyển Dụng (Frontend Interview Q&A)

Câu 1: Phân biệt sự khác nhau giữa <section> và <article>? Khi nào nên dùng thẻ nào?

Trả lời: `<article>` đại diện cho một khối nội dung độc lập, tự có nghĩa và có thể tái sử dụng độc lập ở nơi khác (ví dụ: bài viết blog, bình luận, thẻ sản phẩm). `<section>` đại diện cho một phần nội dung gom nhóm theo chủ đề trong cùng một trang hoặc trong một article (ví dụ: phần giới thiệu, phần tính năng, phần liên hệ). Một `<article>` có thể chứa nhiều `<section>` và ngược lại một `<section>` cũng có thể chứa nhiều `<article>`.

Câu 2: Tại sao việc thay thế bằng Semantic Tags lại cải thiện chỉ số SEO?

Trả lời: Bot tìm kiếm của Google (Googlebot) sử dụng các thẻ Semantic để phân bổ trọng số từ khóa chính xác. Nội dung nằm trong `<main>` và `<article>` được đánh giá điểm ưu tiên cao nhất, trong khi nội dung trong `<aside>` hay `<footer>` được hạ trọng số. Việc sử dụng Semantic Tags giúp Bot hiểu chính xác cấu trúc cây dữ liệu mà không cần đoán dựa trên tên lớp CSS.

Câu 3: Thẻ <figure> và <figcaption> mang lại lợi ích gì hơn so với <img> đi kèm thẻ <p>?

Trả lời: Thẻ `<figure>` liên kết về mặt ngữ nghĩa giữa hình ảnh (hoặc sơ đồ, đoạn code) với phần văn bản chú thích `<figcaption>`. Khi Trình đọc màn hình đọc đến phần tử này, nó sẽ thông báo cho người dùng biết đoạn văn bản chú thích đó thuộc về hình ảnh tương ứng, thay vì đọc rời rạc như hai phần tử độc lập.

3.14.6 Nhóm Thẻ Semantic Mức Văn Bản (Text-Level / Inline Semantics)

Bên cạnh nhóm thẻ Landmark quản lý bố cục khối, HTML5 cung cấp bộ thẻ ngữ nghĩa mức văn bản (**Text-level Semantics**) giúp đánh dấu chính xác ý nghĩa của từng từ, từng ký tự trong câu:

[Bảng] Phân Tích Nhóm Thẻ Semantic Mức Văn Bản Chi Tiết

Thẻ Semantic	Ý nghĩa ngữ nghĩa	Ví dụ cú pháp & Chuẩn kỹ thuật
`<dfn>`	Đánh dấu thuật ngữ lần đầu xuất hiện để định nghĩa.	`Khái niệm <dfn>DOM</dfn> là...`
`<abbr>`	Từ viết tắt kèm giải thích đầy đủ qua `title`.	`<abbr title="HyperText Markup Language">HTML</abbr>`
`<kbd>`	Tổ hợp phím nhập từ bàn phím người dùng.	`Nhấn <kbd>Ctrl</kbd> + <kbd>S</kbd>`
`<samp>`	Kết quả đầu ra mẫu của chương trình/máy tính.	`Hệ thống: <samp>500 Internal Error</samp>`
`<var>`	Tên biến số trong toán học hoặc mã lập trình.	`Phương trình <var>y</var> = <var>f</var>(<var>x</var>)`
`<data>`	Gắn giá trị máy đọc được (`value`) với chữ.	`<data value="15000000">15 triệu</data>`
`<time>`	Thời gian máy đọc được (chuẩn ISO-8601).	`<time datetime="2026-07-26">26/07/2026</time>`
`<del>` / `<ins>`	Văn bản bị xóa / Văn bản được chèn bổ sung.	`<del>500k</del> <ins>400k</ins>`
`<bdi>` / `<bdo>`	Đảo chiều văn bản / Cách ly văn bản đa ngôn ngữ.	`<bdo dir="rtl">Text</bdo>`

Mã Nguồn Mẫu Kết Hợp Các Thẻ Semantic Mức Văn Bản Nội Dòng

```
<p>
  Trong lap trinh Web, khai niem <dfn><abbr title="Document Object Model">DOM</abbr></dfn> la rat quan trong.
  Nguoi dung nhan to hop phim <kbd>Ctrl</kbd> + <kbd>Shift</kbd> + <kbd>I</kbd> de mo DevTools.
  Ket qua man hinh hien thi: <samp>200 OK</samp>.
  Thoi gian dang bai: <time datetime="2026-07-26T00:17:36+07:00">26/07/2026 00:17</time>.
  Khuyen mai tu <del datetime="2026-07-01">200.000d</del> giảm xuống còn <ins datetime="2026-07-26"><data value="150000">150.000đ</data></ins>.
</p>
```

3.14.7 3 Mô Hình Kiến Trúc Bố Cụ Bằng Semantic Tags Trong Thực Tế

Mô Hình 1: Kiến Trúc Trang Bài Báo / Blog Công Nghệ (Article & Comments)

Cấu Trúc Trang Bài Viết Công Nghệ Độc Lập Kèm Bình Luận Lồng Nhau

```
<main>
  <article>
    <header>
      <h1>Huong Dan Lam Chu Semantic HTML5 Tu A Den Z</h1>
      <p>Dang ngay: <time datetime="2026-07-26">26/07/2026</time></p>
      <address>Tac gia: <a href="mailto:admin@example.com">Nguyen Van A</a><
    /address>
    </header>

    <section>
      <h2>1. Gioi thieu tong quan</h2>
      <p>Semantic HTML5 giup cau truc trang web mach lac...</p>
      <figure>
        <img src="semantic-diagram.png" alt="So do Semantic">
        <figcaption>Hinh 1: Kien truc Semantic HTML5 Landmarks</figcaption>
      </figure>
    </section>

    <footer>
      <p>Danh muc: <mark>Frontend Core</mark></p>
    </footer>

    <!-- Khu vuc binh luan (moi comment la mot article doc lap) -->
    <section>
      <h3>Binh luan tu doc gia</h3>
      <article>
        <header><b>Doc gia B</b> vao <time datetime="2026-07-26T09:00">09:00<
    /time></header>
        <p>Bai viet rat chi tiet va de hieu!</p>
      </article>
    </section>
  </article>
</main>
```

Mô Hình 2: Trang Chi Tiết Sản Phẩm Thương Mại Điện Tử (E-Commerce Product)

Cấu Trúc Trang Chi Tiết Sản Phẩm Thương Mại Điện Tử

```html
<main>
  <article class="product-detail">
    <header>
      <h1>Laptop Macbook Pro M3 Max</h1>
      <p>Thuong hieu: <cite>Apple</cite></p>
    </header>

    <figure class="product-gallery">
      <img src="macbook-front.jpg" alt="Macbook Pro goc chinh dien">
      <figcaption>Hinh anh Macbook Pro M3 Max chinh hang</figcaption>
    </figure>

    <section class="pricing">
      <h2>Thong Tin Gia Ban</h2>
      <p>Gia goc: <del>55.000.000d</del></p>
      <p>Gia khuyen mai: <ins><data value="49990000">49.990.000d</data></ins></p>
      <p>Danh gia: <meter min="0" max="5" value="4.8">4.8 / 5 sao</meter></p>
    </section>

    <section class="specifications">
      <h2>Thong So Ky Thuat</h2>
      <ul>
        <li>Chipset: <var>Apple M3 Max</var></li>
        <li>RAM: <data value="36">36 GB Unified Memory</data></li>
      </ul>
    </section>
  </article>
</main>
```

Mô Hình 3: Giao Diện Ứng Dụng Web / SaaS Dashboard Layout

Cấu Trúc Giao Diện Web Application / SaaS Dashboard

```html
<!DOCTYPE html>
<html lang="vi">
<head><title>Admin SaaS Dashboard</title></head>
<body>
  <!-- Thanh App Header tren cung -->
  <header class="app-header">
    <h1>He Thong Quan Tri Enterprise</h1>
    <address>User Login: <a href="#">admin@company.com</a></address>
  </header>

  <div class="dashboard-wrapper">
    <!-- Thanh dieu huong Sidebar ben trai -->
    <nav class="app-sidebar">
      <ul>
        <li><a href="#analytics">Thong ke</a></li>
        <li><a href="#users">Nguoi dung</a></li>
        <li><a href="#settings">Cau hinh</a></li>
      </ul>
    </nav>

    <!-- Trung tam noi dung Dashboard -->
    <main class="app-content">
      <section id="analytics">
        <h2>Bao Cao Doanh Thu</h2>
        <p>Doanh thu thang nay: <data value="1200000000">1.2 Ty VND</data></p>
      </section>
    </main>

    <!-- Sidebar thong bao phu ben phai -->
    <aside class="notifications-panel">
      <h3>Thong Bao He Thong</h3>
      <p>Cap nhat phien ban <time datetime="2026-07-26">v2.4.0</time> thanh cong.
    </p>
    </aside>
  </div>

  <!-- Chan trang Status Bar -->
  <footer class="app-statusbar">
    <p>Trang thai he thong: <mark style="background-color: lightgreen;">ONLINE<
    /mark></p>
  </footer>
</body>
</html>
```

3.14.8 Tích Hợp Semantic HTML Với Dữ Liệu Có Cấu Trúc SEO (Schema.org & Microdata)

Để đạt điểm SEO tuyệt đối trên các công cụ tìm kiếm, trình duyệt không chỉ đọc thẻ Semantic mà còn giải mã bộ thuộc tính dữ liệu có cấu trúc (**Microdata Structured Data**) theo chuẩn **Schema.org**:

Mã Nguồn Kết Hợp Thẻ Semantic Với Microdata Chuẩn Schema.org

```
<article itemscope itemtype="https://schema.org/TechArticle">
  <header>
    <h1 itemprop="headline">Tổng Quan HTML5 Semantic Masterclass</h1>
    <p>Đăng
    bởi <span itemprop="author" itemscope itemtype="https://schema.org/Person">
      <span itemprop="name">Nguyễn Văn A</span>
    </span></p>
    <time itemprop="datePublished" datetime="2026-07-26">26/07/2026</time>
  </header>

  <section itemprop="articleBody">
    <p>Nội dung bài viết về Semantic HTML5...</p>
  </section>
</article>
```

[Bảng] Ma Trận Ánh Xạ Giữa Thẻ Semantic HTML5 Và Đối Tượng Dữ Liệu Schema.org

Thẻ Semantic HTML5	Schema.org Itemtype	Lợi Ích SEO Rich Snippets
`<article>`	`schema.org/Article`	Hiển thị khung tin tức nổi bật trên Google SERP.
`<figure> + <figcaption>`	`schema.org/ImageObject`	Lập chỉ mục ảnh chuẩn trong Google Images Search.
`<address>`	`schema.org/PostalAddress`	Hiển thị Google Maps Local Business Knowledge Graph.
`<main>`	`schema.org/MainContentPart`	Giúp Googlebot xác định nội dung chính để xếp hạng SEO.

3.14.9 Hướng Dẫn Debug Cây Trợ Năng (Accessibility Panel Inspection)

Để kiểm tra các thẻ Semantic HTML5 có hoạt động chuẩn xác trên trình duyệt Chrome hay không, kỹ sư Frontend thực hiện các bước sau:

QUY TRÌNH 4 BƯỚC DEBUG ACCESSIBILITY TREE TRÊN CHROME DEVTOOLS

1. **Bước 1**: Nhấp chuột phải vào phần tử trên trang web → chọn `Inspect` (hoặc nhấn phím `F12`).
2. **Bước 2**: Chọn tab `Elements` trên thanh công cụ Chrome DevTools.
3. **Bước 3**: Tại khung bên phải, chọn tab phụ `Accessibility`.
4. **Bước 4**: Quan sát ô **Computed Properties** để xem các giá trị:
   - `Role`: Vai trò ngữ nghĩa trình duyệt nhận diện (ví dụ: `banner`, `navigation`, `main`).
   - `Name`: Tên phần tử đọc bởi Screen Reader (ví dụ: lấy từ thuộc tính `aria-label` hoặc văn bản thẻ).
   - `Description`: Mô tả bổ sung chi tiết của phần tử.

3.14.10 Bộ Thẻ Semantic HTML5 Hiện Đại Mới Nhất (<search>, <hgroup>, <meter> vs <progress>)

Đặc tả WHATWG và W3C liên tục cập nhật bộ thẻ Semantic để đáp ứng sự phát triển của các ứng dụng web hiện đại. Dưới đây là phân tích kỹ thuật 3 nhóm thẻ ngữ nghĩa nâng cao quan trọng:

1. Thẻ <search> – Chuẩn Ngữ Nghĩa Cho Khung Tìm Kiếm (HTML Specification 2023)

Trước năm 2023, để đánh dấu khu vực tìm kiếm chuẩn A11y, lập trình viên phải gắn thuộc tính `role="search"` thủ công vào thẻ `<form>` hoặc `<div>`. Thẻ `<search>` chính thức ra đời nhằm cung cấp một Landmark Element native đại diện cho chức năng tìm kiếm:

Mã Nguồn Mẫu Khai Báo Thẻ <search> Chuẩn Modern HTML

```
<!-- Bao bọc khu vực tìm kiếm chính của website hoặc ứng dụng -->
<search>
  <form action="/search" method="get">
    <label for="query">Tìm kiếm bài viết:</label>
    <input type="search" id="query" name="q" placeholder="Nhập từ khóa...">
    <button type="submit">Tìm kiếm</button>
  </form>
</search>
```

[Ghi Chú] Lợi Ích Của Thẻ <search>

Trình duyệt tự động gán Landmark Role `search` vào Accessibility Tree, giúp Screen Readers nhảy nhanh tới ô tìm kiếm khi người dùng nhấn phím tắt điều hướng mà không cần khai báo thuộc tính ARIA bổ sung.

2. Thẻ <hgroup> – Nhóm Tiêu Đề Và Tagline/Subtitle

Khi một bài viết hoặc phần tử có tiêu đề chính (`<h1>`) đi kèm tiêu đề phụ, khẩu hiệu hoặc tagline, việc đặt thẻ tiêu đề thứ hai (`<h2>`) trực tiếp bên dưới có thể làm sai lệch thuật toán phân cấp tiêu đề (Outlining

Algorithm). Thẻ `<hgroup>` giải quyết triệt để vấn đề này bằng cách gom nhóm chúng lại:

Mã Nguồn Mẫu Sử Dụng Thẻ <hgroup>

```
<hgroup>
  <h1>Giáo Trình HTML5 Masterclass</h1>
  <p>Hướng Dẫn Biên Soạn Chuyên Sâu Từ Cơ Bản Đến Nâng Cao Cho Frontend
    Developer</p>
</hgroup>
```

3. Ma Trận Phân Biệt Giữa Thẻ `<meter>` Và `<progress>`

Hai thẻ `<meter>` và `<progress>` thường bị nhầm lẫn do giao diện mặc định có dạng thanh ngang trên trình duyệt. Tuy nhiên, về mặt ngữ nghĩa kỹ thuật, chúng phục vụ hai mục đích hoàn toàn riêng biệt:

[Bảng] Ma Trận So Sánh Kỹ Thuật Giữa Thẻ <meter> Và <progress>

Thẻ Semantic	Ngữ Nghĩa Kỹ Thuật	Ngữ Cảnh Sử Dụng Thực Tế
`<progress>`	Biểu thị tiến độ hoàn thành của một tác vụ đang diễn ra (Dynamic Progress).	Tiến trình tải tập tin, tiến độ hoàn tất khóa học, thanh hoàn thành các bước thanh toán checkout.
`<meter>`	Đo lường một giá trị định lượng nằm trong một khoảng cố định (Scalar Gauge).	Dung lượng đĩa cứng đã dùng (80GB/100GB), mức dung lượng pin (45%), điểm đánh giá chất lượng (4.5/5 sao).

3.14.11 Kỹ Thuật Responsive Image Semantics Với Thẻ `<picture>`

Song song với thẻ `<figure>` và `<figcaption>`, thẻ `<picture>` đại diện cho ngữ nghĩa trình diễn hình ảnh đáp ứng (**Responsive Image Semantics**). Thẻ này cho phép các kỹ sư Web thực hiện kỹ thuật **Art Direction** và tự động thương lượng định dạng ảnh tối ưu (**Format Negotiation**):

Mã Nguồn Mẫu Sử Dụng Thẻ <picture> Tối Ưu Định Dạng Ảnh Và Viewport

```
<figure>
  <picture>
    <!-- Ưu tiên định dạng AVIF thế hệ mới cho màn hình lớn Desktop -->
    <source srcset="hero-large.avif" type="image/avif" media="(min-width: 1024px)
    ">
    <!-- Fallback định dạng WebP cho màn hình Tablet -->
    <source srcset="hero-medium.webp" type="image/webp" media="(min-width: 768px)
    ">
    <!-- Fallback mặc định cho trình duyệt cũ hoặc màn hình Mobile -->
    <img src="hero-small.jpg" alt="Hình ảnh minh họa giao diện ứng dụng">
  </picture>
  <figcaption>Hình ảnh mô tả hệ thống responsive tương thích đa thiết
    bị</figcaption>
</figure>
```

3.14.12 Cấu Trúc Hộp Thoại Modal Native Với Thẻ `<dialog>`

Thẻ `<dialog>` cung cấp giải pháp native cho các hộp thoại pop-up, alert hoặc modal mà không cần xây dựng custom modal bằng `<div>` phức tạp:

Mã Nguồn Mẫu Khai Báo Thẻ <dialog> Native

```
<!-- Thẻ dialog đóng vai trò modal popup -->
<dialog id="confirmModal">
  <form method="dialog">
    <h2>Xác Nhận Thao Tác</h2>
    <p>Bạn có chắc chắn muốn xóa dữ liệu này không?</p>
    <button value="cancel">Hủy bỏ</button>
    <button value="confirm" value="default">Xác nhận</button>
  </form>
</dialog>
```

[Khái Niệm] Điểm Cốt Lõi Về Trợ Năng (A11y) Của Thẻ <dialog>

Khi kích hoạt modal bằng phương thức JavaScript `dialog.showModal()`:

- Trình duyệt tự động đưa hộp thoại vào **Top-Layer Stack** (hiển thị trên cùng toàn bộ giao diện mà không bị ảnh hưởng bởi z-index).
- Tự động tạo phần tử giả `::backdrop` làm tối nền xung quanh.
- Kích hoạt cơ chế **Native Focus Trap**: Con trỏ bàn phím (phím `Tab`) bị giữ chặt bên trong modal, không thể thoát ra các phần tử nền dưới.
- Cho phép người dùng nhấn phím `ESC` để đóng modal nhanh chóng.

3.15 Tóm Tắt Chương 3

[Ghi Chú] Tổng Kết Cốt Lõi Chương 3

HTML là ngôn ngữ nền tảng cốt lõi trong phát triển web. Việc làm chủ cấu trúc tài liệu HTML5, bộ thẻ ngữ nghĩa (Semantic HTML5 Masterclass), thực thể HTML, định dạng Bảng (Table) chuẩn mực và kết hợp đúng đắn với bộ thẻ `<meta>` (tối ưu mã hóa UTF-8, Responsive Viewport, mô tả SEO và thẻ Open Graph mạng xã hội) sẽ giúp website vận hành chuẩn mực, đạt thứ hạng cao trên các công cụ tìm kiếm và mang lại trải nghiệm xuất sắc cho người dùng.

CHƯƠNG 4

Thẻ Bố Cục Ngữ Nghĩa (Layout Semantics) & Thẻ Văn Bản

Chương này trình bày chi tiết từng thẻ thuộc nhóm Bố cục ngữ nghĩa và nhóm Văn bản trong HTML5.

4.1 Thẻ Thân Tài Liệu <body>

4.1.1 1. Lý Thuyết & Thuộc Tính Sự Kiện Window

Thẻ <body> chứa toàn bộ nội dung trực quan hiển thị trên cửa sổ trình duyệt (Văn bản, Hình ảnh, Video, Biểu mẫu).

- Các thuộc tính sự kiện cửa sổ: onload, onunload, onresize, onpopstate, ononline, onoffline.

4.1.2 2. Ví Dụ Mã Nguồn Thực Tế

Mã Nguồn: Sử Dụng Thẻ body Với Sự Kiện Trạng Thái Mạng

```
<body ononline="console.log('Đã kết nối Internet')" onoffline="alert('Mất kết nối
    mạng!')">
  <h1>Nội dung chính hiển thị tại đây</h1>
</body>
```

BROWSER PREVIEW — Kết Quả Hiển Thị Trình Duyệt: Thẻ <body>

Nội dung chính hiển thị tại đây

4.2 Thẻ Đỉnh Trang / Đỉnh Phân Đoạn <header>

4.2.1 1. Lý Thuyết & Quy Tắc Sử Dụng

Thẻ <header> đại diện cho nhóm các phần tử dẫn nhập hoặc điều hướng. Một trang web có thể chứa nhiều thẻ <header> (một ở đỉnh trang web và các thẻ <header> con nằm bên trong <article> hoặc <section>).

4.2.2 2. Ví Dụ Mã Nguồn Thực Tế

Mã Nguồn: Thẻ Header Đỉnh Trang Web Chuẩn SEO

```
<header>
  <a href="/" class="logo">
    <img src="logo.svg" alt="EduWeb Academy">
  </a>
  <nav>
    <ul>
      <li><a href="/courses">Khóa Học</a></li>
      <li><a href="/blog">Bài Viết</a></li>
    </ul>
  </nav>
</header>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Thẻ Header Đỉnh Trang**

[LOGO] EduWeb Academy Khóa Học Bài Viết

4.3 Thẻ Thanh Điều Hướng <nav>

4.3.1 1. Lý Thuyết & Accessibility (a11y)

Thẻ <nav> đại diện cho một phần của trang web chứa các liên kết điều hướng chính (Navigation Links). Trình đọc màn hình (Screen Reader) sử dụng thẻ này để giúp người khiếm thị nhảy nhanh đến menu chính.

4.3.2 2. Ví Dụ Mã Nguồn Thực Tế

Mã Nguồn: Menu Điều Hướng Phân Tầng Với Thẻ Nav

```
<nav aria-label="Thanh điều hướng chính">
  <ul>
    <li><a href="/" aria-current="page">Trang Chủ</a></li>
    <li><a href="/about">Giới Thiệu</a></li>
    <li><a href="/contact">Liên Hệ</a></li>
  </ul>
</nav>
```

BROWSER PREVIEW Kết Quả Hiển Thị Trình Duyệt: Thanh Điều Hướng Nav

Trang Chủ • Giới Thiệu • Liên Hệ

4.4 Thẻ Nội Dung Chính <main>

4.4.1 1. Lý Thuyết & Quy Tắc Độc Nhất

Thẻ <main> chứa nội dung độc nhất và trung tâm của tài liệu.

[Cảnh Báo] Quy Tắc Vàng Thẻ <main>

Trong một tệp HTML chỉ được phép có **duy nhất 1 thẻ <main> hiển thị**. Thẻ <main> KHÔNG được là phần tử con của <header>, <nav>, <article>, <aside> hay <footer>.

4.4.2 2. Ví Dụ Mã Nguồn Thực Tế

Mã Nguồn: Khối Nội Dung Chính Độc Nhất <main>

```
<main id="main-content">
  <h1>Khóa Học HTML5 Masterclass</h1>
  <p>Nội dung chi tiết khóa học...</p>
</main>
```

BROWSER PREVIEW Kết Quả Hiển Thị Trình Duyệt: Khối Nội Dung Chính Main

Khóa Học HTML5 Masterclass

Nội dung chi tiết khóa học...

4.5 Thẻ Bài Viết Độc Lập <article>

4.5.1 1. Lý Thuyết & Tiêu Chuẩn Tách Tải

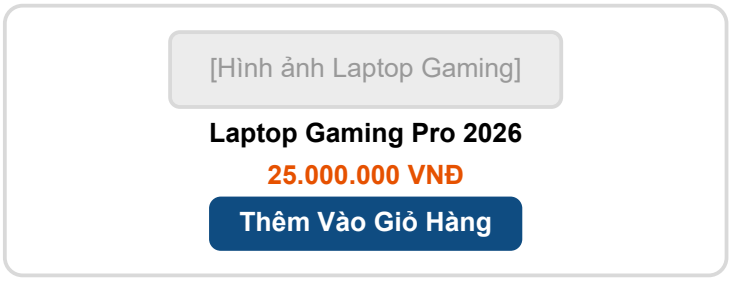
Thẻ <article> đại diện cho một thành phần hoàn chỉnh, độc lập trong trang web mà có thể phân phối hoặc tái sử dụng độc lập (ví dụ: bài báo, sản phẩm thương mại điện tử, bài đăng blog, bình luận).

4.5.2 2. Ví Dụ Mã Nguồn Thực Tế

Mã Nguồn: Thẻ Article Mô Phỏng UI Card Sản Phẩm Độc Lập

```
<article class="product-card">
  <img src="laptop.jpg" alt="Laptop Gaming">
  <h2>Laptop Gaming Pro 2026</h2>
  <p class="price">25.000.000 VNĐ</p>
  <button>Thêm Vào Giỏ Hàng</button>
</article>
```

BROWSER PREVIEW Kết Quả Hiển Thị Trình Duyệt: UI Card Sản Phẩm Article



4.6 Thẻ Phân Đoạn Chủ Đề <section>

4.6.1 1. Lý Thuyết & So Sánh <section> vs <article>

Thẻ <section> đại diện cho một phân đoạn độc lập chứa các nội dung cùng chung một chủ đề. Mỗi <section> luôn luôn nên đi kèm với một thẻ tiêu đề (<h2>-<h6>).

4.6.2 2. Ví Dụ Mã Nguồn Thực Tế

Mã Nguồn: Phân Đoạn Các Section Trong Bài Viết

```
<section id="introduction">
  <h2>1. Giới Thiệu Căn Bản</h2>
  <p>Nội dung giới thiệu...</p>
</section>

<section id="features">
  <h2>2. Các Tính Năng Nổi Bật</h2>
  <p>Nội dung tính năng...</p>
</section>
```

BROWSER PREVIEW Kết Quả Hiển Thị Trình Duyệt: Các Phân Đoạn Section

1. Giới Thiệu Căn Bản

Nội dung giới thiệu...

2. Các Tính Năng Nổi Bật

Nội dung tính năng...

[Bảng] Bảng So Sánh Ngữ Nghĩa: <article> vs <section> vs

Thẻ HTML	Ý Nghĩa Ngữ Nghĩa (Semantics)	Đặc Điểm Độc Lập Nội Dung	Trường Hợp Khuyên Dùng
`<article>`	Thành phần nội dung hoàn chỉnh, độc lập.	**Có thể phân phối độc lập** (Bản tin RSS, Widget, Card).	Bài viết blog, Tin tức, Sản phẩm eCommerce, Bình luận.
`<section>`	Phân đoạn gom nhóm các ý thuộc cùng một chủ đề.	Cần đi kèm tiêu đề (`<h2>`-`<h6>`) để tạo Outline.	Các mục trong trang (Giới thiệu, Tính năng, Báo giá, FAQ).
`<div>`	**Thẻ rỗng vô nghĩa** (Pure Container).	Không mang bất kỳ ý nghĩa ngữ nghĩa nào.	Phục vụ định kiểu CSS Flexbox/Grid hoặc JavaScript DOM.

4.7 Thẻ Thanh Bên Phụ <aside>

4.7.1 1. Lý Thuyết & Mục Đích Sử Dụng

Thẻ `<aside>` chứa các nội dung liên quan gián tiếp đến nội dung chính xung quanh (như thanh bên Sidebar, bài viết liên quan, quảng cáo, ô trích dẫn).

4.7.2 2. Ví Dụ Mã Nguồn Thực Tế

Mã Nguồn: Thanh Bên Phụ Aside Cho Trang Blog

```
<aside class="sidebar">
  <h3>Bài Viết Được Xem Nhiều Nhất</h3>
  <ul>
    <li><a href="/post/1">Làm chủ CSS Grid</a></li>
    <li><a href="/post/2">Học JavaScript ES6</a></li>
  </ul>
</aside>
```

BROWSER PREVIEW — Kết Quả Hiển Thị Trình Duyệt: Thanh Bên Aside

Bài Viết Được Xem Nhiều Nhất

- Làm chủ CSS Grid
- Học JavaScript ES6

4.8 Thẻ Chân Trang <footer>

4.8.1 1. Lý Thuyết & Thành Phần Bên Trong

Thẻ <footer> đại diện cho chân của trang web hoặc chân của một phân đoạn <article>. Thường chứa thông tin bản quyền, liên kết điều khoản dịch vụ, bản đồ trang web (Sitemap) và icon mạng xã hội.

4.8.2 2. Ví Dụ Mã Nguồn Thực Tế

Mã Nguồn: Chân Trang Footer Đầy Đủ

```
<footer>
  <p>&copy; 2026 EduWeb Academy. Tất cả quyền được bảo lưu.</p>
  <ul>
    <li><a href="/privacy">Chính Sách Bảo Mật</a></li>
    <li><a href="/terms">Điều Khoản Sử Dụng</a></li>
  </ul>
</footer>
```

BROWSER PREVIEW — Kết Quả Hiển Thị Trình Duyệt: Chân Trang Footer

© 2026 EduWeb Academy. Tất cả quyền được bảo lưu.
Chính Sách Bảo Mật | Điều Khoản Sử Dụng

4.9 Thẻ Thông Tin Liên Hệ <address>

4.9.1 1. Lý Thuyết & Định Dạng Chuẩn

Thẻ <address> cung cấp thông tin liên hệ cho tác giả hoặc chủ sở hữu của tài liệu/bài viết.

4.9.2 2. Ví Dụ Mã Nguồn Thực Tế

Mã Nguồn: Thông Tin Liên Hệ Tác Giả Với Thẻ Address

```
<address>
  Viết bởi <a href="mailto:tan@eduweb.vn">Nguyễn Trọng Tấn</a>.<br>
  Địa chỉ: Tòa nhà EduWeb, Hà Nội, Việt Nam.<br>
  Điện thoại: <a href="tel:+84987654321">0987 654 321</a>
</address>
```

BROWSER PREVIEW Kết Quả Hiển Thị Trình Duyệt: Thẻ Address

Viết bởi Nguyễn Trọng Tấn.
 Địa chỉ: Tòa nhà EduWeb, Hà Nội, Việt Nam.
 Điện thoại: 0987 654 321

4.10 Nhóm Thẻ Tiêu Đề <h1> đến <h6> & <hgroup>

4.10.1 1. Lý Thuyết & Cây Phân Tầng Tiêu Đề SEO

Các thẻ từ <h1> đến <h6> đại diện cho 6 cấp độ tiêu đề phân tầng. Thẻ <hgroup> dùng để nhóm một tiêu đề chính và các tiêu đề phụ lại với nhau.

4.10.2 2. Ví Dụ Mã Nguồn Thực Tế

Mã Nguồn: Cấu Trúc Hgroup Và Tiêu Đề Phân Tầng

```
<hgroup>
  <h1>Lập Trình Web Chuyên Sâu</h1>
  <p>Giáo trình Bách khoa toàn thư Frontend 2026</p>
</hgroup>
```

BROWSER PREVIEW Kết Quả Hiển Thị Trình Duyệt: Hgroup & Tiêu Đề Phân Tầng

Lập Trình Web Chuyên Sâu

Giáo trình Bách khoa toàn thư Frontend 2026

4.11 Thẻ Khối Chứa Tìm Kiếm <search>

4.11.1 1. Lý Thuyết & Chuẩn WHATWG Mới

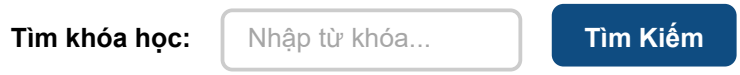
Thẻ `<search>` là thẻ ngữ nghĩa mới được đưa vào HTML Living Standard đại diện cho khối giao diện chứa công cụ tìm kiếm và lọc dữ liệu.

4.11.2 2. Ví Dụ Mã Nguồn Thực Tế

Mã Nguồn: Thẻ Search Chứa Form Tìm Kiếm Chuẩn WHATWG

```html
<search>
  <form action="/search" method="GET">
    <label for="query">Tìm khóa học:</label>
    <input type="search" id="query" name="q" placeholder="Nhập từ khóa...">
    <button type="submit">Tìm Kiếm</button>
  </form>
</search>
```

BROWSER PREVIEW — **Kết Quả Hiển Thị Trình Duyệt: Khối Tìm Kiếm Search**



4.11.3 3. Đọc Thêm: Hệ Thống ARIA Landmarks & Accessibility Tree

Khi trình duyệt phân tích các thẻ Bố cục ngữ nghĩa HTML5, nó tự động ánh xạ (*mapping*) các thẻ này thành các **Vùng Mốc ARIA (Landmark Roles)** trên Cây Trợ Năng (**Accessibility Tree**) mà không cần viết thủ công thuộc tính `role="..."`:

[Bảng] Bảng Ánh Xạ Thẻ Bố Cục HTML5 Sang ARIA Landmarks

Thẻ Bố Cục HTML5	ARIA Role Tương Đương	Hành Vi Trình Đọc Màn Hình (Screen Reader)
`<header>`	`role="banner"`	Giúp người khiếm thị định vị vùng đầu trang chứa Logo và Tiêu đề.
`<nav>`	`role="navigation"`	Cho phép ấn phím tắt nhảy trực tiếp đến danh sách liên kết điều hướng.
`<main>`	`role="main"`	Bỏ qua toàn bộ Menu để đi thẳng vào nội dung chính của trang web.
`<aside>`	`role="complementary"`	Đánh dấu khu vực thông tin phụ hỗ trợ cho nội dung chính.
`<footer>`	`role="contentinfo"`	Định vị vùng thông tin bản quyền và liên kết chân trang.
`<search>`	`role="search"`	Đánh dấu vùng công cụ tìm kiếm dữ liệu.

4.12 Thẻ Đoạn Văn <p> & Khối Văn Bản Giữ Nguyên <pre>

4.12.1 1. Lý Thuyết & Khác Biệt Cốt Lõi

- <p>: Đoạn văn bản thông thường (Trình duyệt tự động nén nhiều khoảng trắng liền nhau thành 1 khoảng trắng). - <pre>: Định dạng văn bản giữ nguyên chính xác từng khoảng trắng, dấu cách và xuống dòng như trong mã nguồn.

4.12.2 2. Ví Dụ Mã Nguồn Thực Tế

Mã Nguồn: So Sánh Thẻ p và Thẻ pre

```
<!-- Đoạn văn bình thường --><p>Đây là một đoạn văn bản chuẩn.</p>

<!-- Khối hiển thị giữ nguyên khoảng trắng --><pre>
  Họ và Tên : Nguyễn Văn A
  Quê Quán : Hà Nội
</pre>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: So Sánh Thẻ p vs Thẻ pre**

Thẻ <p> (Dòng chảy chuẩn):
Đây là một đoạn văn bản chuẩn.

Thẻ <pre> (Giữ nguyên khoảng trắng font Monospace):

```
Họ và Tên : Nguyễn Văn A
Quê Quán : Hà Nội
```

[Khái Niệm] Cơ Chế Nén Khoảng Trắng (Whitespace Collapse)

- **Thẻ <p>**: Áp dụng quy tắc nén khoảng trắng (**Whitespace Collapse**). Mọi dấu cách thừa, dấu Tab và ký tự xuống dòng trong mã nguồn đều bị trình duyệt nén lại thành **duy nhất 1 dấu cách**.
- **Thẻ <pre>**: Vô hiệu hóa quy tắc nén khoảng trắng. Trình duyệt hiển thị chính xác từng ký tự khoảng trắng và dòng mới với phông chữ độ rộng cố định (**Monospace Font**).
- **Mẹo CSS Thay Thế**: Thay vì dùng thẻ <pre>, bạn có thể dùng thẻ <p> đính kèm CSS `white-space: pre-wrap;` để vừa giữ nguyên khoảng trắng vừa hỗ trợ tự động xuống dòng khi màn hình nhỏ.

4.13 Thẻ Trích Dẫn Khối <blockquote> & Trích Dẫn Nội Dòng <q>

4.13.1 1. Lý Thuyết & Thuộc Tính cite

- <blockquote>: Trích dẫn một đoạn văn bản dài từ nguồn khác (Hiển thị lùi lề khối). - <q>: Trích dẫn ngắn nằm trực tiếp trong dòng (Trình duyệt tự động thêm dấu ngoặc kép "..."). - Thuộc tính cite="...": Chứa URL nguồn gốc của bài trích dẫn.

4.13.2 2. Ví Dụ Mã Nguồn Thực Tế

Mã Nguồn: Thẻ Blockquote và Thẻ q Với Thuộc Tính Cite

```
<blockquote cite="https://w3.org/TR/html5">
  <p>HTML5 là chuẩn Living Standard phát triển bởi WHATWG.</p>
</blockquote>

<p>Tim Berners-Lee từng nói: <q cite="https://quote.com">Internet là dành cho tất
    cả mọi người</q>.</p>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Blockquote & Quoted Text**

> *HTML5 là chuẩn Living Standard phát triển bởi WHATWG.*

Tim Berners-Lee từng nói: ”Internet là dành cho tất cả mọi người”.

4.14 Các Thẻ Danh Sách <ol>, <ul>, <li>, <dl>, <dt>, <dd>

4.14.1 1. Lý Thuyết & Các Thuộc Tính Danh Sách

- : Danh sách không thứ tự (Dấu chấm tròn bullet). - : Danh sách có thứ tự (Đánh số 1, 2, 3). Thuộc tính type="1|a|A|i|I", start="5", reversed (đánh số ngược). - <dl>, <dt>, <dd>: Danh sách mô tả / Từ điển (**Description List**) dùng cho các cặp Thuật ngữ - Giải nghĩa.

4.14.2 2. Ví Dụ Mã Nguồn Thực Tế

Mã Nguồn: Danh Sách Đánh Số Ngược Và Danh Sách Từ Điển DL

```
<!-- Danh sách đánh số ngược từ 3 về 1 --><ol reversed start="3">
  <li>Bước 3: Xuất bản ứng dụng</li>
  <li>Bước 2: Viết mã nguồn</li>
  <li>Bước 1: Lập kế hoạch</li>
</ol>

<!-- Danh sách từ điển thuật ngữ --><dl>
  <dt>HTML5</dt>
  <dd>Ngôn ngữ đánh dấu siêu văn bản phiên bản 5.</dd>
  <dt>CSS3</dt>
  <dd>Ngôn ngữ định kiểu trang trí giao diện web.</dd>
</dl>
```

BROWSER PREVIEW Kết Quả Hiển Thị Trình Duyệt: Danh Sách Thống Kê & Từ Điển

Danh sách đánh số ngược (start="3" reversed):

3. Bước 3: Xuất bản ứng dụng
2. Bước 2: Viết mã nguồn
1. Bước 1: Lập kế hoạch

Danh sách từ điển (Description List <dl>):

HTML5
Ngôn ngữ đánh dấu siêu văn bản phiên bản 5.
CSS3
Ngôn ngữ định kiểu trang trí giao diện web.

4.15 Thẻ Đường Kẻ Phân Cách <hr>

4.15.1 1. Lý Thuyết & Ý Nghĩa Ngữ Nghĩa

Thẻ <hr> trong HTML5 đại diện cho một điểm chuyển đổi chủ đề ngữ nghĩa (**Thematic Break**) giữa các đoạn văn chứ không đơn thuần là vẽ một đường kẻ ngang trang trí.

4.15.2 2. Ví Dụ Mã Nguồn Thực Tế

Mã Nguồn: Chuyển Đổi Chủ Đề Với Thẻ Hr

```
<p>Kết thúc chương 1 tại đây.</p>
<hr>
<p>Bắt đầu nội dung chương 2...</p>
```

BROWSER PREVIEW Kết Quả Hiển Thị Trình Duyệt: Đường Kẻ Phân Cách Hr

Kết thúc chương 1 tại đây.

Bắt đầu nội dung chương 2...

4.16 Thẻ Liên Kết Siêu Văn Bản <a> Masterclass

4.16.1 1. Lý Thuyết & Mục Đích Sử Dụng

Thẻ <a> (viết tắt của **Anchor** – điểm neo) được sử dụng để tạo các siêu liên kết (**Hyperlinks**). Đây là "xương sống" của mạng lưới World Wide Web, cho phép người dùng chuyển hướng giữa các trang web hoặc tài nguyên khác nhau.

Thẻ <a> hỗ trợ 6 mục đích liên kết chính:

1. Liên kết đến một trang web khác (Ngoại trang – External Link).
2. Liên kết đến một phần cụ thể trên cùng trang (Liên kết nội bộ – Internal Anchor Link).
3. Mở ứng dụng gửi Email (mailto:).
4. Gọi điện thoại trực tiếp trên thiết bị di động (tel:).
5. Kích hoạt tải xuống tệp tin (download).
6. Thực thi hành động JavaScript.

4.16.2 2. Cú Pháp Cơ Bản & Phân Loại Các Loại Đường Dẫn (href)

Cú pháp cơ bản: Nội dung hiển thị. Thuộc tính href (**Hypertext Reference**) là thuộc tính **bắt buộc** nếu muốn tạo liên kết.

1. **URL Tuyệt Đối (Absolute URL)**: Địa chỉ web đầy đủ bao gồm giao thức (https://), tên miền và đường dẫn.
2. **URL Tương Đối (Relative URL)**: Địa chỉ tương đối so với trang hiện tại (cùng thư mục contact.html, thư mục con pages/about.html, thư mục cha ../index.html).
3. **Liên Kết Nội Bộ (Internal Anchor Links)**: Trỏ đến phần tử có id="..." duy nhất trên trang qua cú pháp href="#id_dich".
4. **Liên Kết Email (mailto:)**: Mở trình gửi email. Hỗ trợ tiêu đề và nội dung mặc định bằng cú pháp mã hóa URL (%20 cho dấu cách).
5. **Liên Kết Điện Thoại (tel:)**: Kích hoạt cuộc gọi trên điện thoại.
6. **Liên Kết Tải Xuống (download)**: Gợi ý trình duyệt tải file thay vì mở trực tiếp.

Mã Nguồn: Các Loại Đường Dẫn Href Phổ Biến Trong Thẻ a

```
<!-- 1. URL Tuyệt đối --><a href="https://www.example.com/pages/about.html">Về chúng tôi</a>
<!-- 2. URL Tương đối --><a href="contact.html">Liên hệ</a>
<a href="../index.html">Trang chủ</a>

<!-- 3. Liên kết nội bộ Anchor Link --><h2 id="section1">Giới thiệu sản phẩm</h2>
<a href="#section1">Đi đến phần giới thiệu</a>

<!-- 4. Liên kết Email (Mailto) --><a href="mailto:info@example.com?subject=Ho%20tro%20san%20pham&body=Toi%20co%20cau%20hoi:">Gửi Email Hỗ Trợ</a>

<!-- 5. Liên kết Điện thoại (Tel) --><a href="tel:+84123456789">Gọi ngay: 0123.456.789</a>

<!-- 6. Liên kết Tải xuống (Download) --><a href="document.pdf" download="BaoCaoQuy3.pdf">Tải báo cáo PDF</a>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Các Loại Đường Dẫn Href**

1. URL Tuyệt đối: Về chúng tôi
2. URL Tương đối: Liên hệ | Trang chủ
3. Anchor Link: Đi đến phần giới thiệu
4. Email: Gửi Email Hỗ Trợ
5. Tel: Gọi ngay: 0123.456.789
6. Download: Tải báo cáo PDF

4.16.3 3. Phân Tích Chuyên Sâu Về href="#" (Masterclass href="#")

Dấu thăng `href="#"` đóng vai trò là một định danh mảnh (**Fragment Identifier**) rỗng. Khi nhấp vào liên kết `href="#"`:

- Trình duyệt không tải lại trang nhưng sẽ **tự động cuộn mượt/giật lên đầu trang** (`<body>` hoặc `<html>`).
- Đôi khi thêm ký tự `#` vào cuối URL trên thanh địa chỉ.

[Cảnh Báo] Cạm Bẫy Giật Cuộn Trang Khi Dùng href="#"

Nếu bạn dùng `href="#"` để chạy hàm JavaScript (như mở Modal, bật Menu) mà không ngăn hành vi mặc định, người dùng sẽ bị giật cuộn lên đầu trang rất khó chịu!

Các Cách Ngăn Hành Vi Mặc Định Của href="#"

Mã Nguồn: Ngăn Hành Vi Cuộn Trang Khi Dùng href="#"

```
<!-- Cách 1: Dùng e.preventDefault() trong JavaScript (Khuyên dùng) --><a href="#" id="openModalBtn">Mở Hộp Thoại</a><script>
  document.getElementById('openModalBtn').addEventListener('click', function(e) {
    e.preventDefault(); // Ngăn hành vi cuộn mượt mặc định
    alert('Hộp thoại đã mở!');
  });
</script>

<!-- Cách 2: Dùng return false trong inline onclick --><a href="#" onclick="alert('Đã click!'); return false;">Click tôi</a>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Thẻ a Ngăn Cuộn Mặc Định**

Mở Hộp Thoại *(Trang web giữ nguyên vị trí cuộn khi người dùng click vào nút!)*

[Bảng] Bảng Thống Kê

Giải Pháp Thay Thế	Ưu / Nhược Điểm	Khuyên Dùng Khi Nào?
`<a href="#">` + `preventDefault()`	Tạo nút bấm giả, giữ được focus phím Tab.	Khi cần giả lập link chạy JS.
`<a href="javascript:void(0)">`	Không cuộn trang nhưng kém a11y	Không khuyến khích.
`<button type="button">`	**Chuẩn ngữ nghĩa a11y 100%**	**Rất khuyên dùng** cho các hành động JS không điều hướng trang.

Bảng 4.1: So sánh giữa href=”#”, href=”javascript:void(0)” và thẻ button

4.16.4 4. Bảng Tra Cứu Các Thuộc Tính Quan Trọng Của Thẻ <a>

[Bảng] Bảng Thống Kê

Thuộc Tính	Giá Trị Hợp Lệ	Chức Năng & Hành Vi
`href`	URL chuẩn	Đường dẫn trỏ đến trang/tài nguyên đích.
`target`	`_self`, `_blank`, `_parent`, `_top`	Nơi hiển thị trang mới (`_blank` mở tab mới).
`rel`	`noopener`, `noreferrer`, `nofollow`, `external`	Khai báo mối quan hệ SEO và bảo mật.
`download`	Tên file tùy chọn	Ép trình duyệt tải file về thay vì mở trên web.
`title`	Chuỗi văn bản	Tooltip hiển thị khi di chuột qua liên kết.
`ping`	URLs phân cách khoảng trắng	Gửi request theo dõi lượt nhấp (Analytics tracking).

Bảng 4.2: Toàn bộ các thuộc tính nâng cao của thẻ <a>

[Cảnh Báo] Bảo Mật Với target=”_blank” & rel=”noopener noreferrer”

Khi dùng `target="_blank"` mở tab mới, trang mới có thể can thiệp vào trang cũ qua `window.opener` (**Tấn công Tabnabbing**). **Luôn bổ sung**: `rel="noopener noreferrer"` khi mở liên kết ngoại trang tab mới!

Mã Nguồn: Liên Kết Mở Tab Mới An Toàn Chuẩn SEO & Bảo Mật

```
<a href="https://www.example.org" target="_blank" rel="noopener noreferrer"
    title="Ghé thăm trang web bên
    ngoài">
  Trang Web Bên Ngoài (Mở Tab Mới)
</a>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Target _blank An Toàn**

Trang Web Bên Ngoài (Mở Tab Mới) (Hiển thị tooltip: "Ghé thăm trang web bên ngoài")

4.16.5 5. Thẻ <a> Với Nội Dung Khối (Clickable UI Cards)

Trong HTML5, bạn có quyền bọc các phần tử khối (, <h3>, <p>, <img>) bên trong thẻ <a> để biến toàn bộ một **Card UI** thành khu vực nhấp chuột chuyển hướng.

Mã Nguồn: Bọc Khối UI Card Bên Trong Thẻ a Để Nhấp Chuyển Trang

```
<!-- HTML5 cho phép bọc khối div bên trong thẻ
    a --><a href="/blog/html5-masterclass" class="blog-card-link">
  <div class="blog-card">
    <img src="thumbnail.jpg" alt="Ảnh đại diện bài viết">
    <h3>Khóa Học HTML5 Masterclass 2026</h3>
    <p>Tìm hiểu toàn bộ thẻ HTML5 ngữ nghĩa chuẩn SEO...</p>
  </div>
</a>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Clickable UI Card**

[Ảnh Thumbnail]

Khóa Học HTML5 Masterclass 2026

Tìm hiểu toàn bộ thẻ HTML5 ngữ nghĩa chuẩn SEO...

4.16.6 6. Tạo Kiểu Thẻ <a> Bằng CSS & Pseudo-Classes (Quy Tắc LVHA)

Trạng thái của thẻ <a> được điều khiển bởi 5 Lớp giả (**Pseudo-classes**) theo quy tắc ưu tiên **LVHA**:

1. a:link: Liên kết ban đầu chưa được truy cập.
2. a:visited: Liên kết đã từng được người dùng nhấp ghé thăm.
3. a:focus: Liên kết đang nhận tiêu điểm bàn phím (ấn phím Tab).
4. a:hover: Con trỏ chuột đang di chuyển đè lên liên kết.
5. a:active: Liên kết đang trong khoảng thời gian đè giữ chuột.

Mã Nguồn: CSS Định Kiểu Thẻ a Thành Button Nút Bấm Với Quy Tắc LVHA

```
<style>
  /* 1. Kỹ thuật tạo nút bấm từ thẻ a */
  .btn-link {
    display: inline-block;
    background: #0F4C81;
    color: #ffffff;
    padding: 10px 20px;
    text-decoration: none;
    border-radius: 5px;
    transition: background 0.3s ease;
  }

  /* 2. Thứ tự ưu tiên LVHA */
  a:link { color: #0F4C81; text-decoration: none; }
  a:visited { color: #5E35B1; }
  a:focus { outline: 2px solid #E65100; outline-offset: 2px; }
  a:hover { color: #E65100; text-decoration: underline; }
  a:active { color: #008080; }
</style>

<a href="/courses" class="btn-link">Khám Phá Khóa Học</a>
```

BROWSER PREVIEW Kết Quả Hiển Thị Trình Duyệt: Thẻ a Tạo Kiểu Thành Button

Khám Phá Khóa Học

4.17 Nhóm Thẻ Nhấn Mạnh Văn Bản , , <mark>, <small>, <s>, <del>, <ins>

4.17.1 1. Lý Thuyết & Mục Đích Ngữ Nghĩa

- : Văn bản có tầm quan trọng đặc biệt (In đậm). - : Nhấn mạnh ngữ điệu nói (In nghiêng). - <mark>: Tô sáng văn bản tìm kiếm (Mặc định nền vàng). - <small>: Văn bản ghi chú nhỏ (Bản quyền, miễn trừ trách nhiệm). - <del> & <ins>: Văn bản bị xóa bỏ (gạch ngang) và văn bản được

thêm mới (gạch chân) kèm thuộc tính `datetime`.

4.17.2 2. Ví Dụ Mã Nguồn Thực Tế

Mã Nguồn: Ứng Dụng Nhóm Thẻ Nhấn Mạnh Văn Bản

```
<p>Giá gốc:<del>500.000 VNĐ</del> <ins datetime="2026-07-20">350.000
    VNĐ</ins></p>
<p>Từ khóa tìm kiếm:<mark>HTML5 Masterclass</mark></p>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Nhóm Thẻ Nhấn Mạnh Văn Bản**

Giá gốc: ~~500.000 VNĐ~~ **350.000 VNĐ**

Từ khóa tìm kiếm: **HTML5 Masterclass**

4.18 Nhóm Thẻ Kỹ Thuật <code>, <kbd>, <var>, <samp>

4.18.1 1. Lý Thuyết & Mục Đích Sử Dụng

- `<code>`: Hiển thị đoạn mã máy tính. - `<kbd>`: Đại diện cho phím bấm bàn phím người dùng ấn. - `<var>`: Biến toán học hoặc lập trình. - `<samp>`: Kết quả mẫu xuất ra từ chương trình máy tính.

4.18.2 2. Ví Dụ Mã Nguồn Thực Tế

Mã Nguồn: Kết Hợp Các Thẻ Kỹ Thuật Code, Kbd, Var

```
<p>Nhấn<kbd>Ctrl</kbd> + <kbd>C</kbd>để sao chép biến<var>x</var>trong
    hàm<code>main()</code>.</p>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Nhóm Thẻ Kỹ Thuật Code, Kbd, Var**

Nhấn `Ctrl` + `C` để sao chép biến *x* trong hàm `main()`.

4.19 Nhóm Thẻ Thời Gian & Mã Số <time> & <data>

4.19.1 1. Lý Thuyết & Thuộc Tính datetime/value

- `<time datetime="YYYY-MM-DDThh:mm:ss">`: Chuẩn hóa mốc thời gian cho con bọ tìm kiếm và trình đọc màn hình. - `<data value="...">`: Gắn mã sản phẩm / ID máy tính đọc được với tên hiển thị.

4.19.2 2. Ví Dụ Mã Nguồn Thực Tế

Mã Nguồn: Thẻ Thời Gian Time Và Thẻ Mã Số Data

```
<p>Đăng ngày<time datetime="2026-07-20T08:30">20 Tháng 7, 2026</time></p>
<p>Sản phẩm:<data value="PROD-9982">Laptop Dell XPS</data></p>
```

BROWSER PREVIEW — **Kết Quả Hiển Thị Trình Duyệt: Thẻ Time & Thẻ Data**

Đăng ngày 20 Tháng 7, 2026
Sản phẩm: Laptop Dell XPS (Thuộc tính ngầm value="PROD-9982")

4.20 Thẻ Từ Viết Tắt <abbr> & Định Nghĩa <dfn>

4.20.1 1. Lý Thuyết & Thuộc Tính title

- `<abbr title="...">`: Từ viết tắt. Thuộc tính `title` chứa cụm từ giải nghĩa đầy đủ hiển thị khi di chuột. - `<dfn>`: Đánh dấu thuật ngữ lần đầu tiên được định nghĩa trong bài viết.

4.20.2 2. Ví Dụ Mã Nguồn Thực Tế

Mã Nguồn: Thẻ Từ Viết Tắt Abbr Và Định Nghĩa Dfn

```
<p><dfn><abbr title="HyperText Markup Language">HTML</abbr></dfn>là ngôn ngữ đánh
    dấu siêu văn bản chuẩn của Web.</p>
```

BROWSER PREVIEW — **Kết Quả Hiển Thị Trình Duyệt: Thẻ Abbr & Thẻ Dfn**

HTML (HyperText Markup Language) là ngôn ngữ đánh dấu siêu văn bản chuẩn của Web.

4.21 Nhóm Thẻ Đa Ngôn Ngữ & Ruby Annotations <bdi>, <bdo>, <ruby>, <rt>, <rp>

4.21.1 1. Lý Thuyết & Chú Thích Phát Âm

- <bdi>: Cô lập đoạn văn bản không rõ hướng đọc (RTL/LTR). - <bdo dir="rtl|ltr">: Ép buộc hướng đọc văn bản. - <ruby>, <rt>, <rp>: Hệ thống thẻ chú thích phiên âm phát âm trên đầu chữ Á Đông.

4.21.2 2. Ví Dụ Mã Nguồn Thực Tế

Mã Nguồn: Chú Thích Phát Âm Chữ Á Đông Với Thẻ Ruby

```
<ruby>
  Kan <rp>(</rp><rt>kan</rt><rp>)</rp>
  Ji <rp>(</rp><rt>ji</rt><rp>)</rp>
</ruby>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Chú Thích Phiên Âm Ruby**

kan ji
Kan **Ji**

4.22 Thẻ Chia Khối & Thẻ Inline

4.22.1 1. Lý Thuyết & Cảnh Báo Lạm Dụng Div/Span

 (Division) và là hai thẻ ”khô” không mang bất kỳ ý nghĩa ngữ nghĩa nào, được sử dụng làm container gom nhóm để định kiểu CSS hoặc thao tác JavaScript.

[Cảnh Báo] Cảnh Báo Hội Chứng Divitis (Div-Soup)

Không sử dụng thay thế cho thẻ <button> hoặc <a>. Việc lạm dụng thẻ div vô nghĩa thay cho thẻ ngữ nghĩa sẽ phá hỏng toàn bộ điểm số SEO và khiến người khiếm thị không thể sử dụng trang web!

4.22.2 2. Ví Dụ Mã Nguồn Thực Tế

Mã Nguồn: Gom Nhóm Container Div Và Span Hợp Lệ

```
<div class="card-wrapper">
  <span class="badge badge-success">Mới</span>
  <p>Nội dung thẻ card...</p>
</div>
```

BROWSER PREVIEW Kết Quả Hiển Thị Trình Duyệt: Gom Nhóm Div & Span

Mới

Nội dung thẻ card...

4.23 Thẻ Khối Chú Thích Hình Ảnh & Sơ Đồ <figure> & <figcaption>

4.23.1 1. Lý Thuyết & Accessibility (a11y)

Thẻ <figure> đại diện cho một đơn vị nội dung tự chứa độc lập (Hình ảnh, sơ đồ, minh họa, đoạn mã nguồn), thường đi kèm với một chú thích <figcaption> bên dưới hoặc bên trên. Trình đọc màn hình và con bọ tìm kiếm Googlebot gắn kết chặt chẽ chú thích này với hình ảnh/sơ đồ tương ứng để tối ưu SEO hình ảnh.

4.23.2 2. Ví Dụ Mã Nguồn Thực Tế

Mã Nguồn: Nhúng Hình Ảnh Minh Họa Kèm Chú Thích Figure Và Figcaption

```
<figure class="article-image">
  <img src="dom-tree.png" alt="Sơ đồ cây DOM trong HTML5" loading="lazy">
  <figcaption>Hình 4.1: Sơ đồ phân cấp cây Document Object Model
    (DOM).</figcaption>
</figure>
```

BROWSER PREVIEW Kết Quả Hiển Thị Trình Duyệt: Khối Figure Kèm Chú Thích Figcaption

[Hình ảnh minh họa: Sơ đồ cây DOM trong HTML5]

Hình 4.1: Sơ đồ phân cấp cây Document Object Model (DOM).

4.24 Thẻ Chỉ Số & Trích Dẫn Nguồn <sub>, <sup>, <cite>

4.24.1 1. Lý Thuyết & Ứng Dụng Trong Khoa Học & Văn Bản

- <sub> (**Subscript**): Chỉ số dưới, dùng hiển thị công thức hóa học (H_2O), chỉ số mảng toán học.
- <sup> (**Superscript**): Chỉ số trên, dùng hiển thị lũy thừa ($E = mc^2$), diện tích (m^2) hoặc chú thích chân trang (**Footnote**).

- <cite>: Đánh dấu tên nguồn trích dẫn của một tác phẩm (Tên sách, bài báo, bài hát, phim, nghiên cứu khoa học).

4.24.2 2. Ví Dụ Mã Nguồn Thực Tế

Mã Nguồn: Sử Dụng Thẻ Sub, Sup Và Cite

```
<!-- Công thức hóa học và toán học --><p>Công thức phân tử nước là H<sub>2</sub>O và diện tích căn phòng là 45 m<sup>2</sup>.</p>

<!-- Trích dẫn tên tác phẩm --><p>Theo cuốn sách<cite>Bách Khoa Lập Trình Web 2026</cite>, HTML5 là nền tảng của mọi trang web.</p>
```

BROWSER PREVIEW Kết Quả Hiển Thị Trình Duyệt: Chỉ Số Sub, Sup & Trích Dẫn Cite

Công thức phân tử nước là H_2O và diện tích căn phòng là 45 m^2.

Theo cuốn sách *Bách Khoa Lập Trình Web 2026*, HTML5 là nền tảng của mọi trang web.

4.25 Thẻ Ngắt Dòng & Ngắt Từ Linh Hoạt
 & <wbr>

4.25.1 1. Lý Thuyết & So Sánh Behavior

-
 (**Line Break**): Ép trình duyệt xuống dòng ngay lập tức tại vị trí đặt thẻ (không được dùng
 để tạo khoảng cách giữa các đoạn văn thay cho CSS margin!).
- <wbr> (**Word Break Opportunity**): Khai báo một vị trí trình duyệt **được phép ngắt dòng nếu cần thiết** khi hiển thị trên màn hình nhỏ di động (rất hữu ích cho các đường dẫn URL siêu dài hoặc các từ ghép tiếng Đức/Anh dài).

4.25.2 2. Ví Dụ Mã Nguồn Thực Tế

Mã Nguồn: Sử Dụng Thẻ Br Ngắt Dòng Và Wbr Ngắt Từ Linh Hoạt

```
<!-- Ngắt dòng trong địa chỉ thơ văn --><p>EduWeb Academy<br>123 Đường Cầu Giấy<br>Hà Nội, Việt Nam</p>

<!-- Ngắt từ linh hoạt cho đường dẫn dài --><p>Truy cập: https://eduweb.vn/courses/html5<wbr>-masterclass<wbr>-advanced<wbr>-2026.html</p>
```

BROWSER PREVIEW Kết Quả Hiển Thị Trình Duyệt: Ngắt Dòng Br & Ngắt Từ Wbr

EduWeb Academy
123 Đường Cầu Giấy
Hà Nội, Việt Nam

Truy cập: https://eduweb.vn/courses/html5-masterclass-advanced-2026.html

4.26 Thẻ Định Dạng Giao Diện Phân Biệt Ngữ Nghĩa <b>, <i>, <u>

4.26.1 1. Lý Thuyết & Phân Biệt Với Strong / Em

HTML5 phân biệt rõ ràng giữa thẻ định kiểu giao diện thuần túy và thẻ mang ngữ nghĩa:

- `<b>` (**Bring Attention To**): Làm đậm văn bản thuần thị giác mà **không làm tăng độ quan trọng** (khác với `<strong>`).
- `<i>` (**Idiomatic Text**): In nghiêng văn bản dành cho thuật ngữ kỹ thuật, từ ngoại ngữ, tên loài khoa học, suy nghĩ nội tâm (khác với `<em>` nhấn mạnh giọng đọc).
- `<u>` (**Unarticulated Annotation**): Gạch chân văn bản đánh dấu từ sai chính tả hoặc tên riêng tiếng Trung/Nhật.

4.26.2 2. Ví Dụ Mã Nguồn Thực Tế

Mã Nguồn: Thẻ b, i Và u Chuẩn Ngữ Nghĩa HTML5

```
<p>Tên khoa học của loài mèo là<i>Felis catus</i>.</p>
<p>Vui lòng chú ý từ<b>lưu ý</b>trong hợp đồng.</p>
```

BROWSER PREVIEW Kết Quả Hiển Thị Trình Duyệt: Định Dạng Thẻ b, i, u

Tên khoa học của loài mèo là *Felis catus*.

Vui lòng chú ý từ **lưu ý** trong hợp đồng.

4.26.3 3. Đọc Thêm: Phân Biệt Ngữ Nghĩa Trợ Năng (Screen Reader Sound Modulation)

[Mẹo] Sự Khác Biệt Giữa Định Kiểu Thị Giác Và Âm Thanh Trợ Năng

- **Thẻ `<b>` & `<i>`**: Chỉ thay đổi diện mạo thị giác trên màn hình (**Visual Styling**). Trình đọc màn hình (Screen Reader) sẽ đọc các từ này với giọng đọc bình thường không thay đổi.
- **Thẻ `<strong>` & `<em>`**: Thay đổi ngữ nghĩa cảm xúc và tầm quan trọng (**Semantic Emphasis**). Trình đọc màn hình như NVDA hoặc JAWS sẽ **tự động tăng âm lượng, thay đổi cao độ giọng đọc** hoặc đọc nhấn mạnh từ đó để thông báo cho người khiếm thị!

4.27 Thẻ Danh Sách Nút Hành Động <menu>

4.27.1 1. Lý Thuyết & Ngữ Nghĩa Toolbar

Thẻ `<menu>` trong chuẩn HTML5 Living Standard là một phiên bản ngữ nghĩa của thẻ `<ul>`, đại diện cho một danh sách không thứ tự chứa các nút bấm hành động (**Toolbar / Context Menu**) thay vì danh sách các liên kết điều hướng.

4.27.2 2. Ví Dụ Mã Nguồn Thực Tế

Mã Nguồn: Thanh Công Cụ Thao Tác Bài Viết Với Thẻ Menu

```
<menu class="editor-toolbar">
  <li><button type="button">Sao Chép</button></li>
  <li><button type="button">Dán</button></li>
  <li><button type="button">Xóa Bài</button></li>
</menu>
```

BROWSER PREVIEW — **Kết Quả Hiển Thị Trình Duyệt: Thanh Menu Công Cụ**

Sao Chép | Dán | Xóa Bài

4.28 Tóm Tắt Chương 4

Chương 4 đã phân tích trọn bộ 27 mục section riêng biệt dành cho toàn bộ các thẻ thuộc nhóm Bố cục ngữ nghĩa và Định dạng văn bản. Chương tiếp theo sẽ tiến sang hệ thống Thẻ Bảng Biểu và Thẻ Tương Tác Nội Tạng.

CHƯƠNG 5

Biểu Mẫu Web (HTML5 Forms) & Các Ô Nhập Liệu

[MỤC LỤC TRỌNG TÂM CHƯƠNG 5] Cấu Trúc 2 Phần Masterclass Biểu Mẫu

PHẦN 1: Nền Tảng Biểu Mẫu Web & Ô Nhập Liệu Cốt Lõi Cấu trúc thẻ `<form>`, giao thức truyền nhận `GET` vs `POST`, 22+ loại input types, `<label>`, `name` / `value`, `<fieldset>` & `<legend>`, so sánh `<select>` vs `<datalist>`, thẻ `<textarea>`, thẻ tính toán `<output>`, cơ chế Native Form Validation và biểu mẫu đăng ký tổng hợp.

PHẦN 2: Chuyên Sâu Từng Ô Nhập Liệu, Định Dạng CSS & Trợ Năng A11y Chi tiết kỹ thuật Checkbox, Radio, Buttons, Number, Range; Kỹ thuật CSS hiện đại (`:placeholder-shown`, Floating Label, `:focus-within`); Chuẩn trợ năng Web Accessibility (WCAG 2.1 / WAI-ARIA).

5.1 PHẦN 1: Nền Tảng Biểu Mẫu Web & Các Ô Nhập Liệu Cốt Lõi

Biểu mẫu HTML (**HTML Form**) là thành phần cốt lõi trên trang web cho phép thu thập dữ liệu từ người dùng (nhập văn bản, chọn radio, tick checkbox, upload tệp tin, nhấp nút Submit...). Khi gửi form, dữ liệu sẽ được đóng gói và chuyển lên máy chủ (**Server**) để xử lý thông qua hai thuộc tính quyết định: `action` và `method`.

5.1.1 1. Cấu Trúc Cơ Bản & Thuộc Tính Thẻ `<form>`

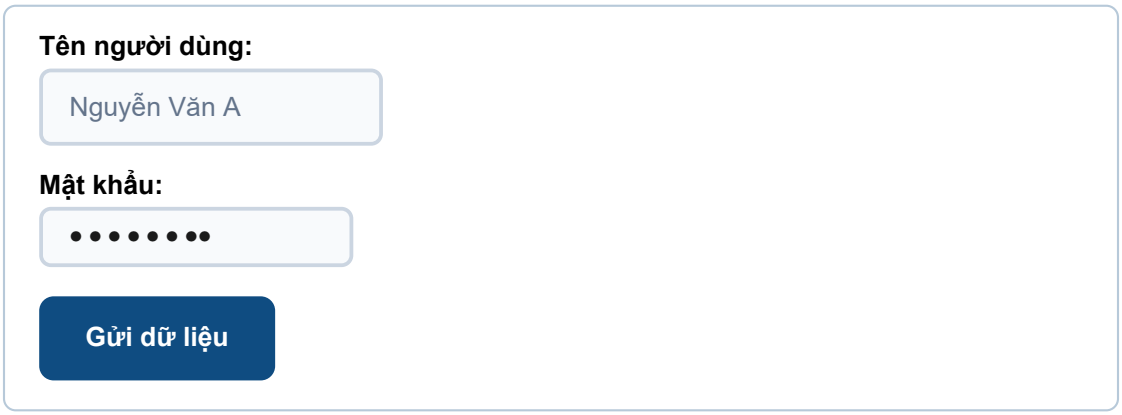
Mã Nguồn Cấu Trúc Khai Báo Biểu Mẫu HTML Cơ Bản

```html
<form action="/submit" method="post">
  <label for="name">Ten nguoi dung:</label>
  <input type="text" id="name" name="username">

  <label for="password">Mat khau:</label>
  <input type="password" id="password" name="password">

  <input type="submit" value="Gui du lieu">
</form>
```

BROWSER PREVIEW Kết Quả Hiển Thị Trình Duyệt: Biểu Mẫu HTML Cơ Bản



Bảng Tra Cứu Các Thuộc Tính Quan Trọng Của Thẻ <form>:

[Bảng] Bảng Tra Cứu Thuộc Tính Cốt Lõi Của Thẻ <form>

Thuộc Tính	Ý Nghĩa Ngữ Nghĩa & Chức Năng	Quy Chuẩn Kỹ Thuật & Giá Trị
`action`	Đường dẫn URL của máy chủ xử lý dữ liệu khi người dùng Submit form.	Đường dẫn tuyệt đối (`https://...`) hoặc tương đối (`/submit.php`).
`method`	Phương thức giao thức HTTP dùng để gửi dữ liệu lên server.	`GET` (nối tham số vào URL) hoặc `POST` (gửi ẩn trong Body).
`enctype`	Kiểu mã hóa dữ liệu khi gửi (bắt buộc khi có upload tệp tin).	`application/x-www-form-urlencoded` (mặc định) hoặc `multipart/form-data` (dùng upload file).
`target`	Vị trí hiển thị phản hồi kết quả từ máy chủ sau khi gửi form.	`_self` (mặc định trang hiện tại), `_blank` (mở cửa sổ/tab mới).
`autocomplete`	Bật hoặc tắt tính năng tự động gợi ý điền dữ liệu của trình duyệt.	`on` (mặc định bật gợi ý) hoặc `off` (tắt gợi ý bảo mật).
`novalidate`	Vô hiệu hóa tính năng tự động kiểm tra dữ liệu (**Client-side Validation**).	Thuộc tính Boolean (khi có mặt sẽ bỏ qua kiểm tra `required`, `pattern`).

Bảng Tra Cứu Toàn Bộ Các Thẻ & Phần Tử Form Thường Dùng:

[Bảng] Ma Trận Phân Loại Các Phần Tử Biểu Mẫu Trong HTML

Phần Tử Form	Công Dụng & Đặc Tính Kỹ Thuật	Ngữ Cảnh Sử Dụng Thực Tế
`<input type="text">`	Ô nhập văn bản một dòng thô thông thường.	Nhập họ tên, địa chỉ, tiêu đề bài viết.
`<input type="password">`	Ô nhập mật khẩu (ký tự tự động che bằng dấu chấm *).	Nhập mật khẩu đăng nhập, mã PIN bảo mật.
`<input type="radio">`	Chọn 1 trong nhiều tùy chọn duy nhất (Single selection).	Chọn giới tính (Nam/Nữ), chọn hình thức thanh toán.
`<input type="checkbox">`	Chọn nhiều tùy chọn độc lập cùng lúc (Multiple selections).	Chọn sở thích, đồng ý điều khoản dịch vụ.
`<input type="file">`	Chọn và tải tệp tin từ thiết bị máy tính người dùng.	Upload ảnh đại diện Avatar, tải file đính kèm PDF.
`<input type="submit">`	Nút gửi toàn bộ dữ liệu biểu mẫu lên server.	Nút ”Đăng ký”, ”Đăng nhập”, ”Gửi phản hồi”.
`<input type="reset">`	Nút đặt lại toàn bộ các ô nhập về giá trị ban đầu.	Khôi phục lại form nhập liệu trống.
`<textarea>`	Vùng nhập văn bản nhiều dòng (**Multiline text input**).	Nhập bài viết, gửi bình luận, nhập nội dung phản hồi.
`<select> + <option>`	Menu danh sách thả xuống (**Dropdown menu**).	Chọn tỉnh/thành phố, chọn quốc gia, chọn năm sinh.
`<button>`	Nút bấm linh hoạt có thể tùy chỉnh (`type="submit/reset/button"`).	Kết hợp với JavaScript kích hoạt sự kiện UI.
`<label>`	Nhãn liên kết tiêu đề với ô nhập (tăng A11y).	Rất quan trọng cho Trình đọc màn hình (Screen Readers).
`<fieldset> + <legend>`	Nhóm các phần tử nhập liệu có liên quan kèm khung viền.	Phân nhóm ”Thông tin cá nhân”, ”Thông tin giao hàng”.

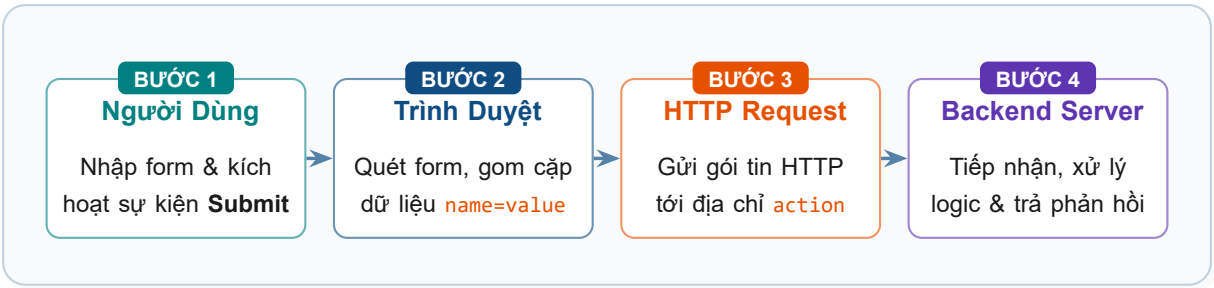
Chuyên Sâu Thuộc Tính action Trong Thẻ `<form>` (Form Action Là Gì?)

[Khái Niệm] Thuộc Tính action Trong HTML Form Là Gì?

Thuộc tính action của thẻ `<form>` xác định địa chỉ **URL** (Uniform Resource Locator) mà dữ liệu trong biểu mẫu sẽ được đóng gói và gửi tới khi người dùng kích hoạt sự kiện Submit (Gửi).

- **Nói cách khác**: Đây là điểm cuối (**Endpoint**) tiếp nhận và xử lý dữ liệu biểu mẫu trên máy chủ Backend (thường là một tập tin xử lý như PHP, Python, Node.js, Java hoặc một điểm cuối RESTful API).
- **Cơ chế hoạt động bên dưới**: Khi sự kiện submit xảy ra, trình duyệt web dừng luồng tương tác hiện tại, quét qua tất cả các phần tử nhập liệu hợp lệ trong thẻ `<form>`, gom các cặp `name=value`, mã hóa dữ liệu theo định dạng `enctype` và khởi tạo một HTTP Request chuyển hướng tới địa chỉ URL khai báo trong `action`.

Sơ Đồ Luồng Hoạt Động Của Form Action:



Phân Loại Địa Chỉ URL Trong Thuộc Tính action:

Khi khai báo đường dẫn `action`, lập trình viên có thể sử dụng các dạng đường dẫn sau:

[Bảng] Phân Loại Các Dạng Đường Dẫn Trong Thuộc Tính action

Dạng Đường Dẫn	Cú Pháp HTML Mẫu	Đặc Điểm & Hành Vi Xử Lý
Đường Dẫn Tương Đối (Relative URL)	`<form action="process.php">` `<form action="../save.php">`	Trỏ tới tệp nằm cùng thư mục hoặc thư mục cha/con trên cùng server. Dễ di chuyển mã nguồn giữa các môi trường (Dev/Staging/Prod).
Đường Dẫn Từ Gốc Root (Root-Relative URL)	`<form action="/api/v1/users">`	Trỏ tới địa chỉ tính từ tên miền gốc (**Domain Root**). Đảm bảo không bị lỗi đường dẫn khi nhúng form ở các trang con có độ sâu cấp thư mục khác nhau.
Đường Dẫn Tuyệt Đối (Absolute URL)	`<form action="https://api.domain.co`	Trỏ trực tiếp tới một tên miền hoặc máy chủ API bên ngoài. Bắt buộc phải khởi chạy qua giao thức an toàn `https://`.
Chuỗi Rỗng Hoặc Bỏ Trống (Self-Submitting Form)	`<form action="">` `<form>`	Trình duyệt mặc định gửi toàn bộ dữ liệu biểu mẫu về chính URL của trang hiện tại. Rất phổ biến khi làm việc với PHP/ASP.NET monolithic hoặc xử lý form 2 giai đoạn (GET hiển thị, POST xử lý).
Dấu Thăng (action="#") (Hash Navigation)	`<form action="#">`	Gửi dữ liệu về trang hiện tại nhưng đính kèm thẻ `#` làm trình duyệt tự động cuộn trang lên trên cùng (**Top Page**). Không khuyến nghị dùng trong thực tế trừ khi có xử lý JS.

Mối Quan Hệ Cốt Lõi Giữa action, method và enctype:

Thuộc tính `action` không hoạt động độc lập mà luôn kết hợp chặt chẽ với phương thức gửi dữ liệu `method` và kiểu mã hóa dữ liệu `enctype`:

1. **Kết hợp với `method="GET"`**: Trình duyệt đính kèm toàn bộ dữ liệu vào sau địa chỉ trong `action` dưới dạng chuỗi truy vấn (**Query String**):

```
URL_action?name1=value1&name2=value2
```

Ứng dụng: Dùng cho các biểu mẫu tìm kiếm, lọc dữ liệu, phân trang mà thông tin không cần bảo mật

và cho phép người dùng lưu bookmark URL.

2. **Kết hợp với `method="POST"`**: Trình duyệt đính kèm dữ liệu ẩn bên trong thân của HTTP Request (**Request Body**). *Ứng dụng*: Dùng cho đăng nhập, đăng ký, thanh toán, cập nhật thông tin cá nhân.
3. **Ảnh hưởng của thuộc tính `enctype` khi gửi tới action**:
   - `application/x-www-form-urlencoded` (*Mặc định*): Tất cả khoảng trắng đổi thành `+` hoặc `%20`, các ký tự đặc biệt được mã hóa theo dạng Hex.
   - `multipart/form-data`: **Bắt buộc phải dùng** khi form có ô tải tệp tin (`<input type="file">`) để dữ liệu tệp nhị phân được phân chia thành từng khối gửi tới `action`.
   - `text/plain`: Gửi dữ liệu dưới dạng văn bản thô không mã hóa (chỉ phục vụ thử nghiệm).

Ví Dụ Minh Họa Chi Tiết:

1. **Gửi đến file xử lý khác (`save_data.php`):**

Form gửi dữ liệu tới file xử lý save_data.php

```
<form action="save_data.php" method="post">
  <input type="text" name="email" placeholder="Nhập email">
  <button type="submit">Đăng ký</button>
</form>
```

BROWSER PREVIEW **Kết Quả Trình Duyệt: Gửi Đến File save_data.php**



→ *Kết quả khi submit*: Dữ liệu nhập ở ô email sẽ được gửi đến tập tin `save_data.php`.

2. **Gửi đến chính trang hiện tại (`action=""`):**

Form tìm kiếm gửi dữ liệu về chính trang hiện tại

```
<form action="" method="get">
  <input type="text" name="q" placeholder="Tìm kiếm...">
  <button type="submit">Search</button>
</form>
```

BROWSER PREVIEW **Kết Quả Trình Duyệt: Form Tìm Kiếm Gửi Về Trang Hiện Tại**



→ *Kết quả khi submit*: Trình duyệt sẽ đổi URL trên thanh địa chỉ thành dạng:

`http://example.com/?q=giatri_nhap`

Lưu Ý Quan Trọng & Cảnh Báo Bảo Mật (Security Best Practices):

[Cảnh Báo] Các Nguyên Tắc Bảo Mật Cần Nắm Vững Khi Cấu Hình Form Action

1. **Bắt buộc sử dụng giao thức HTTPS trong action**: Không bao giờ trỏ thuộc tính `action` tới đường dẫn giao thức không an toàn (`http://`) trong sản phẩm thực tế để tránh nguy cơ bị nghe lén dữ liệu trên đường truyền (**Man-in-the-Middle Attack**).
2. **Phòng chống lỗ hổng Cross-Site Request Forgery (CSRF)**: Khi `action` trỏ tới các điểm cuối thay đổi dữ liệu hệ thống (như đổi mật khẩu, chuyển tiền), bắt buộc Backend phải yêu cầu đính kèm **CSRF Token** hợp lệ bên trong form.
3. **Tránh lỗ hổng Open Redirect**: Không bao giờ cấu hình thuộc tính `action` nhận trực tiếp giá trị URL biến động truyền từ tham số chuỗi truy vấn người dùng nhập mà không qua kiểm tra (**Whitelist validation**).
4. **Phân công vai trò giữa Frontend và Backend**:
   - **Frontend**: Thiết kế giao diện, đảm bảo liên kết thuộc tính `action` chính xác, thu thập và kiểm tra sơ bộ tính hợp lệ (**Client-side Validation**).
   - **Backend**: Xây dựng bộ định tuyến (**Router/Endpoint**) tại đúng đường dẫn khai báo trong `action`, kiểm tra tính hợp lệ chặt chẽ (**Server-side Validation**) và lưu trữ cơ sở dữ liệu.

Minh Họa Thực Tế & Kịch Bản Nhập Thử (Form Action Demonstration):

Mã Nguồn Đầy Đủ Minh Họa Form Action Với Kịch Bản Nhập Thử

```
<!-- Form dang ky tai khoan truyen du lieu den server handler -->
<form action="process_register.php" method="POST">
  <label for="user_email">Email dang ky:</label>
  <input type="email" id="user_email" name="email" placeholder="user@example.com"
     required>

  <label for="user_pass">Mat khau:</label>
  <input type="password" id="user_pass" name="password" placeholder="******"
    required>

  <button type="submit">Dang ky tai khoan</button>
</form>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Minh Họa Form Action Nhập Thử**

Form Đăng Ký Tài Khoản (Gửi tới: `process_register.php`):

Email đăng ký: nguyenvana@gmail.com

Mật khẩu: ••••••••

Đăng ký tài khoản

Mô Phỏng Trạng Thái Khi Người Dùng Nhấn Nút Submit ("Nhập Thử"):

- **Trình duyệt khởi tạo**: HTTP POST Request gửi tới URL `process_register.php`.
- **Dữ liệu đóng gói gửi đi**: `email=nguyenvana%40gmail.com&password=******`
- **Trạng thái xử lý Backend**: Server xử lý tệp `process_register.php` nhận dữ liệu, tiến hành mã hóa mật khẩu và trả về phản hồi chào mừng.

Đọc Thêm: Các Kỹ Thuật Nâng Cao Với Form Action Trong Lập Trình Modern Web

[Khái Niệm] Ghi Đè Đường Dẫn Action Bằng Thuộc Tính formaction (HTML5)

Trong đặc tả HTML5, nếu một form có nhiều nút bấm gửi dữ liệu cho các mục đích khác nhau (ví dụ: Nút "Gửi bài" và Nút "Lưu nháp"), ta có thể dùng thuộc tính `formaction` ngay trên nút submit để ghi đè thuộc tính `action` chung của form:

Ví dụ sử dụng thuộc tính formaction trên từng nút submit

```
<form action="/save-article" method="post">
  <!-- Nút 1: Gửi dữ liệu đến /save-draft -->
  <button type="submit" formaction="/save-draft">Luu Nhap</button>

  <!-- Nút 2: Sử dụng action mặc định của form (/save-article) -->
  <button type="submit">Xuat Ban</button>
</form>
```

[Mẹo] Xử Lý Form Action Trong Các Ứng Dụng Trang Đơn (SPA / AJAX)

Trong lập trình Frontend hiện đại với JavaScript (React, Vue, Angular, Fetch API/Axios):

- Lập trình viên thường gọi hàm `event.preventDefault()` trong sự kiện `onsubmit` để **chặn hành vi điều hướng trang mặc định** của thuộc tính `action`.
- Dữ liệu sau đó được đóng gói và gửi qua AJAX/Fetch API một cách bất đồng bộ mà không làm tải lại trang web, giúp tối ưu trải nghiệm người dùng (**UX**).

[Ghi Chú] Góc Phỏng Vấn: Các Câu Hỏi Thường Gặp Về Form Action

1. **Phân biệt action="", action="#" và bỏ qua thuộc tính action?**
   - `action=""` và bỏ trống `action`: Đều gửi form về chính URL trang hiện tại mà không làm đổi thẻ thăng.
   - `action="#"`: Gửi dữ liệu về trang hiện tại nhưng đính kèm vị trí `#` khiến trình duyệt tự cuộn lên đầu trang.
2. **Tại sao thẻ <form> vẫn cần thuộc tính action khi làm việc với React/Vue/Angular?**
   - Để đảm bảo tính năng **Progressive Enhancement** (Cải tiến tiệm tiến) và **Accessibility (a11y)**: Nếu JavaScript của người dùng bị tắt hoặc gặp sự cố tải, form vẫn có thể gửi dữ liệu lên server backend thông qua đường dẫn `action` truyền thống.

Chuyên Sâu Các Thuộc Tính Điều Khiển: `enctype`, `target`, `novalidate` & `autocomplete`

Bên cạnh `action` và `method`, thẻ `<form>` sở hữu 4 thuộc tính điều khiển nâng cao quyết định trực tiếp đến định dạng mã hóa gói tin HTTP, hành vi điều hướng cửa sổ và cơ chế kiểm soát bảo mật dữ liệu.

1. Chuyên Sâu Thuộc Tính enctype (Encoding Type) & Bí Quyết Upload File:

Thuộc tính `enctype` quy định **MIME type** của nội dung biểu mẫu được mã hóa khi gửi lên máy chủ thông qua phương thức `method="post"` (với `GET`, thuộc tính này hoàn toàn bị trình duyệt bỏ qua).

[Bảng] Ma Trận Phân Biệt 3 Giá Trị enctype Trong HTML Form

Giá Trị enctype	Cơ Chế Mã Hóa & Cấu Trúc Payload	Trường Hợp Ứng Dụng Thực Tế
`application/x-www-for`	**Giá trị mặc định**. Mọi ký tự được mã hóa: khoảng trắng thành dấu `+`, ký tự đặc biệt thành mã Hex (`%XX`), các cặp nối bằng dấu `&`.	Biểu mẫu văn bản thông thường (Đăng nhập, Tìm kiếm, Đăng ký thông tin chữ).
`multipart/form-data`	Dữ liệu **không bị mã hóa ký tự**, mà được chia thành các phân đoạn đa phần tách biệt bởi ranh giới phân cách (**Boundary**).	**Bắt buộc 100%** khi form có ô upload file (`<input type="file">`).
`text/plain`	Dữ liệu gửi đi dưới dạng văn bản thô, chỉ khoảng trắng đổi thành dấu `+`, không mã hóa ký tự đặc biệt.	Chỉ dùng cho mục đích gỡ lỗi (**Debugging**), không dùng cho sản phẩm thực tế.

[Cảnh Báo] Cạm Bẫy Thực Tế: Quên Khai Báo enctype="multipart/form-data" Khi Upload File

Nếu một biểu mẫu chứa thẻ `<input type="file">` nhưng thiếu khai báo `enctype="multipart/form-data"` trên thẻ `<form>`:

- Trình duyệt sẽ chỉ gửi **tên tệp tin** (chuỗi văn bản đơn thuần) thay vì toàn bộ luồng nhị phân (**Binary Stream**) của tệp.
- Phía máy chủ Backend (PHP `$_FILES`, Node.js `multer`, Python Django/FastAPI) sẽ nhận về giá trị rỗng và không thể lưu tệp tin.

Mã Nguồn: Biểu Mẫu Upload Tệp Tin Chuẩn Kỹ Thuật Với multipart/form-data

```
<!-- Bat buoc phai co method="post" va enctype="multipart/form-data" -->
<form action="/api/upload" method="post" enctype="multipart/form-data">
  <label for="user-avatar">Chon anh dai dien (Toi da 5MB):</label>
  <input type="file" id="user-avatar" name="avatar" accept="image/png, image/jpeg
    " required>

  <button type="submit">Tai Len Server</button>
</form>
```

2. Chuyên Sâu Thuộc Tính target (Điều Hướng Cửa Sổ Kết Quả):

Thuộc tính `target` xác định ngữ cảnh duyệt web (**Browsing Context**) nơi kết quả phản hồi từ máy chủ sẽ được hiển thị sau khi người dùng gửi form.

[Bảng] Bảng Tra Cứu 4 Từ Khóa Cốt Lõi Của Thuộc Tính target

Từ Khóa	Hành Vi Hiển Thị Phản Hồi	Kịch Bản Sử Dụng Thực Chiến
`_self`	**Mặc định**. Tải kết quả phản hồi ngay tại tab/khung hình hiện tại.	Luồng đăng nhập, gửi phản hồi thông thường.
`_blank`	Mở một **tab mới hoặc cửa sổ mới** để hiển thị kết quả xử lý.	Xuất hóa đơn PDF, tạo báo cáo thống kê, cổng thanh toán.
`_parent`	Tải phản hồi vào khung hình cha (**Parent Frame**) chứa form hiện tại.	Biểu mẫu nhúng trong iframe phân cấp đa tầng.
`_top`	Tải phản hồi vào toàn bộ cửa sổ trình duyệt cao nhất, phá vỡ mọi khung iframe.	Thoát khỏi iframe khi hết phiên đăng nhập.

[Mẹo] Kỹ Thuật Thực Chiến: Submit Form Không Reload Trang Bằng target Và iframe Ẩn

Trước khi kỹ thuật AJAX và Fetch API ra đời, các lập trình viên sử dụng thuộc tính `target` kết hợp với một thẻ `<iframe name="hidden-frame">` ẩn để gửi form ngầm mà không làm tải lại trang web chính:

`<form action="/save" method="post" target="hidden-frame">` → *Kết quả phản hồi nạp vào iframe ẩn!*

3. Chuyên Sâu Thuộc Tính novalidate (Kiểm Soát Native Validation):

`novalidate` là một thuộc tính boolean. Khi hiện diện trên thẻ `<form>`, nó ra lệnh cho trình duyệt **vô hiệu hóa hoàn toàn cơ chế kiểm tra dữ liệu mặc định (Native HTML5 Validation)**.

[Khái Niệm] Khi Nào Kỹ Sư Frontend Cần Sử Dụng novalidate?

- **Thay thế bằng Custom JavaScript Validation**: Khi giao diện ứng dụng sử dụng các thư viện kiểm tra lỗi chuyên nghiệp (React Hook Form, Formik, Yup, Zod, VeeValidate) và cần hiển thị tooltip/thông báo lỗi tùy biến theo thiết kế UI riêng thay vì hộp thông báo màu xám mặc định của trình duyệt.
- **Tính năng Lưu Bản Nháp (Save Draft)**: Người dùng muốn lưu tạm biểu mẫu khi chưa hoàn thành tất cả các trường `required`.
- **Lưu ý an ninh**: Thuộc tính `novalidate` chỉ tắt kiểm tra ở phía trình duyệt người dùng (**Client-side**). Máy chủ Backend **bắt buộc 100% vẫn phải kiểm tra lại dữ liệu** để phòng chống tấn công chèn mã độc.

Mã Nguồn: Biểu Mẫu Tắt Native Validation Để Tự Xử Lý Bằng JavaScript

```
<!-- Trinh duyet se khong tu dong chan submit khi de trong o required -->
<form action="/api/posts" method="post" novalidate id="postForm">
  <label for="title">Tieu de bai viet:</label>
  <input type="text" id="title" name="title" required>

  <button type="submit" name="action" value="publish">Xuat Ban</button>
  <button type="submit" name="action" value="draft" formnovalidate>Luu Ban Nhap<
    /button>
</form>
```

4. Chuyên Sâu Thuộc Tính autocomplete (Kiểm Soát Gợi Ý Cấp Độ Form):

Thuộc tính `autocomplete` trên thẻ `<form>` có hai giá trị: `on` (mặc định) và `off`.

- `autocomplete="on"`: Trình duyệt tự động lưu và hiển thị danh sách gợi ý các giá trị người dùng đã từng nhập trước đó cho tất cả các ô input bên trong form.
- `autocomplete="off"`: Yêu cầu trình duyệt **không ghi nhớ và không gợi ý dữ liệu** cho toàn bộ form.

[Cảnh Báo] Khuyến Cáo Bảo Mật Thực Chiến Đối Với autocomplete="off"

Quy chuẩn an ninh mạng ngành tài chính ngân hàng (**FinTech**) và thương mại điện tử khuyến nghị tắt tự động điền đối với các biểu mẫu giao dịch nhạy cảm:

- Biểu mẫu nhập mã xác thực OTP, mật khẩu giao dịch hoặc mã PIN thẻ ngân hàng.
- Tránh rò rỉ dữ liệu khi người dùng sử dụng máy tính công cộng hoặc thiết bị dùng chung.

5.1.2 2. Phân Tích Chuyên Sâu Phương Thức Gửi Dữ Liệu GET vs POST (`method`)

Thuộc tính `method` trong thẻ `<form>` xác định phương thức giao thức truyền tải siêu văn bản (**HTTP Method**) được trình duyệt sử dụng để chuyển phát dữ liệu người dùng đến tài nguyên máy chủ được chỉ định ở `action`. Trong chuẩn W3C HTML5 và IETF RFC 7231, hai phương thức kinh điển và quan trọng nhất mà mọi kỹ sư Frontend bắt buộc phải làm chủ là `GET` và `POST`.

1. Sơ Đồ Kiến Trúc & Luồng Vận Hành Dữ Liệu (GET vs POST Architecture)

[Khái Niệm] Bản Chất Vận Hành Cốt Lõi Theo Chuẩn HTTP/1.1 & HTTP/2

Sự khác biệt cốt tử giữa GET và POST nằm ở **kênh truyền tải dữ liệu** trong cấu trúc gói tin HTTP Request:

- **Phương thức GET**: Đóng gói dữ liệu trực tiếp lên dòng yêu cầu URL (**Request Line**) dưới dạng chuỗi truy vấn (**Query String**). Thân của gói tin yêu cầu (**Request Body**) hoàn toàn để trống.
- **Phương thức POST**: Giữ cho đường dẫn URL sạch sẽ và đóng gói toàn bộ tham số vào phần thân dữ liệu ngầm (**Request Payload / HTTP Body**) qua luồng dữ liệu TCP, cho phép truyền tải an toàn và không giới hạn dung lượng.

Sơ Đồ Đối Sánh Kiến Trúc Kênh Truyền Dữ Liệu: GET vs POST



2. Phương Thức GET: Cấu Trúc Query String & Kịch Bản Lọc Dữ Liệu

Mục tiêu & Cách thức hoạt động:

Phương thức `GET` được sử dụng để gửi yêu cầu lấy dữ liệu từ máy chủ. Dữ liệu nhập trong form sẽ được đính kèm trực tiếp vào đuôi thanh địa chỉ URL của trình duyệt dưới dạng chuỗi truy vấn (**Query String**):

- Nối dữ liệu biểu mẫu vào URL theo các cặp `name=value`.
- Các cặp dữ liệu được phân cách bằng ký tự `?` cho cặp đầu tiên và `&` cho các cặp tiếp theo (cú pháp: `?key1=value1&key2=value2`).
- Dung lượng dữ liệu bị giới hạn nghiêm ngặt bởi trình duyệt (thường khoảng $\sim$ 2048 ký tự).
- Dữ liệu xuất hiện công khai trên thanh địa chỉ và tự động lưu vào lịch sử trình duyệt (**Browser History**).
- Nếu không khai báo thuộc tính `method`, trình duyệt sẽ mặc định coi form sử dụng phương thức `GET`.

Giải Phẫu Cấu Trúc Chuỗi Truy Vấn (Query String Anatomy):

Một đường link URL gửi bằng phương thức GET luôn bao gồm cấu trúc thành phần giải phẫu chuẩn như sau:

```
https://www.example.com/search?q=HTML5&page=1
```

Đường dẫn tĩnh (Path) — URL Endpoint | ? — Bắt đầu Query | q=HTML5 — Cặp 1 (Key = Value) | & — Phân cách | page=1 — Cặp 2 (Key = Value)

Mã Nguồn Form Tìm Kiếm Sử Dụng Phương Thức GET

```html
<form action="https://www.example.com/search" method="GET">
    <label for="query">Tìm kiếm bài viết:</label>
    <input type="text" id="query" name="q">
    <button type="submit">Tìm Kiếm</button>
</form>
```

BROWSER PREVIEW — **Mô Phỏng Trình Duyệt: Form Tìm Kiếm GET**

Thanh địa chỉ URL trình duyệt (Nối Query String trực tiếp):

`https://www.example.com/search?q=HTML5+Masterclass`

Giao diện hiển thị trong trang:

Tìm kiếm bài viết:

HTML5 Masterclass [Tìm Kiếm]

[Mẹo] Nguyên Tắc Lựa Chọn: Khi Nào NÊN và KHÔNG NÊN Dùng GET?

- **NÊN DÙNG**: Khi thực hiện các thao tác chỉ đọc dữ liệu (**Read-only**), tìm kiếm (**Search**), lọc sản phẩm (**Filter**), phân trang danh sách (**Pagination**), hoặc khi muốn người dùng có thể lưu Bookmark và sao chép liên kết URL có chứa đầy đủ tham số để chia sẻ cho người khác.
- **KHÔNG NÊN DÙNG**: Khi gửi dữ liệu nhạy cảm (mật khẩu tài khoản, số thẻ tín dụng, mã bảo mật OTP) vì dữ liệu sẽ bị lộ hoàn toàn trên URL bar, lịch sử trình duyệt và máy chủ Proxy/Log; hoặc khi cần gửi dữ liệu kích thước lớn (upload tệp tin, tải ảnh, gửi nội dung bài viết dài).

Ví Dụ Thực Tế 1: Form Lọc Sản Phẩm Thương Mại Điện Tử (Filter Form)

Trong các ứng dụng thương mại điện tử (Shopee, Tiki, Lazada), khi người dùng chọn lọc sản phẩm theo Thương hiệu và Khoảng giá, form bắt buộc sử dụng phương thức `GET` để người dùng dễ dàng lưu Bookmark hoặc chia sẻ link kết quả lọc cho bạn bè:

Mã Nguồn Form Lọc Sản Phẩm Bằng Phương Thức GET

```html
<form action="https://shopee.vn/search" method="GET">
  <label for="brand">Thương hiệu:</label>
  <select id="brand" name="brand">
    <option value="apple">Apple</option>
    <option value="samsung">Samsung</option>
  </select>

  <label for="max_price">Giá tối đa:</label>
  <input type="number" id="max_price" name="max_price" value="20000000">

  <button type="submit">Lọc Sản Phẩm</button>
</form>
```

BROWSER PREVIEW **Mô Phỏng Trình Duyệt: Kết Quả Lọc Sản Phẩm GET Với Link Chia Sẻ**

Thanh địa chỉ URL kết quả (Cho phép Copy/Paste chia sẻ link cho bạn bè):

`https://shopee.vn/search?brand=apple&max_price=20000000`

Giao diện kết quả:

Kết quả lọc: Đang hiển thị danh sách sản phẩm **Apple** có giá dưới **20.000.000đ**.

3. Phương Thức POST: Đóng Gói Request Body & Kịch Bản Bảo Mật

Mục tiêu & Cách thức hoạt động:

Phương thức `POST` được sử dụng để gửi dữ liệu cần xử lý hoặc thay đổi trạng thái trên máy chủ. Dữ liệu biểu mẫu được đóng gói ngầm bên trong thân của yêu cầu HTTP (**HTTP Request Body**):

- Dữ liệu được gửi hoàn toàn ẩn ngầm, không xuất hiện trên thanh địa chỉ URL.
- **Không giới hạn kích thước** dung lượng dữ liệu gửi đi (cho phép upload file vài GB hoặc bài viết dài).
- Dữ liệu không bị lưu vào lịch sử trình duyệt hay lưu cache tự động.
- Đảm bảo tính an toàn và bảo mật cao hơn so với GET.

Mã Nguồn Form Đăng Nhập Sử Dụng Phương Thức POST

```
<form action="https://www.example.com/login" method="POST">
    <label for="username">Tên đăng nhập:</label>
    <input type="text" id="username" name="user">

    <label for="password">Mật khẩu:</label>
    <input type="password" id="password" name="pass">

    <button type="submit">Đăng Nhập</button>
</form>
```

BROWSER PREVIEW — **Mô Phỏng Trình Duyệt: Form Đăng Nhập POST**

Thanh địa chỉ URL trình duyệt (Dữ liệu ngầm trong Body, URL sạch):

`https://www.example.com/login` (Không hiện username hay password!)

Giao diện hiển thị trong trang:

Tên đăng nhập:

admin_user

Mật khẩu:

••••••

Đăng Nhập

[Mẹo] Nguyên Tắc Lựa Chọn: Khi Nào NÊN và KHÔNG NÊN Dùng POST?

- **NÊN DÙNG**: Khi gửi thông tin định danh và nhạy cảm (đăng nhập tài khoản, đổi mật khẩu, xác thực thẻ ngân hàng, thanh toán), khi gửi dữ liệu đính kèm tệp tin đa phương tiện (`<input type="file">`), hoặc khi thực hiện các tác vụ tạo mới (**Create**), cập nhật (**Update**) hay làm thay đổi trạng thái tài nguyên cơ sở dữ liệu trên máy chủ.
- **KHÔNG NÊN DÙNG**: Khi chỉ muốn thực hiện các truy vấn tìm kiếm nhẹ, đọc dữ liệu công khai hoặc khi muốn người dùng có khả năng chia sẻ liên kết kết quả trực tiếp cho người khác.

Ví Dụ Thực Tế 2: Form Thanh Toán & Upload Tệp Tin (Checkout & Upload Form)

Khi người dùng thực hiện thanh toán đơn hàng hoặc đính kèm ảnh hóa đơn lên server, form bắt buộc sử dụng phương thức `POST` để bảo mật dữ liệu thẻ và truyền tải tệp tin dung lượng lớn:

Mã Nguồn Form Thanh Toán & Tải File Bảo Mật Bằng POST

```
<form action="https://store.com/api/checkout" method="POST" enctype="
    multipart/form-data">
  <label for="card">Số thẻ tín dụng:</label>
  <input type="text" id="card" name="credit_card">

  <label for="receipt">Ảnh hóa đơn nhận hàng:</label>
  <input type="file" id="receipt" name="receipt_img">

  <button type="submit">Xác Nhận Thanh Toán</button>
</form>
```

BROWSER PREVIEW Mô Phỏng Trình Duyệt: Form Thanh Toán POST Bảo Mật An Toàn

Thanh địa chỉ URL trình duyệt (Được bảo mật hoàn toàn, ẩn ngầm thông tin thẻ):

`https://store.com/api/checkout` (Không hiện số thẻ ngân hàng!)

Giao diện giao dịch:

Trạng thái: Dữ liệu số thẻ •••• •••• •••• **8888** và tệp tin ảnh hóa đơn được mã hóa và đóng gói ẩn ngầm trong HTTP Request Body.

4. Ma Trận So Sánh Kỹ Thuật Toàn Diện (GET vs POST Comparison Matrix)

Dưới đây là bảng ma trận tổng hợp toàn bộ các tiêu chí kỹ thuật chuẩn mực đối sánh giữa hai phương thức theo chuẩn đặc tả W3C và IETF RFC 7231:

[Bảng] Ma Trận So Sánh Kỹ Thuật Toàn Diện Giữa Phương Thức GET Và POST

Tiêu Chí So Sánh	Phương Thức GET	Phương Thức POST
Hiển thị trên URL	**Có** (Nối đính kèm sau dấu `?` dạng Query String).	**Không** (Đóng gói ẩn hoàn toàn trong HTTP Request Body).
Giới hạn dung lượng	**Bị giới hạn** ($\sim$ 2048 ký tự tùy trình duyệt).	**Không giới hạn** dung lượng dữ liệu gửi đi.
Mức độ bảo mật	**Kém** (Dễ lộ mật khẩu/thông tin trên thanh URL).	**Bảo mật cao hơn** (Dữ liệu gửi ẩn trong Body).
Lưu trong Lịch sử (History)	**Có** (Lưu đầy đủ URL chứa tham số).	**Không** (Không lưu tham số form vào History).
Lưu Cache trình duyệt	**Có** (Trình duyệt tự cache để tăng tốc tải lại).	**Không** (Mỗi lần submit là một request riêng biệt).
Bookmark URL kết quả	**Có thể Bookmark** và chia sẻ liên kết dễ dàng.	**Không thể Bookmark** vì URL không chứa dữ liệu.
Khả năng Upload File	**Không thể** dùng để tải tệp tin lên server.	**Hỗ trợ tốt** (kết hợp với `enctype="multipart/form-data"`).
Tính An Toàn (Safe / Idempotent)	**Safe & Idempotent** (Gửi lặp lại nhiều lần không làm thay đổi trạng thái server).	**Non-Idempotent** (Mỗi lần gửi lặp lại có thể tạo ra đơn hàng mới).
Ngữ cảnh sử dụng chính	Tìm kiếm, lọc sản phẩm, truy vấn chỉ đọc (**Read-only**).	Đăng nhập, đăng ký, thanh toán, upload file, cập nhật dữ liệu.

5. Góc Phỏng Vấn Chuyên Sâu & Cạm Bẫy Thực Tế (Engineering Insights)

[Khái Niệm] 2 Câu Hỏi Phỏng Vấn Thường Gặp Về Form Methods

1. **Câu hỏi 1: "Phương thức POST có thực sự mã hóa bảo mật dữ liệu tuyệt đối không nếu trang web dùng giao thức HTTP thường?"**
   - **Trả lời chuẩn**: **KHÔNG!** Phương thức POST chỉ có tác dụng giấu dữ liệu khỏi thanh địa chỉ URL của màn hình người dùng. Nếu trang web chạy trên giao thức `http://` thông thường (chưa có chứng chỉ SSL/TLS `https://`), toàn bộ Request Body vẫn được truyền qua mạng Internet dưới dạng văn bản thô (**Plaintext**). Bất kỳ ai sử dụng phần mềm bắt gói tin (như Wireshark) trên cùng mạng Wi-Fi đều có thể đọc trọn vẹn mật khẩu của bạn. Do đó, bảo mật biểu mẫu **bắt buộc phải kết hợp POST với HTTPS**.
2. **Câu hỏi 2: "Tại sao khi bấm F5 hoặc nút Reload trang sau khi gửi Form POST, trình duyệt lại bật cửa sổ cảnh báo 'Confirm Form Resubmission'?"**
   - **Nguyên nhân**: POST là phương thức **Non-Idempotent** (Không có tính bất biến). Khi người dùng Reload lại trang nhận kết quả, trình duyệt hỏi xác nhận để ngăn chặn việc người dùng vô tình gửi lại dữ liệu cũ lần 2 (ví dụ: bị trừ tiền hai lần cho cùng một đơn hàng).
   - **Giải pháp kiến trúc chuẩn ngành**: Áp dụng mô hình thiết kế **PRG Pattern (Post/Redirect/Get)**. Khi Server Backend nhận được yêu cầu POST thành công, server không trả về trang HTML trực tiếp mà trả về mã trạng thái `HTTP 302 / 303 Redirect` chuyển hướng trình duyệt của người dùng sang một đường dẫn GET thông báo thành công. Khi đó, người dùng có F5 bao nhiêu lần cũng chỉ kích hoạt yêu cầu GET an toàn!

6. Kịch Bản Thực Tế (Case Studies) & Bí Mật Triết Lý Mặc Định Của Phương Thức GET

a. Tại Sao POST Tiện Lợi Và Kín Đáo Nhưng GET Lại Được Chọn Làm Mặc Định?

Nhiều kỹ sư mới vào nghề thường thắc mắc: *"Phương thức POST vừa kín đáo, không lộ thông tin trên thanh địa chỉ, lại không bị giới hạn độ dài dữ liệu, vậy tại sao W3C và Tim Berners-Lee lại đặt GET làm phương thức mặc định cho thẻ `<form>`?"*

Bí mật đằng sau quyết định kiến trúc này gồm 3 lý do sống còn đối với mạng Internet toàn cầu:

1. **Triết lý Siêu văn bản & Khả năng chia sẻ liên kết (Shareable URLs)**: Mạng Web (World Wide Web) sinh ra để tra cứu và liên kết tài nguyên qua URL (**Hyperlinks**). Với phương thức GET, toàn bộ tham số tìm kiếm/lọc nằm ngay trên URL. Người dùng có thể **sao chép đường dẫn gửi cho bạn bè qua Zalo/Messenger**, lưu vào **Bookmark trình duyệt**, hoặc sử dụng nút **Back / Forward** mượt mà. Nếu mặc định là POST, dữ liệu bị giấu trong thân gói tin (Body), URL chỉ trơ trọi một địa chỉ tĩnh, người dùng không thể chia sẻ liên kết cho nhau và tính liên kết của Internet sẽ hoàn toàn bị phá vỡ.
2. **Tận dụng Bộ nhớ đệm (Caching) & Hạ tầng CDN toàn cầu**: Yêu cầu GET có tính năng **Cacheable** (cho phép trình duyệt và mạng lưới máy chủ trung gian CDN như Cloudflare, Akamai lưu bản sao). Khi hàng triệu người dùng cùng tra cứu chung một từ khóa hay xem cùng một trang danh mục sản phẩm, máy chủ chỉ cần xử lý một lần, các lượt sau được trả về tức thì từ bộ nhớ đệm. Trái lại, POST mặc định không được cache; nếu form mặc định là POST, hệ thống máy chủ toàn cầu sẽ nhanh chóng

bị quá tải.

3. **Tính an toàn khi người dùng tải lại trang (Idempotency)**: GET là phương thức mang tính **Idempotent** (chỉ đọc dữ liệu, bấm F5 tải lại 100 lần cũng không làm thay đổi hay mất mát dữ liệu). POST mang tính **Non-Idempotent** (tạo đơn hàng, chuyển tiền). Nếu mặc định là POST, mỗi khi tải lại trang người dùng sẽ liên tục bị làm phiền bởi hộp thoại cảnh báo *"Confirm Form Resubmission"*.

b. Case Study Thực Chiến: Bài Toán Phân Trang & Trình Chiếu Slide (State in URL):

[Khái Niệm] Hiện Tượng: Trình Chiếu 33 Trang Slide — Tại Trang 2, Bấm F5 Vẫn Ở Đúng Trang 2

Đặt vấn đề: Trên một trang web trình chiếu tài liệu gồm 33 trang slide, bạn đang xem ở trang 2. Khi bấm phím F5 để tải lại trang, trình duyệt vẫn mở đúng vị trí slide trang 2 mà không bị bật ngược về trang 1. Kỹ thuật đằng sau hiện tượng này là gì?

Bản chất kỹ thuật: Đó chính là nguyên lý **Lưu trạng thái vào URL (State in URL)** thông qua tham số `GET Query String`:

- Khi bấm Next sang trang 2, đường dẫn URL được cập nhật thành: `https://example.com/slides?page=2`.
- Khi bấm F5, trình duyệt gửi lại yêu cầu GET mang theo tham số `page=2`. Mã nguồn máy chủ (Backend) hoặc Router JavaScript phía Client đọc tham số này và tức thời hiển thị đúng slide số 2.
- **Lợi ích thực chiến**: Bạn có thể gửi thẳng đường link `.../slides?page=2` cho đồng nghiệp hay giảng viên, người nhận nhấp chuột vào sẽ nhảy thẳng tới đúng trang 2 để thảo luận mà không cần lật lại từ đầu.

c. Bảng Đối Sánh Kịch Bản Thực Tế: Khi Nào Bắt Buộc Dùng GET vs POST?

[Bảng] Bảng Tra Cứu Kịch Bản Ứng Dụng Thực Tế Của GET Và POST Trên Các Nền Tảng Lớn

Nền Tảng	Tính Năng Thực Tế	Phương Thức & Dữ Liệu	Lý Do Kỹ Thuật Bắt Buộc
Google Search	Ô tìm kiếm từ khóa	`GET (/search?q=HTML5)`	Cho phép chia sẻ URL kết quả, lưu Bookmark, lưu Cache mạng.
Shopee / Tiki	Bộ lọc giá, danh mục sản phẩm	`GET (/search?cat=1&price_mi`	Lưu trạng thái bộ lọc trên URL, hỗ trợ SEO và gửi link cho bạn bè.
YouTube	Tìm kiếm video	`GET (/results?search_query=`	Hỗ trợ duyệt lịch sử xem và chia sẻ danh sách kết quả tra cứu.
Facebook / TikTok	Form Đăng nhập tài khoản	`POST (Gửi ngầm trong Body)`	Giấu mật khẩu khỏi URL, không lưu thông tin nhạy cảm vào Browser History.
Cổng VNPAY / MoMo	Nút "Xác nhận chuyển tiền"	`POST (Payload giao dịch ẩn)`	Ngăn chặn việc bấm F5 vô tình kích hoạt chuyển tiền lần 2 (**Non-Idempotent**).
Canva / Google Drive	Nút tải lên (Upload tệp tin)	`POST (multipart/form-data)`	Kích thước tệp lớn vượt xa giới hạn độ dài ký tự của URL.

5.1.3 3. Chuyên Sâu Thẻ `<input>` & 22+ Loại Input Types Trong HTML5

Thẻ `<input>` là phần tử đa năng nhất trong biểu mẫu HTML. Phần tử này không có thẻ đóng (**Self-closing tag**) và thay đổi hoàn toàn giao diện cũng như cơ chế hoạt động dựa trên thuộc tính `type`.

Cấu trúc cơ bản:

Mã Nguồn Cấu Trúc Khai Báo Thẻ Input Cơ Bản

```
<input type="text" name="username" placeholder="Nhap ten nguoi dung...">
```

[Bảng] Nhóm 1: 10 Loại Input Nhập Văn Bản, Số Liệu & Tìm Kiếm

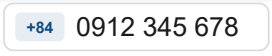
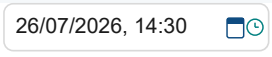
Loại type	Ý Nghĩa & Cơ Chế Hoạt Động	Cú Pháp Mã Nguồn
`text`	Ô nhập văn bản một dòng thô (mặc định nếu không ghi type).	`<input type="text">`
`password`	Ô nhập mật khẩu (tự động ẩn ký tự bằng dấu chấm *).	`<input type="password">`
`email`	Nhập địa chỉ Email (trình duyệt tự động kiểm tra định dạng @).	`<input type="email">`
`url`	Nhập đường dẫn website (`https://...`).	`<input type="url">`
`number`	Nhập số nguyên/số thực (có mũi tên tăng giảm).	`<input type="number" min="1" max="10">`
`tel`	Nhập số điện thoại (tự động bật bàn phím số trên Mobile).	`<input type="tel">`
`search`	Ô nhập tìm kiếm (có nút biểu tượng xóa nhanh `x`).	`<input type="search">`
`date`	Bộ chọn Ngày / Tháng / Năm dạng hộp lịch (**DatePicker**).	`<input type="date">`
`time`	Bộ chọn Giờ / Phút / Giây.	`<input type="time">`
`datetime-local`	Chọn cả Ngày và Giờ theo giờ địa phương.	`<input type="datetime-local">`

[Bảng] Nhóm 2: 12 Loại Input Lựa Chọn, Color, File & Nút Bấm

Loại type	Ý Nghĩa & Cơ Chế Hoạt Động	Cú Pháp Mã Nguồn
`month`	Chọn Tháng và Năm.	`<input type="month">`
`week`	Chọn Tuần và Năm trong năm.	`<input type="week">`
`color`	Bộ chọn màu sắc dạng bảng màu (**Color Picker**).	`<input type="color">`
`checkbox`	Ô vuông tick chọn nhiều phương án độc lập cùng lúc.	`<input type="checkbox">`
`radio`	Nút tròn chọn 1 trong nhiều phương án duy nhất trong cùng nhóm `name`.	`<input type="radio">`
`range`	Thanh trượt chọn giá trị số nằm trong khoảng Min – Max.	`<input type="range" min="0" max="100">`
`file`	Chọn và tải tệp tin từ máy tính lên server.	`<input type="file" multiple>`
`hidden`	Trường dữ liệu ẩn (gửi tham số ngầm, không hiển thị giao diện UI).	`<input type="hidden" value="123">`
`submit`	Nút gửi biểu mẫu lên máy chủ xử lý.	`<input type="submit" value="Gui">`
`reset`	Nút khôi phục toàn bộ các ô nhập về giá trị ban đầu.	`<input type="reset" value="Lam lai">`
`button`	Nút bấm thông thường (dùng kết hợp với sự kiện JavaScript).	`<input type="button" value="Click">`
`image`	Nút bấm gửi form dạng hình ảnh đồ họa tùy chỉnh.	`<input type="image" src="btn.png">`

Bảng Phụ Tra Cứu Giao Diện Minh Họa Trực Quan (Modern UI Controls):

[Bảng] Bảng Phụ 1: Minh Họa Giao Diện (10 Loại Input Nhập Liệu & Tìm Kiếm)

Loại Input	Giao Diện Minh Họa (Modern UI)	Đặc Điểm Trực Quan
`text`		Ô nhập văn bản tiêu chuẩn, có con trỏ văn bản nhấp nháy.
`password`		Che giấu ký tự nhập bằng các chấm tròn bảo mật kèm biểu tượng ẩn/hiện mật khẩu.
`email`		Định dạng email kèm biểu tượng phong bì thư nhận diện ngữ cảnh.
`url`		Tích hợp tiền tố giao thức `https://` dạng khối nhãn dán hiện đại.
`number`		Tích hợp cụm nút Stepper hai chiều tăng/giảm giá trị số ở cạnh phải.
`tel`		Tích hợp tiền tố mã quốc gia di động chuẩn quốc tế (ví dụ: +84).
`search`		Hộp tìm kiếm bo cong mềm mại với biểu tượng kính lúp và nút xóa nhanh văn bản.
`date`		Hộp chọn ngày kèm biểu tượng lịch DatePicker trực quan.
`time`		Hộp chọn thời gian kèm biểu tượng đồng hồ kim chuyên dụng.
`datetime-local`		Bộ chọn kết hợp Ngày và Giờ địa phương với biểu tượng đồng bộ.

[Bảng] Bảng Phụ 2: Minh Họa Giao Diện (12 Loại Input Lựa Chọn, Color, File & Nút Bấm)

Loại Input	Giao Diện Minh Họa (Modern UI)	Đặc Điểm Trực Quan
`month`	Tháng 07, 2026	Bộ chọn Tháng và Năm với biểu tượng nhận diện DatePicker.
`week`	Tuần 30, 2026 W30	Bộ chọn Tuần chuyên dụng với nhãn hiển thị số tuần (ví dụ: W30).
`color`	#0F4C81	Hộp chọn màu sắc Color Swatch với mã màu Hex và biểu tượng bảng màu.
`checkbox`	Đồng ý điều khoản	Hộp kiểm vuông bo góc với dấu tick vector sắc nét cho phép chọn nhiều mục.
`radio`	Thanh toán thẻ	Nút tròn radio đồng tâm đại diện cho lựa chọn duy nhất trong một nhóm.
`range`	60%	Thanh trượt Slider hiện đại hiển thị trực quan tiến trình và con chạy Thumb.
`file`	Choose File avatar.png	Nút tải tệp đính kèm bo góc hiện đại kèm tên tệp đã chọn.
`hidden`	*<input type="hidden"> (0px DOM*	Trường dữ liệu ngầm, hoàn toàn không chiếm diện tích hiển thị trên giao diện (0px).
`submit`	Gửi Dữ Liệu	Nút gửi biểu mẫu màu chủ đạo nổi bật kèm mũi tên hành động.
`reset`	Làm Lại	Nút khôi phục dữ liệu ban đầu kèm biểu tượng xoay hoàn tác.
`button`	Click Action	Nút bấm tùy chỉnh phục vụ gắn sự kiện JavaScript tương tác.
`image`	[IMG: Submit Button]	Nút gửi biểu mẫu thay thế bằng hình ảnh đồ họa tùy biến.

Bảng tra cứu các thuộc tính quan trọng của thẻ `<input>`:

[Bảng] Bảng Tra Cứu Các Thuộc Tính Cốt Lõi Của Thẻ <input>

Thuộc Tính	Ý Nghĩa Ngữ Nghĩa & Tác Dụng	Ví Dụ Khai Báo
`name`	Tên của trường dữ liệu dùng để đóng gói gửi lên máy chủ.	`name="email"`
`value`	Giá trị dữ liệu mặc định hoặc giá trị do người dùng nhập.	`value="Nguyen Van A"`
`placeholder`	Dòng chữ văn bản mờ gợi ý hiển thị bên trong ô nhập.	`placeholder="Nhap email..."`
`required`	Thuộc tính Boolean bắt buộc người dùng phải điền trước khi submit.	`<input required>`
`readonly`	Đặt ô nhập ở chế độ chỉ đọc, không cho sửa (vẫn gửi dữ liệu).	`<input readonly>`
`disabled`	Vô hiệu hóa ô nhập (không cho sửa và KHÔNG gửi dữ liệu).	`<input disabled>`
`maxlength`	Giới hạn số ký tự tối đa người dùng có thể gõ vào.	`<input maxlength="10">`
`min / max`	Đặt giới hạn giá trị nhỏ nhất / lớn nhất cho ô number, date, range.	`<input type="number" min="1" max="100">`
`step`	Bước nhảy tăng / giảm cho ô nhập số hoặc thanh trượt range.	`<input type="number" step="5">`
`multiple`	Cho phép chọn nhiều tệp tin hoặc nhập nhiều địa chỉ email.	`<input type="file" multiple>`
`checked`	Đánh dấu chọn sẵn mặc định cho nút radio hoặc ô checkbox.	`<input type="checkbox" checked>`
`autofocus`	Tự động đặt con trỏ chuột vào ô nhập ngay khi trang web tải xong.	`<input autofocus>`
`pattern`	Biểu thức chính quy (**Regex**) kiểm tra cú pháp dữ liệu nhập.	`<input pattern="[0-9]10">`
`inputmode`	Ép bàn phím ảo hiển thị đúng dạng (`numeric`, `decimal`, `tel`).	`<input inputmode="numeric">`
`enterkeyhint`	Đổi nhãn nút Enter trên bàn phím di động (`search`, `send`, `done`).	`<input enterkeyhint="send">`
`autocomplete`	Gợi ý điền nhanh hoặc đọc OTP SMS (`one-time-code`).	`autocomplete="one-time-code"`

[Khái Niệm] Các Thuộc Tính Tối Ưu Trải Nghiệm Di Động (Mobile UX)

- `inputmode="numeric|decimal|tel|search|email"`: Ép bàn phím ảo hiển thị đúng layout tối ưu tốc độ gõ phím.
- `enterkeyhint="search|send|done|next"`: Thay đổi nhãn nút Enter trên bàn phím mềm của smartphone.
- `autocomplete="one-time-code"`: Cho phép trình duyệt/hệ điều hành tự động đọc mã OTP gửi qua SMS để người dùng nhấn điền nhanh chỉ với 1 chạm.

Mã Nguồn Mẫu Tổng Hợp Các Loại Input Thường Dùng

```
<form action="/register" method="post">
  <label for="username">Ho va Ten:</label>
  <input type="text" id="username" name="username" placeholder="Ten" required>

  <label for="email">Email:</label>
  <input type="email" id="email" name="email" placeholder="Email" required>

  <label for="pass">Mat khau:</label>
  <input type="password" id="pass" name="pass" placeholder="Mat khau">

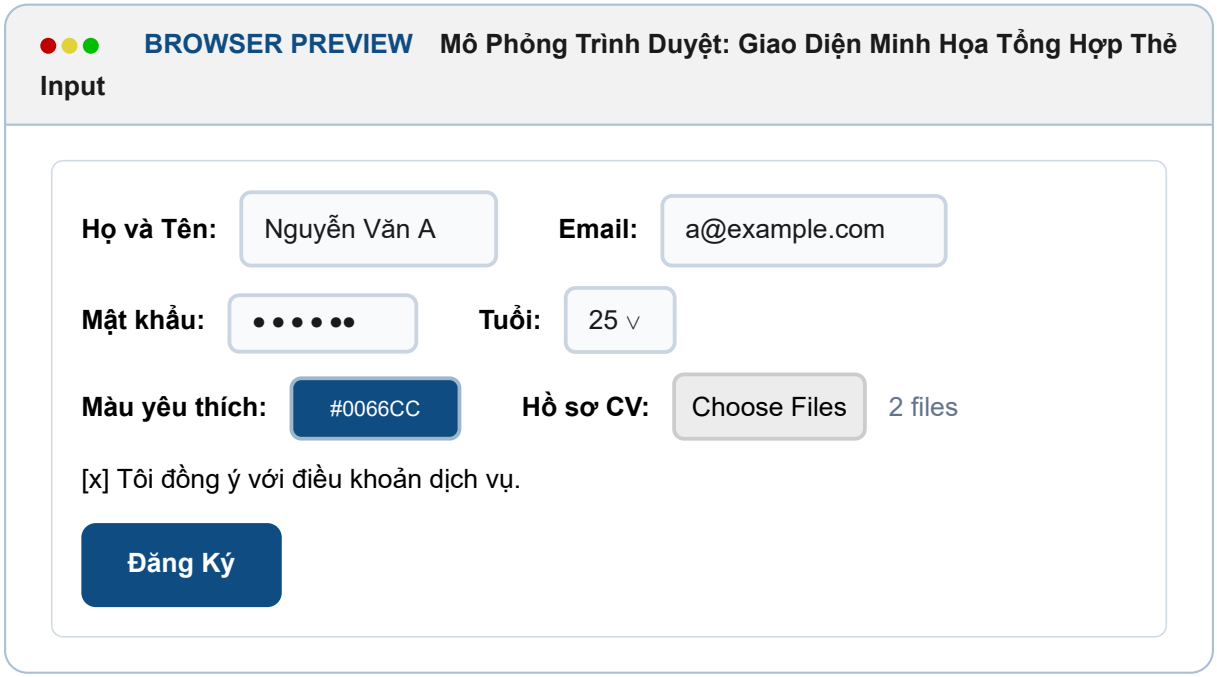
  <label for="age">Tuoi:</label>
  <input type="number" id="age" name="age" min="18" max="60" step="1">

  <label for="favcolor">Mau yeu thich:</label>
  <input type="color" id="favcolor" name="favcolor" value="#0066cc">

  <label for="cv">Ho so CV (upload nhieu file):</label>
  <input type="file" id="cv" name="cv" multiple>

  <label>
    <input type="checkbox" name="agree" required checked> Toi dong y
  </label>

  <input type="submit" value="Dang Ky">
</form>
```



5.1.4 4. Chuyên Sâu Thẻ `<label>`: Quy Tắc 2 Cách Dùng & Chuẩn Trợ Năng A11y

Thẻ `<label>` là phần tử ngữ nghĩa được sử dụng để gắn nhãn mô tả cho các ô nhập liệu (`<input>`, `<textarea>`, `<select>`). Mặc định, `<label>` là một phần tử Nội dòng (**Inline Element**).

[Khái Niệm] 3 Tác Dụng Cốt Lõi Của Thẻ <label>

- **1. Mô tả rõ ràng mục đích nhập liệu**: Giúp người dùng biết chính xác dữ liệu nào cần nhập vào ô biểu mẫu mà không bị nhầm lẫn.
- **2. Cơ chế kích hoạt Focus tự động (Click-to-Focus)**: Khi người dùng nhấp chuột vào dòng chữ mô tả của `<label>`, con trỏ nhập liệu sẽ tự động nhảy vào ô tương ứng. Điều này cực kỳ hữu ích với các nút nhỏ khó bấm như `checkbox` hay `radio` trên màn hình di động.
- **3. Chuẩn Trợ Năng (Accessibility - A11y)**: Trình đọc màn hình cho người khiếm thị (**Screen Readers**) phụ thuộc hoàn toàn vào `<label>` để đọc thành tiếng mô tả ô nhập khi con trỏ di chuyển tới.

Phân Tích Chi Tiết 2 Cách Sử Dụng Thẻ <label> Trong HTML5:

1. **Cách 1: Gói Trực Tiếp Input Trong Label (Implicit Labeling)**
   - Đặt trực tiếp thẻ `<input>` nằm bên trong thẻ `<label>...</label>`.
   - Không bắt buộc dùng thuộc tính `for`, mã nguồn ngắn gọn, thường áp dụng cho `checkbox` hoặc `radio`.

Mã Nguồn Cách 1: Gói Trực Tiếp Input Trong Label

```
<!-- Cách 1: Implicit Labeling -->
<label>
  Tên đăng nhập:
  <input type="text" name="username">
</label>
```

2. **Cách 2: Liên Kết Khớp for Và id (Explicit Labeling – QUY CHUẨN KHUYÊN DÙNG)**
   - Thuộc tính `for="..."` của `<label>` phải **trùng khớp chính xác 100%** với thuộc tính `id="..."` của `<input>`.
   - **Ưu điểm vượt trội**: Tách biệt cấu trúc HTML, cho phép đặt Label và Input ở hai vị trí khác nhau trong giao diện (ví dụ trong bảng hoặc cột bố cục CSS Flexbox/Grid).

Mã Nguồn Cách 2: Liên Kết Khớp Giá Trị for Và id

```
<!-- Cách 2: Explicit Labeling (Khai báo khuyên dùng) -->
<label for="user-email">Địa chỉ Email:</label>
<input type="email" id="user-email" name="email" placeholder="user@example.com
    ">
```

Ví Dụ Thực Tế & Giao Diện Trực Quan Tương Tác Focus:

Mã Nguồn Mẫu Form Đăng Ký Có Gắn Nhãn Label

```
<form action="/register" method="POST">
  <!-- Cấu trúc Explicit Labeling chuẩn W3C -->
  <label for="fullname">Họ và tên:</label>
  <input type="text" id="fullname" name="fullname" placeholder="Nhập họ tên...">

  <!-- Cấu trúc Implicit Labeling tiện lợi cho Checkbox -->
  <label>
    <input type="checkbox" name="terms" value="agreed">
    Tôi đồng ý với các điều khoản dịch vụ
  </label>
</form>
```

BROWSER PREVIEW **Mô Phỏng Trình Duyệt: Giao Diện Tương Tác Nhấp Chữ Focus Tự Động**

Họ và tên: Nguyễn Văn A | ← **Con trỏ tự động nhấp nháy focus khi nhấp vào chữ "Họ và tên"**

Tôi đồng ý với các điều khoản dịch vụ ← **Tự động tích chọn checkbox khi nhấp bất kỳ đâu trên dòng chữ!**

Ma Trận So Sánh Kỹ Thuật Khi Dùng Và Không Dùng Thẻ <label>:

[Bảng] Ma Trận So Sánh Tác Động Kỹ Thuật Khi Dùng Và Không Dùng Thẻ <label>

Tiêu Chí So Sánh	Biểu Mẫu CÓ Thẻ <label>	Biểu Mẫu KHÔNG CÓ Thẻ <label>
Hiển thị mô tả	**Đạt chuẩn**: Mô tả ngữ nghĩa độc lập, rõ ràng cho ô nhập.	**Kém**: Phải dùng `placeholder` tạm thời, dễ bị biến mất khi nhập chữ.
Khả năng Focus nhấp chữ	**Tự động focus**: Nhấp chuột vào chữ mô tả là con trỏ nhảy ngay vào ô nhập.	**Không thể focus**: Phải click chính xác vào ô nhập nhỏ → bất tiện trên Mobile.
Trợ năng (A11y)	**Xuất sắc**: Trình đọc màn hình đọc rõ tên ô nhập cho người khiếm thị.	**Rất kém**: Screen Reader không thể nhận diện mục đích của ô nhập liệu.
Thao tác Mobile	**Dễ bấm**: Vùng chạm (**Touch Target**) rộng, dễ bấm nút checkbox/radio.	**Khó bấm**: Nút quá nhỏ, rất dễ bấm nhầm trên màn hình cảm ứng di động.
Chuẩn W3C HTML5	**100% W3C Standard**: Mã nguồn chuẩn mực, dễ bảo trì và mở rộng.	**Không đạt chuẩn**: Code thiếu ngữ nghĩa, ảnh hưởng xấu tới điểm SEO & A11y.

[Ghi Chú] Kết Luận & Quy Tắc Vàng Khi Dùng Thẻ <label>

- **QUY TẮC BẮT BUỘC**: Khai báo thẻ `<label>` cho **100% ô nhập liệu** (`text`, `email`, `password`, `select`, `textarea`, `checkbox`, `radio`).
- **NGOẠI LỆ DUY NHẤT**: Chỉ bỏ `<label>` với nút bấm (`submit`, `button`) hoặc trường dữ liệu ẩn (`hidden`). Trong trường hợp thiết kế nút bấm Icon đặc thù không chữ, phải thay thế bằng thuộc tính `aria-label="..."`.

5.1.5 5. Chuyên Sâu Thuộc Tính value & name Trong HTML Forms

Trong kiến trúc của biểu mẫu web, `name` và `value` là hai thuộc tính có mối liên kết mật thiết nhất, đóng vai trò tạo nên nền tảng của giao thức truyền thông điệp Client – Server. Mọi cơ chế đóng gói dữ liệu của trình duyệt từ thời sơ khai của HTML cho đến các tiêu chuẩn HTML5 hiện đại đều vận hành dựa trên nguyên lý bắt cặp **Key = Value** này.

1. Phân Tích Chi Tiết Thuộc Tính value (Giá Trị Gửi Dữ Liệu)

[Khái Niệm] Định Nghĩa & Vai Trò Của Thuộc Tính value

Thuộc tính `value` định nghĩa **giá trị đại diện** của phần tử Form (`<input>`, `<textarea>`, `<button>`, `<option>`...). Đây chính là dữ liệu thực sự sẽ được đóng gói và gửi lên Server khi biểu mẫu được Submit.

Bảng Tra Cứu Chi Tiết Cách Hoạt Động Của Thuộc Tính value Theo Từng Loại Thẻ:

[Bảng] Bảng Tra Cứu Cách Hoạt Động Của Thuộc Tính value Theo Từng Loại Thẻ Form

Loại Thẻ	Vai Trò Của value	Ví Dụ Cú Pháp Mã Nguồn
`<input type="text">`	Giá trị mặc định hiển thị trong ô nhập. Người dùng có thể thay đổi.	`<input type="text" value="Nguyen Van A">`
`<input type="password">`	Mật khẩu mặc định (không nên dùng vì lý do bảo mật).	`<input type="password" value="123456">`
`<input type="email/number/`	Giá trị ban đầu của trường dữ liệu.	`<input type="number" value="18">`
`<input type="radio">`	Mỗi radio có value riêng. Khi chọn, chỉ value của radio được chọn gửi đi.	`<input type="radio" name="gender" value="male"> Nam`
`<input type="checkbox">`	Nếu được tick → gửi value. Nếu không tick → không gửi.	`<input type="checkbox" name="agree" value="yes">`
`<input type="hidden">`	Luôn gửi value nhưng không hiển thị trên giao diện.	`<input type="hidden" name="userid" value="123">`
`<input type="submit/reset/`	value chính là nhãn chữ hiển thị trên nút bấm.	`<input type="submit" value="Gửi form">`
`<input type="file">`	value chứa đường dẫn file (chỉ đọc, không gán trước được vì bảo mật).	`[x] Không gán sẵn được.`
`<option> (trong <select>)`	value được gửi khi option được chọn. Nếu không có value, text hiển thị sẽ được dùng.	`<option value="vn">Việt Nam</option>`
`<button>`	value chỉ được gửi nếu button có thuộc tính name.	`<button type="submit" name="action" value="save">Lưu</button>`

[Cảnh Báo] 4 Lưu Ý Vàng Bắt Buộc Phải Nhớ Về Thuộc Tính value

1. **Nếu không khai báo value**:
   - Ô nhập văn bản (`text`, `number`, `email`...): Mặc định gửi chuỗi rỗng `""`.
   - Ô chọn (`checkbox`, `radio`): Mặc định trình duyệt tự gán giá trị `"on"`.
2. **Ghi đè giá trị ban đầu**: Khi người dùng gõ thay đổi nội dung ô nhập → Lúc submit sẽ gửi **giá trị mới nhất**, ghi đè lên `value` mặc định ban đầu.
3. **Trường ẩn (`hidden`)**: Thuộc tính `value` là **cách duy nhất** để truyền dữ liệu ngầm (ví dụ: User ID, Session Token, CSRF Token).
4. **Checkbox không tick**: Nếu người dùng không tick chọn Checkbox, trình duyệt sẽ **không gửi bất kỳ tham số nào** (không phải gửi `value=""`).

2. Phân Tích Chi Tiết Thuộc Tính name (Khóa Nhận Diện Key-Value)

[Khái Niệm] Định Nghĩa & Cơ Chế Cặp Key = Value

Thuộc tính `name` dùng để **đặt tên định danh** cho phần tử Form. Khi biểu mẫu Submit, trình duyệt sẽ rà soát toàn bộ cây DOM bên trong thẻ `<form>`, trích xuất các điều khiển thành công (**Successful Controls**) và đóng gói dữ liệu dưới dạng cặp **Key = Value**:

Key (Tên tham số) = Giá trị của name & Value (Dữ liệu) = Nội dung nhập / Thuộc tính value

Sơ Đồ Cơ Chế Đóng Gói Dữ Liệu Form (Cặp Khóa name – Giá Trị value)



Ví Dụ Minh Họa Cơ Chế Gửi Dữ Liệu Dạng Query String Trên URL

```
<form action="/submit" method="GET">
  <input type="text" name="username" value="Alice">
  <input type="password" name="pass" value="1234">
  <input type="submit" value="Gửi">
</form>
```

BROWSER PREVIEW — **Kết Quả Truyền Dữ Liệu Khi Submit Form**

Khi người dùng bấm nút **"Gửi"**, địa chỉ URL trên trình duyệt sẽ tự động nối chuỗi Query String:

https://example.com/submit?username=Alice&pass=1234

[→] Trong đó `username` và `pass` chính là giá trị của thuộc tính `name`!

Mẹo Kỹ Thuật: Nhóm Nhiều Checkbox Dùng Chung Một name:

Mã Nguồn Nhiều Checkbox Sử Dụng Cùng Thuộc Tính name

```
<form action="/submit" method="GET">
  <label><input type="checkbox" name="hobby" value="music"> Âm nhạc</label>
  <label><input type="checkbox" name="hobby" value="sport"> Thể thao</label>
  <label><input type="checkbox" name="hobby" value="travel"> Du lịch</label>
  <input type="submit" value="Gửi">
</form>
```

[Ghi Chú] Giải Thích Kết Quả Khi Tick Chọn Nhiều Checkbox

Nếu người dùng tick chọn **"Âm nhạc"** và **"Du lịch"**, trình duyệt sẽ đóng gói và đính kèm nhiều tham số có chung tên hobby vào URL:

hobby=music&hobby=travel

So Sánh Thực Tế: Trường Hợp Có Sẵn value vs Không Có value Ban Đầu:

[Bảng] So Sánh Dữ Liệu Gửi Đi Khi Form Submit Với Các Trường Hợp value

Trường Hợp Mã Nguồn	Thao Tác Người Dùng	Chuỗi Dữ Liệu Submit Gửi Đi
`<input name="username" value="Tan">`	Bấm nút Submit ngay không sửa.	`username=Tan` (Lấy value sẵn)
`<input name="username">`	Gõ chữ **"Khoa"** rồi bấm Submit.	`username=Khoa` (Lấy chữ mới gõ)
`<input name="username">`	Để trống ô nhập rồi bấm Submit.	`username=` (Gửi chuỗi rỗng)

Ví Dụ Thực Tế Chuyên Sâu: 3 Kịch Bản Real-World Của value & name:

1. **Kịch bản 1: Nhóm Radio Đánh Giá Khảo Sát (Chung name, Khác value)**

Mã Nguồn Khảo Sát Mức Độ Hài Lòng Khách Hàng

```
<form action="/feedback" method="POST">
  <!-- Tất cả 3 nút Radio dùng chung name="rating" -->
  <label><input type="radio" name="rating" value="5"> Rất hài lòng</label>
  <label><input type="radio" name="rating" value="3"> Bình thường</label>
  <label><input type="radio" name="rating" value="1"> Không hài lòng</label>
  <input type="submit" value="Gửi đánh giá">
</form>
```

BROWSER PREVIEW — Kết Quả Đóng Gói Dữ Liệu Khảo Sát

Khi người dùng tick chọn nút **"Rất hài lòng"**, Form Submit POST sẽ đóng gói duy nhất 1 cặp Key-Value lên máy chủ:

rating = 5

2. **Kịch bản 2: Bộ Lọc Tìm Kiếm Sản Phẩm Thương Mại Điện Tử (Form GET Đa Tham Số)**

Mã Nguồn Form Lọc Sản Phẩm Đa Thuộc Tính

```
<form action="/search" method="GET">
  <input type="text" name="q" value="iPhone">
  <input type="number" name="price_min" value="1000">
  <input type="number" name="price_max" value="2000">
  <input type="hidden" name="in_stock" value="1">
  <input type="submit" value="Lọc kết quả">
</form>
```

BROWSER PREVIEW — **Chuỗi URL Query String Được Nối Tự Động**

Khi bấm **"Lọc kết quả"**, trình duyệt nối tất cả thuộc tính `name` và `value` thành URL:

/search?q=iPhone&price_min=1000&price_max=2000&in_stock=1

3. **Kịch bản 3: Biểu Mẫu Đặt Hàng Trực Tuyến (Kết Hợp select, textarea & hidden)**

Mã Nguồn Form Mua Hàng Sản Phẩm

```
<form action="/order" method="POST">
  <input type="hidden" name="product_id" value="PROD-9988">
  <label for="pay">Phương thức thanh toán:</label>
  <select id="pay" name="payment">
    <option value="momo">Ví MoMo</option>
    <option value="cod">Thanh toán khi nhận hàng (COD)</option>
  </select>
  <label for="note">Ghi chú đơn hàng:</label>
  <textarea id="note" name="note">Giao giờ hành chính</textarea>
  <button type="submit" name="btn_action" value="checkout">Đặt Hàng
    Ngay</button>
</form>
```

BROWSER PREVIEW — **Mô Phỏng Payload Đơn Hàng Gửi Lên Server**

Dữ liệu Request Body được đóng gói hoàn chỉnh gửi tới Server:

product_id=PROD-9988 & payment=momo & note=Giao giờ hành chính & btn_action=checkout

3. Quy Tắc & So Sánh Nhanh Giữa Thuộc Tính name Và id

[Bảng] Bảng Quy Tắc Và Lưu Ý Kỹ Thuật Của Thuộc Tính name

Thuộc Tính / Quy Tắc	Ý Nghĩa & Cơ Chế Hoạt Động
Bắt buộc để gửi dữ liệu	Nếu input không có `name`, trình duyệt bỏ qua và dữ liệu hoàn toàn không được gửi về máy chủ.
Có thể trùng name	Nhiều input có thể chia sẻ cùng một `name` để tạo nhóm lựa chọn hoặc mảng dữ liệu (ví dụ: `checkbox`, `radio`).
Khác biệt hoàn toàn với id	`id` phải duy nhất trên toàn tài liệu để liên kết CSS/JS/`<label>`, còn `name` đóng vai trò đặt tên khóa cho gói tin HTTP.
Truy xuất DOM thuận tiện	Trong JavaScript, có thể truy xuất nhanh phần tử qua cây Collection: `document.forms["orderForm"].elements["payment"]`.

[Bảng] Ma Trận So Sánh Kỹ Thuật Toàn Diện Giữa Thuộc Tính name Và id

Tiêu Chí Kỹ Thuật	Thuộc Tính name	Thuộc Tính id
Mục Đích Chính	Đặt tên định danh trường dữ liệu để gửi lên Backend Server qua HTTP Request.	Xác định duy nhất một phần tử DOM trong phạm vi toàn bộ trang web.
Tính Duy Nhất	**Không bắt buộc**. Có thể trùng nhau (dành cho Radio Group, Checkbox Array).	**Bắt buộc duy nhất 100%** trên toàn bộ tài liệu HTML.
Gửi Lên Server	**CÓ**. Trình duyệt lấy giá trị của `name` làm Key của tham số.	**KHÔNG**. Máy chủ hoàn toàn không biết `id` của phần tử là gì.
Liên Kết với Label	Không có tác dụng liên kết trực tiếp với thuộc tính `for` của `<label>`.	**Bắt buộc**: Khớp chính xác với `<label for="elementId">` để tối ưu tương tác & A11y.
Bộ Chọn CSS	Sử dụng cú pháp Attribute Selector: `input[name="user"]`.	Sử dụng cú pháp ID Selector: `#elementId` (độ ưu tiên Specificity cao hơn).
Truy Xuất JavaScript	`form.elements["user"]` `document.getElementsByName()`	`document.getElementById()` `document.querySelector("#id")`

[Mẹo] Góc Phỏng Vấn Frontend: Cạm Bẫy Thực Tế Khi Xử Lý name & value

- **Cạm bẫy "Input có dữ liệu nhưng Server nhận null/undefined"**: Đây là lỗi sơ đẳng phổ biến nhất của lập trình viên mới. Nguyên nhân 99% là do người viết khai báo nhầm `id="username"` mà quên thuộc tính `name="username"`. Trình duyệt chỉ gửi những điều khiển có thuộc tính `name`!
- **Checkbox không gửi giá trị "false"**: Khi một ô Checkbox không được người dùng đánh dấu tick, trình duyệt sẽ **hoàn toàn không gửi tên trường đó lên Server**. Do đó ở Backend (PHP/NodeJS/Java), ta cần kiểm tra sự tồn tại (`isset()` hoặc kiểm tra `undefined`) thay vì kiểm tra giá trị bằng `false`.
- **Đặt tên dạng mảng (`name="skills[]"`)**: Trong các ngôn ngữ Backend như PHP, việc thêm cặp dấu ngoặc vuông `[]` vào cuối thuộc tính name giúp máy chủ tự động gom các giá trị đã chọn thành một mảng dữ liệu (Array) thay vì chỉ lưu giá trị cuối cùng.

5.1.6 6. Phân Tích Chuyên Sâu Thuộc Tính placeholder, required & autocomplete

1. Phân Tích Chuyên Sâu Thuộc Tính placeholder

[Khái Niệm] Định Nghĩa & Vai Trò Của Thuộc Tính placeholder

Thuộc tính `placeholder` tạo ra văn bản gợi ý (**Hint**) xuất hiện mờ bên trong các ô nhập liệu (`<input>`, `<textarea>`). Văn bản này sẽ tự động biến mất khi người dùng gõ ký tự đầu tiên, giúp người dùng hiểu rõ loại dữ liệu cần nhập.

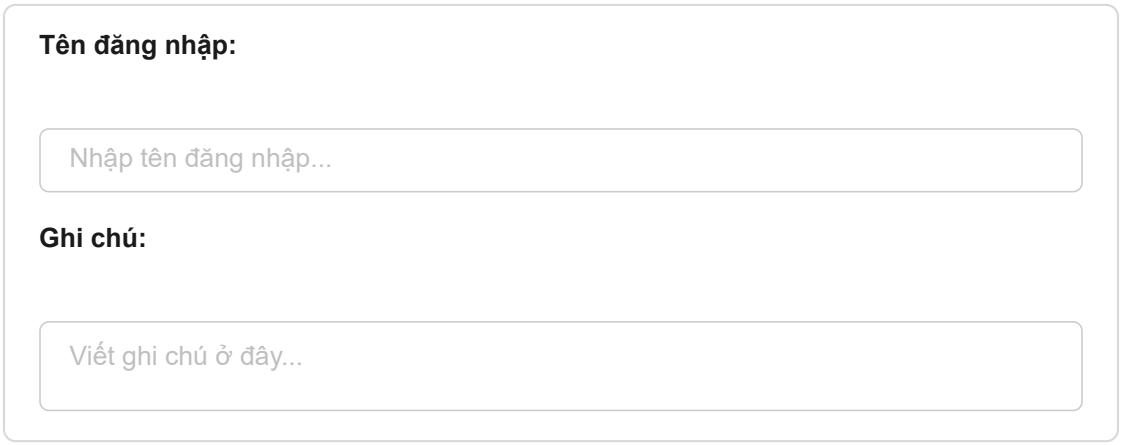
Mã Nguồn Cú Pháp Khai Báo Thuộc Tính placeholder

```
<!-- Cú pháp placeholder trên input văn bản -->
<input type="text" name="username" placeholder="Nhập tên đăng nhập">

<!-- Cú pháp placeholder trên textarea -->
<textarea placeholder="Viết ghi chú ở đây..."></textarea>
```

BROWSER PREVIEW — **Giao Diện Trình Duyệt Hiển Thị placeholder**

Khi ô nhập **chưa có dữ liệu**, trình duyệt hiển thị placeholder dạng chữ mờ gợi ý:



[→] Khi người dùng gõ ký tự đầu tiên → Placeholder biến mất. Xóa hết nội dung → Placeholder xuất hiện lại.

Bảng Đặc Điểm Kỹ Thuật & Lưu Ý Khi Sử Dụng placeholder:

[Bảng] Bảng Đặc Điểm Kỹ Thuật Của Thuộc Tính placeholder

Đặc Điểm	Mô Tả Chi Tiết & Hành Vi
Chỉ hiển thị tạm	Không được coi là giá trị (`value`). Khi Submit Form, thuộc tính `placeholder` **KHÔNG gửi đi**.
Không thay thế label	Chỉ mang tính gợi ý phụ. Vẫn bắt buộc dùng thẻ `<label>` để mô tả chính xác (tối ưu Trợ năng A11y).
Mất khi nhập	Khi người dùng nhập ký tự đầu tiên → `placeholder` biến mất. Xóa hết nội dung → `placeholder` hiện lại.
Hỗ trợ input/textarea	Sử dụng được cho hầu hết các loại `type=text, email, number, password, search...` và thẻ `<textarea>`.
Ngôn ngữ tự nhiên	Nên viết ngắn gọn, dễ hiểu (ví dụ: `"Email của bạn"`, `"MM/DD/YYYY"`).

Ví Dụ Thực Tế Kết Hợp label, input, textarea Có placeholder:

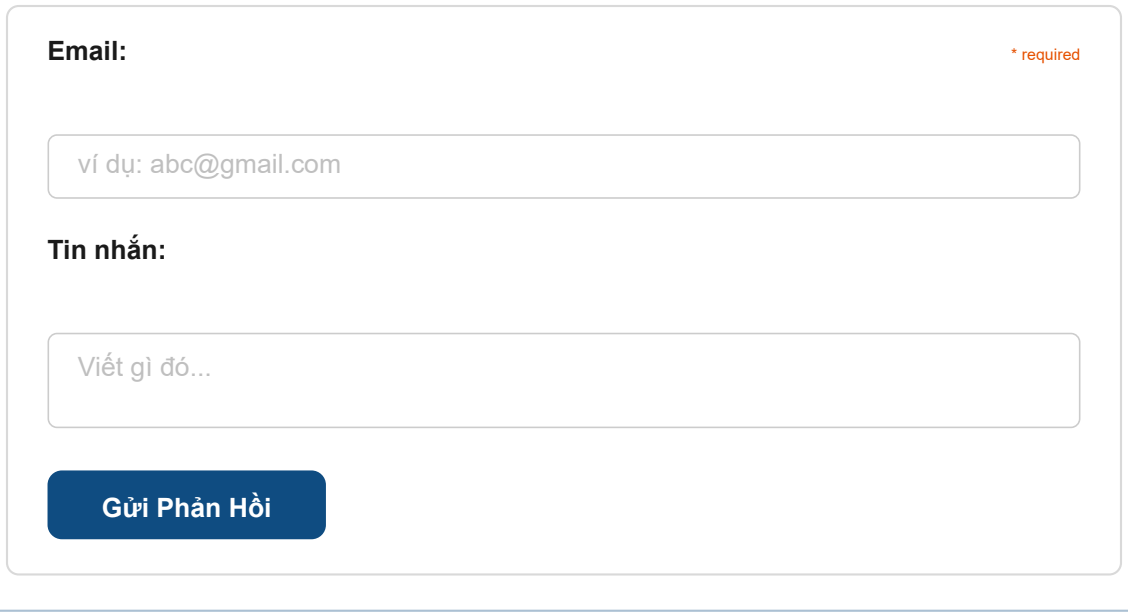
Mã Nguồn Form Phản Hồi Có Gắn Nhãn Và placeholder

```
<form action="/contact" method="POST">
  <label for="email">Email:</label>
  <input type="email" id="email" name="email" placeholder="ví dụ:
    abc@gmail.com" required>

  <label for="msg">Tin nhắn:</label>
  <textarea id="msg" name="message" placeholder="Viết gì đó..."></textarea>

  <input type="submit" value="Gửi">
</form>
```

BROWSER PREVIEW Giao Diện Form Phản Hồi Với label Kết Hợp placeholder



Ma Trận So Sánh Trực Diện Giữa Thuộc Tính placeholder Và value:

[Bảng] Ma Trận So Sánh Đối Chiếu Giữa Thuộc Tính placeholder Và value

Thuộc Tính	Vai Trò Trực Quan	Hành Vi Khi Submit Form
placeholder	Văn bản gợi ý mờ, tự động biến mất khi người dùng gõ chữ.	✗ Không được gửi đi.
value	Giá trị dữ liệu thực sự đang nằm trong ô nhập.	✓ Được đóng gói gửi đi.

2. Phân Tích Chuyên Sâu Thuộc Tính required

[Khái Niệm] Định Nghĩa & Vai Trò Của Thuộc Tính required

Thuộc tính `required` là thuộc tính Boolean (chỉ cần viết `required` hoặc `required="true"`). Khi được khai báo, trình duyệt sẽ **bắt buộc người dùng phải nhập dữ liệu hợp lệ** vào trường đó trước khi cho phép Submit Form.

Mã Nguồn Khai Báo Thuộc Tính required

```
<input type="text" name="username" required>
<input type="email" name="email" required>
```

Bảng Tác Động Của required Theo Từng Loại Input:

[Bảng] Bảng Quy Tắc Kiểm Tra required Theo Từng Loại Ô Nhập HTML5

Loại Input HTML5	Ảnh Hưởng & Quy Tắc Kiểm Tra Của required
`text, password, search, tel`	Không được để trống. Nếu để trống → Trình duyệt ngăn chặn Submit và hiển thị thông báo cảnh báo.
`email, url, number`	Ngoài việc không được để trống, dữ liệu nhập vào còn **bắt buộc phải đúng định dạng** (ví dụ: email phải chứa ký tự `@`).
`checkbox`	Bắt buộc phải được tick chọn. Nếu có nhiều checkbox độc lập có `required`, ít nhất ô đó phải được tick.
`radio (cùng name)`	**Ít nhất 1 nút Radio trong cùng nhóm name** phải được người dùng tick chọn.
`select`	Phải chọn một tùy chọn khác với Option trống (`<option value="">`).
`file`	Bắt buộc người dùng phải chọn tệp tin từ thiết bị trước khi gửi.

Ví Dụ Thực Tế Khai Báo required Cho Toàn Bộ Form:

Mã Nguồn Form Đăng Ký Yêu Cầu Nhập Đầy Đủ Các Trường

```html
<form action="/register" method="POST">
  <label>Tên:</label>
  <input type="text" name="username" required>

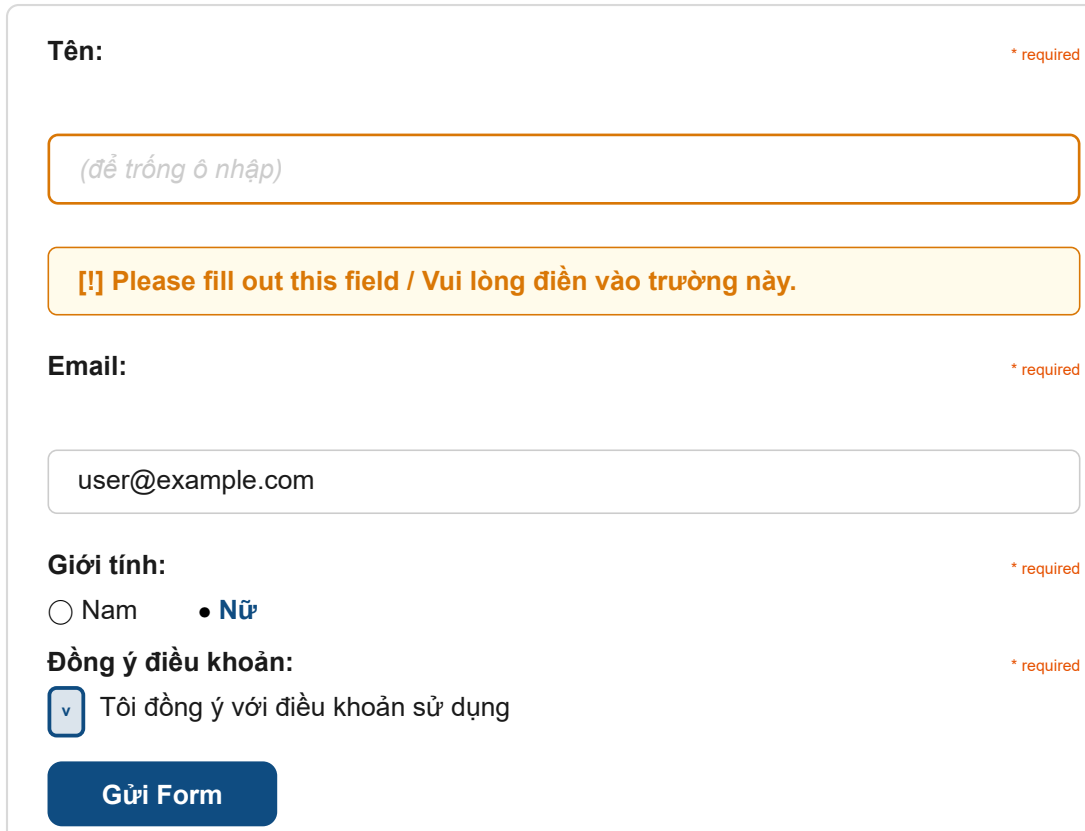
  <label>Email:</label>
  <input type="email" name="email" required>

  <label>Giới tính:</label>
  <input type="radio" name="gender" value="male" required> Nam
  <input type="radio" name="gender" value="female"> Nữ

  <label>Đồng ý điều khoản:</label>
  <input type="checkbox" name="agree" required>

  <input type="submit" value="Gửi form">
</form>
```

BROWSER PREVIEW — Giao Diện Form Đăng Ký Với Trường required



[→] Trình duyệt sẽ tự động chặn Submit và hiển thị thông báo cảnh báo bên dưới trường đầu tiên chưa điền đầy đủ.

[Cảnh Báo] 4 Lưu Ý Vàng Bắt Bắt Buộc Nhớ Về Thuộc Tính required

1. **Chỉ hoạt động khi Submit**: Thuộc tính `required` chỉ được kích hoạt kiểm tra khi biểu mẫu được Submit.
2. **Giao diện thông báo mặc định**: Trình duyệt tự hiển thị thông báo lỗi (giao diện thông báo có thể khác nhau tùy thuộc vào Chrome, Firefox, Edge hay Safari).
3. **Tích hợp kiểm soát chặt chẽ**: Có thể kết hợp `required` với các thuộc tính `pattern`, `min`, `max`, `maxlength` để ràng buộc dữ liệu toàn diện.
4. **Cần Validate ở Backend**: Thuộc tính này chỉ kiểm tra ở phía Client → **vẫn bắt buộc phải kiểm tra lại dữ liệu ở Server (Backend)** để phòng chống tấn công vô hiệu hóa HTML.

3. Phân Tích Chuyên Sâu Thuộc Tính autocomplete

[Khái Niệm] Định Nghĩa & Vai Trò Của Thuộc Tính autocomplete

Thuộc tính `autocomplete` dùng để **bật hoặc tắt tính năng tự động điền gợi ý** của trình duyệt khi người dùng nhập dữ liệu. Trình duyệt sẽ lưu các thông tin thường nhập (Tên, Email, Số điện thoại, Mật khẩu, Địa chỉ...) và tự động đề xuất điền nhanh khi gặp các Form tương tự.

Mã Nguồn Cú Pháp Bật/Tắt Thuộc Tính autocomplete

```
<!-- Tắt gợi ý tự động điền cho toàn bộ Form -->
<form action="/login" method="POST" autocomplete="off">

<!-- Bật gợi ý tự động điền riêng cho ô nhập Email -->
<input type="email" name="email" autocomplete="on">
```

Bảng Tra Cứu Các Giá Trị Tiêu Chuẩn Của Thuộc Tính autocomplete:

[Bảng] Bảng Tra Cứu Các Giá Trị Chuẩn W3C Của Thuộc Tính autocomplete

Giá Trị	Ý Nghĩa Gợi Ý Của Trình Duyệt	Ví Dụ Mã Nguồn Khai Báo
`on`	Bật tự động điền gợi ý dữ liệu (Mặc định).	`<input type="text" autocomplete="on">`
`off`	Tắt tự động điền gợi ý dữ liệu.	`<input type="password" autocomplete="off">`
`name`	Gợi ý Họ và Tên đầy đủ.	`<input type="text" autocomplete="name">`
`username`	Gợi ý Tên đăng nhập tài khoản.	`<input type="text" autocomplete="username">`
`email`	Gợi ý địa chỉ Email cá nhân.	`<input type="email" autocomplete="email">`
`new-password`	Gợi ý mật khẩu mới (dùng khi Đăng ký tài khoản).	`<input type="password" autocomplete="new-password">`
`current-password`	Gợi ý mật khẩu hiện tại (dùng khi Đăng nhập).	`<input type="password" autocomplete="current-password">`
`tel`	Gợi ý số điện thoại liên lạc.	`<input type="tel" autocomplete="tel">`
`address-line1`	Gợi ý địa chỉ nhà / tên đường.	`<input type="text" autocomplete="address-line1">`
`postal-code`	Gợi ý mã bưu điện (Zip / Postal Code).	`<input type="text" autocomplete="postal-code">`
`cc-number`	Gợi ý số thẻ ngân hàng / thẻ tín dụng.	`<input type="text" autocomplete="cc-number">`

Ví Dụ Thực Tế Khai Báo autocomplete Chuẩn Đăng Nhập:

Mã Nguồn Form Đăng Nhập Chuẩn Autocomplete

```
<form action="/login" method="POST" autocomplete="on">
  <label for="user">Tên đăng nhập:</label>
  <input type="text" id="user" name="username" autocomplete="username" required>

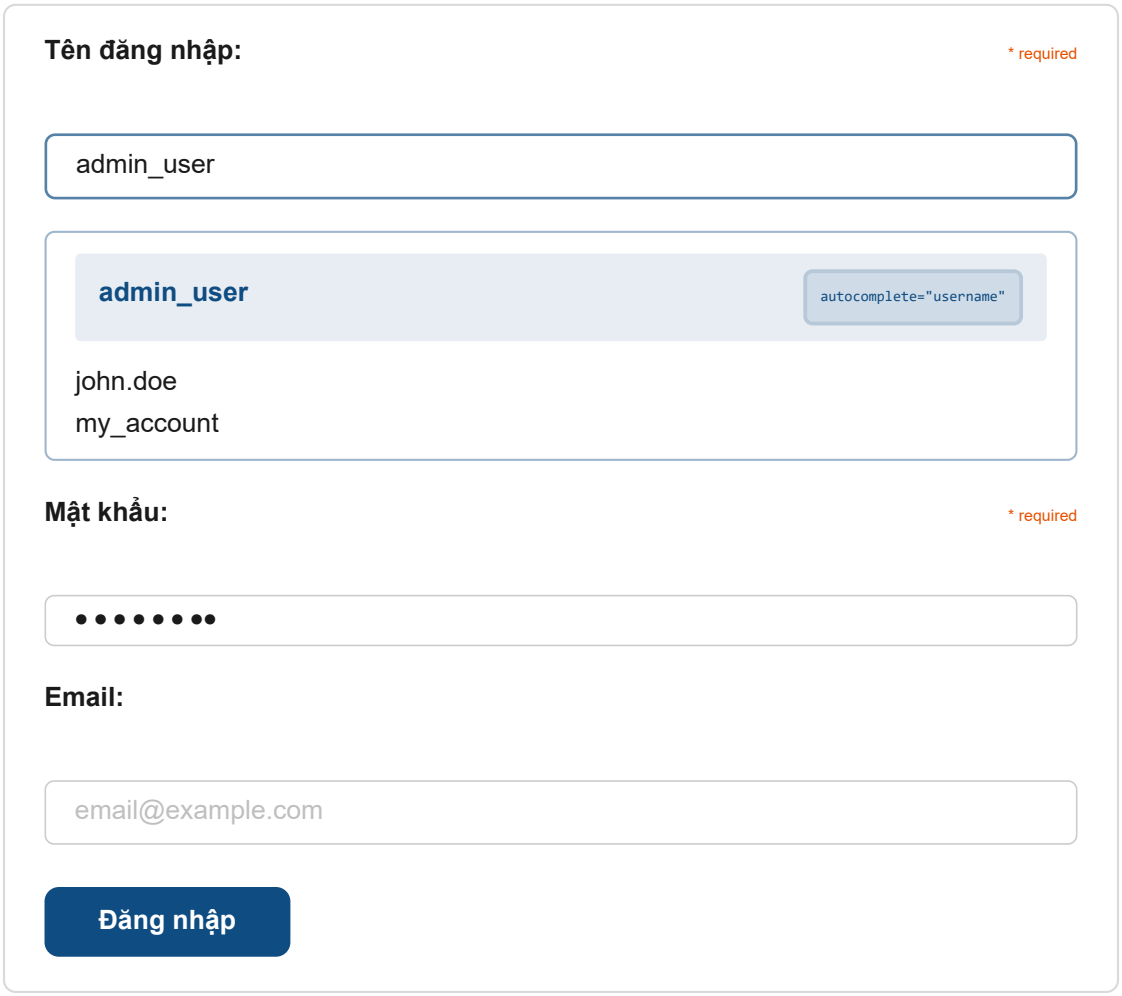
  <label for="pass">Mật khẩu:</label>
  <input type="password" id="pass" name="password" autocomplete="current-password
    " required>

  <label for="email">Email:</label>
  <input type="email" id="email" name="email" autocomplete="email">

  <input type="submit" value="Đăng nhập">
</form>
```

BROWSER PREVIEW **Giao Diện Trình Duyệt Gợi Ý Tự Động Điền (Autocomplete)**

Khi người dùng click vào ô nhập, trình duyệt hiển thị danh sách gợi ý từ dữ liệu đã lưu:



[Mẹo] 4 Lưu Ý Quan Trọng Khi Sử Dụng Thuộc Tính autocomplete

1. **Mặc định là "on"**: Trình duyệt mặc định gán `autocomplete="on"` nếu không có khai báo khác.
2. **Bảo mật ô nhạy cảm**: Dùng `autocomplete="off"` cho các ô nhập nhạy cảm (như Mã OTP, Mật khẩu tạm thời, Mã PIN ngân hàng).
3. **Hành vi của trình duyệt**: Một số trình duyệt hiện đại (Chrome, Safari) có thể vẫn cố tình hiển thị gợi ý mật khẩu ngay cả khi đặt `off` vì lý do tiện ích bảo mật Password Manager.
4. **Khai báo linh hoạt**: Có thể khai báo `autocomplete` cho toàn bộ thẻ `<form>` hoặc khai báo đè lên từng ô `<input>` riêng lẻ.

4. Phân Tích Chuyên Sâu Thuộc Tính novalidate & formnovalidate

[Khái Niệm] Định Nghĩa & Vai Trò Của Thuộc Tính novalidate

Thuộc tính `novalidate` là thuộc tính Boolean áp dụng cho thẻ `<form>`. Khi được khai báo, trình duyệt sẽ **bỏ qua hoàn toàn** mọi kiểm tra hợp lệ (Validation) mặc định của HTML5 khi Submit Form. Nghĩa là các thuộc tính `required`, `type="email"`, `pattern`, `min`, `max`... sẽ **không bị kiểm tra ở phía Client**.

Mã Nguồn Khai Báo Form Có Thuộc Tính novalidate

```
<form action="/submit" method="post" novalidate>
  <input type="email" name="email" required>
  <input type="submit" value="Gửi">
</form>
```

BROWSER PREVIEW — **Kết Quả Khi Form Có Thuộc Tính novalidate**

Dù `required` và `type="email"` có được khai báo, trình duyệt **vẫn cho phép Submit ngay** cả khi để trống hoặc nhập sai định dạng:

Email: (required + type="email")

abc123 [!] Sai định dạng Email

[v] Form vẫn được Submit thành công! Trình duyệt KHÔNG chặn (do có novalidate).

[→] So sánh: Nếu **không có** `novalidate`, trình duyệt sẽ chặn Submit và hiển thị bong bóng lỗi *"Please include an '@' in the email address"*.

Bảng Đặc Điểm Kỹ Thuật & Lưu Ý Khi Sử Dụng novalidate:

[Bảng] Bảng Đặc Điểm Kỹ Thuật Của Thuộc Tính novalidate

Đặc Điểm	Mô Tả Chi Tiết
Tắt Validation mặc định	Ngăn không cho trình duyệt chặn Form khi dữ liệu sai hoặc thiếu.
Không xóa thuộc tính input	Các thuộc tính `required`, `pattern`, `type`... vẫn tồn tại trong mã nguồn, chỉ là không được kiểm tra ở phía Client.
Dùng khi Validation thủ công	Thường sử dụng khi muốn tự kiểm tra bằng JavaScript hoặc xử lý Validation hoàn toàn ở phía Server (Backend).
Có thể dùng cho từng nút Submit	Ngoài thẻ `<form>`, có thể áp dụng thuộc tính `formnovalidate` riêng cho từng `<button type="submit">`.

Ví Dụ Thực Tế: Hai Nút Submit Với Và Không Có Validation:

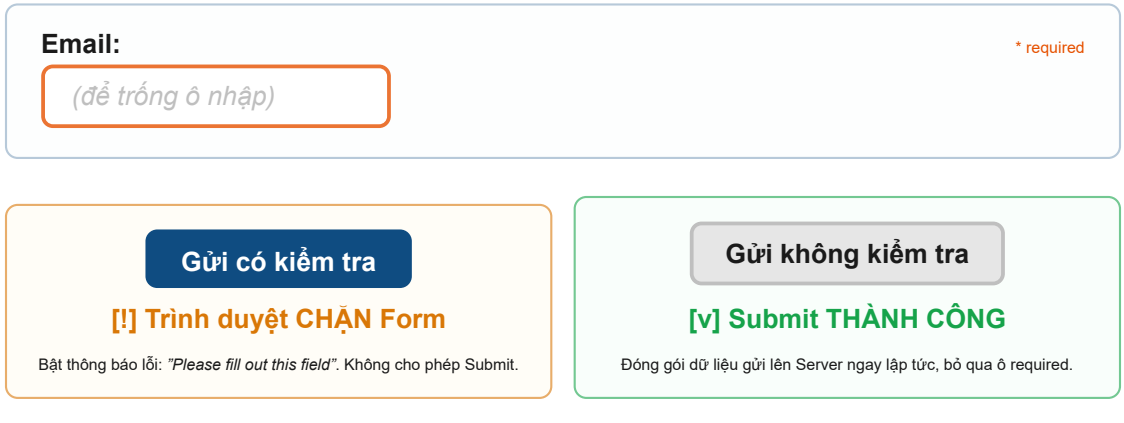
Mã Nguồn Form Có 2 Nút Submit: Có Kiểm Tra vs Bỏ Qua Kiểm Tra

```html
<form action="/submit" method="post">
  <input type="email" name="email" required>

  <!-- Nút này sẽ kiểm tra Validation bình thường -->
  <button type="submit">Gửi có kiểm tra</button>

  <!-- Nút này bỏ qua Validation nhờ formnovalidate -->
  <button type="submit" formnovalidate>Gửi không kiểm tra</button>
</form>
```

BROWSER PREVIEW So Sánh Trực Quan Hành Vi 2 Nút Submit Với Thuộc Tính formnovalidate



[Ghi Chú] Khi Nào Nên Và Không Nên Sử Dụng novalidate?

- **Nên dùng:** Khi bạn muốn tự viết logic kiểm tra bằng JavaScript hoặc khi Validation phía Client không cần thiết (ví dụ: Form nhỏ, dữ liệu chỉ xử lý ở Server).
- **Không nên dùng:** Nếu muốn tận dụng HTML5 Validation tự động (tiết kiệm code, có UI cảnh báo sẵn, cải thiện trải nghiệm người dùng).

5.1.7 7. Các Thuộc Tính Kiểm Soát Nhập Liệu Advanced Trong HTML5

[Khái Niệm] Vai Trò Của Bộ Thuộc Tính Kiểm Soát (Form Control Attributes)

HTML5 cung cấp bộ thuộc tính tích hợp sẵn cho phép kiểm soát quy tắc nhập liệu, hỗ trợ hiển thị giao diện UI và tự động Ràng buộc/Validate dữ liệu ngay trên phía Client (Trình duyệt) mà **chưa cần viết bất kỳ dòng mã JavaScript nào**.

Bảng Tra Cứu 8 Thuộc Tính Tiên Tiến Hỗ Trợ Nhập Liệu Trong HTML5:

[Bảng] Bảng Tra Cứu Bộ Thuộc Tính Nâng Cao Trong Thẻ Input HTML5

Thuộc Tính	Ý Nghĩa & Cơ Chế Hoạt Động	Cú Pháp Mã Nguồn Ví Dụ
`placeholder`	Chữ gợi ý mờ hiển thị tạm thời khi ô trống. Tự động ẩn đi khi người dùng bắt đầu gõ văn bản.	`<input type="text" placeholder="(*@Nhập họ tên...@*)">`
`required`	Bắt buộc người dùng phải nhập/chọn dữ liệu trước khi Form có thể Submit thành công.	`<input type="email" required>`
`readonly`	Chỉ cho phép đọc nội dung ô nhập. Người dùng không sửa được nhưng **vẫn gửi dữ liệu** khi Submit.	`<input type="text" value="VN-2026" readonly>`
`disabled`	Vô hiệu hóa ô nhập (mờ UI). Người dùng không thể tương tác và **KHÔNG gửi dữ liệu** khi Submit.	`<input type="text" value="Locked" disabled>`
`autofocus`	Tự động di chuyển con trỏ nhập liệu vào ô này ngay khi trang Web vừa tải xong.	`<input type="text" name="query" autofocus>`
`pattern`	Kiểm tra định dạng dữ liệu nhập vào bằng biểu thức chính quy (**Regex**) trực tiếp trên HTML5.	`<input type="tel" pattern="[0-9]10">`
`min, max, step`	Khống chế giá trị tối thiểu, tối đa và bước nhảy số cho các loại input `number` và `range`.	`<input type="number" min="1" max="100" step="5">`
`multiple`	Cho phép chọn nhiều tệp tin cùng lúc (với `file`) hoặc nhập nhiều email phân cách dấu phẩy.	`<input type="file" name="docs" multiple>`

Ma Trận So Sánh Trực Diện Giữa Thuộc Tính readonly Và disabled:

[Bảng] Ma Trận So Sánh Đối Chiếu Giữa Thuộc Tính readonly Và disabled

Tiêu Chí So Sánh	Thuộc Tính readonly	Thuộc Tính disabled
Hiển thị giao diện UI	Giữ nguyên màu chữ và khung bình thường (hoặc xám nhẹ tùy CSS).	Làm mờ tối ô nhập (thường là màu xám nhạt, con trỏ chuột dạng `not-allowed`).
Khả năng tương tác	Người dùng **không thể chỉnh sửa**, nhưng vẫn có thể bôi đen copy chữ và Focus con trỏ.	Người dùng **hoàn toàn không thể tương tác**, không bôi đen copy, không Focus con trỏ.
Hành vi khi Form Submit	✅ **Có gửi dữ liệu lên Server**.	❌ **Bị bỏ qua, KHÔNG gửi dữ liệu**.

5.1.8 8. Gom Nhóm Biểu Mẫu Chuẩn Ngữ Nghĩa Với `<fieldset>` & `<legend>`

[Khái Niệm] Tổng Quan Về Thẻ <fieldset> Và Thẻ <legend>

Thẻ `<fieldset>` dùng để **gom nhóm các phần tử liên quan** trong một biểu mẫu Form lớn thành một khối logic trực quan có đường viền bo quanh. Thẻ `<legend>` đóng vai trò là **tiêu đề (caption)** cho nhóm phần tử đó, hiển thị đè trực tiếp lên đường viền phía trên. Việc kết hợp bộ đôi này giúp tổ chức form rõ ràng, tăng tính truy cập (**Accessibility / a11y**) cho người dùng và thiết bị Trình đọc màn hình (**Screen Reader**).

8.1. Chi Tiết Thẻ `<fieldset>` Trong HTML

a. <fieldset> Là Gì?

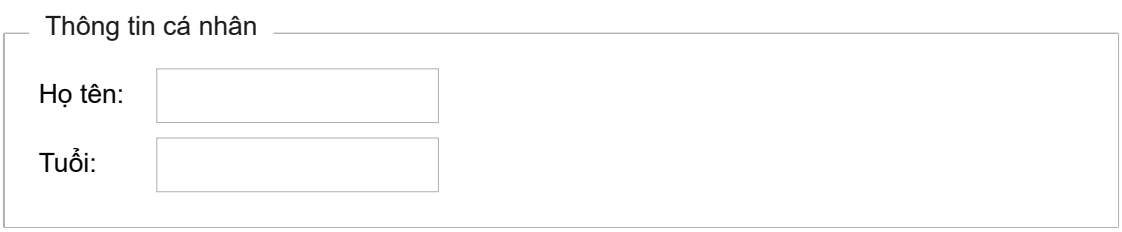
- Là thẻ HTML dùng để **nhóm các phần tử trong form lại thành một khối logic**.
- Thường **đi kèm với** `<legend>` để hiển thị tiêu đề cho nhóm.
- Giúp **tổ chức form rõ ràng, tăng tính Accessibility** (dễ hiểu với người dùng và screen reader).

b. Cú Pháp Cơ Bản Về <fieldset>:

Cú Pháp Cơ Bản Thẻ <fieldset>

```
<form>
  <fieldset>
    <legend>Thông tin cá nhân</legend>
    <label for="name">Họ tên:</label>
    <input type="text" id="name" name="name"><br><br>
    <label for="age">Tuổi:</label>
    <input type="number" id="age" name="age">
  </fieldset>
</form>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Khung Fieldset Mặc Định**



[Ghi Chú] Giải Thích Cú Pháp

Ở đây: `<fieldset>` tạo khung viền bao quanh, `<legend>` hiển thị chữ **"Thông tin cá nhân"** chèn ngay lên đường viền phía trên.

c. Các Thuộc Tính Của <fieldset>:

[Bảng] Bảng Thuộc Tính Của Thẻ <fieldset>

Thuộc tính	Ý nghĩa & Hành vi kỹ thuật
`disabled`	Vô hiệu hóa toàn bộ nội dung trong fieldset (tất cả input bên trong đều bị khóa).
`form`	Liên kết fieldset với một form cụ thể (nếu fieldset nằm ngoài form).
`name`	Đặt tên cho fieldset (ít dùng, chủ yếu cho script).

d. Các Ví Dụ Nâng Cao Thẻ <fieldset>:

Fieldset Với Thuộc Tính disabled

```
<fieldset disabled>
  <legend>Thông tin bị khóa</legend>
  <label>Email:</label>
  <input type="email" value="user@gmail.com">
</fieldset>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Fieldset Bị Vô Hiệu Hóa (disabled)**

Thông tin bị khóa

Email: user@gmail.com *(Bị khóa / không thể nhập)*

→ **Người dùng không thể nhập vào các input.** (Tất cả ô nhập bên trong tự động thừa hưởng trạng thái bị khóa/vô hiệu hóa).

Fieldset Nằm Ngoài Form (Sử Dụng Thuộc Tính form)

```
<form id="myForm">
  <input type="submit" value="Gửi">
</form>

<fieldset form="myForm">
  <legend>Thông tin thêm</legend>
  <input type="text" placeholder="Nhập dữ liệu">
</fieldset>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Fieldset Liên Kết Form**

Gửi

Thông tin thêm

Nhập dữ liệu: *Nhập dữ liệu...*

→ **Mặc dù `<fieldset>` nằm ngoài `<form>`, nhưng nhờ `form="myForm"` mà nó vẫn gửi dữ liệu.**

e. Tùy Chỉnh Giao Diện Fieldset Bằng CSS:

Định Dạng CSS Cho Fieldset

```css
fieldset {
  border: 2px solid #3498db;
  padding: 15px;
  margin-bottom: 20px;
  border-radius: 6px;
}
legend {
  font-weight: bold;
  color: #2c3e50;
}
```

BROWSER PREVIEW Kết Quả Hiển Thị Trình Duyệt: Fieldset Được Tùy Chỉnh CSS

Thông tin cá nhân

Khung viền màu xanh `#3498db` dày 2px, bo góc 6px, tiêu đề in đậm màu `#2c3e50`.

f. Các Lưu Ý Quan Trọng Khi Sử Dụng <fieldset>:

1. `<fieldset>` **không bắt buộc**, nhưng nên dùng khi form có nhiều nhóm dữ liệu.
2. **Luôn kết hợp với** `<legend>` để mô tả ý nghĩa nhóm.
3. Nếu chỉ cần trang trí đơn thuần mà không mang ý nghĩa gom nhóm ngữ nghĩa, không nhất thiết dùng `<fieldset>`, có thể dùng `<div>` + CSS.

[Mẹo] Tóm Lại Về <fieldset>

- `<fieldset>` = **khung nhóm các input trong form**.
- `<legend>` = **tiêu đề mô tả nhóm**.
- Giúp form **dễ đọc, dễ dùng, chuẩn accessibility**.

8.2. Chi Tiết Thẻ `<legend>` Trong HTML

a. <legend> Là Gì?

- Là thẻ HTML dùng để **tạo tiêu đề (caption)** cho nhóm form được bao bởi thẻ `<fieldset>`.
- Giúp người dùng **hiểu rõ nội dung** của nhóm input trong form.
- Thường dùng để **cải thiện Accessibility** (dễ đọc với screen reader).

b. Cú Pháp Sử Dụng Thẻ <legend>:

Cú Pháp Thẻ <legend>

```
<fieldset>
  <legend>Thông tin cá nhân</legend>
  <label for="name">Họ tên:</label>
  <input type="text" id="name" name="name"><br>
  <label for="age">Tuổi:</label>
  <input type="number" id="age" name="age">
</fieldset>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Thẻ <legend> Làm Tiêu Đề Mặc Định**



→ Ở đây, **"Thông tin cá nhân"** là tiêu đề cho toàn bộ nhóm input.

c. Đặc Điểm Quan Trọng Của Thẻ <legend>:

1. **Chỉ hợp lệ bên trong** `<fieldset>` (phải là con đầu tiên của `<fieldset>`, không được đứng một mình).
2. Có thể chứa text hoặc inline elements (ví dụ `<strong>`, `<em>`), nhưng **không nên quá phức tạp**.
3. **Vị trí hiển thị mặc định**: trên cùng và bên trái của fieldset (chèn ngang lên đường viền khung).

d. Thuộc Tính Hỗ Trợ Của Thẻ <legend>:

- `align` (**deprecated** – không khuyến khích): canh trái, phải, giữa, hoặc top.

Thuộc Tính align Lỗi Thời Trên Thẻ legend

```
<legend align="center">Thông tin</legend>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Thẻ <legend align="center">**

Thông tin

(Tiêu đề legend được đẩy ra vị trí chính giữa khung viền)

→ **Thuộc tính này đã lỗi thời (deprecated), nên dùng CSS thay thế.**

e. Tùy Chỉnh Bằng CSS Cho Thẻ <legend> & <fieldset>:

Tùy Biến CSS Cho Legend Và Fieldset

```
legend {
  font-weight: bold;
  font-size: 18px;
  color: #2c3e50;
}
fieldset {
  border: 2px solid #3498db;
  padding: 10px;
  border-radius: 6px;
}
```

f. Ứng Dụng Thực Tế Của Thẻ <legend>:

- **Nhóm các trường liên quan trong form**:
  - Thông tin cá nhân
  - Thông tin liên hệ
  - Cài đặt tài khoản
- **Tăng tính dễ đọc và chuyên nghiệp** cho biểu mẫu web.

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Biểu Mẫu Nhóm Thực Tế**

Thông Tin Cá Nhân

Họ tên: Nguyễn Văn A Tuổi: 25

Cài Đặt Tài Khoản

Email: user@gmail.com *(Bị khóa)*

[Mẹo] Tóm Lại Về <legend>

- `<fieldset>` = **khung nhóm input**.
- `<legend>` = **tiêu đề mô tả nhóm đó**.

8.3. Đọc Thêm: Chuyên Sâu Về Accessibility (a11y), DOM API JavaScript & Best Practices

a. Nguyên Lý Hoạt Động Của Screen Reader Với Fieldset & Legend (WCAG 2.1 Standard):

Khi thiết kế các nhóm ô chọn Checkbox hoặc Radio Button, người dùng thị giác có thể nhận biết câu hỏi chung nhờ dòng chữ nằm phía trên. Tuy nhiên, người dùng khiếm thị dùng Trình đọc màn hình (**Screen Reader**) chuyển ô bằng phím Tab sẽ bị mất ngữ cảnh nếu câu hỏi không được kết nối chuẩn.

- **Không có `<fieldset>`/`<legend>`**: Khi di chuyển vào Radio Button "Nam", Screen Reader chỉ đọc: *"Radio button, unchecked, Nam"*. Người dùng không thể biết đây là câu hỏi về Giới tính hay Thành viên!
- **Có `<fieldset>`/`<legend>`**: Screen Reader sẽ đọc vang cả nhóm: *"Giới tính, group. Radio button, unchecked, Nam (1 of 3)"*.

b. Cơ Chế DOM JavaScript Của HTMLFieldSetElement:

Đối tượng `HTMLFieldSetElement` trong DOM sở hữu các thuộc tính vô cùng hữu ích cho việc quản lý trạng thái form bằng JavaScript:

- `fieldset.disabled = true/false`: Bật/tắt toàn bộ các trường nhập con trong một dòng lệnh duy nhất mà không cần lặp qua từng phần tử `input`.
- `fieldset.elements`: Trả về danh sách `HTMLFormControlsCollection` chứa tất cả các ô nhập liệu bên trong fieldset đó.
- `fieldset.form`: Trả về phần tử `<form>` chứa fieldset này (hoặc form được liên kết qua thuộc tính `form=""`).

c. Cảnh Báo CSS Layout Bug Với Fieldset Trong Trình Duyệt Old WebKit/Blink:

Trong một số phiên bản trình duyệt cũ, thẻ `<fieldset>` có cơ chế tính toán rendering đặc biệt (**anonymous block box**), dẫn đến việc áp dụng CSS `display: flex` hoặc `display: grid` trực tiếp lên `<fieldset>` có thể bị hỏng layout.

[Mẹo] Mẹo Khắc Phục CSS Layout Trong Fieldset

Best Practice hiện đại là đặt một thẻ wrapper `<div class="fieldset-content">` nằm ngay bên trong `<fieldset>` (ngay sau `<legend>`), và áp dụng CSS Flexbox / Grid lên thẻ `div` đó thay vì áp dụng trực tiếp lên `<fieldset>`.

5.1.9 9. So Sánh Chuyên Sâu Bộ Chọn `<select>` vs Bộ Gợi Ý Autocomplete `<datalist>`

[Khái Niệm] Phân Biệt Cơ Chế Thuộc Thẻ <select> Và <datalist>

- **Thẻ `<select>`**: Danh sách thả xuống (**Dropdown Menu**) có giá trị **cố định 100%**. Người dùng bắt buộc phải chọn 1 trong các tùy chọn sẵn có, **không thể tự gõ thêm chữ**.
- **Thẻ `<datalist>`**: Danh sách gợi ý tự động (**Autocomplete Suggestions**). Người dùng **vừa có thể chọn từ danh sách gợi ý, vừa có thể tự gõ văn bản tự do** theo ý muốn.

Mã Nguồn Triển Khai Thẻ datalist Kết Hợp Với Input

```
<!-- Khai báo Input kết nối tới datalist qua thuộc tính list -->
<label for="browser-choice">Chọn trình duyệt yêu thích:</label>
<input list="browsers" id="browser-choice" name="browser" placeholder="Gõ hoặc
    chọn trình duyệt...">

<!-- Danh sách tùy chọn gợi ý datalist -->
<datalist id="browsers">
  <option value="Google Chrome">
  <option value="Mozilla Firefox">
  <option value="Microsoft Edge">
  <option value="Apple Safari">
  <option value="Brave Browser">
</datalist>
```

Ma Trận So Sánh Trực Diện Giữa Thẻ <select> Và <datalist>:

[Bảng] Ma Trận So Sánh Kỹ Thuật Giữa Thẻ select Và datalist

Tiêu Chí So Sánh	Thẻ <select>	Thẻ <datalist>
Khả năng gõ văn bản tự do	**Không thể**. Người dùng chỉ được bấm chọn từ danh sách đóng kín.	**Hoàn toàn có thể**. Người dùng tự do nhập bất kỳ chuỗi chữ nào.
Tự động lọc tìm kiếm	Không tự lọc (trừ khi dùng thêm thư viện JavaScript bên thứ ba).	**Tự động lọc** các tùy chọn chứa từ khóa người dùng đang gõ theo thời gian thực.
Giá trị gửi lên Server	Gửi thuộc tính `value` của tùy chọn `<option>` được chọn.	Gửi chuỗi văn bản thực tế có trong ô `<input>`.
Trường hợp sử dụng khuyên dùng	Dùng cho dữ liệu chuẩn hóa cố định (ví dụ: Thành phố, Quốc gia, Giới tính).	Dùng cho từ khóa tìm kiếm, nhãn Tag, Tên trình duyệt, Hãng xe...

5.1.10 10. Nhập Văn Bản Đa Dòng Với Thẻ <textarea>

[Khái Niệm] Đặc Điểm Thẻ <textarea>

Thẻ `<textarea>` tạo ra một ô nhập văn bản nhiều dòng (**Multi-line Text Input**), thường được sử dụng cho các mục nhập Ý kiến phản hồi, Khung bình luận, Bài viết hoặc Địa chỉ chi tiết.

Bảng Tra Cứu Chi Tiết Thuộc Tính Thẻ <textarea>:

[Bảng] Bảng Tra Cứu Các Thuộc Tính Thẻ textarea Trong HTML5

Thuộc Tính	Ý Nghĩa & Tác Dụng Chi Tiết	Cú Pháp Ví Dụ
`rows`	Số dòng văn bản hiển thị mặc định theo chiều cao.	`<textarea rows="5"></textarea>`
`cols`	Số cột (ký tự) hiển thị mặc định theo chiều rộng.	`<textarea cols="40"></textarea>`
`maxlength`	Giới hạn tổng số ký tự tối đa người dùng được phép nhập.	`<textarea maxlength="500"></textarea>`
`wrap`	Điều hướng xuống dòng: `soft` (chỉ xuống dòng UI) vs `hard` (gửi kèm ký tự xuống dòng `\n`).	`<textarea wrap="hard"></textarea>`

[Cảnh Báo] Lưu Ý CSS Khống Chế Kích Thước Tránh Vỡ Bố Cục Với textarea

Mặc định trình duyệt cho phép người dùng kéo góc ô `<textarea>` để thay đổi kích thước. Trong các thiết kế Web chuyên nghiệp, ta nên khống chế thuộc tính CSS `resize` để tránh ô nhập bị kéo làm vỡ giao diện:

```
textarea  resize: vertical; min-height: 100px; max-height: 300px;
```
(Chỉ cho kéo theo chiều dọc)

5.1.11 11. Thẻ Hiển Thị Kết Quả Tính Toán <output>

Thẻ `<output>` đại diện cho kết quả của một phép tính toán hoặc hành động người dùng thực hiện trong form thông qua các giá trị nhập liệu.

[Khái Niệm] Thuộc Tính Của Thẻ Output

- `for="id1 id2"`: Khai báo danh sách ID các ô input tham gia vào phép tính toán.
- `name="..."`: Đặt tên cho phần tử output để truy xuất qua form hoặc gửi kèm dữ liệu.

Mã Nguồn: Tự Động Tính Toán Phép Nhân Bằng Output Và JS oninput

```
<form oninput="result.value = parseInt(a.value) * parseInt(b.value)">
  <input type="number" id="a" value="10"> x
  <input type="number" id="b" value="5"> =
  <output name="result" for="a b">50</output> VNĐ
</form>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trực Quan Trình Duyệt: Thẻ Output Tính Toán Trực Tiếp**

10 × 5 = **50 VNĐ**

5.1.12 12. Cơ Chế HTML5 Form Validation & Thuộc Tính Ràng Buộc

[Khái Niệm] Form Validation Trong HTML5 Là Gì?

Form Validation (Kiểm tra tính hợp lệ của biểu mẫu) là quá trình xác minh dữ liệu người dùng nhập vào ô input có đáp ứng đúng định dạng và quy tắc yêu cầu trước khi gửi lên máy chủ Backend hay không.

- **Native Validation**: Trình duyệt tự động kiểm tra và hiển thị thông báo lỗi giao diện mặc định mà không cần viết mã JavaScript phức tạp.
- **Lợi ích**: Tăng trải nghiệm người dùng (**UX**), giảm tải số lượng yêu cầu không hợp lệ gửi tới máy chủ và bảo vệ tính toàn vẹn dữ liệu.

[Bảng] Bảng Tổng Hợp Các Thuộc Tính Ràng Buộc Dữ Liệu Form Validation

Thuộc Tính	Ý Nghĩa Ràng Buộc	Cú Pháp HTML Mẫu
`required`	Bắt buộc phải nhập/chọn dữ liệu trước khi gửi form.	`<input type="text" required>`
`pattern`	Biểu thức chính quy (**Regex**) kiểm tra định dạng chuỗi.	`<input pattern="[0-9]10">`
`minlength / maxlength`	Số lượng ký tự tối thiểu và tối đa cho phép.	`<input minlength="6" maxlength="20">`
`min / max`	Giá trị số hoặc ngày tháng nhỏ nhất và lớn nhất.	`<input type="number" min="1" max="100">`
`step`	Bước nhảy hợp lệ của giá trị số.	`<input type="number" step="0.5">`
`type`	Định dạng dữ liệu tự động (`email`, `url`, `number`).	`<input type="email">`

Ví Dụ Áp Dụng Thuộc Tính Validation Thuần HTML5

```
<form action="/register" method="POST">
  <!-- Bắt buộc nhập và định dạng Email -->
  <input type="email" name="email" required placeholder="user@example.com">

  <!-- Mật khẩu tối thiểu 8 ký tự -->
  <input type="password" name="password" minlength="8" required placeholder="Mật
    khẩu (>= 8 ký tự)">

  <!-- Số điện thoại đúng định dạng 10 chữ số -->
  <input type="tel" name="phone" pattern="[0-9]{10}" placeholder="Số điện thoại
    (10 chữ số)">

  <button type="submit">Đăng Ký</button>
</form>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Form Validation HTML5**

Đăng Ký Tài Khoản (HTML5 Validation Demo):

nhap_email@example.com *(Required & Email type)*

Mật khẩu (≥8 ký tự) *(Minlength = 8)*

Số điện thoại (10 số) *(Regex Pattern = [0-9]{10})*

Đăng Ký

5.1.13 13. Mã Nguồn Minh Họa Biểu Mẫu Đăng Ký Tổng Hợp

Mã Nguồn Mẫu Form Đăng Ký Tài Khoản Tổng Hợp Đầy Đủ Tính Năng

```
<form action="/register" method="post" enctype="multipart/form-data">
  <fieldset>
    <legend>Dang Ky Tai Khoan Nguoi Dung</legend>

    <label for="username">Ho va Ten:</label>
    <input type="text" id="username" name="username" required placeholder="Nhap ho ten...">

    <label for="email">Dia chi Email:</label>
    <input type="email" id="email" name="email" required placeholder="name@example.com">

    <label for="password">Mat khau:</label>
    <input type="password" id="password" name="password" required>

    <label>Gioi tinh:</label>
    <input type="radio" id="male" name="gender" value="male"> <label for="male">Nam</label>
    <input type="radio" id="female" name="gender" value="female"> <label for="female">Nu</label>

    <label>So thich:</label>
    <input type="checkbox" id="sport" name="hobby" value="sport"> <label for="sport">The thao</label>
    <input type="checkbox" id="music" name="hobby" value="music"> <label for="music">Am nhac</label>

    <label for="country">Quoc gia:</label>
    <select id="country" name="country">
      <option value="vn">Việt Nam</option>
```

```
25        <option value="us">Mỹ</option>
26        <option value="jp">Nhật Bản</option>
27      </select>
28
29      <label for="bio">Gioi thieu ban than:</label>
30      <textarea id="bio" name="bio" rows="4" placeholder="Vai dong gioi thieu..."><
        /textarea>
31
32      <label for="avatar">Anh dai dien:</label>
33      <input type="file" id="avatar" name="avatar" accept="image/*">
34
35      <button type="submit">Dang Ky Ngay</button>
36      <button type="reset">Lam Lai</button>
37    </fieldset>
38  </form>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Biểu Mẫu Đăng Ký Tài Khoản Đầy Đủ Thành Phần**

Đăng Ký Tài Khoản Người Dùng

Họ và Tên: Nguyễn Văn A **Email:** name@example.com

Giới tính: (o) Nam ( ) Nữ **Sở thích:** [x] Thể thao [x] Âm nhạc [ ] Đọc sách

Quốc gia: Việt Nam ∨

Giới thiệu bản thân:

Xin chào, tôi là lập trình viên Frontend đang học HTML5 và CSS3...

Ảnh đại diện: Choose File avatar.jpg

Đăng Ký Ngay **Làm Lại**

[Ghi Chú] 4 Lưu Ý Nguyên Tắc Vàng Khi Xây Dựng Biểu Mẫu

1. **Bắt buộc kết hợp `<label for="...">` và `id="..."`**: Giúp tăng khả năng truy cập (Accessibility - A11y), cho phép người dùng nhấp chuột vào chữ nhãn để trỏ thẳng con trỏ vào ô nhập.
2. **Kiểm tra dữ liệu phía Client (Client-side Validation)**: Nên kết hợp các thuộc tính HTML5 native như `required`, `pattern`, `maxlength`, `min/max` để chặn dữ liệu sai trước khi gửi lên server.
3. **Chọn phương thức `method="post"` khi gửi thông tin quan trọng**: Tuyệt đối không dùng GET cho dữ liệu nhạy cảm như mật khẩu hay số thẻ ngân hàng.
4. **Khai báo `enctype="multipart/form-data"` khi upload file**: Nếu thiếu thuộc tính này trên thẻ `<form>`, tệp tin tải lên sẽ bị lỗi biến thành chuỗi văn bản rỗng khi gửi lên server.

PHẦN 2: CHUYÊN SÂU TỪNG Ô NHẬP LIỆU, ĐỊNH DẠNG CSS & TRỢ NĂNG A11Y

Sau khi đã nắm vững toàn bộ nền tảng cốt lõi về cấu trúc thẻ `<form>`, các thuộc tính điều khiển và phương thức truyền nhận dữ liệu ở **Phần 1**, **Phần 2** sẽ đào sâu vào từng loại ô nhập liệu chuyên biệt (Checkbox, Radio, Range, Number, Button...), các kỹ thuật tạo kiểu giao diện CSS hiện đại (`:placeholder-shown`, Floating Label, `:focus-within`) và quy chuẩn Trợ năng Web (**Web Accessibility - A11y** theo chuẩn W3C/WAI-ARIA).

5.2 PHẦN 2: Chuyên Sâu Từng Ô Nhập Liệu, Định Dạng CSS & Chuẩn Trợ Năng A11y

Phần này đi sâu vào chi tiết kỹ thuật từng phần tử nhập liệu chuyên biệt, các thuộc tính điều khiển nâng cao, kỹ thuật định dạng giao diện CSS hiện đại và quy chuẩn Trợ năng Web (**Web Accessibility - A11y**) đạt chuẩn công nghiệp.

5.3 Thẻ Ô Chọn Checkbox (`<input type="checkbox">`)

[Khái Niệm] Định Nghĩa Checkbox

Checkbox là một loại phần tử nhập liệu (`<input>`) dạng ô chọn nhiều (**multi-choice**), cho phép người dùng chọn **0, 1 hoặc nhiều lựa chọn** trong cùng một nhóm.

- **Khác biệt với Radio Button**: Radio button chỉ cho phép chọn duy nhất 1 trong nhiều lựa chọn (**single-choice**).
- **Cơ chế gửi dữ liệu**: Nếu được tích chọn, checkbox sẽ gửi giá trị tương ứng về máy chủ. Nếu không được chọn, giá trị của nó sẽ không được gửi đi.
- **Ứng dụng phổ biến**: Chọn nhiều tùy chọn (sở thích, kỹ năng, ngôn ngữ lập trình...), xác nhận đồng ý điều khoản, hoặc bật/tắt một tính năng.

5.3.1 Cú Pháp Cơ Bản & Thuộc Tính Thường Gặp

Cú pháp tạo ô chọn Checkbox đơn giản

```
<input type="checkbox" name="vehicle" value="Bike"> Tôi có xe đạp
<input type="checkbox" name="vehicle" value="Car"> Tôi có ô tô
```

BROWSER PREVIEW Kết Quả Hiển Thị Trình Duyệt: Checkbox Cú Pháp Cơ Bản

☐ Tôi có xe đạp

☐ Tôi có ô tô

Dưới đây là bảng tổng hợp các thuộc tính phổ biến nhất được sử dụng cùng với thẻ Checkbox:

[Bảng] Bảng Thuộc Tính Checkbox Cần Nắm Vững

Thuộc Tính	Ý Nghĩa	Ví Dụ Đầy Đủ
type="checkbox"	Xác định phần tử nhập liệu là một checkbox.	<input type="checkbox">
name	Tên trường dữ liệu, dùng để gom nhóm các checkbox liên quan.	<input type="checkbox" name="hobby">
value	Giá trị sẽ gửi đi lên máy chủ nếu checkbox được chọn.	<input type="checkbox" value="football">
checked	Đặt checkbox ở trạng thái mặc định được chọn sẵn khi tải trang.	<input type="checkbox" checked>
required	Bắt buộc phải chọn ô này trước khi gửi form.	<input type="checkbox" required>
disabled	Vô hiệu hóa ô chọn, không cho phép người dùng thay đổi trạng thái.	<input type="checkbox" disabled>

5.3.2 Ví Dụ Đầy Đủ & Cơ Chế Gửi Dữ Liệu Lên Server

Form chọn sở thích sử dụng Checkbox

```
<form action="/submit" method="GET">
  <p>Sở thích của bạn:</p>
  <input type="checkbox" name="hobby" value="music"> Nghe nhạc<br>
  <input type="checkbox" name="hobby" value="sport" checked> Thể thao<br>
  <input type="checkbox" name="hobby" value="travel"> Du lịch<br>
  <br>
  <input type="submit" value="Gửi">
</form>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Form Chọn Sở Thích**

Sở thích của bạn:

- ☐ Nghe nhạc
- ☑ **Thể thao** *(Checked mặc định)*
- ☐ Du lịch

Gửi

[Mẹo] Kết Quả Truy Vấn HTTP Request

Nếu người dùng chọn đồng thời hai ô **Nghe nhạc** và **Thể thao**, dữ liệu URL Chuỗi truy vấn (Query String) gửi lên máy chủ sẽ là:

```
hobby=music&hobby=sport
```

Giải thích: Thuộc tính `name` đóng vai trò là **Tên Biến (Key)**, còn thuộc tính `value` đóng vai trò là **Giá Trị (Value)** trong cặp dữ liệu truyền đi `key=value`.

Lưu Ý Quan Trọng Khi Gửi Dữ Liệu:

1. **Không chọn (Unchecked)**: Nếu checkbox không được tích chọn, trình duyệt sẽ **hoàn toàn không gửi bất kỳ dữ liệu nào** tương ứng với `name` đó (Đây là cơ chế **Successful Control** theo chuẩn đặc tả W3C HTML5).
2. **Cùng tên (`name`)**: Khi nhiều checkbox có cùng `name`, khi submit trình duyệt sẽ gửi đi một danh sách các giá trị đã chọn dưới dạng dãy tham số trùng tên.
3. **Checkbox độc lập**: Nếu muốn các ô chọn hoạt động hoàn toàn độc lập với nhau (thu nhận từng giá trị riêng biệt), hãy đặt các giá trị `name` khác nhau.

5.3.3 So Sánh Checkbox name Giống Nhau (Gom Nhóm) vs name Khác Nhau (Độc Lập)

Trong thiết kế biểu mẫu HTML, cách đặt tên thuộc tính `name` quyết định trực tiếp bản chất cấu trúc dữ liệu mà máy chủ (Backend) thu nhận được.

[Bảng] Ma Trận So Sánh thuộc tính name Giống Nhau vs Khác Nhau

Đặc Điểm So Sánh	Cú Pháp HTML Mẫu	Dữ Liệu Submit (Âm nhạc + Du lịch)	Ngữ Cảnh Ứng Dụng
name Giống Nhau (Gom Nhóm)	<input name="hobby"> *(Dùng chung name)*	hobby=music& hobby=travel *(Gửi mảng 1 key)*	Nhóm tùy chọn cùng chủ đề (sở thích, kỹ năng). Backend nhận mảng giá trị.
name Khác Nhau (Lựa Chọn Độc Lập)	<input name="music"> <input name="travel"> *(Dùng name riêng)*	music=yes& travel=yes *(Gửi từng key riêng)*	Các cài đặt/tính năng độc lập (bật/tắt thông báo). Backend nhận từng key riêng.

Trường Hợp 1: Gom Nhóm Checkbox Cùng Name (name="hobby"):

Khi các tùy chọn thuộc về cùng một danh mục (ví dụ: Danh sách sở thích), ta đặt chung name="hobby". Khi submit, trình duyệt sẽ gom tất cả các ô được chọn thành một danh sách gửi về máy chủ:

Form gom nhóm Checkbox sử dụng chung name="hobby"

```
<form action="/submit" method="GET">
  <p>Sở thích của bạn:</p>
  <input type="checkbox" name="hobby" value="music" checked> Âm nhạc
  <input type="checkbox" name="hobby" value="sport"> Thể thao
  <input type="checkbox" name="hobby" value="travel" checked> Du lịch
  <input type="submit" value="Gửi">
</form>
```

BROWSER PREVIEW Mô Phỏng Trình Duyệt: Gom Nhóm Checkbox Chung name="hobby"

Sở thích của bạn:

☑ **Âm nhạc** ☐ Thể thao ☑ **Du lịch**

Gửi

[Mẹo] Kết Quả Chuỗi Truy Vấn Gửi Về Máy Chủ (Query String)

Khi chọn đồng thời **Âm nhạc** và **Du lịch**, dữ liệu nhận được ở Backend là:

```
hobby=music&hobby=travel
```

Trường Hợp 2: Checkbox Độc Lập Khác Name (name Riêng Biệt):

Khi mỗi ô chọn đại diện cho một cài đặt/tính năng riêng biệt (như bật/tắt nhận tin tức, khuyến mãi, cập nhật), ta khai báo các thuộc tính name riêng lẻ:

Form tùy chọn Checkbox khai báo name riêng biệt

```
<form action="/submit" method="GET">
  <p>Tùy chọn cài đặt hệ thống:</p>
  <input type="checkbox" name="newsletter" value="yes" checked> Nhận tin tức
  <input type="checkbox" name="offers" value="yes"> Nhận khuyến mãi
  <input type="checkbox" name="updates" value="yes" checked> Nhận cập nhật
  <input type="submit" value="Gửi">
</form>
```

BROWSER PREVIEW — **Mô Phỏng Trình Duyệt: Tùy Chọn Độc Lập Khác name**

Tùy chọn cài đặt hệ thống:

☑ **Nhận tin tức** ☐ Nhận khuyến mãi ☑ **Nhận cập nhật**

Gửi

[Mẹo] Kết Quả Chuỗi Truy Vấn Gửi Về Máy Chủ (Query String)

Khi chọn đồng thời **Nhận tin tức** và **Nhận cập nhật**, dữ liệu nhận được ở Backend là:

```
newsletter=yes&updates=yes
```

Ghi chú: Tương tự nếu khai báo riêng tên từng sở thích (name="music", name="travel"), dữ liệu gửi về sẽ là: music=yes&travel=yes.

[Khái Niệm] Quy Tắc Quyết Định Chọn Thuộc Tính name Chuẩn Senior Frontend Engineer

- **Sử dụng name GIỐNG NHAU** khi các ô chọn thuộc về cùng 1 tập hợp danh mục (sở thích, danh sách kỹ năng, danh mục sản phẩm). Backend sẽ xử lý dữ liệu dưới dạng mảng (**Array**).
- **Sử dụng name KHÁC NHAU** khi mỗi ô chọn đại diện cho một cờ bật/tắt (**Boolean Flag**) độc lập (như đồng ý điều khoản, đăng ký bản tin, bật thông báo qua Email).

5.3.4 Tối Ưu Trải Nghiệm Người Dùng Với Thẻ <label>

Nếu không có thẻ <label>, người dùng vẫn có thể chọn checkbox bằng cách nhấp trực tiếp vào ô vuông nhỏ. Tuy nhiên, việc không sử dụng label sẽ mất đi các lợi ích quan trọng sau:

- **Dễ dàng thao tác**: Khi có label, người dùng có thể nhấp vào cả dòng văn bản để bật/tắt checkbox, thay vì phải căn chỉnh con trỏ vào ô vuông nhỏ.
- **Khắc phục khó khăn trên di động**: Giúp tương tác thuận tiện hơn khi kích thước checkbox nhỏ hoặc nằm gần các thành phần khác.
- **Tạo sự liên kết logic**: Khi sử dụng label for="id", nhãn được gắn kết trực tiếp với checkbox thông qua thuộc tính id. Ngược lại nếu không có label, văn bản chỉ là text đơn thuần không có liên kết logic.

Ví dụ hoàn chỉnh kết hợp Checkbox với thẻ label và thuộc tính action

```
<form action="https://www.w3schools.com/action_page.php" target="_blank">
    <p>Những ngôn ngữ lập trình bạn đã học:</p>
    <input type="checkbox" id="ngonNguC" name="language" value="c">
    <label for="ngonNguC">Ngôn ngữ C</label><br>

    <input type="checkbox" id="ngonNguJava" name="language" value="java">
    <label for="ngonNguJava">Ngôn ngữ Java</label><br>

    <input type="checkbox" id="ngonNguPhp" name="language" value="php">
    <label for="ngonNguPhp">Ngôn ngữ PHP</label><br>

    <input type="submit" value="Gửi">
</form>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Tương Tác Chọn Ngôn Ngữ Qua Thẻ Label**

Những ngôn ngữ lập trình bạn đã học:

☑ **Ngôn ngữ C** ← **Nhấp vào chữ "Ngôn ngữ C" cũng kích hoạt ô chọn**

☐ Ngôn ngữ Java

☐ Ngôn ngữ PHP

Gửi

[Ghi Chú] Giải Thích Cấu Trúc Mã Nguồn

- `action="https://www.w3schools.com/action_page.php"`: Khi gửi form, dữ liệu sẽ được chuyển tiếp đến trang này để xử lý.
- `target="_blank"`: Kết quả xử lý form sẽ được mở ra trong một tab mới trên trình duyệt.
- `name="language"`: Gom nhóm các ô chọn ngôn ngữ lập trình lại với nhau.
- `value="c|java|php"`: Khi tích chọn ô nào, giá trị tương ứng của ô đó sẽ được gửi đi. Ví dụ: nhấp chọn ô Ngôn ngữ C → tương ứng với lựa chọn `language=c`.
- `label for="id"`: Liên kết nhãn văn bản với thẻ input tương ứng qua ID.

5.3.5 Các Kỹ Thuật Sử Dụng Checkbox Nâng Cao

Checkbox Bắt Buộc Chọn (`required`):

Thêm thuộc tính `required` để bắt buộc người dùng phải tích chọn trước khi cho phép submit form (thường áp dụng cho ô Đồng ý điều khoản):

Bắt buộc chọn checkbox điều khoản

```
<form>
    <label>
        <input type="checkbox" name="agree" required>
        Tôi đồng ý với điều khoản
    </label>
    <input type="submit" value="Gửi">
</form>
```

BROWSER PREVIEW Mô Phỏng Trình Duyệt: Checkbox Bắt Buộc Chọn (Required Validation)

☐ Tôi đồng ý với điều khoản * (Bắt buộc)

Gửi

Checkbox Mặc Định Được Chọn (`checked`):

Sử dụng thuộc tính `checked` để ô chọn luôn ở trạng thái đã được tích sẵn khi trang web tải xong:

Khởi tạo checkbox mặc định checked

```
<input type="checkbox" name="subscribe" value="yes" checked> Nhận tin tức
```

Kỹ Thuật Checkbox Ẩn (Hidden Checkbox Fallback):

Mặc định trong chuẩn HTML, nếu checkbox không được tích chọn, trình duyệt sẽ **hoàn toàn không gửi bất kỳ dữ liệu nào** lên máy chủ. Việc này gây khó khăn cho hệ thống backend khi cần ghi nhận giá trị `false` hoặc `"no"`.

Giải pháp: Sử dụng một thẻ `<input type="hidden">` đặt nằm **ngay trước** thẻ checkbox có cùng thuộc tính `name`.

Kỹ thuật gửi giá trị mặc định khi không chọn checkbox

```
<!-- Thẻ Hidden chứa giá trị mặc định khi KHÔNG chọn -->
<input type="hidden" name="subscribe" value="no">

<!-- Thẻ Checkbox chứa giá trị khi ĐƯỢC chọn -->
<input type="checkbox" name="subscribe" value="yes"> Nhận tin tức
```

[Khái Niệm] Nguyên Lý Hoạt Động Của Hidden Fallback

- **Khi KHÔNG chọn Checkbox**: Trình duyệt bỏ qua ô checkbox và gửi giá trị của ô Hidden → Dữ liệu nhận được: `subscribe=no`.
- **Khi CÓ chọn Checkbox**: Cả hai thẻ đều được gửi đi. Do ô Checkbox nằm ở sau, giá trị `subscribe=yes` của nó sẽ đè lên (**override**) giá trị `no` đứng trước → Dữ liệu nhận được: `subscribe=yes`.

5.3.6 Phân Biệt HTML Attribute checked vs DOM Property

[Bảng] So Sánh HTML Attribute vs DOM Property

Đặc Điểm	HTML Attribute (checked)	DOM Property (element.checked)
Bản chất	Xác định trạng thái ban đầu khi tải trang.	Phản ánh trạng thái thời gian thực (Real-time).
Tương tác người dùng	**KHÔNG thay đổi** khi nhấp chọn.	**TỰ ĐỘNG thay đổi** (true / false).
Truy cập JavaScript	elem.getAttribute('checked')	elem.checked hoặc elem.defaultChecked
Khôi phục Form	Dùng để Reset form về ban đầu.	Dùng để kiểm tra trạng thái trước khi xử lý Logic.

5.3.7 Bắt Buộc Chọn Ít Nhất 1 Checkbox Trong Nhóm

[Cảnh Báo] Hạn Chế Của Thuộc Tính required Mặc Định

Khi gán required cho nhiều checkbox có cùng name, chuẩn HTML5 yêu cầu người dùng phải tích chọn **TẤT CẢ các ô đó** thay vì chọn ít nhất 1 ô.

Giải pháp: Sử dụng JavaScript Constraint Validation API (setCustomValidity) để kiểm tra cả nhóm:

Giải pháp Validate bắt buộc chọn ít nhất 1 Checkbox trong nhóm

```
function validateCheckboxGroup() {
  const checkboxes = document.querySelectorAll('input[name="hobby"]');
  // Kiểm tra xem có ít nhất 1 ô nào được checked hay không
  const isOneChecked = Array.from(checkboxes).some(cb => cb.checked);

  checkboxes.forEach(cb => {
    if (!isOneChecked) {
      // Gán thông báo lỗi nếu chưa chọn ô nào
      cb.setCustomValidity('Vui lòng chọn ít nhất một sở thích!');
    } else {
      // Xóa thông báo lỗi khi đã chọn hợp lệ
      cb.setCustomValidity('');
    }
  });
}
```

5.3.8 Thu Thập Dữ Liệu Trong JavaScript Modern & React

Lấy Toàn Bộ Mảng Checkbox Được Chọn Với FormData:

API `FormData.getAll()` trong ES6+ giúp thu thập danh sách cực kỳ gọn gàng:

Thu thập mảng dữ liệu Checkbox với FormData

```
const form = document.querySelector('form');
form.addEventListener('submit', (e) => {
  e.preventDefault();
  const formData = new FormData(form);

  // Trả về mảng chứa tất cả các value được chọn của 'hobby'
  const selectedHobbies = formData.getAll('hobby');
  console.log('Các sở thích đã chọn:', selectedHobbies);
});
```

Quản Lý State Checkbox Trong React (Controlled Component):

Trong các Framework như React, trạng thái của danh sách Checkbox thường được quản lý dưới dạng mảng State:

Xử lý nhóm Checkbox trong React Component

```
import React, { useState } from 'react';

function HobbySelector() {
  const [selectedHobbies, setSelectedHobbies] = useState([]);

  const handleCheckboxChange = (event) => {
    const { value, checked } = event.target;
    if (checked) {
      // Thêm giá trị vào mảng state nếu được chọn
      setSelectedHobbies([...selectedHobbies, value]);
    } else {
      // Lọc bỏ giá trị khỏi mảng state nếu bỏ chọn
      setSelectedHobbies(selectedHobbies.filter(item => item !== value;
    }
  };

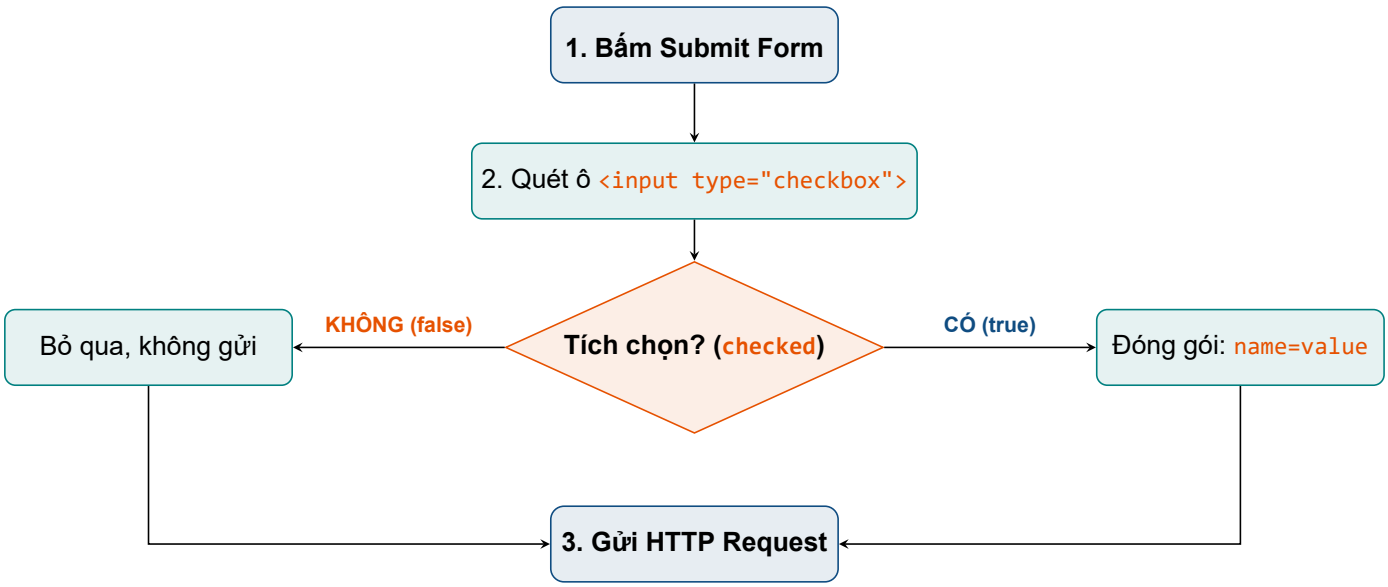
  return (
    <div>
      <label>
        <input type="checkbox" value="music" onChange={handleCheckboxChange} />
      Nghe
      nhạc
      </label>
      <label>
        <input type="checkbox" value="sport" onChange={handleCheckboxChange} />
      Thể
      thao
      </label>
    </div>
  );
}
```

5.3.9 Cảnh Báo Bảo Mật (Security Notes)

[Cảnh Báo] Cảnh Báo Bảo Mật Dữ Liệu Checkbox

- **Không bao giờ tin tưởng giá trị `value` gửi từ Client**: Người dùng có thể dễ dàng mở Dev-Tools trên trình duyệt để sửa đổi thuộc tính `value="free"` thành `value="admin"` trước khi bấm Submit.
- **Khử trùng dữ liệu (Sanitization & Validation ở Backend)**: Máy chủ (Backend) luôn phải kiểm tra danh sách giá trị nhận được có nằm trong tập danh mục cho phép (**Whitelist**) hay không trước khi lưu vào cơ sở dữ liệu.

5.3.10 Minh Họa Sơ Đồ Luồng Xử Lý Dữ Liệu Checkbox Khi Submit



Hình 5.1: Sơ đồ quy trình đóng gói dữ liệu Checkbox của trình duyệt khi submit Form

5.3.11 Bảng Tổng Kết & Ghi Nhớ Cốt Lõi

[Bảng] Tóm Tắt Các Thuộc Tính Và Kỹ Thuật Checkbox

Thuộc Tính / Thẻ	Mô Tả	Ghi Chú Thực Hành
`checked`	Mặc định được chọn khi tải trang.	Giúp tăng tỷ lệ chuyển đổi cho lựa chọn ưu tiên.
`required`	Bắt buộc phải chọn ô này mới được submit.	Dùng khi yêu cầu đồng ý điều khoản dịch vụ.
`name`	Tên của trường dữ liệu gửi đi.	Thuộc tính quan trọng nhất để server nhận diện.
`value`	Giá trị gửi lên máy chủ khi ô được chọn.	Nếu không khai báo value, trình duyệt mặc định gửi ”on”.
`hidden input`	Thẻ ẩn gửi giá trị mặc định khi không chọn.	Tránh bị mất dữ liệu hay lỗi khuyết key ở Backend.

[Mẹo] Những Điểm Bắt Buộc Phải Ghi Nhớ

1. Checkbox **không chọn thì không gửi dữ liệu** → Sử dụng `<input type="hidden">` để khắc phục.
2. Các checkbox có **cùng name** khi submit sẽ gửi dữ liệu dưới dạng một danh sách giá trị (dãy các tham số trùng tên).
3. Luôn phải khai báo thuộc tính `name` để máy chủ thu nhận được dữ liệu.
4. Nên luôn bọc hoặc kết hợp với thẻ `<label>` để tăng tính thân thiện và trải nghiệm người dùng.

5.3.12 Đọc Thêm (Mở Rộng Kiến Thức Chuyên Sâu)

Để nâng cao năng lực thiết kế và phát triển ứng dụng web thực tế, lập trình viên Frontend cần nắm vững các khía cạnh chuyên sâu sau đây liên quan đến Checkbox:

a. Trạng Thái Không Xác Định (Indeterminate State) Trong JavaScript:

Ngoài hai trạng thái hiển thị cơ bản là `checked = true` và `checked = false`, HTML Checkbox còn sở hữu trạng thái thứ 3 gọi là **Indeterminate** (Lơ lửng / Không xác định). Trạng thái này thường xuất hiện trong giao diện danh sách cây (Tree-view) hoặc ô "Chọn tất cả" (**Select All**) khi chỉ có một số ô con được chọn.

Thiết lập trạng thái Indeterminate bằng JavaScript

```
const parentCheckbox = document.getElementById('selectAll');
// Trạng thái này KHÔNG THỂ thiết lập trực tiếp bằng thuộc tính HTML
parentCheckbox.indeterminate = true; // Hiển thị dấu gạch ngang (-) trên ô chọn
```

BROWSER PREVIEW Mô Phỏng Trình Duyệt: Trạng Thái Indeterminate (Lơ lửng)

- [-] **Chọn tất cả danh mục** ← Trạng thái Indeterminate (Dấu gạch ngang)
- [x] Danh mục 1 (Đã chọn)
- [ ] Danh mục 2 (Chưa chọn)

b. Gom Nhóm Truy Cập Chuẩn Accessibility Với <fieldset> Và <legend>:

Khi có một nhóm nhiều Checkbox liên quan đến cùng một câu hỏi (như danh sách sở thích), việc gom nhóm bằng các thẻ chuẩn Accessibility là yêu cầu bắt buộc đối với Trình đọc màn hình (Screen Reader):

Gom nhóm Checkbox hợp chuẩn WCAG/a11y

```
<fieldset>
  <legend>Chọn các kỹ năng Frontend bạn làm chủ:</legend>
  <label><input type="checkbox" name="skill" value="html"> HTML5</label>
  <label><input type="checkbox" name="skill" value="css"> CSS3</label>
  <label><input type="checkbox" name="skill" value="js"> JavaScript</label>
</fieldset>
```

BROWSER PREVIEW **Mô Phỏng Trình Duyệt: Khung Gom Nhóm Thẻ <fieldset> & <legend>**

Chọn các kỹ năng Frontend bạn làm chủ: *(Tiêu đề <legend>)*

☐ HTML5 ☑ **CSS3** ☑ **JavaScript**

c. Tùy Biến Giao Diện Checkbox Bằng CSS Modern (appearance: none):

Các ô chọn mặc định của trình duyệt thường có giao diện thô cứng và không đồng nhất giữa các hệ điều hành (Windows, macOS, iOS). Kỹ thuật CSS hiện đại cho phép tái thiết kế giao diện Checkbox hoàn toàn theo thiết kế Figma:

Custom giao diện Checkbox đẹp mắt bằng CSS3

```css
input[type="checkbox"] {
  appearance: none; /* Bỏ giao diện mặc định của hệ điều hành */
  width: 20px;
  height: 20px;
  border: 2px solid #0056b3;
  border-radius: 4px;
  cursor: pointer;
  transition: all 0.2s ease-in-out;
}

input[type="checkbox"]:checked {
  background-color: #0056b3;
  position: relative;
}

/* Tạo dấu tích chọn (Checkmark) bằng Pseudo-element ::after */
input[type="checkbox"]:checked::after {
  content: '';
  position: absolute;
  left: 6px; top: 2px;
  width: 5px; height: 10px;
  border: solid white;
  border-width: 0 2px 2px 0;
  transform: rotate(45deg);
}
```

BROWSER PREVIEW **Mô Phỏng Trình Duyệt: Giao Diện Custom Checkbox Bằng CSS Modern**

☑ **Checkbox Đã Được Styling Với Bo Góc 4pt & Màu Brand Primary Blue**

☐ Checkbox Chưa Chọn Với Viền Xanh Đẹp Mắt

d. Xử Lý Mảng Checkbox Ở Backend (PHP, Node.js, Express):

Trong các ngôn ngữ server-side như PHP, nếu muốn tự động nhận nhiều checkbox gửi lên dưới dạng một mảng (Array) chính thức, cú pháp đặt tên `name` cần có thêm ký tự ngoặc vuông `[]`:

Đặt name dạng mảng cho PHP Backend

```
<input type="checkbox" name="hobby[]" value="music"> Nghe nhạc
<input type="checkbox" name="hobby[]" value="sport"> Thể thao
<!-- Trên PHP Backend sẽ thu nhận được $_POST['hobby'] dưới dạng Array: ['music',
    'sport'] -->
```

5.3.13 Bộ Câu Hỏi Phỏng Vấn Frontend Thực Chiến (Interview Q&A)

[Ghi Chú] Câu Hỏi Phỏng Vấn 1: Phân Biệt Checkbox Và Radio Button

Hỏi: Khi nào nên sử dụng Checkbox và khi nào nên sử dụng Radio Button trong thiết kế giao diện Form?

Trả lời:

- **Checkbox**: Dùng cho bài toán chọn nhiều lựa chọn độc lập (**Multi-choice**, chọn 0, 1 hoặc nhiều) hoặc trường hợp bật/tắt một tính năng đơn lẻ (**Toggle switch**).
- **Radio Button**: Dùng cho bài toán bắt buộc chọn duy nhất 1 trong danh sách các lựa chọn loại trừ lẫn nhau (**Single-choice**).

[Ghi Chú] Câu Hỏi Phỏng Vấn 2: Xử Lý Vấn Đề Khuyết Dữ Liệu Unchecked

Hỏi: Tại sao ô Checkbox không tích chọn lại không gửi dữ liệu về máy chủ? Làm sao để khắc phục ở Frontend?

Trả lời: Theo đặc tả kỹ thuật HTML, trình duyệt chỉ đóng gói các phần tử ở trạng thái **successful control** (có `name` và được `checked`). Để khắc phục, ta đặt một thẻ `<input type="hidden">` có cùng `name` với giá trị mặc định (ví dụ `value="no"`) ngay trước thẻ checkbox.

[Ghi Chú] Câu Hỏi Phỏng Vấn 3: Thiết Lập Trạng Thái Indeterminate

Hỏi: Trạng thái lơ lửng (**Indeterminate**) của Checkbox có thể khai báo trực tiếp bằng thuộc tính HTML được không?

Trả lời: KHÔNG. Trạng thái **Indeterminate** không có thuộc tính HTML tương ứng mà bắt buộc phải thiết lập thông qua thuộc tính DOM JavaScript: `checkboxElement.indeterminate = true`.

5.3.14 Bảng Độ Tương Thích Trình Duyệt & Best Practices Checklist

[Bảng] Bảng Độ Tương Thích Trình Duyệt Của Các Tính Năng Checkbox

Tính Năng	Chrome / Edge	Firefox	Safari (iOS)	Ghi Chú Hỗ Trợ
`<input type="checkbox">`	Full (v1+)	Full (v1+)	Full (v1+)	Hỗ trợ 100% trên tất cả trình duyệt.
`elem.indeterminate`	Full (v1+)	Full (v1+)	Full (v1+)	Thuộc tính DOM JS chuẩn W3C.
`CSS appearance: none`	Full (v84+)	Full (v80+)	Full (v15.4+)	Tùy biến giao diện không cần prefix.
`FormData.getAll()`	Full (v50+)	Full (v44+)	Full (v11.1+)	Gom mảng value checkbox được chọn.

[Mẹo] Checklist 5 Quy Tắc Vàng Khi Lập Trình Checkbox

1. **Luôn kết hợp với thẻ `<label>`**: Tăng diện tích tương tác click cho người dùng trên thiết bị di động.
2. **Khai báo thuộc tính `name` chính xác**: Bắt buộc phải có `name` để máy chủ nhận dạng được trường dữ liệu.
3. **Cung cấp `value` có nghĩa**: Đặt giá trị `value` rõ ràng (ví dụ `value="email"` thay vì bỏ trống để mặc định gửi `"on"`).
4. **Gom nhóm với `<fieldset>` & `<legend>`**: Đảm bảo tính khả truy cập (Accessibility / a11y) đạt chuẩn WCAG 2.1 cho Trình đọc màn hình.
5. **Luôn Validate & Sanitization ở Backend**: Không tin tưởng giá trị `value` gửi từ Client, kiểm tra Whitelist trên Server.

5.4 Thẻ Nút Chọn Đơn Radio Button (<input type="radio">)

[Khái Niệm] Định Nghĩa Radio Button

Radio Button là một loại phần tử nhập liệu (`<input>`) dạng ô chọn đơn (**single-choice**), cho phép người dùng chọn **duy nhất 1 lựa chọn** trong một nhóm các tùy chọn loại trừ lẫn nhau (**mutually exclusive**).

- **Cùng `name` gom nhóm**: Trong một nhóm các nút Radio có cùng thuộc tính `name`, khi người dùng tích chọn một ô mới, trình duyệt sẽ tự động bỏ chọn (**uncheck**) ô đã chọn trước đó.
- **Khác biệt với Checkbox**: Checkbox cho phép chọn nhiều lựa chọn cùng lúc (**multi-choice**), trong khi Radio Button bắt buộc chọn duy nhất một giá trị.
- **Ứng dụng thực tế**: Chọn giới tính (Nam/Nữ/Khác), Phương thức thanh toán (Thẻ tín dụng/PayPal/COD), Hình thức vận chuyển (Giao hàng nhanh/Tiết kiệm), Mức độ hài lòng,...

5.4.1 Cú Pháp Cơ Bản & Bảng Thuộc Tính Thường Gặp

Cú pháp tạo Radio Button cơ bản

```
<input type="radio" name="gender" value="male"> Nam
<input type="radio" name="gender" value="female"> Nữ
```

BROWSER PREVIEW — **Kết Quả Hiển Thị Trình Duyệt: Radio Button Cơ Bản**



[Mẹo] Giải Thích Cú Pháp Gom Nhóm

Vì cả hai ô chọn trên đều có cùng thuộc tính `name="gender"`, trình duyệt hiểu rằng chúng thuộc về cùng một nhóm câu hỏi → Người dùng chỉ có thể nhấp chọn **duy nhất 1 trong 2** tùy chọn.

Dưới đây là bảng tổng hợp các thuộc tính phổ biến nhất được sử dụng với Radio Button:

[Bảng] Bảng Thuộc Tính Radio Button Cần Nắm Vững

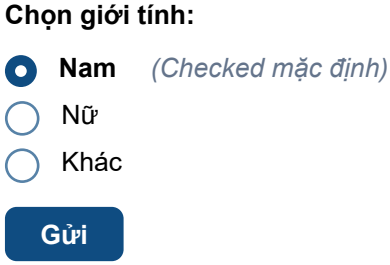
Thuộc Tính	Ý Nghĩa Kỹ Thuật	Ví Dụ Đầy Đủ
`type="radio"`	Xác định phần tử nhập liệu là một nút Radio Button.	`<input type="radio">`
`name`	Gom nhóm các nút Radio với nhau (các ô cùng nhóm bắt buộc có `name` giống hệt nhau).	`<input type="radio" name="gender">`
`value`	Giá trị dữ liệu được gửi về máy chủ khi nút Radio đó được tích chọn.	`<input type="radio" value="male">`
`checked`	Thiết lập nút Radio ở trạng thái mặc định được chọn sẵn khi tải trang.	`<input type="radio" checked>`
`required`	Bắt buộc người dùng phải chọn 1 nút Radio trong nhóm trước khi submit form.	`<input type="radio" name="gender" required>`
`disabled`	Vô hiệu hóa tùy chọn, không cho phép người dùng tương tác hay thay đổi.	`<input type="radio" disabled>`

5.4.2 Ví Dụ Đầy Đủ & Cơ Chế Gửi Dữ Liệu Lên Server

Form chọn giới tính hoàn chỉnh sử dụng Radio Button

```
<form action="/submit" method="POST">
  <p>Chọn giới tính:</p>
  <input type="radio" name="gender" value="male" checked> Nam<br>
  <input type="radio" name="gender" value="female"> Nữ<br>
  <input type="radio" name="gender" value="other"> Khác<br>
  <br>
  <button type="submit">Gửi</button>
</form>
```

BROWSER PREVIEW — **Kết Quả Hiển Thị Trình Duyệt: Form Chọn Giới Tính**



[Mẹo] Kết Quả Dữ Liệu Gửi Về Máy Chủ (HTTP Request)

Khi form được submit:

- Nếu chọn **Nữ**, dữ liệu gửi đi là: `gender=female`.
- Nếu chọn **Nam**, dữ liệu gửi đi là: `gender=male`.

Nguyên lý: Chỉ duy nhất giá trị `value` của nút Radio được tích chọn mới được đóng gói gửi đi lên máy chủ.

5.4.3 Tối Ưu Trải Nghiệm Tương Tác Với Thẻ <label>

Nếu không có thẻ `<label>`, người dùng bắt buộc phải căn chỉnh con trỏ chuột chính xác vào hình tròn nhỏ để chọn. Việc kết hợp với thẻ `<label>` giúp nhấp vào chữ để chọn và tăng trải nghiệm sử dụng (UX).

Cách 1: Liên kết qua thuộc tính `id` và `for`:

Liên kết Radio Button với thẻ label qua ID

```
<form>
    <input type="radio" id="male" name="gender" value="male">
    <label for="male">Nam</label><br>

    <input type="radio" id="female" name="gender" value="female">
    <label for="female">Nữ</label><br>

    <input type="radio" id="other" name="gender" value="other">
    <label for="other">Khác</label><br>
</form>
```

BROWSER PREVIEW **Mô Phỏng Trình Duyệt: Tương Tác Chọn Radio Qua Thẻ Label (Liên kết for=id)**

(●) **Nam** ← Nhấp chữ "Nam" (liên kết for="male") để chọn
( ) Nữ
( ) Khác

Cách 2: Bọc trực tiếp thẻ input bên trong thẻ label:

Bọc trực tiếp Radio Button trong thẻ label

```
<form>
  <p>Phương thức thanh toán:</p>
  <label>
    <input type="radio" name="payment" value="credit"> Thẻ tín dụng
  </label><br>
  <label>
    <input type="radio" name="payment" value="paypal"> PayPal
  </label><br>
</form>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Tương Tác Chọn Phương Thức Thanh Toán**

Phương thức thanh toán:

◯ Thẻ tín dụng

◉ **PayPal** ← **Nhấp vào chữ "PayPal" cũng kích hoạt chọn**

5.4.4 Các Lưu Ý Quan Trọng & Lỗi Thường Gặp

[Cảnh Báo] 4 Lưu Ý Cốt Lõi Khi Sử Dụng Radio Button

1. **Cùng `name` để gom nhóm**: Các nút Radio phải có cùng thuộc tính `name` thì mới tạo thành một nhóm lựa chọn duy nhất.
2. **Lỗi quên `name` hoặc khác `name`**: Nếu để thuộc tính `name` khác nhau (hoặc bỏ trống `name`), người dùng có thể chọn nhiều nút Radio cùng một lúc → Mất đi bản chất chọn đơn của Radio Button.
3. **Xử lý khi không chọn**: Nếu người dùng không chọn nút Radio nào và form không có thuộc tính `required`, khi submit trình duyệt sẽ **không gửi bất kỳ giá trị nào** đại diện cho nhóm đó.
4. **Luôn dùng `<label>`**: Giúp tăng diện tích nhấp trên thiết bị di động, tạo sự thân thiện cho người dùng.

Minh Hoạ Lỗi Thường Gặp (Khác `name` gây chọn nhiều):

CẢNH BÁO LỖI: Radio Button không có cùng name

```
<!-- LỖI: Hai nút Radio có name/id riêng lẻ, không có name chung gom nhóm -->
<input type="radio" id="opt1" value="A"> Tùy chọn A
<input type="radio" id="opt2" value="B"> Tùy chọn B
<!-- HẬU QUẢ: Người dùng có thể chọn cả A và B cùng lúc (Sai logic Radio!) -->
```

5.4.5 So Sánh Chi Tiết Checkbox vs Radio Button

Dưới đây là bảng ma trận so sánh toàn diện giúp lập trình viên đưa ra quyết định thiết kế chính xác giữa Checkbox và Radio Button:

[Bảng] Ma Trận So Sánh Checkbox vs Radio Button

Đặc Điểm So Sánh	Checkbox (<input type="checkbox">)	Radio Button (<input type="radio">)
Số lượng lựa chọn	Chọn **nhiều lựa chọn** cùng lúc (0, 1 hoặc nhiều).	Chọn **duy nhất 1 giá trị** trong nhóm.
Tác dụng của name	Có thể giống nhau (gom mảng) hoặc khác nhau (độc lập).	**Bắt buộc phải giống nhau** để gom nhóm chọn duy nhất 1.
Cơ chế gửi dữ liệu	Gửi nhiều cặp tham số (`hobby=music&hobby=sport`).	Gửi **duy nhất 1 giá trị** của ô được chọn (`gender=male`).
Mặc định chọn trước	Thuộc tính `checked` có thể đặt trên **nhiều ô**.	Thuộc tính `checked` chỉ nên đặt trên **1 ô duy nhất**.
Ngữ cảnh sử dụng	Chọn sở thích, danh mục kỹ năng, ngôn ngữ lập trình...	Chọn giới tính, phương thức thanh toán, hình thức vận chuyển...

Ví Dụ Mã Nguồn So Sánh Trực Quan:

Ví dụ đối chiếu cú pháp Radio vs Checkbox

```
<!-- Radio Button: Bắt buộc chọn duy nhất 1 giới tính -->
<h3>Radio (Chỉ chọn 1)</h3>
<input type="radio" name="gender" value="male"> Nam
<input type="radio" name="gender" value="female"> Nữ

<!-- Checkbox: Có thể chọn nhiều sở thích cùng lúc -->
<h3>Checkbox (Có thể chọn nhiều)</h3>
<input type="checkbox" name="hobby" value="reading"> Đọc sách
<input type="checkbox" name="hobby" value="travel"> Du lịch
```

BROWSER PREVIEW **Mô Phỏng Trình Duyệt: Giao Diện So Sánh Radio vs Checkbox**

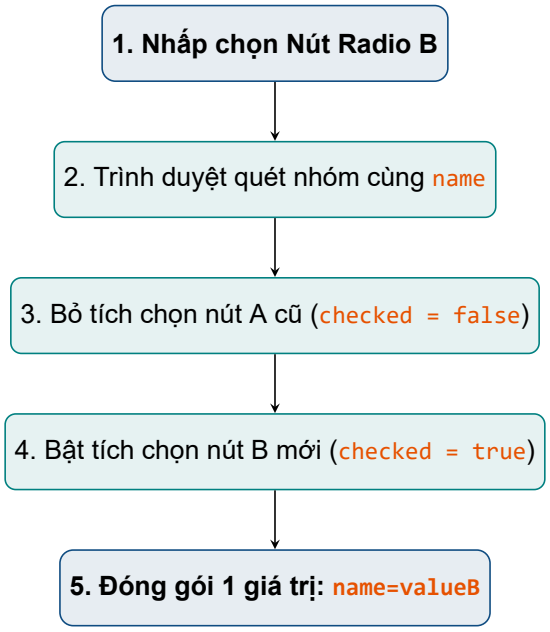
Radio Button (Chỉ chọn 1):

(•) Nam ( ) Nữ

Checkbox (Có thể chọn nhiều):

☑ **Đọc sách** ☑ **Du lịch**

5.4.6 Minh Họa Sơ Đồ Luồng Xử Lý Radio Button



Hình 5.2: Sơ đồ quy trình loại trừ lẫn nhau (Mutually Exclusive) của nhóm Radio Button

5.4.7 Đọc Thêm (Mở Rộng Kiến Thức Chuyên Sâu)

a. Điều Hướng Bàn Phím Chuẩn Accessibility (a11y Keyboard Navigation):

Radio Button sở hữu cơ chế điều hướng bàn phím rất đặc thù theo tiêu chuẩn WCAG:

- **Phím Tab**: Nhảy vào nhóm Radio (chỉ dừng lại ở nút Radio đang được tích chọn trong nhóm).
- **Phím mũi tên (←→↑↓)**: Di chuyển và tự động chuyển đổi tích chọn giữa các nút Radio trong cùng nhóm.

b. Tùy Biến Giao Diện Radio Button Bằng CSS Modern (`appearance: none`):

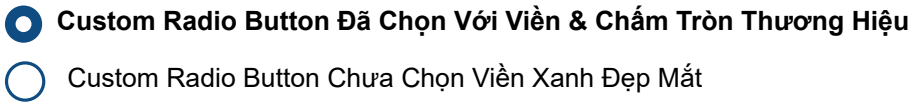
Tương tự Checkbox, ta có thể dùng CSS3 để tạo chấm tròn Radio đẹp mắt đồng bộ thiết kế thương hiệu:

Custom giao diện Radio Button đẹp mắt với CSS3

```
input[type="radio"] {
  appearance: none; /* Xóa giao diện mặc định hệ điều hành */
  width: 20px; height: 20px;
  border: 2px solid #0056b3;
  border-radius: 50%; /* Tạo hình tròn chuẩn Radio */
  outline: none; cursor: pointer;
}

input[type="radio"]:checked {
  background-color: white;
  border-color: #0056b3;
  box-shadow: inset 0 0 0 4px #0056b3; /* Tạo chấm tròn nhỏ bên trong */
}
```

BROWSER PREVIEW **Mô Phỏng Trình Duyệt: Custom Radio Button CSS Modern**



c. Quản Lý State Radio Button Trong React (Controlled Component):

Trong React, nhóm Radio Button thường được liên kết với 1 biến State chuỗi duy nhất:

Xử lý nhóm Radio Button trong React

```jsx
import React, { useState } from 'react';

function GenderForm() {
  const [gender, setGender] = useState('male'); // State lưu 1 giá trị duy nhất

  return (
    <div>
      <label>
        <input
          type="radio"
          name="gender"
          value="male"
          checked={gender === 'male'}
          onChange={(e) => setGender(e.target.value)}
        /> Nam
      </label>
      <label>
        <input
          type="radio"
          name="gender"
          value="female"
          checked={gender === 'female'}
          onChange={(e) => setGender(e.target.value)}
        /> Nữ
      </label>
    </div>
  );
}
```

5.4.8 Bộ Câu Hỏi Phỏng Vấn Frontend Thực Chiến (Interview Q&A)

[Ghi Chú] Câu Hỏi Phỏng Vấn 1: Điều Kiện Để Radio Button Gom Nhóm

Hỏi: Điều kiện bắt buộc để nhiều nút Radio Button hoạt động loại trừ lẫn nhau (chỉ chọn được 1) là gì? Điều gì xảy ra nếu quên khai báo thuộc tính này?

Trả lời: Điều kiện bắt buộc là các nút Radio đó phải có cùng giá trị thuộc tính `name`. Nếu quên khai báo hoặc đặt tên `name` khác nhau, trình duyệt sẽ coi chúng là các nút riêng biệt và cho phép người dùng chọn tất cả cùng lúc (sai nguyên tắc UX).

[Ghi Chú] Câu Hỏi Phỏng Vấn 2: Dữ Liệu Radio Gửi Đi Khi Submit

Hỏi: Khi Submit Form, dữ liệu của nhóm Radio Button được trình duyệt đóng gói gửi đi như thế nào?

Trả lời: Trình duyệt chỉ đóng gói duy nhất 1 cặp `name=value` tương ứng với nút Radio đang được tích chọn (`checked = true`). Các nút không được chọn trong cùng nhóm sẽ hoàn toàn bị bỏ qua.

5.4.9 Bảng Độ Tương Thích Trình Duyệt & Best Practices Checklist

[Bảng] Bảng Độ Tương Thích Trình Duyệt Của Radio Button

Tính Năng	Chrome / Edge	Firefox	Safari (iOS)	Ghi Chú Hỗ Trợ
`<input type="radio">`	Full (v1+)	Full (v1+)	Full (v1+)	Hỗ trợ 100% trên mọi thiết bị.
`Phím mũi tên (Arrow Keys)`	Full (v1+)	Full (v1+)	Full (v1+)	Chuẩn WCAG a11y keyboard.
`CSS appearance: none`	Full (v84+)	Full (v80+)	Full (v15.4+)	Tùy biến hình tròn Radio đẹp mắt.

[Mẹo] Checklist 5 Quy Tắc Vàng Khi Sử Dụng Radio Button

1. **Kiểm tra thuộc tính `name` trùng khớp**: Luôn đảm bảo các nút trong cùng nhóm có tên `name` giống hệt nhau.
2. **Khai báo giá trị `value` rõ ràng**: Đặt `value` cụ thể cho từng nút (ví dụ `value="credit"` thay vì để trống).
3. **Nên chọn mặc định 1 ô (`checked`)**: Đặt `checked` trên tùy chọn phổ biến nhất để giảm bớt 1 thao tác cho người dùng.
4. **Luôn kết hợp với thẻ `<label>`**: Giúp người dùng nhấp vào văn bản để chọn radio dễ dàng trên điện thoại.
5. **Sử dụng đúng mục đích UX**: Chỉ dùng Radio Button khi số lượng lựa chọn ít (từ 2 - 5 lựa chọn). Nếu danh sách quá dài (> 5 lựa chọn), hãy sử dụng thẻ chọn `<select>` (Dropdown Menu).

5.5 Các Thẻ Nút Bấm Trong Form (<input type="submit">, <input type="button">, <button>)

[Khái Niệm] Tổng Quan Về Nút Bấm Trong Form

Nút bấm (**Buttons**) là thành phần kích hoạt tương tác cốt lõi trong biểu mẫu HTML. Trình duyệt cung cấp hai cách chính để khai báo nút bấm: thông qua thẻ nhập liệu tự đóng `<input>` hoặc thẻ mở/đóng linh hoạt `<button>`.

- **Thẻ `<input type="submit">`**: Nút gửi form truyền thống, chỉ chứa văn bản đơn thuần.
- **Thẻ `<input type="button">`**: Nút bấm đơn giản dùng để kích hoạt các kịch bản JavaScript.
- **Thẻ `<button>`**: Nút bấm thế hệ mới cực kỳ linh hoạt, cho phép chứa văn bản, icon, thẻ `<span>`, hình ảnh và các phần tử HTML bên trong.

5.5.1 Thẻ Nút Gửi Form (<input type="submit">)

Định Nghĩa & Bản Chất:

`<input type="submit">` là loại nút dùng để gửi toàn bộ dữ liệu nhập trong form lên URL được chỉ định trong thuộc tính `action` thông qua phương thức `method` (`GET` hoặc `POST`). Đây là loại nút **mặc định** trong thẻ `<form>` nếu một ô nút bấm không được khai báo rõ thuộc tính `type`.

Cú pháp cơ bản của nút input type="submit"

```
<form action="/submit" method="post">
  <input type="text" name="username" placeholder="Tên đăng nhập" required>
  <input type="submit" value="Đăng nhập">
</form>
```

BROWSER PREVIEW — **Kết Quả Hiển Thị Trình Duyệt: Nút Input Submit**



[Mẹo] Giải Thích Luồng Gửi Dữ Liệu

Khi người dùng nhấp vào nút **Đăng nhập**, trình duyệt sẽ gom tất cả dữ liệu có thuộc tính `name` trong form (ví dụ `username`) và đóng gói gửi tới đường dẫn `/submit`.

Các Thuộc Tính Thường Gặp Của <input type="submit">:

[Bảng] Bảng Thuộc Tính Hỗ Trợ Nút Submit

Thuộc Tính	Ý Nghĩa Kỹ Thuật	Ví Dụ Cú Pháp
value	Nội dung văn bản hiển thị trên nút (nếu không khai báo, mặc định trình duyệt hiển thị "Submit").	<input type="submit" value="Gửi">
formaction	Ghi đè đường dẫn action của <form> (chỉ áp dụng khi bấm nút này).	<input type="submit" formaction="/login">
formmethod	Ghi đè phương thức method của form (GET hoặc POST).	<input type="submit" formmethod="get">
formtarget	Quy định nơi mở trang kết quả (_self, _blank, _parent, _top).	<input type="submit" formtarget="_blank">
formnovalidate	Bỏ qua tất cả các kiểm tra hợp lệ (Validation) của HTML5 khi bấm nút này.	<input type="submit" formnovalidate>
disabled	Vô hiệu hóa nút gửi, không cho phép người dùng nhấp chọn.	<input type="submit" disabled>

Ví Dụ Nâng Cao (Ghi Đè Thuộc Tính Form Với Nút Submit):

Ví dụ sử dụng nhiều nút submit với các thuộc tính ghi đè formaction và formnovalidate

```
<form action="/register" method="post">
  <input type="email" name="email" required>

  <!-- Nút gửi bình thường: Gửi đến /register và kiểm tra required -->
  <input type="submit" value="Đăng ký">

  <!-- Nút này gửi nhưng BỎ QUA kiểm tra required -->
  <input type="submit" value="Đăng ký nhanh" formnovalidate>

  <!-- Nút này GHI ĐÈ chuyển tiếp gửi dữ liệu đến trang /login -->
  <input type="submit" value="Đăng nhập" formaction="/login">
</form>
```

BROWSER PREVIEW Mô Phỏng Trình Duyệt: Nhóm Nút Submit Ghi Đè Action

user@example.com

Đăng ký | Đăng ký nhanh | Đăng nhập

[Cảnh Báo] 3 Lưu Ý Quan Trọng Khi Dùng Nút Submit

1. **Nhiều nút submit trong cùng form**: Khi bấm nút nào, trình duyệt sẽ áp dụng các thuộc tính riêng (`formaction`, `formnovalidate`...) của chính nút đó để xử lý việc gửi dữ liệu.
2. **Mặc định hiển thị chữ "Submit"**: Nếu bỏ trống thuộc tính `value`, văn bản hiển thị trên nút sẽ mặc định là "Submit" (hoặc từ tương đương tùy theo ngôn ngữ trình duyệt người dùng).
3. **Đặt nút ngoài form bằng thuộc tính `form="id"`**: Nút submit có thể nằm bất kỳ đâu ngoài thẻ `<form>` miễn là khai báo thuộc tính `form="id_form"`.

Ví dụ đặt nút submit nằm bên ngoài thẻ form

```
<!-- Form được gắn ID riêng -->
<form id="myForm" action="/save">
  <input type="text" name="data">
</form>

<!-- Nút Submit nằm BÊN NGOÀI form nhưng vẫn kích hoạt gửi myForm -->
<input type="submit" form="myForm" value="Lưu dữ liệu">
```

BROWSER PREVIEW Mô Phỏng Trình Duyệt: Nút Submit Nằm Bên Ngoài Form Kết Nối Qua form="id"

Khung Form (id="myForm"): Dữ liệu nhập...

Nút submit nằm ngoài DOM Form nhưng khai báo form="myForm": Lưu dữ liệu

5.5.2 Thẻ Nút Bấm JavaScript (<input type="button">)

Định Nghĩa & Đặc Điểm:

`<input type="button">` tạo ra một nút bấm bình thường. Nút này **không có bất kỳ hành động mặc định nào** (không gửi form, không reset form). Nó được thiết kế chuyên biệt để kết hợp với ngôn ngữ JavaScript nhằm thực thi các sự kiện tương tác (`onclick`, `addEventListener`).

Cú pháp nút input type="button" kích hoạt sự kiện onclick

```
<input type="button" value="Bấm tôi" onclick="alert('Xin chào!')">
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Nút Input Button Kích Hoạt Alert**

Khi người dùng nhấp vào nút: **Bấm tôi** → Trình duyệt hiển thị hộp thoại thông báo: **"Xin chào!"** (Form không bị reload hay gửi đi).

Phân Biệt <input type="button"> vs <input type="submit"> Trong Form:

Form kết hợp cả input type="button" và input type="submit"

```
<form action="/submit" method="POST">
  <input type="text" placeholder="Nhập họ tên">

  <!-- Nút này chỉ chạy JS kiểm tra, KHÔNG gửi form -->
  <input type="button" value="Kiểm tra" onclick="alert('Nút này không gửi
    form!')">

  <!-- Nút này GỬI FORM lên máy chủ -->
  <input type="submit" value="Gửi form">
</form>
```

BROWSER PREVIEW **Mô Phỏng Trình Duyệt: Phân Biệt Nút JS Button Và Nút Submit Form**

Nhập họ tên... | Kiểm tra (Chạy JS) | Gửi form (Submit)

[Bảng] Ma Trận So Sánh <input type="button"> vs <button type="button">

Tiêu Chí	Thẻ <input type="button">	Thẻ <button type="button">
Cú pháp mã HTML	<input type="button" value="Click">	<button type="button">Click</button>
Nội dung hiển thị	**Chỉ chứa văn bản thuần** lấy từ thuộc tính value.	**Linh hoạt chứa HTML** (văn bản, icon SVG, hình ảnh, thẻ ...).
Mức độ tùy biến UI	Hạn chế hơn khi thiết kế giao diện phức tạp.	Rất linh hoạt, dễ dàng định dạng CSS Modern.
Thực tế sử dụng	Ít được sử dụng hơn trong các dự án hiện đại.	**Được ưu tiên lựa chọn** trong hầu hết ứng dụng web.

[Mẹo] Lưu Ý Về Nút Khôi Phục Dữ Liệu <input type="reset">

Bên cạnh <input type="submit"> và <input type="button">, HTML5 còn cung cấp nút khôi phục dữ liệu <input type="reset" value="Nhập lại">. Nút này có tác dụng đưa toàn bộ các phần tử nhập liệu trong form trở về trạng thái giá trị mặc định ban đầu.

Lời khuyên UX/UI: Tránh lạm dụng nút Reset trong thiết kế giao diện ứng dụng web hiện đại vì người dùng có thể nhấp nhầm làm mất toàn bộ thông tin vừa điền!

5.5.3 Thẻ Nút Bấm Đa Năng Linh Hoạt (<button>)

[Khái Niệm] Định Nghĩa & Bản Chất Vượt Trội Của Thẻ <button>

Thẻ <button> tạo ra một nút bấm tương tác đa năng trong biểu mẫu HTML. Điểm cải tiến vượt trội cốt lõi của thẻ <button> so me với thẻ tự đóng <input type="submit"> là nó được thiết kế dưới dạng **Container Element** (thẻ cặp có thẻ mở <button> và thẻ đóng </button>).

- **Khả năng chứa nội dung HTML phong phú**: Bạn có thể lồng bất kỳ văn bản, thẻ định dạng (,), biểu tượng SVG, hình ảnh (<img>), hoặc thậm chí cả badge hiệu ứng Spinner Loading vào bên trong nút.
- **Linh hoạt tùy biến giao diện (CSS UI/UX)**: Dễ dàng áp dụng kỹ thuật Flexbox/Grid, hiệu ứng chuyển cảnh (**Transition**), Hover, Active, và Ripple effect chuẩn Modern Web.
- **Tương thích Accessibility (a11y)**: Hỗ trợ hoàn hảo cho trình đọc màn hình (Screen Reader) và dễ dàng kết hợp các thuộc tính ARIA (aria-expanded, aria-controls, aria-label).

Ví Dụ Đơn Giản: Nút Bấm Chứa Icon SVG & Thẻ HTML Con:

Cú pháp thẻ button chứa icon SVG và văn bản HTML

```
<form action="/submit" method="post">
  <button type="submit" class="btn-send">
    <!-- Hình ảnh hoặc biểu tượng SVG lồng bên trong nút -->
    <svg width="16" height="16" viewBox="0 0 24 24" fill="currentColor">
      <path d="M2.01 21L23 12 2.01 3 2 10l15 2-15 2z"/>
    </svg>
    <span>Gửi Thông Tin Đăng Ký</span>
  </button>
</form>
```

BROWSER PREVIEW Mô Phỏng Trình Duyệt: Giao Diện Thẻ Button Lồng Icon SVG & Text



3 Giá Trị Thuộc Tính type Trong Thẻ <button>:

Khác với thẻ <input>, thuộc tính type trong thẻ <button> quy định trực tiếp hành động mặc định của nút khi người dùng nhấp chuột:

[Bảng] Bảng Phân Loại Giá Trị Thuộc Tính type Trong Thẻ <button>

Giá Trị type	Hành Động Mặc Định	Ngữ Cảnh Áp Dụng Thực Tế	Ví Dụ Cú Pháp
type="submit" (Mặc định)	Tự động gom dữ liệu và gửi form lên Server (giống <input type="submit">).	Dùng cho nút gửi form đăng ký, đăng nhập, thanh toán.	<button type="submit">Gửi</button>
type="reset"	Tự động xóa và khôi phục tất cả ô nhập về trạng thái ban đầu.	Dùng cho nút xóa dữ liệu vừa nhập để điền lại từ đầu.	<button type="reset">Xóa làm lại</button>
type="button"	**Vô hướng (Dead-end)**, không có bất kỳ hành động mặc định nào.	Dùng để kích hoạt kịch bản JavaScript (Modal, Dropdown, AJAX).	<button type="button" onclick="...">Bấm</button>

[Cảnh Báo] CẢNH BÁO NGUY HIỂM 1: Bẫy Mặc Định Khi Bỏ Quên Thuộc Tính 'type'

Nếu bạn khai báo thẻ `<button>` nằm bên trong thẻ `<form>` mà **bỏ quên thuộc tính** `type`, trình duyệt sẽ tự động coi đó là `type="submit"`.

→ **Hậu quả nghiêm trọng**: Khi người dùng nhấp vào một nút bấm vốn chỉ dành để mở Popup/Modal hoặc chạy đoạn mã JavaScript, trình duyệt sẽ tự động kích hoạt gửi form và reload lại toàn bộ trang ngoài ý muốn!

→ **Quy Tắc Vàng Senior**: Luôn luôn khai báo rõ ràng `type="button"` cho tất cả các nút bấm xử lý JavaScript.

[Cảnh Báo] CẢNH BÁO NGUY HIỂM 2: Lỗi Thẻ HTML Không Hợp Lệ (Invalid HTML Nesting)

Theo chuẩn đặc tả W3C HTML5 Specification, thẻ `<button>` thuộc nhóm **Interactive Content**. Do đó:

- **Tuyệt đối KHÔNG lồng thẻ `<a>` (chứa `href`) vào trong `<button>`.**
- **Tuyệt đối KHÔNG lồng một thẻ `<button>` khác hoặc thẻ `<input>` vào trong `<button>`.**

Việc lồng sai này sẽ khiến trình duyệt phá vỡ cây DOM (**DOM Parsing Error**) và làm hỏng trải nghiệm người dùng.

Cơ Chế Đặc Biệt: Gửi Dữ Liệu Với Thuộc Tính `name` & `value` Trên Nút Submit:

Một tính năng cực kỳ mạnh mẽ của thẻ `<button type="submit">` là nó cho phép gán thuộc tính `name` và `value`. Khi người dùng nhấp vào một nút Submit cụ thể trong Form, **chỉ duy nhất `name` và `value` của nút được nhấp mới được đóng gói gửi đi cùng dữ liệu Form**!

[Mẹo] Mẹo Senior: Xử Lý Nhiều Hành Động Submit Trong 1 Form Mà Không Cần JavaScript

Bạn có thể tạo ra các nút Submit khác nhau trong cùng 1 form (ví dụ nút "Lưu nháp" và nút "Xuất bản bài viết"). Máy chủ Backend dựa vào giá trị `value` nhận được để phân loại hành động xử lý:

Ví dụ xử lý nhiều nút Submit với name và value khác nhau

```
<form action="/article/save" method="post">
  <input type="text" name="title" placeholder="Tiêu đề bài viết" required>

  <!-- Nút 1: Gửi name="action" và value="draft" -->
  <button type="submit" name="action" value="draft">Lưu Nháp</button>

  <!-- Nút 2: Gửi name="action" và value="publish" -->
  <button type="submit" name="action" value="publish">Xuất Bản Ngay</button>
</form>
```

BROWSER PREVIEW **Mô Phỏng Trình Duyệt: Form 2 Nút Submit Phân Loại Hành Động**

Tiêu đề bài viết mới...

Lưu Nháp (action=draft) Xuất Bản Ngay (action=publish)

Bảng Thuộc Tính Mở Rộng Đầy Đủ Của Thẻ <button>:

[Bảng] Bảng Thuộc Tính Mở Rộng Hỗ Trợ Bởi Thẻ <button>

Thuộc Tính	Ý Nghĩa Chức Năng Kỹ Thuật	Ví Dụ Cú Pháp
type	Quy định loại nút (submit, reset, button).	<button type="button">Click</button>
name	Tên trường đại diện cho nút bấm khi submit form.	<button type="submit" name="btn">
value	Giá trị gửi về máy chủ khi nút đó được nhấp.	<button type="submit" value="delete">
form	Liên kết nút bấm với ID của một thẻ <form> nằm bất kỳ đâu ngoài DOM.	<button type="submit" form="mainForm">
formaction	Ghi đè đường dẫn action của Form khi gửi bằng nút này.	<button type="submit" formaction="/save">
formmethod	Ghi đè phương thức gửi dữ liệu (GET hoặc POST).	<button type="submit" formmethod="get">
formtarget	Quy định nơi mở cửa sổ kết quả (_blank, _self...).	<button type="submit" formtarget="_blank">
formnovalidate	Gửi form mà không chạy kiểm tra Validation HTML5.	<button type="submit" formnovalidate>
disabled	Vô hiệu hóa nút, đổi giao diện xám và không thể nhấp chọn.	<button type="submit" disabled>
autofocus	Tự động trỏ con trỏ/focus vào nút ngay khi trang tải xong.	<button type="button" autofocus>

Ví Dụ Tổng Hợp Đầy Đủ Tất Cả Các Loại Nút Bấm Trong Form:

Ví dụ tổng hợp đầy đủ các loại nút submit, reset, JS button và nút ngoài form

```html
<form id="regForm" action="/register" method="post">
  <input type="text" name="user" required placeholder="Tên đăng nhập">

  <!-- 1. Nút Submit chính: Gửi dữ liệu form -->
  <button type="submit" class="btn-primary">Đăng ký tài khoản</button>

  <!-- 2. Nút Reset: Khôi phục ô nhập về giá trị ban đầu -->
  <button type="reset" class="btn-secondary">Xóa dữ liệu</button>

  <!-- 3. Nút JS Button: Kích hoạt hàm kiểm tra JavaScript -->
  <button type="button" onclick="alert('Đã bấm nút JS!')">Nhấn thử JS</button>
</form>

<!-- 4. Nút Submit nằm BÊN NGOÀI form nhưng liên kết qua form="regForm" -->
<button type="submit" form="regForm" formaction="/login" class="btn-outer">
  Đăng nhập (Nút ngoài Form)
</button>
```

BROWSER PREVIEW **Mô Phỏng Trình Duyệt: Form Đầy Đủ Tất Cả Các Loại Nút Bấm**

Tên đăng nhập...

Đăng ký tài khoản | Xóa dữ liệu | Nhấn thử JS

Nút bấm nằm bên ngoài form: Đăng nhập (formaction="/login")

5.5.4 Thẻ Nút Khôi Phục Dữ Liệu Reset Form (<input type="reset"> & <button type="reset">)

[Khái Niệm] Định Nghĩa & Bản Chất Của Nút Reset Form

Nút `type="reset"` (khai báo qua `<input type="reset">` hoặc `<button type="reset">`) là loại nút bấm chức năng đặc biệt dùng để **xóa toàn bộ thông tin người dùng vừa nhập** trong biểu mẫu và khôi phục tất cả các ô nhập liệu về **trạng thái / giá trị khởi tạo ban đầu** (**initial default state**).

- **Phạm vi hoạt động**: Chỉ có tác dụng khi được đặt bên trong thẻ `<form>` (hoặc liên kết qua thuộc tính `form="id"`).
- **Cơ chế đối với Server**: **KHÔNG gửi dữ liệu đi**, không phát sinh bất kỳ HTTP Request nào tới máy chủ Backend và không làm reload lại trang.

Cú Pháp Cơ Bản & Ví Dụ Minh Họa Khôi Phục Giá Trị Mặc Định:

Dưới đây là ví dụ minh họa chi tiết sự khác biệt giữa các ô nhập có khai báo thuộc tính `value` mặc định và không có thuộc tính `value`:

Cú pháp sử dụng nút reset với ô nhập có giá trị mặc định

```
<form action="/update-profile" method="post">
  <!-- Ô 1: Có giá trị mặc định value="Nguyễn Văn A" -->
  <label>Họ và tên:</label>
  <input type="text" name="fullname" value="Nguyễn Văn A">

  <!-- Ô 2: Không có giá trị mặc định (rỗng) -->
  <label>Email mới:</label>
  <input type="email" name="email" placeholder="nhập email">

  <!-- Nút Reset dạng input và button -->
  <input type="reset" value="Xóa làm lại">
  <button type="reset">Reset Form</button>
  <button type="submit">Gửi Form</button>
</form>
```

BROWSER PREVIEW **Mô Phỏng Trình Duyệt: Giao Diện Form & Tương Tác Khi Bấm Nút Reset Form**

Giao diện Form hiển thị trên trình duyệt:

Họ và tên: Nguyễn Văn A Email mới: nhập email...

Xóa làm lại Reset Form Gửi Form

Cơ chế khi nhấp chọn nút Reset Form (<button type="reset">):
→ Ô **Họ và tên** khôi phục về giá trị mặc định: **"Nguyễn Văn A"** (lấy từ thuộc tính `value`).
→ Ô **Email mới** khôi phục về trạng thái rỗng: **""** (do không có thuộc tính `value`).

So Sánh Chi Tiết Hành Động Của 3 Loại Nút Bấm Trong Form:

[Bảng] Ma Trận So Sánh Hành Động Mặc Định Của Các Loại Nút Bấm

Loại Nút (type)	Hành Động Mặc Định	Tác Động Dữ Liệu	Phát Request Server?
`type="submit"`	Gửi dữ liệu form tới đường dẫn `action`.	Đóng gói toàn bộ `name=value`.	**CÓ** (Tải lại trang hoặc AJAX).
`type="reset"`	Xóa/khôi phục ô nhập về giá trị ban đầu.	Trả các input về `initial value`.	**KHÔNG** (Chỉ xử lý ở Client).
`type="button"`	**Vô hướng**, không làm gì mặc định.	Không tự động thay đổi dữ liệu.	**KHÔNG** (Chờ gắn sự kiện JS).

3 Lưu Ý Kỹ Thuật Cốt Lõi Khi Sử Dụng Nút Reset:

[Cảnh Báo] LƯU Ý QUAN TRỌNG: 3 Quy Tắc Cần Nhớ Về Nút Reset

1. **Reset đưa về giá trị ban đầu, KHÔNG PHẢI xóa rỗng hoàn toàn**:
   - Nếu ô nhập có sẵn `<input value="abc">` → Khi bấm Reset, giá trị quay về **"abc"**.
   - Nếu ô nhập không có `value` → Khi bấm Reset, giá trị quay về rỗng **""**.
2. **Phạm vi tác động gói gọn trong Form**: Nút Reset chỉ xóa các ô nhập liệu thuộc cùng thẻ `<form>` chứa nó (hoặc liên kết qua thuộc tính `form="id"`). Các ô nằm ngoài form sẽ không bị ảnh hưởng.
3. **Xóa sạch 100% về rỗng phải dùng JavaScript**: Nếu muốn xóa sạch tất cả các ô về rỗng (bất chấp ô đó có `value` mặc định hay không), bạn bắt buộc phải dùng đoạn mã JavaScript để can thiệp vào cây DOM.

[Cảnh Báo] CẢNH BÁO NGUY HIỂM UX: Tại Sao Nên Hạn Chế Sử Dụng Nút Reset Trên Form Hiện Đại?

Trong thiết kế trải nghiệm người dùng (UX/UI Design) hiện đại, các chuyên gia hàng đầu **KHUYÊN NÊN TRÁNH DÙNG NÚT RESET** trên các biểu mẫu giao diện:

→ **Rủi ro nghiêm trọng**: Người dùng thường vô tình nhấp nhầm vào nút "Reset" (xóa) đặt ngay cạnh nút "Submit" (gửi), dẫn đến việc toàn bộ thông tin họ vừa tốn nhiều thời gian điền bị xóa sạch!

5.5.5 So Sánh Chi Tiết <input type="submit"> vs <button type="submit">

Dưới đây là ma trận so sánh toàn diện giúp bạn đưa ra lựa chọn chính xác khi thiết kế biểu mẫu HTML5:

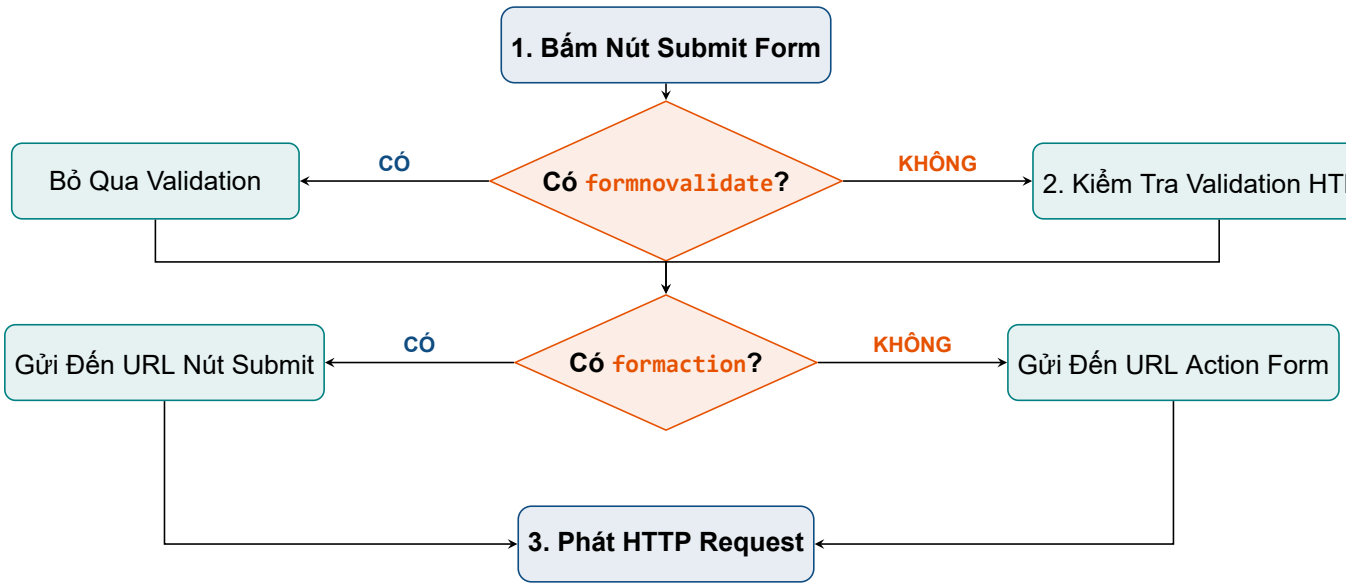
[Bảng] Ma Trận So Sánh <input type="submit"> vs <button type="submit">

Tiêu Chí So Sánh	Thẻ <input type="submit">	Thẻ <button type="submit">
Cấu trúc phần tử HTML	Thẻ tự đóng (**Void Element**), không có phần thân chứa thẻ con.	Thẻ cặp (**Container Element**), chứa được văn bản, icon, thẻ HTML con.
Nội dung hiển thị	**Chỉ chứa văn bản thuần** lấy từ thuộc tính `value`.	**Cực kỳ linh hoạt**: Văn bản, icon SVG, hình ảnh, spinner badge,...
Tùy biến giao diện (UI)	Hạn chế, khó tạo các hiệu ứng nút bấm hiện đại.	**Rất dễ dàng tùy biến CSS Modern** (Flexbox, Pseudo-elements, Ripple).
Hỗ trợ thuộc tính	Hỗ trợ các thuộc tính ghi đè `formaction`, `formnovalidate`...	Hỗ trợ đầy đủ các thuộc tính tương tự.
Khuyên dùng thực tế	Dùng cho các biểu mẫu đơn giản, không yêu cầu thiết kế icon.	**Khuyên dùng cho 100% dự án ứng dụng Web hiện đại**.

[Khái Niệm] Tóm Tắt Quy Tắc Lựa Chọn Thẻ Nút Bấm Chuẩn Senior Engineer

- **Ưu tiên 100% sử dụng thẻ** `<button>` khi thiết kế giao diện ứng dụng Web hiện đại để dễ dàng làm chủ CSS, thêm icon SVG và cải thiện trải nghiệm người dùng (UX).
- **Luôn luôn khai báo thuộc tính** `type` rõ ràng (`type="submit"`, `type="button"`, `type="reset"`) để tránh bẫy tự động submit form ngoài ý muốn.

5.5.6 Minh Họa Sơ Đồ Luồng Xử Lý Submit Form Khi Nhấp Nút



Hình 5.3: Sơ đồ luồng quyết định gửi dữ liệu và xử lý thuộc tính ghi đè (Override Attributes) của nút Submit

5.5.7 Đọc Thêm (Mở Rộng Kiến Thức Chuyên Sâu)

a. Trải Nghiệm Phím Enter Trong Form (Enter Key Form Submit):

Mặc định trên các trình duyệt, khi người dùng đang nhập dữ liệu trong một ô `<input type="text">` và nhấn phím **Enter**, trình duyệt sẽ tự động tìm kiếm nút `type="submit"` đầu tiên trong form và kích hoạt sự kiện Submit.

b. Styling Nút Bấm Đẹp Với CSS Modern (Flexbox, Icon SVG & CSS Pseudo-classes):

Kỹ thuật thiết kế nút bấm chuẩn UI/UX hiện đại với hiệu ứng Hover, Active và Disabled:

Mẫu CSS nút bấm chuẩn UI/UX hiện đại

```css
.btn-primary {
  display: inline-flex;
  align-items: center;
  gap: 8px;
  padding: 10px 20px;
  background-color: #0f4c81;
  color: white;
  border: none;
  border-radius: 6px;
  font-weight: 600;
  cursor: pointer;
  transition: all 0.2s ease-in-out;
}

.btn-primary:hover {
  background-color: #0a355c;
  transform: translateY(-1px);
}

.btn-primary:active {
  transform: translateY(0);
}

.btn-primary:disabled {
  opacity: 0.6;
  cursor: not-allowed;
}
```

BROWSER PREVIEW — **Mô Phỏng Trình Duyệt: Custom Modern CSS Button**



c. Xử Lý Submit Form Trong React (e.preventDefault()):

Trong các ứng dụng Single Page Application (SPA) như React, ta sử dụng `e.preventDefault()` để ngăn trình duyệt reload lại trang khi nhấp nút Submit:

Xử lý sự kiện Submit Form trong React

```
import React from 'react';

function LoginForm() {
  const handleSubmit = (event) => {
    event.preventDefault(); // Ngăn trình duyệt reload lại trang
    console.log('Đã gửi dữ liệu qua API Ajax/Fetch');
  };

  return (
    <form onSubmit={handleSubmit}>
      <input type="text" name="username" />
      {/* Bắt buộc khai báo type="submit" cho nút gửi form */}
      <button type="submit">Đăng nhập</button>
    </form>
  );
}
```

5.5.8 Bộ Câu Hỏi Phỏng Vấn Frontend Thực Chiến (Interview Q&A)

[Ghi Chú] Câu Hỏi Phỏng Vấn 1: Sự Khác Biệt Giữa `<input type="submit">` và `<button type="submit">`

Hỏi: Tại sao trong các dự án ứng dụng web hiện đại, các lập trình viên Frontend thường ưu tiên sử dụng thẻ `<button type="submit">` hơn thẻ `<input type="submit">`?

Trả lời: Bởi vì thẻ `<button>` cho phép lồng các phần tử HTML phức tạp bên trong (như icon SVG, spinner loading, hình ảnh, thẻ `<span>` định dạng chữ), giúp việc tùy biến giao diện (UI/UX) và tạo hiệu ứng linh hoạt hơn rất nhiều so với thẻ `<input>` vốn chỉ nhận chuỗi văn bản thuần qua thuộc tính `value`.

[Ghi Chú] Câu Hỏi Phỏng Vấn 2: Sự Cố Bỏ Quên `type` Trong Thẻ `<button>`

Hỏi: Điều gì xảy ra nếu bạn đặt một thẻ `<button>` xử lý sự kiện nhấp chuột bằng JavaScript trong form nhưng quên khai báo thuộc tính `type="button"`?

Trả lời: Mặc định theo đặc tả HTML5, thẻ `<button>` không có thuộc tính `type` sẽ tự động mặc định là `type="submit"`. Do đó khi người dùng nhấp vào, ngoài việc chạy đoạn mã JavaScript, trình duyệt sẽ tự động kích hoạt gửi form và reload lại trang, gây ra lỗi ngoài ý muốn.

[Ghi Chú] Câu Hỏi Phỏng Vấn 3: Cơ Chế Khôi Phục Dữ Liệu Của Nút Reset Form

Hỏi: Tại sao khi nhấp vào nút `<button type="reset">`, một số ô nhập liệu lại không bị xóa rỗng mà lại hiển thị lại nội dung cũ? Muốn xóa sạch 100% về rỗng thì làm thế nào?

Trả lời: Nút Reset trong HTML5 không có chức năng "xóa sạch về rỗng" mà là "khôi phục về giá trị mặc định khởi tạo ban đầu (**initial HTML value attribute**)". Nếu ô nhập có sẵn `value="abc"`, nút Reset sẽ đưa ô đó về lại chữ `"abc"`. Muốn xóa sạch 100% tất cả các ô về rỗng (bất kể có `value` hay không), lập trình viên phải dùng đoạn mã JavaScript can thiệp đặt `input.value = ''` hoặc lặp qua các phần tử điều khiển của Form.

5.5.9 Bảng Độ Tương Thích Trình Duyệt & Best Practices Checklist

[Bảng] Bảng Độ Tương Thích Trình Duyệt Của Thẻ Nút Bấm

Tính Năng	Chrome / Edge	Firefox	Safari (iOS)	Ghi Chú Hỗ Trợ
`<input type="submit">`	Full (v1+)	Full (v1+)	Full (v1+)	Hỗ trợ 100% trên mọi trình duyệt.
Thẻ `<button>`	Full (v1+)	Full (v1+)	Full (v1+)	Hỗ trợ tốt các phần tử con HTML.
`formaction / form...`	Full (v9+)	Full (v4+)	Full (v5.1+)	Ghi đè thuộc tính form ở từng nút.
`Thuộc tính form="id"`	Full (v10+)	Full (v4+)	Full (v5.1+)	Đặt nút ngoài thẻ form.

[Mẹo] Checklist 5 Quy Tắc Vàng Khi Làm Việc Với Nút Bấm Form

1. **Luôn khai báo thuộc tính `type` rõ ràng**: Luôn viết rõ `type="submit"`, `type="button"` hoặc `type="reset"` cho tất cả các thẻ `<button>`.
2. **Ưu tiên dùng `<button>` cho giao diện hiện đại**: Giúp dễ dàng thêm icon, văn bản đa dạng và làm chủ thiết kế CSS.
3. **Sử dụng thuộc tính `disabled` đúng lúc**: Chuyển nút sang trạng thái `disabled` khi đang gửi dữ liệu (loading state) để tránh người dùng nhấp trùng lặp (Double submit).
4. **Tận dụng thuộc tính `formaction`**: Khi form có nhiều hành động gửi khác nhau (ví dụ: Lưu nháp, Đăng ký, Đăng nhập).
5. **Đảm bảo Accessibility (a11y)**: Nút bấm phải hiển thị rõ trạng thái Focus khi điều hướng bằng phím Tab và có văn bản mô tả mục đích hành động rõ ràng.

5.6 Thẻ Ô Nhập Số (<input type="number">)

[Khái Niệm] Định Nghĩa Thẻ Ô Nhập Số input type="number"

Thẻ `<input type="number">` là phần tử nhập liệu chuyên biệt cho phép người dùng nhập giá trị số (**Integer** hoặc **Float**).

- **Giao diện trình duyệt**: Trình duyệt thường hiển thị thêm hai nút mũi tên tăng/giảm (**Spinner buttons**) ở cạnh phải để người dùng tăng hoặc giảm số nhanh chóng bằng chuột.
- **Bàn phím di động**: Khi truy cập trên điện thoại (iOS, Android), màn hình sẽ tự động bật bàn phím số (**Numeric Keypad**).
- **Giá trị mặc định**: `min` và `max` không giới hạn, `step = 1`, `value` mặc định là rỗng (`""`).

5.6.1 Cú Pháp Cơ Bản & Bảng Thuộc Tính Thường Gặp

Cú pháp khởi tạo ô nhập số đơn giản

```
<input type="number">
```

BROWSER PREVIEW — **Mô Phỏng Trình Duyệt: Ô Nhập Số Cú Pháp Mặc Định**

Ô nhập số mặc định: Nhập số... *(Nút Spinner tăng/giảm ở cạnh phải)*

Dưới đây là bảng tổng hợp đầy đủ các thuộc tính thường dùng cùng thẻ `<input type="number">`:

[Bảng] Bảng Thuộc Tính Thẻ input type="number" Cần Nắm Vững

Thuộc Tính	Ý Nghĩa	Giá Trị Mặc Định
min	Giá trị nhỏ nhất được phép nhập vào form.	Không giới hạn
max	Giá trị lớn nhất được phép nhập vào form.	Không giới hạn
value	Giá trị mặc định hiển thị khi tải trang.	Rỗng ("")
step	Bước nhảy tăng/giảm (ví dụ step="2" chọn số chẵn, step="0.1" nhập số thập phân).	1
name	Tên trường dữ liệu gửi đi khi Submit form.	Không có
required	Bắt buộc người dùng phải nhập số mới cho submit.	false
readonly / disabled	Không cho sửa hoặc vô hiệu hóa hoàn toàn ô nhập.	false

5.6.2 Ví Dụ Minh Họa Chi Tiết

a. Ví Dụ Cơ Bản Giới Hạn Khoảng Giá Trị (1–100):

Ví dụ biểu mẫu yêu cầu người dùng nhập độ tuổi hợp lệ trong khoảng từ 1 đến 100 với giá trị mặc định là 18:

Ví dụ giới hạn độ tuổi với min, max và value

```
<form>
  <label for="userAge">Nhập tuổi (1-100):</label>
  <input type="number" id="userAge" name="age" min="1" max="100" value="18">
  <button type="submit">Gửi Đính Kèm</button>
</form>
```

BROWSER PREVIEW Mô Phỏng Trình Duyệt: Biểu Mẫu Nhập Độ Tuổi Hợp Lệ



b. Ví Dụ Nâng Cao Nhập Số Thập Phân Với Thuộc Tính step:

Mặc định step="1" chỉ cho phép nhập số nguyên. Khi cần nhập số thập phân (như điểm số thang điểm 10.0), ta khai báo step="0.1":

Ví dụ cho phép nhập số thập phân với step="0.1"

```
<label for="score">Điểm số (thang 0.0 - 10.0):</label>
<input type="number" id="score" name="score" min="0" max="10" step="0.1" value=
    "7.5">
```

BROWSER PREVIEW **Mô Phỏng Trình Duyệt: Ô Nhập Điểm Số Thập Phân (step="0.1")**

Điểm số (thang 0.0 - 10.0): 7.5 ← Cho phép chọn 7.5, 7.6...

c. Ví Dụ Custom Giao Diện Nút Tăng/Giảm Số Lượng Chuẩn E-Commerce:

Trong các website bán hàng, người ta thường ẩn nút spinner mặc định và tạo 2 nút tăng/giảm đẹp mắt bằng HTML/CSS & JS:

Ví dụ custom nút tăng/giảm số lượng sản phẩm chuẩn UX E-Commerce

```
<div class="quantity-picker">
  <button type="button" onclick="qty.value--">-</button>
  <input type="number" id="qty" name="quantity" min="1" max="99" value="18">
  <button type="button" onclick="qty.value++">+</button>
</div>
```

BROWSER PREVIEW **Mô Phỏng Trình Duyệt: Custom UI Ô Chọn Số Lượng E-Commerce**

Số lượng đặt mua: − 18 + *(Thiết kế nút tăng/giảm tùy biến chuẩn E-Commerce UI/UX)*

5.6.3 Các Lưu Ý Quan Trọng & Bẫy Thường Gặp

[Cảnh Báo] Cảnh Báo Lỗi Kiểm Lỗi Dữ Liệu (Validation Errors)

- **Vượt quá khoảng min/max**: Nếu người dùng cố tình nhập số nhỏ hơn `min` hoặc lớn hơn `max`, trình duyệt sẽ ngăn Submit form và hiển thị thông báo lỗi HTML5 Validation (trừ khi dùng thuộc tính `novalidate`).
- **Lỗi không chia hết cho step**: Nếu giá trị nhập không chia hết cho bội số của `step` tính từ mốc `min`, form cũng bị báo lỗi không hợp lệ (Ví dụ: `step="2"`, người dùng nhập `value="3"` → Lỗi!).

[Cảnh Báo] Bẫy UX Quan Trọng: Khi Nào KHÔNG NÊN Dùng input type="number"?

- **Số điện thoại, Số thẻ ngân hàng, Mã bưu chính (Zip code)**: KHÔNG NÊN dùng `type="number"`.
- **Lý do**:
  1. Số điện thoại bắt đầu bằng số `0` (ví dụ `0912345678`) khi ép về kiểu số sẽ bị **mất số 0 ở đầu** (thành `912345678`).
  2. Nút mũi tên tăng/giảm (**Spinner**) hoặc thao tác cuộn chuột vô tình làm thay đổi số thẻ tín dụng/số điện thoại của người dùng mà họ không hề hay biết!
- **Giải pháp Senior**: Sử dụng `<input type="tel">` hoặc `<input type="text" inputmode="numeric" pattern="[0-9]*">`.

[Mẹo] Lưu Ý Thực Chiến Khi Dùng Ô Nhập Số

1. Người dùng vẫn có thể gõ các ký tự chữ từ bàn phím trên một số trình duyệt cũ, nhưng khi Submit form, trình duyệt sẽ tự động kích hoạt HTML5 Validation để chặn.
2. Để cho phép nhập bất kỳ số thập phân nào mà không bị gò bó bởi bội số, bạn có thể gán `step="any"`.
3. Nếu cần kiểm soát logic số phức tạp hơn (ví dụ tính tổng tiền real-time), hãy luôn kết hợp kiểm tra bằng mã JavaScript.

5.6.4 Bảng Tổng Kết & Ghi Nhớ Cốt Lõi

[Bảng] Tóm Tắt Kỹ Thuật Ô Nhập Số

Thuộc Tính / Kỹ Thuật	Mô Tả	Ghi Chú Thực Hành
`min / max`	Giới hạn vùng giá trị hợp lệ.	Tránh người dùng nhập số âm hoặc số quá lớn.
`step`	Quy định khoảng tăng giảm	Dùng `0.1` cho số thập phân, `any` cho tự do.
`value`	Giá trị khởi tạo ban đầu.	Nên khai báo sẵn nếu có giá trị gợi ý cho người dùng.

5.6.5 Bộ Câu Hỏi Phỏng Vấn Frontend Thực Chiến (Interview Q&A)

[Ghi Chú] Câu Hỏi Phỏng Vấn 1: Nhập Số Thập Phân Trong input type="number"

Hỏi: Mặc định ô `<input type="number">` báo lỗi khi nhập số có dấu phẩy thập phân (như 3.14). Làm sao để cho phép nhập số thập phân?

Trả lời: Do mặc định thuộc tính `step` nhận giá trị là `1` (chỉ cho nhập số nguyên). Để nhập số thập phân, ta khai báo thuộc tính `step="0.1"` (hoặc số lẻ nhỏ hơn tùy độ chính xác) hoặc dùng `step="any"` để chấp nhận mọi số thực.

[Ghi Chú] Câu Hỏi Phỏng Vấn 2: Ẩn Nút Tăng Giảm Spinner Bằng CSS

Hỏi: Làm thế nào để ẩn hai nút mũi tên tăng/giảm (**Spinner buttons**) mặc định của trình duyệt trên ô nhập số?

Trả lời: Ta sử dụng thuộc tính `-webkit-appearance: none` cho các trình duyệt WebKit (Chrome, Safari, Edge) và `-moz-appearance: textfield` cho Firefox để loại bỏ hoàn toàn nút spinner:

Đoạn mã CSS ẩn nút spinner tăng/giảm trên mọi trình duyệt

```
/* Ẩn nút spinner trên Chrome, Safari, Edge */
input[type="number"]::-webkit-outer-spin-button,
input[type="number"]::-webkit-inner-spin-button {
  -webkit-appearance: none;
  margin: 0;
}

/* Ẩn nút spinner trên Firefox */
input[type="number"] {
  -moz-appearance: textfield;
}
```

5.6.6 Đọc Thêm (Mở Rộng Kiến Thức Chuyên Sâu)

a. Thuộc Tính inputmode=”numeric” vs pattern=”[0-9]*”:

Khi thiết kế biểu mẫu nhập số điện thoại hoặc số tài khoản ngân hàng, thay vì dùng `type="number"` (vốn bị lỗi mất số `0` ở đầu), lập trình viên Frontend chuẩn Senior sẽ kết hợp:

Tối ưu bàn phím di động chuẩn UX không bị mất số 0

```
<input type="text" inputmode="numeric" pattern="[0-9]*" placeholder="Nhập số tài
    khoản...">
```

b. Chặn Sự Kiện Wheel (Cuộn Chuột) Làm Đổi Giá Trị Ngoài Ý Muốn:

Mặc định khi người dùng di chuột qua ô `<input type="number">` và cuộn bánh xe chuột (**Mouse wheel**), trình duyệt sẽ tự động tăng hoặc giảm số. Để tránh người dùng bị nhảy số vô tình khi lướt trang, ta dùng đoạn mã JavaScript:

Chặn cuộn chuột đổi số trên input type=”number”

```
document.querySelectorAll('input[type="number"]').forEach(input => {
  input.addEventListener('wheel', (e) => e.preventDefault());
});
```

5.7 Thẻ Ô Chọn Khoảng Giá Trị (<input type="range">)

[Khái Niệm] Định Nghĩa Thẻ Range Input (Thanh Trượt Slider)

Thẻ `<input type="range">` là phần tử biểu mẫu hiển thị dưới dạng một thanh trượt (**Slider**), cho phép người dùng lựa chọn một giá trị số nằm trong một khoảng xác định (**Range**).

- **Cơ chế hiển thị**: Trình duyệt hiển thị một thanh ngang cùng một nút kéo (**Thumb**) có thể di chuyển bằng chuột hoặc thao tác vuốt cảm ứng.
- **Bản chất dữ liệu**: Giá trị chọn luôn là một chuỗi số.
- **Giá trị mặc định chuẩn HTML5**:
  - `min = 0`
  - `max = 100`
  - `value = 50`
- **Công thức tự động tính value mặc định**: Nếu khai báo `min` và `max` riêng biệt (ví dụ `min="20"` và `max="200"`) mà không khai báo thuộc tính `value`, trình duyệt sẽ tự động tính giá trị mặc định theo công thức trung bình cộng:

$$\text{value}_{\text{mặc định}} = \frac{\text{min} + \text{max}}{2} = \frac{20 + 200}{2} = 110$$

5.7.1 Cú Pháp Cơ Bản & Bảng Thuộc Tính Thường Gặp

Cú pháp khởi tạo thanh trượt Slider đơn giản

```
<input type="range">
```

BROWSER PREVIEW **Mô Phỏng Trình Duyệt: Thanh Trượt Slider Mặc Định (Vị trí 50%)**

Thanh trượt mặc định (0–100): *(Vị trí mặc định: 50)*

Dưới đây là bảng tổng hợp các thuộc tính thường dùng cho thanh trượt `<input type="range">`:

[Bảng] Bảng Thuộc Tính Thẻ input type="range" Cần Nắm Vững

Thuộc Tính	Ý Nghĩa	Giá Trị Mặc Định
`min`	Giá trị nhỏ nhất ở đầu thanh trượt bên trái.	`0`
`max`	Giá trị lớn nhất ở đầu thanh trượt bên phải.	`100`
`value`	Giá trị khởi tạo của con trỏ Slider.	`50` hoặc $\frac{min+max}{2}$
`step`	Bước nhảy khoảng cách giữa các điểm dừng hợp lệ khi kéo.	`1`
`name`	Tên tham số dữ liệu gửi lên máy chủ khi submit form.	`Không có`

5.7.2 Ví Dụ Minh Họa Chi Tiết

a. Ví Dụ Biểu Mẫu Điều Chỉnh Âm Lượng (0–100):

Ví dụ tạo thanh trượt điều chỉnh âm lượng bắt đầu ở mức 30:

Ví dụ thanh trượt điều chỉnh âm lượng mức 30

```
<form>
  <label for="vol">Âm lượng (0-100):</label>
  <input type="range" id="vol" name="volume" min="0" max="100" value="30">
</form>
```

BROWSER PREVIEW Mô Phỏng Trình Duyệt: Thanh Trượt Âm Lượng Bắt Đầu Ở Mức 30

Âm lượng (0-100): 30

b. Ví Dụ Nâng Cao Hiển Thị Giá Trị Thời Gian Thực Với Thẻ <output>:

Mặc định thanh trượt Slider không tự hiển thị con số bên cạnh. Ta kết hợp sự kiện JavaScript `oninput` và thẻ `<output>` để hiển thị giá trị thời gian thực khi người dùng kéo con trỏ:

Ví dụ hiển thị giá trị kéo Slider thời gian thực với thẻ output

```
<label for="brightness">Độ sáng:</label>
<input type="range" id="brightness" name="brightness" min="0" max="200" step="10"
      value="100"
        oninput="output.value = this.value">
<output id="output">100</output>
```

BROWSER PREVIEW **Mô Phỏng Trình Duyệt: Thanh Trượt Độ Sáng Cập Nhật Con Số Real-time**

Độ sáng: 100

c. Kỹ Thuật Nâng Cao: Tạo Các Nấc Điểm Dừng (Tick Marks) Với Thẻ <datalist>:

Chuẩn HTML5 cho phép kết nối thanh trượt Slider với thẻ `<datalist>` để hiển thị các vạch nấc dừng (**Tick marks / Snap points**) trực quan:

Ví dụ tạo các nấc điểm dừng trên Slider với datalist

```
<label for="temp">Nhiệt độ phòng (°C):</label>
<input type="range" id="temp" name="temp" min="0" max="40" step="10" list="ticks"
    >

<!-- Khai báo danh sách các vạch mốc nấc dừng -->
<datalist id="ticks">
  <option value="0" label="0 deg C"></option>
  <option value="10"></option>
  <option value="20" label="20 deg C"></option>
  <option value="30"></option>
  <option value="40" label="40 deg C"></option>
</datalist>
```

BROWSER PREVIEW **Mô Phỏng Trình Duyệt: Slider Có Các Vạch Nấc Dừng (Tick Marks)**

Nhiệt độ (°C): *(Các vạch mốc: 0, 10, 20, 30, 40)*

5.7.3 Các Lưu Ý Quan Trọng & Bẫy Thường Gặp

[Mẹo] 4 Quy Tắc Cốt Lõi Cần Nhớ Về Thẻ Range

1. **Tự động sửa giá trị (Clamping)**: Nếu gán thuộc tính `value` nhỏ hơn `min` hoặc lớn hơn `max`, trình duyệt sẽ tự động điều chỉnh con trỏ về mốc giới hạn gần nhất.
2. **Tác dụng của step**: Thuộc tính `step` giúp hạn chế các mốc số dư (ví dụ `step="10"` chỉ cho phép dừng tại 0, 10, 20, 30...).
3. **Dữ liệu gửi trong Form**: Khi Submit form, trình duyệt chỉ gửi **giá trị số duy nhất** (ví dụ `brightness=100`), không gửi tọa độ con trỏ hay tỷ lệ phần trăm.
4. **Trải nghiệm người dùng (UX)**: Nên luôn đi kèm thẻ `<output>` hoặc văn bản hiển thị con số để người dùng biết chính xác giá trị họ đang chọn.

5.7.4 Bảng Tổng Kết & Ghi Nhớ Cốt Lõi

[Bảng] Tóm Tắt Thuộc Tính & Tính Toán Mặc Định Thẻ Range

Trường Hợp	Giá Trị Mặc Định	Ví Dụ Minh Họa
`Không khai báo gì`	`min=0`, `max=100`, `value=50`	`<input type="range">` → Vị trí 50.
`Khai báo min, max`	`value` = `(min + max) / 2`	`min="20" max="200"` → `value = 110`.

5.7.5 Bộ Câu Hỏi Phỏng Vấn Frontend Thực Chiến (Interview Q&A)

[Ghi Chú] Câu Hỏi Phỏng Vấn 1: Cập Nhật Con Số Real-time Khi Kéo Slider

Hỏi: Làm sao để hiển thị con số thay đổi liên tục theo thời gian thực khi người dùng đang kéo thanh trượt Slider?

Trả lời: Ta lắng nghe sự kiện `oninput` (hoặc sự kiện `input` trong JavaScript) trên thẻ range và gán giá trị `this.value` vào một thẻ `<output>` hoặc phần tử hiển thị văn bản. Không nên dùng sự kiện `change` vì `change` chỉ nhả ra kết quả khi người dùng đã thả tay khỏi chuột.

[Ghi Chú] Câu Hỏi Phỏng Vấn 2: Tùy Biến Giao Diện Thanh Trượt Bằng CSS

Hỏi: Giao diện thanh trượt Slider mặc định trên các hệ điều hành trông rất thô. Làm sao để custom màu sắc và nút kéo bằng CSS?

Trả lời: Ta sử dụng thuộc tính `appearance: none` để loại bỏ giao diện mặc định, sau đó tùy biến đường rãnh (**Track**) và nút kéo (**Thumb**) thông qua các Pseudo-elements của từng trình duyệt:

- Chrome / Safari / Edge: `::-webkit-slider-runnable-track` và `::-webkit-slider-thumb`
- Firefox: `::-moz-range-track` và `::-moz-range-thumb`

5.7.6 Đọc Thêm (Mở Rộng Kiến Thức Chuyên Sâu)

a. Thiết Lập Thanh Trượt Dọc (Vertical Range Slider):

Mặc định thanh trượt Slider hiển thị theo chiều ngang. Khi thiết kế equalizer âm thanh hoặc cột đo chiều cao, ta có thể xoay thanh trượt thành chiều dọc bằng thuộc tính CSS `writing-mode`:

Thiết lập thanh trượt Slider xoay dọc

```
<input type="range" id="volVertical" min="0" max="100" value="70"
       style="writing-mode: vertical-lr; direction: rtl; width: 16px; height: 120
    px;">
```

b. Kỹ Thuật Đổi Màu Tiến Trình Kéo Slider (Dynamic Gradient Range Fill):

Để làm nổi bật phần thanh trượt đã kéo qua (màu xanh brand) khác với phần chưa kéo (màu xám nhạt), ta lắng nghe sự kiện `input` trong JavaScript để cập nhật dải dốc màu nền `linear-gradient` tương ứng với tỷ lệ phần trăm:

Custom hiệu ứng màu tiến trình kéo Slider như Spotify

```
const rangeInput = document.querySelector('input[type="range"]');
rangeInput.addEventListener('input', function() {
  const percent = this.value - this.min) / (this.max - this.min * 100;
  // Cập nhật nền gradient động theo tỷ lệ phần trăm
  this.style.background = `linear-gradient(to right, #0056b3 ${percent}%, #e0e0e0
     ${percent}%)`;
});
```

5.8 Thẻ Ô Nhập Văn Bản Nhiều Dòng Textarea (<textarea>)

[Khái Niệm] Định Nghĩa Thẻ Textarea

Textarea là thẻ HTML chuyên dụng dùng để tạo một ô nhập văn bản nhiều dòng (**Multi-line text input field**). Khác với thẻ `<input type="text">` vốn chỉ cho phép nhập văn bản trên một dòng đơn lẻ, `<textarea>` cho phép người dùng xuống dòng và nhập các đoạn văn bản dài.

- **Ứng dụng phổ biến**: Nhập ý kiến phản hồi (**Feedback**), bình luận bài viết (**Comment**), mô tả sản phẩm (**Description**), ghi chú, hoặc nhập địa chỉ giao hàng (**Address**).
- **Khi nào nên dùng**: Sử dụng khi cần thu thập dữ liệu dạng văn bản dài hoặc dữ liệu có cấu trúc nhiều dòng thay vì chuỗi ký tự ngắn.

5.8.1 Cú Pháp Cơ Bản & Cấu Trúc Khai Báo

Cú pháp cơ bản tạo ô nhập liệu Textarea

```
<label for="msg">Tin nhan:</label>
<textarea id="msg" name="message"></textarea>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Textarea Cú Pháp Cơ Bản**

Tin nhắn:

(Người dùng có thể gõ nhiều dòng văn bản vào đây...)

[Mẹo] Giải Thích Thành Phần Cấu Trúc

- `name="message"`: Tên trường dữ liệu đóng gói gửi về máy chủ khi submit form. Nếu không khai báo `name`, dữ liệu trong ô sẽ không được gửi đi.
- `id="msg"`: Định danh duy nhất kết nối logic với thuộc tính `for="msg"` của thẻ `<label>`.

5.8.2 Các Thuộc Tính Quan Trọng Của Thẻ <textarea>

Dưới đây là bảng tổng hợp chi tiết tất cả các thuộc tính quan trọng thường được sử dụng cùng thẻ `<textarea>`:

[Bảng] Bảng Tổng Hợp Thuộc Tính Của Thẻ Textarea

Thuộc Tính	Ý Nghĩa & Chức Năng	Giá Trị Mặc Định	Ví Dụ Cú Pháp
`name`	Tên trường dữ liệu gửi lên máy chủ khi submit form.	Không có	`name="comment"`
`id`	Định danh duy nhất để liên kết với thẻ `<label>`.	Không có	`id="commentMsg"`
`rows`	Số lượng hàng (chiều cao) hiển thị mặc định của ô nhập.	2 dòng	`rows="5"`
`cols`	Số lượng cột (chiều rộng ký tự) hiển thị mặc định trên mỗi dòng.	20 ký tự	`cols="40"`
`placeholder`	Đoạn văn bản gợi ý nội dung ẩn hiển thị khi ô nhập rỗng.	Không có	`placeholder="Nhập nội dung..."`
`maxlength`	Giới hạn số lượng ký tự tối đa cho phép nhập vào ô.	Không giới hạn	`maxlength="200"`
`readonly`	Thiết lập ô chỉ đọc, người dùng không thể chỉnh sửa nội dung.	false	`readonly`
`disabled`	Vô hiệu hóa ô nhập, không thể tương tác và không gửi dữ liệu.	false	`disabled`
`required`	Bắt buộc người dùng phải điền dữ liệu trước khi submit form.	false	`required`
`wrap`	Quy định cơ chế xử lý xuống dòng khi gửi form (`soft`, `hard`, `off`).	soft	`wrap="soft"`

5.8.3 Các Giá Trị Mặc Định Của <textarea> Trong HTML

Khi không chỉ định bất kỳ thuộc tính nào, trình duyệt web sẽ áp dụng bộ giá trị mặc định tiêu chuẩn sau cho thẻ `<textarea>`:

[Bảng] Bảng Chi Tiết Giá Trị Mặc Định Của Thẻ Textarea

Thuộc Tính	Giá Trị Mặc Định	Giải Thích Hành Vi Trình Duyệt
Nội dung bên trong	Rỗng ("")	Thẻ <textarea></textarea> không chứa bất kỳ văn bản nào.
rows	2	Trình duyệt mặc định hiển thị chiều cao tương đương 2 dòng văn bản.
cols	20	Trình duyệt mặc định hiển thị chiều rộng tương đương 20 ký tự trên mỗi dòng.
wrap	soft	Văn bản tự động hiển thị xuống dòng trên giao diện, nhưng khi gửi về server chỉ có ký tự xuống dòng do người dùng nhấn Enter thực sự mới được đính kèm.
spellcheck	true	Trình duyệt tự động bật tính năng kiểm tra chính tả (nếu trình duyệt có hỗ trợ ngôn ngữ).
resize	both (CSS)	Cho phép người dùng kéo thả chuột để thay đổi kích thước ô nhập theo cả chiều ngang và chiều dọc.
maxlength	Không giới hạn	Người dùng có thể nhập văn bản với độ dài tùy ý.
placeholder	Không có	Không hiển thị văn bản mờ gợi ý khi ô nhập trống.
readonly	false	Người dùng có thể thoải mái nhập và chỉnh sửa nội dung.
disabled	false	Ô nhập hoạt động bình thường và sẵn sàng thu nhận tương tác.
required	false	Biểu mẫu không bắt buộc phải điền ô này mới cho phép submit.
autofocus	false	Không tự động đặt con trỏ chuột vào ô nhập khi tải xong trang web.

5.8.4 So Sánh Textarea Mặc Định vs Textarea Cấu Hình Thuộc Tính

Để thấy rõ sự khác biệt giữa cài đặt mặc định và khi khai báo thuộc tính, hãy xem ví dụ so sánh dưới đây:

Mã nguồn so sánh Textarea mặc định và có thuộc tính

```
<!-- 1. Textarea mac dinh -->
<label>Textarea mac dinh:</label>
<textarea></textarea>

<!-- 2. Textarea co thuoc tinh -->
<label>Textarea co thuoc tinh:</label>
<textarea rows="5" cols="40" placeholder="Nhập ghi chú tại đây..."></textarea>
```

BROWSER PREVIEW Mô Phỏng Trình Duyệt: So Sánh Mặc Định vs Có Thuộc Tính

1. Textarea mặc định (2 dòng × 20 ký tự):

2. Textarea có thuộc tính (5 dòng × 40 ký tự + Placeholder):

Nhập ghi chú tại đây...

5.8.5 Các Ví Dụ Minh Họa Chi Tiết Thực Tế

Ví Dụ 1: Ô nhập phản hồi có giới hạn số dòng, cột và tối đa ký tự:

Form nhập phản hồi giới hạn 4 dòng, 50 cột và tối đa 200 ký tự

```
<textarea name="feedback" rows="4" cols="50" maxlength="200"
          placeholder="Nhập phản hồi của bạn..."></textarea>
```

BROWSER PREVIEW Kết Quả Trình Duyệt: Ô Nhập Phản Hồi Giới Hạn 200 Ký Tự

Nhập phản hồi của bạn...

→ *Hiển thị ô nhập rộng 4 dòng × 50 ký tự mỗi dòng, giới hạn tối đa 200 ký tự.*

Ví Dụ 2: Ô nhập chứa văn bản giá trị mặc định sẵn có:

Form với nội dung mặc định đặt giữa thẻ mở và đóng

```
<textarea name="bio" rows="3" cols="40">Tôi là sinh viên CNTT.</textarea>
```

BROWSER PREVIEW — **Kết Quả Trình Duyệt: Textarea Có Giá Trị Mặc Định**

Tôi là sinh viên CNTT.

→ *Khi tải trang, văn bản "Tôi là sinh viên CNTT." đã xuất hiện sẵn trong ô nhập.*

Ví Dụ 3: Ô nhập trạng thái chỉ đọc (readonly):

Form cấu hình thuộc tính readonly không cho sửa nội dung

```
<textarea name="info" rows="3" cols="40" readonly>Thông tin chỉ đọc, không thể
    sửa...</textarea>
```

BROWSER PREVIEW — **Kết Quả Trình Duyệt: Textarea Trạng Thái Readonly**

Thông tin chỉ đọc, không thể sửa...

→ *Người dùng có thể bôi đen copy nhưng hoàn toàn KHÔNG THỂ chỉnh sửa hay xóa văn bản.*

Ví Dụ 4: Ô nhập trạng thái bị vô hiệu hóa (disabled):

Form cấu hình thuộc tính disabled vô hiệu hóa tương tác

```
<textarea name="note" rows="3" cols="40" disabled>Không thể nhập dữ liệu vào
    đây...</textarea>
```

BROWSER PREVIEW **Kết Quả Trình Duyệt: Textarea Trạng Thái Disabled**

Không thể nhập dữ liệu vào đây...

→ *Ô nhập bị mờ đi, không thể tương tác và dữ liệu sẽ KHÔNG ĐƯỢC gửi đi khi submit form.*

Ví Dụ 5: Ô nhập bắt buộc điền dữ liệu (required):

Form bắt buộc nhập bình luận trước khi submit

```
<textarea name="comment" rows="4" cols="40" required
          placeholder="Vui lòng nhập bình luận..."></textarea>
```

5.8.6 Các Lưu Ý Quan Trọng Cần Ghi Nhớ Nằm Lòng

[Cảnh Báo] Các Lưu Ý Bắt Buộc Phải Thuộc Khi Làm Việc Với Thẻ Textarea

1. **CỰC KỲ QUAN TRỌNG: Không dùng thuộc tính `value="..."`**: Không giống như các thẻ `<input>`, thẻ `<textarea>` **HOÀN TOÀN KHÔNG HỖ TRỢ thuộc tính `value`**. Muốn đặt giá trị văn bản mặc định khi tải trang, bạn **bắt buộc phải đặt văn bản nằm giữa cặp thẻ mở và thẻ đóng**:

```
<textarea>Văn bản giá trị mặc định</textarea>
```

   Cảnh báo lỗi thường gặp: Cú pháp `<textarea value="abc">` là **SAI HOÀN TOÀN** và sẽ bị trình duyệt bỏ qua!

2. **Cấu hình kích thước với CSS (`resize`)**: Mặc định, trình duyệt cho phép người dùng kéo chuột ở góc dưới ô để thay đổi kích thước (`resize: both`). Trong thiết kế thực tế, để tránh làm phá vỡ giao diện web (**Layout Break**), lập trình viên thường dùng CSS vô hiệu hóa tính năng này hoặc chỉ cho phép kéo theo chiều dọc:

```
/* Không cho phép người dùng thay đổi kích thước */
textarea { resize: none; }

/* Chỉ cho phép thay đổi kích thước theo chiều dọc */
textarea { resize: vertical; }
```

3. **Tự động xuất hiện thanh cuộn (`overflow`)**: Khi nội dung văn bản do người dùng nhập vào vượt quá số dòng (`rows`) hoặc số cột (`cols`) hiển thị, trình duyệt sẽ tự động thêm thanh cuộn đứng (**Vertical Scrollbar**) để cho phép cuộn xem toàn bộ văn bản.

4. **Bắt buộc phải có thuộc tính `name`**: Nếu bạn muốn thu thập và gửi dữ liệu văn bản trong `<textarea>` về máy chủ Backend, thẻ `<textarea>` bắt buộc phải có thuộc tính `name`. Nếu thiếu `name`, dữ liệu sẽ bị bỏ qua khi submit form.

5.8.7 Đọc Thêm: Kỹ Thuật Nâng Cao & Tối Ưu Trải Nghiệm Lập Trình Textarea

[Mẹo] Tự Động Giãn Chiều Cao Textarea Theo Nội Dung Nhập (Auto-expand)

Trong các ứng dụng nhắn tin hoặc mạng xã hội hiện đại (như Facebook, Messenger, Zalo), ô nhập văn bản thường tự động cao lên khi người dùng gõ xuống dòng mà không xuất hiện thanh cuộn xấu xí. Ta có thể đạt được hiệu ứng này bằng 1 dòng JavaScript đơn giản:

Mã JavaScript tự động thay đổi chiều cao Textarea theo scrollHeight

```
const textarea = document.querySelector('textarea');
textarea.addEventListener('input', function() {
  this.style.height = 'auto'; // Reset chieu cao ban dau
  this.style.height = (this.scrollHeight) + 'px'; // Cap nhat chieu cao
    theo noi dung
});
```

[Ghi Chú] Kỹ Thuật Đếm Số Ký Tự Còn Lại (Real-time Character Counter)

Kết hợp thuộc tính `maxlength="200"` với JavaScript để hiển thị bộ đếm số ký tự thời gian thực giúp người dùng biết mình còn bao nhiêu ký tự được phép gõ:

Mã nguồn đếm ký tự thời gian thực cho Textarea

```
<textarea id="myText" maxlength="200" placeholder="Nhap noi dung..."><
    /textarea>
<p>So ky tu con lai: <span id="charCount">200</span>/200</p>

<script>
const myText = document.getElementById('myText');
const charCount = document.getElementById('charCount');

myText.addEventListener('input', function() {
  const remaining = 200 - this.value.length;
  charCount.textContent = remaining;
});
</script>
```

5.9 Thẻ Danh Sách Thả Xuống Select (<select>, <option> & <optgroup>)

[Khái Niệm] Định Nghĩa Thẻ Select & Option

Thẻ `<select>` trong HTML được sử dụng để tạo một **danh sách thả xuống** (**Dropdown List**) hoặc một danh sách chọn (**List Box**), cho phép người dùng lựa chọn một hoặc nhiều tùy chọn từ danh sách các mục có sẵn.

- **Thẻ `<option>`**: Được đặt bên trong thẻ `<select>` để định nghĩa từng mục lựa chọn cụ thể trong danh sách.
- **Thẻ `<optgroup>`**: Dùng để phân nhóm các tùy chọn có cùng chủ đề liên quan, giúp giao diện danh sách ngắn gọn và dễ tìm kiếm hơn.

Lợi Ích Cốt Lõi Khi Sử Dụng Thẻ Select:

1. **Tiết kiệm không gian giao diện**: Danh sách ẩn đi và chỉ mở ra khi người dùng kích chuột vào.
2. **Tránh nhập sai dữ liệu**: Ép người dùng chỉ được phép chọn từ danh sách chuẩn cố định (ví dụ: Chọn Tỉnh/Thành phố, Quốc gia, Giới tính).
3. **Dễ quản lý và mở rộng**: Dễ dàng bổ sung hoặc chỉnh sửa danh sách các tùy chọn trên Frontend hoặc lấy từ Cơ sở dữ liệu Backend.

5.9.1 Cú Pháp Cơ Bản & Cấu Trúc Khai Báo

Cú pháp cơ bản tạo danh sách thả xuống Select

```
<label for="country">Chon quoc gia:</label>
<select id="country" name="country">
  <option value="vn">Việt Nam</option>
  <option value="us">Hoa Kỳ</option>
  <option value="jp">Nhật Bản</option>
</select>
```

BROWSER PREVIEW **Mô Phỏng Trình Duyệt: Danh Sách Thả Xuống Select Cơ Bản**

Chọn quốc gia:

Việt Nam ∨

5.9.2 Các Thuộc Tính Quan Trọng Của Thẻ <select>

Dưới đây là bảng tổng hợp chi tiết tất cả các thuộc tính quan trọng của thẻ `<select>`:

[Bảng] Bảng Tổng Hợp Thuộc Tính Của Thẻ Select

Thuộc Tính	Ý Nghĩa & Chức Năng	Giá Trị Mặc Định	Ví Dụ Cú Pháp
`name`	Tên của trường dữ liệu để đóng gói gửi về máy chủ Backend.	Không có	`name="country"`
`id`	Định danh duy nhất để liên kết với thuộc tính `for` của thẻ `<label>`.	Không có	`id="countrySelect"`
`multiple`	Cho phép người dùng chọn nhiều tùy chọn cùng lúc (dùng Ctrl/Shift hoặc Cmd).	`false`	`multiple`
`size`	Số lượng tùy chọn hiển thị trực tiếp cùng lúc (chuyển sang dạng hộp danh sách Listbox).	`1` (hiển thị dạng Dropdown)	`size="3"`
`disabled`	Vô hiệu hóa toàn bộ danh sách select, người dùng không thể tương tác.	`false`	`disabled`
`required`	Bắt buộc người dùng phải chọn một tùy chọn trước khi submit form.	`false`	`required`
`autofocus`	Tự động đặt con trỏ tiêu điểm vào danh sách khi trang vừa tải xong.	`false`	`autofocus`

5.9.3 Các Thuộc Tính Quan Trọng Của Thẻ <option> Bên Trong

Mỗi thẻ `<option>` đại diện cho một mục lựa chọn và sở hữu các thuộc tính sau:

[Bảng] Bảng Thuộc Tính Của Thẻ Option

Thuộc Tính	Ý Nghĩa & Chức Năng	Giá Trị Mặc Định	Ví Dụ Cú Pháp
`value`	Giá trị thực tế được đóng gói gửi về máy chủ khi tùy chọn đó được người dùng chọn.	Nếu không có, lấy văn bản hiển thị làm value	`value="vn"`
`selected`	Đặt tùy chọn này làm mặc định được chọn sẵn khi trang vừa tải xong.	Tùy chọn đầu tiên	`selected`
`disabled`	Làm mờ tùy chọn này, khiến người dùng không thể bấm chọn.	`false`	`disabled`
`label`	Nhãn văn bản ngắn gọn hiển thị thay thế nội dung văn bản bên trong thẻ `<option>`.	Nội dung bên trong thẻ	`label="VN"`

5.9.4 Cơ Chế Hoạt Động Của name và value Khi Submit Form

Sự kết hợp giữa thuộc tính `name` trên thẻ `<select>` và thuộc tính `value` trên các thẻ `<option>` quyết định chính xác dữ liệu nhận được trên máy chủ Backend:

[Khái Niệm] Cơ Chế Đóng Gói Dữ Liệu Form (Name & Value)

- `name`: Định danh tên trường dữ liệu. Giúp máy chủ phân biệt dữ liệu gửi về thuộc về ô nhập nào (ví dụ: `fruit`, `country`, `color`).
- `value`: Giá trị dữ liệu thực tế được gửi đi khi một tùy chọn được chọn.

Form chọn trái cây gửi dữ liệu bằng phương thức GET

```
<form action="submit.php" method="GET">
  <label for="fruit">Chon trai cay:</label>
  <select name="fruit" id="fruit">
    <option value="apple">Táo</option>
    <option value="banana">Chuối</option>
    <option value="mango">Xoài</option>
  </select>
  <button type="submit">Gửi</button>
</form>
```

[Mẹo] Phân Tích URL Chuỗi Truy Vấn (Query String)

Kịch bản: Người dùng chọn mục **"Chuối"** (`value="banana"`) và nhấn nút **Gửi**.

Trình duyệt sẽ chuyển hướng tới địa chỉ URL:

```
submit.php?fruit=banana
```

Trong đó: `fruit` là giá trị thuộc tính `name` của thẻ `<select>`, và `banana` là giá trị thuộc tính `value` của thẻ `<option>` được chọn.

Trường Hợp Có value vs Không Có value:

1. **Nếu thẻ có thuộc tính `value`:**

Ví dụ tùy chọn khai báo value rõ ràng

```
<select name="color">
  <option value="red">Đỏ</option>
  <option value="blue">Xanh</option>
  <option value="green">Lục</option>
</select>
```

BROWSER PREVIEW Kết Quả Hiển Thị Trình Duyệt: Tùy Chọn Khai Báo Value Rõ Ràng

Chọn màu sắc:

Xanh ∨

→ *Khi người dùng chọn "Xanh", dữ liệu gửi về server là value="blue".*

2. **Nếu thẻ KHÔNG KHAI BÁO thuộc tính `value`:**

Ví dụ tùy chọn không có thuộc tính value

```
<select name="color">
  <option>Đỏ</option>
  <option>Xanh</option>
  <option>Lục</option>
</select>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Tùy Chọn Không Khai Báo Value**

Chọn màu sắc:

Xanh ∨

→ *Khi người dùng chọn "Xanh", trình duyệt tự lấy chữ "Xanh" làm value để gửi về server.*

5.9.5 Thiết Lập Tùy Chọn Mặc Định & Tùy Chọn Trống (Placeholder Option)

1. Tùy chọn mặc định tự động của trình duyệt:

Theo mặc định, nếu không có bất kỳ thuộc tính `selected` nào được chỉ định, trình duyệt sẽ tự động chọn **mục tùy chọn đầu tiên** trong danh sách:

Mục đầu tiên (Táo) tự động được chọn mặc định

```
<select name="fruit">
  <option value="apple">Táo</option>  <!-- Muc nay se duoc chon mac dinh -->
  <option value="banana">Chuối</option>
  <option value="mango">Xoài</option>
</select>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Mục Đầu Tiên Tự Động Chọn Mặc Định**

Chọn trái cây:

Táo ∨

→ *Mặc định khi nạp trang, mục đầu tiên ("Táo") tự động hiển thị.*

2. Đặt một tùy chọn khác làm mặc định với thuộc tính `selected`:

Nếu muốn chọn một mục nằm ở vị trí khác làm mặc định khi tải trang, hãy thêm thuộc tính `selected` vào thẻ `<option>` đó:

Đặt mục Chuối làm tùy chọn mặc định sẵn

```
<select name="fruit">
  <option value="apple">Táo</option>
  <option value="banana" selected>Chuối</option>  <!-- Duoc chon mac dinh -->
  <option value="mango">Xoài</option>
</select>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Chọn Sẵn Mục Chuối Bằng Thuộc Tính Selected**

Chọn trái cây:

Chuối ∨

→ Nhờ thuộc tính selected, mục "Chuối" hiển thị sẵn ngay khi trang vừa nạp xong.

3. Kỹ thuật tạo dòng lựa chọn hướng dẫn (Placeholder Option) ép người dùng phải chọn:

Trong thực tế phát triển phần mềm, ta thường tạo một mục đầu tiên rỗng kết hợp với thuộc tính `required` để buộc người dùng phải chủ động thao tác chọn một giá trị hợp lệ trước khi gửi form:

Kỹ thuật Placeholder Option bắt buộc chọn

```
<select name="fruit" required>
  <option value="" selected disabled>-- Chon trai cay --</option>
  <option value="apple">Táo</option>
  <option value="banana">Chuối</option>
  <option value="mango">Xoài</option>
</select>
```

BROWSER PREVIEW **Kết Quả Trình Duyệt: Tùy Chọn Hướng Dẫn Ban Đầu**

Chọn trái cây:

– Chọn trái cây – ∨

→ Dòng "– Chọn trái cây –" hiển thị ban đầu, bị mờ (disabled) và có value="" khiến form bắt buộc chọn mục khác.

5.9.6 Các Ví Dụ Minh Họa Chi Tiết Thực Tế

Ví Dụ 1: Select cho phép chọn nhiều giá trị cùng lúc (Multiple Select):

Mã HTML Select hỗ trợ chọn nhiều mục bằng thuộc tính multiple

```
<label for="pets">Chon thu cung yeu thich:</label>
<select id="pets" name="pets[]" multiple size="3">
  <option value="dog">Chó</option>
  <option value="cat">Mèo</option>
  <option value="bird">Chim</option>
  <option value="fish">Cá</option>
</select>
```

BROWSER PREVIEW **Kết Quả Trình Duyệt: Hộp Chọn Nhiều Mục (Multiple Selection Listbox)**

Chọn thú cưng yêu thích (Giữ Ctrl / Cmd để chọn nhiều):



→ *Người dùng giữ phím Ctrl (Windows) hoặc Cmd (Mac) để bấm chọn cùng lúc Chó và Mèo.*

5.9.7 Chi Tiết Về Thẻ Phân Nhóm Tùy Chọn <optgroup>

[Khái Niệm] Định Nghĩa Thẻ <optgroup>

Thẻ <optgroup> (**Option Group**) trong HTML được sử dụng để **gom nhóm các tùy chọn** (<option>) có cùng chủ đề lại với nhau bên trong một thẻ <select>.

Mục Đích: Giúp danh sách thả xuống có cấu trúc phân tầng rõ ràng, mạch lạc, dễ đọc và dễ tìm kiếm, đặc biệt khi danh sách chứa rất nhiều mục lựa chọn.

1. Cú pháp cơ bản của thẻ <optgroup>:

Mã HTML phân nhóm danh sách thực đơn với optgroup

```html
<select name="food">
  <optgroup label="Trái cây">
    <option value="apple">Táo</option>
    <option value="banana">Chuối</option>
  </optgroup>
  <optgroup label="Đồ uống">
    <option value="coffee">Cà phê</option>
    <option value="tea">Trà</option>
  </optgroup>
</select>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Dropdown Hiển Thị 2 Nhóm (Trái cây & Đồ uống)**

Chọn thực đơn:

TRÁI CÂY
Táo
Chuối
ĐỒ UỐNG
Cà phê
Trà

→ Dropdown hiển thị rõ ràng 2 nhóm: Trái cây và Đồ uống với các tiêu đề nhóm được in đậm.

2. Bảng thuộc tính quan trọng của thẻ <optgroup>:

[Bảng] Bảng Thuộc Tính Của Thẻ Optgroup

Thuộc Tính	Ý Nghĩa & Chức Năng	Giá Trị Mặc Định	Ví Dụ Cú Pháp
`label`	Tiêu đề văn bản của nhóm (**bắt buộc phải có** để hiển thị tên nhóm).	`Không có`	`label="Xe Đức"`
`disabled`	Vô hiệu hóa toàn bộ nhóm tùy chọn (người dùng không thể chọn các mục trong nhóm này).	`false`	`disabled`

3. Các ví dụ minh họa chi tiết:

Ví dụ 1: Dropdown nhóm tùy chọn xe thương hiệu Đức và Nhật:

Mã HTML phân nhóm danh sách hãng xe bằng optgroup

```
<select name="car">
  <optgroup label="Xe Duc">
    <option value="bmw">BMW</option>
    <option value="audi">Audi</option>
  </optgroup>
  <optgroup label="Xe Nhat">
    <option value="toyota">Toyota</option>
    <option value="honda">Honda</option>
  </optgroup>
</select>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Danh Sách Nhóm Xe Đức & Xe Nhật**

Chọn hãng xe:

XE ĐỨC
BMW
Audi
XE NHẬT
Toyota
Honda

Ví dụ 2: Nhóm bị vô hiệu hóa bằng thuộc tính disabled:

Mã HTML nhóm sản phẩm Hết hàng bị vô hiệu hóa bằng disabled

```
<select name="drink">
  <optgroup label="Còn hàng">
    <option value="coffee">Cà phê</option>
    <option value="tea">Trà</option>
  </optgroup>
  <optgroup label="Hết hàng" disabled>
    <option value="beer">Bia</option>
    <option value="wine">Rượu</option>
  </optgroup>
</select>
```

●●● **BROWSER PREVIEW** **Kết Quả Hiển Thị Trình Duyệt: Nhóm Hết Hàng Bị Vô Hiệu Hóa**

Chọn đồ uống tại quán:

CÒN HÀNG
- Cà phê
- Trà

HẾT HÀNG (Vô hiệu hóa)
- Bia
- Rượu

→ Nhóm "Hết hàng" (bao gồm Bia và Rượu) bị mờ đi và người dùng hoàn toàn không thể bấm chọn.

[Cảnh Báo] 4 Lưu Ý Cốt Lõi Khi Sử Dụng Thẻ <optgroup>

1. **Phải nằm bên trong thẻ `<select>`**: Thẻ `<optgroup>` chỉ hoạt động khi nằm trực tiếp bên trong thẻ `<select>`, không thể đứng độc lập ngoài form.
2. **Thuộc tính `label` là bắt buộc**: Nếu không khai báo thuộc tính `label`, nhóm sẽ không hiển thị tên tiêu đề trên giao diện.
3. **Khả năng kết hợp với `multiple`**: Có thể kết hợp thuộc tính `multiple` trên thẻ `<select>` để cho phép người dùng chọn cùng lúc nhiều tùy chọn nằm ở các nhóm khác nhau.
4. **CSS tùy biến hạn chế**: Các thuộc tính CSS dành cho `<optgroup>` bị trình duyệt giới hạn rất nhiều (không thể thay đổi kiểu phông, màu nền phức tạp trên một số trình duyệt mặc định).

5.9.8 Các Lưu Ý Quan Trọng Cần Ghi Nhớ Nằm Lòng

[Cảnh Báo] Các Lưu Ý Cốt Lõi Khi Sử Dụng Thẻ Select & Option

1. **Tên trường dữ liệu cho Multiple Select (`name="pets[]"`)**: Khi sử dụng thuộc tính `multiple`, để các ngôn ngữ xử lý Backend (như PHP, Node.js) nhận đầy đủ tất cả các giá trị được chọn dưới dạng **một mảng (Array)**, bạn nên đặt dấu ngoặc vuông vuông sau tên thuộc tính `name`: `name="pets[]"`.
2. **Giá trị mặc định của thuộc tính `value`**: Nếu bỏ quên không ghi thuộc tính `value` trong thẻ `<option>`, trình duyệt sẽ tự động lấy toàn bộ nội dung văn bản hiển thị làm giá trị gửi đi. Hãy luôn chủ động khai báo `value` bằng chữ thường không dấu để dễ xử lý dữ liệu Back-end.
3. **Giới hạn khả năng tùy biến CSS của thẻ <select> mặc định**: Giao diện thẻ `<select>` mặc định do hệ điều hành và trình duyệt render nên rất khó để tùy biến CSS phức tạp (như thêm icon, hiệu ứng hover đẹp). Trong các dự án thực tế đòi hỏi UX/UI cao cấp, lập trình viên thường dùng các thư viện UI JavaScript (như **Select2**, **Bootstrap Select**, **React-Select**).

5.9.9 Đọc Thêm: Kỹ Thuật Nâng Cao & Tối Ưu Lập Trình Dropdown

[Mẹo] Kỹ Thuật Tạo Dropdown Động Tự Động Thay Đổi Theo Danh Sách Cha (Cascading Select)

Trong các form đăng ký địa chỉ (Tỉnh/Thành phố → Quận/Huyện → Phường/Xã), khi người dùng chọn Tỉnh/Thành phố ở thẻ ‘<select>‘ thứ nhất, JavaScript sẽ lắng nghe sự kiện `change` để nạp danh sách Quận/Huyện tương ứng vào thẻ ‘<select>‘ thứ hai:

Mã JavaScript tự động cập nhật danh sách Select con

```
const provinceSelect = document.getElementById('province');
const districtSelect = document.getElementById('district');

provinceSelect.addEventListener('change', function() {
  const selectedProvince = this.value;
  // Gọi API hoặc lấy dữ liệu Quận/Huyện dựa theo Tỉnh/Thành đã chọn
  updateDistrictOptions(selectedProvince);
});
```

[Ghi Chú] Tối Ưu Truy Cập (a11y) Cho Thẻ Select

- Luôn luôn liên kết thẻ `<select>` với thẻ `<label>` bằng cặp thuộc tính `for` - `id`.
- Khi dùng thuộc tính `multiple`, nên bổ sung văn bản hướng dẫn rõ ràng (ví dụ: *"Giữ phím Ctrl hoặc Cmd để chọn nhiều mục"*) giúp người dùng thiết bị di động hoặc trình đọc màn hình dễ dàng thao tác.

[Bảng] Bảng So Sánh Chi Tiết: <select> vs <datalist>

Tiêu Chí So Sánh	Thẻ Danh Sách Thả Xuống <select>	Thẻ Gợi Ý Tự Động <datalist>
Giới hạn lựa chọn	Ép buộc người dùng **chỉ được chọn** các mục trong danh sách có sẵn.	Cho phép vừa chọn mục gợi ý vừa **nhập văn bản tự do**.
Trải nghiệm tìm kiếm	Mặc định không hỗ trợ ô gõ từ khóa tìm kiếm lọc dữ liệu.	Tự động lọc danh sách gợi ý theo ký tự người dùng vừa gõ.
Đóng gói dữ liệu	Gửi giá trị `value` của thẻ `<option>` được chọn.	Gửi chuỗi ký tự hiển thị trong ô `<input>`.
Trường hợp sử dụng	Chọn Tỉnh/Thành, Quốc gia, Giới tính, Trạng thái đơn hàng.	Ô tìm kiếm từ khóa, Chọn thương hiệu, Ô chọn có từ gợi ý.

[Mẹo] Mẹo Phỏng Vấn Tuyển Dụng: Xử Lý Select Đa Giá Trị Trong JavaScript

Câu hỏi phỏng vấn: Làm thế nào để trích xuất mảng tất cả các giá trị được chọn từ thẻ `<select multiple>` bằng JavaScript?

Cú pháp tối ưu:

Trích xuất mảng giá trị từ Select Multiple trong JS

```
const selectElem = document.getElementById('pets');
// Chuyển danh sách HTMLOptionsCollection thành mảng Array và lọc các mục selected
const selectedValues = Array.from(selectElem.selectedOptions).map(opt => opt.value);
console.log(selectedValues); // Kết quả mảng: ['dog', 'cat']
```

[Cảnh Báo] Lưu Ý Bảo Mật: Kiểm Tra Dữ Liệu Select Phía Server (Backend Validation)

Tuyệt đối **không tin tưởng dữ liệu từ thẻ `<select>` gửi về**! Dù trên giao diện Frontend bạn chỉ cho chọn 3 tùy chọn `["vn", "us", "jp"]`, người dùng xấu hoàn toàn có thể mở Developer Tools (F12) sửa giá trị `value="hacker"` trước khi bấm Gửi. Máy chủ Backend luôn luôn phải xác thực giá trị nhận được thuộc Danh sách Trắng (**Whitelist Validation**).

5.10 Thay Đổi Font Chữ Mặc Định Trong Form HTML

[Khái Niệm] Vấn Đề Kế Thừa Font Trong Form HTML

Trong HTML, các phần tử điều khiển biểu mẫu như `<input>`, `<textarea>`, `<select>` và `<button>` **không tự động kế thừa `font-family` từ thẻ `<body>`**.

Nếu bạn chỉ khai báo font chữ cho thẻ `<body>`, các ô nhập liệu trong form vẫn sẽ hiển thị phông chữ mặc định của trình duyệt/hệ điều hành (như Times New Roman hoặc Segoe UI), làm mất tính đồng bộ thiết kế giao diện!

5.10.1 1. Phương Pháp Thay Đổi Font Đồng Bộ Cực Kỳ Đơn Giản

Để ép tất cả các phần tử trong form dùng chung một phông chữ chuẩn với trang web, bạn có thể áp dụng thuộc tính CSS `font-family: inherit;`:

Mã CSS đồng bộ phông chữ toàn bộ form bằng font-family: inherit

```
/* Khai bao font chinh cho toan trang */
body {
  font-family: "Roboto", "Segoe UI", sans-serif;
}

/* Ep tat ca phan tu form ke thua font tu body */
form, input, textarea, select, button, label {
  font-family: inherit;
  font-size: 16px;
  color: #333;
}
```

5.10.2 2. Bảng Các Thuộc Tính Liên Quan Đến Font Chữ Trong Form

[Bảng] Bảng Thuộc Tính Định Kiểu Font Chữ Trong Form

Thuộc Tính CSS	Ý Nghĩa & Tác Dụng	Ví Dụ Cú Pháp
`font-family`	Chọn họ phông chữ hiển thị.	`font-family: Arial, sans-serif;`
`font-size`	Điều chỉnh kích thước văn bản.	`font-size: 16px;`
`font-weight`	Độ đậm của chữ (100-900 hoặc `normal`, `bold`).	`font-weight: bold;`
`font-style`	Kiểu dáng chữ (`normal`, `italic`, `oblique`).	`font-style: italic;`
`line-height`	Chiều cao dòng (giúp văn bản thoáng và dễ đọc hơn).	`line-height: 1.5;`
`color`	Màu sắc văn bản nội dung.	`color: #333333;`

5.10.3 3. Ví Dụ Mã Nguồn HTML & CSS Đầy Đủ

Mã HTML & CSS hoàn chỉnh đổi phông chữ form với Google Fonts

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <title>Form Doi Font</title>
  <!-- Nhap Google Font Roboto -->
  <link href="https://fonts.googleapis.com/css2?family=Roboto&display=swap" rel="
    stylesheet">
  <style>
    body {
      font-family: "Roboto", sans-serif;
    }
    form, input, textarea, select, button, label {
      font-family: inherit; /* Ke thua tu body */
      font-size: 16px;
      color: #333;
    }
  </style>
</head>
<body>
  <h2>Form Vi Du</h2>
  <form>
    <label for="name">Họ tên:</label><br>
    <input type="text" id="name" name="name"><br><br>

    <label for="email">Email:</label><br>
    <input type="email" id="email" name="email"><br><br>

    <label for="msg">Tin nhan:</label><br>
    <textarea id="msg" name="msg"></textarea><br><br>

    <button type="submit">Gửi</button>
  </form>
</body>
</html>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Form Đồng Bộ Font Chữ Roboto**

Form Ví Dụ (Đồng Bộ Font Roboto)

Họ tên:

Nguyễn Trọng Tấn

Email:

contact@domain.com

Tin nhắn:

Nội dung tin nhắn...

Gửi

[Cảnh Báo] 4 Lưu Ý Cốt Lõi Khi Thay Đổi Font Chữ Trong Form

1. **Dùng `font-family: inherit;`**: Giúp các phần tử form tự động kế thừa font từ `<body>`, giúp dễ quản lý trung tâm (chỉ cần đổi font 1 nơi ở body là toàn bộ form đổi theo).
2. **Tránh chỉ đặt font cho `<body>`**: Nếu quên bộ chọn `input, select, textarea, button`, trình duyệt sẽ rơi về phông mặc định của hệ điều hành.
3. **Nạp font trước khi dùng**: Nếu dùng Google Fonts hoặc Custom Font, phải nạp thẻ `<link>` hoặc `@import` trước khi áp dụng CSS.
4. **Ghi đè cho nút bấm**: Nút bấm (`<button>`, `input[type="submit"]`) luôn sở hữu kiểu dáng mặc định riêng của trình duyệt, do đó luôn phải ghi đè `font-family: inherit;`.

5.11 Giả Lớp CSS :placeholder-shown & Kỹ Thuật Floating Label

[Khái Niệm] Giả Lớp :placeholder-shown Là Gì?

`:placeholder-shown` là một CSS Pseudo-class (Giả lớp) được trình duyệt tự động kích hoạt cho các phần tử `<input>` hoặc `<textarea>` **khi phần tử đó đang hiển thị văn bản hướng dẫn placeholder** (tức là ô input đang rỗng).

- **Input trống** → Placeholder hiển thị → `:placeholder-shown` được kích hoạt.
- **Input có dữ liệu** → Placeholder biến mất → `:placeholder-shown` bị vô hiệu hóa.

5.11.1 1. Cú Pháp Cơ Bản & Ví Dụ Minh Họa

Mã CSS thay đổi viền và nền ô Input khi placeholder đang hiển thị

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <title>Vi du placeholder-shown</title>
  <style>
    input {
      padding: 8px;
      border: 2px solid #ccc;
      border-radius: 4px;
    }
    /* Khi placeholder con hien thi (o input trong) */
    input:placeholder-shown {
      border-color: orange;
      background-color: #fff8e1;
    }
    /* Khi nguoi dung da nhap text vao */
    input:not(:placeholder-shown) {
      border-color: green;
      background-color: #f0fff4;
    }
  </style>
</head>
<body>
  <form>
    <label for="name">Tên:</label><br>
    <input type="text" id="name" placeholder="Nhập tên của bạn">
  </form>
</body>
</html>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Trạng Thái Ban Đầu vs Khi Nhập Dữ Liệu**

1. Trạng thái ban đầu (Input trống - Placeholder hiện):

Nhập tên của bạn **(Viền cam, Nền vàng nhạt)**

2. Khi người dùng nhập dữ liệu (Placeholder biến mất):

Nguyễn Trọng Tấn **(Viền xanh lá, Nền xanh nhạt)**

[Cảnh Báo] 4 Lưu Ý Cốt Lõi Khi Sử Dụng :placeholder-shown

1. **Chỉ dùng cho trường có placeholder**: Giả lớp này chỉ hoạt động với các thẻ `<input>` và `<textarea>` có khai báo thuộc tính `placeholder="..."`.
2. **Không áp dụng cho thẻ <select>**: Thẻ `<select>` không có thuộc tính placeholder nên không thể dùng `:placeholder-shown`.
3. **Phân biệt với ::placeholder**: Selector `::placeholder` dùng để định kiểu văn bản chữ gợi ý bên trong, còn `:placeholder-shown` dùng để định kiểu toàn bộ ô input dựa theo trạng thái dữ liệu.
4. **Tối ưu UX Form Validation**: Giúp đánh dấu trực quan ô chưa nhập dữ liệu mà không cần viết mã JavaScript.

5.12 Giả Lớp CSS :not(:placeholder-shown) & Ứng Dụng Floating Label

[Khái Niệm] Ý Nghĩa Của :not(:placeholder-shown)

Đây là sự kết hợp giữa hai giả lớp `:not()` và `:placeholder-shown`.

Ý Nghĩa: Chọn các phần tử `<input>` hoặc `<textarea>` **không còn hiển thị placeholder** (tức là khi người dùng đã chủ động nhập dữ liệu vào ô).

5.12.1 1. Mã Nguồn Form Validation Thuần CSS với :not(:placeholder-shown)

Mã HTML & CSS đổi màu viền khi ô Input có dữ liệu

```html
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <title>Vi du :not(:placeholder-shown)</title>
  <style>
    input {
      padding: 8px;
      border: 2px solid #ccc;
      border-radius: 4px;
    }
    /* Khi field con trong viền đỏ */
    input:placeholder-shown {
      border-color: red;
      background-color: #fff5f5;
    }
    /* Khi field da nhap du lieu viền xanh lá */
    input:not(:placeholder-shown) {
      border-color: green;
      background-color: #f0fff4;
    }
  </style>
</head>
<body>
  <form>
    <label for="email">Email:</label><br>
    <input type="email" id="email" placeholder="Nhập email...">
  </form>
</body>
</html>
```

BROWSER PREVIEW — **Kết Quả Hiển Thị Trình Duyệt: Validation Thuần CSS**

1. Khi chưa nhập (Input trống):

Nhập email... **(Viền đỏ - Chưa nhập)**

2. Khi đã nhập text (Dữ liệu sẵn sàng):

tan@domain.com **(Viền xanh lá - Đã nhập)**

5.12.2 2. Các Ứng Dụng Thực Tế Đỉnh Cao

[Mẹo] 3 Ứng Dụng Thực Tế Đỉnh Cao Của :not(:placeholder-shown)

1. **Form Validation Nhẹ Bằng CSS**: Đánh dấu trạng thái ô nhập dữ liệu ngay khi người dùng hoàn thành gõ chữ mà không cần lắng nghe sự kiện JavaScript.
2. **Kỹ Thuật Floating Label (Nhãn Trượt Nổi Thuần CSS)**: Khi người dùng bấm vào ô và bắt đầu nhập text, thẻ `<label>` tự động thu nhỏ và bay lên mép trên ô input:

Mã CSS tạo hiệu ứng Floating Label không dùng JS

```
/* Khi input da co du lieu -> nhan label bay len tren */
input:not(:placeholder-shown) + label {
  transform: translateY(-20px);
  font-size: 12px;
  color: #1a73e8;
}
```

3. **Tối Ưu Hiệu Năng**: Loại bỏ mã JavaScript lắng nghe sự kiện `input` hay `change`, giúp trình duyệt render giao diện mượt mà và tiết kiệm RAM.

[Cảnh Báo] Lưu Ý Quan Trọng Với Thuộc Tính Placeholder

- **Yêu cầu thuộc tính placeholder**: Thuộc tính `:placeholder-shown` bắt buộc ô input phải có thuộc tính `placeholder="..."`. Nếu bạn quên khai báo thuộc tính này, `:placeholder-shown` sẽ không bao giờ khớp, khiến giả lớp `:not(:placeholder-shown)` luôn luôn đúng (luôn có tác dụng ngay cả khi input trống!).
- **Mẹo nhỏ**: Nếu muốn dùng kỹ thuật Floating Label nhưng không muốn hiện chữ gợi ý, hãy khai báo thuộc tính rỗng: `placeholder=" "`.

5.12.3 3. Đọc Thêm: Phân Biệt Phân Cấp Pseudo-class :placeholder-shown vs Pseudo-element ::placeholder

[Bảng] Bảng So Sánh Chi Tiết: :placeholder-shown vs ::placeholder

Tiêu Chí So Sánh	Giả Lớp :placeholder-shown (Pseudo-class)	Phần Tử Giả ::placeholder (Pseudo-element)
Đối tượng tác động	Định kiểu cho **toàn bộ ô input / textarea** (viền, nền, bóng đổ, vị trí label con).	Chỉ định kiểu cho **chữ văn bản gợi ý** nằm bên trong ô.
Cú pháp CSS	Sử dụng dấu hai chấm đơn `:placeholder-shown`.	Sử dụng dấu hai chấm kép `::placeholder`.
Các thuộc tính CSS hỗ trợ	Hỗ trợ toàn bộ thuộc tính CSS (`border`, `background`, `transform`, `padding`).	Chỉ hỗ trợ các thuộc tính văn bản (`color`, `font-style`, `opacity`).
Mục đích sử dụng	Kiểm tra trạng thái dữ liệu đã nhập hay chưa để tạo hiệu ứng UI/UX.	Đổi màu chữ gợi ý mờ hơn hoặc nghiêng để tránh nhầm với chữ user gõ.

[Mẹo] Kỹ Thuật Kết Hợp Định Kiểu Hoàn Hảo Cả 2 Giả Lớp

Mã CSS kết hợp định kiểu cả văn bản gợi ý lẫn trạng thái ô input

```
/* 1. Dinh kieu chu goi y nghieng va mo 50% */
input::placeholder {
  color: #888888;
  font-style: italic;
  opacity: 0.7;
}

/* 2. Dinh kieu o input khi dang cho nhap du lieu */
input:placeholder-shown {
  border: 1.5px solid #1a73e8;
  box-shadow: 0 0 5px rgba(26, 115, 232, 0.2);
}
```

5.13 Giả Lớp CSS :focus-within & Hiệu Ứng Focus Nhóm

[Khái Niệm] Giả Lớp :focus-within Trong CSS Là Gì?

`:focus-within` là một CSS Pseudo-class (Giả lớp). Nó chọn một phần tử khi **chính bản thân nó** hoặc **bất kỳ phần tử con nào bên trong nó** đang được nhận tiêu điểm focus.

- **Cơ chế hoạt động**: Nếu người dùng nhấp chuột hoặc dùng phím Tab focus vào một ô `<input>` nằm bên trong một khối `<form>` hoặc `<div class="form-group">`, thì toàn bộ khối cha đó cũng sẽ lập tức khớp với giả lớp `:focus-within`.

5.13.1 1. Cú Pháp Cơ Bản

Cú pháp CSS đổi viền khối container khi có con được focus

```
.container:focus-within {
  border: 2px solid blue;
}
```

5.13.2 2. Ví Dụ Minh Họa Chi Tiết

Mã HTML & CSS hoàn chỉnh ứng dụng :focus-within làm nổi bật khối Form Group

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <title>Vi du :focus-within</title>
  <style>
    .form-group {
      border: 2px solid #ccc;
      padding: 10px;
      border-radius: 6px;
      transition: 0.3s;
    }
    /* Khi co input ben trong duoc focus */
    .form-group:focus-within {
      border-color: blue;
      background-color: #f0f8ff;
    }
    input {
      padding: 6px;
      border: 1px solid #aaa;
    }
  </style>
</head>
<body>
  <div class="form-group">
    <label for="name">Họ tên:</label><br>
    <input type="text" id="name" placeholder="Nhập tên...">
  </div>
</body>
</html>
```

BROWSER PREVIEW Kết Quả Hiển Thị Trình Duyệt: Trạng Thái Focus Nhóm Form

1. Khi chưa focus (Trạng thái đơn thuần bình thường):

Họ tên: Nhập tên... **(Viền xám, Nền trắng)**

2. Khi người dùng focus - nhấp chuột vào ô Input bên trong:

Họ tên: Nguyễn Trọng Tấn **(Khối cha đổi viền xanh, Nền xanh nhạt #f0f8ff)**

5.13.3 3. Ví Dụ 2: Khung Tìm Kiếm Tự Động Hiển Thị Menu Gợi Ý (Search Box Dropdown)

Mã HTML & CSS hiển thị danh sách từ khóa gợi ý khi ô Input Search được focus

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <title>Search Box :focus-within</title>
  <style>
    .search-box {
      position: relative;
      width: 320px;
      border: 2px solid #ddd;
      border-radius: 20px;
      padding: 8px 16px;
      transition: all 0.3s ease;
      background-color: #fff;
    }
    /* Khi input ben trong duoc focus -> viem xanh & do bong Glow */
    .search-box:focus-within {
      border-color: #1a73e8;
      box-shadow: 0 4px 16px rgba(26, 115, 232, 0.25);
    }
    .search-input {
      border: none;
      outline: none;
      width: 100%;
      font-size: 14px;
    }
    /* Dynamic Dropdown */
    .search-dropdown {
      display: none; /* Mac dinh an */
      position: absolute;
      top: 100%; left: 0; right: 0;
      background: #fff;
      border: 1px solid #ddd;
      border-radius: 8px;
      margin-top: 6px;
      box-shadow: 0 4px 12px rgba(0,0,0,0.1);
    }
    /* Tu dong hien thi dropdown khi co focus bên trong */
    .search-box:focus-within .search-dropdown {
      display: block;
    }
  </style>
</head>
<body>
  <div class="search-box">
    <input type="search" class="search-input" placeholder="Tìm kiếm từ khóa...">
    <div class="search-dropdown">
      <div style="padding: 8px;">1. Lập trình Web HTML5/CSS3</div>
      <div style="padding: 8px;">2. Giả lớp CSS :focus-within</div>
    </div>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Thanh Tìm Kiếm Đổ Bóng & Mở Menu Gợi Ý**

1. Trạng thái ban đầu (Chưa focus):

[Kính lúp] Tìm kiếm từ khóa... **(Menu ẩn)**

2. Khi người dùng nhấp chuột focus vào ô tìm kiếm:

[Kính lúp] Lập trình web |

[Kính lúp] Từ khóa gợi ý tự động:
1. Lập trình Web HTML5/CSS3
2. Giả lớp CSS :focus-within

(Viền xanh sáng, Tự hiện Menu Gợi ý không cần JS)

5.13.4 4. Ví Dụ 3: Floating Label Màu Tím Neon Tối Ưu Tương Tác

Mã HTML & CSS kỹ thuật Floating Label nhãn trượt nổi màu tím neon

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <style>
    .field-wrapper {
      position: relative;
      margin-top: 15px;
      border: 2px solid #ccc;
      border-radius: 8px;
      padding: 12px;
      transition: 0.3s;
    }
    .field-wrapper input {
      border: none;
      outline: none;
      width: 100%;
    }
    .field-wrapper label {
      position: absolute;
      top: 12px; left: 12px;
      color: #777;
      transition: 0.2s ease-all;
      background-color: white;
      padding: 0 4px;
    }
    /* Khi nhap text HOAC dang focus -> Nhan label bay len mep tren */
    .field-wrapper:focus-within label,
    .field-wrapper input:not(:placeholder-shown) + label {
      top: -10px;
      left: 10px;
      font-size: 12px;
      color: #6200ee;
      font-weight: bold;
    }
    .field-wrapper:focus-within {
      border-color: #6200ee;
    }
  </style>
</head>
<body>
  <div class="field-wrapper">
    <input type="text" id="phone" placeholder=" ">
    <label for="phone">Số điện thoại liên hệ</label>
  </div>
</body>
</html>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Floating Label Màu Tím Neon**

1. Trạng thái rỗng chưa focus:

Số điện thoại liên hệ

2. Khi focus nhấp chuột (Label trượt nổi lên mép trên):

[Số điện thoại liên hệ] 0988 123 456

(Label nổi viền tím Neon)

5.13.5 5. Ví Dụ 4: Giao Diện Multi-Card Form – Nổi Bật Thẻ Card Đang Thao Tác

Mã HTML & CSS làm nổi bật Card form đang được người dùng thao tác

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <style>
    .form-card {
      border: 1.5px solid #e0e0e0;
      border-radius: 10px;
      padding: 16px;
      margin-bottom: 15px;
      opacity: 0.65; /* Mo nhe khi chua thao tac */
      transition: all 0.3s ease;
      background-color: #fafafa;
    }
    /* Card nao chua o input dang focus se sang ro va noi len */
    .form-card:focus-within {
      opacity: 1;
      border-color: #00c853;
      background-color: #ffffff;
      box-shadow: 0 6px 20px rgba(0, 200, 83, 0.15);
      transform: translateY(-2px);
    }
  </style>
</head>
<body>
  <div class="form-card">
    <h3>Thẻ 1: Thông tin cá nhân</h3>
    <input type="text" placeholder="Họ và tên">
  </div>
  <div class="form-card">
    <h3>Thẻ 2: Địa chỉ giao hàng</h3>
    <input type="text" placeholder="Số nhà, Tên đường">
  </div>
</body>
</html>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Nổi Bật Card Tương Tác**

1. Card 1 (Chưa focus):

Thẻ 1: Thông tin cá nhân (Mờ 65%)

2. Card 2 (Đang thao tác gõ số nhà - Card tự nổi sáng 100%):

Thẻ 2: Địa chỉ giao hàng 123 Đường Nguyễn Huệ, Q1 | **(Nổi sáng 100%, Viền xanh lá, Bóng mờ Card)**

[Mẹo] 3 Ứng Dụng Thực Tế Đỉnh Cao Của :focus-within

1. **Highlight nhóm form**: Làm nổi bật toàn bộ thẻ cha chứa ô nhập liệu khi người dùng thao tác, giúp cải thiện tầm nhìn trực quan.
2. **Floating label**: Thẻ label tự động đổi màu hoặc nổi lên khi input bên trong nhận focus.
3. **Tạo hiệu ứng trực quan cho UX**: Xây dựng các hiệu ứng tương tác cao cấp hoàn toàn bằng CSS mà không cần viết mã JavaScript.

[Cảnh Báo] Các Lưu Ý Khi Sử Dụng Giả Lớp :focus-within

- **Khác biệt hoàn toàn với `:focus`**: Giả lớp `:focus` chỉ áp dụng trực tiếp cho chính bản thân phần tử đang nhận tiêu điểm, trong khi `:focus-within` áp dụng cho phần tử cha chứa nó (rất tiện lợi khi muốn định kiểu toàn bộ khối **Block**).
- **Độ tương thích trình duyệt**: Được hỗ trợ đầy đủ trên tất cả các trình duyệt hiện đại (Chrome, Firefox, Edge, Safari).

5.14 Truy Cập Web Cho Biểu Mẫu (Form Accessibility - a11y)

[Khái Niệm] Tiêu Chuẩn Accessibility (a11y) Trong HTML Form

Web Accessibility (a11y) trong biểu mẫu là việc thiết kế và lập trình sao cho tất cả người dùng, bao gồm cả **người khiếm thị hoặc người khuyết tật sử dụng trình đọc màn hình (Screen Reader)** hoặc chỉ dùng bàn phím (Keyboard navigation), đều có thể hiểu và tương tác với form dễ dàng.

5.14.1 1. Các Quy Tắc Vàng Giúp Form Đạt Chuẩn WCAG

1. **Liên kết thẻ `<label>` chuẩn xác**: Mỗi ô input bắt buộc phải có thẻ `<label>` liên kết bằng cặp thuộc tính `for="..."` và `id="..."`.

2. **Sử dụng thuộc tính ARIA**:
   - `aria-describedby="..."`: Liên kết ô input với đoạn văn bản hướng dẫn hoặc thông báo lỗi bên dưới.
   - `aria-invalid="true"`: Thông báo cho trình đọc màn hình biết ô input đang bị lỗi dữ liệu.
   - `aria-required="true"`: Đánh dấu trường bắt buộc nhập cho công cụ trợ năng.
3. **Cung cấp nhãn ẩn cho Screen Reader (`.sr-only`)**: Đối với các nút bấm chỉ có Icon (như nút Tìm kiếm kính lúp), hãy thêm văn bản ẩn để Screen Reader đọc được tên nút.

Mã HTML form chuẩn truy cập Accessibility a11y

```html
<form>
  <!-- 1. Lien ket label bang for va id -->
  <label for="username">Ten dang nhap:</label>
  <input type="text" id="username" name="username"
         aria-describedby="username-hint" aria-required="true">
  <small id="username-hint">Ten dang nhap tu 6-20 ky tu.</small>

  <!-- 2. Nung Nut bam Icon co nhan an Screen Reader -->
  <button type="submit" aria-label="Tim kiem san pham">
    <svg width="16" height="16"><path d="..."/></svg>
    <span class="sr-only">Tim kiem</span>
  </button>
</form>
```

BROWSER PREVIEW — **Kết Quả Trình Duyệt: Form Đạt Chuẩn Trợ Năng a11y**

Tên đăng nhập:

nguyenvana

Tên đăng nhập từ 6-20 ký tự (Đã liên kết aria-describedby)

[SVG Icon] Tìm kiếm

5.14.2 2. Thuộc Tính Điều Hướng Bàn Phím tabindex Trong HTML

[Khái Niệm] Thuộc Tính tabindex Trong HTML Là Gì?

`tabindex` là một thuộc tính toàn cục (**Global Attribute**) trong HTML. Nó xác định thứ tự tiêu điểm focus của các phần tử khi người dùng nhấn phím **Tab** trên bàn phím.

- **Tác dụng cốt lõi**: Giúp cải thiện trải nghiệm điều hướng bằng bàn phím (**Keyboard Navigation**), là yêu cầu sống còn cho tiêu chuẩn truy cập Web Accessibility (WCAG, ARIA).

[Bảng] Bảng Chi Tiết Các Giá Trị Của Thuộc Tính tabindex

Giá Trị tabindex	Ý Nghĩa & Hành Vi Tiêu Điểm	Ví Dụ HTML
`tabindex="0"`	Phần tử có thể focus được, và thứ tự focus tuân theo thứ tự xuất hiện tự nhiên trong mã DOM HTML.	`<div tabindex="0">Có thể focus</div>`
`tabindex="-1"`	Phần tử có thể focus bằng mã JavaScript (qua phương thức `element.focus()`), nhưng sẽ **bị bỏ qua** khi người dùng bấm phím Tab.	`<div tabindex="-1">Bỏ qua khi Tab</div>`
`tabindex="n"` ($n > 0$)	Đặt thứ tự focus cụ thể ưu tiên trước thứ tự DOM tự nhiên (số nhỏ hơn được focus trước).	`<input type="text" tabindex="1">`

5.14.3 3. Ví Dụ Minh Họa Chi Tiết Về Thứ Tự Tab

Mã HTML minh họa các giá trị tabindex khác nhau

```html
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <title>Vi du tabindex</title>
</head>
<body>
  <h2>Thứ tự Tab</h2>
  <!-- Theo DOM mac dinh (tabindex=0) -->
  <input type="text" placeholder="Tên (DOM)" tabindex="0"><br>
  <input type="text" placeholder="Email (DOM)" tabindex="0"><br>

  <!-- Bo qua khi nhan phim Tab (tabindex=-1) -->
  <button tabindex="-1">Nút bị bỏ qua</button><br>

  <!-- Custom thu tu uu tien (tabindex > 0) -->
  <input type="text" placeholder="Custom 1" tabindex="2"><br>
  <input type="text" placeholder="Custom 2" tabindex="1"><br>
</body>
</html>
```

BROWSER PREVIEW **Kết Quả Hành Vi Điều Hướng Phím Tab Khi Người Dùng Thao Tác**

Thứ tự di chuyển tiêu điểm khi nhấn phím Tab liên tiếp:

1. Custom 2 **(tabindex=1 được focus đầu tiên)**
2. Custom 1 **(tabindex=2 được focus thứ hai)**
3. Tên (DOM) **(Tiếp tục sang phần tử DOM mặc định 1)**
4. Email (DOM) **(Tiếp tục sang phần tử DOM mặc định 2)**
5. Nút bị bỏ qua **(Đã bị BỎ QUA hoàn toàn do tabindex=-1)**

[Cảnh Báo] 4 Lưu Ý Nằm Lòng Khi Sử Dụng Thuộc Tính tabindex

1. **Tránh lạm dụng số dương (`tabindex > 0`)**: Việc đặt các số nguyên dương sẽ ghi đè thứ tự DOM tự nhiên, dễ gây rối loạn và mất phương hướng cho người dùng chỉ dùng bàn phím. **Khuyên dùng**: Chỉ sử dụng `0` hoặc `-1`.
2. **Biến thẻ không tương tác thành focusable (`tabindex="0"`)**: Dùng cho các thẻ mặc định không thể focus (như `<div>`, `<span>`) để người dùng bàn phím có thể Tab tới và tương tác.
3. **Focus bằng JavaScript với `tabindex="-1"`**: Hữu ích khi cần chuyển tiêu điểm bằng script: `document.getElementById("myDiv").focus();`, nhưng không muốn người dùng tự Tab tới phần tử đó.
4. **Các thẻ form mặc định không cần `tabindex="0"`**: Các phần tử như `<input>`, `<button>`, `<select>`, `<a href>` đã mặc định có thể nhận focus, do đó không cần khai báo thêm `tabindex="0"`.

CHƯƠNG 6

Thẻ Đa Phương Tiện (Multimedia), Nhúng & Graphics

Chương này phân tích chi tiết từng thẻ thuộc nhóm Đa phương tiện, Nhúng khung web và Đồ họa Vector SVG/Canvas trong HTML5.

6.1 Thẻ Hình Ảnh <img>

6.1.1 1. Lý Thuyết & Bảng Toàn Bộ Thuộc Tính Tối Ưu Hiệu Năng

Thẻ <img> dùng để nhúng hình ảnh vào tài liệu.

[Bảng] Bảng Thống Kê

Thuộc Tính	Giá Trị Hợp Lệ	Chức Năng & Tối Ưu Hiệu Năng
src	URL ảnh	Đường dẫn tệp ảnh.
alt	Chuỗi văn bản	Văn bản thay thế khi lỗi ảnh hoặc cho Screen Reader.
srcset	Danh sách URLs kèm mật độ pixel	Cung cấp các định dạng ảnh theo độ phân giải màn hình.
sizes	Media queries ((max-width: 600px) 100vw)	Khai báo kích thước hiển thị dự kiến của ảnh.
loading	lazy, eager	Tải ngầm Lazy Loading khi cuộn tới ảnh.
decoding	async, sync	Giải mã hình ảnh bất đồng bộ không làm giật trang.
fetchpriority	high, low, auto	Ép trình duyệt tải ảnh Banner Hero trước tiên.

Bảng 6.1: Toàn bộ thuộc tính tối ưu hình ảnh của thẻ <img>

6.1.2 2. Ví Dụ Mã Nguồn Thực Tế

```
<!-- Ảnh Hero banner tải ngay lập tức với ưu tiên cao nhất --><img src="hero-1200.webp"
     srcset="hero-600.webp 600w, hero-1200.webp 1200w"
     sizes="(max-width: 600px) 100vw, 1200px"
     alt="Banner Khóa Học HTML5"
     width="1200" height="600"
     fetchpriority="high"
     decoding="async">

```

```
<!-- Ảnh bên dưới cuộn tới đâu tải tới đó (Lazy Loading) --><img src="course-thumb.jpg" alt="Minh họa bài học" loading="lazy">
```

Listing 6.1: Ví dụ ảnh chuẩn Responsive kèm Lazy Loading và Fetch Priority

6.2 Thẻ Nguồn Ảnh Thích Ứng <picture> & <source>

6.2.1 1. Lý Thuyết & Kỹ Thuật Art Direction

Thẻ <picture> là wrapper chứa một hoặc nhiều thẻ <source> và một thẻ <img> dự phòng. Thẻ này áp dụng kỹ thuật **Art Direction** (Đổi khung ảnh theo màn hình di động/máy tính) và tự động chọn định dạng ảnh thế hệ mới (AVIF, WebP).

6.2.2 2. Ví Dụ Mã Nguồn Thực Tế

```
<picture>
  <!-- 1. Ưu tiên cao nhất: Định dạng AVIF siêu nhẹ --> <source srcset="image.avif" type="image/avif">

  <!-- 2. Ưu tiên 2: Định dạng WebP --> <source srcset="image.webp" type="image/webp">

  <!-- 3. Dự phòng cho trình duyệt cũ --> <img src="image.jpg" alt="Mô tả hình ảnh">
</picture>
```

Listing 6.2: Ví dụ đổi định dạng ảnh AVIF/WebP với thẻ picture

6.3 Thẻ Bản Đồ Ảnh <map> & <area>

6.3.1 1. Lý Thuyết & Thuộc Tính Tọa Độ

- <map name="mapname">: Khai báo một bản đồ hình ảnh chứa các vùng liên kết nhấp chuột. - <area>: Khai báo một vùng nhấp chuột cụ thể. Thuộc tính shape="rect|circle|poly", coords="x1,y1,x2,y2", href="...".

6.3.2 2. Ví Dụ Mã Nguồn Thực Tế

```
<img src="planets.jpg" alt="Hệ Mặt Trời" usemap="#image-map">

<map name="image-map">
  <!-- Vùng hình tròn nhấp vào Mặt Trời --> <area shape="circle" coords="100,100,50" href="sun.html" alt="Mặt Trời">
  <!-- Vùng hình chữ nhật nhấp vào Trái Đất --> <area shape="rect" coords="200,80,260,140" href="earth.html" alt="Trái Đất">
</map>
```

Listing 6.3: Ví dụ tạo vùng nhấp chuột tương tác trên ảnh

6.4 Thẻ Phát Video <video>

6.4.1 1. Lý Thuyết & Bảng Thuộc Tính

Thẻ <video> phát trình chiếu các tệp video gốc (MP4, WebM, Ogg).

- controls: Hiển thị thanh điều khiển phát/dừng/âm lượng.
- poster="...": Ảnh đại diện hiển thị trước khi ấn play.
- autoplay, muted, loop: Tự động phát ngầm (Phải kèm muted mới chạy trên iOS/Android).
- playsinline: Phát video trực tiếp trong dòng trên trình duyệt iOS Safari.

6.4.2 2. Ví Dụ Mã Nguồn Thực Tế

```
<video controls poster="thumbnail.jpg" width="800" height="450" playsinline preload="
    metadata">
  <source src="course-intro.webm" type="video/webm">
  <source src="course-intro.mp4" type="video/mp4">
  Trình duyệt của bạn không hỗ trợ thẻ video.
</video>
```

Listing 6.4: Ví dụ thẻ video phát chuẩn đa thiết bị

6.5 Thẻ Phát Âm Thanh <audio>

6.5.1 1. Lý Thuyết & Định Dạng Hỗ Trợ

Thẻ <audio> nhúng trình phát âm thanh gốc (MP3, WAV, OGG).

6.5.2 2. Ví Dụ Mã Nguồn Thực Tế

```
<audio controls preload="none">
  <source src="podcast-ep1.mp3" type="audio/mpeg">
  <source src="podcast-ep1.ogg" type="audio/ogg">
</audio>
```

Listing 6.5: Ví dụ thẻ audio phát nhạc podcast

6.6 Thẻ Nhúng Phụ Đề <track>

6.6.1 1. Lý Thuyết & Chuẩn Phụ Đề WebVTT

Thẻ <track> nhúng phụ đề văn bản chuẩn **WebVTT** (.vtt) vào các thẻ <video> hoặc <audio>.

- kind="subtitles|captions|descriptions|chapters": Loại phụ đề.
- srclang="vi|en": Mã ngôn ngữ của phụ đề.
- label="...": Tên hiển thị trên menu chọn phụ đề.
- default: Bật sẵn phụ đề này.

6.6.2 2. Ví Dụ Mã Nguồn Thực Tế

```
<video controls src="lecture.mp4">
```

```
  <track kind="subtitles" src="subtitles-vi.vtt" srclang="vi" label="Tiếng
    Việt" default>
  <track kind="subtitles" src="subtitles-en.vtt" srclang="en" label="English">
</video>
```

Listing 6.6: Ví dụ nhúng phụ đề tiếng Việt và tiếng Anh cho video

6.7 Thẻ Khung Nhúng Trang Web <iframe>

6.7.1 1. Lý Thuyết & Bảo Mật Sandbox

Thẻ <iframe> (Inline Frame) nhúng một tài liệu HTML độc lập khác vào trang web hiện tại.

[Bảng] Bảng Thống Kê

Thuộc Tính	Giá Trị Hợp Lệ	Chức Năng & Bảo Mật
src	URL	Địa chỉ trang web nhúng.
srcdoc	Mã HTML	Nhúng trực tiếp chuỗi HTML vào iframe mà không cần URL.
sandbox	allow-scripts, allow-forms, allow-same-origin	Bật tường lửa bảo mật cô lập iframe.
allow	camera; microphone; fullscreen	Phân quyền truy cập thiết bị phần cứng.

Bảng 6.2: Các thuộc tính bảo mật của thẻ iframe

6.7.2 2. Ví Dụ Mã Nguồn Thực Tế

```
<iframe width="560" height="315"
        src="https://www.youtube.com/embed/dQw4w9WgXcQ"
        title="Trình chiếu Video YouTube"
        allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope;
    picture-in-picture"
        sandbox="allow-scripts allow-same-origin allow-popups"
        loading="lazy"
        allowfullscreen></iframe>
```

Listing 6.7: Ví dụ nhúng video YouTube an toàn với sandbox và allow

6.8 Thẻ Đồ Họa Vector Inline <svg> & Các Thẻ Con

6.8.1 1. Lý Thuyết & Đồ Họa Độc Lập Độ Phân Giải

Thẻ <svg> nhúng mã đồ họa Vector 2D trực tiếp vào trang web. Các thẻ con chính: <path>, <circle>, <rect>, <line>, <text>, <defs>, <use>.

6.8.2 2. Ví Dụ Mã Nguồn Thực Tế

```
<svg width="100" height="100" viewBox="0 0 100 100" xmlns="http://www.w3.org/2000/svg"
    >
  <circle cx="50" cy="50" r="40" stroke="#0F4C81" stroke-width="4" fill="#008080" />
</svg>
```

Listing 6.8: Ví dụ vẽ icon hình tròn và hình sao bằng SVG

6.9 Thẻ Khung Vẽ Điểm Ảnh <canvas>

6.9.1 1. Lý Thuyết & 2D Context API

Thẻ <canvas> tạo ra một vùng trống tính bằng điểm ảnh (Pixels) cho phép JavaScript vẽ đồ họa 2D, biểu đồ động hoặc game WebGL.

6.9.2 2. Ví Dụ Mã Nguồn Thực Tế

```
<canvas id="myCanvas" width="200" height="100" style="border:1px solid #ccc;"></canvas
    >

<script>
  const canvas = document.getElementById('myCanvas');
  const ctx = canvas.getContext('2d');
  ctx.fillStyle = '#0F4C81';
  ctx.fillRect(10, 10, 150, 80);
</script>
```

Listing 6.9: Ví dụ vẽ hình chữ nhật đỏ bằng Canvas API

6.10 Thẻ Công Thức Toán Học <math> (MathML)

6.10.1 1. Lý Thuyết & Các Phần Tử MathML

Thẻ <math> nhúng công thức toán học chuẩn W3C MathML. Các thẻ con: <mfrac> (Phân số), <msqrt> (Căn bậc hai), <msup> (Số mũ).

6.10.2 2. Ví Dụ Mã Nguồn Thực Tế

```
<math>
  <mfrac>
    <mi>a</mi>
    <mi>b</mi>
  </mfrac>
</math>
```

Listing 6.10: Ví dụ hiển thị công thức toán học Phân số

6.11 Thẻ Nhúng Đối Tượng Khác <object>, <embed>, <param>

6.11.1 1. Lý Thuyết & Nhúng File PDF

- `<object data="..." type="...">`: Nhúng tài nguyên ngoài như file PDF. - `<embed src="..." type="...">`: Nhúng đa phương tiện ứng dụng ngoài.

6.11.2 2. Ví Dụ Mã Nguồn Thực Tế

```
<object data="document.pdf" type="application/pdf" width="100%" height="500px">
  <p>Trình duyệt không hỗ trợ xem PDF.<a href="document.pdf">Tải tệp tại đây</a>.</p>
</object>
```

Listing 6.11: Ví dụ nhúng tệp PDF hiển thị trực tiếp

6.12 Tóm Tắt Chương 6

[Khái Niệm] Tóm Lại Cốt Lõi Kiến Thức Chương 6

Chương 6 đã phân tích trọn bộ các mục riêng biệt dành cho từng thẻ Đa phương tiện (`<img>`, `<picture>`, `<video>`, `<audio>`, `<track>`), Nhúng khung web (`<iframe>`, `<object>`) và Đồ họa Vector/Canvas (`<svg>`, `<canvas>`, `<math>`). Chương tiếp theo sẽ tiến sang nhóm Thẻ Tương Tác Hiện Đại, Web Components, Thuộc Tính Toàn Cục và SEO Metadata.

CHƯƠNG 7

Thẻ Tương Tác Hiện Đại, Web Components, Global Attributes & SEO

Chương này làm chủ các thẻ Tương tác nội tạng hiện đại như `<dialog>`, Popover API, `<details>`, `<progress>`, `<meter>`; kiến trúc Web Components với `<template>` và `<slot>`; bộ Thuộc Tính Toàn Cục (Global Attributes); cùng kỹ thuật tối ưu hóa công cụ tìm kiếm SEO Metadata và Dữ liệu cấu trúc JSON-LD.

7.1 Thẻ Accordion `<details>` & `<summary>`

[Khái Niệm] Thẻ Accordion Thuần HTML5

Thẻ `<details>` và `<summary>` tạo ra widget ẩn/hiện nội dung (Accordion/Collapsible) thuần HTML5 mà không cần dùng JavaScript hay thư viện bên thứ ba.

- `<summary>`: Đóng vai trò là tiêu đề nhấp chọn (kích hoạt mở/đóng).
- Thuộc tính `open`: Đặt trạng thái mặc định mở nội dung khi tải trang.
- Thuộc tính `name="..."`: Gom nhóm các thẻ `<details>` lại với nhau, tự động đóng các ô khác khi một ô mới được mở (chuẩn HTML Living Standard).

Ví dụ danh sách FAQ Accordion với details và summary

```
<details name="faq-group" open>
  <summary><strong>Khoa hoc keo dai trong bao lau?</strong></summary>
  <p>Khoa hoc keo dai trong 3 thang voi 50 bai giang chuyen sau.</p>
</details>

<details name="faq-group">
  <summary><strong>Toi co duoc nhan chung chi khong?</strong></summary>
  <p>Co, ban se nhan duoc chung chi hoan thanh chuan quoc te.</p>
</details>
```

BROWSER PREVIEW — Kết Quả Trình Duyệt: Widget Accordion FAQ

[-] Khóa học kéo dài trong bao lâu? *(Đang mở open)*

Khóa học kéo dài trong 3 tháng với 50 bài giảng chuyên sâu.

[+] Tôi có được nhận chứng chỉ không? *(Đang đóng)*

7.2 Thẻ Cửa Sổ Modal <dialog>

[Khái Niệm] Thẻ Cửa Sổ Hộp Thoại Dialog Native

Thẻ `<dialog>` cung cấp giải pháp chuẩn native để xây dựng cửa sổ Popup / Modal / Hộp thoại xác nhận trong HTML5.

- **Phương thức JS `showModal()`**: Mở dialog ở chế độ Modal chính thức trên lớp trên cùng (**Top Layer**) với phần nền mờ (`::backdrop`).
- **Phương thức JS `close()`**: Đóng cửa sổ hộp thoại.
- **Cấu hình `method="dialog"`**: Khi nằm trong form, submit nút bấm sẽ tự động đóng dialog mà không làm tải lại trang.

Ví dụ tạo Modal Popup chuẩn với thẻ dialog

```
<!-- Nut mo modal -->
<button onclick="document.getElementById('confirmModal').showModal()">Xoa Bai
    Viet</button>

<!-- The dialog modal -->
<dialog id="confirmModal">
  <h3>Xac Nhan Hanh Dong</h3>
  <p>Ban co chac chan muon xoa bai viet nay vinh vien khong?</p>
  <form method="dialog">
    <button value="cancel">Huy Bo</button>
    <button value="confirm" style="color: red;">Xoa Vinh Vien</button>
  </form>
</dialog>
```

BROWSER PREVIEW — Mô Phỏng Trình Duyệt: Hộp Thoại Modal Dialog



7.3 Thẻ Popover API (popover, popovertarget)

[Khái Niệm] Tính Năng Đột Phá Popover API (WHATWG Standard)

Popover API cho phép biến bất kỳ phần tử HTML nào thành Tooltip, Dropdown Menu hoặc Popup nổi hiển thị ở lớp trên cùng (**Top Layer**) mà không bị giới hạn bởi thuộc tính `overflow: hidden` của phần tử cha.

- Thuộc tính `popover="auto|manual"`: Gán tính năng popover cho phần tử.
- Thuộc tính `popovertarget="id"`: Liên kết nút bấm kích hoạt tới ID của khung popover.
- Khởi chạy hoàn toàn thuần HTML5 không cần JavaScript!

Ví dụ Tooltip Popover thuần HTML5 không cần JS

```
<!-- Nut kich hoat Popover -->
<button popovertarget="userTooltip">Xem Thong Tin Tac Gia</button>

<!-- Khung noi dung Popover -->
<div id="userTooltip" popover="auto">
  <h4>Nguyen Trong Tan</h4>
  <p>Chuyen gia Kien truc Frontend & Tac gia sach.</p>
</div>
```

7.4 Thẻ Thước Đo <meter> & Tiến Trình <progress>

[Khái Niệm] Phân Biệt <progress> và <meter>

- <progress>: Biểu diễn tiến trình đang chạy (đang tải tệp, hoàn tất đăng ký...).
- <meter>: Biểu diễn thước đo của một giá trị nằm trong phạm vi xác định (dung lượng ổ đĩa, nhiệt độ, điểm đánh giá).

[Bảng] Bảng Thuộc Tính Của Thẻ meter Và progress

Thuộc Tính	Áp Dụng Cho	Ý Nghĩa
value	<progress>, <meter>	Giá trị hiện tại.
min / max	<progress>, <meter>	Giá trị tối thiểu và tối đa.
low / high / optimum	Chỉ <meter>	Các dải mốc giá trị thấp, cao và tối ưu.

Ví dụ sử dụng thẻ progress và thẻ meter

```
<!-- Thanh tien trinh tai tep 70% -->
<label for="fileDown">Tien trinh tai tep:</label>
<progress id="fileDown" value="70" max="100">70%</progress>

<!-- Thuoc do dung luong o dia -->
<label for="diskSpace">Dung luong o dia:</label>
<meter id="diskSpace" value="85" min="0" max="100" low="30" high="80" optimum=
    "20">85%</meter>
```

BROWSER PREVIEW — Kết Quả Trình Duyệt: Tiến Trình & Thước Đo

Tiến trình tải tệp: 70%

Dung lượng ổ đĩa: 85% (Cảnh báo dung lượng cao)

7.5 Thẻ Mẫu Thụ Động <template> & Thẻ Điểm Chèn <slot>

7.5.1 1. Lý Thuyết & Web Components Architecture

- <template>: Chứa đoạn mã HTML thụ động không dựng hình khi tải trang, chỉ được nhân bản bằng JavaScript. - <slot name="..."></slot>: Điểm chèn nội dung động trong kiến trúc Web Components và Shadow DOM.

7.5.2 2. Ví Dụ Mã Nguồn Thực Tế

```
<template id="user-card-template">
  <style>
    .card { padding: 16px; border: 1px solid #ccc; border-radius: 8px; }
  </style>
  <div class="card">
    <h2><slot name="username">Tên Mặc Định</slot></h2>
    <p><slot name="bio">Mô tả mặc định...</slot></p>
  </div>
</template>
```

Listing 7.1: Ví dụ định nghĩa thẻ template và named slot

7.6 Tất Cả Thuộc Tính Toàn Cục (Global Attributes Masterclass)

7.6.1 1. Lý Thuyết & Bảng Tra Cứu Toàn Bộ Thuộc Tính Toàn Cục

Thuộc tính toàn cục (**Global Attributes**) là các thuộc tính có thể áp dụng cho **tất cả mọi thẻ HTML5** không ngoại lệ.

[Bảng] Bảng Thống Kê

Thuộc Tính	Giá Trị Hợp Lệ	Chức Năng & Tác Dụng
`id`	Mã định danh duy nhất	Đặt ID duy nhất trong trang (Dùng cho CSS/JS/Anchor).
`class`	Danh sách tên lớp	Gom nhóm các phần tử để định kiểu CSS.
`style`	Chuỗi mã CSS	Nhúng mã CSS trực tiếp trong thẻ (Inline Style).
`title`	Chuỗi văn bản	Chú thích bổ sung hiển thị khi di chuột (Tooltip).
`hidden`	`hidden`, `until-found`	Ẩn phần tử khỏi màn hình và Trình đọc màn hình.
`tabindex`	`-1`, `0`, `1+`	Cấu hình thứ tự di chuyển con trỏ Focus phím Tab.
`accesskey`	Ký tự bàn phím	Phím tắt kích hoạt phần tử (`Alt + Key`).
`autofocus`	Boolean	Tự động Focus con trỏ chuột vào thẻ khi nạp trang.
`contenteditable`	`true`, `false`	Cho phép người dùng chỉnh sửa văn bản trực tiếp.
`spellcheck`	`true`, `false`	Kiểm tra lỗi chính tả văn bản.
`inert`	Boolean	Vô hiệu hóa toàn bộ tương tác và focus của vùng phần tử.
`data-*`	Giá trị tùy biến	Lưu trữ dữ liệu tùy biến máy tính đọc được trong `dataset`.

Bảng 7.1: Toàn bộ các thuộc tính toàn cục trong HTML5

7.6.2 2. Ví Dụ Mã Nguồn Thực Tế

```
<!-- Cho phép người dùng trực tiếp sửa văn
    bản --><h1 contenteditable="true" spellcheck="false">Nhấp vào đây để chỉnh sửa
    tiêu đề</h1>

<!-- Khung bị vô hiệu hóa hoàn toàn tương tác với inert --><div inert>
  <button>Nút này không thể nhấp</button>
</div>
```

```
<!-- Lưu trữ dữ liệu tùy biến với data-* --><article data-product-id="9982"
    data-category="laptop" data-price="25000000">
  <h2>Laptop Gaming Pro</h2>
</article>
```

Listing 7.2: Ví dụ ứng dụng các thuộc tính contenteditable, inert và data-*

7.7 Bộ Thẻ Social Media Metadata (Open Graph & Twitter Cards)

7.7.1 1. Lý Thuyết & Thẻ Hiển Thị Xem Trước Khi Chia Sẻ

Bộ thẻ Open Graph (`og:*`) do Facebook phát triển và Twitter Cards (`twitter:*`) điều khiển hình ảnh, tiêu đề và mô tả hiển thị khi người dùng chia sẻ liên kết trang web lên mạng xã hội.

7.7.2 2. Ví Dụ Mã Nguồn Thực Tế

```
<!-- Facebook Open Graph -->
<meta property="og:type" content="article">
<meta property="og:title" content="Khóa Học HTML5 Masterclass 2026">
<meta property="og:description" content="Giáo trình bách khoa toàn thư lập trình Web
    chuyên sâu.">
<meta property="og:image" content="https://eduweb.vn/assets/og-cover.png">
<meta property="og:url" content="https://eduweb.vn/courses/html5">

<!-- Twitter Card -->
<meta name="twitter:card" content="summary_large_image">
<meta name="twitter:title" content="Khóa Học HTML5 Masterclass 2026">
<meta name="twitter:image" content="https://eduweb.vn/assets/og-cover.png">
```

Listing 7.3: Ví dụ bộ thẻ Open Graph chuẩn khi chia sẻ Facebook/Zalo

7.8 Dữ Liệu Cấu Trúc Schema.org JSON-LD

7.8.1 1. Lý Thuyết & Cú Pháp <script type="application/ld+json">

Dữ liệu cấu trúc Schema.org theo chuẩn **JSON-LD** giúp Googlebot hiểu chính xác ngữ nghĩa của trang web (Bài viết, Sản phẩm, Khóa học, FAQ) để hiển thị các kết quả tìm kiếm nổi bật (**Rich Snippets**).

7.8.2 2. Ví Dụ Mã Nguồn Thực Tế

```
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "Course",
  "name": "HTML5 Masterclass Toàn Tập",
  "description": "Giáo trình bách khoa toàn thư thiết kế web hiện đại.",
  "provider": {
    "@type": "Organization",
    "name": "EduWeb Academy",
    "sameAs": "https://eduweb.vn"
```

```
  }
}
</script>
```

Listing 7.4: Ví dụ khai báo dữ liệu cấu trúc Khóa Học chuẩn Google Rich Results

7.9 Kỹ Thuật Bảo Mật Với HTML Header Tags

7.9.1 1. Lý Thuyết & CSP, SRI

- Content Security Policy (CSP): Ngăn chặn tấn công XSS bằng cách quy định các nguồn tài nguyên hợp lệ. - Subresource Integrity (SRI): Bảo vệ tệp CDN khỏi bị sửa đổi mã độc bằng mã băm Hash.

7.9.2 2. Ví Dụ Mã Nguồn Thực Tế

```
<meta http-equiv="Content-Security-Policy"
      content="default-src 'self'; script-src 'self' https://trustedscripts.example.
    com; style-src 'self' 'unsafe-inline';">
```

Listing 7.5: Ví dụ cấu hình Content Security Policy trong thẻ meta

7.10 Tóm Tắt Chương 7

[Khái Niệm] Tóm Lại Cốt Lõi Kiến Thức Chương 7

Chương 7 đã hoàn tất hệ thống Bách Khoa Toàn Thư HTML5 hiện đại với toàn bộ các thẻ Tương tác (`<details>`, `<dialog>`, Popover API, `<meter>`, `<progress>`), kiến trúc Web Components (`<template>`, `<slot>`), bảng tra cứu Thuộc tính toàn cục Global Attributes, cùng bộ thẻ Social Media Metadata và Dữ liệu cấu trúc JSON-LD. Toàn bộ nền tảng HTML5 đã được trang bị vững chắc, chuẩn bị cho Phần 3: CSS3 Thiết Kế Giao Diện & Bố Cục Trang Web.

CHƯƠNG 8

Phụ Lục A: Bách Khoa Tra Cứu Toàn Bộ 110+ Thẻ HTML & Thuộc Tính

Phụ lục này tổng hợp toàn bộ **110+ thẻ HTML** thuộc chuẩn chính thức **WHATWG HTML Living Standard**, phân loại theo nhóm chức năng, liệt kê đầy đủ các thuộc tính hợp lệ (*Attributes*) và mục đích sử dụng.

8.1 1. Các Thẻ Khởi Tạo & Siêu Dữ Liệu (<head>)

[Bảng] Bảng Thống Kê

Thẻ HTML	Thuộc Tính Đặc Thù	Mô Tả CHức Năng & Ý Nghĩa
`<!DOCTYPE>`	không có	Khai báo chuẩn HTML5 kích hoạt Standards Mode.
`<html>`	`lang`, `dir`, `xmlns`	Thẻ gốc tài liệu.
`<head>`	không có	Chứa siêu dữ liệu không hiển thị trực tiếp.
`<title>`	không có	Tiêu đề trang hiển thị trên thẻ trình duyệt.
`<base>`	`href`, `target`	Đặt URL cơ sở cho tất cả liên kết tương đối.
`<meta>`	`charset`, `name`, `content`, `http-equiv`, `media`	Siêu dữ liệu SEO, mã hóa, viewport, CSP.
`<link>`	`rel`, `href`, `sizes`, `type`, `as`, `crossorigin`, `integrity`	Liên kết CSS, Favicon, Resource Hints (preload).
`<style>`	`media`, `blocking`	Nhúng mã CSS nội bộ (Inline CSS).
`<script>`	`src`, `async`, `defer`, `type`, `integrity`, `crossorigin`	Nhúng mã JavaScript, ES Modules, mã băm SRI.
`<noscript>`	không có	Khối nội dung dự phòng khi trình duyệt tắt JS.

Bảng 8.1: Các thẻ cấu trúc gốc và siêu dữ liệu Metadata

8.2 2. Các Thẻ Bố Cục & Ngữ Nghĩa Khối (Sectioning & Structural Semantics)

[Bảng] Bảng Thống Kê

Thẻ HTML	Thuộc Tính Đặc Thù	Mô Tả CHức Năng & Ý Nghĩa
`<body>`	Thuộc tính sự kiện Window	Chứa toàn bộ nội dung trực quan hiển thị.
`<header>`	Thuộc tính toàn cục	Đỉnh trang web hoặc đỉnh phân đoạn/bài viết.
`<nav>`	Thuộc tính toàn cục	Thanh chứa danh sách liên kết điều hướng.
`<main>`	Thuộc tính toàn cục	Khối nội dung chính duy nhất của trang.
`<article>`	Thuộc tính toàn cục	Bài viết độc lập, card sản phẩm, comment.
`<section>`	Thuộc tính toàn cục	Phân đoạn nội dung có cùng chủ đề.
`<aside>`	Thuộc tính toàn cục	Thanh bên phụ, trích dẫn, liên kết xem thêm.
`<footer>`	Thuộc tính toàn cục	Chân trang, bản quyền, liên kết sitemap.
`<address>`	Thuộc tính toàn cục	Thông tin liên hệ tác giả/doanh nghiệp.
`<h1>` đến `<h6>`	Thuộc tính toàn cục	Tiêu đề phân tầng từ bậc 1 đến bậc 6.
`<hgroup>`	Thuộc tính toàn cục	Nhóm tiêu đề chính và tiêu đề phụ.
`<search>`	Thuộc tính toàn cục	Khối chứa công cụ tìm kiếm (WHATWG mới).

Bảng 8.2: Các thẻ ngữ nghĩa cấu trúc bố trúc trang web

8.3 3. Thẻ Định Dạng Văn Bản Nội Dòng (Inline Text Semantics)

[Bảng] Bảng Thống Kê

Thẻ HTML	Thuộc Tính Đặc Thù	Mô Tả CHức Năng & Ý Nghĩa
`<a>`	`href`, `target`, `download`, `rel`, `ping`, `referrerpolicy`	Liên kết siêu văn bản, tải file.
`<strong>`	Thuộc tính toàn cục	Văn bản có tầm quan trọng cao.
`<em>`	Thuộc tính toàn cục	Nhấn mạnh ngữ điệu phát âm.
`<small>`	Thuộc tính toàn cục	Chữ nhỏ chú thích pháp lý/bản quyền.
`<s>`	Thuộc tính toàn cục	Văn bản bị hủy bỏ hoặc hết hạn.
`<cite>`	Thuộc tính toàn cục	Nguồn trích dẫn tác phẩm/tài liệu.
`<q>`	`cite`	Trích dẫn ngắn trong dòng.
`<dfn>`	Thuộc tính toàn cục	Thuật ngữ đang được định nghĩa.
`<abbr>`	`title`	Từ viết tắt kèm giải nghĩa đầy đủ.
`<time>`	`datetime`	Đánh dấu mốc thời gian ISO-8601.
`<data>`	`value`	Gắn giá trị mã số đọc bởi máy tính.
`<code>`	Thuộc tính toàn cục	Đoạn mã máy tính nội dòng.
`<kbd>`	Thuộc tính toàn cục	Phím bấm bàn phím.
`<var>`	Thuộc tính toàn cục	Biến số toán học hoặc lập trình.
`<samp>`	Thuộc tính toàn cục	Kết quả đầu ra mẫu từ máy tính.
`<sub>`, `<sup>`	Thuộc tính toàn cục	Chỉ số dưới (H_2O) và chỉ số trên ($E = mc^2$).
`<mark>`	Thuộc tính toàn cục	Tô sáng đoạn văn bản được tìm kiếm.
`<bdi>`, `<bdo>`	`dir`	Xử lý hướng đọc văn bản RTL/LTR.
`<ruby>`, `<rt>`, `<rp>`	Thuộc tính toàn cục	Chú thích phát âm chữ Á Đông (Ruby).
`<span>`	Thuộc tính toàn cục	Thẻ inline vô nghĩa dùng để CSS/JS.
` `, `<wbr>`	Thuộc tính toàn cục	Ngắt dòng và điểm ngắt từ linh hoạt.

Bảng 8.3: Tất cả các thẻ văn bản nội dòng và thuộc tính đi kèm

8.4 4. Thẻ Form & Điều Khiển Đầu Vào (Forms & Controls)

[Bảng] Bảng Thống Kê

Thẻ HTML	Thuộc Tính Đặc Thù	Mô Tả CHức Năng & Ý Nghĩa
`<form>`	`action`, `method`, `enctype`, `autocomplete`, `novalidate`, `target`	Khung chứa biểu mẫu gửi dữ liệu.
`<input>`	22+ `type`, `name`, `value`, `required`, `pattern`, `min`, `max`, `step`, `placeholder`, `inputmode`, `enterkeyhint`, `popovertarget`	Trường nhập dữ liệu đa dạng.
`<button>`	`type`, `disabled`, `form`, `popovertarget`, `popovertargetaction`	Nút bấm thực thi hành động.
`<select>`	`name`, `multiple`, `size`, `disabled`, `required`	Danh sách chọn xổ xuống (Dropdown).
`<optgroup>`	`label`, `disabled`	Nhóm các tùy chọn trong select.
`<option>`	`value`, `selected`, `disabled`, `label`	Một phần tử tùy chọn trong select.
`<textarea>`	`rows`, `cols`, `maxlength`, `placeholder`, `wrap`	Khung nhập văn bản nhiều dòng.
`<label>`	`for`	Nhãn dán mô tả ô nhập liệu.
`<fieldset>`	`disabled`, `form`, `name`	Khung gom nhóm các trường form.
`<legend>`	Thuộc tính toàn cục	Tiêu đề cho khối fieldset.
`<datalist>`	không có	Danh sách gợi ý tự động cho input.
`<output>`	`for`, `form`, `name`	Hiển thị kết quả tính toán form.

Bảng 8.4: Toàn bộ thẻ điều khiển Form và thuộc tính nhập liệu

8.5 5. Đa Phương Tiện, Graphics & Web Components

[Bảng] Bảng Thống Kê

Thẻ HTML	Thuộc Tính Đặc Thù	Mô Tả CHức Năng & Ý Nghĩa
`<img>`	`src`, `alt`, `srcset`, `sizes`, `loading`, `decoding`, `fetchpriority`	Hiển thị hình ảnh đa thiết bị.
`<picture>`	không có	Khung chứa các nguồn ảnh thích ứng.
`<source>`	`src`, `srcset`, `type`, `media`, `sizes`	Nguồn tài nguyên cho picture/video.
`<video>`	`src`, `controls`, `autoplay`, `muted`, `loop`, `poster`, `preload`, `playsinline`	Phát trình trình chiếu video gốc.
`<audio>`	`src`, `controls`, `autoplay`, `muted`, `loop`, `preload`	Phát âm thanh mp3/wav/ogg.
`<track>`	`kind`, `src`, `srclang`, `label`, `default`	Nhúng phụ đề chuẩn WebVTT.
`<iframe>`	`src`, `srcdoc`, `sandbox`, `allow`, `loading`, `width`, `height`	Nhúng khung trang web bên ngoài.
`<svg>`	`viewBox`, `width`, `height`, `xmlns`	Đồ họa Vector nhị thức Inline.
`<canvas>`	`width`, `height`	Khung vẽ đồ họa 2D / WebGL.
`<math>`	`xmlns`	Nhúng công thức toán học MathML.
`<template>`	không có	Khối mã HTML thụ động dùng cho JS.
`<slot>`	`name`	Điểm chèn dữ liệu Shadow DOM.
`<details>`	`open`, `name`	Khung Accordion ẩn/hiện nội dung.
`<summary>`	Thuộc tính toàn cục	Tiêu đề cho khối details.
`<dialog>`	`open`	Cửa sổ Popup Modal gốc trình duyệt.

Bảng 8.5: Các thẻ nhúng tài nguyên đa phương tiện và kỹ thuật mới

8.6 6. Tất Cả Thuộc Tính Toàn Cục (Global Attributes)

Mọi thẻ HTML trong tài liệu đều có quyền sử dụng các **Thuộc tính Toàn Cục (Global Attributes)** sau:

- `id`: Định danh duy nhất cho phần tử trong toàn bộ cây DOM.
- `class`: Tên lớp định kiểu cho CSS và truy vấn bằng JavaScript.
- `style`: Viết mã CSS trực tiếp trên phần tử (Inline style).
- `title`: Văn bản trợ giúp hiển thị dạng chú thích khi di chuột vào (Tooltip).
- `lang`: Khai báo ngôn ngữ riêng cho phần tử (ví dụ: `lang="en"`).
- `dir`: Hướng đọc văn bản (`ltr`, `rtl`, `auto`).

- `hidden`: Ẩn phần tử khỏi màn hình hoàn toàn (tương đương `display: none`).
- `tabindex`: Thứ tự nhận focus bàn phím khi ấn phím Tab.
- `accesskey`: Gán phím tắt kích hoạt nhanh phần tử từ bàn phím.
- `autofocus`: Tự động trỏ con trỏ vào phần tử khi trang vừa tải xong.
- `contenteditable`: Cho phép người dùng chỉnh sửa trực tiếp nội dung thẻ trên màn hình.
- `spellcheck`: Bật/Tắt kiểm tra lỗi chính tả ngôn ngữ của trình duyệt.
- `inputmode`: Bật đúng bàn phím di động thích hợp.
- `enterkeyhint`: Đổi nhãn nút Enter trên bàn phím di động.
- `inert`: Vô hiệu hóa toàn bộ tương tác bàn phím, chuột và trình đọc màn hình.
- `popover`: Biến phần tử thành khung Popover hiển thị ở lớp trên cùng (Top Layer).
- `data-*`: Thuộc tính lưu trữ dữ liệu tùy chỉnh riêng của ứng dụng (truy cập qua `dataset`).

PHẦN III

CSS3 – Thiết Kế Giao Diện & Bố Cục Trang Web

CHƯƠNG 9

Nền Tảng CSS: Cascading Style Sheets, Box Model & Thuộc Tính Outline

9.1 Sơ Lược & Khái Niệm Về CSS

CSS (**Cascading Style Sheets**) giúp bạn thiết kế và định dạng giao diện trang web. Nó quyết định mọi thứ về hình thức hiển thị, từ màu sắc, kích thước chữ đến bố cục và hiệu ứng.

[Khái Niệm] Khái Niệm CSS Là Gì?

CSS (Cascading Style Sheets) là ngôn ngữ dùng để mô tả cách trình bày của một tài liệu HTML hoặc XML. Nó giúp định dạng màu sắc, kiểu chữ, bố cục và hiệu ứng cho trang web. CSS giúp **tách biệt phần nội dung (HTML) với phần hiển thị**, giúp trang web dễ quản lý, tăng tính bảo trì và tối ưu hiệu suất tải trang.

9.1.1 Mục Đích Cốt Lõi Của Ngôn Ngữ CSS

1. **Tách biệt nội dung và trình bày**: Thẻ HTML tập trung vào nội dung ngữ nghĩa, CSS lo phần thiết kế giao diện thị giác.
2. **Giúp trang web đẹp và chuyên nghiệp**: Mang lại giao diện hiện đại, nâng cao trải nghiệm thị giác.
3. **Dễ bảo trì và tái sử dụng**: Một tập tin CSS duy nhất có thể dùng và định kiểu đồng bộ cho nhiều trang web.
4. **Cải thiện trải nghiệm người dùng và tối ưu tốc độ tải trang**: Giảm kích thước file HTML, tối ưu hóa bộ nhớ đệm trình duyệt (**browser cache**).

9.2 Cú Pháp CSS (CSS Syntax)

9.2.1 Cấu Trúc Cơ Bản Của Cú Pháp CSS

Một quy tắc CSS bao gồm hai thành phần chính:

- **Bộ chọn (Selector)** – Xác định phần tử HTML cần áp dụng CSS.
- **Khối khai báo (Declaration Block)** – Chứa các thuộc tính và giá trị đặt trong cặp dấu ngoặc nhọn `{}`.

Cú Pháp Định Dạng CSS Chuẩn

```
selector {
    thuoc_tinh: gia_tri;
    thuoc_tinh: gia_tri;
}
```

9.2.2 Ví Dụ Cụ Thể & Giải Thích Cú Pháp

Khai Báo Style Cho Thẻ h1

```
h1 {
    color: red;
    font-size: 24px;
}
```

BROWSER PREVIEW — **Kết Quả Hiển Thị: Thẻ h1 Chữ Đỏ, Cỡ Chữ 24px**

Tiêu Đề Văn Bản h1

(→ Kết quả: Thẻ <h1> có chữ màu đỏ và kích thước 24px)

[Ghi Chú] Giải Thích Chi Tiết Cú Pháp

- `h1` → Chọn tất cả thẻ `<h1>` trên trang.
- `color: red;` → Thiết lập màu chữ đỏ.
- `font-size: 24px;` → Thiết lập cỡ chữ `24px`.

9.3 Các Cách Thêm Style Vào Trang Web

CSS có thể được áp dụng vào HTML theo 3 cách:

[Bảng] Bảng So Sánh 3 Cách Thêm Style Vào Trang Web

Cách viết	Mô tả	Ưu điểm	Nhược điểm
`Inline CSS`	Viết trực tiếp vào thẻ HTML bằng thuộc tính `style`.	**Dễ dùng, nhanh chóng.**	**Khó bảo trì, không tái sử dụng được.**
`Internal CSS`	Viết trong thẻ `<style>` trong file HTML.	**Dễ chỉnh sửa toàn bộ trang, không cần file ngoài.**	**Không tách biệt nội dung và kiểu dáng.**
`External CSS`	Link tới file `.css` bên ngoài qua thẻ `<link>`.	**Tái sử dụng cao, dễ bảo trì.**	**Phải tải thêm file, trang load lâu hơn chút.**

9.3.1 Inline CSS (CSS Trong Thẻ HTML)

Viết trực tiếp vào thẻ HTML bằng thuộc tính `style`.

Mã Nguồn Inline CSS

```html
<h1 style="color: red; font-size: 30px;">Header Title</h1>
```

BROWSER PREVIEW — **Kết Quả Hiển Thị Inline CSS**



→ **Hạn chế**: **Không khuyến khích** vì làm code khó bảo trì.

9.3.2 Internal CSS (CSS Nội Bộ)

Viết trong thẻ `<style>` trong file HTML (bên trong `<head>`). Dùng khi chỉ áp dụng CSS cho một trang web duy nhất.

Mã Nguồn Internal CSS

```html
<head>
  <style>
    h1 {
      color: blue;
      font-size: 24px;
    }
  </style>
</head>
```

BROWSER PREVIEW — **Kết Quả Hiển Thị Internal CSS**

Tiêu Đề Xanh Dương

9.3.3 External CSS (CSS Bên Ngoài)

Viết CSS trong file `.css` riêng biệt, sau đó liên kết vào HTML bằng thẻ `<link>`.

Liên Kết File CSS Qua Thẻ Link

```html
<head>
    <link rel="stylesheet" href="style.css">
</head>
```

Nội Dung Trong File style.css

```css
/* Sample Text */
p {
    color: green;
    font-size: 16px;
}
```

BROWSER PREVIEW — **Kết Quả Hiển Thị External CSS**

Đoạn văn bản màu xanh lá được định kiểu từ file style.css bên ngoài.

→ **Lời khuyên**: **Khuyến khích sử dụng** vì giúp code gọn gàng và dễ bảo trì.

9.4 Một Số Thuộc Tính CSS Cơ Bản & Đơn Vị Đo Kích Thước

9.4.1 Bảng Tra Cứu Một Số Thuộc Tính CSS Cơ Bản

[Bảng] Bảng Tra Cứu Các Thuộc Tính CSS Cơ Bản Phổ Biến

Thuộc tính	Chức năng	Ví dụ
`color`	Đặt màu chữ.	`color: red;`
`font-size`	Đặt kích thước chữ.	`font-size: 20px;`
`background`	Đặt màu nền hoặc ảnh nền.	`background: #f0f0f0;`
`border`	Tạo viền cho phần tử.	`border: 2px solid black;`
`padding`	Khoảng cách bên trong phần tử.	`padding: 10px;`
`margin`	Khoảng cách bên ngoài phần tử.	`margin: 20px;`
`text-align`	Căn chỉnh văn bản.	`text-align: center;`
`display`	Quyết định kiểu hiển thị của phần tử.	`display: flex;`

Ví Dụ Kết Hợp Thuộc Tính Định Kiểu Cho Thẻ p:

Ví Dụ Định Kiểu Thẻ p Tổng Hợp

```
p {
    color: green;
    font-size: 18px;
    background: #e0f7fa;
    padding: 10px;
    border: 1px solid #00796b;
}
```

BROWSER PREVIEW — **Kết Quả Hiển Thị Của Thẻ p Định Kiểu**

> **Đoạn văn có chữ màu xanh lá, nền xanh nhạt (#e0f7fa), viền xanh đậm (#00796b), khoảng cách nội dung padding 10px.**

→ **Kết quả**: Đoạn văn sẽ có chữ màu xanh lá, nền xanh nhạt, viền xanh đậm, khoảng cách nội dung 10px.

b. Các Đơn Vị Đo Trong CSS:

[Bảng] Bảng Tra Cứu Đơn Vị Đo Trong CSS

Đơn vị	Ý nghĩa
`px`	Pixel – Kích thước cố định.
`em`	Tương đối với `font-size` của phần tử cha.
`rem`	Tương đối với `font-size` của thẻ `<html>`.
`%`	Phần trăm so với phần tử cha.
`vw / vh`	Chiều rộng / chiều cao của viewport.

9.4.2 Chuyên Đề Hệ Thống Màu Sắc Trong CSS (CSS Colors Masterclass Từ A đến Z)

Màu sắc là yếu tố sống còn quyết định tính thẩm mỹ, khả năng truyền tải thông điệp, trải nghiệm người dùng (**UX**) và khả năng truy cập (**Accessibility - a11y**) của mọi trang web. Trong CSS, lập trình viên có thể biểu diễn màu sắc qua nhiều hệ thống biểu diễn khác nhau từ đơn giản đến nâng cao.

1. Mục Đích Và Vai Trò Của Màu Sắc Trong CSS:

[Bảng] Bảng Phân Tích Vai Trò Của Màu Sắc Trên Giao Diện Web

Thành phần giao diện	Vai trò màu sắc đảm nhận	Mục tiêu trải nghiệm (UX)
Văn bản (Text)	Định kiểu chữ chính, chữ phụ, tiêu đề.	Tăng độ đọc (**Readability**), phân cấp thông tin rõ ràng.
Nút bấm & Liên kết	Nhấn mạnh các điểm tương tác chính (CTA).	Hướng người dùng thực hiện hành động chính.
Nền (Background)	Phân tách khối nội dung, tạo khoảng không.	Tạo bố cục cân đối, dễ chịu cho mắt.
Cảnh báo & Thông báo	Thể hiện trạng thái hệ thống.	Báo lỗi (Đỏ), Cảnh báo (Vàng), Thành công (Xanh lá).

2. Phân Loại & Tổng Quan Các Kiểu Màu Trong CSS:

[Bảng] Bảng Tổng Quan Chi Tiết Các Kiểu Biểu Diễn Màu CSS

Loại màu	Cú pháp	Ví dụ minh họa	Đặc tính & Ghi chú
`Color Name`	`tên_màu`	`color: red;`	Có sẵn 147 tên màu chuẩn HTML/CSS.
`Hexadecimal`	`#RRGGBB`	`color: #ff0000;`	Mã 6 ký tự Hex, chính xác tuyệt đối.
`Shorthand Hex`	`#RGB`	`color: #f00;`	Rút gọn 3 ký tự khi từng cặp trùng nhau.
`RGB`	`rgb(R, G, B)`	`rgb(255, 0, 0)`	Dải giá trị $0 \to 255$ cho 3 kênh Red, Green, Blue.
`RGBA`	`rgba(R, G, B, A)`	`rgba(255, 0, 0, 0.5)`	Thêm kênh Alpha độ trong suốt $(0 \to 1)$.
`HSL`	`hsl(H, S%, L%)`	`hsl(0, 100%, 50%)`	Hue (góc màu), Saturation (độ bão hòa), Lightness (độ sáng).
`HSLA`	`hsla(H, S%, L%, A)`	`hsla(0, 100%, 50%, 0.5)`	Thêm kênh Alpha độ trong suốt $(0 \to 1)$.
`Transparent`	`transparent`	`background: transparent;`	Từ khóa đặc biệt tạo độ trong suốt tuyệt đối (100%).
`CurrentColor`	`currentColor`	`border-color: currentColor;`	Từ khóa đặc biệt tự động kế thừa màu chữ (`color`) hiện tại.

3. Chi Tiết Từng Hệ Thống Biểu Diễn Màu Sắc:

3.1. Color Name (Tên Màu Định Danh Trong CSS):

[Khái Niệm] Khái Niệm Cơ Bản Về Tên Màu Định Danh (Color Name)

1. Color Name Là Gì?:

- **Color Name**: Là tên màu sắc đã được định nghĩa sẵn trong tiêu chuẩn CSS. Người lập trình chỉ cần gọi trực tiếp tên tiếng Anh của màu sắc đó mà không cần sử dụng mã Hex hay hàm toán học nào (không cần dấu #, không cần đối số RGB).
- **Số lượng màu chuẩn**: CSS hỗ trợ tổng cộng **147 tên màu chuẩn** được quy định chính thức bởi W3C.

2. Cú Pháp Sử Dụng Color Name trong CSS:

Cú Pháp Định Kiểu Bằng Tên Màu Định Danh

```
/* Cu phap tong quat */
element {
  color: <color-name>;
  background-color: <color-name>;
}

/* Vi du thuc te */
p {
  color: tomato;
  background-color: lightgray;
}
```

3. Phân Nhóm Các Tên Màu Phổ Biến Trong CSS:

[Bảng] Bảng Phân Nhóm Các Tên Màu Từ Khóa Thường Dùng

Nhóm màu	Ví dụ tên màu định sẵn	Đặc điểm & Cảm giác thị giác
Màu Đỏ (Red)	`red, crimson, tomato, firebrick`	Ấm áp, nổi bật, dùng cho cảnh báo hoặc điểm nhấn.
Màu Xanh lá (Green)	`green, lime, olive, seagreen`	Tươi mát, tự nhiên, trạng thái thành công.
Màu Xanh dương (Blue)	`blue, navy, skyblue, dodgerblue`	Tin cậy, chuyên nghiệp, liên kết và điều hướng.
Màu Trung tính	`black, white, gray, silver`	Màu văn bản chính, màu nền và đường viền phụ.
Màu Vàng / Cam	`yellow, gold, orange, khaki`	Năng động, rực rỡ, điểm nhấn lưu ý nhẹ.
Màu Tím (Purple)	`purple, violet, indigo, orchid`	Sang trọng, sáng tạo, thương hiệu hiện đại.

4. Các Ví Dụ Mã Nguồn Thực Tế Với Color Name:

Ví Dụ 1: Định Kiểu Thẻ Bài Viết Tin Tức (Article Card) Bằng Color Name

```
<!-- Structure HTML -->
<div class="article-card">
  <h2 class="title">Tin Tuc Cong Nghe Moi Nhat</h2>
  <p class="desc">Kham pha cac tinh nang CSS3 hien dai giup toi uu giao dien Web.
    </p>
  <span class="tag">HOT</span>
</div>

<!-- Styling CSS -->
<style>
.article-card {
  background-color: whitesmoke;
  border: 2px solid navy;
  padding: 16px;
  border-radius: 8px;
}
.title {
  color: crimson;
  font-size: 18px;
}
.desc {
  color: darkslategrey;
}
.tag {
  background-color: darkred;
  color: white;
  padding: 4px 8px;
}
</style>
```

Ví Dụ 2: Định Kiểu Huy Hiệu Trạng Thái Hệ Thống (Notification Badges)

```css
/* Huy hieu thanh cong */
.badge-success {
  background-color: seagreen;
  color: white;
}

/* Huy hieu loi khan cap */
.badge-error {
  background-color: firebrick;
  color: white;
}

/* Huy hieu canh bao */
.badge-warning {
  background-color: darkorange;
  color: black;
}
```

BROWSER PREVIEW **Kết Quả Hiển Thị Của Thẻ Bài Viết & Huy Hiệu Trạng Thái Bằng Color Name**

Tin Tức Công Nghệ Mới Nhất (`color: crimson;`)

Khám phá các tính năng CSS3 hiện đại giúp tối ưu giao diện Web. (`color: darkslategrey;`)

THÀNH CÔNG (seagreen)

LỖI HỆ THỐNG (firebrick)

CẢNH BÁO (darkorange)

5. Đánh Giá Ưu Điểm Và Hạn Chế Của Color Name:

[Bảng] Bảng So Sánh Ưu Điểm Và Hạn Chế Của Tên Màu Định Danh

Ưu điểm (Đúng)	Hạn chế (Không)
Cực kỳ dễ nhớ, dễ viết: Đọc tên tiếng Anh là viết được ngay.	**Rất ít tùy biến**: Bị giới hạn cố định trong 147 tên màu.
Không cần tính toán: Không phải ghi nhớ hay chuyển đổi mã Hex/RGB.	**Không có kênh Alpha**: Không hỗ trợ tạo độ mờ trong suốt.
Tương thích tuyệt đối: Hỗ trợ 100% trên tất cả trình duyệt web.	**Thiếu chính xác**: Không đáp ứng được các thiết kế UI/UX phức tạp.

6. Danh Sách Một Số Tên Màu Đẹp & Phổ Biến NÊN Biết:

[Bảng] Bảng Tra Cứu Danh Sách Tên Màu Đẹp & Mã HEX Tương Đương

Tên màu (Color Name)	Mã HEX tương đương	Mô tả sắc thái màu
`tomato`	`#FF6347`	Màu cam đỏ tươi tắn, năng động.
`dodgerblue`	`#1E90FF`	Màu xanh lam sáng rực rỡ, hiện đại.
`gold`	`#FFD700`	Màu vàng ánh kim sang trọng.
`slategray`	`#708090`	Màu xám xanh trung tính dịu mắt.
`darkorchid`	`#9932CC`	Màu tím lan đậm sáng tạo.
`lightcoral`	`#F08080`	Màu hồng san hô nhạt nữ tính.
`indigo`	`#4B0082`	Màu tím chàm sâu thẳm.

7. Hướng Dẫn Ra Quyết Định: Lưu Ý Khi Dùng Color Name:

[Mẹo] Quy Tắc Sử Dụng Tên Màu Trong Dự Án Web

- **Không phân biệt chữ hoa / chữ thường**: CSS không phân biệt hoa thường với tên màu. Các cách viết `red`, `Red`, `RED` đều cho kết quả hoàn toàn giống nhau.
- **Nên dùng khi**: Dựng khung demo nhanh, tạo ví dụ giảng dạy đơn giản hoặc thử nghiệm ý tưởng giao diện sơ khởi.
- **Không nên dùng khi**: Thiết kế giao diện sản phẩm chuyên nghiệp (Figma/Adobe), cần độ chính xác cao theo bộ nhận diện thương hiệu, hoặc khi cần độ mờ trong suốt (Alpha) và dải chuyển màu (Gradient).

3.2. Hệ Màu HEX Trong CSS (Hexadecimal Color Model):

[Khái Niệm] Khái Niệm Cơ Bản Về Hệ Màu HEX (Hexadecimal)

1. HEX Là Gì?:

- **HEX (viết tắt của Hexadecimal)**: Là hệ màu biểu diễn giá trị màu sắc bằng hệ số **Thập lục phân** (cơ số 16).
- **Cấu trúc tổng quát**: Một mã màu HEX chuẩn có dạng: `#RRGGBB`. Trong đó:
  - **RR**: Giá trị thuộc về kênh màu Đỏ (**Red**).
  - **GG**: Giá trị thuộc về kênh màu Xanh lá (**Green**).
  - **BB**: Giá trị thuộc về kênh màu Xanh dương (**Blue**).
- **Bản chất kênh màu**: Mỗi cặp giá trị (RR, GG, BB) gồm 2 chữ số thập lục phân ($00 \rightarrow$ FF), tương đương với giá trị từ 0 đến 255 trong hệ thập phân (Decimal).

2. Cách Đọc Và Giải Mã Một Mã Màu HEX: Ví dụ phân tích mã màu `#FF5733`:

Thành phần	Giá trị HEX	Giá trị thập phân	Giải thích mức độ phát sáng
Red (RR)	FF	$15 \times 16^1 + 15 \times 16^0 = \mathbf{255}$	Cường độ phát sáng màu Đỏ cực đại (100%).
Green (GG)	57	$5 \times 16^1 + 7 \times 16^0 = \mathbf{87}$	Cường độ màu Xanh lá trung bình (34%).
Blue (BB)	33	$3 \times 16^1 + 3 \times 16^0 = \mathbf{51}$	Cường độ màu Xanh dương thấp (20%).

$\rightarrow$ Kết quả phối hợp 3 ánh sáng tạo ra màu **Cam Đỏ (Orange-Red)** rực rỡ và ấm áp.

3. Cú Pháp Khai Báo Chuẩn Trong CSS & Ví Dụ Minh Họa:

Cú Pháp Định Dạng Mã Màu HEX Trong CSS

```
/* Cu phap chung */
color: #RRGGBB;

/* Cac mau co ban pho bien */
.black-text { color: #000000; } /* Den tuyen (R=0, G=0, B=0) */
.white-text { color: #ffffff; } /* Trang tinh (R=255, G=255, B=255) */
.red-text   { color: #ff0000; } /* Do thuan (R=255, G=0, B=0) */
.green-text { color: #00ff00; } /* Xanh la thuan (R=0, G=255, B=0) */
.blue-text  { color: #0000ff; } /* Xanh duong thuan (R=0, G=0, B=255) */
```

BROWSER PREVIEW Kết Quả Hiển Thị Trực Quan Của Các Mã Màu HEX Cơ Bản

.black-text (#000000): Chữ màu Đen tuyền tuyệt đối

.white-text (#ffffff): Chữ màu Trắng tinh nét trên nền xám nhẹ

.red-text (#ff0000): Chữ màu Đỏ thuần tươi tắn

.green-text (#00ff00): Chữ màu Xanh lá thuần chuẩn

.blue-text (#0000ff): Chữ màu Xanh dương thuần đậm

4. Cú Pháp Viết Gọn Mã HEX (Shorthand Hex Syntax):

[Khái Niệm] Quy Trình Rút Gọn Mã Màu HEX

- **Điều kiện rút gọn**: Nếu mỗi cặp ký tự đại diện cho từng kênh màu hoàn toàn giống nhau (`#AABBCC` trong đó AA, BB, CC), bạn có thể rút gọn mã Hex 6 ký tự thành mã Hex 3 ký tự `#ABC`.
- **Công thức quy đổi**: `#ABC` $\Leftrightarrow$ `#AABBCC`.

[Bảng] Bảng So Sánh Các Mã HEX Đầy Đủ Và Dạng Viết Gọn

Viết đầy đủ (6 ký tự)	Viết gọn (3 ký tự)	Hợp lệ?	Ghi chú chi tiết
`#FFFFFF`	`#FFF`	Đúng	Cả 3 cặp FF, FF, FF đều trùng ký tự.
`#000000`	`#000`	Đúng	Cả 3 cặp 00, 00, 00 đều trùng ký tự.
`#FF0000`	`#F00`	Đúng	Tương ứng FF → F, 00 → 0, 00 → 0.
`#112233`	`#123`	Đúng	Tương ứng 11 → 1, 22 → 2, 33 → 3.
`#123456`	`Không thể viết gọn`	Không	Các cặp 12, 34, 56 không có ký tự trùng lặp trong cặp!

[Cảnh Báo] Lưu Ý Quan Trọng Khi Viết Gọn Mã HEX

Bạn chỉ được phép rút gọn mã Hex khi **từng cặp ký tự đơn lẻ** trùng nhau (ví dụ: `FF`, `00`, `33`). Tuyệt đối không được rút gọn nếu hai ký tự trong cùng một cặp khác nhau (ví dụ: `#FF5733` không thể viết gọn thành `#F53`).

5. Danh Sách Các Mã Màu HEX Phổ Biến Trong Thiết Kế Web:

[Bảng] Bảng Tra Cứu Danh Sách Mã Màu HEX Phổ Biến

Tên màu	Mã HEX	Mã RGB tương đương	Gợi ý ứng dụng thực tế
Đen (Black)	`#000000`	`rgb(0, 0, 0)`	Màu tối hoàn toàn, màu chữ chính trên nền sáng.
Trắng (White)	`#FFFFFF`	`rgb(255, 255, 255)`	Màu sáng hoàn toàn, màu nền trang chính.
Đỏ (Red)	`#FF0000`	`rgb(255, 0, 0)`	RGB Full Red – Thể hiện trạng thái lỗi khẩn cấp.
Xanh lá (Green)	`#00FF00`	`rgb(0, 255, 0)`	RGB Full Green – Trạng thái thành công, hoàn thành.
Xanh dương (Blue)	`#0000FF`	`rgb(0, 0, 255)`	RGB Full Blue – Liên kết (Link) và thông tin.
Cam (Orange)	`#FFA500`	`rgb(255, 165, 0)`	Nút cảnh báo nhẹ, nút hành động nhấn mạnh.
Vàng (Yellow)	`#FFFF00`	`rgb(255, 255, 0)`	Kết hợp Red + Green – Điểm nhấn lưu ý.
Tím (Purple)	`#800080`	`rgb(128, 0, 128)`	Kết hợp Red + Blue – Thương hiệu sáng tạo.

6. Thuật Toán Chuyển Đổi Giữa Mã HEX Và Hệ RGB:

[Khái Niệm] Quy Trình Chuyển Đổi Mã HEX Sang RGB

Chuyển đổi mã `#FF5733` sang cú pháp `rgb(...)`:

1. Tách mã Hex thành 3 cặp byte: Red $= \text{FF}_{16}$, Green $= 57_{16}$, Blue $= 33_{16}$.
2. Đổi từ hệ Hex (cơ số 16) sang hệ Thập phân (cơ số 10):
   - Kênh Red (FF_{16}): $\rightarrow$ **255**.
   - Kênh Green (57_{16}): $\rightarrow$ **87**.
   - Kênh Blue (33_{16}): $\rightarrow$ **51**.
3. Kết quả cú pháp tương đương: $\rightarrow$ `rgb(255, 87, 51)`.

Mẹo nhỏ: Bạn có thể tra cứu và chuyển đổi nhanh chóng tại trang web `https://www.colorhexa.com`.

7. So Sánh Chi Tiết Giữa Mã HEX, RGB Và RGBA:

[Bảng] Bảng So Sánh Toàn Diện Giữa Mã HEX, RGB Và RGBA

Tiêu chí	Mã HEX (#RRGGBB)	Hệ RGB (rgb(r,g,b))	Hệ RGBA (rgba(r,g,b,a))
Độ dễ nhớ	Khó ghi nhớ bằng mắt thường.	Dễ hiểu trực quan theo giá trị số từ $0 \rightarrow 255$.	Dễ hiểu trực quan theo giá trị số từ $0 \rightarrow 255$.
Dạng viết gọn	**Có hỗ trợ** (mã 3 ký tự `#FFF`).	**Không** (phải viết đầy đủ 3 tham số).	**Không** (phải viết đầy đủ 4 tham số).
Kênh Alpha (Trong suốt)	**Không** (ở chuẩn truyền thống 6 ký tự).	**Không** (chỉ màu đặc 100%).	**Có hỗ trợ** (có kênh Alpha $0.0 \rightarrow 1.0$).
Thân thiện thiết kế	Rất phổ biến, xuất chuẩn từ Figma / Adobe.	Dùng nhiều khi tính toán màu động trong JavaScript.	Hoàn hảo cho hiệu ứng Overlay, bóng mờ.

8. Hướng Dẫn Ra Quyết Định: Khi Nào Nên Và Không Nên Dùng HEX?

[Khái Niệm] Ma Trận Hướng Dẫn Trường Hợp Sử Dụng Mã HEX

- **Nên dùng HEX khi (Khuyên dùng)**:
  - Cần chọn màu tĩnh nhanh chóng cho các phần tử UI cố định.
  - Làm việc trực tiếp với file thiết kế giao diện (Figma, Sketch, Adobe XD).
  - Ghi mã màu ngắn gọn để tối ưu dung lượng file CSS nhỏ gọn.
- **Không nên dùng HEX khi (Hạn chế)**:
  - Cần tạo màu nền bán trong suốt (**Alpha opacity**) cho Modal/Popup.
  - Cần thay đổi màu sắc động tăng giảm mượt mà bằng lập trình JavaScript.
  - Cần hòa trộn dải màu (**Overlay Blend Mode**) dịu mắt với hình ảnh nền bên dưới.

9. Cơ Chế Hoạt Động Nội Bộ Của Máy Tính & Trình Duyệt Khi Xử Lý HEX:

[Khái Niệm] Quy Trình Xử Lý Nội Bội Của Trình Duyệt Khi Gặp Mã HEX

Các bước xử lý từ mã nguồn đến mắt người xem:

1. **Người lập trình**: Khai báo mã màu trong CSS (ví dụ: `color: #FF5733;`).
2. **Trình duyệt Web**: Giải mã chuỗi Hex 16 sang hệ RGB 10: FF $\rightarrow 255, 57 \rightarrow 87, 33 \rightarrow 51$.
3. **Hệ thống phần cứng (Màn hình)**: Truyền tín hiệu điện điều khiển 3 điểm ảnh phụ (**Subpixels**) Red, Green, Blue phát sáng theo đúng tỷ lệ cường độ.
4. **Mắt người xem**: Mắt tiếp nhận chùm ánh sáng cộng phát ra từ màn hình và cảm nhận được màu Cam Đỏ.

Mở rộng chuyên sâu – Mã HEX 8 Ký Tự (CSS Color Module Level 4): Trong tiêu chuẩn CSS hiện đại, bạn có thể bổ sung kênh Alpha trực tiếp vào mã Hex bằng cú pháp 8 ký tự `#RRGGBBAA` (ví dụ: `#FF573380` tương đương với độ mờ 50%).

10. Ví Dụ Thực Tế Component UI Với Mã Màu HEX:

Ví Dụ 1: Định Kiểu Thẻ Sản Phẩm E-Commerce Với Mã HEX 6 Ký Tự Chuẩn Figma

```
<!-- Product Card Structure -->
<div class="product-card">
  <span class="sale-badge">-25%</span>
  <h3 class="product-name">Tai Nghe Bluetooth Wireless</h3>
  <p class="price">1.250.000 VND</p>
  <button class="btn-buy">Them Vao Gio Hang</button>
</div>

<!-- Styling with HEX 6-digit colors -->
<style>
.product-card {
  background-color: #f8fafc; /* Nen xam trang sang diu */
  border: 1px solid #e2e8f0;    /* Vien xam nhat */
  border-radius: 8px;
  padding: 16px;
}
.sale-badge {
  background-color: #ef4444; /* Do tuoi giam gia */
  color: #ffffff;            /* Trang tinh */
  padding: 2px 6px;
}
.product-name {
  color: #1e293b;            /* Xam den dam de doc */
}
.price {
  color: #059669;            /* Xanh la dam gia tien */
  font-weight: bold;
}
.btn-buy {
  background-color: #3b82f6; /* Xanh duong chinh (Primary) */
  color: #ffffff;
  border: none;
}
</style>
```

Ví Dụ 2: Sử Dụng Mã HEX 8 Ký Tự Có Kênh Alpha (#RRGGBBAA) Trong CSS4

```css
/* Lop phu Modal nen den mo 50% opacity (#00000080) */
.modal-backdrop {
  background-color: #00000080; /* 80 trong Hex = 128 trong Decimal = 50% Alpha */
}

/* Tag thong bao xanh duong nhat mo 20% (#3b82f633) */
.info-tag {
  background-color: #3b82f633; /* 33 trong Hex = 51 trong Decimal = 20% Alpha */
  color: #1d4ed8;
  border: 1px solid #93c5fd;
}
```

BROWSER PREVIEW Kết Quả Hiển Thị Của Thẻ Sản Phẩm & Mã HEX 8 Ký Tự Alpha

-25% (#ef4444)

Tai Nghe Bluetooth Wireless (#1e293b)

Giá: 1.250.000 VNĐ (#059669)

Thêm Vào Giỏ Hàng (#3b82f6)

Tag Nổi Bật với Mã HEX 8 Ký Tự: `background-color: #3b82f633;` (Nền Xanh dương mờ 20%)

3.3. Hệ Màu RGB Và RGBA (Red, Green, Blue, Alpha):

[Khái Niệm] Khái Niệm Cơ Bản Về Mô Hình Màu RGB & Kênh RGBA

1. Bản Chất Kỹ Thuật Của Hệ Màu RGB:

- **RGB (Red - Green - Blue)**: Là mô hình biểu diễn màu dựa trên **Nguyên lý ánh sáng cộng** (**Additive Color Model**). Trong mô hình này, các màu sắc trên màn hình được tạo thành từ việc phát chiếu và phối hợp 3 nguồn chùm ánh sáng màu cơ bản: Đỏ (Red), Xanh lá (Green) và Xanh dương (Blue). Đây là tiêu chuẩn mô hình màu được sử dụng phổ biến nhất trên các thiết bị điện tử như màn hình máy tính, TV, điện thoại và giao diện Web.
- **Thang cường độ phát sáng**: Mỗi kênh màu R, G, B nhận một giá trị số nguyên từ `0` đến `255` ($0 \rightarrow 255$), đại diện cho 256 mức cường độ sáng phát ra từ điểm ảnh (0 là tắt hoàn toàn, 255 là phát sáng cực đại). Khi phối hợp cả 3 kênh, hệ màu RGB tạo ra tổng cộng $256 \times 256 \times 256 = 16.777.216$ màu sắc khác nhau (đạt chuẩn màu **True Color 24-bit**).

2. Sự Mở Rộng Kênh Alpha (A) Trong RGBA:

- **RGBA (Red - Green - Blue - Alpha)**: Là biến thể mở rộng trực tiếp từ RGB, bổ sung thêm kênh thứ tư là **Alpha (A)** đại diện cho chỉ số **Độ trong suốt** (**Opacity / Transparency**).
- **Dải giá trị Alpha**: Kênh Alpha nhận giá trị số thực (**Float**) trong khoảng từ `0.0` (hoàn toàn trong suốt 0%, đè lên nội dung khác nhưng nhìn xuyên thấu hoàn toàn) đến `1.0` (đặc nguyên bản 100%, không trong suốt).

2. Cú Pháp Định Dạng Chuẩn Trong CSS & Bảng Tham Số Chi Tiết:

Cú Pháp Khai Báo RGB & RGBA Trong CSS

```
1 /* Cu phap RGB truyen thong (3 tham so kenh mau) */
2 color: rgb(R, G, B);
3
4 /* Cu phap RGBA bo sung kenh Alpha (4 tham so) */
5 color: rgba(R, G, B, A);
```

[Bảng] Bảng Giải Thích Chi Tiết Các Tham Số RGB / RGBA

Tham số	Ý nghĩa & Vai trò chuyên môn	Dải giá trị chuẩn
R	Cường độ phát sáng của kênh màu Đỏ (Red).	$0 \rightarrow 255$
G	Cường độ phát sáng của kênh màu Xanh lá (Green).	$0 \rightarrow 255$
B	Cường độ phát sáng của kênh màu Xanh dương (Blue).	$0 \rightarrow 255$
A	Kênh độ trong suốt (Alpha Channel) kiểm soát mức độ nhìn xuyên qua.	$0.0 \rightarrow 1.0$

3. Quy Tắc Sử Dụng Dấu Phẩy & Sự Khác Biệt Giữa Cú Pháp Truyền Thống vs CSS Color Level 4:

[Khái Niệm] Phân Tích Quy Tắc Sử Dụng Dấu Phẩy Trong Cú Pháp RGB / RGBA

a. Cú Pháp Truyền Thống (Phải Có Dấu Phẩy ‘,‘):

- Khi sử dụng cú pháp hàm `rgb(R, G, B)` hoặc `rgba(R, G, B, A)` tiêu chuẩn truyền thống, **bắt buộc phải sử dụng dấu phẩy** giữa các tham số.
- **Sai cú pháp (Không có dấu phẩy)**: `color: rgba(255 0 0 0.5);` → **KHÔNG hợp lệ** trong chuẩn CSS thông thường. Trình duyệt sẽ coi đây là lỗi cú pháp và không áp dụng thuộc tính màu này.
- **Đúng cú pháp (Có dấu phẩy chuẩn)**: `color: rgba(255, 0, 0, 0.5);` → **Đúng chuẩn**, hiển thị màu đỏ trong suốt 50%.

b. Cú Pháp Hiện Đại CSS Color Module Level 4 (Dùng Khoảng Trắng & Dấu Gạch Chéo ‘/‘):

- Theo tiêu chuẩn mới (**CSS Color Module Level 4**), trình duyệt cho phép dùng khoảng trắng thay dấu phẩy và dùng dấu gạch chéo `/` để ngăn kênh Alpha với cú pháp `rgb()`:
- **Cú pháp hợp lệ trong trình duyệt hiện đại**: `color: rgb(255 0 0 / 0.5);` → **Hợp lệ chuẩn CSS4**, tương đương với `rgba(255, 0, 0, 0.5)`.

[Mẹo] Ghi Nhớ Mẹo Sử Dụng Cú Pháp Khai Báo Màu

- Với cú pháp `rgb(...)` hoặc `rgba(...)` kiểu truyền thống → **Dùng dấu phẩy ‘,‘**.
- Với cú pháp kiểu mới CSS4 → **Dùng dấu cách và dấu gạch chéo ‘/‘** để ngăn kênh độ trong suốt Alpha.

[Bảng] Bảng Tóm Tắt Tính Hợp Lệ Của Các Cách Viết Cú Pháp RGBA / RGB

Cách viết cú pháp	Hợp lệ?	Ghi chú & Độ tương thích trình duyệt
`rgba(255, 0, 0, 0.5)`	Hợp lệ	Chuẩn truyền thống, hỗ trợ 100% trên tất cả mọi trình duyệt.
`rgb(255 0 0 / 0.5)`	Hợp lệ	Chuẩn mới CSS4 – Cần trình duyệt phiên bản hiện đại.
`rgba(255 0 0 0.5)`	Không hợp lệ	Sai cú pháp hoàn toàn – Trình duyệt không chấp nhận.

4. Ví Dụ Cụ Thể & Kết Quả Hiển Thị Trực Quan:

Khai Báo Định Dạng RGB Cơ Bản Và RGBA Trong Suốt

```
/* RGB co ban: do dac 100% va nen xanh la dac 100% */
.basic-text {
  color: rgb(255, 0, 0);          /* Mau do thuan */
  background-color: rgb(0, 255, 0); /* Mau nen xanh la */
  padding: 10px;
}

/* RGBA co do trong suot: xanh duong mo 50% */
.translucent-box {
  background-color: rgba(0, 0, 255, 0.5); /* Alpha = 0.5 */
  color: white;
  padding: 12px;
}
```

BROWSER PREVIEW — **Kết Quả Hiển Thị Trực Quan Của Mã CSS Trên**

.basic-text: Chữ màu Đỏ rgb(255, 0, 0) trên nền Xanh lá rgb(0, 255, 0)

Dòng văn bản nền phía dưới trang web... (Lớp giao diện nằm ở phía đằng sau)

.translucent-box: Nền Xanh dương mờ 50% rgba(0, 0, 255, 0.5)
(Hiệu ứng độ mờ bán trong suốt giúp nhìn xuyên qua nền)

5. Nguyên Lý Phối Màu Tuỳ Chỉnh Bằng RGB:

[Bảng] Bảng Nguyên Lý Phối Màu Tuỳ Chỉnh Phổ Biến Bằng RGB

Tên màu	Cú pháp RGB	Mô tả nguyên lý phối ánh sáng
Đen (Black)	`rgb(0, 0, 0)`	Không có ánh sáng phát ra ở cả 3 kênh ($R = G = B = 0$).
Trắng (White)	`rgb(255, 255, 255)`	Tất cả 3 kênh màu phát sáng ở mức tối đa ($R = G = B = 255$).
Xám (Gray)	`rgb(128, 128, 128)`	Cả 3 kênh màu có giá trị bằng nhau ($R = G = B$) tạo ra màu xám trung tính.
Vàng (Yellow)	`rgb(255, 255, 0)`	Kết hợp ánh sáng Đỏ + Xanh lá cực đại ($R = 255, G = 255, B = 0$).
Tím (Purple)	`rgb(128, 0, 128)`	Kết hợp ánh sáng Đỏ + Xanh dương ($R = 128, G = 0, B = 128$).
Cam (Orange)	`rgb(255, 165, 0)`	Kết hợp Đỏ tối đa + một ít Xanh lá ($R = 255, G = 165, B = 0$).

6. Đánh Giá Ưu Điểm Và Hạn Chế Của Hệ Màu RGB:

[Khái Niệm] Phân Tích Ưu Điểm & Hạn Chế Của Hệ Màu RGB / RGBA

- **Ưu điểm (Nổi bật)**:
  - Hỗ trợ độ trong suốt (**Alpha Channel**) linh hoạt với cú pháp `rgba()`.
  - Cực kỳ dễ dàng điều chỉnh màu sắc động bằng JavaScript (ví dụ: `element.style.backgroundColor = `rgba(255, 100, 0, 0.8)``).
  - Hỗ trợ tạo dải màu chuyển tiếp (`linear-gradient`) và hiệu ứng lớp phủ (**Overlay**) mượt mà.
- **Hạn chế (Hạn chế)**:
  - Trực quan không dễ đọc hoặc đoán màu bằng mắt thường so với mã Hex hoặc HSL.
  - Cú pháp viết dài hơn so với mã Hex rút gọn hoặc tên từ khóa màu có sẵn.
  - Khó ghi nhớ giá trị số chính xác nếu không sử dụng công cụ Color Picker chuyên dụng.

7. Hiệu Ứng Khi Thay Đổi Kênh Alpha (A) Trong RGBA:

[Bảng] Bảng Mức Độ Mờ Quan Sát Được Theo Kênh Alpha

Giá trị Alpha (A)	Hiệu ứng quan sát trực quan trên giao diện
`1.0`	Màu đậm đặc nguyên bản, rõ ràng 100%, không xuyên thấu.
`0.5`	Màu trong mờ 50%, cho phép nhìn xuyên qua 50% nội dung lớp nền bên dưới.
`0.1`	Gần như trong suốt hoàn toàn, chỉ còn một vệt màu cực kỳ nhẹ.
`0.0`	Hoàn toàn trong suốt 100% (ẩn màu hoàn toàn trên giao diện).

[Khái Niệm] Ứng Dụng Phổ Biến Của RGBA Trong Thiết Kế Giao Diện UI/UX

- **Overlay (Lớp phủ bán trong suốt)**: Phủ một lớp màu đen/tối mờ che phủ ảnh hoặc video phía dưới khi mở cửa sổ Modal/Popup.
- **Hover nhẹ (Hiệu ứng di chuột)**: Tạo màu nền mờ mịn khi người dùng di chuột qua thẻ, nút bấm.
- **Box Shadow (Bóng đổ mịn)**: Khai báo màu bóng đổ có kênh Alpha giúp phần bóng mềm mại, dịu mắt và chuyên nghiệp hơn so với bóng đặc.
- **Gradient (Dải chuyển màu trong suốt)**: Kết hợp RGBA với dải màu chuyển tiếp để tạo hiệu ứng chuyển mờ dần vào ảnh nền.

Ví Dụ Ứng Dụng RGBA Làm Lớp Phủ Overlay Modal

```
.overlay {
  background-color: rgba(0, 0, 0, 0.5); /* Nen den mo 50% che phut trang */
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
}
```

BROWSER PREVIEW **Kết Quả Mô Phỏng Lớp Phủ Trong Suốt RGBA (Modal Overlay)**



8. Ví Dụ Thực Tế Component UI & Kết Hợp Nâng Cao Của RGBA:

Ví Dụ Thực Tế Thẻ Thông Báo Error Box & Gradient RGBA

```
<!-- Component HTML -->
<div class="note">Thong bao loi he thong</div>
<div class="gradient-banner">Banner chuyen mau RGBA</div>

<!-- CSS Dinh dang -->
<style>
.note {
  padding: 15px 20px;
  background-color: rgba(255, 0, 0, 0.1); /* Do nhat 10% lam nen */
  border: 1px solid rgb(255, 0, 0);       /* Vien do dac 100% */
  color: rgb(200, 0, 0);                   /* Mau chu do sam de doc */
  border-radius: 6px;
}

/* Ket hop RGBA voi dai mau chuyen tiep */
.gradient-banner {
  padding: 15px 20px;
  background: linear-gradient(to right, rgba(255, 0, 0, 0.5), rgba(0, 0, 255,
    0.5));
  color: white;
  border-radius: 6px;
}
</style>
```

BROWSER PREVIEW Kết Quả Hiển Thị Thực Tế Thẻ Error Box & Banner Gradient

.note: Thông báo lỗi hệ thống

`background-color: rgba(255, 0, 0, 0.1);` | `border: 1px solid rgb(255, 0, 0);`

.gradient-banner: Dải Chuyển Màu RGBA (Red 50% → Blue 50%)

```
background: linear-gradient(to right, rgba(255, 0, 0, 0.5), rgba(0, 0, 255, 0.5));
```

9. So Sánh Chi Tiết Giữa Cú Pháp 'rgba()' Và Thuộc Tính 'opacity':

[Bảng] Bảng So Sánh Toàn Diện Giữa rgba() Và Thuộc Tính opacity

Tiêu chí	Hàm rgba()	Thuộc tính opacity
Phạm vi áp dụng	Chỉ ảnh hưởng duy nhất màu được chỉ định (màu nền, màu chữ hoặc đường viền).	Làm mờ toàn bộ phần tử bao gồm cả tất cả phần tử con bên trong.
Tính linh hoạt	**Linh hoạt cao**: Có thể làm mờ nền mà chữ vẫn đặc 100%.	**Không chọn lọc**: Mờ cả văn bản, hình ảnh, icon bên trong.
Ứng dụng Overlay	**Hoàn hảo cho Modal/Popup**: Tạo nền tối mờ nhưng chữ hiển thị sắc nét.	**Hạn chế**: Khiến nội dung trên Modal cũng bị mờ mờ khó đọc.

10. Các Lưu Ý Quan Trọng Khi Sử Dụng RGB / RGBA:

[Mẹo] Lưu Ý Best Practices Khi Dùng RGBA Trong Thiết Kế Web

- **Nên sử dụng** `rgba()` khi chỉ muốn màu nền hoặc đường viền mờ nhẹ mà chữ và biểu tượng bên trong vẫn giữ độ rõ 100%.
- **Tránh dùng Alpha quá thấp**: Chỉ số Alpha quá nhỏ (< 0.1) khiến màu sắc quá nhạt, khó nhìn và dễ vi phạm tiêu chuẩn độ tương phản khả năng truy cập (WCAG Contrast Ratio).
- **Chú ý thứ tự đối số**: Cú pháp `rgba(R, G, B, A)` luôn bắt buộc thứ tự 3 kênh màu R, G, B trước, kênh A đứng cuối cùng.
- **Điều chỉnh động bằng JavaScript**: Bạn dễ dàng dùng JS để thay đổi màu sắc bán trong suốt linh hoạt: `element.style.backgroundColor = \`rgba(255, 100, 0, 0.8)\`;`

[Cảnh Báo] Các Lỗi Thường Gặp Khi Làm Việc Với RGB / RGBA

- **Nhầm lẫn giữa 'rgba()' và 'opacity'**: Dùng 'opacity' cho container khiến chữ bên trong bị mờ nhạt khó đọc. Giải pháp: Thay thế 'background-color: rgba(...)'.
- **Nhập sai dải giá trị**: Nhập giá trị kênh màu vượt quá 255 (ví dụ: 'rgb(300, 0, 0)') sẽ bị trình duyệt bỏ qua hoặc tự động ép về 255.
- **Cơ chế thừa kế màu trong DOM**: 'rgba()' không bị thừa kế xuống con, nhưng nếu gán 'opacity' lên cha thì con bắt buộc bị mờ theo mà không có cách nào bù đắp lại bằng CSS.

11. Tổng Kết Nhanh So Sánh RGB Và RGBA:

[Bảng] Bảng Tổng Kết Nhanh Phân Biệt RGB Và RGBA

Thuộc tính	Cú pháp	Công dụng chính	Trường hợp sử dụng phổ biến
RGB	`rgb(R, G, B)`	Màu sắc đặc 100% cơ bản.	Màu chữ chính, đường viền đặc, màu nền cố định.
RGBA	`rgba(R, G, B, A)`	Màu sắc kết hợp độ mờ trong suốt.	Nền mờ, overlay popup, hiệu ứng hover, bóng đổ mịn.

12. Tra Cứu Danh Sách Các Mã Màu RGB Phổ Biến Trong Giao Diện Web:

I. Nhóm Màu Cơ Bản:

[Bảng] Bảng Mã Màu RGB Phổ Biến - Nhóm Màu Cơ Bản

Tên màu	Cú pháp RGB	Mã Hex	Minh họa màu
Red (Đỏ)	`rgb(255, 0, 0)`	`#FF0000`	Đỏ thuần
Green (Xanh lá)	`rgb(0, 128, 0)`	`#008000`	Xanh lá chuẩn
Blue (Xanh dương)	`rgb(0, 0, 255)`	`#0000FF`	Xanh dương thuần
Yellow (Vàng)	`rgb(255, 255, 0)`	`#FFFF00`	Vàng rực
Cyan / Aqua	`rgb(0, 255, 255)`	`#00FFFF`	Xanh lơ
Magenta / Fuchsia	`rgb(255, 0, 255)`	`#FF00FF`	Hồng tím
White (Trắng)	`rgb(255, 255, 255)`	`#FFFFFF`	Trắng tinh
Black (Đen)	`rgb(0, 0, 0)`	`#000000`	Đen tuyền

II. Màu Sắc Phổ Biến Dùng Trong Thiết Kế Giao Diện (UI):

[Bảng] Bảng Mã Màu RGB Phổ Biến - Nhóm Màu Giao Diện UI

Tên màu	Cú pháp RGB	Mã Hex	Gợi ý sử dụng thực tế
Dark Gray	`rgb(64, 64, 64)`	`#404040`	Văn bản chính trên nền sáng hoặc văn bản phụ nền tối.
Light Gray	`rgb(211, 211, 211)`	`#D3D3D3`	Nền phụ, đường phân cách (divider), viền thẻ nhẹ.
Orange	`rgb(255, 165, 0)`	`#FFA500`	Nút cảnh báo, nút kêu gọi hành động (CTA).
Tomato	`rgb(255, 99, 71)`	`#FF6347`	Nút bấm nổi bật, trạng thái nhấn mạnh.
Lime	`rgb(0, 255, 0)`	`#00FF00`	Màu sáng tươi, điểm nhấn nổi bật trên giao diện tối.
Sky Blue	`rgb(135, 206, 235)`	`#87CEEB`	Màu nền dịu nhẹ, banner thông tin mượt mà.
Violet	`rgb(238, 130, 238)`	`#EE82EE`	Màu nhấn accent sáng tạo, thương hiệu hiện đại.

III. Nhóm Màu Pastel (Mềm, Nhẹ, Dễ Nhìn):

[Bảng] Bảng Mã Màu RGB Phổ Biến - Nhóm Màu Pastel Dịu Mắt

Tên màu	Cú pháp RGB	Mã Hex	Mô tả & Phong cách
Pastel Red	`rgb(255, 179, 186)`	`#FFB3BA`	Đỏ nhạt mềm mại, dùng cho thẻ cảnh báo nhẹ.
Pastel Green	`rgb(186, 255, 201)`	`#BAFFC9`	Xanh lá nhạt, màu thành công dịu mắt.
Pastel Blue	`rgb(179, 217, 255)`	`#B3D9FF`	Xanh dương nhạt, dùng làm nền card hoặc tag.
Light Pink	`rgb(255, 182, 193)`	`#FFB6C1`	Hồng nhẹ dễ chịu cho giao diện thẩm mỹ.
Peach	`rgb(255, 229, 180)`	`#FFE5B4`	Màu đào nhạt, rất hợp làm màu nền section.
Mint	`rgb(189, 252, 201)`	`#BDFCC9`	Màu bạc hà hiện đại, giao diện tươi sáng.

IV. Nhóm Màu Dành Cho Nền Tối (Dark Theme):

[Bảng] Bảng Mã Màu RGB Phổ Biến - Nhóm Màu Dark Theme

Tên màu	Cú pháp RGB	Mã Hex	Gợi ý sử dụng Dark Theme
Dark Blue	`rgb(18, 32, 47)`	`#12202F`	Nền chính cho giao diện Dark Mode sang trọng.
Gray-900	`rgb(33, 37, 41)`	`#212529`	Nền card hoặc thanh điều hướng (Navbar) tối.
Slate	`rgb(108, 117, 125)`	`#6C757D`	Văn bản phụ, icon xám trung tính trên nền tối.
Teal	`rgb(0, 128, 128)`	`#008080`	Màu nhấn Accent xanh ngọc hiện đại.
Charcoal	`rgb(54, 69, 79)`	`#36454F`	Nền trung tính cao cấp, hạn chế mỏi mắt.

V. Thuật Toán Tự Chuyển Đổi Mã Hex → RGB:

[Khái Niệm] Quy Trình Chuyển Đổi Từ Mã Hex Sang Cú Pháp RGB

Ví dụ: Chuyển đổi mã Hex `#FF5733` sang cú pháp `rgb(R, G, B)`.

Các bước thực hiện:

1. Tách mã Hex 6 ký tự thành 3 cặp ký tự đại diện cho 3 kênh: Red $=$ FF_{16}, Green $=$ 57_{16}, Blue $=$ 33_{16}.
2. Chuyển đổi từng cặp số Thập lục phân (Hex - cơ số 16) sang Thập phân (Decimal - cơ số 10):
   - Kênh Red (FF_{16}): $15 \times 16^1 + 15 \times 16^0 = 240 + 15 = \mathbf{255}$.
   - Kênh Green (57_{16}): $5 \times 16^1 + 7 \times 16^0 = 80 + 7 = \mathbf{87}$.
   - Kênh Blue (33_{16}): $3 \times 16^1 + 3 \times 16^0 = 48 + 3 = \mathbf{51}$.
3. Ghép lại vào cú pháp chuẩn CSS: → `rgb(255, 87, 51)`.

[Khái Niệm] Câu Hỏi Phỏng Vấn Thường Gặp Về Hệ Màu Trong CSS

- **Câu hỏi 1**: Sự khác biệt cốt lõi giữa ‘background: rgba(0,0,0,0.5)‘ và ‘background: black; opacity: 0.5‘ là gì?
  - *Trả lời*: ‘rgba()‘ chỉ áp dụng độ mờ 50% duy nhất lên màu nền, giữ cho chữ và icon bên trong rõ đặc 100%. Ngược lại, ‘opacity: 0.5‘ làm mờ toàn bộ phần tử và tất cả phần tử con bên trong.
- **Câu hỏi 2**: Khi nào nên dùng mã Hex và khi nào nên dùng ‘rgb()‘ / ‘rgba()‘?
  - *Trả lời*: Mã Hex phù hợp nhất cho màu cố định 100% vì ngắn gọn và chuẩn Figma. ‘rgba()‘ bắt buộc phải dùng khi cần màu nền mờ trong suốt, đổ bóng mịn hoặc thao tác đổi màu động bằng JavaScript.

[Bảng] Đọc Thêm: Mở Rộng Kiến Thức Chuyên Sâu Về RGB / RGBA

1. Cú Pháp Hiện Đại CSS Color Module Level 4 (Space-separated Syntax): Trong các tiêu chuẩn CSS hiện đại (CSS Color Module Level 4), W3C hỗ trợ cú pháp không sử dụng dấu phẩy và dùng dấu gạch chéo / cho kênh Alpha:

Cú Pháp CSS Color Level 4 Không Dấu Phẩy

```
/* Tuong duong rgba(255, 87, 51, 0.5) */
.modern-box {
  color: rgb(255 87 51 / 50%);
  background: rgb(0 0 0 / 0.7);
}
```

2. CSS Color Module Level 5 & Hàm 'color-mix()': CSS Color Level 5 giới thiệu hàm hòa trộn màu mạnh mẽ trực tiếp bằng CSS thuần:

Ví Dụ Hòa Trộn Màu Với color-mix()

```
/* Hoa tron 80% mau do voi 20% mau trang */
.soft-red-mix {
  background-color: color-mix(in srgb, red 80%, white 20%);
}
```

3. Nguyên Lý Xử Lý Alpha Blending Trên Phần Cứng Card Đồ Họa (GPU): Khi một phần tử có màu RGBA bán trong suốt đè lên phần tử khác, trình duyệt sẽ thực hiện thuật toán tính toán hòa trộn điểm ảnh (**Alpha Blending**):

$$C_{\text{final}} = C_{\text{src}} \times A_{\text{src}} + C_{\text{dst}} \times (1 - A_{\text{src}})$$

Quá trình này được tăng tốc phần cứng (**GPU Hardware Acceleration**) thông qua việc tạo ra một lớp tổng hợp (**Compositor Layer**) riêng biệt trên card đồ họa.

4. Lưu Ý Khả Năng Truy Cập (Accessibility & WCAG Contrast): Khi thiết kế chữ mờ hoặc background bán trong suốt bằng RGBA, cần đảm bảo tỷ lệ độ tương phản (**Contrast Ratio**) đạt tối thiểu $4.5 : 1$ đối với văn bản thường và $3.0 : 1$ đối với văn bản lớn theo tiêu chuẩn WCAG 2.1 AA.

3.4. Hệ Màu HSL Và HSLA (Hue, Saturation, Lightness, Alpha) Masterclass:

[Khái Niệm] Khái Niệm Cơ Bản Về Mô Hình Màu HSL & HSLA

1. Bản Chất Kỹ Thuật Của Hệ Màu HSL:

- **HSL (Hue - Saturation - Lightness)**: Là hệ thống biểu diễn màu sắc dựa trên góc màu và cách mắt người cảm nhận trực quan trong thực tế (thay vì dựa trên cường độ phát sáng máy tính như RGB/HEX).
- **4 Kênh thông số của HSL / HSLA**:
  - **Hue (Góc màu - H)**: Góc xoay trên vòng tròn màu sắc từ $0°$ đến $360°$. Trong đó: $0°$ (hoặc $360°$) là Đỏ, $60°$ là Vàng, $120°$ là Xanh lá, $180°$ là Xanh lơ, $240°$ là Xanh dương, $300°$ là Hồng tím.
  - **Saturation (Độ bão hòa - S%)**: Mức độ tươi rực rỡ của màu (0% là màu xám xịt trung tính, 100% là sắc màu tươi nguyên bản).
  - **Lightness (Độ sáng - L%)**: Mức độ sáng/tối của màu (0% là màu đen tuyền, 50% là tông màu chuẩn nguyên bản, 100% là màu trắng tinh).
  - **Alpha (Độ trong suốt - A)**: Chỉ số độ mờ nhìn xuyên thấu từ 0.0 (trong suốt 0%) đến 1.0 (đặc nguyên bản 100%).

2. Cú Pháp Định Dạng Chuẩn Trong CSS & Bảng Giá Trị:

Cú Pháp Khai Báo HSL & HSLA Trong CSS

```
/* Cu phap HSL truyen thong */
color: hsl(H, S%, L%);

/* Cu phap HSLA co kenh Alpha */
color: hsla(H, S%, L%, A);
```

3. Các Ví Dụ Mã Nguồn Thực Tế Về Hệ Màu HSL & HSLA:

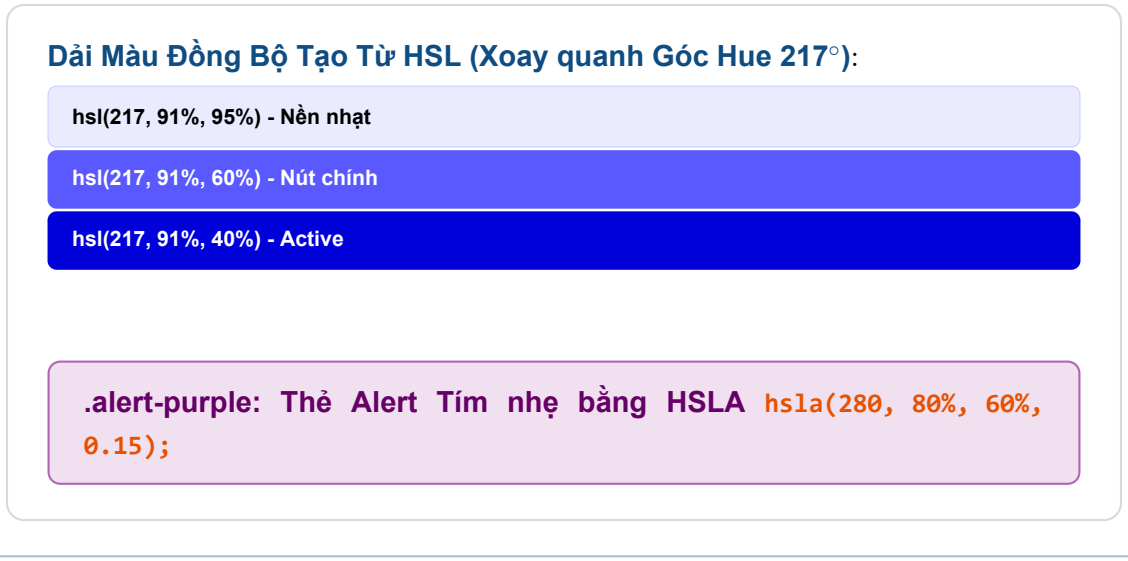
Ví Dụ 1: Tạo Dải Màu Đồng Bộ (Color Palette) Chỉ Bằng Việc Đổi Chỉ Số Lightness L%

```
/* Giu nguyen goc Hue=217 (Xanh duong) va Saturation=91% */
.bg-lightest  { background-color: hsl(217, 91%, 95%); color: #1e293b; } /* Nen
    xanh cuc nhat */
.btn-primary  { background-color: hsl(217, 91%, 60%); color: white; }   /* Mau
    nut nhan */
.btn-hover    { background-color: hsl(217, 91%, 50%); color: white; }   /* Mau
    nut hover dam hon */
.btn-active   { background-color: hsl(217, 91%, 40%); color: white; }   /* Mau
    nut active dam nhat */
```

Ví Dụ 2: Định Kiểu Thẻ Alert Trong Suốt Nhẹ Bằng HSLA

```css
/* Mau tim trong suot mo 15% bang HSLA */
.alert-purple {
  background-color: hsla(280, 80%, 60%, 0.15); /* Alpha = 0.15 */
  border: 1px solid hsl(280, 80%, 50%);
  color: hsl(280, 80%, 30%);
  padding: 12px 16px;
  border-radius: 6px;
}
```

BROWSER PREVIEW Kết Quả Hiển Thị Của Dải Màu HSL Palette & Thẻ Alert HSLA



4. Lợi Thế Vượt Trội Của HSL Trong Lập Trình Hệ Màu Động (Theme System):

[Bảng] Bảng Phân Tích Lợi Thế Vượt Trội Của Hệ Màu HSL

Ưu điểm cốt lõi	Giải thích & Ứng dụng thực tế
Tạo Palette màu đồng bộ	Chỉ cần giữ nguyên góc Hue H, thay đổi chỉ số $L\%$ để tạo ra dải sắc thái từ nhạt đến đậm nhất quán.
Tối ưu hiệu ứng Hover / Focus	Dễ dàng tạo trạng thái di chuột nhẹ hơn hoặc đậm hơn mà không cần đoán mã Hex ngẫu nhiên.
Hợp tác tốt với Biến CSS	Khai báo góc màu trong biến: `--brand-hue: 217; background: hsl(var(--brand-hue), 90%, 50%);`.

3.5. Từ Khóa Đặc Biệt: transparent Và currentColor Masterclass:

[Khái Niệm] Tổng Quan Về Hai Từ Khóa Đặc Biệt Trong CSS

Trong hệ thống màu sắc của CSS, ngoài các mô hình mã màu số (HEX, RGB, HSL), W3C cung cấp hai từ khóa đặc biệt cực kỳ linh hoạt và có sức mạnh lớn trong thiết kế giao diện UI/UX hiện đại:

1. `transparent`: Từ khóa đại diện cho màu sắc hoàn toàn trong suốt (100% transparency / 0% opacity).
2. `currentColor`: Từ khóa động đại diện cho giá trị màu sắc của thuộc tính `color` hiện tại.

I. Chuyên Đề Từ Khóa Đặc Biệt `transparent`:

[Khái Niệm] Bản Chất Kỹ Thuật Của Từ Khóa transparent

1. transparent Là Gì?:

- Từ khóa `transparent` tạo ra một màu hoàn toàn nhìn xuyên thấu (100% trong suốt), cho phép các phần tử nền hoặc nội dung phía dưới hiển thị trọn vẹn.
- **Bản chất toán học trong hệ màu**: Từ khóa `transparent` tương đương hoàn toàn với giá trị `rgba(0, 0, 0, 0)` hoặc `hsla(0, 0%, 0%, 0)`.
- Trong tiêu chuẩn CSS Color Module Level 3 trở đi, `transparent` được coi là một tên màu chuẩn hợp lệ và có thể áp dụng cho bất kỳ thuộc tính màu sắc nào (`color`, `background-color`, `border-color`, `shadow`, `stroke`).

2. Các Ứng Dụng Kỹ Thuật Kinh Điển Của `transparent` Trong Thiết Kế Web:

[Bảng] Bảng Tổng Hợp 4 Ứng Dụng Kỹ Thuật Thực Tế Của transparent

Ứng dụng kỹ thuật	Mô tả & Kỹ thuật triển khai trong CSS
Vẽ hình khối tam giác / Tooltip	Kết hợp các đường viền `border` với 3 cạnh mang màu `transparent` và 1 cạnh có màu thực tế để tạo mũi tên Tooltip.
Nút bấm viền mỏng (Ghost Button)	Khai báo `background-color: transparent;` để lộ nền trang web, chỉ giữ đường viền `border` tinh tế.
Phủ dải mờ Gradient đè ảnh nền	Sử dụng `linear-gradient(to top, #1e293b, transparent)` giúp văn bản hiển thị rõ ràng trên nền hình ảnh complex.
Reset nền mặc định ô nhập liệu	Tháo bỏ màu nền xám mặc định của trình duyệt cho thẻ `<input>`, `<select>` hoặc đường ray thanh cuộn `::-webkit-scrollbar-track`.

Ví Dụ 1: Định Kiểu Nút Bấm Ghost Button & Mũi Tên Tooltip Bằng transparent

```css
/* 1. Nut bam Ghost Button sang trong */
.btn-ghost {
  background-color: transparent; /* Nền trong suốt nhìn xuyên qua */
  border: 2px solid #3b82f6;
  color: #3b82f6;
  padding: 8px 16px;
  border-radius: 6px;
  transition: all 0.3s ease;
}

.btn-ghost:hover {
  background-color: #3b82f6;     /* Khi hover moi to mau dac */
  color: #ffffff;
}

/* 2. Mui ten tam giac Tooltip (Triangle Arrow) */
.tooltip-arrow {
  width: 0;
  height: 0;
  border-left: 8px solid transparent;  /* Vien trai trong suot */
  border-right: 8px solid transparent; /* Vien phai trong suot */
  border-bottom: 10px solid #1e293b;   /* Vien duoi tao thanh dinh tam giac */
}

/* 3. Gradient mo dan vao anh nen (Fading Gradient Overlay) */
.hero-card {
  background: linear-gradient(to top, rgba(15, 23, 42, 0.9), transparent);
}
```

BROWSER PREVIEW Kết Quả Hiển Thị Của Nút Bấm Ghost Button & Mũi Tên Tooltip

Nút Bấm Ghost Button (`background: transparent;`):

Khám Phá Khóa Học (Ghost Button)

Mũi Tên Tooltip Tam Giác (Kết hợp `transparent`):

▲ Thẻ Tooltip Thông Tin (`border-left/right: transparent;`)

[Bảng] Bảng So Sánh Chi Tiết: transparent vs opacity: 0 vs visibility: hidden

Tiêu chí	transparent	opacity: 0	visibility: hidden
`Bản chất tác động`	Chỉ làm mờ màu sắc nền/viền được gán.	Làm mờ toàn bộ phần tử và tất cả con bên trong.	Ẩn hiển thị phần tử nhưng vẫn giữ nguyên khoảng trống DOM.
`Sự tương tác chuột (Click/Hover)`	**Vẫn tương tác tốt** (Pointer events hoạt động).	**Vẫn nhận click** (Rất dễ gây bẫy bấm nhầm).	**Vô hiệu hóa click** (Không nhận sự kiện chuột).
`Khả năng kế thừa phần tử con`	Phần tử con giữ màu sắc riêng biệt 100%.	Tất cả phần tử con bị mờ theo 100%.	Phần tử con có thể hiện lại bằng `visibility: visible`.

II. Chuyên Đề Từ Khóa Đặc Biệt `currentColor`:

[Khái Niệm] Bản Chất Kỹ Thuật Của Từ Khóa Đặc Biệt currentColor

1. currentColor Là Gì?:

- `currentColor`: Là một giá trị từ khóa động trong CSS, đại diện cho giá trị màu sắc hiện tại của thuộc tính `color` đang áp dụng trực tiếp hoặc kế thừa lên phần tử đó.
- **Nguyên lý cốt lõi**: currentColor ≡ giá trị hiện tại của thuộc tính color.
- **Công dụng chính**: Đồng bộ tự động màu sắc cho `border-color`, `background-color`, `box-shadow`, `outline-color`, và đồ họa SVG `fill` / `stroke`.

Ví Dụ 2: Hiệu Ứng Hover Nút Bấm & Đồng Bộ Icon SVG Với currentColor

```css
/* 1. Nut bam dong bo mau vien khi Hover */
button.btn-sync {
  color: #2563eb;                /* Mau chu xanh duong */
  border: 2px solid currentColor; /* Vien tu dong xanh duong */
  background-color: transparent;
  padding: 8px 16px;
  border-radius: 6px;
  transition: color 0.3s ease;
}

button.btn-sync:hover {
  color: #d97706;                /* Chi can doi color, vien tu dong sang mau cam! */
}

/* 2. Dong bo mau khung nhap lieu va Icon SVG khi Focus */
.input-group {
  color: #64748b;                /* Mau icon va text phu xam */
  border: 2px solid currentColor; /* Vien xam theo color */
  padding: 8px 12px;
  border-radius: 6px;
}

.input-group:focus-within {
  color: #2563eb;                /* Ca vien va Icon SVG tu dong sang xanh duong! */
}
```

BROWSER PREVIEW Kết Quả Hiển Thị Đồng Bộ Trạng Thái Focus & Hover Bằng currentColor

Trạng Thái Focus Input Đồng Bộ Icon SVG (`currentColor`):

[Icon SVG] Nhập email... (`border` và Icon tự động chuyển màu Xanh dương khi Focus)

[Bảng] Tóm Tắt Tổng Kết Nhanh So Sánh transparent Và currentColor

Từ khóa	Bản chất biểu diễn	Trường hợp ứng dụng hàng đầu (Use Cases)
`transparent`	**Màu trong suốt 100% (Alpha = 0).**	Ghost Button, Mũi tên tam giác Tooltip, Gradient mờ dần vào ảnh nền, Reset nền mặc định.
`currentColor`	**Giá trị màu động đại diện cho `color`.**	Đồng bộ màu viền/bóng đổ với chữ, icon SVG tự đổi màu theo theme, hiệu ứng Hover/Focus linh hoạt.

5. Các Lưu Ý Kỹ Thuật Quan Trọng Khi Sử Dụng currentColor:

[Cảnh Báo] Các Cảnh Báo & Bẫy Thường Gặp Với currentColor

- **Phải có thuộc tính `color` xác định**: Nếu phần tử không gán `color` và không kế thừa từ cha, trình duyệt sẽ tự động lấy giá trị mặc định của trình duyệt (`initial`).
- **Không dùng thay thế thuộc tính `color`**: Tuyệt đối không khai báo `color: currentColor;` trên cùng phần tử vì sẽ gây ra vòng lặp tham chiếu vô nghĩa.
- **Hạn chế trong dải dốc màu (`linear-gradient()`)**: Một số phiên bản trình duyệt cũ không hỗ trợ tốt việc dùng `currentColor` làm điểm mốc màu (**Color Stop**) trong gradient.

6. Tóm Tắt Nhanh Bản Chất Của Từ Khóa currentColor:

[Bảng] Bảng Tổng Kết Nhanh Đặc Tính Của currentColor

Tính năng	Giá trị & Mô tả
Loại giá trị	Giá trị từ khóa đặc biệt trong tiêu chuẩn CSS.
Đại diện cho	Giá trị màu chữ (`color`) hiện tại của phần tử.
Ứng dụng phổ biến	`border-color`, `outline-color`, `box-shadow`, SVG `fill`/`stroke`, màu hiệu ứng `:hover`.
Tính kế thừa	Tự động kế thừa theo màu chữ của phần tử cha trong cây DOM.
Gợi ý sử dụng Best Practice	Tạo nút bấm (Button), biểu tượng Icon SVG, đường viền nhất quán màu sắc. Khi chuyển đổi theme chỉ cần đổi duy nhất 1 biến `color`.

3.7. Mẹo Chọn Màu Sắc Chuyên Nghiệp Cho Giao Diện Web (UI/UX Best Practices):

[Bảng] Bảng Gợi Ý Phối Màu Thực Tế Chuẩn Chuyên Nghiệp

Mục đích sử dụng	Mã màu gợi ý	Nguyên tắc & Lý do chọn
Văn bản chính (Primary text)	`#212529, #333333`	Dùng xám đậm thay vì đen tuyền `#000` để dịu mắt.
Văn bản phụ (Secondary text)	`#6c757d, #888888`	Xám nhạt hơn để tạo tương phản phân cấp thông tin.
Nền chính (Main Background)	`#ffffff, #f8f9fa`	Trắng hoặc xám kem rất nhẹ tạo cảm giác sạch sẽ.
Nút gọi hành động (CTA Button)	`#007bff, #ff6347`	Màu xanh/cam tươi nổi bật kích thích hành động nhấp.
Cảnh báo lỗi (Danger / Error)	`#dc3545, #ff4d4f`	Đỏ cảnh báo nổi bật đạt chuẩn tương phản W3C.
Giao diện tối (Dark Mode)	`#121212, #1a1a1a`	Nền tối kết hợp văn bản xám sáng (`#e0e0e0`).

3.8. Ứng Dụng Màu Động Với CSS Variables & JavaScript:

Trong phát triển ứng dụng Web thực tế, việc quản lý hệ thống màu sắc bằng Biến CSS (**CSS Variables**) và điều khiển linh hoạt qua JavaScript giúp hỗ trợ tính năng chuyển đổi giao diện sáng/tối (**Dark Mode / Theme Switch**) cực kỳ dễ dàng.

Khai Báo Hệ Thống Biến Màu Trong CSS

```
:root {
  --primary-color: #3498db;
  --text-color: #333333;
  --bg-color: #ffffff;
}

/* Ap dung bien mau cho giao dien */
body {
  background-color: var(--bg-color);
  color: var(--text-color);
}

.button-main {
  background-color: var(--primary-color);
  color: #ffffff;
}
```

Cập Nhật Màu Động Bằng JavaScript

```
// Thao tac thay doi mau nen bang JavaScript
const heroElement = document.querySelector('.hero-section');
heroElement.style.backgroundColor = 'rgba(255, 0, 0, 0.6)';

// Thay doi gia tri bien CSS truc tiep qua JavaScript
document.documentElement.style.setProperty('--primary-color', '#2ecc71');
```

3.9. Bảng So Sánh Tổng Quát Các Hệ Màu Trong CSS:

[Bảng] Bảng So Sánh Toàn Diện Các Hệ Biểu Diễn Màu

Hệ màu	Độ phổ biến	Dễ viết tay?	Hỗ trợ Alpha?	Ứng dụng phù hợp nhất
`Color Name`	Thấp	**Rất dễ**	**Không**	Học tập, ví dụ nhanh, chạy thử nghiệm code.
`Hexadecimal`	**Cao nhất**	**Khá dễ**	**Không**	Dự án web thực tế, sao chép mã từ Figma/Photoshop.
`RGB / RGBA`	**Rất cao**	Dài hơn	**Có (RGBA)**	Hiệu ứng mờ overlay, đổ bóng, di chuột hover.
`HSL / HSLA`	Trung bình	Khó ban đầu	**Có (HSLA)**	Lập trình hệ màu động, theme sáng/tối, dải màu sắc.
`CSS Variables`	**Rất cao**	Chuẩn mực	**Tùy thuộc biến**	Xây dựng hệ thiết kế Design System chuyên nghiệp.

3.10. Kết Luận – Khi Nào Nên Dùng Hệ Màu Nào?

[Khái Niệm] Quy Tắc Vàng Lựa Chọn Hệ Màu Cho Dự Án

1. **Dự án nhỏ, làm bài tập nhanh**: Sử dụng `Color Name` hoặc `Hex`.
2. **Giao diện trang web thực tế / Sản phẩm nghiệp dư & chuyên nghiệp**: Ưu tiên kết hợp **Mã Hex** với **CSS Variables** để quản lý tập trung.
3. **Cần hiệu ứng trong suốt, lớp phủ overlay, bóng mờ**: Sử dụng **RGBA** hoặc **HSLA**.
4. **Xây dựng dải màu chủ đạo, hệ màu động sáng/tối (Dark Mode Switcher)**: Sử dụng **HSL kết hợp Biến CSS (CSS Variables) và JavaScript**.

3.6. Tính Kế Thừa Của Thuộc Tính Color (Color Property Inheritance Masterclass):

[Khái Niệm] Khái Niệm Cơ Bản Về Tính Kế Thừa Của color

1. Tính Kế Thừa Mặc Định (Inherited Property):

- Thuộc tính `color` trong CSS là một **thuộc tính kế thừa mặc định** (**Inherited Property**).
- **Nguyên lý hoạt động**: Nếu bạn không khai báo thuộc tính `color` trực tiếp cho một phần tử con, phần tử đó sẽ tự động kế thừa giá trị `color` từ phần tử cha chứa nó trong cây DOM.

2. Phân Loại Thuộc Tính CSS Theo Tính Kế Thừa:

[Bảng] Bảng So Sánh Thuộc Tính CSS Có Kế Thừa Và Không Kế Thừa

Phân loại	Các thuộc tính tiêu biểu	Có kế thừa?	Ghi chú đặc tính
Kế thừa (Inherited)	`color`, `font-family`, `line-height`, `visibility`, `text-align`	**Có**	Tự động lan truyền xuống các thẻ con trong cây DOM.
Không kế thừa (Non-inherited)	`margin`, `padding`, `border`, `width`, `height`, `background-color`	**Không**	Chỉ có tác dụng duy nhất trên phần tử được khai báo.

3. Các Ví Dụ Mã Nguồn Về Kế Thừa Màu Sắc:

Ví Dụ 1: Kế Thừa Màu Sắc Đơn Giản Từ Thẻ Cha

```
<!-- HTML Structure -->
<div class="parent">
  <p>Đoạn văn này tự động kế thừa màu đỏ từ thẻ div.parent.</p>
</div>

<!-- CSS Styling -->
<style>
.parent {
  color: red; /* Thiet lap mau do cho the cha */
}
</style>
```

Ví Dụ 2: Kế Thừa Màu Sắc Lồng Nhau Qua Nhiều Cấp Thẻ DOM

```
<!-- Structure DOM long nhau 3 cap -->
<div style="color: blue;">
  <section>
    <p>Color nay la blue do ke thua tu div cha ngoai cung quan section!</p>
  </section>
</div>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Kế Thừa Màu Sắc Đơn Giản & Lồng Nhau**

Đoạn văn tự động có màu Đỏ do kế thừa từ `.parent { color: red; }`

Đoạn văn trong `<section>` kế thừa màu Xanh dương (`color: blue;`) từ thẻ `<div>` ngoài cùng.

4. Ép Bắt Buộc Kế Thừa Hoặc Reset Màu Với `inherit` Và `initial`:

[Khái Niệm] Ý Nghĩa Của Hai Giá Trị Từ Khóa inherit Và initial

Trong các trường hợp đặc biệt (như thẻ liên kết `<a>` hoặc nút bấm `<button>` bị trình duyệt gán kiểu mặc định User Agent Style), màu chữ có thể không tự kế thừa. Bạn có thể điều khiển trực tiếp bằng hai từ khóa:

- `color: inherit;`: Ép buộc phần tử phải nhận giá trị màu chữ từ phần tử cha (vượt qua mọi thiết lập mặc định của trình duyệt).
- `color: initial;`: Reset giá trị màu chữ về lại giá trị mặc định ban đầu của CSS (thường là màu đen `#000000`).

Ví Dụ 3: Ép Thẻ Liên Kết <a> Và Button Kế Thừa Màu Bằng color: inherit

```
/* Custom Reset cho the a va button de ke thua mau tu the cha */
.navbar {
  color: darkblue;
}

/* Thong thuong the <a> co mau xanh tim va gach chan mac dinh cua trinh duyet */
.navbar a {
  color: inherit; /* Ep the <a> ke thua mau darkblue tu .navbar! */
  text-decoration: none;
}
```

5. Kết Hợp Tính Kế Thừa Với Từ Khóa `currentColor`:

Ví Dụ 4: Phối Hợp Kế Thừa color Và currentColor Đồng Bộ Viền

```
<!-- HTML Structure -->
<div class="parent">
  <div class="child">
    <p>Doan van ben trong the child.</p>
  </div>
</div>

<!-- CSS Styling -->
<style>
.parent {
  color: darkblue; /* Mau chu darkblue lan truyen xuong child va p */
}

.child {
  border: 2px solid currentColor; /* Vien tu dong mang mau darkblue do ke thua tu
     parent! */
  padding: 12px;
}
</style>
```

BROWSER PREVIEW Kết Quả Hiển Thị Kết Hợp Tính Kế Thừa Và currentColor

.child: Đường viền tự động mang màu `darkblue` nhờ sự phối hợp giữa tính kế thừa `color` từ `.parent` và từ khóa `border: 2px solid currentColor;`

6. Các Lưu Ý Kỹ Thuật Quan Trọng Khi Làm Việc Với Tính Kế Thừa:

[Cảnh Báo] Những Cảnh Báo Cần Nhớ Về Kế Thừa Màu Sắc

- **Cây DOM quyết định cấp kế thừa**: Phần tử con luôn nhận màu từ phần tử cha trực tiếp gần nhất có khai báo thuộc tính `color`.
- **Bị ghi đè ngay khi con có `color` riêng**: Nếu bạn khai báo `color` trực tiếp trên phần tử con, thuộc tính kế thừa từ cha sẽ lập tức bị vô hiệu hóa.
- **Ngoại lệ User Agent Style của trình duyệt**: Các thẻ HTML như `<a>`, `<button>`, `<input>`, `<select>` có quy tắc màu mặc định riêng của trình duyệt. Bạn bắt buộc phải khai báo `color: inherit;` nếu muốn chúng kế thừa màu từ thẻ cha.

7. Bảng Tổng Kết Đặc Tính Kế Thừa Của Thuộc Tính Color:

[Bảng] Bảng Tổng Kết Các Giá Trị Kế Thừa Của color

Cú pháp giá trị	Có kế thừa?	Ý nghĩa & Mục đích sử dụng
`color: <value>`	Có (Mặc định)	Mặc định tự động lan truyền màu xuống tất cả thẻ con.
`color: inherit`	Có (Ép buộc)	Ép phần tử nhận màu từ cha (khắc phục CSS Reset / User Agent Style).
`color: initial`	Không	Reset màu chữ về mặc định của trình duyệt (thường là đen `#000000`).

9.4.3 Chuyên Đề Phông Chữ & Typography Trong CSS (Generic Font Families Masterclass)

[Khái Niệm] Khái Niệm Cơ Bản Về Generic Font Families

1. Generic Font Families Là Gì?:

- **Generic Font Families (Nhóm phông chữ tổng quát)**: Là các nhóm phông chữ mặc định chuẩn hóa bởi W3C mà tất cả trình duyệt Web trên mọi hệ điều hành (Windows, macOS, Linux, Android, iOS) luôn luôn hiểu và hỗ trợ sẵn.
- **Vai trò cốt lõi trong cơ chế Fallback Font**: Khi người lập trình khai báo một font chữ cụ thể (ví dụ: `"Times New Roman"`), nếu thiết bị người dùng chưa cài đặt font đó, trình duyệt sẽ tự động dự phòng (**Fallback**) sang nhóm font tổng quát tương ứng được chỉ định ở cuối danh sách (ví dụ: `serif`).

2. Sự Khác Biệt Trực Quan Giữa Phông Chữ Có Chân (Serif) Và Không Chân (Sans-Serif):

BROWSER PREVIEW Minh Họa Trực Quan Sự Khác Biệt Giữa Serif Và Sans-serif



[Bảng] Bảng So Sánh Nhanh Giữa Serif Và Sans-serif

Tiêu chí so sánh	Phông chữ Serif (Có chân)	Phông chữ Sans-serif (Không chân)
Nét chữ	Có các nét gạch nhỏ ở đầu và chân ký tự.	Không có chân chữ, nét chữ đơn giản gọn phẳng.
Phong cách thiết kế	Cổ điển, sang trọng, nghiêm túc, trang trọng.	Hiện đại, sạch sẽ, gọn gàng, thân thiện.
Độ dễ đọc trên giấy in	Dễ đọc hơn nhờ các chân dẫn hướng mắt khi đọc bài dài.	Đọc ổn nhưng có thể thiếu điểm nhấn phân dòng.
Độ dễ đọc trên màn hình	Có thể mỏi mắt/khó đọc ở độ phân giải màn hình thấp.	Rất dễ đọc, đặc biệt trên màn hình di động nhỏ.
Ứng dụng phổ biến	Sách in, báo chí, luận văn, tài liệu chính thức.	Trang web, ứng dụng di động, giao diện UI/UX.

3. Bảng Phân Tích Chi Tiết 5 Nhóm Generic Font Families Chuẩn W3C:

[Bảng] Bảng Phân Tích Chi Tiết 5 Nhóm Phông Chữ Tổng Quát (Generic Font Families)

Tên nhóm	Đặc điểm kỹ thuật	Cảm nhận thị giác	Ví dụ font cụ thể	Ứng dụng phù hợp nhất
`serif`	Nét gạch nhỏ (serif) ở chân/đầu ký tự.	Truyền thống, nghiêm túc, chuyên nghiệp.	`Times New Roman, Georgia, Garamond`	Văn bản dài, sách báo, luận văn, tiêu đề trang trọng.
`sans-serif`	Không chân chữ, nét phẳng đều gọn gàng.	Hiện đại, sạch sẽ, dễ nhìn kỹ thuật số.	`Arial, Helvetica, Roboto, Open Sans`	Giao diện người dùng Web/App, email, blog cá nhân.
`monospace`	Chiều rộng các ký tự bằng nhau tuyệt đối.	Rõ ràng, đồng đều, dễ soi từng ký tự.	`Courier New, Consolas, Lucida Console`	Code editor, terminal, log hệ thống, bảng dữ liệu.
`cursive`	Nét nghiêng mô phỏng chữ viết tay.	Cá tính, nghệ thuật, mềm mại, trang nhã.	`Brush Script, Comic Sans, Pacifico`	Thiệp mời, chữ ký, tiêu đề trang trí nhẹ, blog cá nhân.
`fantasy`	Kiểu chữ cách điệu trang trí độc đáo.	Độc lạ, vui nhộn, gây chú ý mạnh mẽ.	`Impact, Papyrus, Jokerman, Lobster`	Poster quảng cáo, game, tiêu đề thu hút sự chú ý.

4. Các Phông Chữ Hay Dùng Nhất Trong Thiết Kế Web Theo Từng Nhóm:

[Bảng] Danh Sách Các Phông Chữ Tiêu Chuẩn Phổ Biến Trong Thiết Kế Web

Nhóm họ phông	Tên font cụ thể	Đặc điểm & Trường hợp ứng dụng thực tế
Serif (Có chân)	`Times New Roman`	Cổ điển, trang trọng (dùng nhiều trong báo chí, tài liệu in).
	`Georgia`	Thanh lịch, thiết kế tối ưu dễ đọc trên màn hình máy tính hơn Times.
	`Merriweather`	Thân thiện, nét chữ rõ ràng lý tưởng cho blog đọc nội dung dài.
Sans-serif (Không chân)	`Arial`	Đơn giản, phổ biến nhất thế giới trên mọi hệ điều hành.
	`Roboto`	Font mặc định của hệ điều hành Android, hiện đại, sạch sẽ.
	`Open Sans`	Dễ đọc tuyệt vời, phù hợp cho hệ thống Web/App quy mô lớn.
	`Lato`	Mềm mại, cân bằng, hợp cho cả tiêu đề lẫn văn bản chính.
	`Montserrat`	Đậm chất thiết kế hiện đại, tạo cảm giác chuyên nghiệp sáng tạo.
Monospace (Rộng đều)	`Courier New`	Phong cách máy đánh chữ cổ điển truyền thống.
	`Consolas`	Font chuẩn mực khi viết mã nguồn code, gọn gàng dễ đọc.
Cursive (Viết tay)	`Pacifico`	Nét chữ viết tay đáng yêu, hợp cho tiêu đề nghệ thuật hoặc Logo.
	`Dancing Script`	Nét chữ uyển chuyển, trang nhã cho thiệp mừng / banner.
Fantasy (Cách điệu)	`Impact`	Nét chữ cực đậm, thu hút sự chú ý ngay lập tức cho tiêu đề.
	`Playfair Display`	Sang trọng, phù hợp cho giao diện thời trang cao cấp.
	`Lobster`	Độc đáo, nghệ thuật, tạo điểm nhấn bắt mắt sơ khởi.

5. Cú Pháp Sử Dụng Trong CSS & Cơ Chế Fallback Font List:

Cú Pháp Khai Báo Danh Sách Phông Chữ Dự Phòng (Fallback Chain)

```css
/* 1. Dinh kieu van ban chinh voi chuoi Fallback Sans-serif */
body {
  font-family: "Open Sans", Arial, Helvetica, sans-serif;
}

/* 2. Dinh kieu ma nguoi va khoi Code voi Monospace */
code, pre {
  font-family: Consolas, "Courier New", monospace;
}

/* 3. Dinh kieu tieu de trang trong voi Serif */
h1.academic-title {
  font-family: "Playfair Display", Georgia, "Times New Roman", serif;
}
```

BROWSER PREVIEW Kết Quả Hiển Thị Của Các Kiểu Phông Chữ Khác Nhau Trên Giao Diện

1. Sans-serif (Roboto / Arial):
Đây là đoạn văn bản không chân hiện đại, gọn gàng và rất dễ đọc trên màn hình kỹ thuật số.

2. Serif (Georgia / Times New Roman):
Đây là đoạn văn bản có chân trang trọng, mang phong cách cổ điển báo chí và tài liệu chính thức.

3. Monospace (Consolas / Courier New):
`const greeting = "Hello World!"; // Ký tự rộng bằng nhau tuyệt đối`

4. Cursive & Fantasy (Chữ nghệ thuật & Tiêu đề cách điệu):
Chữ viết tay nghệ thuật (Cursive) và chữ cách điệu đậm nét (Fantasy) dùng tạo điểm nhấn Tooltip / Banner.

6. Tóm Tắt Nhanh Cheatsheet Lựa Chọn Phông Chữ:

[Bảng] Bảng Tóm Tắt Cheatsheet Lựa Chọn Loại Font Cho Dự Án

Loại font	Cảm giác chính thị giác	Trường hợp ứng dụng hàng đầu
`serif`	**Trang trọng, cổ điển, trang nhã.**	Ấn phẩm in ấn, sách báo, luận văn học thuật.
`sans-serif`	**Gọn gàng, hiện đại, thân thiện.**	Trang web, giao diện UI/UX, ứng dụng Mobile.
`monospace`	**Kỹ thuật, rõ ràng, chính xác.**	Trình soạn thảo Code, terminal, bảng số liệu.
`cursive`	**Nghệ thuật, mềm mại, tinh tế.**	Thiệp mừng, chữ ký, banner trang trí nhẹ.
`fantasy`	**Kỳ lạ, độc đáo, thu hút mạnh.**	Tiêu đề quảng cáo đặc biệt, Poster, Game UI.

7. Chuyên Đề System Font Stack Trong CSS (System-UI Masterclass):

System Font (phông chữ hệ thống) là phông chữ mặc định được cài đặt sẵn trên hệ điều hành mà người dùng đang sử dụng (như Windows, macOS, iOS, Android, Linux). Khi bạn khai báo system font trong CSS, trình duyệt sẽ hiển thị văn bản bằng đúng phông chữ hệ thống đó, giúp giao diện trang web đồng bộ tuyệt đối với các thành phần của hệ điều hành (menu, tiêu đề cửa sổ, nút bấm hệ thống...).

[Khái Niệm] Khái Niệm Cốt Lõi Về System Font

1. System Font Là Gì?:

- **Khái niệm**: System Font là bộ phông chữ giao diện người dùng mặc định (**Native UI Font**) của hệ điều hành.
- **Cơ chế hiển thị**: Trình duyệt truy xuất trực tiếp font chữ của hệ điều hành mà không cần tải bất kỳ tệp font nào từ máy chủ web hoặc dịch vụ bên ngoài (như Google Fonts).

2. Tài Liệu & Nguồn Tham Khảo Chuẩn Kỹ Thuật:

- Tham khảo cấu trúc chuỗi System Font Stack tại: `CSS-Tricks System Font Stack`.
- Kiểm tra mức độ phổ biến của các phông chữ tại: `CSS Font Stack`.

Ưu Điểm Và Lý Do Nên Sử Dụng System Font:

[Bảng] Bảng Phân Tích Chi Tiết Các Ưu Điểm Của System Font

Ưu điểm cốt lõi	Lý do kỹ thuật	Lợi ích trải nghiệm người dùng (UX)
Tốc độ cực nhanh (Fast)	**Không cần tải font từ web (Google Fonts / Custom Web Fonts).**	Trang web hiển thị tức thì, loại bỏ hoàn toàn hiện tượng nhấp nháy chữ (**FOUT / FOIT**), tối ưu chỉ số Core Web Vitals.
Tương thích cao (High Compatibility)	**Sử dụng phông chữ hệ thống được tối ưu hóa cho màn hình thiết bị.**	Hiển thị chuẩn xác, thân thuộc và thân thiện với người dùng trên mọi hệ điều hành.
Thống nhất giao diện (UI Consistency)	**Phông chữ giống hệt với giao diện hệ điều hành (OS/UI).**	Giao diện trang web hòa nhập tự nhiên như một ứng dụng Native App của thiết bị.
Responsive tốt (High Adaptability)	**Tự động tối ưu trên nhiều thiết bị và độ phân giải khác nhau.**	Hiển thị sắc nét trên cả màn hình Retina / High-DPI và màn hình tiêu chuẩn.

Cách Sử Dụng system-ui Trong CSS (Cú Pháp Đơn Giản vs Fallback Đầy Đủ):

CSS Fonts Module Level 4 giới thiệu giá trị từ khóa `system-ui`, cho phép gọi trực tiếp phông chữ mặc định của hệ điều hành hiện tại chỉ với một dòng mã.

1. Cú Pháp Đơn Giản Nhất Với system-ui (CSS Fonts Level 4)

```
/* Tu dong goi font mac dinh cua he dieu hanh hien tai */
body {
  font-family: system-ui;
}
```

[Bảng] Danh Sách Phông Chữ Mặc Định (UI Font) Theo Từng Hệ Điều Hành

Hệ điều hành (OS)	Font mặc định (UI Font)	Ghi chú đặc tính thị giác
macOS	`San Francisco`	Phông chữ tối giản, sắc nét trên màn hình Retina của Apple.
iOS	`San Francisco`	Font chuẩn mực trên iPhone / iPad.
Windows 10+	`Segoe UI`	Phông chữ giao diện chuẩn mực của Microsoft Windows.
Android	`Roboto`	Phông chữ mặc định hệ sinh thái Google / Android.
Ubuntu / Linux	`Ubuntu`	Phông chữ tự do chuẩn mực của hệ điều hành Ubuntu.
Older Windows (7 / XP / Vista)	`Tahoma, Arial`	Các phông chữ dự phòng cổ điển trên hệ điều hành Windows cũ.

2. Chuỗi Khai Báo Dự Phòng Đầy Đủ (Full System Font Stack Fallback)

```
/* Chuoi Fallback System Font Stack giup tuong thich tuyet doi moi trinh duyet & OS */
body {
  font-family:
    -apple-system,       /* macOS & iOS (Safari / Apple WebKit) */
    BlinkMacSystemFont,  /* Chrome & Opera tren macOS (Blink Engine) */
    "Segoe UI",          /* Windows 10 / 11 */
    Roboto,              /* Android */
    Ubuntu,              /* Ubuntu Linux */
    "Helvetica Neue",    /* macOS / iOS phien ban cu */
    sans-serif;          /* Generic Fallback cuoi cung */
}
```

[Mẹo] Cơ Chế Hoạt Động Của Chuỗi Fallback Font

Trình duyệt sẽ duyệt danh sách từ trên xuống dưới và chọn phông chữ đầu tiên có sẵn trên hệ điều hành của người dùng. Nếu không tìm thấy font nào trong danh sách, trình duyệt sẽ tự động dự phòng xuống nhóm font tổng quát `sans-serif`.

Ví Dụ Thực Tế & Kết Quả Hiển Thị System Font:

Ví Dụ Mã Nguồn HTML & CSS Sử Dụng System Font

```html
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <title>Vi Du System Font</title>
  <style>
    body {
      font-family: system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
      font-size: 16px;
      line-height: 1.6;
      color: #212529;
      padding: 20px;
    }
  </style>
</head>
<body>
  <h1>Tiêu đề theo system font</h1>
  <p>Đây là đoạn văn bản sử dụng font mặc định của hệ điều hành bạn đang dùng.</p>
</body>
</html>
```

BROWSER PREVIEW Kết Quả Hiển Thị Thực Tế Của System Font Trên Trình Duyệt



Lưu Ý Quan Trọng Khi Sử Dụng System Font:

[Bảng] Bảng Phân Tích Lưu Ý & Giải Thích Kỹ Thuật Khi Dùng System Font

Lưu ý kỹ thuật	Giải thích chi tiết & Giải pháp
Không đồng nhất giữa các hệ điều hành	Mỗi hệ điều hành (macOS, Windows, Android) sử dụng một phông chữ hệ thống riêng, khiến giao diện văn bản có sự chênh lệch nhẹ về nét chữ giữa các thiết bị khác nhau.
Không thể chọn chính xác từng glyph / font cụ thể	Phụ thuộc hoàn toàn vào hệ điều hành của người dùng, nhà phát triển không thể can thiệp tùy chỉnh chi tiết từng nét glyph cố định.
Không hỗ trợ tốt các hệ thống cũ	Từ khóa `system-ui` đòi hỏi trình duyệt hiện đại (Chrome 56+, Firefox 43+, Safari 11+). Bắt buộc phải kết hợp chuỗi fallback đầy đủ (`-apple-system, "Segoe UI", Roboto...`) để đảm bảo hiển thị ổn định trên thiết bị cũ.

Tổng Kết Chuyên Đề System Font trong CSS:

[Bảng] Bảng Tổng Kết Kiến Thức Cốt Lõi Về System Font

Thuộc tính / Khái niệm	Ý nghĩa & Giá trị sử dụng
`system-ui`	Gọi trực tiếp phông chữ giao diện mặc định của hệ điều hành hiện tại.
`font-family fallback`	Chuỗi phông chữ dự phòng đảm bảo hiển thị ổn định khi trình duyệt cũ không hỗ trợ `system-ui`.
`Ưu điểm vượt trội`	Tốc độ tải cực nhanh (không tốn dung lượng mạng), giao diện thân thuộc với OS, hiệu suất cao.
`Khuyết điểm hạn chế`	Không kiểm soát chi tiết 100% về kiểu dáng chữ cố định trên mọi thiết bị người dùng.

9.5 Các Loại Selectors Trong CSS (CSS Selectors Masterclass)

Bộ chọn (**Selector**) là cách để xác định các phần tử HTML mà bạn muốn áp dụng kiểu dáng (**CSS styles**). CSS cung cấp nhiều loại selectors để bạn có thể chọn phần tử theo tên thẻ, ID, class, trạng thái, hoặc cấu trúc của chúng.

9.5.1 Phân Loại Hệ Thống: 5 Nhóm CSS Selectors Cốt Lõi

Trong tiêu chuẩn kỹ thuật W3C và kiến trúc hiển thị của trình duyệt, toàn bộ hệ thống bộ chọn CSS được phân loại một cách khoa học thành **5 nhóm chính** dựa trên cơ chế truy vấn phần tử:

[Bảng] BẢNG PHÂN LOẠI 5 NHÓM CSS SELECTORS CHÍNH

STT	Nhóm Selector	Cú pháp tiêu biểu	Chức năng & Phạm vi áp dụng
1	**Bộ chọn đơn giản (Simple Selectors)**	`p, .box, #header, *`	Chọn phần tử trực tiếp dựa theo tên thẻ, class, ID hoặc bộ chọn toàn cục (*).
2	**Bộ chọn tổ hợp (Combinator Selectors)**	`div p, div > p, h2 + p, h2 ~ p`	Chọn phần tử dựa trên mối quan hệ phân cấp gia phả (hậu duệ, con trực tiếp, anh em liền kề, anh em chung) trong cây DOM.
3	**Bộ chọn lớp giả (Pseudo-class Selectors)**	`:hover, :focus, :first-child, :nth-child`	Chọn phần tử dựa trên trạng thái tương tác động của người dùng hoặc vị trí cấu trúc của phần tử trong tài liệu.
4	**Bộ chọn phần tử giả (Pseudo-element Selectors)**	`::before, ::after, ::first-letter, ::marker`	Chọn và định dạng một phần nội dung cụ thể hoặc chèn thêm phần tử đồ họa ảo vào cây DOM.
5	**Bộ chọn thuộc tính (Attribute Selectors)**	`[type="text"], [required], [href^="https"]`	Chọn phần tử dựa trên sự tồn tại hoặc giá trị cụ thể của các thuộc tính HTML.

9.5.2 Bảng Tổng Quan Chi Tiết Các Bộ Chọn CSS Quan Trọng

[Bảng] Bảng Tra Cứu Danh Sách Đầy Đủ Các Bộ Chọn CSS Quan Trọng

Loại Selector	Cú pháp	Công dụng
Bộ chọn theo thẻ (Type Selector)	`tagname {}`	Chọn tất cả phần tử có tên thẻ cụ thể.
Bộ chọn theo class (Class Selector)	`.classname {}`	Chọn tất cả phần tử có class chỉ định.
Bộ chọn theo ID (ID Selector)	`#idname {}`	Chọn phần tử có ID cụ thể (duy nhất).
Bộ chọn nhóm (Group Selector)	`tag1, tag2 {}`	Áp dụng chung CSS cho nhiều phần tử.
Bộ chọn chung (Universal Selector)	`* {}`	Chọn tất cả các thành phần HTML trên trang.
Bộ chọn con trực tiếp (Child Selector)	`parent > child {}`	Chọn phần tử con trực tiếp của phần tử cha.
Bộ chọn con (mọi cấp) (Descendant Selector)	`parent child {}`	Chọn tất cả phần tử con bên trong phần tử cha.
Bộ chọn anh em liền kề (Adjacent Sibling)	`element1 + element2 {}`	Chọn phần tử ngay sau một phần tử khác cùng cấp.
Bộ chọn anh em chung (General Sibling)	`element1 ~ element2 {}`	Chọn tất cả phần tử cùng cấp phía sau một phần tử.
Bộ chọn thuộc tính (Attribute Selector)	`[attribute] {}`	Chọn phần tử có một thuộc tính cụ thể.
Bộ chọn giá trị thuộc tính cụ thể	`[attribute="value"] {}`	Chọn phần tử có thuộc tính bằng giá trị cụ thể.
Bộ chọn giá trị thuộc tính chứa đoạn	`[attribute*="value"] {}`	Chọn phần tử có giá trị thuộc tính chứa đoạn text nhất định.
Pseudo-class (Lớp giả)	`selector:pseudo-class {}`	Chọn phần tử dựa vào trạng thái (hover, focus, checked, first-child, last-child, nth-child,...).
Pseudo-element (Phần tử giả)	`selector::pseudo-elem {}`	Chọn một phần của nội dung (first-letter, first-line, before, after).

9.5.3 2. Chi Tiết Các Bộ Chọn Cơ Bản (Basic Selectors)

[Bảng] Bảng Tổng Hợp Chi Tiết Các Bộ Chọn Cơ Bản (Basic Selectors)

Loại Selector	Cú pháp	Công dụng
Bộ chọn theo thẻ (Type Selector)	`tagname {}`	Chọn tất cả phần tử có tên thẻ cụ thể.
Bộ chọn theo class (Class Selector)	`.classname {}`	Chọn tất cả phần tử có class chỉ định.
Bộ chọn theo ID (ID Selector)	`#idname {}`	Chọn phần tử có ID cụ thể (duy nhất).
Bộ chọn nhóm (Group Selector)	`tag1, tag2 {}`	Áp dụng chung CSS cho nhiều phần tử.
Bộ chọn chung (Universal Selector)	`* {}`	Chọn tất cả các thành phần HTML trên trang.

2.1. Bộ Chọn Theo Thẻ (Type Selector / Tag Selector)

a. Khái niệm:

Bộ chọn theo thẻ (Type Selector) là bộ chọn cơ bản trong CSS, dùng để chọn tất cả các phần tử có cùng tên thẻ HTML và áp dụng chung một quy tắc CSS cho chúng.

b. Cú pháp:

Cú Pháp Type Selector

```
tagname {
    thuoc_tinh: gia_tri;
}
```

- `tagname`: Tên thẻ HTML cần chọn.
- `thuoc_tinh`: Thuộc tính CSS sẽ được áp dụng.
- `gia_tri`: Giá trị của thuộc tính CSS.

c. Các ví dụ minh họa:

Ví Dụ 1: Định Dạng Tất Cả Đoạn Văn <p>

```
p {
    color: blue;
    font-size: 18px;
}
```

[OK] Kết quả: Tất cả các thẻ `<p>` trong trang web sẽ có màu chữ xanh và kích thước chữ 18px.

Ví Dụ 2: Định Dạng Các Tiêu Đề h1, h2, h3

```
h1 {
    color: red;
    text-align: center;
}
h2 {
    color: green;
}
h3 {
    color: orange;
}
```

[OK] Kết quả: `<h1>` màu đỏ căn giữa, `<h2>` màu xanh, `<h3>` màu cam.

Ví Dụ 3: Áp Dụng Font Chữ Cho Nhiều Thẻ Cùng Lúc

```
h1, h2, h3 {
    font-family: Arial, sans-serif;
}
```

[OK] Kết quả: Các tiêu đề `<h1>`, `<h2>`, `<h3>` đều sử dụng font chữ Arial.

d. Cách hoạt động:

Trình duyệt quét HTML để tìm tất cả các phần tử có tên thẻ phù hợp → Áp dụng quy tắc CSS → Hiển thị kết quả đồng bộ.

Mã HTML Và CSS Kiểm Thử Cách Hoạt Động Thẻ

```
<h1>Header Title</h1>
<h2>Header Title</h2>
<p>Paragraph text</p>
<p>Paragraph text</p>
```

```
p {
    color: blue;
    font-size: 16px;
}
h1 {
    text-align: center;
    color: red;
}
```

BROWSER PREVIEW — Kết Quả Hiển Thị Cách Hoạt Động Type Selector

Tiêu đề chính

Tiêu đề phụ

Đây là một đoạn văn bản.

Đây là một đoạn khác.

e. Ưu điểm & Hạn chế của Type Selector:

[Mẹo] Ưu Điểm & Khi Nào Nên Dùng Type Selector

- **Khi nào nên dùng**: Áp dụng quy tắc CSS chung cho tất cả phần tử cùng loại mà không cần gán class hay ID (ví dụ: tất cả `<p>`, `<h1>`, `<table>`, `<button>`).
- **Ưu điểm**: Dễ sử dụng, tiết kiệm thời gian, tăng tính nhất quán giao diện và tạo bố cục nhanh chóng.

Ví Dụ Thiết Lập Kiểu Dáng Đồng Bộ Cho Nút Bấm <button>

```css
button {
    background-color: #ff6600;
    color: white;
    padding: 10px 20px;
    border: none;
    cursor: pointer;
    border-radius: 4px;
}
```

BROWSER PREVIEW — Kết Quả Hiển Thị: Nút Bấm <button> Đồng Bộ Giao Diện

Đăng Nhập Đăng Ký

(→ Kết quả: Tất cả các nút bấm <button> trên trang sẽ có cùng kiểu dáng da cam đồng bộ)

[Cảnh Báo] Hạn Chế Của Type Selector

Hạn chế: Nếu chỉ muốn định dạng một số phần tử cụ thể thay vì tất cả, nên dùng **Class Selector** (`.classname`) hoặc **ID Selector** (`#idname`) để tránh ảnh hưởng ngoài ý muốn đến các phần tử cùng loại khác trên trang.

2.2. Bộ Chọn Theo Lớp (Class Selector)

a. Khái niệm & Cú pháp:

Bộ chọn class chọn các phần tử HTML có thuộc tính `class` nhất định.

Cú Pháp Class Selector

```
.classname {
    thuoc_tinh: gia_tri;
}
```

[Ghi Chú] Lưu Ý Đặt Tên Class

Tên class không được bắt đầu bằng số và không chứa ký tự đặc biệt (trừ dấu gạch ngang - và gạch dưới _). Một phần tử có thể có nhiều class, ngăn cách bằng dấu cách.

b. Ví dụ minh họa HTML + CSS:

Mã HTML Sử Dụng Nhiều Class

```html
<h1 class="title">Welcome</h1>
<p class="content">Paragraph text</p>
<p class="content highlight">Paragraph text</p>
<button class="btn">Sample Text</button>
```

```css
/* Select element */
.title {
    color: #ff6600;
    text-align: center;
    font-size: 28px;
}
/* Select element */
.content {
    color: #333;
    font-size: 16px;
    line-height: 1.5;
}
/* Select element */
.highlight {
    background-color: yellow;
    font-weight: bold;
}
/* Select element */
.btn {
    background-color: #007bff;
    color: white;
    padding: 10px 20px;
    border: none;
    cursor: pointer;
    border-radius: 5px;
}
```

BROWSER PREVIEW — **Kết Quả Hiển Thị Class Selector**

Chào mừng bạn đến với trang web!

Đây là đoạn văn bản nội dung.

Đây là đoạn văn bản nổi bật.

Nhấn vào đây

c. Ưu/Nhược điểm & Tóm tắt nhanh:

[Mẹo] Ưu Điểm & Khi Nào Nên Dùng Class Selector

- **Tái sử dụng cao**: Một class có thể áp dụng cho nhiều phần tử khác nhau trên toàn bộ trang web.
- **Dễ bảo trì**: Thay đổi kiểu dáng ở một nơi duy nhất trong file CSS, tất cả thẻ HTML có class đó sẽ tự động cập nhật.
- **Tránh ảnh hưởng toàn cục**: Không như bộ chọn theo thẻ, class chỉ tác động đến các phần tử cụ thể được khai báo.

[Cảnh Báo] Hạn Chế Của Class Selector

- **Độ ưu tiên thấp hơn ID**: Nếu một phần tử có cả class và ID, kiểu dáng trong ID sẽ ghi đè class.
- **Có thể làm CSS phức tạp**: Nếu lạm dụng quá nhiều class chồng chéo, mã CSS sẽ trở nên khó đọc và khó bảo trì.

[Bảng] Tóm Tắt Nhanh Về Class Selector

Đặc điểm	Chi tiết
Cú pháp	`.classname { thuộc_tính: giá_trị; }`
Chọn phần tử	Chọn tất cả các phần tử có class tương ứng
Ký hiệu	Dấu chấm `.` đứng trước tên class
Độ ưu tiên	Thấp hơn ID, nhưng cao hơn bộ chọn theo thẻ
Tính tái sử dụng	Cao – có thể áp dụng cùng class cho nhiều thẻ
Ứng dụng thực tế	Thiết kế layout linh hoạt, tái sử dụng kiểu dáng, giảm lặp mã

2.3. Bộ Chọn Theo ID (ID Selector)

a. Khái niệm & Cú pháp:

ID là một định danh duy nhất cho mỗi phần tử HTML. Bộ chọn ID dùng dấu thăng #.

Cú Pháp ID Selector

```
#idname {
    thuoc_tinh: gia_tri;
}
```

b. Ví dụ minh họa HTML + CSS:

Mã HTML Sử Dụng ID Duy Nhất

```html
<h1 id="main-title">Welcome</h1>
<p id="intro">Paragraph text</p>
<button id="submit-btn">Submit</button>
```

```css
/* Select element */
#main-title {
    color: #ff6600;
    font-size: 32px;
    text-align: center;
}
/* Select element */
#intro {
    color: #444;
    font-style: italic;
    font-size: 18px;
}
/* Select element */
#submit-btn {
    background-color: #007bff;
    color: white;
    padding: 10px 20px;
    border: none;
    border-radius: 5px;
    cursor: pointer;
}
```

BROWSER PREVIEW — Kết Quả Hiển Thị ID Selector



c. Ưu/Nhược điểm & Khi nào nên dùng ID Selector:

[Mẹo] Ưu Điểm & Khi Nào Nên Dùng ID Selector

- **Chính xác tuyệt đối**: Định dạng chính xác một phần tử duy nhất (như tiêu đề chính, header, footer).
- **Độ ưu tiên cao**: Dễ dàng ghi đè các kiểu dáng class và tag khi cần thiết.
- **Tối ưu với JavaScript**: Rất tiện lợi khi kết hợp với `document.getElementById()`.

[Cảnh Báo] Hạn Chế Của ID Selector

- **Không tái sử dụng**: Mỗi ID là duy nhất trên trang, dùng nhiều lần là vi phạm chuẩn W3C HTML.
- **Khó bảo trì**: Lạm dụng ID sẽ làm tăng độ phức tạp khi muốn mở rộng giao diện.

[Bảng] Tóm Tắt Nhanh Về ID Selector

Đặc điểm	Chi tiết
Cú pháp	`#idname { thuộc_tính: giá_trị; }`
Ký hiệu	Dấu thăng `#` đứng trước tên ID
Độ ưu tiên	Rất cao – cao hơn class và type selector
Tính tái sử dụng	Không – mỗi ID chỉ dùng cho 1 phần tử duy nhất
Ứng dụng thực tế	Định dạng phần tử duy nhất, làm việc với JavaScript

2.4. Bộ Chọn Chung (Universal Selector - *)

Bộ chọn chung dùng dấu sao * giúp bạn chọn tất cả các phần tử HTML trên trang web.

Cú Pháp Universal Selector

```
* {
    thuoc_tinh: gia_tri;
}
```

Các ví dụ ứng dụng phổ biến:

Ví Dụ 1: Reset Margin, Padding Và Thiết Lập Box-Sizing

```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}
```

Ví Dụ 2: Đặt Font Chữ Và Màu Nền Mặc Định Toàn Trang

```
* {
    font-family: Arial, sans-serif;
    background-color: #f0f0f0;
    color: #333;
}
```

Ví Dụ 3: Thêm Viền Đỏ Để Kiểm Tra Layout (Debug)

```
* {
    border: 1px solid red;
}
```

[Ghi Chú] Mẹo Giới Hạn Phạm Vi Của Universal Selector

Để tránh ảnh hưởng toàn trang, bạn có thể giới hạn phạm vi bằng cách kết hợp với bộ chọn cha: `div * { margin: 0; padding: 0; }` → Chỉ áp dụng reset cho các phần tử nằm bên trong thẻ `<div>`.

[Bảng] Tóm Tắt Nhanh Về Universal Selector

Đặc điểm	Chi tiết
Cú pháp	`* { thuộc_tính: giá_trị; }`
Chức năng	Chọn và áp dụng quy tắc CSS cho tất cả phần tử
Ưu điểm	Nhanh, đơn giản, tiện lợi khi cần áp dụng toàn trang
Nhược điểm	Ảnh hưởng quá rộng, giảm hiệu suất, dễ gây xung đột
Trường hợp dùng	Reset CSS, cấu hình toàn trang, kiểm tra layout

9.5.4 3. Bộ Chọn Kết Hợp (Combinator Selectors) Chi Tiết & Ví Dụ Thực Tế

[Khái Niệm] Khái Niệm Về Bộ Chọn Kết Hợp (Combinator Selectors)

Trong CSS, **combinator selectors (bộ chọn kết hợp)** là các bộ chọn dùng để kết nối hoặc liên kết các phần tử dựa trên mối quan hệ của chúng trong cây DOM. Chúng giúp bạn chọn phần tử con, phần tử liền kề, hoặc phần tử nằm cùng cấp cực kỳ linh hoạt!

[Bảng] Bảng Tổng Quan Các Loại Bộ Chọn Kết Hợp (Combinator Selectors)

Loại Selector	Cú pháp	Công dụng
Bộ chọn con trực tiếp (Child Selector)	`parent > child {}`	Chọn phần tử con trực tiếp của phần tử cha.
Bộ chọn con (mọi cấp) (Descendant Selector)	`parent child {}`	Chọn tất cả phần tử con bên trong phần tử cha (mọi cấp).
Bộ chọn anh em liền kề (Adjacent Sibling)	`element1 + element2 {}`	Chọn phần tử ngay sau một phần tử khác cùng cấp.
Bộ chọn anh em chung (General Sibling)	`element1 ~ element2 {}`	Chọn tất cả phần tử cùng cấp phía sau một phần tử.

3.1. Descendant Selector (Bộ Chọn Hậu Duệ) – Chi Tiết & Ví Dụ Thực Tế

[Khái Niệm] Khái Niệm Descendant Selector (Bộ Chọn Hậu Duệ)

Descendant Selector (Bộ chọn hậu duệ) trong CSS dùng để chọn **tất cả các phần tử con, cháu, chắt…** nằm bên trong một phần tử cha, bất kể cấp độ lồng nhau sâu bao nhiêu trong cây DOM.

a. Cú pháp và nguyên tắc hoạt động:

Cú Pháp Descendant Selector (Khoảng Trắng)

```
cha con {
    thuoc_tinh: gia_tri;
}
```

[Ghi Chú] Khoảng Trắng – Ký Hiệu Kỳ Diệu Của Descendant Selector

Khoảng trắng giữa hai bộ chọn chính là ký hiệu của Descendant Selector.

1. **Bước 1**: Tìm phần tử cha (`cha`).
2. **Bước 2**: Tìm tất cả phần tử hậu duệ bên trong nó (`con`), bất kể cấp độ lồng nhau.
3. **Bước 3**: Áp dụng quy tắc CSS cho tất cả các phần tử hậu duệ thỏa mãn.

b. Ví dụ 1: Cơ bản về mức độ lồng nhau (Con, Cháu, Chắt):

Ví Dụ 1: Định Dạng Tất Cả Thẻ p Nằm Trong .container

```
<div class="container">
    <p>Sample Text</p>
    <div class="box">
        <p>Sample Text</p>
        <span>
            <p>Sample Text</p>
        </span>
    </div>
</div>
```

```
.container p {
    color: blue;
    font-weight: bold;
}
```

BROWSER PREVIEW — **Kết Quả Hiển Thị Ví Dụ 1 (.container p)**

.container

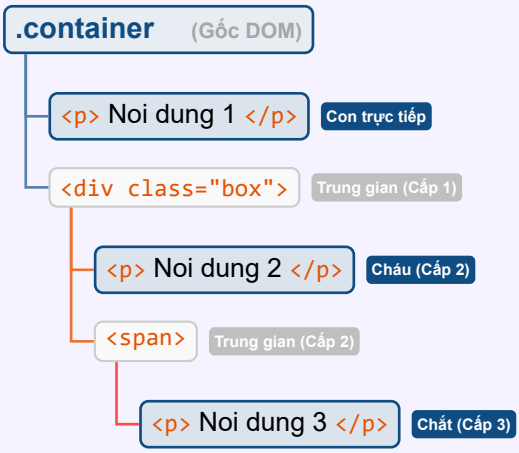
Noi dung 1 ← Con trực tiếp (Được chọn)

.box

Noi dung 2 ← Cháu (Được chọn)

 Noi dung 3 ← Chắt (Được chọn)

[Khái Niệm] Sơ Đồ Phân Cấp Cây DOM Trực Quan Của Ví Dụ 1



Phân tích: `.container p` quét từ gốc `.container` và chọn tất cả thẻ `<p>` bên trong. Hậu duệ bao gồm con trực tiếp (`<p>Noi dung 1</p>`), cháu (`<p>Noi dung 2</p>`) và chắt (`<p>Noi dung 3</p>`).

c. Ví dụ 2: Phân tích chi tiết các cấp P1, P2, P3 trong cây DOM:

Ví Dụ 2: Thử Nghiệm P1, P2, P3 Với .parent p

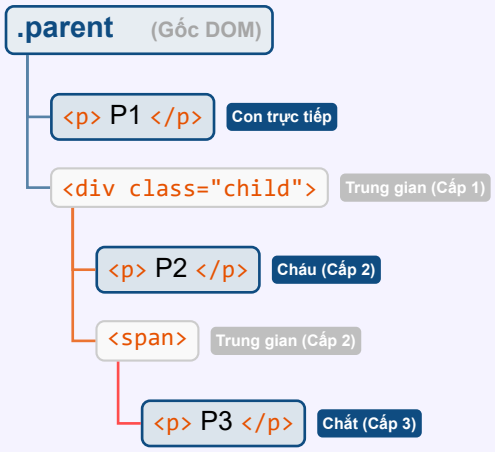
```html
<div class="parent">
    <p>Sample Text</p>
    <div class="child">
        <p>Sample Text</p>
        <span>
            <p>Sample Text</p>
        </span>
    </div>
</div>
```

```css
.parent p {
    color: blue;
    font-weight: bold;
}
```

BROWSER PREVIEW — **Kết Quả Hiển Thị Của Ví Dụ 2**

P1 (trong .parent) → Màu xanh, in đậm
P2 (trong .child, nằm trong .parent) → Màu xanh, in đậm
P3 (trong , nằm trong .child, nằm trong .parent) → Màu xanh, in đậm

[Khái Niệm] Sơ Đồ Phân Cấp Cây DOM Trực Quan Của Ví Dụ 2



Giải thích: Không quan trọng `<p>` nằm ở cấp độ lồng sâu bao nhiêu, chỉ cần nó là hậu duệ của `.parent`, nó sẽ bị ảnh hưởng bởi CSS!

d. Ví dụ 3: So sánh chuyên sâu giữa Selector cụ thể (.parent .child) vs Selector toàn cục (p):

Mã HTML Thử Nghiệm So Sánh Mức Độ Tác Động

```html
<p>Sample Text</p>
<div class="parent">
    <p class="child">Sample Text</p>
    <p class="child">Sample Text</p>
    <div class="child">
        <p>Sample Text</p>
        <p>Sample Text</p>
    </div>
    <span class="child">
        <p>Sample Text</p>
    </span>
</div>
```

Trường hợp 1: Sử Dụng Selector Hậu Duệ Cụ Thể .parent .child

```css
/* Select element */
.parent .child {
    color: red;
    font-weight: bold;
}
```

BROWSER PREVIEW — **Kết Quả Trường Hợp 1 (.parent .child)**

Noi dung 0 ← Không ảnh hưởng (ngoài .parent)

.parent

Noi dung 1 ← Đỏ, đậm (thẻ p có class .child)
Noi dung 2 ← Đỏ, đậm (thẻ p có class .child)

div.child (Đổi màu toàn bộ nội dung trong div)

Noi dung 3 ← Đỏ, đậm (kế thừa từ div.child)
Noi dung 4 ← Đỏ, đậm (kế thừa từ div.child)

span.child: Noi dung 5 ← Đỏ, đậm (nằm trong span.child)

[Ghi Chú] Mở Rộng Thẻ Con Cụ Thể Và Selector Toàn Bộ *

- `.parent .child h3 { color: red; }` → Chỉ chọn thẻ `<h3>` nằm trong phần tử `.child` của `.parent`.
- `.parent .child * { color: red; }` → Chọn **tất cả** các thẻ con mọi cấp bên trong `.child`.

Trường hợp 2: Sử Dụng Selector Thẻ Toàn Cục p

```
/* Select element */
p {
    color: red;
    font-weight: bold;
}
```

[Bảng] So Sánh Tiêu Chí Giữa Selector Hậu Duệ Cụ Thể vs Selector Toàn Cục

Tiêu chí	Trường hợp 1 (.parent .child)	Trường hợp 2 (p)
Phạm vi tác động	Chỉ các phần tử có class `.child` nằm trong `.parent`.	Tất cả thẻ `<p>` trên toàn bộ trang web.
Tính linh hoạt	Cụ thể hơn, dễ quản lý theo cấu trúc HTML từng mô-đun.	Bao trùm toàn bộ, dễ gây tác động ngoài ý muốn.
Tính bảo trì	High (Cao), dễ dàng chỉnh sửa theo từng thành phần nhỏ.	Low (Thấp), vì mọi thay đổi đều ảnh hưởng tới toàn bộ thẻ `<p>`.
Độ ưu tiên (Specificity)	Cao hơn (**20 điểm**: 2 class).	Thấp hơn (**1 điểm**: 1 element).

[Mẹo] Khi Nào Nên Dùng Selector Nào?

- **Nên dùng** `.parent .child`: Khi muốn quản lý phần tử theo cấu trúc cụ thể, định dạng chỉ nội dung trong 1 khu vực nhất định, tăng độ chính xác và tránh vô tình làm hỏng các phần tử ngoài ý muốn.
- **Nên dùng** `p`: Khi muốn định dạng nhanh toàn bộ văn bản chung cho toàn trang hoặc khi viết CSS reset / style tổng quát ban đầu.

e. So sánh Descendant Selector với các loại Combinators khác:

[Bảng] Bảng So Sánh Chi Tiết Các Loại Bộ Chọn Kết Hợp Combinators

Cú pháp Selector	Cách hoạt động	Ví dụ minh họa
`.cha con`	Chọn **tất cả hậu duệ** của `.cha` (mọi cấp).	`.parent p (Chọn P1, P2, P3)`
`.cha > con`	Chỉ chọn **con trực tiếp** của `.cha`.	`.parent > p (Chỉ chọn P1)`
`element1 + element2`	Chọn phần tử `element2` **ngay liền kề** sau `element1`.	`div + p (Chọn p đứng ngay sau div)`
`element1 ~ element2`	Chọn **tất cả anh chị em** `element2` nằm phía sau `element1`.	`h2 ~ p (Chọn tất cả p sau h2)`

So Sánh Trực Quan: Descendant (.parent p) vs Child (.parent > p)

```css
/* Select element */
.parent p {
    color: blue;
}

/* Sample Text */
.parent > p {
    color: red;
}
```

BROWSER PREVIEW — **Kết Quả So Sánh Descendant vs Child**

P1 (trong .parent) → **Đỏ** (với `.parent > p` vì là con trực tiếp)
P2, P3 → **Xanh** (với `.parent p` vì là hậu duệ)

So Sánh Trực Quan: Descendant (.parent p) vs Adjacent Sibling (div + p)

```css
/* Select element */
.parent p {
    color: blue;
}

/* Sample Text */
div + p {
    color: red;
}
```

f. 3 Ứng dụng thực tế phổ biến:

[Mẹo] 3 Ứng Dụng Thực Tế Hàng Đầu Của Descendant Selector

- **Ứng dụng 1**: Định dạng tất cả tiêu đề `<h2>` nằm bên trong bài viết `<article>`, bất kể cấp độ lồng nhau.
- **Ứng dụng 2**: Thiết kế liên kết menu điều hướng nhiều cấp `nav ul li a`.
- **Ứng dụng 3**: Định dạng đồng bộ tất cả các ô nhập liệu `input` nằm trong thẻ `<label>` của `<form>`.

Ứng Dụng 1: Định Dạng Tất Cả Tiêu Đề h2 Trong Bài Viết article

```
article h2 {
    color: #ff6347; /* Sample Text */
    font-size: 24px;
}
```

BROWSER PREVIEW **Kết Quả: Tiêu Đề h2 Trong Article Đổi Màu Đỏ Cà Chua**

<article> (Khối bài viết)

Tiêu Đề Bài Viết h2 ← được chọn (màu #ff6347, 24px)

Nội dung chi tiết của bài viết được trình bày tại đây...

Ứng Dụng 2: Thiết Kế Menu Điều Hướng Nhiều Cấp (nav ul li a)

```
nav ul li a {
    text-decoration: none;
    color: #333;
    font-weight: bold;
}
```

BROWSER PREVIEW **Kết Quả: Các Thẻ a Trong Menu Điều Hướng Phân Cấp**

Trang chủ | Khóa học ↓

(→ Chọn tất cả thẻ <a> trong menu, bất kể lồng sâu bao nhiêu cấp trong <ul> và <li>)

Ứng Dụng 3: Định Dạng Đồng Bộ Tất Cả Ô Input Trong Form

```
form label input {
    border: 1px solid #ccc;
    padding: 8px;
    border-radius: 4px;
}
```

BROWSER PREVIEW — **Kết Quả: Ô Input Nằm Trong Thẻ Label Của Form**

Họ và tên: Nhập họ tên...

(→ Chọn tất cả thẻ <input> nằm bên trong <label> của <form>)

g. Lưu ý quan trọng & Tóm tắt:

[Cảnh Báo] 2 Lưu Ý Cực Kỳ Quan Trọng Khi Sử Dụng Descendant Selector

- **Hiệu suất (Performance)**: Nếu cây DOM quá lớn và lồng nhau quá sâu (ví dụ: `body div main section article div p`), trình duyệt phải quét nhiều cấp để so khớp, có thể làm chậm tốc độ render trang web.
- **Độ ưu tiên (Specificity)**: Nếu có nhiều bộ chọn cùng tác động lên một phần tử, trình duyệt sẽ tính điểm Specificity để quyết định kiểu dáng nào thắng.

[Mẹo] Tóm Tắt Về Descendant Selector (Khoảng Trắng)

- **Descendant Selector** rất mạnh mẽ và tiện lợi khi cần áp dụng style cho nhiều phần tử con và hậu duệ.
- Chỉ cần một **khoảng trắng**, bạn đã có thể chọn mọi phần tử nằm bên trong phần tử cha, bất kể cấp độ sâu.
- **Nhớ rằng**: Hãy kiểm soát độ sâu của bộ chọn để tránh tác động ngoài mong muốn đến các thành phần UI khác!

3.2. Child Selector (Bộ Chọn Con Trực Tiếp) – Chi Tiết & Cơ Chế Xử Lý

[Khái Niệm] Khái Niệm Child Selector (bộ chọn con trực tiếp)

Child Selector (Bộ chọn con trực tiếp) trong CSS là một bộ chọn dùng để chọn các **phần tử con trực tiếp** của phần tử cha, tức là các phần tử nằm ngay bên trong phần tử cha mà không đi qua trung gian. Nếu phần tử con nằm sâu hơn một cấp (là cháu, chắt…), thì **không** bị ảnh hưởng bởi bộ chọn này.

a. Cú pháp và cách hoạt động:

Cú Pháp Child Selector (>)

```
A > B {
    /* CSS Commentp dCommentng cho B */
}
```

- `A`: Phần tử cha.
- `B`: Phần tử con trực tiếp của A (ngay bên trong A, không lồng thêm cấp nào).

[Ghi Chú] Cơ Chế Hoạt Động Của Trình Duyệt Khi Xử Lý A > B

Khi bạn viết `A > B` trong CSS, trình duyệt sẽ thực hiện theo 2 bước:

1. **Bước 1**: Tìm phần tử `A` (phần tử cha).
2. **Bước 2**: Kiểm tra các phần tử bên trong `A`:
   - Nếu phần tử `B` là **con trực tiếp** của `A` → **Áp dụng CSS**.
   - Nếu phần tử `B` **không** phải con trực tiếp (là cháu, chắt) → **Bỏ qua, không áp dụng CSS**.

Ghi chú: Con trực tiếp là phần tử nằm ngay bên trong phần tử cha, không qua trung gian.

b. Trình tự xử lý của trình duyệt (Browser Pipeline):

[Khái Niệm] 3 Giai Đoạn Trình Duyệt Xử Lý CSS

Trình duyệt xử lý CSS theo thứ tự duyệt cây DOM:

1. **1. Phân tích cấu trúc HTML** → Xây dựng **DOM Tree** (Cây DOM).
2. **2. Áp dụng quy tắc CSS**:
   - Kiểm tra bộ chọn (`A > B`).
   - So khớp phần tử: Nếu `B` là con trực tiếp của `A`, áp dụng CSS.
   - Bỏ qua phần tử không khớp quan hệ trực tiếp.
3. **3. Kết xuất giao diện (Render)** → Hiển thị kết quả ra màn hình.

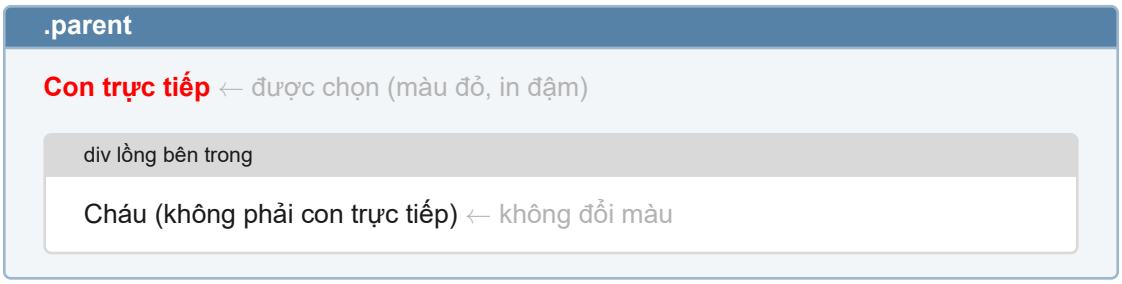
c. Ví dụ minh họa cơ bản (Con vs Cháu):

Ví Dụ 1: Phân Biệt Con Trực Tiếp Và Cháu

```html
<div class="parent">
    <p>Sample Text</p>
    <div>
        <p>Sample Text</p>
    </div>
</div>
```

```css
.parent > p {
    color: red;
    font-weight: bold;
}
```

BROWSER PREVIEW — Kết Quả Hiển Thị Ví Dụ 1 (.parent > p)



[Ghi Chú] Phân Tích Chi Tiết Quá Trình So Khớp Ví Dụ 1

Trình duyệt đọc CSS và tìm phần tử `.parent`, sau đó kiểm tra các thẻ `<p>`:

- **Thẻ <p> đầu tiên**: Thỏa mãn (là con trực tiếp) → Áp dụng CSS (màu đỏ, in đậm).
- **Thẻ <p> thứ hai**: Không thỏa mãn (là cháu, nằm trong thẻ `<div>`) → Bỏ qua, không áp dụng CSS.

d. Ví dụ nâng cao (Phân tích lồng ghép nhiều cấp):

Ví Dụ 2: Phân Tích Cấu Trúc Lồng Nhiều Cấp VỚI div > p

```html
<p>Sample Text</p>
<div>
    <p>Sample Text</p>
    <p>Sample Text</p>
    <div>
        <p>Sample Text</p>
        <p>Sample Text</p>
    </div>
    <span>
        <p>Sample Text</p>
    </span>
</div>
```

```css
div > p {
    color: red;
    font-weight: bold;
}
```

[Ghi Chú] Phân Tích Từng Phần Tử Trong Ví Dụ 2

1. `<p>Noi dung 0</p>`: **Không được chọn** vì đứng độc lập, không nằm trong `<div>`.
2. `<p>Noi dung 1</p>` & `<p>Noi dung 2</p>`: **Được chọn** vì là con trực tiếp của thẻ `<div>` ngoài.
3. `<p>Noi dung 3</p>` & `<p>Noi dung 4</p>`: **Được chọn** vì là con trực tiếp của thẻ `<div>` trong.
4. `<p>Noi dung 5</p>`: **Không được chọn** vì nó là con trực tiếp của `<span>`, không phải con trực tiếp của `<div>`.

BROWSER PREVIEW — **Kết Quả Hiển Thị Chi Tiết Ví Dụ 2 (div > p)**

Noi dung 0 ← Bình thường (ngoài div)

div ngoài

Noi dung 1 ← màu đỏ, in đậm
Noi dung 2 ← màu đỏ, in đậm

div trong

Noi dung 3 ← màu đỏ, in đậm (con trực tiếp của div trong)
Noi dung 4 ← màu đỏ, in đậm (con trực tiếp của div trong)

**** Noi dung 5 ← Bình thường (con của span)

e. Ví dụ chuỗi Selector 3 cấp (.container > .box > p):

Ví Dụ 3: Chuỗi Selector 3 Cấp Độ Lồng Nhau

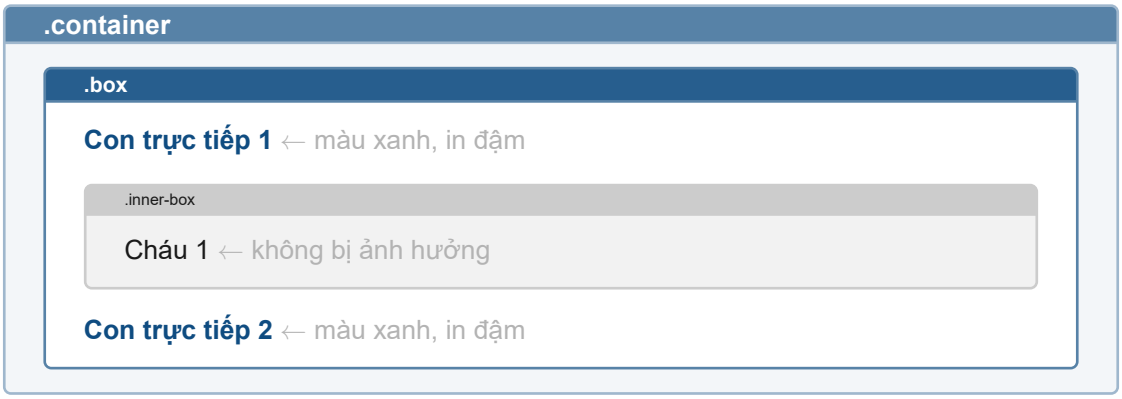
```html
<div class="container">
    <div class="box">
        <p>Sample Text</p>
        <div class="inner-box">
            <p>Sample Text</p>
        </div>
        <p>Sample Text</p>
    </div>
</div>
```

```css
.container > .box > p {
    color: blue;
    font-weight: bold;
}
```

[Ghi Chú] Phân Tích Chuỗi Selector .container > .box > p

1. Tìm `.container`.
2. Tìm `.box` là con trực tiếp của `.container`.
3. Tìm các thẻ `<p>` là con trực tiếp của `.box`.
4. Áp dụng màu xanh và in đậm cho `<p>` con trực tiếp của `.box` (bỏ qua `<p>` nằm trong `.inner-box`).

BROWSER PREVIEW — Kết Quả Hiển Thị Chuỗi Selector 3 Cấp



f. So sánh với Descendant Selector (Bộ chọn hậu duệ):

[Bảng] So Sánh Child Selector (>) vs Descendant Selector (Khoảng Trắng)

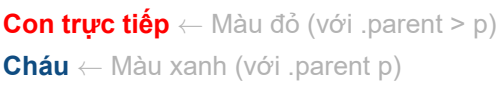
Bộ chọn	Ý nghĩa	Phạm vi chọn
`A > B`	`Chọn con trực tiếp của A`	Chỉ phần tử con ngay bên trong A
`A B`	`Chọn tất cả hậu duệ của A`	Chọn tất cả phần tử bên trong A (bao gồm con, cháu, chắt…)

So Sánh Trực Quan: .parent p vs .parent > p

```
/* Select element */
.parent p {
    color: blue;
}

/* Sample Text */
.parent > p {
    color: red;
}
```

BROWSER PREVIEW — Kết Quả So Sánh: Con Trực Tiếp vs Cháu



g. 3 Quy tắc quan trọng cốt lõi:

[Ghi Chú] 3 Quy Tắc Quan Trọng Khi Sử Dụng Child Selector

1. **Chọn đúng con trực tiếp**: Chỉ áp dụng CSS cho phần tử nằm ngay bên trong phần tử cha.
2. **Không ảnh hưởng đến hậu duệ sâu hơn**: Nếu phần tử là cháu, chắt (không trực tiếp), trình duyệt tự động bỏ qua.
3. **Tính thứ tự trong DOM**: Chỉ xét theo cấu trúc HTML, không quan tâm vị trí hiển thị trực quan trên giao diện màn hình.

h. Trường hợp không hoạt động:

[Cảnh Báo] Trường Hợp Child Selector KHÔNG Hoạt Động

Không có quan hệ cha-con trực tiếp: Nếu phần tử không nằm ngay bên trong phần tử cha (bị bọc bởi một thẻ khác), bộ chọn `>` sẽ **không** áp dụng.

Ví Dụ Sai: Thẻ p Nằm Trong article Nên Không Phải Con Trực Tiếp Của section

```html
<section>
    <article>
        <p>Sample Text</p>
    </article>
</section>
```

```css
section > p {
    color: green;
}
```

BROWSER PREVIEW — **Kết Quả: Không Có Thay Đổi**

Không phải con trực tiếp ← Không có gì thay đổi vì thẻ `<p>` nằm trong `<article>`, không phải con trực tiếp của `<section>`.

i. Các ứng dụng thực tế phong phú:

[Mẹo] 5 Ứng Dụng Thực Tế Mạnh Mẽ Của Child Selector

- **1. Cấu trúc menu điều hướng (`.nav > li > a`)**: Chỉ chọn thẻ `<a>` là con trực tiếp của `<li>`, tránh áp dụng CSS lên các thẻ con lồng sâu hơn trong menu dropdown.
- **2. Layout card sản phẩm (`.card > .title`)**: Chỉ chọn tiêu đề là con trực tiếp của thẻ card, giữ cấu trúc gọn gàng.
- **3. Bố cục lưới Grid / Flexbox (`.grid > .item`)**: Chọn phần tử item trực tiếp trong grid, giúp dễ dàng căn chỉnh và quản lý khoảng cách.
- **4. Ẩn/hiện phần tử footer con trực tiếp (`.card > .footer`)**: Ẩn phần footer nếu nó là con trực tiếp của card.
- **5. Định dạng cấu trúc bảng (`table > tbody > tr`)**: Thêm đường viền dưới mỗi hàng của bảng, chỉ khi là trực tiếp trong `<tbody>`.

Ứng Dụng 1: Cấu Trúc Menu Điều Hướng (.nav > li > a)

```
.nav > li > a {
    text-decoration: none;
    padding: 10px;
    color: #007bff;
}
```

Ứng Dụng 2: Card Sản Phẩm (.card > .title)

```
.card > .title {
    font-size: 20px;
    font-weight: bold;
    color: #333;
}
```

Ứng Dụng 3: Bố Cục Lưới Flexbox/Grid (.grid > .item)

```
.grid > .item {
    flex: 1;
    margin: 10px;
}
```

[Mẹo] Tóm Tắt Về Child Selector (>)

- **Child Selector (>)** giúp chọn chính xác phần tử con trực tiếp của một phần tử cha mà không ảnh hưởng đến phần tử lồng sâu hơn.
- **Hoạt động theo cấu trúc DOM**: Phân tích, so khớp, và áp dụng CSS theo thứ tự cây DOM.
- **Ứng dụng rộng rãi** trong thiết kế layout, menu điều hướng, card sản phẩm, và cấu trúc trang web phức tạp.

3.3. Adjacent Sibling Selector (Bộ Chọn Anh Chị Liền Kề) – Chi Tiết

[Khái Niệm] Khái Niệm Adjacent Sibling Selector

Adjacent Sibling Selector (Bộ chọn anh chị liền kề) là một loại bộ chọn trong CSS dùng để chọn phần tử ngay liền kề sau một phần tử khác trong cây DOM, cùng cấp cha. Bộ chọn này rất hữu ích khi bạn muốn tác động chỉ đến phần tử đứng ngay sau một phần tử cụ thể, thay vì chọn tất cả các phần tử sau đó.

a. Cú pháp:

Cú Pháp Adjacent Sibling Selector (+)

```
element1 + element2 {
    /* CSS styles */
}
```

- `element1`: Phần tử đứng trước.
- `element2`: Phần tử liền kề ngay sau phần tử 1.

[Ghi Chú] Lưu Ý Quan Trọng

Hai phần tử phải **cùng cấp** (sibling elements) và **liền kề nhau** trong cây DOM. Nếu có bất kỳ phần tử nào chen giữa, bộ chọn sẽ không hoạt động.

b. Ví dụ cơ bản:

Ví Dụ Cơ Bản Adjacent Sibling Selector

```
<div class="box">Box</div>
<div class="text">Sample Text</div>
<div class="text">Sample Text</div>
```

```
.box + .text {
    color: red;
}
```

BROWSER PREVIEW — Kết Quả Hiển Thị Ví Dụ Cơ Bản (.box + .text)

Box

Văn bản 1 ← được chọn (liền kề ngay sau .box)

Văn bản 2 ← không bị ảnh hưởng (không nằm ngay sát .box)

c. Nguyên tắc hoạt động:

[Ghi Chú] 3 Nguyên Tắc Hoạt Động Của Adjacent Sibling Selector

1. **Cùng cấp (siblings)**: Hai phần tử phải cùng nằm trong một phần tử cha.
2. **Liền kề nhau (adjacent)**: Phần tử thứ hai phải ngay sau phần tử thứ nhất (không có phần tử nào chen giữa).
3. **Một phần tử duy nhất**: CSS sẽ chỉ chọn phần tử liền kề đầu tiên, không áp dụng cho các phần tử sau đó.

d. Phân tích Element 1 và Element 2:

Với Adjacent Sibling Selector (`+`), chỉ phần tử thứ hai (`element2`) — nếu nằm ngay sau `element1` — mới bị ảnh hưởng bởi CSS. **Element 1 hoàn toàn không bị tác động.**

Phân Tích Chi Tiết: Element 1 vs Element 2

```html
<div class="box">Box</div>
<div class="text">Sample Text</div>
<div class="text">Sample Text</div>
```

```css
.box + .text {
    color: red;
}
```

BROWSER PREVIEW — Phân Tích Kết Quả: Phần Tử Nào Bị Ảnh Hưởng?

Box ← .box (element1) → **không bị ảnh hưởng**

Văn bản 1 ← .text ngay sau .box → **đỏ**

Văn bản 2 ← không liền kề trực tiếp .box → **không thay đổi**

e. Khi có phần tử chen giữa:

Trường Hợp Phần Tử Chen Giữa – Adjacent Sibling Thất Bại

```html
<div class="box">Box</div>
<span>Sample Text</span>
<div class="text">Sample Text</div>
```

```css
.box + .text {
    color: red;
}
```

BROWSER PREVIEW — **Kết Quả Khi Có Phần Tử Chen Giữa**

Box

Chen giữa

Văn bản 1 ← không đổi màu (bị `<span>` chen giữa)

[Cảnh Báo] Khi Nào Adjacent Sibling KHÔNG Hoạt Động?

Nếu có **bất kỳ phần tử nào chen giữa** `element1` và `element2`, bộ chọn sẽ không áp dụng. Nếu bạn muốn chọn tất cả các phần tử phía sau mà không cần liền kề, hãy dùng **General Sibling Selector (~)**!

f. Khái niệm "Cùng cấp" (Sibling Elements):

[Khái Niệm] Hiểu Rõ "Cùng Cấp" Trong Cây DOM

Trong HTML, khi nói **cùng cấp**, ta đang nói về các **sibling elements** (phần tử anh chị em). Chúng là những phần tử nằm cùng một cấp bậc trong cây DOM và có cùng phần tử cha. Hãy hình dung cấu trúc như một cây gia đình:

- **Phần tử cha (parent element)**: Giống như bố mẹ.
- **Phần tử con (child element)**: Là con cái.
- **Phần tử cùng cấp (sibling element)**: Là anh chị em ruột — các phần tử con của cùng một phần tử cha.

Cùng nằm trên 1 đường thẳng sẽ là cùng cấp với nhau!

Ví Dụ Minh Họa Khái Niệm Sibling Elements

```html
<div class="parent">
    <div class="child-1">Child 1</div>
    <div class="child-2">Child 2</div>
    <div class="child-3">Child 3</div>
</div>
```

```css
.child-1 + .child-2 {
    color: red;
}
```

BROWSER PREVIEW — **Kết Quả Minh Họa Sibling Elements**

.parent (Phần tử cha)

Child 1 ← .child-1 (element1, không bị ảnh hưởng)

Child 2 ← .child-2 (liền kề, đổi đỏ)

Child 3 ← .child-3 (không liền kề .child-1, không đổi)

Child 1, Child 2, Child 3 là các phần tử cùng cấp (siblings) vì chúng đều là con trực tiếp của .parent.

g. Ví dụ chi tiết hơn:

Ví Dụ Chi Tiết: h1 + p

```html
<h1>Header Title</h1>
<p>Sample Text</p>
<p>Sample Text</p>
```

```css
h1 + p {
    font-weight: bold;
    color: blue;
}
```

BROWSER PREVIEW — **Kết Quả Hiển Thị h1 + p**

Tiêu đề

Đoạn văn đầu tiên ← in đậm + xanh dương

Đoạn văn thứ hai ← không bị ảnh hưởng

h. Trường hợp không hoạt động:

Trường Hợp Không Hoạt Động: Phần Tử Chen Giữa h1 và p

```html
<h1>Header Title</h1>
<span>Sample Text</span>
<p>Sample Text</p>
```

```css
h1 + p {
    color: green;
}
```

BROWSER PREVIEW — **Kết Quả: Không Có Phần Tử Nào Đổi Màu**



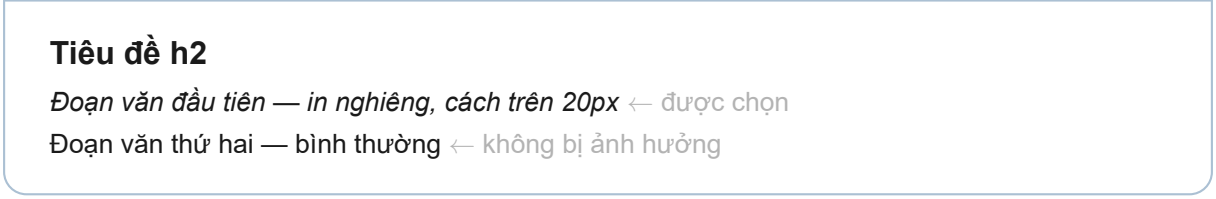
[Mẹo] Ứng Dụng Thực Tế Của Adjacent Sibling Selector

- **Ứng dụng 1**: Tạo khoảng cách hoặc định dạng đặc biệt cho phần tử ngay sau tiêu đề.
- **Ứng dụng 2**: Định dạng menu hoặc nút bấm liền kề nhau.

Ứng Dụng 1: Định Dạng Đoạn Văn Ngay Sau Tiêu Đề

```css
h2 + p {
    margin-top: 20px;
    font-style: italic;
}
```

BROWSER PREVIEW — **Kết Quả: Đoạn Văn Đầu Tiên Sau h2 Được In Nghiêng**



Ứng Dụng 2: Định Dạng Nút Bấm Liền Kề

```
button.primary + button.secondary {
    margin-left: 10px;
    background-color: gray;
}
```

BROWSER PREVIEW — **Kết Quả: Nút Primary và Secondary Liền Kề**

Primary Secondary

(→ Nút Secondary (liền kề sau Primary) có nền xám và cách trái 10px.)

j. Tóm tắt nhanh & Kết luận:

[Bảng] Tóm Tắt Nhanh Về Adjacent Sibling Selector

Đặc điểm	Chi tiết
Cú pháp	`element1 + element2 { }`
Ký hiệu	Dấu cộng `+` giữa 2 phần tử
Phạm vi chọn	Chỉ chọn phần tử liền kề ngay sau (1 phần tử duy nhất)
Yêu cầu	Hai phần tử phải cùng cấp và liền kề, không có phần tử chen giữa
Element 1	Không bị ảnh hưởng bởi CSS
Element 2	Bị ảnh hưởng bởi CSS (nếu liền kề)
So sánh với ~	`+` chọn 1 phần tử liền kề, `~` chọn tất cả phía sau

[Mẹo] Kết Luận Về Adjacent Sibling Selector

- **Adjacent sibling selector (+)** cực kỳ mạnh khi bạn muốn kiểm soát phần tử ngay sau một phần tử khác.
- Tuy nhiên, nếu bạn muốn chọn **tất cả phần tử phía sau**, hãy dùng **General Sibling Selector (~)**!

3.4. General Sibling Selector (Bộ Chọn Anh Chị Tổng Quát) – Chi Tiết

[Khái Niệm] Khái Niệm General Sibling Selector

General Sibling Selector (bộ chọn anh chị tổng quát) là một công cụ mạnh trong CSS giúp bạn chọn **tất cả** các phần tử cùng cấp nằm sau phần tử được chọn, không cần phải liền kề như với Adjacent Sibling Selector (+). Nó hoạt động bằng cách quét từ trên xuống dưới trong DOM và chọn tất cả các phần tử phù hợp nằm sau phần tử gốc.

a. Cú pháp:

Cú Pháp General Sibling Selector (~)

```
A ~ B {
    /* CSS styles */
}
```

- A: Phần tử gốc (anchor element).
- B: Phần tử anh chị em tổng quát của A (cùng cấp và nằm sau A).

[Ghi Chú] Điều Kiện Hoạt Động

- A và B phải **cùng cấp** (cùng cha) trong cây DOM.
- B phải **nằm sau** A, nhưng **không cần liền kề** ngay lập tức.

b. Ví dụ cơ bản:

Ví Dụ Cơ Bản: Chọn Tất Cả Thẻ p Sau h2

```
<h2>Header Title</h2>
<p>Sample Text</p>
<p>Sample Text</p>
<p>Sample Text</p>
```

```
h2 ~ p {
    color: blue;
    font-weight: bold;
}
```

BROWSER PREVIEW — Kết Quả Hiển Thị h2 ~ p

Tiêu đề

Đoạn 1 ← được chọn
Đoạn 2 ← được chọn
Đoạn 3 ← được chọn

(→ Tất cả các thẻ `<p>` sau `<h2>` đều có màu xanh và chữ đậm.)

c. Ứng dụng thực tế:

[Mẹo] 4 Ứng Dụng Thực Tế Mạnh Mẽ Của General Sibling Selector

- **Ứng dụng 1**: Ẩn/hiện nội dung bằng checkbox (không cần JavaScript!).
- **Ứng dụng 2**: Hover làm sáng các mục menu phía sau.
- **Ứng dụng 3**: Đánh dấu viền đỏ cho ghi chú sau tiêu đề.
- **Ứng dụng 4**: Ẩn tất cả thông báo khi nhấn nút.

Ứng Dụng 1: Toggle Ẩn/Hiện Bằng Checkbox

```html
<input type="checkbox" id="toggle">
<p>Content text</p>
<p>Content text</p>
```

```css
/* Paragraph text */
#toggle:checked ~ p {
    display: none;
}
```

BROWSER PREVIEW — Kết Quả: Toggle Checkbox Ẩn/Hiện Nội Dung

☐ x **Ẩn nội dung**

~~Nội dung 1~~ ← ẩn đi
~~Nội dung 2~~ ← ẩn đi

(→ Khi check vào ô, tất cả các đoạn văn sau checkbox sẽ biến mất!)

Ứng Dụng 2: Hover Làm Sáng Các Mục Phía Sau

```html
<div class="menu">Sample Text</div>
<div class="menu">Sample Text</div>
<div class="menu">Sample Text</div>
```

```css
.menu:hover ~ .menu {
    background-color: yellow;
}
```

BROWSER PREVIEW — **Kết Quả: Hover Vào Mục 1 Làm Sáng Mục 2 và 3**

Mục 1 ← đang hover

Mục 2 ← đổi nền vàng

Mục 3 ← đổi nền vàng

Ứng Dụng 3: Đánh Dấu Viền Đỏ Cho Ghi Chú Sau Tiêu Đề

```html
<h3>Header Title</h3>
<div class="note">Sample Text</div>
<div class="note">Sample Text</div>
```

```css
h3 ~ .note {
    border-left: 3px solid red;
    padding-left: 10px;
}
```

BROWSER PREVIEW — **Kết Quả: Viền Đỏ Bên Trái Cho Ghi Chú**

Tiêu đề chính

Lưu ý 1

Lưu ý 2

(→ Tất cả các phần tử .note sau thẻ `<h3>` sẽ có viền đỏ bên trái.)

Ứng Dụng 4: Ẩn Tất Cả Thông Báo Khi Nhấn Nút

```html
<button id="hideBtn">Sample Text</button>
<div class="alert">Sample Text</div>
<div class="alert">Sample Text</div>
```

```css
#hideBtn:active ~ .alert {
    display: none;
}
```

BROWSER PREVIEW — **Kết Quả: Nhấn Giữ Nút Ẩn Thông Báo**

Ẩn thông báo

~~Thông báo 1~~ ← biến mất

~~Thông báo 2~~ ← biến mất

(→ Khi nhấn giữ nút, tất cả thông báo phía sau sẽ biến mất!)

d. Phân biệt với Adjacent Sibling Selector (+):

[Bảng] So Sánh Adjacent Sibling (+) vs General Sibling (~)

Bộ chọn	Ý nghĩa	Hoạt động
`A + B`	Chọn phần tử B liền kề ngay sau A	Chỉ chọn phần tử đầu tiên nằm ngay sau A
`A ~ B`	Chọn tất cả phần tử B nằm sau A	Chọn tất cả các phần tử sau A, không cần liền kề

So Sánh Trực Quan: + vs ~

```css
/* Paragraph text */
h2 + p { color: red; }

/* Paragraph text */
h2 ~ p { color: blue; }
```

e. Nguyên tắc hoạt động:

[Ghi Chú] 3 Nguyên Tắc Hoạt Động Của General Sibling Selector

1. **Cùng cấp trong cây DOM**: Phần tử được chọn phải cùng cấp (sibling) với phần tử gốc.
2. **Xét theo thứ tự xuất hiện**: CSS sẽ quét từ trên xuống dưới trong DOM và chọn tất cả các phần tử phù hợp.
3. **Không chọn phần tử phía trước**: Chỉ các phần tử nằm **sau** phần tử gốc mới bị ảnh hưởng.

f. Ví dụ nâng cao (phần tử xen kẽ):

Ví Dụ Nâng Cao: Phần Tử Xen Kẽ Không Ảnh Hưởng

```html
<div class="container">
    <h2>Header Title</h2>
    <p>Sample Text</p>
    <span>Sample Text</span>
    <p>Sample Text</p>
    <p>Sample Text</p>
</div>
```

```css
h2 ~ p {
    background-color: yellow;
}
```

BROWSER PREVIEW Kết Quả: Chỉ Thẻ p Được Chọn, span Không Bị Ảnh Hưởng

.container

Tiêu đề

Đoạn văn 1 ← nền vàng

Span không bị ảnh hưởng ← không phải <p>

Đoạn văn 2 ← nền vàng

Đoạn văn 3 ← nền vàng

g. Trường hợp không hoạt động:

[Cảnh Báo] 2 Trường Hợp General Sibling KHÔNG Hoạt Động

1. **Không cùng cấp**: Nếu A và B không cùng cấp (khác phần tử cha), thì ~ sẽ không hoạt động.
2. **B nằm trước A**: Nếu phần tử B xuất hiện **trước** A trong DOM, bộ chọn cũng không áp dụng CSS cho B.

Ví Dụ Sai: B Nằm Trước A

```html
<p>Sample Text</p>
<h2>Header Title</h2>
<p>Sample Text</p>
```

```css
/* Header Title */
p ~ h2 {
    color: red;
}
```

BROWSER PREVIEW — **Kết Quả: Thứ Tự Trong DOM Rất Quan Trọng**

Đoạn văn 1 ← nằm trước, không bị ảnh hưởng
Tiêu đề ← nằm sau `<p>`, bị đổi đỏ
Đoạn văn 2 ← không phải `<h2>`, không bị chọn

h. Mẹo sử dụng hiệu quả:

[Mẹo] Mẹo Sử Dụng General Sibling Selector Hiệu Quả

- **Kết hợp với pseudo-classes** như `:hover`, `:checked`, hoặc `:nth-child` để tạo hiệu ứng phức tạp mà không cần JavaScript.
- **Kết hợp với position và z-index** để xây dựng giao diện động, ví dụ như tooltip, popup menu, hoặc progress bar.
- **Khi nào nên dùng?** Khi bạn muốn chọn **nhiều phần tử cùng cấp** nằm sau một phần tử cụ thể. Rất hữu ích trong tạo menu động, ẩn/hiện nội dung, progress bar, và form validation.

i. Tóm tắt nhanh & Kết luận:

[Bảng] Tóm Tắt Nhanh Về General Sibling Selector

Đặc điểm	Chi tiết
Cú pháp	`A ~ B { }`
Ký hiệu	Dấu ngã ~ giữa 2 phần tử
Phạm vi chọn	Chọn tất cả phần tử B cùng cấp nằm sau A
Yêu cầu	A và B phải cùng cha, B phải nằm sau A
Liền kề?	Không cần liền kề, có phần tử xen kẽ vẫn hoạt động
So sánh với +	+ chọn 1 phần tử liền kề, ~ chọn tất cả phía sau
Ứng dụng	Toggle checkbox, hover nhóm, viền ghi chú, ẩn thông báo

[Mẹo] Kết Luận Về General Sibling Selector

- **General Sibling Selector (~)** giúp chọn tất cả phần tử cùng cấp nằm sau phần tử gốc, không cần liền kề.
- **Thứ tự trong DOM rất quan trọng**: Chỉ chọn các phần tử xuất hiện sau phần tử gốc.
- **Ứng dụng cực kỳ linh hoạt**: Tạo hiệu ứng hover nhóm, ẩn/hiện nội dung bằng checkbox, làm nổi bật các phần tử liên quan… mà không cần JavaScript!

3.5. Bảng Tóm Tắt Dễ Nhớ Về Combinators

[Bảng] Bảng Tóm Tắt Dễ Nhớ Các Bộ Chọn Kết Hợp Combinator

Combinator	Ký hiệu	Ý nghĩa	Ví dụ
Descendant	(Khoảng trắng)	Chọn tất cả hậu duệ (con/cháu mọi cấp)	`.parent .child`
Child	`>`	Chọn con trực tiếp	`.parent > .child`
Adjacent	`+`	Chọn phần tử liền kề ngay sau	`.box + .text`
General	`~`	Chọn tất cả anh chị phía sau	`.box ~ .text`

3.6. Ví Dụ Thực Tế: Tạo Menu Điều Hướng Với Trạng Thái Hover

Tạo Menu Dropdown Điều Hướng Bằng Child Selector & Hover

```css
/* Sample Text */
nav ul > li:hover > ul {
    display: block;
}
```

```html
<nav>
    <ul>
        <li>Menu 1
            <ul>
                <li>Sample Text</li>
                <li>Sample Text</li>
            </ul>
        </li>
    </ul>
</nav>
```

BROWSER PREVIEW **Kết Quả Hiển Thị: Menu Dropdown Xuất Hiện Khi Hover**

Menu 1 ↓

Mục 1
Mục 2

(→ Kết quả: Khi di chuột (hover) vào Menu 1, danh sách menu con dropdown tự động xuất hiện!)

[Mẹo] Kết Luận Về Bộ Chọn Kết Hợp Combinator Selectors

- **Linh hoạt mã nguồn**: Combinator selectors giúp bạn viết CSS linh hoạt hơn, chọn đúng phần tử mà không cần thêm class hoặc ID thừa vào HTML.
- **Tối ưu hiệu suất**: Hiểu sâu về các selector này sẽ giúp bạn xây dựng giao diện gọn gàng, dễ bảo trì và tối ưu hiệu suất tải trang!

9.5.5 4. Chuyên Đề Phân Biệt Chi Tiết: ”Phần Tử Cùng Cấp” & ”Phần Tử Con Trực Tiếp” (Masterclass Structural Combinators)

[Khái Niệm] Khái Niệm Phần Tử Cùng Cấp (Sibling Element) Là Gì?

Phần tử cùng cấp (Sibling Element) là những phần tử nằm trong **cùng một phần tử cha (parent)**.

Nói cách khác: Chúng có cùng ”cha trực tiếp” trong cây DOM.

[Mẹo] Mẹo Dễ Nhớ Về Phần Tử Cùng Cấp

Hai phần tử HTML là cùng cấp (**sibling**) nếu chúng **đứng ngang hàng nhau** trong cùng một khối (`<div>`, `<section>`, `<body>`…).

a. Ví dụ minh họa phần tử cùng cấp vs KHÔNG cùng cấp:

Ví Dụ 1: Các Phần Tử Cùng Cấp (Sibling Elements)

```
<div class="parent">
    <h3>Header Title</h3>        <!-- A -->
    <p>Sample Text</p>        <!-- B -->
    <p>Sample Text</p>        <!-- C -->
</div>
```

[Ghi Chú] Giải Thích Ví Dụ 1 (Cùng Cấp)

Ở đây: `<h3>`, `<p>` (Đoạn 1), và `<p>` (Đoạn 2) đều nằm trong cùng 1 thẻ `<div class="parent">`.

→ **Vì vậy, chúng là phần tử cùng cấp (sibling elements).**

Ví Dụ 2: Các Phần Tử KHÔNG Cùng Cấp

```
<div class="parent">
    <h3>Header Title</h3>
    <div>
        <p>Sample Text</p>  <!-- Sample Text -->
    </div>
</div>
```

[Cảnh Báo] Giải Thích Ví Dụ 2 (KHÔNG Cùng Cấp)

Ở đây:

- `<h3>` là con của `div.parent`.
- `<p>` là con của `<div>` khác nằm bên trong `div.parent`.

→ **`<h3>` và `<p>` KHÔNG cùng cấp.**

b. Phân biệt rõ ràng: ”Phần tử con trực tiếp” và ”Phần tử cùng cấp”:

[Bảng] Bảng Phân Biệt Khái Niệm: Phần Tử Con Trực Tiếp vs Phần Tử Cùng Cấp

Khái niệm	Phần tử con trực tiếp (Direct Child)	Phần tử cùng cấp (Sibling Element)
Là gì?	Là phần tử nằm bên trong phần tử cha, ngay trực tiếp – không qua trung gian.	Là hai phần tử cùng nằm trong một phần tử cha – tức là anh em với nhau.

Ví Dụ Minh Họa Phân Biệt Cấu Trúc Cha - Con - Cùng Cấp

```
<div class="cha">
    <h3>Header Title</h3>        <!-- Sample Text -->
    <p>Sample Text</p>           <!-- Sample Text -->
    <div>
        <p>Sample Text</p>         <!-- Sample Text -->
    </div>
</div>
```

[Ghi Chú] Giải Thích Chi Tiết Cấu Trúc Trong HTML Trên

- `<h3>` và `<p>Đoạn 1` → **Cùng cấp**, vì cùng nằm trực tiếp trong `<div class="cha">`.
- `<p>Đoạn 1` là **con trực tiếp** của `div.cha`.
- `<p>Đoạn 2` **KHÔNG phải con trực tiếp** của `div.cha` mà là con của một thẻ `<div>` khác bên trong.
- `<p>Đoạn 2` **KHÔNG cùng cấp với <h3>**, vì nằm sâu hơn một cấp.

[Bảng] Tóm Lại So Sánh Đặc Điểm

Tiêu chí so sánh	Phần tử con trực tiếp	Phần tử cùng cấp	
Cùng nằm trong phần tử cha	Có	Có	
Phải là con của cha	Có	Có	
Phải là ngay bên trong	Có	**Không bắt buộc** (chỉ cần chung cha)	

[Mẹo] Mẹo Ghi Nhớ Cốt Lõi

- **Cùng cấp** = ”Ngang hàng” (Anh em ruột cùng cha).
- **Con trực tiếp** = ”Con ruột” của phần tử cha.

c. Chuyên đề thực nghiệm 1: Bộ chọn anh chị em tổng quát (`h3 ~ p`):

Đoạn Mã HTML Và CSS Thực Nghiệm Bộ Chọn h3 ~ p

```html
<div class='Parent'>
    <h3>lorem different</h3>
    <p>lorem is apple </p>       <!-- 1 -->
    <p>lorem is apple </p>       <!-- 2 -->
    <div class='child_1'>
        <p>lorem is apple </p>     <!-- 3 -->
        <p>lorem is apple </p>     <!-- 4 -->
    </div>
    <p>lorem is apple </p>       <!-- 5 -->
    <div class='child_2'>
        <p>lorem is apple </p>     <!-- 6 -->
        <p>lorem is apple </p>     <!-- 7 -->
    </div>
</div>
```

```css
h3 ~ p {
    color: red;
}
```

[Khái Niệm] Giải Thích Cơ Chế Hoạt Động Của Selector h3 ~ p

Selector `h3 ~ p` là **General Sibling Combinator**, nghĩa là tất cả các phần tử `<p>` **cùng cấp (siblings)** với `<h3>` và **đứng sau** `<h3>` trong cùng một phần tử cha sẽ được áp dụng style.

BROWSER PREVIEW — **Kết Quả Tô Màu Đỏ Cho Selector h3 ~ p**

.Parent

<h3> lorem different </h3>

** lorem is apple (Số 1)** ← Đỏ (Sibling trực tiếp sau h3)

** lorem is apple (Số 2)** ← Đỏ (Sibling trực tiếp sau h3)

.child_1

 lorem is apple (Số 3) ← Không đổi màu (nằm trong .child_1)

 lorem is apple (Số 4) ← Không đổi màu (nằm trong .child_1)

** lorem is apple (Số 5)** ← Đỏ (Sibling trực tiếp sau h3)

.child_2

 lorem is apple (Số 6) ← Không đổi màu (nằm trong .child_2)

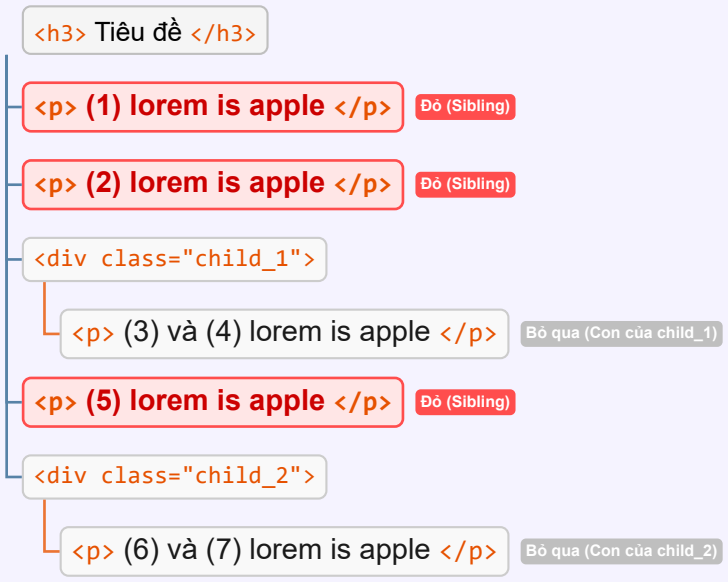
 lorem is apple (Số 7) ← Không đổi màu (nằm trong .child_2)

[Ghi Chú] Chi Tiết Phân Tích Kết Quả Áp Dụng

- **Áp dụng CSS color: red cho**: `<p>` số 1, `<p>` số 2, và `<p>` số 5. Vì cả 3 thẻ này là sibling trực tiếp của `<h3>`, nằm trong cùng thẻ cha `.Parent` và đứng sau `<h3>`.
- **KHÔNG áp dụng cho**:
  - `<p>` số 3 và số 4: nằm bên trong `.child_1`.
  - `<p>` số 6 và số 7: nằm bên trong `.child_2`.

  Vì chúng không phải sibling trực tiếp của `<h3>`, mà là con của các thẻ `<div>` khác.

[Khái Niệm] Sơ Đồ Phân Cấp Cây DOM Tổng Kết Cho h3 ~ p



[Cảnh Báo] Lưu Ý Nâng Cao & Hướng Xử Lý Trong Thực Tế

Trong CSS, với bộ chọn `h3 ~ p`, chỉ các phần tử `<p>` nào là **"cùng cấp"** (tức là sibling trực tiếp của `<h3>`) và nằm sau `<h3>` mới bị ảnh hưởng. Còn những phần tử `<p>` không cùng cấp (ví dụ như nằm trong `div.child_1`, `div.child_2`) thì **KHÔNG** bị ảnh hưởng bởi selector này.

Minh Họa Nhanh Khác Biệt Cấp Độ

```
<div class="Parent">
    <h3>Header Title</h3>          <!-- Sample Text -->
    <p>Sample Text</p>        <!-- Sample Text -->
    <div>
        <p>Sample Text</p>  <!-- Sample Text -->
    </div>
</div>
```

Giải thích: Cùng cấp nghĩa là 2 phần tử có chung cha trực tiếp. Nếu phần tử đó nằm lồng bên trong một thẻ khác → không còn cùng cấp.

Giải pháp: Nếu bạn muốn tất cả các `<p>` nằm sau `<h3>` đều bị ảnh hưởng, bất kể cấp lồng nào, bạn không thể làm điều đó chỉ bằng CSS hiện tại (vì CSS không có bộ chọn "mọi phần tử nằm sau ở mọi cấp"). Bạn cần dùng JavaScript để quét DOM hoặc thay đổi cấu trúc HTML / thêm class phù hợp.

d. Chuyên đề thực nghiệm 2: Bộ chọn con trực tiếp (`.Parent > p` và `div > p`):

Ví Dụ 1: Sử Dụng Selector .Parent > p

```
.Parent > p {
    color: red;
}
```

[Khái Niệm] Ý Nghĩa Selector .Parent > p

Selector `.Parent > p` chọn tất cả các phần tử `<p>` là **con trực tiếp (direct children)** của phần tử có class `.Parent`.

[Bảng] Phân Tích Từng Thẻ Trong HTML Khi Áp Dụng .Parent > p

Thẻ HTML	Kết quả	Giải thích
`<h3>lorem different</h3>`	Không tác động	Không phải thẻ `<p>`, bỏ qua.
`<p>lorem is apple </p> (Số 1)`	**Tô đỏ**	Con trực tiếp của `.Parent`.
`<p>lorem is apple </p> (Số 2)`	**Tô đỏ**	Con trực tiếp của `.Parent`.
`Thẻ <p> số 3 & 4 trong .child_1`	Bỏ qua	Con của `div.child_1`, KHÔNG phải con trực tiếp của `.Parent`.
`<p>lorem is apple </p> (Số 5)`	**Tô đỏ**	Con trực tiếp của `.Parent`.
`Thẻ <p> số 6 & 7 trong .child_2`	Bỏ qua	Con của `div.child_2`, KHÔNG phải con trực tiếp của `.Parent`.

BROWSER PREVIEW **Kết Luận Cho Ví Dụ 1 (.Parent > p): 3 Thẻ p Bị Tô Đỏ**

.Parent

** lorem is apple (Số 1 sau <h3>)** → Tô đỏ

** lorem is apple (Số 2 trước div.child_1)** → Tô đỏ

.child_1

 lorem is apple (Số 3 & 4) → Không bị ảnh hưởng

** lorem is apple (Số 5 sau div.child_1)** → Tô đỏ

.child_2

 lorem is apple (Số 6 & 7) → Không bị ảnh hưởng

Ví Dụ 2: Sử Dụng Selector div > p

```
1 div > p {
2     color: red;
3 }
```

[Khái Niệm] Ý Nghĩa Selector div > p

Selector `div > p` sẽ chọn tất cả các thẻ `<p>` là **con trực tiếp** của **bất kỳ thẻ `<div>` nào**.

[Ghi Chú] Phân Tích Tác Động Của div > p Lên Mã HTML

- Thẻ `<p>` 1, 2, 5 → là con trực tiếp của `div.Parent` → **Tô đỏ**.
- Thẻ `<p>` 3, 4 → là con trực tiếp của `div.child_1` → **Tô đỏ**.
- Thẻ `<p>` 6, 7 → là con trực tiếp của `div.child_2` → **Tô đỏ**.
- Thẻ `<h3>` → Không phải thẻ `<p>`, bỏ qua.

Kết luận: **TẤT CẢ 7 THẺ ĐỀU BỊ TÔ ĐỎ!**

[Cảnh Báo] Cảnh Báo Về Bộ Chọn Quá Tổng Quát

Selector càng tổng quát (`div > p`) thì càng dễ ”vô tình” áp dụng lên nhiều phần tử hơn mong muốn. Nếu bạn chỉ muốn định dạng các thẻ `<p>` trực thuộc `.Parent` mà không làm ảnh hưởng tới `.child_1` hay `.child_2`, bắt buộc phải dùng bộ chọn cụ thể hơn: `.Parent > p`.

e. Chuyên đề thực nghiệm 3: Bộ chọn hậu duệ (`.Parent p`):

Đoạn Mã Selector Hậu Duệ .Parent p

```
.Parent p {
    color: red;
}
```

[Khái Niệm] Ý Nghĩa Selector .Parent p

Selector `.Parent p` chọn **tất cả các thẻ `<p>` nằm bên trong (bất kỳ cấp lồng nào)** của phần tử có class `.Parent`.

Lưu ý: Không cần phải là con trực tiếp – chỉ cần nằm lồng trong `.Parent` là được!

BROWSER PREVIEW **Kết Quả Cho .Parent p: TẤT CẢ 7 Thẻ p Đều Tô Đỏ**

TẤT CẢ 7 THẺ <p> ĐỀU BỊ TÔ ĐỎ vì chúng đều nằm bên trong phần tử `.Parent`, dù là con trực tiếp (P1, P2, P5) hay con gián tiếp/cháu (P3, P4, P6, P7).

f. Bảng tổng hợp so sánh sự khác biệt giữa các Selector:

[Bảng] Bảng So Sánh Tổng Hợp Sự Khác Biệt Giữa Các Structural Selectors

Selector	Mô tả phạm vi chọn	Số thẻ <p> bị ảnh hưởng
`.Parent p`	Tất cả `<p>` nằm bên trong `.Parent` (mọi cấp lồng)	**7 thẻ (Tất cả)**
`.Parent > p`	Chỉ các `<p>` là **con trực tiếp** của `.Parent`	**3 thẻ (P1, P2, P5)**
`div > p`	Bất kỳ `<p>` nào là **con trực tiếp** của bất kỳ `<div>` nào	**7 thẻ (Tất cả)**
`h3 ~ p`	Các `<p>` **cùng cấp (siblings)**, nằm phía sau `<h3>`	**3 thẻ (P1, P2, P5)**

[Mẹo] Kết Luận Lựa Chọn Selector Phù Hợp

Việc lựa chọn bộ chọn chặt chẽ (`.Parent > p`, `h3 ~ p`) hay tổng quát (`.Parent p`, `div > p`) tùy thuộc hoàn toàn vào mục tiêu thiết kế giao diện của bạn. Hãy ưu tiên các bộ chọn cụ thể và có phạm vi kiểm soát tốt để tránh lỗi giao diện phát sinh ngoài ý muốn!

9.5.6 5. So Sánh Tổng Hợp Chi Tiết Và Dễ Hiểu Nhất Về Tất Cả Các Bộ Chọn Kết Hợp Trong CSS

[Khái Niệm] Tổng Quan: 4 Loại Bộ Chọn Kết Hợp (Combinator Selectors)

CSS cung cấp 4 loại bộ chọn kết hợp (Combinator Selectors) giúp bạn chọn phần tử dựa trên **mối quan hệ phân cấp** và **vị trí tương đối** trong cây DOM. Hiểu rõ từng loại, cách hoạt động và ứng dụng thực tế sẽ giúp bạn viết CSS tinh gọn, dễ bảo trì và hiệu suất cao.

5.1. Bảng So Sánh Chi Tiết 4 Bộ Chọn Kết Hợp

[Bảng] Bảng So Sánh Tổng Hợp 4 Bộ Chọn Kết Hợp CSS Combinator

Bộ chọn	Cú pháp	Ý nghĩa	Ví dụ
Descendant Selector	`cha con`	Chọn **tất cả phần tử con cháu** bên trong phần tử cha, không phân biệt cấp độ lồng nhau.	`.container p { color: blue; }`
Child Selector	`cha > con`	Chọn **chỉ con trực tiếp** của phần tử cha (cháu, chắt sẽ bị bỏ qua).	`.container > p { color: red; }`
Adjacent Sibling	`e1 + e2`	Chọn phần tử **anh chị em liền kề ngay sau** phần tử đầu tiên, cùng cấp trong DOM.	`h1 + p { color: green; }`
General Sibling	`e1 ~ e2`	Chọn **tất cả phần tử anh chị em cùng cấp** nằm sau phần tử đầu tiên.	`h1 ~ p { color: purple; }`

5.2. Ví Dụ Minh Họa Trực Quan – Áp Dụng Đồng Thời Cả 4 Bộ Chọn

HTML Mẫu: Cấu Trúc DOM Dùng Chung Cho Cả 4 Bộ Chọn Kết Hợp

```html
<div class="container">
    <h1>Header Title</h1>
    <p>Sample Text</p>
    <p>Sample Text</p>
    <div>
        <p>Sample Text</p>
    </div>
    <p>Sample Text</p>
</div>
```

CSS: Áp Dụng Đồng Thời Tất Cả 4 Bộ Chọn Kết Hợp

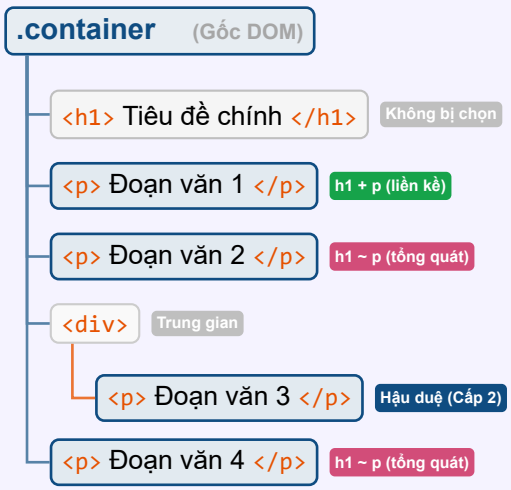
```css
/* Select element */
.container p {
    color: blue;
}

/* Sample Text */
.container > p {
    color: red;
}

/* Select element */
h1 + p {
    color: green;
}

/* Select element */
h1 ~ p {
    color: purple;
}
```

[Khái Niệm] Sơ Đồ Phân Cấp Cây DOM Cho Ví Dụ Tổng Hợp



5.3. Bảng Kết Quả Trực Quan – Phân Tích Từng Phần Tử

[Bảng] Kết Quả Trực Quan: Màu Sắc Của Từng Phần Tử Khi Áp Dụng Cả 4 Selector

Nội dung	Màu sắc	Giải thích
`Tiêu đề chính`	Mặc định (đen)	Không bị ảnh hưởng vì không có bộ chọn nào tác động đến `<h1>`.
`Đoạn văn 1`	**Xanh lá (green)**	Bộ chọn `h1 + p` (Adjacent Sibling) tác động, vì nó là thẻ `<p>` nằm **ngay liền sau** `<h1>`.
`Đoạn văn 2`	**Tím (purple)**	Bộ chọn `h1 ~ p` (General Sibling) tác động, vì nó nằm **sau** `<h1>` và **cùng cấp**.
`Đoạn văn 3 (trong <div>)`	**Xanh dương (blue)**	Bộ chọn `.container p` (Descendant) tác động, vì nó là **hậu duệ** của `.container` (dù lồng cấp 2 trong `<div>`).
`Đoạn văn 4`	**Tím (purple)**	Bộ chọn `h1 ~ p` (General Sibling) tác động, vì nó nằm **sau** `<h1>` và **cùng cấp**.

BROWSER PREVIEW — Kết Quả Hiển Thị Trực Quan Trên Trình Duyệt

Tiêu đề chính

Đoạn văn 1 ← h1 + p (green)

Đoạn văn 2 ← h1 ~ p (purple)

Đoạn văn 3 ← .container p (blue) – lồng trong

Đoạn văn 4 ← h1 ~ p (purple)

[Ghi Chú] Tại Sao Đoạn Văn 1 Là Xanh Lá Chứ Không Phải Tím Hoặc Đỏ?

Đoạn văn 1 thỏa mãn **cả 4 bộ chọn** cùng lúc:

1. `.container p` (Descendant) – vì P1 là hậu duệ của `.container`.
2. `.container > p` (Child) – vì P1 là con trực tiếp.
3. `h1 + p` (Adjacent Sibling) – vì P1 liền kề ngay sau `<h1>`.
4. `h1 ~ p` (General Sibling) – vì P1 nằm sau `<h1>` cùng cấp.

Khi nhiều bộ chọn cùng tác động lên 1 phần tử, CSS sẽ áp dụng theo quy tắc **Cascade (thứ tự xuất hiện)**: bộ chọn được viết **sau cùng** trong file CSS sẽ thắng. Vì `h1 + p` nằm sau `.container > p`, nên màu xanh lá (green) được hiển thị. Nếu `h1 ~ p` (purple) được viết cuối cùng, nó sẽ đè lên. Nhưng vì `h1 + p` cụ thể hơn `h1 ~ p` trong trường hợp liền kề, P1 vẫn nhận màu green!

5.4. Ứng Dụng Thực Tế Của Từng Bộ Chọn

[Bảng] Bảng Ứng Dụng Thực Tế Của 4 Bộ Chọn Kết Hợp

Bộ chọn	Ứng dụng thực tế
`Descendant Selector` `cha con`	Styling toàn bộ nội dung trong section: Định dạng tất cả tiêu đề, đoạn văn, nút… bên trong một khu vực cụ thể.
`Child Selector` `cha > con`	Menu đa cấp: Chọn chỉ các mục cấp 1 của menu để định dạng đặc biệt, bỏ qua các mục lồng cấp 2, 3…
`Adjacent Sibling` `e1 + e2`	Tạo khoảng cách sau tiêu đề: Định dạng đoạn văn ngay sau tiêu đề để cách dòng, tạo khoảng thở cho bố cục.
`General Sibling` `e1 ~ e2`	Styling danh sách thông báo: Tô màu hoặc highlight tất cả các thông báo sau tiêu đề “Thông báo mới”.

5.5. Khi Nào Nên Dùng Bộ Chọn Nào?

[Mẹo] Hướng Dẫn Chọn Đúng Bộ Chọn Kết Hợp Cho Từng Tình Huống

- Cần định dạng **toàn bộ phần tử con** (bất kể cấp độ) → **Descendant Selector** (`cha con`).
- Chỉ muốn tác động đến **con trực tiếp**, tránh ảnh hưởng sâu → **Child Selector** (`cha > con`).
- Muốn định dạng phần tử **liền kề** để tạo bố cục đẹp → **Adjacent Sibling Selector** (`e1 + e2`).
- Muốn tác động đến **nhiều phần tử cùng cấp** mà không cần liền kề → **General Sibling Selector** (`e1 ~ e2`).

5.6. Mẹo Tối Ưu CSS Với Bộ Chọn Kết Hợp

[Cảnh Báo] 3 Mẹo Tối Ưu Hiệu Suất CSS Khi Dùng Bộ Chọn Kết Hợp

1. **Tránh lạm dụng bộ chọn quá sâu**: `div ul li a span {}` có thể làm trình duyệt chậm vì phải tìm kiếm qua nhiều cấp. Hãy tối giản bộ chọn xuống tối đa 3–4 cấp.
2. **Ưu tiên bộ chọn cụ thể hơn**: Child Selector (`>`) nhanh hơn Descendant Selector vì chỉ tìm con trực tiếp, không phải quét toàn bộ cây DOM con.
3. **Kết hợp thông minh**: Dùng kết hợp Sibling Selector với Pseudo-class (ví dụ: `:hover`, `:nth-child`) để tạo hiệu ứng đẹp mắt mà không cần JavaScript!

5.7. Kết Luận Tổng Quan

[Mẹo] Kết Luận Tổng Hợp Về Bộ Chọn Kết Hợp (Combinator Selectors)

Bộ chọn kết hợp là công cụ mạnh mẽ giúp CSS linh hoạt và chính xác hơn. Hiểu rõ từng loại, cách hoạt động và ứng dụng thực tế sẽ giúp bạn:

- **Viết CSS tinh gọn**: Chọn đúng phần tử mà không cần thêm class hoặc ID thừa vào HTML.
- **Tối ưu hiệu suất**: Hiểu sâu về các selector này giúp xây dựng giao diện gọn gàng, dễ bảo trì và tối ưu tốc độ render trang!
- **Tạo hiệu ứng không cần JavaScript**: Kết hợp Sibling Selectors với Pseudo-class để tạo menu dropdown, toggle panel, highlight nhóm phần tử… hoàn toàn bằng CSS thuần!

9.5.7 6. Chuyên Đề Phân Tích Chi Tiết & Thực Nghiệm Về Bộ Chọn Thuộc Tính (Attribute Selectors Masterclass)

[Khái Niệm] Khái Niệm Về Bộ Chọn Thuộc Tính (Attribute Selectors)

Bộ chọn thuộc tính (Attribute Selector) trong CSS cho phép bạn chọn và áp dụng định kiểu cho các phần tử HTML dựa trên sự **tồn tại của một thuộc tính** hoặc **giá trị cụ thể của thuộc tính đó** (ví dụ: `href`, `type`, `src`, `title`, `target`, `disabled`, `required`, các thuộc tính tùy chỉnh `data-*`, hay thuộc tính trợ năng `aria-*`).

Lợi ích cốt lõi: Giúp bạn định dạng giao diện linh hoạt trực tiếp từ ngữ nghĩa HTML mà không cần phải gán thêm thẻ `class` hay `id` dư thừa vào mã nguồn!

a. Bảng tổng quan 7 loại bộ chọn thuộc tính & Cờ phân biệt hoa/thường:

[Bảng] Bảng Tổng Quan Chi Tiết 7 Loại Bộ Chọn Thuộc Tính Trong CSS

Tên Bộ Chọn	Cú Pháp	Mô Tả Quy Tắc So Khớp
Tồn tại thuộc tính	`[attr]`	Chọn phần tử **có chứa thuộc tính** `attr`, bất kể giá trị là gì.
Giá trị chính xác	`[attr="val"]`	Chọn phần tử có giá trị thuộc tính **bằng đúng tuyệt đối** `val`.
Chứa từ độc lập	`[attr~="val"]`	Giá trị thuộc tính chứa `val` như **một từ độc lập** (phân cách bởi khoảng trắng).
Bắt đầu bằng tiền tố	`[attr|="val"]`	Giá trị bằng đúng `val` hoặc **bắt đầu bằng** `val-` (thường dùng cho mã ngôn ngữ).
Bắt đầu bằng chuỗi	`[attr^="val"]`	Giá trị thuộc tính **bắt đầu bằng** chuỗi `val`.
Kết thúc bằng chuỗi	`[attr$="val"]`	Giá trị thuộc tính **kết thúc bằng** chuỗi `val`.
Chứa chuỗi con	`[attr*="val"]`	Giá trị thuộc tính **chứa chuỗi con** `val` ở bất kỳ vị trí nào.
Không phân biệt hoa/thường	`[attr="val" i]`	Cờ `i` giúp quy tắc so khớp **không phân biệt chữ hoa / chữ thường**.

b. Phân tích chi tiết 7 loại bộ chọn thuộc tính kèm ví dụ thực tế:

6.1. Bộ Chọn Sự Tồn Tại Của Thuộc Tính ([attr])

[Khái Niệm] Ý Nghĩa Selector [attr]

Bộ chọn `[attr]` sẽ khớp với tất cả các phần tử có chứa thuộc tính được chỉ định, không quan tâm giá trị của thuộc tính đó là gì hay có rỗng hay không.

Ví Dụ 1: Định Dạng Thẻ Input Có Thuộc Tính required

```css
/* Sample Text */
input[required] {
    border: 2px solid red;
    background-color: #fff0f0;
}
```

```html
<form>
    <input type="text" placeholder="Sample Text">
    <input type="email" required placeholder="Sample Text"> <!-- Sample Text -->
</form>
```

BROWSER PREVIEW — Kết Quả Hiển Thị: Ô Email Bắt Buộc Được Viền Đỏ



6.2. Bộ Chọn Giá Trị Chính Xác ([attr="val"])

[Khái Niệm] Ý Nghĩa Selector [attr="val"]

So khớp chính xác từng ký tự của giá trị thuộc tính. Chỉ khi giá trị khớp 100% thì CSS mới được áp dụng.

Ví Dụ 2: Định Kiểu Nút Bấm Submit Trong Form

```css
/* Submit */
input[type="submit"] {
    background-color: #28a745;
    color: white;
    font-weight: bold;
    border-radius: 4pt;
    padding: 6px 12px;
}
```

```html
<input type="text" placeholder="Sample Text">
<input type="submit" value="Submit"> <!-- Sample Text -->
```

BROWSER PREVIEW — Kết Quả Hiển Thị: Nút Submit Được Tô Màu Xanh Lá



6.3. Bộ Chọn Chứa Từ Độc Lập ([attr~=”val”])

[Khái Niệm] Ý Nghĩa Selector [attr~=”val”]

Bộ chọn này tìm giá trị thuộc tính chứa từ `val` như **một từ riêng biệt hoàn chỉnh** (được phân cách bởi khoảng trắng xung quanh). Thường dùng khi một thuộc tính chứa danh sách nhiều từ (như `class="btn primary active"`).

Ví Dụ 3: Tìm Thẻ Có Class Chứa Từ ”active”

```css
/* Select element */
[class~="active"] {
    color: #007bff;
    font-weight: bold;
}
```

```html
<div class="card active main">Sample Text</div>
<div class="card interactive">Sample Text</div>
```

BROWSER PREVIEW Kết Quả Hiển Thị: Thẻ Chứa Từ Độc Lập ”active” Được Tô Màu

Khớp - Được chọn (Class chứa từ ”active”)

Không khớp (Class interactive)

6.4. Bộ Chọn Bắt Đầu Tiền Tố / Mã Ngôn Ngữ ([attr|=”val”])

[Khái Niệm] Ý Nghĩa Selector [attr|=”val”]

Khớp với các phần tử có giá trị thuộc tính **chính xác là** `val` hoặc **bắt đầu bằng** `val-` (theo sau là dấu gạch nối). Bộ chọn này sinh ra đặc biệt để chọn mã ngôn ngữ chuẩn ISO (như `lang="en"`, `lang="en-US"`, `lang="en-GB"`).

Ví Dụ 4: Định Dạng Thẻ Có Thuộc Tính Ngôn Ngữ Tiếng Anh

```css
/* Sample Text */
p[lang|="en"] {
    font-style: italic;
    color: #4a5568;
}
```

```html
<p lang="en">Hello World</p>       <!-- Sample Text -->
<p lang="en-US">Hello America</p> <!-- Sample Text -->
<p lang="fr">Bonjour</p>          <!-- Sample Text -->
```

BROWSER PREVIEW Kết Quả Hiển Thị: Định Dạng Nghiêng Cho Thẻ Thuộc Nhóm Tiếng Anh

Hello World ← lang="en" (In nghiêng)
Hello America ← lang="en-US" (In nghiêng)
Bonjour ← lang="fr" (Mặc định)

6.5. Bộ Chọn Bắt Đầu Chuỗi ([attr^="val"])

[Khái Niệm] Ý Nghĩa Selector [attr^="val"]

Khớp với tất cả các phần tử có giá trị thuộc tính **bắt đầu bằng chuỗi** `val` (Prefix Match).

Ví Dụ 5: Tự Động Thêm Khung Cho Đường Dẫn Bảo Mật HTTPS

```css
/* Sample Text */
a[href^="https://"] {
    color: #2b6cb0;
    font-weight: bold;
}
```

```html
<a href="https://google.com">Sample Text</a> <!-- Sample Text -->
<a href="http://unsafe.com">Sample Text</a> <!-- Sample Text -->
```

BROWSER PREVIEW Kết Quả Hiển Thị: Link HTTPS Tự Động Được Nổi Bật

Trang an toàn HTTPS ← a[href^="https://"] | Trang không an toàn HTTP

6.6. Bộ Chọn Kết Thúc Chuỗi ([attr$="val"])

[Khái Niệm] Ý Nghĩa Selector [attr$="val"]

Khớp với tất cả các phần tử có giá trị thuộc tính **kết thúc bằng chuỗi** `val` (Suffix Match). Ứng dụng phổ biến nhất là nhận diện đuôi mở rộng của tệp tin tải về (`.pdf`, `.docx`, `.zip`).

Ví Dụ 6: Tự Động Nhận Diện Tệp Tải Về PDF & Thêm Icon

```css
/* Sample Text */
a[href$=".pdf"]::after {
    content: " [PDF]";
    color: red;
    font-weight: bold;
    font-size: 0.8em;
}
```

```html
<a href="tailieu.pdf">Sample Text</a> <!-- Sample Text -->
<a href="trangchu.html">Sample Text</a>          <!-- Sample Text -->
```

BROWSER PREVIEW — **Kết Quả Hiển Thị: Tự Động Chèn Nhãn [PDF**

Đỏ Mắt Nhìn] Tải Báo Cáo Tài Chính **[PDF]** ← a[href$=".pdf"]::after

6.7. Bộ Chọn Chứa Chuỗi Con ([attr*="val"])

[Khái Niệm] Ý Nghĩa Selector [attr*="val"]

Khớp với các phần tử có giá trị thuộc tính **chứa chuỗi** `val` **ở bất kỳ vị trí nào** (Substring Match).

Ví Dụ 7: Định Kiểu Cho Tất Cả Đường Dẫn Đến Mạng Xã Hội Facebook

```css
/* Select element */
a[href*="facebook"] {
    color: #1877f2;
    font-weight: bold;
}
```

```html
<a href="https://www.facebook.com/profile">Sample Text</a> <!-- Sample Text -->
```

BROWSER PREVIEW **Kết Quả Hiển Thị: Link Facebook Tự Động Được Tô Màu Xanh Nhận Diện**

Trang Fanpage ← a[href*="facebook"] (Tô màu xanh Facebook)

6.8. Cờ Phân Biệt Chữ Hoa / Chữ Thường ([attr="val" i])

[Khái Niệm] Ý Nghĩa Cờ Case-Insensitive Flag "i"

Mặc định trong HTML5, một số thuộc tính phân biệt hoa thường. Thêm cờ `i` trước dấu đóng ngoặc `]` giúp bộ chọn **bỏ qua phân biệt hoa/thường**, giúp khớp cả `.pdf`, `.PDF`, `.Pdf`.

Ví Dụ 8: Bắt File PDF Không Phân Biệt Chữ Hoa/Thường

```
/* Sample Text */
a[href$=".pdf" i] {
    color: red;
}
```

BROWSER PREVIEW **Kết Quả Hiển Thị: Bắt Cả .PDF và .Pdf Nhờ Cờ "i"**

tailieu_baocao.PDF ← Khớp nhờ cờ "i"
huongdan_sudung.Pdf ← Khớp nhờ cờ "i"

c. Bảng tra cứu tóm tắt các ký hiệu so khớp Attribute Selectors:

[Bảng] Bảng Tra Cứu So Sánh Các Ký Hiệu Attribute Selectors

Ký Hiệu	Loại So Khớp	Trường Hợp KHỚP (Match)	Trường Hợp BỎ QUA (No match)
`[attr]`	Tồn tại	`<div title="hello">`	`<div>` (không có title)
`=`	Bằng đúng tuyệt đối	`<a target="_blank">`	`<a target="_self">`
`~=`	Từ độc lập	`<div class="btn active">`	`<div class="btn-active">`
`|=`	Tiền tố ngôn ngữ	`<p lang="en-US">`	`<p lang="us-en">`
`^=`	Bắt đầu chuỗi	`<a href="https://a.com">`	`<a href="http://a.com">`
`$=`	Kết thúc chuỗi	`<a href="file.pdf">`	`<a href="file.pdf.png">`
`*=`	Chứa chuỗi con	`<img src="user_avatar.jpg">`	`<img src="user_photo.jpg">`

d. 4 Ứng dụng thực tế đỉnh cao của Attribute Selectors trong Frontend Development:

[Bảng] 4 Pattern Thực Tế Sử Dụng Attribute Selectors Vô Cùng Hiệu Quả

Mẫu Thiết Kế (UI Pattern)	Mô Tả & Cách Triển Khai Thực Tế
1. Form Validation & UI States	Định kiểu trạng thái nhập liệu trực tiếp từ HTML5 attributes: `input[disabled]` → Nền xám nhạt, cấm trỏ chuột (`cursor: not-allowed`). `input[readonly]` → Không cho chỉnh sửa nhưng vẫn chọn được text. `input:invalid` → Hiển thị khung cảnh báo viền đỏ khi nhập sai format.
2. Tự Động Nhận Diện Link Ngoại Mạng & File Tải Về	Tăng trải nghiệm người dùng (UX) tự động: `a[target="_blank"]::after` → Tự chèn icon mũi tên chỉ ra ngoài. `a[href$=".zip"]` → Tự thêm icon tập tin nén ZIP.
3. Custom Data Attributes (data-*)	Quản lý State giao diện trong React / Vue / Vanilla JS: `div[data-status="success"]` → Tô màu xanh thông báo thành công. `body[data-theme="dark"]` → Tự động chuyển đổi toàn bộ màu giao diện Dark Mode!
4. Web Accessibility ARIA Attributes (aria-*)	Đảm bảo trợ năng chuẩn WCAG cho người khuyết tật: `button[aria-disabled="true"]` → Giảm độ đục opacity, vô hiệu hóa nút bấm chuẩn a11y. `div[aria-expanded="true"]` → Hiển thị menu accordion đang mở.

[Mẹo] Best Practices Checklist Cho Attribute Selectors

- **Khi nào nên dùng Attribute Selector?** Dùng khi thuộc tính đó mang ý nghĩa HTML gốc (như `type`, `href`, `disabled`, `data-*`, `aria-*`) để tránh phải viết thêm class thừa.
- **Khi nào nên dùng Class Selector?** Dùng Class cho các thành phần giao diện tái sử dụng linh hoạt nhiều nơi (như `.card`, `.btn-primary`).
- **Lưu ý hiệu năng**: Attribute selector rất nhanh trên các trình duyệt hiện đại, nhưng hãy hạn chế dùng selector chứa chuỗi quá phức tạp `[attr*="..."]` trên cây DOM quá lớn hàng nghìn node.

9.5.8 7. Chuyên Đề Phân Tích Chi Tiết & Thực Nghiệm Về Bộ Chọn Lớp Giả (Pseudo-class Selectors Masterclass)

Pseudo-class selectors (Bộ chọn lớp giả) là một phần cực kỳ mạnh mẽ trong CSS3 giúp bạn chọn và định kiểu các phần tử HTML dựa trên **trạng thái tương tác đặc biệt** hoặc **vị trí vị thế cấu trúc** của chúng trong cây DOM – mà **không cần phải thêm bất kỳ class hay ID thủ công nào vào file HTML**!

[Khái Niệm] Bản Chất & Cú Pháp Tổng Quát Về Pseudo-Class

- **Bản chất**: Chọn và định kiểu phần tử dựa trên **trạng thái tương tác đặc biệt** (như di chuột, click bấm) hoặc **vị trí vị thế cấu trúc** trong cây DOM mà **không cần thêm class/ID vào HTML**.
- **Cú pháp chuẩn**: Bắt đầu bằng **một dấu hai chấm** (`:`).
- **Công thức tổng quát**:

```
selector:pseudo-class { property: value; }
```

- **Ví dụ minh họa**: `button:hover { color: red; }` (Đổi chữ sang màu đỏ khi rê chuột vào nút bấm).
- **Ứng dụng thực tế**: Phản hồi hành vi rê chuột, con trỏ focus nhập liệu, trạng thái check-box/radio được chọn, hoặc tự động định kiểu dòng chẵn/lẻ cho bảng dữ liệu.

7.1. Bảng Tra Cứu Tổng Hợp Toàn Bộ Pseudo-Classes

Dưới đây là phân tích chi tiết toàn bộ các bộ chọn lớp giả (Pseudo-classes) được nhóm theo 4 phân loại chức năng thực tế trong thiết kế giao diện Web:

7.1. Nhóm Trạng Thái Tương Tác & Điều Hướng (User Action & Links)

Nhóm này kiểm soát các hành vi động của người dùng khi di chuột, click bấm, focus ô nhập liệu hoặc truy cập đường dẫn:

[Bảng] Bảng Tra Cứu Pseudo-Classes Trạng Thái Tương Tác & Liên Kết

Selector	Phạm vi áp dụng	Ví dụ cú pháp	Chức năng & Ý nghĩa
`:hover`	Tất cả phần tử	`button:hover { color: red; }`	Kích hoạt đổi màu đỏ khi trỏ chuột vào phần tử.
`:active`	Tất cả phần tử	`a:active { color: green; }`	Kích hoạt chuyển xanh khi đang giữ chuột bấm xuống.
`:focus`	Input, Button, Link	`input:focus { border: 2px solid blue; }`	Kích hoạt viền xanh khi nhận tiêu điểm nhập liệu.
`:link`	Thẻ `<a href="...">`	`a:link { color: blue; }`	Áp dụng cho liên kết chưa từng được truy cập.
`:visited`	Thẻ `<a href="...">`	`a:visited { color: purple; }`	Áp dụng cho liên kết đã từng lưu trong lịch sử web.
`:target`	Phần tử có `id`	`#section:target { background: yellow; }`	Khớp với id trên URL anchor (`#section`) đổi nền vàng.

a. Bộ Chọn :link (Liên Kết Chưa Truy Cập)

Bộ chọn `:link` trong CSS được sử dụng để chọn các liên kết (`<a>`) chưa được truy cập. Điều này có nghĩa là liên kết đó chưa được người dùng click vào hoặc trình duyệt chưa ghi nhận nó là đã truy cập.

[Khái Niệm] Đặc Điểm & Cú Pháp Của :link

- **Cú pháp**:

Mã CSS Định Kiểu Liên Kết Chưa Truy Cập

```css
a:link {
    color: blue;
    text-decoration: none;
}
```

- **Ý nghĩa**:
  - `a:link` chỉ áp dụng cho các thẻ `<a>` có thuộc tính `href`.
  - Liên kết chưa truy cập sẽ được áp dụng style này.

Ví Dụ Minh Họa Đầy Đủ HTML & CSS Cho :link Và :visited

```html
<!DOCTYPE html>
<html lang="vi">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Sample Text</title>
    <style>
        a:link {
            color: blue;
            font-weight: bold;
            text-decoration: none;
        }
        a:visited {
            color: purple;
        }
    </style>
</head>
<body>
    <h2>Sample Text</h2>
    <a href="https://www.example.com">Sample Text</a>
    <br>
    <a href="https://www.google.com">Sample Text</a>
</body>
</html>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trực Quan Trạng Thái :link Và :visited**

- **Liên kết chưa truy cập (`a:link`)**:

  Link chưa truy cập (Hiển thị chữ in đậm, màu xanh dương rực rỡ).

- **Liên kết đã từng nhấp truy cập (`a:visited`)**:

  Link đã truy cập (Trình duyệt tự động đổi sang màu tím dịu mắt dựa trên lịch sử duyệt web).

[Mẹo] Ứng Dụng Thực Tế Của :link

- Tạo giao diện rõ ràng cho người dùng biết link nào đã hoặc chưa click.
- Giúp người dùng dễ dàng điều hướng và tránh nhầm lẫn khi duyệt trang web.

b. Bộ Chọn :visited (Liên Kết Đã Truy Cập)

Trong CSS, `a:visited` là một pseudo-class được sử dụng để chọn các liên kết (thẻ `<a>`) đã được người dùng truy cập. Điều này giúp bạn tùy chỉnh giao diện của liên kết sau khi người dùng đã nhấp vào và truy cập trang đích.

[Khái Niệm] Cú Pháp & Giải Thích :visited

Cú Pháp Định Kiểu :visited

```
a:visited {
    color: purple; /* Sample Text */
    text-decoration: underline; /* Sample Text */
}
```

Giải thích:

- `a`: Chọn thẻ liên kết (anchor).
- `:visited`: Chọn các liên kết đã được nhấp vào và truy cập trước đó (dựa trên lịch sử trình duyệt).

[Cảnh Báo] Lưu Ý Bảo Mật Cực Kỳ Quan Trọng Cho a:visited

Vì lý do bảo mật, trình duyệt chỉ cho phép bạn thay đổi các thuộc tính không ảnh hưởng đến bố cục khi sử dụng `:visited` để tránh cuộc tấn công lộ lịch sử duyệt web (**History Sniffing Attack**):

- **Các thuộc tính ĐƯỢC CHO PHÉP**: `color`, `background-color`, `border-color`, `outline-color`, `text-decoration`, `cursor`, `fill`, `stroke`.
- **Các thuộc tính BỊ CẤM HOÀN TOÀN**: Các thuộc tính làm thay đổi kích thước hoặc bố cục như `display`, `visibility`, `content`, `font-size`, `padding`, `margin` sẽ không có tác dụng với `:visited`.

Ví Dụ Hoàn Chỉnh Kết Hợp a, a:hover Và a:visited

```
a {
    color: blue; /* Sample Text */
    text-decoration: none;
}
a:hover {
    color: red; /* Sample Text */
    text-decoration: underline;
}
a:visited {
    color: purple; /* Sample Text */
}
```

BROWSER PREVIEW Kết Quả Hiển Thị Trực Quan Chu Trình Chuyển Đổi Trạng Thái Link

- **Trạng thái 1 – Ban đầu (Chưa click)**:
  Trang chủ Website (Chữ màu xanh dương mặc định).
- **Trạng thái 2 – Di chuột vào (`a:hover`)**:
  Trang chủ Website (Lập tức đổi sang màu đỏ rực kèm đường gạch chân).
- **Trạng thái 3 – Sau khi đã nhấp vào xem (`a:visited`)**:
  Trang chủ Website (Đổi sang màu tím đại diện cho liên kết trong quá khứ).

c. Bộ Chọn :hover (Trạng Thái Di Chuột Vào Phần Tử)

Pseudo-class `:hover` là một công cụ cực kỳ phổ biến và hữu ích trong CSS! Nó giúp bạn thay đổi giao diện của phần tử khi người dùng di chuột vào, tạo ra trải nghiệm tương tác trực quan và sinh động

hơn.

[Khái Niệm] 1. :hover Là Gì & Cú Pháp Tổng Quát

:hover là gì? `:hover` là một pseudo-class trong CSS, áp dụng cho một phần tử khi người dùng di chuột qua nó (không cần nhấp chuột, chỉ cần đưa chuột vào).

Cú pháp:

Cú Pháp Khai Báo :hover

```
/* selector:hover { Sample Text: Sample Text; } */
selector:hover {
    property: value;
}
```

Ý nghĩa: Khi con trỏ chuột nằm trên phần tử được chọn, các thuộc tính CSS bên trong `:hover` sẽ được kích hoạt.

2. Ví Dụ Cơ Bản HTML & CSS Với :hover

```
<!DOCTYPE html>
<html lang="vi">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Sample Text</title>
    <style>
        a:hover {
            color: red;
            text-decoration: underline;
        }
        button:hover {
            background-color: #007bff;
            color: #fff;
            cursor: pointer;
        }
    </style>
</head>
<body>
    <h2>Sample Text</h2>
    <a href="#">Sample Text</a>
    <br><br>
    <button>Sample Text</button>
</body>
</html>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trực Quan Trạng Thái Di Chuột :hover**

- **Khi rê con trỏ chuột vào liên kết <a>**:

  Di chuột vào đây để thấy hiệu ứng! (Chữ lập tức chuyển màu đỏ rực rỡ kèm đường gạch chân).

- **Khi rê con trỏ chuột vào nút bấm <button>**:

  Di chuột vào nút này (Nền chuyển sang xanh dương đậm `#007bff`, chữ trắng nổi bật, con trỏ đổi thành hình bàn tay `pointer`).

[Khái Niệm] 3. Các Phần Tử Có Thể Dùng Với :hover

Hầu như mọi phần tử HTML đều có thể áp dụng `:hover`, không chỉ các thẻ tương tác: `<a>`, `<button>`, `<div>`, `<p>`, `<img>`, `<span>`, v.v.

Ví dụ với hình ảnh:

Hiệu Ứng Phóng To Ảnh Khi Hover

```
1 img:hover {
2     transform: scale(1.1);
3     transition: 0.3s ease;
4 }
```

BROWSER PREVIEW **Kết Quả Hiển Thị Hiệu Ứng Phóng To Ảnh**

- **Ban đầu (Normal State)**:

  Hình ảnh (100%) (Ảnh hiển thị với tỷ lệ mặc định 100%).

- **Khi di chuột vào (Hover State)**:

  Hình ảnh (110%) (Nhờ thuộc tính `transform: scale(1.1)`, ảnh tự động phóng to mượt mà 0.3s).

[Mẹo] 4. Ứng Dụng Thực Tế Của :hover

- **Thiết kế menu điều hướng**: Khi hover vào mục menu sẽ đổi màu hoặc hiển thị menu thả xuống (dropdown).
- **Nút bấm nổi bật**: Nút thay đổi màu sắc hoặc hiệu ứng khi người dùng di chuột qua.
- **Thẻ sản phẩm (hover card)**: Khi hover vào thẻ sản phẩm sẽ hiển thị thêm thông tin chi tiết.
- **Hiệu ứng ảnh**: Phóng to ảnh, làm mờ hoặc thêm bộ lọc khi hover.
- **Tooltip**: Hiển thị chú thích hoặc thông tin bổ sung khi di chuột vào phần tử.

Ví Dụ Menu Với Dropdown Khi Hover

```
<style>
    .menu {
        position: relative;
        display: inline-block;
    }
    .dropdown {
        display: none;
        position: absolute;
        background-color: #f9f9f9;
        padding: 10px;
        box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
    }
    .menu:hover .dropdown {
        display: block;
    }
</style>
<div class="menu">
    <button>Sample Text</button>
    <div class="dropdown">
        <p>Sample Text</p>
        <p>Sample Text</p>
        <p>Sample Text</p>
    </div>
</div>
```

BROWSER PREVIEW **Minh Họa Trực Quan Trạng Thái Giao Diện Dropdown Menu**

- **Trạng thái 1 – Ban đầu (Chưa di chuột vào nút Menu)**:

  [Menu] (Thẻ `.dropdown` ở trạng thái `display: none` nên hoàn toàn bị ẩn).

- **Trạng thái 2 – Khi di chuột vào nút Menu (:hover)**:

  [Menu] → Menu thả xuống hiện ra ngay phía dưới:

  Nội Dung Dropdown Xuất Hiện:
  - **Mục 1**
  - **Mục 2**
  - **Mục 3**

[Cảnh Báo] 5. Lưu Ý Quan Trọng Khi Sử Dụng :hover

- **Hover không hoạt động trên thiết bị cảm ứng**: Trên điện thoại hoặc tablet không có con trỏ chuột, nên bạn có thể kết hợp với `:focus` hoặc JavaScript để hỗ trợ trải nghiệm người dùng.
- **Thứ tự viết CSS (Quy tắc LVHA)**: Khi kết hợp với các pseudo-class khác (như `:link`, `:visited`, `:active`), hãy viết đúng thứ tự:

  `:link → :visited → :hover → :active`

[Khái Niệm] 6. Tổng Kết Về :hover

`:hover` là một trong những pseudo-class mạnh mẽ nhất trong CSS, giúp bạn:

- **Tăng tính tương tác**: Làm nổi bật các phần tử khi người dùng di chuột qua.
- **Cải thiện UX/UI**: Tạo hiệu ứng hình ảnh, nút bấm, và dropdown mượt mà.
- **Dễ dàng triển khai**: Chỉ cần vài dòng CSS là đã có thể làm giao diện sinh động hơn!

d. Bộ Chọn :active (Trạng Thái Khi Nhấp Bấm Giữ Chuột)

Trong CSS, `:active` là một pseudo-class được sử dụng để chọn một phần tử khi nó đang được nhấp vào (tức là khi bạn giữ chuột trái trên phần tử đó nhưng chưa thả ra). Nó thường được dùng để tạo hiệu ứng nhấn, giúp giao diện sinh động hơn!

[Khái Niệm] 1. Cú Pháp Tổng Quát

Cú Pháp Định Kiểu :active

```
selector:active {
    /* Sample Text */
}
```

[Khái Niệm] 2. Ví Dụ Cơ Bản Với Nút Bấm

Mã Nguồn HTML & CSS Tạo Hiệu Ứng Nút Bấm Với :active

```
<button class="btn">Sample Text</button>
<style>
    .btn {
        padding: 10px 20px;
        background: blue;
        color: white;
        border: none;
        border-radius: 5px;
    }
    .btn:active {
        background: red;
        transform: scale(0.95);
    }
</style>
```

Giải thích:

- Khi nhấn giữ nút, nền sẽ chuyển sang màu đỏ và nút sẽ hơi thu nhỏ lại (`transform: scale(0.95)` – hiệu ứng nhấn).
- Khi thả chuột, nút quay về trạng thái bình thường.

BROWSER PREVIEW Kết Quả Hiển Thị Demo Nút Bấm Cơ Bản Với :active

- **Trạng thái ban đầu**: Nút màu xanh dương (`blue`), chữ trắng, kích thước 100%.
- **Khi nhấp giữ chuột trái (:active)**: Nút lập tức đổi nền sang màu đỏ (`red`) và thu nhỏ về 95% kích thước ban đầu.
- **Khi thả chuột ra**: Nút quay về nền màu xanh và kích thước bình thường.

[Mẹo] 3. Ứng Dụng Thực Tế Của :active

- **Tạo hiệu ứng bấm nút**: Đổi màu hoặc kích thước khi nhấn nút.
- **Hiệu ứng nhấn menu hoặc link**: Làm nổi bật liên kết khi nhấp vào.
- **Phản hồi hành động người dùng**: Giúp người dùng thấy ngay lập tức họ đã nhấn vào đâu.

4. Ví Dụ Định Kiểu Tương Tác Với Thẻ Liên Kết (Link):

Mã Nguồn CSS Kết Hợp :hover Và :active Cho Thẻ <a>

```
<a href="#" class="link">Sample Text</a>
<style>
    .link {
        color: blue;
        text-decoration: none;
        padding: 5px;
    }
    .link:hover {
        color: green;
    }
    .link:active {
        color: red;
    }
</style>
```

[Khái Niệm] Giải Thích Chi Tiết Các Trạng Thái Tương Tác

- Khi di chuột vào link → màu xanh lá (`green`).
- Khi nhấn giữ link → màu đỏ (`red` – trạng thái active).
- Khi thả chuột → màu trở về bình thường (`blue`).

BROWSER PREVIEW **Kết Quả Hiển Thị Minh Họa Tương Tác Link Với :active**

- **Di chuột vào link**: Chữ đổi màu xanh lá (`green`).
- **Nhấn giữ chuột trái**: Chữ chuyển ngay sang màu đỏ (`red`).
- **Thả chuột ra**: Chữ trở về màu xanh dương (`blue`) mặc định.

[Cảnh Báo] 5. Lưu Ý Quan Trọng Khi Sử Dụng :active

1. **Thời gian hiệu lực**: Chỉ hoạt động trong lúc nhấn giữ chuột. Khi thả ra, trạng thái `:active` sẽ mất ngay lập tức.
2. **Thứ tự ưu tiên khi viết CSS (Quy tắc LVHA)**:
   Nếu kết hợp nhiều pseudo-class cho liên kết, thứ tự đúng bắt buộc phải là:
   `a:link → a:visited → a:hover → a:active`
   Lý do: Vì trình duyệt áp dụng theo thứ tự này, nếu bạn viết `:active` trước `:hover`, hiệu ứng `:active` có thể không hiển thị đúng do bị `:hover` ghi đè.
3. **Khả năng kích hoạt của các phần tử**:
   - Phần tử có thể nhấp mặc định: `button`, `a`, `input`, `label`, v.v.
   - Với thẻ không tương tác (ví dụ `div`, `span`), bạn cần thêm thuộc tính `tabindex="0"` hoặc dùng JavaScript để bắt sự kiện.

[Khái Niệm] 6. Ví Dụ Nâng Cao: Tạo Hiệu Ứng Nhấn Cho Khối Div Bằng Tabindex

Mã Nguồn HTML & CSS Sử Dụng Tabindex Cho Card Div

```
<div class="card" tabindex="0">
    Sample Text
</div>
<style>
    .card {
        width: 200px;
        height: 100px;
        background: lightblue;
        display: flex;
        align-items: center;
        justify-content: center;
        border: 2px solid blue;
    }
    .card:active {
        background: orange;
        border: 2px solid red;
    }
</style>
```

Giải thích:

- `tabindex="0"` giúp phần tử `<div>` có thể nhận focus, từ đó sử dụng được `:active`.
- Khi nhấn vào, nền đổi thành màu cam (`orange`), viền đổi thành màu đỏ (`red`).

BROWSER PREVIEW **Kết Quả Hiển Thị Khối Div Nâng Cao Với Tabindex**

- **Trạng thái ban đầu**:

  Nhấn vào tôi! (Khối `card` có nền xanh nhạt, viền xanh dương).

- **Khi nhấn giữ chuột vào khối card (:active)**:

  Nhấn vào tôi! (Nền đổi ngay sang màu cam `orange`, viền chuyển thành màu đỏ `red`).

[Mẹo] 7. Khi Nào Nên Dùng :active

- **Nút bấm, liên kết**: Tạo phản hồi nhanh khi người dùng nhấn vào.
- **Trò chơi hoặc ứng dụng web**: Phát hiện hành động nhấn giữ để thực hiện thao tác (ví dụ như giữ nút bắn trong game).
- **Giao diện kéo thả**: Làm nổi bật phần tử đang được nhấn giữ.

[Khái Niệm] 8. Tóm Lại Về :active

- `:active` giúp chọn phần tử trong lúc nhấn giữ chuột.
- **Ứng dụng**: Tạo hiệu ứng bấm nút, nhấn link, hoặc làm nổi bật phần tử đang được chọn.
- Hoạt động tốt với các phần tử tương tác và có thể mở rộng với thuộc tính `tabindex`.

e. Chuyên Đề Phân Tích Sâu: Bộ Chọn :focus, :focus-visible & :focus-within (Trạng Thái Tiêu Điểm Nhập Liệu & Bàn Phím)

Trạng thái tiêu điểm (**Focus State**) là một trong những khía cạnh quan trọng nhất trong thiết kế giao diện Web hiện đại và Trợ năng (**Web Accessibility - WCAG**). Nó giúp người dùng nhận biết chính xác phần tử nào đang nhận tương tác từ bàn phím hoặc thiết bị nhập liệu.

[Khái Niệm] 1. Bản Chất & Cú Pháp Của Pseudo-Class :focus

- **Bản chất**: Pseudo-class `:focus` kích hoạt khi một phần tử nhận tiêu điểm điều hướng – bằng cách nhấp chuột vào, chạm ngón tay trên thiết bị cảm ứng, dùng phím `Tab` di chuyển tới, hoặc gọi hàm `element.focus()` trong JavaScript.
- **Các phần tử có khả năng nhận focus mặc định**: Thẻ ô nhập liệu (`<input>`, `<textarea>`, `<select>`), nút bấm (`<button>`), liên kết (`<a>`), thẻ tóm tắt (`<summary>`) và bất kỳ phần tử nào có thuộc tính `tabindex`.
- **Cú pháp tổng quát**:

Cú Pháp Định Kiểu :focus

```
selector:focus {
    outline: none; /* Sample Text */
    border-color: #007bff; /* Sample Text */
    box-shadow: 0 0 8px rgba(0, 123, 255, 0.4); /* Sample Text */
}
```

[Khái Niệm] 2. Ví Dụ Tùy Biến Giao Diện Form Chuẩn UI/UX

Mã Nguồn HTML & CSS Định Kiểu Ô Nhập Liệu Khi Focus

```
<style>
    .form-input {
        width: 100%;
        padding: 12px 16px;
        font-size: 14px;
        border: 2px solid #ccc;
        border-radius: 8px;
        outline: none;
        transition: all 0.3s ease;
    }
    .form-input:focus {
        border-color: #007bff;
        box-shadow: 0 0 10px rgba(0, 123, 255, 0.35);
        background-color: #f8fbff;
    }
</style>

<label for="username">Sample Text</label>
<input type="text" id="username" class="form-input" placeholder="Sample
    Text">
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trực Quan Khi Focus Ô Nhập Liệu**

- **Trạng thái ban đầu (Unfocused)**: Ô nhập liệu hiển thị viền xám nhạt (`#ccc`), nền màu trắng phẳng.
- **Khi nhấp chuột hoặc bấm phím Tab vào ô (Focused)**: Viền ô lập tức sáng lên màu xanh dương đậm (`#007bff`), nền chuyển sang màu nhạt (`#f8fbff`), kèm theo quầng sáng tỏa mượt (`box-shadow glow`) xung quanh khung nhập.

[Khái Niệm] 3. Chuyên Đề Phân Biệt :focus Và :focus-visible (Modern Accessibility)

- **Vấn đề của :focus truyền thống**: Khi người dùng nhấp chuột vào nút bấm (`<button>`), trình duyệt sẽ vẽ một đường viền đen (**Focus Ring / Outline**) gây mất thẩm mỹ giao diện. Nếu lập trình viên xóa viền này bằng `outline: none`, người dùng phím `Tab` (người khuyết tật dùng thiết bị trợ năng) sẽ hoàn toàn không thấy tiêu điểm ở đâu!
- **Giải pháp Modern CSS :focus-visible**: Chỉ hiển thị đường viền tiêu điểm khi phần tử được điều hướng bằng **bàn phím (Phím Tab)**, còn khi nhấp bằng chuột thì trình duyệt sẽ **tự động ẩn đường viền**!

Mã Nguồn CSS Tối Ưu Tiêu Điểm Chuẩn WCAG Với :focus-visible

```css
/* Sample Text */
button:focus {
    outline: none;
}

/* Sample Text */
button:focus-visible {
    outline: 3px solid #ff9800;
    outline-offset: 3px;
}
```

BROWSER PREVIEW **So Sánh Trải Nghiệm Chuột Vs Bàn Phím Với :focus-visible**

- **Khi nhấp chuột vào nút bấm (Mouse User)**:

  Nút Bấm (Nút kích hoạt bình thường mà **KHÔNG xuất hiện viền đen** — `outline` bị ẩn).

- **Khi dùng phím Tab di chuyển tới nút (Keyboard User)**:

  Nút Bấm (Xuất hiện viền cam rực rỡ `#ff9800` cách mép nút 3px giúp định vị tiêu điểm).

[Khái Niệm] 4. Chuyên Đề Phân Tích :focus-within (Tiêu Điểm Thẻ Cha Khi Con Focus)

- **Bản chất**: Pseudo-class `:focus-within` kích hoạt trên phần tử cha nếu **chính nó** HOẶC **bất kỳ phần tử con/cháu nào nằm bên trong nó** nhận tiêu điểm focus!
- **Ứng dụng thực tế**: Định kiểu lại khung chứa Search Bar, Card Form, hoặc nhóm nút bấm khi người dùng tương tác với bất kỳ ô nhập liệu nào bên trong.

Mã Nguồn CSS Tạo Thanh Tìm Kiếm Đổi Màu Khung Với :focus-within

```
<style>
    .search-box {
        display: flex;
        align-items: center;
        padding: 8px 16px;
        border: 2px solid #ddd;
        border-radius: 25px;
        background: #f9f9f9;
        transition: all 0.3s ease;
    }
    /* Sample Text */
    .search-box:focus-within {
        border-color: #28a745;
        background: #fff;
        box-shadow: 0 4px 12px rgba(40, 167, 69, 0.25);
    }
    .search-input {
        border: none;
        outline: none;
        background: transparent;
        width: 100%;
    }
</style>

<div class="search-box">
    <span class="icon">\textbf{[KiTextm Tra]}</span>
    <input type="text" class="search-input" placeholder="Sample Text">
</div>
```

BROWSER PREVIEW **Minh Họa Trực Quan Thanh Tìm Kiếm Với :focus-within**

- **Khi chưa nhấp vào ô nhập (Normal State)**: Khung chứa `.search-box` có viền màu xám (`#ddd`) và nền xám nhạt (`#f9f9f9`).
- **Khi nhấp vào ô input tìm kiếm (:focus-within)**: Nhờ bộ chọn `:focus-within`, toàn bộ khung bao ngoài `.search-box` (chứa cả icon kính lúp và input) đồng loạt chuyển nền sang trắng rực rỡ, viền đổi thành màu xanh lá (`#28a745`) kèm hiệu ứng bóng đổ nổi khối nổi bật.

[Bảng] Bảng So Sánh Bộ Chọn :focus vs :focus-visible vs :focus-within

Selector	Điều kiện kích hoạt	Ví dụ ứng dụng	Trải nghiệm UX / a11y
`:focus`	**Bất kỳ khi nào phần tử nhận tiêu điểm (Chuột, Tab, JS).**	`input:focus { border-color: blue; }`	Tuyệt vời cho Form input nhưng có thể hiện viền đen xấu trên Nút bấm khi nhấp chuột.
`:focus-visible`	**CHỈ khi phần tử nhận tiêu điểm từ bàn phím (Phím Tab).**	`button:focus-visible { outline: 2px solid orange; }`	Chuẩn WCAG hiện đại: Ẩn viền khi dùng chuột, hiện viền rõ nét khi dùng bàn phím.
`:focus-within`	**Khi CHÍNH NÓ hoặc BẤT KỲ THẺ CON NÀO nhận focus.**	`.form-group:focus-within { background: lightyellow; }`	Tô sáng toàn bộ khối khung chứa (Form Group, Search Box) khi con bên trong tương tác.

[Cảnh Báo] Lưu Ý Quan Trọng Cho Focus Selectors (Accessibility Checklist)

- **TUYỆT ĐỐI KHÔNG DÙNG `outline: none` ĐƠN ĐỘC**: Nếu bạn xóa `outline`, bắt buộc phải cung cấp style thay thế (`border-color`, `box-shadow`, hoặc `:focus-visible`) để người dùng bàn phím không bị mất tiêu điểm.
- **Thứ tự ưu tiên khi viết CSS (Quy tắc LVHFA)**: Khi viết cùng lúc cho thẻ liên kết `<a>`, thứ tự chuẩn là:

  `:link → :visited → :hover → :focus → :active`

[Khái Niệm] Tóm Tắt Cốt Lõi Về Focus Selectors

- `:focus`: Kiểm soát tiêu điểm cá nhân cho ô nhập liệu.
- `:focus-visible`: Tối ưu trải nghiệm bàn phím chuẩn Accessibility.
- `:focus-within`: Quản lý tiêu điểm cho khối khung chứa cha từ phần tử con.

f. Bộ Chọn Lớp Giả :target (Trạng Thái Định Vị Thẻ Theo URL Anchor ID)

Bộ chọn lớp giả `:target` trong CSS được dùng để chọn phần tử HTML được nhắm đến (target) bởi liên kết có chứa `#id` trong URL, tức là khi phần tử đó được ”kích hoạt” qua một đường dẫn có gắn `#id`.

Cú Pháp Cơ Bản Của Bộ Chọn :target

```
#section1:target {
    background: yellow;
}
```

[Khái Niệm] Cách Hoạt Động Cơ Bản Của :target

1. **Bước 1**: Khi bạn click vào một liên kết như: `<a href="#section1">Xem mục 1</a>`.
2. **Bước 2**: Trình duyệt sẽ tự động cuộn đến phần tử có thuộc tính `id="section1"` trong trang web.
3. **Bước 3**: CSS `#section1:target` sẽ lập tức áp dụng lên phần tử này ngay tại thời điểm nó được ”target”.

Ví Dụ Đầy Đủ & Phân Tích So Sánh Trước / Sau Khi Dùng :target:

Trường Hợp 1 – Khi KHÔNG Dùng :target (Chưa Nhấp Link Đã Có Style)

```
<a href="#about">Sample Text</a>
<div id="about">Sample Text</div>

<style>
  #about {
    background-color: yellow;
    padding: 10px;
    border: 2px solid red;
  }
</style>
```

[Cảnh Báo] Phân Tích Lỗi Khi KHÔNG Dùng :target

CSS ở ví dụ trên đang áp dụng mặc định cho phần tử có `id="about"`. Điều này **không liên quan gì đến :target cả** – tức là khối `#about` luôn luôn có màu nền vàng và viền đỏ ngay từ đầu, ngay cả khi người dùng chưa click vào link!

Trường Hợp 2 – Khi DÙNG :target (Chỉ Kích Hoạt Style Khi Click Link)

```
<a href="#about">Sample Text</a>
<div id="about">Sample Text</div>

<style>
  #about:target {
    background-color: yellow;
    padding: 10px;
    border: 2px solid red;
  }
</style>
```

BROWSER PREVIEW **Kết Quả So Sánh Trước Và Sau Khi Nhấp Link**

- **Trước khi nhấp vào link**: Phần tử `#about` hiển thị hoàn toàn bình thường (không có nền vàng, không có viền đỏ).
- **Khi bạn click vào liên kết "Xem giới thiệu"**: Phần tử `#about` sẽ:
  - Được trình duyệt tự động cuộn tới.
  - Lập tức chuyển sang màu nền vàng kẽm viền đỏ nổi bật nhờ kích hoạt bộ chọn `#about:target`.

[Mẹo] Lưu Ý Cốt Lõi Về :target

- `:target` **chỉ áp dụng khi URL có chứa #id**.
- Khi bạn truy cập lại URL gốc (không chứa `#`) hoặc click sang phần tử khác, thì CSS của `:target` trên phần tử cũ sẽ tự động mất tác dụng.

[Bảng] Các Ứng Dụng Thực Tế Của Bộ Chọn :target

Ứng dụng thực tế	Mô tả chi tiết
`Điều hướng trong 1 trang`	Highlight mục đang xem trong trang (kiểu như Table of Contents / Mục lục bài viết).
`Accordion / Toggle đơn giản`	Mở / đóng khối nội dung thả xuống mà không cần sử dụng JavaScript.
`Hiệu ứng nhấn nút cuộn xuống`	Nhấn nút rồi scroll + đổi màu mục tiêu được cuộn đến.

So Sánh :target Với Các Selector Trạng Thái Khác:

[Bảng] Bảng So Sánh :hover vs :focus vs :target

Selector	Ý nghĩa & Thời điểm kích hoạt
`:hover`	Kích hoạt khi con trỏ chuột rê vào phần tử.
`:focus`	Kích hoạt khi phần tử nhận tiêu điểm nhập liệu (thường dùng cho input, button).
`:target`	Kích hoạt khi URL có chứa mã băm `#id` trùng khớp hoàn toàn với `id` của phần tử đó.

Chi Tiết Về Cách Hoạt Động Của :target:

[Khái Niệm] 1. Cơ Chế Xử Lý Của Trình Duyệt Khi Duyệt URL

- **Khi URL có chứa #id** (ví dụ: `example.com/page.html#about`):
  - Trình duyệt tự động cuộn màn hình đến phần tử có `id="about"`.
  - Nếu CSS có khai báo `#about:target`, quy tắc CSS này sẽ được áp dụng.
- **Khi KHÔNG có #id trong URL**:
  - Không có phần tử nào bị xem là `:target`.
  - CSS `:target` sẽ không áp dụng cho bất kỳ phần tử nào trên trang.

Ví Dụ Minh Họa Đơn Giản URL Target

```
<!-- Sample Text -->
<a href="#section1">Sample Text</a>
<div id="section1">Sample Text</div>

<!-- Sample Text -->
<style>
  #section1:target {
    background-color: yellow;
    border: 2px solid red;
  }
</style>
```

BROWSER PREVIEW — Kết Quả Khi Click Và Khi Không Có Target

- **Khi click vào liên kết**:
  1. URL đổi thành: `.../page.html#section1`.
  2. Thẻ `<div id="section1">` trở thành target hiện tại.
  3. CSS `:target` được áp dụng → Hiển thị nền vàng, viền đỏ.
- **Khi bạn Refresh lại trang (mất #), Click sang mục khác, hoặc Chạy trang lần đầu**:
  – Không có phần tử nào là `:target`, nên không có CSS `:target` nào được áp dụng.

[Bảng] Bảng So Sánh Tình Huống: Không Dùng :target vs Dùng :target

Tình huống tương tác	Không dùng :target	Dùng :target (Có CSS :target)
Click link có `href="#abc"`	Trình duyệt chỉ cuộn đến phần tử `id="abc"`.	Trình duyệt cuộn + áp dụng CSS `:target`.
Click vào phần tử không có id	Không cuộn, không có gì xảy ra.	Không cuộn, không có gì xảy ra.
Không click gì	Không có thay đổi giao diện.	Không có thay đổi giao diện.
Muốn thay đổi giao diện khi cuộn	Không làm được bằng CSS thuần.	Có thể highlight tự động phần tử được nhắm.

[Bảng] Bảng Phân Tích Các Lưu Ý Khi Dùng :target

Lưu ý kỹ thuật	Giải thích nguyên lý
`:target chỉ áp dụng khi URL chứa #id trùng với id phần tử`	CSS không tự động chạy nếu không có sự kiện dẫn tới mã băm `#id`.
`Không dùng được để kiểm tra trạng thái thường xuyên`	Nếu bạn muốn thay đổi giao diện dựa trên trạng thái khác (như toggle phức tạp), dùng JS sẽ linh hoạt hơn.
`Có thể lợi dụng :target để tạo hiệu ứng toggle, accordion, modal...`	Thực hiện hoàn toàn bằng CSS thuần mà không cần JavaScript nếu HTML được cấu trúc hợp lý.
`Không nên lạm dụng :target cho UI phức tạp`	Vì không kiểm soát được nhiều trạng thái như hover, active hoặc nhiều phần tử cùng lúc.

[Bảng] Tóm Tắt Nhanh Về Đặc Điểm Bộ Chọn :target

Đặc điểm	Chi tiết bộ chọn :target
Điều kiện áp dụng	URL chứa `#id` trùng khớp với `id` của phần tử.
Phạm vi ảnh hưởng	CHỈ 1 phần tử duy nhất tại 1 thời điểm.
Dùng để làm gì?	Highlight bài viết, Accordion toggle, Scroll-to-section.
Ưu điểm chính	Không cần JavaScript vẫn làm được hiệu ứng chuyển màu / highlight.
Nhược điểm chính	Không có trạng thái ”tắt”, khó linh hoạt khi cần chuyển trạng thái phức tạp.

Mã Nguồn Hoàn Chỉnh Thử Nghiệm "Chuyện Tình Bông Hoa Màu Tím":

Mã Nguồn HTML5 & CSS3 Đầy Đủ Thử Nghiệm :target

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8" />
  <title>Sample Text</title>
  <style>
    h2:target {
      color: blue;
      background-color: #eef;
      padding: 10px;
      border-left: 5px solid blue;
      transition: all 0.3s ease;
    }
  </style>
</head>
<body>
  <h1>Sample Text</h1>
  <ul>
    <li><a href="#mobai">Sample Text</a></li>
    <li><a href="#thanbai">Sample Text</a></li>
    <li><a href="#ketbai">Sample Text</a></li>
  </ul>
  <h2 id="mobai">Sample Text</h2>
  <p>Paragraph text</p>
  <h2 id="thanbai">Sample Text</h2>
  <p>Paragraph text</p>
  <h2 id="ketbai">Sample Text</h2>
  <p>Paragraph text</p>
</body>
</html>
```

BROWSER PREVIEW **Kết Quả Hiển Thị: Trạng Thái Target Khi Nhấp Liên Kết Anchor**

- **Thanh điều hướng liên kết**:
  - `Chuyển đến Mở bài`
  - **Chuyển đến Thân bài** ← *(Đã nhấp liên kết)*
  - `Chuyển đến Kết bài`
- **Giao diện trực quan bài viết**:
  - **Mở bài**: Tiêu đề `<h2>` hiển thị màu đen mặc định.
  - **Thân bài**: Kích hoạt `h2:target` → Tiêu đề chuyển sang **chữ màu xanh dương**, viền trái màu xanh đậm `5px solid blue`, nền xanh nhạt nổi bật thu hút ánh nhìn!
  - **Kết bài**: Tiêu đề `<h2>` hiển thị màu đen mặc định.

[Khái Niệm] Khẳng Định Đúng Cốt Lõi Về :target

"Khi người dùng click vào link nào thì CSS với lớp giả :target được chỉ định sẽ hoạt động tại đó."

Giải thích chi tiết:

1. Khi người dùng click vào thẻ `<a>` có `href="#id"` (Ví dụ `<a href="#thanbai">`), trình duyệt cuộn xuống phần tử có `id="thanbai"` và phần tử đó trở thành `:target`.
2. CSS `h2:target { color: blue; }` chỉ áp dụng duy nhất cho phần tử được nhắm đến. Các phần tử `<h2>` khác không bị ảnh hưởng.
3. Khi nhấp sang link khác, `:target` chuyển sang phần tử mới. Phần tử cũ lập tức mất hiệu ứng. Chỉ duy nhất một phần tử là `:target` tại một thời điểm.

Ví Dụ Thực Tế Nhiều Box & Bảng Phân Tích Ảnh Hưởng:

Mã Nguồn Thử Nghiệm 3 Khối Box Với :target

```
<!-- Sample Text -->
<a href="#box1">Sample Text</a>
<a href="#box2">Sample Text</a>

<!-- Sample Text -->
<div id="box1" class="box">Box 1</div>
<div id="box2" class="box">Box 2</div>
<div id="box3" class="box">Box 3</div>

<!-- Sample Text -->
<style>
  .box:target {
    background-color: yellow;
    border: 2px solid red;
  }
</style>
```

[Bảng] Bảng Phân Tích Tác Động Tương Tác Khi Click Link

Thao tác Click link	Phần tử bị ảnh hưởng	Phân tích tác động
Click link `href="#box1"`	Chỉ `div#box1`	`div#box1` có nền vàng, viền đỏ. Các box khác bình thường.
Click link `href="#box2"`	Chỉ `div#box2`	`div#box2` có nền vàng, viền đỏ. `div#box1` mất hiệu ứng.
Không click hoặc click ngoài	Không box nào	Không có phần tử nào là `:target`.
Box 3 (`id="box3"`)	Không bao giờ	Vì không có liên kết nào trỏ tới `#box3`.

BROWSER PREVIEW **Kết Quả Hiển Thị Trực Quan Giao Diện Thử Nghiệm 3 Khối Box**

- **Tình huống A (Khi chưa nhấp liên kết nào – URL gốc)**:
  - Khối `Box 1`, `Box 2`, `Box 3` hiển thị chữ màu xám đen bình thường, không có nền vàng hay viền đỏ.
- **Tình huống B (Khi người dùng click vào link "Xem Box 1" → URL: `.../index.html#box1`)**:
  - **Box 1** (`div#box1`): Lập tức chuyển sang **màu nền vàng**, viền phát sáng **màu đỏ** (`2px solid red`).
  - **Box 2** & **Box 3**: Giữ nguyên trạng thái bình thường.
- **Tình huống C (Khi nhấp tiếp vào link "Xem Box 2" → URL: `.../index.html#box2`)**:
  - **Box 1**: Tự động ngắt hiệu ứng `:target`, trở về trạng thái bình thường.
  - **Box 2** (`div#box2`): Trở thành target mới → Chuyển sang nền vàng, viền đỏ!
  - **Box 3** (`div#box3`): Không bao giờ bị tác động vì không có thẻ `<a>` nào chứa `href="#box3"`.

[Mẹo] Kết Luận Quan Trọng Về :target

Chỉ phần tử nào có `id` trùng với `#` trong URL mới được CSS `:target` áp dụng. Các phần tử khác dù giống nhau về kiểu hay class nhưng không được target thì không bị ảnh hưởng!

g. Chuyên Đề Thực Nghiệm Sâu: Ẩn/Hiện Phần Tử Con Với :hover & Bộ Chọn Hậu Duệ

Dòng mã này tạo ra một hộp màu vàng có nội dung và một biểu tượng ẩn đi. Khi bạn di chuột vào hộp, nền sẽ chuyển sang màu cam và biểu tượng xuất hiện.

Mã Nguồn Cấu Trúc HTML

```
<div class="box">Content text
    <i>Icon</i>
</div>
```

- `<div class="box">`: Tạo một khối chứa nội dung và icon.
- `<i>Icon</i>`: Thẻ `<i>` đại diện cho icon hoặc nội dung bổ sung. Ban đầu bị ẩn nhờ CSS.

Mã Nguồn CSS Định Kiểu Ẩn/Hiện Khi Hover

```css
.box {
    background: yellow;
    width: 300px;
    height: 200px;
    margin-top: 20px;
}
.box:hover {
    background: orange;
}
.box i {
    display: none;
}
.box:hover i {
    display: inline;
}
```

[Khái Niệm] Phân Tích Chi Tiết Các Thẻ CSS

- `background: yellow`: Đặt nền màu vàng cho `.box`.
- `width: 300px` và `height: 200px`: Kích thước của hộp là 300x200 pixel.
- `margin-top: 20px`: Tạo khoảng cách 20px phía trên hộp.
- `.box:hover`: Khi di chuột vào `.box`, nền đổi sang màu cam.
- `.box i`: Ban đầu, thẻ `<i>` không hiển thị nhờ `display: none`.
- `.box:hover i`: Khi di chuột vào `.box`, thẻ `<i>` hiện ra dưới dạng `inline` (nằm trên cùng dòng với nội dung khác).
- Khi di chuột vào phần tử có class `.box`, tất cả thẻ `<i>` bên trong sẽ được hiển thị theo kiểu inline.

[Khái Niệm] Giải Thích Sâu Kỹ Thuật: Bộ Chọn Hậu Duệ (Descendant Selector)

Câu lệnh này viết theo bộ chọn hậu duệ (Descendant Selector):

Cú Pháp Bộ Chọn Hậu Duệ

```
.box:hover i {
    display: inline;
}
```

- `.box`: Phần tử cha.
- `i`: Phần tử con nằm bên trong `.box`.
- `:hover`: Kích hoạt khi di chuột vào phần tử cha `.box`.

→ **Bộ chọn hậu duệ chọn tất cả thẻ `<i>` nằm bên trong `.box`, bất kể mức lồng nhau bao nhiêu.**

Ví dụ, cả hai thẻ `<i>` dưới đây đều bị ảnh hưởng khi hover vào `.box`:

Ví Dụ Mức Lồng Nhau Của Bộ Chọn Hậu Duệ

```
<div class="box">
    <i>Icon 1</i> <!-- Sample Text -->
    <div>
        <i>Icon 2</i> <!-- Sample Text -->
    </div>
</div>
```

Nếu bạn muốn chỉ ảnh hưởng trực tiếp lên thẻ `<i>` con trực tiếp thì phải dùng Child Selector:

Sử Dụng Child Selector (>)

```
.box:hover > i {
    display: inline;
}
```

BROWSER PREVIEW **So Sánh Kết Quả Minh Họa Thực Nghiệm Cho 2 Bộ Chọn**

- **Trường hợp 1 – Sử dụng Descendant Selector (`.box:hover i`)**:
  - **Ban đầu**: Hộp màu vàng (`background: yellow`), chỉ hiển thị chữ "Nội dung Box". Cả `Icon 1` và `Icon 2` đều ẩn.
  - **Khi di chuột vào .box**: Nền chuyển sang màu cam (`background: orange`). Cả hai thẻ `<i>` (**Icon 1** và **Icon 2**) nằm bên trong khối `.box` (dù ở bất kỳ cấp lồng nhau nào) đều chuyển thành `display: inline` và **xuất hiện cùng lúc**.
- **Trường hợp 2 – Sử dụng Child Selector (`.box:hover > i`)**:
  - **Ban đầu**: Hộp màu vàng, tất cả Icon đều ẩn.
  - **Khi di chuột vào .box**: Nền chuyển sang màu cam. **CHỈ CÓ** thẻ `<i>Icon 1</i>` (con trực tiếp của `.box`) mới xuất hiện; còn `<i>Icon 2</i>` (nằm trong thẻ `<div>` cháu) **vẫn tiếp tục bị ẩn**.

[Mẹo] Ứng Dụng Thực Tế Nâng Cao

- **Hiệu ứng tooltip**: Khi hover vào thẻ, hiển thị mô tả hoặc ghi chú.
- **Thông báo nhỏ**: Ẩn/hiện thông báo hoặc biểu tượng trạng thái.
- **Menu thả xuống**: Hiện menu con khi hover vào nút hoặc thanh điều hướng.

e. Đọc Thêm: Chuyên Đề Nâng Cao Về Pseudo-Classes Trạng Thái Tương Tác

Để giúp bạn làm chủ hoàn toàn các trạng thái tương tác trong môi trường sản xuất thực tế (Production), dưới đây là các phân tích chuyên sâu, kinh nghiệm thực chiến và câu hỏi phỏng vấn tuyển dụng thường gặp:

[Khái Niệm] 1. Xử Lý Lỗi "Sticky Hover" Trên Thiết Bị Cảm Ứng (Touch Devices)

- **Vấn đề thực tế**: Trên smartphone hoặc tablet (màn hình cảm ứng), khi người dùng chạm ngón tay vào nút bấm, trình duyệt sẽ **dính chặt trạng thái :hover** ngay cả sau khi người dùng đã nhấc tay ra, gây ra lỗi giao diện rất khó chịu.
- **Giải pháp Modern CSS**: Sử dụng Media Query kiểm tra khả năng rơ chuột của phần cứng (`@media (hover: hover)`):

Tối Ưu :hover Chỉ Cho Thiết Bị Dùng Chuột

```
/* Sample Text */
@media (hover: hover) and (pointer: fine) {
    button:hover {
        background-color: #0056b3;
        transform: translateY(-2px);
    }
}
```

BROWSER PREVIEW Minh Họa Trực Quan Trải Nghiệm Trên Các Màn Hình

- **Trên máy tính Desktop / Laptop (Chuột cứng)**: Điều kiện `(hover: hover)` thỏa mãn. Khi di chuột qua nút → Nút nhích lên 2px và đổi màu xanh thẫm. Nhấc con trỏ ra → Nút trở lại vị trí ban đầu.
- **Trên Điện thoại / Máy tính bảng (Touchscreen)**: Điều kiện `(hover: hover)` không thỏa mãn → Trình duyệt bỏ qua khối lệnh hover này. Người dùng chạm ngón tay vào nút → Thực hiện thao tác Click mà **KHÔNG bị dính màu hover (Sticky Hover Bug)**.

[Mẹo] 2. Bí Quyết Ghi Nhớ Quy Tắc LVHA Đạt Chuẩn Phỏng Vấn

- **Mẹo ghi nhớ từ khóa**: Hãy nhớ cụm từ **"LoVe HAte"** (**L**ink → **V**isited → **H**over → **A**ctive) hoặc **"Lord Vader Has Arrived"**.
- **Giải thích kỹ thuật sâu**: Do cả 4 pseudo-classes này có **Độ ưu tiên Specificity bằng nhau** $(0, 1, 0)$. Nếu bạn viết `:link` sau `:hover`, khi di chuột vào liên kết chưa truy cập, thuộc tính của `:link` xuất hiện sau sẽ ghi đè và làm mất hiệu ứng của `:hover`!
- **Mở rộng thứ tự khi có :focus (Quy tắc LVHFA)**:

`:link` → `:visited` → `:hover` → `:focus` → `:active`

[Cảnh Báo] 3. Tối Ưu Hiệu Năng Render (60 FPS Performance Notes)

Khi định kiểu hiệu ứng `:hover`, hãy tuân thủ nguyên tắc tối ưu GPU:

- **KHÔNG NÊN**: Thay đổi các thuộc tính kích thước như `width`, `height`, `margin`, `padding`, `top`, `left` khi hover vì sẽ ép trình duyệt tính toán lại bố cục toàn trang (**Reflow / Layout Shift**), gây giật lag.
- **NÊN DÙNG**: Thay đổi `transform` (`scale`, `translate`) và `opacity`. Các thuộc tính này được đẩy thẳng lên card màn hình xử lý (**GPU Hardware Acceleration**), đảm bảo mượt mà 60 khung hình/giây (60fps).

[Mẹo] 4. Tiêu Chuẩn Trợ Năng Bàn Phím (Web Accessibility - a11y)

Để trang web thân thiện với người khuyết tật di chuyển bằng bàn phím (phím `Tab`):

- Luôn đi kèm `:hover` với `:focus` hoặc `:focus-visible`.
- Ví dụ: `button:hover, button:focus-visible { outline: 2px solid blue; }` giúp người dùng duyệt bằng phím Tab nhìn thấy rõ tiêu điểm đang nằm ở đâu.

7.2. Nhóm Vị Trí Vị Thế Cấu Trúc Cây DOM (Structural Selectors)

Cho phép chọn phần tử dựa trên thứ tự xuất hiện hoặc vị thế anh chị em trong cây HTML mà không cần gán class:

[Bảng] Bảng Tra Cứu Structural Pseudo-Classes Trong Cây DOM

Selector	Phạm vi	Ví dụ cú pháp	Chức năng & Ý nghĩa
`:first-child`	Tất cả phần tử	`li:first-child { color: red; }`	Chọn phần tử là con đầu tiên của phần tử cha.
`:last-child`	Tất cả phần tử	`li:last-child { color: blue; }`	Chọn phần tử là con cuối cùng của phần tử cha.
`:nth-child(n)`	Tất cả phần tử	`li:nth-child(2) { color: purple; }`	Chọn phần tử con thứ n (truyền số, odd, even, 2n+1).
`:nth-last-child(n)`	Tất cả phần tử	`li:nth-last-child(1) { color: teal; }`	Chọn phần tử con thứ n đếm ngược từ dưới lên.
`:first-of-type`	Tất cả phần tử	`p:first-of-type { color: green; }`	Chọn phần tử đầu tiên trong số các phần tử cùng loại thẻ.
`:last-of-type`	Tất cả phần tử	`p:last-of-type { color: orange; }`	Chọn phần tử cuối cùng trong số các phần tử cùng loại thẻ.
`:nth-of-type(n)`	Tất cả phần tử	`p:nth-of-type(1) { font-weight: bold; }`	Chọn phần tử thứ n trong số các phần tử cùng loại thẻ.
`:nth-last-of-type(n)`	Tất cả phần tử	`p:nth-last-of-type(1) { color: brown; }`	Chọn phần tử thứ n đếm ngược từ dưới lên trong số các phần tử cùng loại thẻ.
`:only-child`	Tất cả phần tử	`div:only-child { border: 1px solid black; }`	Chọn phần tử nếu nó là con độc nhất của thẻ cha.
`:only-of-type`	Tất cả phần tử	`p:only-of-type { font-style: italic; }`	Chọn phần tử nếu nó là con duy nhất thuộc loại thẻ đó trong thẻ cha.
`:empty`	Tất cả phần tử	`div:empty { display: none; }`	Chọn phần tử hoàn toàn rỗng (không con, không chữ).
`:root`	Thẻ gốc Document	`:root { --main-color: red; }`	Chọn phần tử gốc của cây DOM (thường là thẻ html).

a. Mã nguồn ví dụ cấu trúc DOM đính kèm kết quả visual:

Mã CSS Định Kiểu Danh Sách Theo Vị Trí Vị Thế DOM

```css
/* Sample Text */
li:first-child {
    color: green;
    font-weight: bold;
}

/* Sample Text */
li:nth-child(2) {
    color: red;
}
```

BROWSER PREVIEW Kết Quả Hiển Thị: Danh Sách Được Định Kiểu Theo Vị Trí

- Mục 1 (li:first-child – Màu xanh lá)
- Mục 2 (li:nth-child(2) – Màu đỏ)
- Mục 3 (Mặc định)

b. Tổng Quan & Bản Chất Cốt Lõi Của Pseudo-Classes Cấu Trúc

[Khái Niệm] Định Nghĩa & Bản Chất Của Pseudo-Classes Cấu Trúc

Structural Pseudo-Classes (Các lớp giả cấu trúc) là nhóm bộ chọn đặc biệt trong CSS3 cho phép bạn chọn và trang trí các phần tử HTML dựa hoàn toàn vào **vị trí tương đối của chúng trong cây DOM** (Document Object Model) mà không cần phải chèn thêm bất kỳ class hay id thủ công nào vào mã HTML.

Ẩn dụ tư duy dễ nhớ: Hãy hình dung cây DOM HTML như một danh sách học sinh đang xếp hàng. Thay vì phải dán nhãn tên từng người (gán class), bạn có thể ra lệnh cho CSS: *"Tô màu đỏ cho học sinh đứng đầu hàng"*, *"Tô màu xanh cho học sinh đứng cuối hàng"*, hoặc *"Tô màu vàng cho các học sinh ở vị trí số lẻ (1, 3, 5...)"*. Đó chính là sức mạnh của Pseudo-Classes Cấu trúc!

3 Lợi ích cốt lõi trong phát triển Web hiện đại:

- **Giữ mã HTML sạch tuyệt đối**: Tránh việc phình to mã nguồn HTML do gán hàng loạt class lặp lại (ví dụ dán `class="first-item"`, `class="even-row"` vào từng thẻ).
- **Tự động hóa giao diện động**: Khi dữ liệu gửi từ Server/API thay đổi số lượng dòng (thêm/bớt sản phẩm), giao diện vẫn tự động áp dụng đúng quy luật mà không cần can thiệp CSS.
- **Tối ưu bảo trì mã nguồn**: Thay đổi quy luật thiết kế giao diện tại một nơi duy nhất trong file CSS.

[Bảng] Bảng Phân Loại 10 Structural Pseudo-Classes Theo Cơ Chế Hoạt Động

Bộ chọn CSS	Nguyên lý chọn phần tử trong DOM
1. Nhóm Vị Trí Tuyệt Đối (Đếm tất cả các phần tử con bất kể loại thẻ)	
`:first-child`	Chọn phần tử nếu nó là con đầu tiên tuyệt đối của cha.
`:last-child`	Chọn phần tử nếu nó là con cuối cùng tuyệt đối của cha.
`:nth-child(n)`	Chọn phần tử đứng ở vị trí thứ n từ trên xuống.
`:nth-last-child(n)`	Chọn phần tử đứng ở vị trí thứ n đếm từ dưới lên.
`:only-child`	Chọn phần tử nếu nó là con duy nhất tuyệt đối trong cha.
2. Nhóm Lọc Theo Kiểu Thẻ (Chỉ lọc và đếm các phần tử cùng kiểu thẻ)	
`:first-of-type`	Chọn phần tử cùng kiểu thẻ đầu tiên trong cha.
`:last-of-type`	Chọn phần tử cùng kiểu thẻ cuối cùng trong cha.
`:nth-of-type(n)`	Chọn phần tử cùng kiểu thẻ thứ n từ trên xuống.
`:nth-last-of-type(n)`	Chọn phần tử cùng kiểu thẻ thứ n đếm từ dưới lên.
`:only-of-type`	Chọn phần tử nếu nó là thẻ duy nhất thuộc loại đó trong cha.

c. Chuyên Đề 1: Bộ Chọn Phần Tử Con Đầu Tiên (:first-child vs :first-of-type)

Trong phát triển giao diện Web, việc chọn phần tử con xuất hiện đầu tiên trong một khối container (ví dụ: tạo kiểu làm nổi bật tab "Trang chủ" ở menu, tạo chữ hoa lớn cho đoạn văn đầu bài viết **Drop Cap**, hoặc xóa margin-top thừa của phần tử đỉnh) là một thao tác căn bản. CSS cung cấp hai pseudo-class selector cho mục đích này: `:first-child` và `:first-of-type`. Phần này sẽ phân tích chi tiết và tách biệt hai bộ chọn.

[Khái Niệm] Tổng Quan Nhanh: 2 Cơ Chế Chọn Phần Tử Đỉnh

- `:first-child`: Chọn phần tử nếu nó là **con đầu tiên tuyệt đối** (ở vị trí số 1 trong danh sách tất cả các con của cha).
- `:first-of-type`: Chọn phần tử nếu nó là **con đầu tiên thuộc loại thẻ đó** trong phần tử cha (bỏ qua các thẻ khác loại đứng phía trước).

c1. Phân Tích Chuyên Sâu :first-child (Con Đầu Tiên Tuyệt Đối) `:first-child` là một pseudo-class selector trong CSS dùng để chọn phần tử con đứng ở vị trí đầu tiên tuyệt đối (vị trí $index = 1$) trong danh sách tất cả phần tử con của cha.

[Khái Niệm] Định Nghĩa Chi Tiết & Cơ Chế Hoạt Động Của :first-child

`:first-child` chọn phần tử nếu nó nằm ở vị trí con đầu tiên tuyệt đối trong phần tử cha:

- Phần tử con đứng đầu tiên trong cây DOM có vị trí $index = 1$.
- Trình duyệt đếm tất cả các loại thẻ con (element nodes).
- Nếu phần tử ở vị trí 1 không khớp với thẻ chỉ định trong CSS → **Bỏ qua, không tô màu!**

Tương đương logic: `selector:first-child` ≡ `selector:nth-child(1)`.

Cú Pháp Cơ Bản Của :first-child

```
selector:first-child {
    /* CSS properties apply here */
}

/* Equivalent to: */
selector:nth-child(1) {
    /* Same result */
}
```

Ví Dụ 1: Chọn Thẻ <li> Đầu Tiên Trong Danh Sách

```
<ul>
    <li>Item 1</li> <!-- 1st child of <ul> -->
    <li>Item 2</li>
    <li>Item 3</li>
</ul>

<style>
    /* Select the 1st <li> element inside list */
    li:first-child {
        color: red;
        font-weight: bold;
    }
</style>
```

BROWSER PREVIEW — Kết Quả Hiển Thị Trực Quan Ví Dụ 1

- **Item 1** (← li:first-child khớp – Chữ màu đỏ, in đậm vì là phần tử con đầu tiên)
- Item 2 (Mặc định)
- Item 3 (Mặc định)

(Giải thích: li:first-child chọn phần tử <li> đầu tiên trong danh sách. Kết quả "Item 1" sẽ có màu đỏ và chữ in đậm. Nếu không áp dụng :first-child thì tất cả các thẻ <li> nằm trong <ul> đều bị ảnh hưởng giống nhau.)

Ví Dụ 2: Chọn Phần Tử Con Đầu Tiên Trong Container Cụ Thể

```html
<div class="container">
    <p>Paragraph 1</p> <!-- 1st child of .container -->
    <p>Paragraph 2</p>
</div>

<style>
    /* Select 1st <p> child inside .container */
    .container p:first-child {
        color: blue;
        font-weight: bold;
    }
</style>
```

BROWSER PREVIEW — **Kết Quả Hiển Thị Ví Dụ 2: Container Cụ Thể**

Paragraph 1 (← p:first-child khớp – Chữ xanh dương, in đậm)

Paragraph 2 (Mặc định)

(Giải thích: Chọn <p> đầu tiên trực tiếp trong .container và đổi màu chữ thành xanh dương. Chỉ áp dụng cho phần tử con đầu tiên, không ảnh hưởng các phần tử khác.)

Ví Dụ 3: Phối Hợp :first-child Với Pseudo-Class Hover

```css
/* Highlight 1st item on mouse hover */
li:first-child:hover {
    color: purple;
    text-decoration: underline;
    cursor: pointer;
}
```

[Mẹo] Phối Hợp :first-child Với Hover

[Hiệu Ứng]: Phần tử đầu tiên trong danh sách khi người dùng rê chuột (hover) vào sẽ tự động chuyển sang màu tím, xuất hiện đường gạch chân và đổi con trỏ chuột dạng pointer.

Ví Dụ 4: Tình Huống Thất Bại Phổ Biến Của :first-child

```html
<!-- Mixed container: <h1> is at position 1 -->
<div class="card">
    <h1>Header Title</h1> <!-- 1st child of .card -->
    <p>First paragraph text</p>
</div>

<style>
    /* FAILS: 1st child is <h1>, NOT <p> */
    .card p:first-child { color: green; }
</style>
```

[Cảnh Báo] Phân Tích Chi Tiết: Tại Sao :first-child Thất Bại & Thuật Toán 2 Bước Của Trình Duyệt

Cơ chế duyệt 2 bước của Trình duyệt khi xử lý `p:first-child`:

1. **Bước 1 (Đếm vị trí):** Trình duyệt tìm phần tử con nằm ở vị trí số 1 tuyệt đối bên trong phần tử cha.
2. **Bước 2 (Kiểm tra thẻ):** Trình duyệt kiểm tra xem phần tử ở vị trí số 1 đó có đúng là thẻ `<p>` hay không.

Tại sao `p:first-child` thất bại trong DOM hỗn hợp?

- Nếu vị trí số 1 là thẻ `<h1>` (Tiêu đề), còn thẻ `<p>` nằm ở vị trí số 2 → Trình duyệt thấy vị trí số 1 là `<h1>` chứ không phải `<p>` → **Điều kiện SAI! Bỏ qua, không tô màu đoạn văn nào!**

Giải pháp hoàn hảo – Sử dụng `p:first-of-type`:

- `p:first-of-type` thay đổi quy trình: Trình duyệt sẽ **lọc tất cả các thẻ `<p>` thành một nhóm riêng trước**, sau đó chọn thẻ `<p>` đầu tiên trong nhóm đó (bất chấp phía trước có bao nhiêu thẻ `<h1>` hay `<div>`) → **Tô màu xanh thành công 100%!**

[Khái Niệm] 4 Ứng Dụng Thực Tế Của :first-child

- **Tạo kiểu đặc biệt cho phần tử đầu tiên**: Làm nổi bật tiêu đề đầu tiên hoặc mục đầu tiên trong danh sách.
- **Thiết kế Menu điều hướng**: Nhấn mạnh tab đầu tiên (ví dụ nút "Trang chủ") hoặc thêm khoảng cách/màu sắc đặc biệt.
- **Căn chỉnh Bố cục trang**: Áp dụng padding/margin riêng cho phần tử đầu tiên để căn chỉnh layout cân đối, không bị dính lề trên.
- **Danh sách sản phẩm & Bài viết**: Đánh dấu sản phẩm đầu tiên hoặc bài viết mới nhất trong danh sách.

c2. Phân Tích Chuyên Sâu :first-of-type (Con Đầu Tiên Theo Loại Thẻ) `:first-of-type` là pseudo-class selector chọn phần tử con là **phần tử đầu tiên thuộc loại thẻ đó** (cùng tag name) trong phần tử cha, bất kể phía trước nó có bao nhiêu thẻ thuộc loại khác.

[Khái Niệm] Định Nghĩa Chi Tiết & Cú Pháp :first-of-type

`:first-of-type` chọn phần tử đầu tiên thuộc loại (thẻ) đó trong các phần tử con của một phần tử cha.

- Trình duyệt gom tất cả các thẻ cùng tag name (ví dụ tất cả các thẻ `<p>`).
- Chọn phần tử xuất hiện **đầu tiên** trong nhóm phần tử cùng loại đó.
- Các thẻ thuộc loại khác đứng phía trước hoàn toàn **bị bỏ qua** và không ảnh hưởng đến kết quả.

Khác với `:first-child`:

- `:first-child`: Chọn phần tử đầu tiên bất kể loại nào.
- `:first-of-type`: Chọn phần tử đầu tiên thuộc loại đó (thẻ giống nhau).

Tương đương logic: `selector:first-of-type` ≡ `selector:nth-of-type(1)`.

Cú Pháp Cơ Bản Của :first-of-type

```
selector:first-of-type {
    /* CSS properties apply here */
}

/* Equivalent to: */
selector:nth-of-type(1) {
    /* Same result */
}
```

Ví Dụ 1: Mã Nguồn Chuẩn Chọn Thẻ <p> Đầu Tiên Trong .box

```
<div class="box">
  <h2>Tieu de</h2>
  <p>Doan 1</p> <!-- First <p> in .box -->
  <p>Doan 2</p>
</div>

<style>
  p:first-of-type {
    color: red;
  }
</style>
```

BROWSER PREVIEW — **Kết Quả Hiển Thị Ví Dụ 1: .box p:first-of-type**

Tieu de (Thẻ <h2> đứng trước – Bị bỏ qua)
Doan 1 (← p:first-of-type KHỚP – Tô màu đỏ vì là đầu tiên trong các thẻ con của .box)
Doan 2 (Thẻ thứ 2 – Mặc định)

Ví Dụ 2: Trường Hợp Có Thẻ Chen Giữa Phía Trước

```
<div class="box">
  <h2>Tieu de</h2>
  <span>span</span> <!-- Sibling <span> before <p> -->
  <p>Doan 1</p> <!-- Still first-of-type among <p> tags -->
  <p>Doan 2</p>
</div>

<style>
  p:first-of-type {
    color: red;
  }
</style>
```

BROWSER PREVIEW — **Kết Quả Hiển Thị Ví Dụ 2: Thẻ Chen Giữa**

Tieu de (Thẻ <h2>)
span (Thẻ)
Doan 1 (← p:first-of-type VẪN KHỚP – Chọn được Doan 1 vì là thẻ đầu tiên, dù không phải con đầu tiên của .box)
Doan 2 (Thẻ thứ 2)

Ví Dụ 3: Mỗi Loại Thẻ Đều Có :first-of-type Riêng (Chọn Thẻ Đầu Tiên)

```
<div>
  <h2>Tieu de</h2>
  <p>Doan van</p>
  <span>Span 1</span> <!-- 1st <span> in <div> -->
  <p>Doan van khac</p>
</div>

<style>
  span:first-of-type {
    color: blue;
    font-weight: bold;
  }
</style>
```

BROWSER PREVIEW — **Kết Quả Hiển Thị Ví Dụ 3: span:first-of-type**

Tieu de (Thẻ <h2>)
Doan van (Thẻ <p> thứ 1)
Span 1 (← span:first-of-type KHỚP – Chỉ tác động đến span đầu tiên (trong số các span) trong div)
Doan van khac (Thẻ <p> thứ 2)

[Cảnh Báo] 5 Lưu Ý Cốt Lõi Khi Sử Dụng :first-of-type

1. **1. Là phần tử đầu tiên cùng loại trong cha:** Chỉ áp dụng nếu phần tử đó là loại đầu tiên (cùng tên thẻ: `p`, `li`, `div`...) trong danh sách phần tử con.
2. **2. Không cần phải là phần tử con đầu tiên tuyệt đối:** Dù có thẻ `<span>` hay `<h2>` đứng trước, `p:first-of-type` vẫn hoạt động thành công.
3. **3. Không hoạt động cho phần tử cùng loại thứ 2 trở đi:** Khi có `<p>Doan 1</p>` và `<p>Doan 2</p>`, chỉ `<p>Doan 1</p>` được tô đỏ, `<p>Doan 2</p>` bị bỏ qua.
4. **4. Khác biệt cốt lõi với :first-child:** `:first-child` yêu cầu là phần tử đầu tiên tuyệt đối của cha (bất kể loại gì); `:first-of-type` yêu cầu là phần tử đầu tiên theo loại (thẻ) của cha.
5. **5. Thường dùng trong danh sách nhiều loại phần tử con:** Giúp định kiểu độc lập từng loại thẻ trong giao diện hỗn hợp.

[Bảng] Bảng Kiểm Tra Tính Đúng / Sai Của Các Lưu Ý Về :first-of-type

Phát biểu lưu ý	Đúng / Sai	Ghi chú ngắn
Chọn phần tử đầu tiên theo loại	Đúng	Đúng loại mới được áp dụng
Bị chặn nếu có anh em cùng loại đứng trước	Đúng	Chỉ 1 phần tử đầu tiên được áp dụng
Không quan tâm đến vị trí tổng thể trong DOM	Đúng	Miễn là loại đầu tiên trong nhóm
Là phần tử đầu tiên bất kể loại thẻ?	Sai	Đó là `:first-child`, không phải `:first-of-type`

[Khái Niệm] 4 Ứng Dụng Thực Tế Của :first-of-type

- **Đoạn văn mở đầu bài viết (Lead Paragraph / Drop Cap)**: Định dạng cỡ chữ lớn hoặc chữ hoa hoa mỹ cho đoạn văn đầu tiên trong bài blog.
- **Form ô nhập liệu**: Căn chỉnh lề trên (`margin-top: 0`) cho ô `<input>` đầu tiên trong Form.
- **Ảnh đại diện bài viết**: Tự động định dạng kích thước hoặc border cho hình ảnh `<img>` đầu tiên.
- **Hero section**: Làm nổi bật thẻ tiêu đề `<h1>` xuất hiện đầu tiên trong trang.

c3. Bảng So Sánh Chi Tiết :first-child vs :first-of-type

Ví Dụ Trực Quan So Sánh :first-child vs :first-of-type

```
<div class="demo">
    <h1>Title</h1>
    <p>Paragraph 1</p>
    <p>Paragraph 2</p>
</div>

<style>
    /* FAILS: 1st child of .demo is <h1>, not <p> */
    .demo p:first-child { color: red; }

    /* WORKS: Selects 1st <p> (Paragraph 1), ignoring <h1> */
    .demo p:first-of-type { color: green; font-weight: bold; }
</style>
```

BROWSER PREVIEW — **Kết Quả So Sánh Trong Trình Duyệt**

Title (Thẻ <h1> – Con đầu tiên thực tế)

Paragraph 1 (← p:first-of-type KHỚP – Tô xanh lá vì là <p> đầu tiên)

Paragraph 2 (Thẻ <p> thứ 2)

Lưu ý: p:first-child KHÔNG khớp vì con đầu tiên tuyệt đối là <h1>, không phải <p>.

[Bảng] Bảng So Sánh Chi Tiết Cụ Thể: :first-child vs :first-of-type

Pseudo-class	Quan tâm đến vị trí?	Quan tâm đến loại thẻ?
:first-child	**Có** (Phải là con đầu tiên tuyệt đối của cha)	**Có** (Phải trùng khớp với thẻ chỉ định)
:first-of-type	**Không** (Không cần đứng đầu danh sách con)	**Có** (Là phần tử cùng loại đầu tiên trong cha)

[Bảng] Bảng So Sánh Toàn Diện :first-child vs :first-of-type

Tiêu chí so sánh	:first-child	:first-of-type
Cơ chế đếm	Kiểm tra phần tử đứng ở **vị trí số 1 tuyệt đối**	Lọc riêng nhóm phần tử **cùng loại thẻ** và chọn phần tử đầu
Ảnh hưởng bởi thẻ khác loại?	**Có** (Bất kỳ thẻ khác loại nào đứng ở vị trí 1 đều làm thất bại)	**Không** (Thẻ khác loại đứng phía trước bị bỏ qua)
Tương đương logic	`:nth-child(1)`	`:nth-of-type(1)`
Đối ứng vị trí đáy	`:last-child`	`:last-of-type`
Mức độ nghiêm ngặt	Nghiêm ngặt (Cha phải chỉ có thẻ này ở vị trí đỉnh)	Linh hoạt (Phù hợp với container chứa nhiều thẻ hỗn hợp)
Khi nào nên dùng?	Khi danh sách thuần nhất (ví dụ toàn `<li>`)	Khi container có cấu trúc hỗn hợp nhiều loại thẻ

[Mẹo] Mẹo Ghi Nhớ Nhanh Phân Biệt :first-child vs :first-of-type

Bộ chọn	Vị trí đếm	Quan tâm loại thẻ?	Dùng khi nào?
:first-child	Đầu tiên trong mọi con	**Không**	Style phần tử đầu tiên bất kỳ
:first-of-type	Đầu tiên trong cùng loại	**Có**	Style phần tử đầu tiên của loại thẻ cụ thể

d. Chuyên Đề 2: Bộ Chọn Phần Tử Con Cuối Cùng (:last-child vs :last-of-type)

Trong thiết kế giao diện Web, việc chọn phần tử con cuối cùng trong một khối container (ví dụ: xóa đường gạch kẻ dưới của mục menu cuối cùng, căn chỉnh khoảng cách cho nút bấm cuối form, hoặc định dạng đoạn văn cuối bài) là nhu cầu diễn ra vô cùng thường xuyên. CSS cung cấp hai pseudo-class selector cho mục đích này: `:last-child` và `:last-of-type`. Phần này sẽ phân tích chi tiết và tách biệt hai bộ chọn.

[Khái Niệm] Tổng Quan Nhanh: 2 Cơ Chế Chọn Phần Tử Đáy

- `:last-child`: Chọn phần tử nếu nó là **con cuối cùng tuyệt đối** (ở vị trí cuối cùng trong danh sách tất cả các con của cha).
- `:last-of-type`: Chọn phần tử nếu nó là **con cuối cùng thuộc loại thẻ đó** trong phần tử cha (bỏ qua các thẻ khác loại đứng phía sau).

d1. Phân Tích Chuyên Sâu :last-child (Con Cuối Cùng Tuyệt Đối) `:last-child` là một pseudo-class selector trong CSS dùng để chọn phần tử con đứng ở vị trí cuối cùng tuyệt đối trong danh sách tất cả phần tử con của cha.

[Khái Niệm] Định Nghĩa Chi Tiết & Cơ Chế Hoạt Động Của :last-child

`:last-child` khớp với một phần tử khi và chỉ khi hai điều kiện sau được thỏa mãn đồng thời:

- Phần tử nằm ở **vị trí cuối cùng tuyệt đối** trong danh sách các phần tử con của cha.
- Phần tử đó khớp với bộ chọn `selector` phía trước.
- **Lưu ý quan trọng:** Trình duyệt kiểm tra vị trí cuối cùng trong DOM trước. Nếu phần tử đứng cuối cùng là một loại thẻ khác với `selector` → **Bỏ qua, không áp dụng CSS!**

Tương đương logic: `selector:last-child` ≡ `selector:nth-last-child(1)`.

Cú Pháp Cơ Bản Của :last-child

```
selector:last-child {
    /* CSS properties apply here */
}

/* Equivalent to: */
selector:nth-last-child(1) {
    /* Same result */
}
```

Ví Dụ 1: Chọn Thẻ <li> Cuối Cùng Trong Danh Sách

```
<ul>
    <li>Item 1</li>
    <li>Item 2</li>
    <li>Item 3</li> <!-- Last child of <ul> -->
</ul>

<style>
    /* Select the last <li> element inside list */
    li:last-child {
        color: blue;
        font-weight: bold;
    }
</style>
```

BROWSER PREVIEW — Kết Quả Hiển Thị Trực Quan Ví Dụ 1

- Item 1 (Mặc định)
- Item 2 (Mặc định)
- **Item 3** (← li:last-child khớp – Chữ xanh, in đậm vì là phần tử cuối cùng trong <ul>)

Ví Dụ 2: Chọn Đoạn Văn Cuối Cùng Trong Container Cụ Thể

```
<div class="container">
    <p>Paragraph 1</p>
    <p>Paragraph 2</p> <!-- Last child of .container -->
</div>

<style>
    /* Select last <p> child inside .container */
    .container p:last-child {
        color: red;
        font-weight: bold;
    }
</style>
```

BROWSER PREVIEW — Kết Quả Hiển Thị Ví Dụ 2

Paragraph 1 (Mặc định)

Paragraph 2 (← p:last-child khớp – Chữ đỏ vì là <p> đứng cuối cùng trong .container)

Ví Dụ 3: Phối Hợp :last-child Với Pseudo-Class Hover

```css
/* Highlight last item on mouse hover */
li:last-child:hover {
    color: purple;
    text-decoration: underline;
    cursor: pointer;
}
```

[Mẹo] Phối Hợp :last-child Với Hover

[Hiệu Ứng]: Phần tử cuối cùng trong danh sách khi di chuột vào (hover) sẽ tự động chuyển sang màu tím, xuất hiện đường gạch chân và đổi con trỏ chuột dạng pointer.

Ví Dụ 4: Tình Huống Thất Bại Phổ Biến Của :last-child

```html
<!-- Mixed container: <span> is the actual last child -->
<div class="card">
    <p>First paragraph text</p>
    <span>Span element text</span> <!-- Actual last child -->
</div>

<style>
    /* FAILS: Last child of .card is <span>, NOT <p> */
    .card p:last-child {
        color: red;
    }
</style>
```

BROWSER PREVIEW **Kết Quả Ví Dụ 4: :last-child Thất Bại Trong DOM Hỗn Hợp**

First paragraph text (KHÔNG đỏ – Thẻ <p> không phải là con cuối cùng trong .card)

Span element text (Thẻ – Con cuối cùng thực tế trong .card)

Giải thích: .card p:last-child kiểm tra con cuối cùng trong .card (là). Vì ≠ <p> nên không có thẻ <p> nào được áp dụng CSS.

[Cảnh Báo] Các Lưu Ý Quan Trọng Khi Sử Dụng :last-child

1. Phải là con cuối cùng tuyệt đối: Nếu đứng sau thẻ bạn chọn còn bất kỳ thẻ anh em nào khác (dù khác loại), `:last-child` sẽ thất bại.

2. Text node không bị đếm: Khoảng trắng, xuống dòng trong HTML không bị tính là phần tử con đứng cuối.

3. Giải pháp khi có thẻ khác loại đứng sau: Chuyển sang sử dụng `:last-of-type`.

[Mẹo] Mẹo Ghi Nhớ :last-child

Hình dung: `:last-child` giống như "người xếp hàng ở vị trí cuối cùng tuyệt đối". Nếu người đứng cuối cùng không mặc áo đỏ (không phải loại thẻ cần tìm), bộ chọn `p:last-child` sẽ không chọn được ai.

[Khái Niệm] 5 Ứng Dụng Thực Tế Của :last-child

1. **Tạo viền / Cách điệu phần tử cuối**: Bỏ đường viền dưới (`border-bottom: none`) hoặc đường gạch kẻ ngang ở mục cuối cùng trong danh sách.
2. **Điều hướng Menu**: Nhấn mạnh tab cuối cùng hoặc thêm biểu tượng đặc biệt (nút CTA) cho menu navigation.
3. **Bố cục giao diện**: Điều chỉnh margin/padding dưới (`margin-bottom: 0`) của phần tử cuối để tránh làm phình lề dưới của container.
4. **Danh sách sản phẩm & Footer**: Làm nổi bật phần tử sản phẩm cuối hoặc tạo kiểu đặc biệt cho liên kết cuối trong Footer.
5. **Form nhập liệu**: Điều chỉnh khoảng cách hoặc căn chỉnh nút Submit ở cuối biểu mẫu.

d2. Phân Tích Chuyên Sâu :last-of-type (Con Cuối Cùng Theo Loại Thẻ) `:last-of-type` là pseudo-class selector dùng để chọn phần tử con cuối cùng của một loại (tag name) trong một phần tử cha.

[Khái Niệm] Định Nghĩa Chi Tiết & Cơ Chế Hoạt Động Của :last-of-type

`:last-of-type` lọc và hoạt động theo các bước:

- Trình duyệt lọc riêng danh sách tất cả phần tử con cùng loại thẻ (tag name) trong cha.
- Chọn phần tử xuất hiện **cuối cùng** trong nhóm phần tử cùng loại đó.
- Các phần tử khác loại thẻ đứng phía sau hoàn toàn **bị bỏ qua** và không ảnh hưởng đến kết quả.

So sánh với `:last-child`:

- `:last-child`: Phần tử con cuối cùng trong tất cả các con (Không quan tâm loại thẻ).
- `:last-of-type`: Phần tử con cuối cùng thuộc một loại (thẻ) (Có quan tâm loại thẻ).

Tương đương logic: `selector:last-of-type` ≡ `selector:nth-last-of-type(1)`.

Cú Pháp Cơ Bản Của :last-of-type

```
selector:last-of-type {
    /* style apply */
}

/* Equivalent to: */
selector:nth-last-of-type(1) {
    /* Same result */
}
```

Ví Dụ 1: Mã Nguồn Cơ Bản Với Thẻ Đứng Sau

```
<div>
  <h2>Tieu de</h2>
  <p>Doan 1</p>
  <p>Doan 2</p> <!-- Applied: Last <p> tag -->
  <span>Chu thich</span>
</div>

<style>
  p:last-of-type {
    color: red;
  }
</style>
```

BROWSER PREVIEW — Kết Quả Hiển Thị Ví Dụ 1: p:last-of-type

Tieu de (Thẻ <h2>)

Doan 1 (Thẻ <p> thứ 1)

Doan 2 (← p:last-of-type KHỚP – Doan 2 bị tô đỏ vì nó là phần tử <p> cuối cùng trong cha div)

Chu thich (Thẻ đứng sau – Bị bỏ qua)

Ví Dụ 2: Ví Dụ Phân Biệt Chi Tiết Với :last-child

```html
<div>
  <p>Doan 1</p>
  <p>Doan 2</p>
  <span>Ghi chu</span>
</div>

<style>
  /* FAILS: p is not the absolute last child */
  p:last-child { color: red; }

  /* SUCCEEDS: Last <p> in <p> group */
  p:last-of-type { color: green; font-weight: bold; }
</style>
```

BROWSER PREVIEW — Kết Quả Hiển Thị Ví Dụ 2: Phân Biệt Với :last-child

Doan 1 (Thẻ <p> thứ 1)

Doan 2 (← p:last-of-type KHỚP – p:last-child THẤT BẠI vì p không phải con cuối cùng, còn p:last-of-type THÀNH CÔNG vì p là cuối cùng trong nhóm p)

Ghi chu (Thẻ – Con cuối cùng thực tế)

[Cảnh Báo] Các Lưu Ý Quan Trọng Khi Sử Dụng :last-of-type

1. Áp dụng cho thẻ cùng loại: Không cần là phần tử con cuối cùng tuyệt đối trong container.

2. Trường hợp phần tử đơn lẻ: Nếu container chỉ có đúng 1 phần tử thuộc loại thẻ đó, `:last-of-type` sẽ chọn đúng phần tử đó (trùng kết quả với `:first-of-type` và `:only-of-type`).

[Mẹo] Mẹo Ghi Nhớ :last-of-type

Hình dung: `:last-of-type` giống như ”người cuối cùng của từng nhóm”. Nếu có nhóm nam và nhóm nữ, `:last-of-type` chọn người đứng cuối của nhóm nam và người đứng cuối của nhóm nữ.

[Khái Niệm] 4 Ứng Dụng Thực Tế Của :last-of-type

- **Đoạn văn bài viết**: Xóa `margin-bottom` cho đoạn văn `<p>` cuối bài viết blog để lề bottom ôm sát wrapper.
- **Chữ ký / Nguồn tác giả**: Định dạng kiểu chữ nghiêng hoặc chèn đường kẻ phía trên thẻ `<p>` cuối cùng.
- **Form ô nhập liệu**: Reset khoảng cách lề cho ô `<input>` cuối cùng trước nút bấm.
- **Gallery ảnh bài viết**: Bo góc tròn hoặc thêm border đặc biệt cho hình ảnh `<img>` cuối cùng.

d3. Bảng So Sánh Chi Tiết :last-child vs :last-of-type

[Bảng] Bảng So Sánh Cốt Lõi: :last-child vs :last-of-type

Selector	Ý nghĩa	Quan tâm đến loại thẻ?
:last-child	Phần tử con cuối cùng trong tất cả các con	Không
:last-of-type	Phần tử con cuối cùng thuộc một loại (thẻ)	Có

[Bảng] Bảng Tổng Kết Quy Tắc Tra Cứu :last-child & :last-of-type

Thuộc tính	Vị trí	Quan tâm loại?	Ví dụ dùng
:last-child	Con cuối cùng bất kỳ	Không	Kiểm tra phần tử kết thúc tuyệt đối
:last-of-type	Con cuối cùng trong nhóm cùng loại	Có	Kiểm tra phần tử cuối cùng của một loại thẻ cụ thể

[Bảng] Bảng So Sánh Toàn Diện :last-child vs :last-of-type

Tiêu chí so sánh	:last-child	:last-of-type
Cơ chế đếm	Kiểm tra phần tử đứng ở **vị trí cuối cùng tuyệt đối**	Lọc riêng nhóm phần tử **cùng loại thẻ** và chọn phần tử cuối
Ảnh hưởng bởi thẻ khác loại?	**Có** (Bất kỳ thẻ khác loại nào đứng cuối đều làm thất bại)	**Không** (Thẻ khác loại đứng phía sau bị bỏ qua)
Tương đương logic	`:nth-last-child(1)`	`:nth-last-of-type(1)`
Đối ứng vị trí đầu	`:first-child`	`:first-of-type`
Mức độ nghiêm ngặt	Nghiêm ngặt (Cha phải chỉ có thẻ này ở vị trí đáy)	Linh hoạt (Phù hợp với container chứa nhiều thẻ hỗn hợp)
Khi nào nên dùng?	Khi danh sách thuần nhất (ví dụ toàn `<li>`)	Khi container có cấu trúc hỗn hợp nhiều loại thẻ

[Mẹo] Tóm Kết 4 Bộ Chọn Đầu & Cuối Cơ Bản

- `:first-child`: Con đầu tiên tuyệt đối.
- `:first-of-type`: Con đầu tiên theo loại thẻ.
- `:last-child`: Con cuối cùng tuyệt đối.
- `:last-of-type`: Con cuối cùng theo loại thẻ.

e. Chuyên Đề 3: Bộ Chọn Vị Trí Vạn Năng (:nth-child(n) vs :nth-of-type(n))

`:nth-child(n)` và `:nth-of-type(n)` là hai pseudo-class selector mạnh mẽ nhất trong CSS3 để chọn các phần tử dựa trên vị trí thứ tự của chúng trong danh sách con của cha. Chúng cho phép lập trình viên định kiểu theo chỉ số cố định, theo chu kỳ lặp số học, hoặc theo vị trí chẵn/lẻ. Phần này sẽ phân tích tách biệt và chuyên sâu từng bộ chọn.

[Khái Niệm] Tổng Quan Nhanh: 2 Cơ Chế Đếm Vị Trí Từ Trên Xuống

- `:nth-child(an + b)`: Đếm vị trí **từ trên xuống dưới** (từ con đầu tiên = 1), đếm **tất cả phần tử con** không phân biệt loại thẻ.
- `:nth-of-type(an + b)`: Đếm vị trí **từ trên xuống dưới**, nhưng **chỉ đếm các phần tử cùng loại thẻ** (cùng tag name).

e1. Phân Tích Chuyên Sâu :nth-child(an + b) (Đếm Vị Trí Tuyệt Đối Trong DOM) `:nth-child(n)` là một pseudo-class selector dùng để chọn phần tử con dựa trên vị trí thứ tự tuyệt đối của nó trong danh sách tất cả các phần tử con của cha (đếm từ trên xuống).

[Khái Niệm] Định Nghĩa Chi Tiết & Cơ Chế Đếm Của :nth-child(n)

`:nth-child(an + b)` chọn phần tử con theo thứ tự chỉ định n hoặc công thức đại số $an + b$:

- Phần tử con đầu tiên trong phần tử cha có chỉ số $index = 1$.
- Trình duyệt đếm **tất cả phần tử con** (element nodes) mà không phân biệt loại thẻ (`<div>`, `<p>`, `<span>`, `<h2>`...).
- **Lưu ý cốt lõi:** `:nth-child(an + b)` **không chọn dựa trên loại thẻ**, mà chọn theo vị trí thứ tự tuyệt đối của phần tử con trong cây DOM.
- Nếu một phần tử ở đúng vị trí tính ra từ công thức $an + b$ và khớp với thẻ được chỉ định, nó sẽ bị áp dụng CSS. Ngược lại, dù cùng loại thẻ nhưng sai vị trí DOM $\rightarrow$ không bị ảnh hưởng.

Cú Pháp Cơ Bản Của :nth-child(n)

```
/* Standard nth-child syntax */
selector:nth-child(n) {
    /* CSS properties apply here */
}

/* Formula syntax */
selector:nth-child(an + b) {
    /* CSS properties apply here */
}
```

[Khái Niệm] Giải Thích Thành Phần Cú Pháp :nth-child(an + b)

- `selector`: Là phần tử bạn muốn chọn (ví dụ `li`, `p`, `tr`).
- `a`: **Bước nhảy** (khoảng cách giữa các phần tử sau mỗi lần lặp).
- `n`: **Biến lặp index** (Số nguyên không âm $n = 0, 1, 2, 3, 4, \ldots$).
- `b`: **Vị trí bắt đầu** (Offset ban đầu).

Ví Dụ 1: Minh Họa Vị Trí Đếm DOM Của :nth-child(3)

```html
<div class="container">
  <h2>Header Title</h2>   <!-- Child position 1 -->
  <p>Paragraph 1</p>      <!-- Child position 2 -->
  <p>Paragraph 2</p>      <!-- Child position 3 -->
  <span>Span Note</span>   <!-- Child position 4 -->
  <p>Paragraph 3</p>      <!-- Child position 5 -->
</div>

<style>
  /* Select 3rd child IF it is a <p> tag */
  p:nth-child(3) {
    color: red;
    font-weight: bold;
  }
</style>
```

BROWSER PREVIEW — Kết Quả Hiển Thị Chi Tiết Ví Dụ 1: p:nth-child(3)

Header Title (Child vị trí 1 – Thẻ <h2>)

Paragraph 1 (Child vị trí 2 – Thẻ , không bị tác động)

Paragraph 2 (← p:nth-child(3) KHỚP – Vừa là vừa là child vị trí thứ 3 trong DOM)

Span Note (Child vị trí 4 – Thẻ)

Paragraph 3 (Child vị trí 5 – Thẻ , không bị tác động)

(Giải thích: Paragraph 2 bị tác động vì nó vừa là thẻ vừa là phần tử con thứ 3 đúng yêu cầu. Paragraph 1 và Paragraph 3 không bị tác động vì vị trí đếm trong DOM không khớp 3.)

[Khái Niệm] Công Thức Tổng Quát: nth-child(an + b)

Công thức tổng quát đại số có dạng:

$$\texttt{nth-child}(an + b)$$

Phân tích chi tiết công thức $3n + 1$:

- $a = 3$: Nhảy cách 3 phần tử mỗi lần.
- $b = 1$: Bắt đầu xuất phát từ phần tử thứ 1.

[Bảng] Bảng Thử Thay Số n Vào Công Thức 3n + 1 Qua Các Bước Lặp

Giá trị n	Tính toán biểu thức $3n+1$	Vị trí phần tử trong DOM
$n = 0$	`3(0) + 1 = 1`	**Vị trí 1** (Phần tử con đầu tiên)
$n = 1$	`3(1) + 1 = 4`	**Vị trí 4** (Nhảy tiếp 3 bước)
$n = 2$	`3(2) + 1 = 7`	**Vị trí 7** (Nhảy tiếp 3 bước)
$n = 3$	`3(3) + 1 = 10`	**Vị trí 10** (Nhảy tiếp 3 bước)
$n = 4$	`3(4) + 1 = 13`	**Vị trí 13** (Nhảy tiếp 3 bước)

[Mẹo] Mô Hình Tư Duy Trực Quan Về Công Thức 3n + 1

- `3n` tạo ra bước nhảy 3 phần tử một lần.
- `+1` đẩy điểm bắt đầu xuất phát về phần tử số 1.
- **Luồng nhảy trực quan:** Điểm xuất phát từ phần tử 1 → nhảy 3 bước đến 4 → nhảy 3 bước đến 7 → 10 → 13 . . .

Mã Nguồn Minh Họa Trực Quan Công Thức 3n + 1 Trên Danh Sách 10 Phần Tử

```
<ul>
    <li>Item 1</li>
    <li>Item 2</li>
    <li>Item 3</li>
    <li>Item 4</li>
    <li>Item 5</li>
    <li>Item 6</li>
    <li>Item 7</li>
    <li>Item 8</li>
    <li>Item 9</li>
    <li>Item 10</li>
</ul>

<style>
    /* Select items at positions 1, 4, 7, 10 */
    li:nth-child(3n + 1) {
        color: red;
        font-weight: bold;
    }
</style>
```

BROWSER PREVIEW — **Kết Quả Hiển Thị 10 Items Với 3n + 1**

- **Item 1** (→ Đỏ, in đậm (Vị trí 1))
- Item 2 (Mặc định)
- Item 3 (Mặc định)
- **Item 4** (→ Đỏ, in đậm (Vị trí 4))
- Item 5 (Mặc định)
- Item 6 (Mặc định)
- **Item 7** (→ Đỏ, in đậm (Vị trí 7))
- Item 8 (Mặc định)
- Item 9 (Mặc định)
- **Item 10** (→ Đỏ, in đậm (Vị trí 10))

Ví Dụ Chi Tiết Chọn Phần Tử Lẻ (odd) Và Chẵn (even)

```
<style>
    /* Odd items get light gray background */
    li:nth-child(odd) {
        background-color: #f0f0f0;
    }
    /* Even items get darker gray background */
    li:nth-child(even) {
        background-color: #d3d3d3;
    }
</style>
```

BROWSER PREVIEW — **Kết Quả Hiển Thị Xen Kẽ Lẻ / Chẵn**

- **Item 1, 3, 5...** (← Có màu nền xám nhạt #f0f0f0)
- Item 2, 4, 6... (Màu nền trắng mặc định)

Ví Dụ Chọn Với Công Thức Bội Số (3n) & Công Thức Offset (n + 4)

```css
/* Example 1: Select multiples of 3 (Item 3, 6, 9...) */
li:nth-child(3n) {
    color: blue;
    font-weight: bold;
}

/* Example 2: Select all items after item 3 (Item 4, 5, 6...) */
li:nth-child(n + 4) {
    text-decoration: underline;
}
```

BROWSER PREVIEW — Kết Quả Hiển Thị Công Thức 3n Và n + 4

- **Item 3** (→ li:nth-child(3n) chọn màu xanh, in đậm)
- Item 4, 5, 6... (→ li:nth-child(n + 4) gạch chân từ phần tử 4 trở đi)

[Cảnh Báo] Phân Tích Chi Tiết: Các Trường Hợp :nth-child(an + b) KHÔNG Hoạt Động Như Kỳ Vọng

Mặc dù `:nth-child(an + b)` (bao gồm `odd`, `even`, `n`, `2n+1`, `-n+3`, v.v.) rất mạnh trong việc chọn phần tử theo vị trí, nhưng có những trường hợp cụ thể nó **không thỏa mãn hoặc không hoạt động như bạn kỳ vọng**:

1. Khi phần tử không nằm ở đúng vị trí tính từ đầu:

- Bạn nghĩ `p:nth-child(1)` sẽ tô đỏ đoạn `<p>Paragraph 1</p>` vì nó là đoạn văn đầu tiên.
- **SAI!** Vì thẻ `<p>` đứng sau thẻ `<h2>` nên nó nằm ở vị trí con thứ 2 trong DOM. Do đó `nth-child(1)` không áp dụng cho `p`.
- **Cách sửa:** Dùng `p:first-of-type { color: red; }` hoặc `p:nth-of-type(1) { color: red; }`.

2. Số lượng phần tử không đủ để khớp với công thức:

- Bạn viết `li:nth-child(3n)` với mong muốn chọn các phần tử ở vị trí chia hết cho 3 trong danh sách `<ul>`.
- **SAI!** Nếu danh sách chỉ có 2 thẻ `<li>`, **không có gì xảy ra** vì công thức $3n$ chỉ khớp tại các vị trí 3, 6, 9...
- **Cách hiểu đúng:** Công thức $3n$ chỉ tác động khi tồn tại phần tử ở vị trí index 3, 6, 9...

3. Phối hợp odd, even sai đối tượng trong DOM hỗn hợp:

- Bạn viết `p:nth-child(odd)` với mong muốn tô đỏ thẻ `<p>` ở vị trí lẻ.
- **SAI!** Nếu cấu trúc DOM xen kẽ `<span>1</span>`, `<p>2</p>`, `<span>3</span>`, `<p>4</p>`:
  - `<span>` là con vị trí 1 → odd (lẻ)
  - `<p>` là con vị trí 2 → even (chẵn)
  - `<span>` là con vị trí 3 → odd (lẻ)
  - `<p>` là con vị trí 4 → even (chẵn)

  Do thẻ `<p>` nằm ở vị trí 2 và 4 (chẵn), bộ chọn `p:nth-child(odd)` tìm thẻ `<p>` ở vị trí lẻ nhưng không có thẻ `<p>` nào ở vị trí lẻ → **Không có thẻ p nào được tô màu!**
- **Cách đúng:** Dùng `p:nth-of-type(odd) { color: red; }` để chỉ đếm các thẻ `<p>` ở vị trí lẻ trong tập hợp các thẻ `<p>`.

[Khái Niệm] Ứng Dụng Thực Tế Của :nth-child(n)

1. **Danh sách sản phẩm**: Tô màu xen kẽ cho từng dòng sản phẩm, làm nổi bật sản phẩm theo nhóm.
2. **Bảng dữ liệu (Tables)**: Làm nổi bật từng dòng hoặc cột theo thứ tự lẻ/chẵn (**Zebra Striping**).
3. **Menu hoặc thanh điều hướng**: Thêm hiệu ứng đặc biệt cho nút đầu tiên, nút cuối hoặc các nút theo quy luật.
4. **Thẻ bài viết hoặc ảnh (Grid Gallery)**: Chọn bài viết/ảnh theo quy luật để tạo bố cục lưới đẹp mắt, phân trang linh hoạt.

e2. Phân Tích Chuyên Sâu :nth-of-type(an + b) (Đếm Vị Trí Theo Loại Thẻ) `:nth-of-type(an + b)` là pseudo-class selector chọn phần tử dựa trên vị trí thứ tự của nó khi **chỉ lọc đếm riêng tập hợp các phần tử cùng loại thẻ** (cùng tag name) trong phần tử cha (đếm từ trên xuống).

[Khái Niệm] Định Nghĩa Chi Tiết & Cơ Chế Đếm Của :nth-of-type(n)

Khác với `:nth-child(n)`, `:nth-of-type(n)` **bỏ qua các phần tử khác loại thẻ** và tạo một danh sách đếm độc lập:

- Trình duyệt gom tất cả các thẻ cùng tag name (ví dụ tất cả các thẻ `<p>`).
- Thẻ `<p>` đầu tiên xuất hiện trong container có chỉ số $index = 1$ (trong nhóm thẻ `<p>`).
- Thẻ `<p>` thứ hai xuất hiện trong container có chỉ số $index = 2$ (trong nhóm thẻ `<p>`).
- Các thẻ khác loại (`<h2>`, `<span>`, `<div>`...) nằm chen giữa hoàn toàn **không làm gián đoạn hay tăng chỉ số đếm** của nhóm thẻ `<p>`.

Cú Pháp Cơ Bản Của :nth-of-type(n)

```
/* Select specific tag by index in its tag group */
selector:nth-of-type(n) {
    /* CSS properties apply here */
}

/* Formula syntax for specific tag group */
selector:nth-of-type(an + b) {
    /* CSS properties apply here */
}
```

Mã Nguồn Minh Họa Phân Biệt Chi Tiết: .container :nth-child(3) vs .container p:nth-of-type(3)

```html
<div class="container">
  <p>Doan 1</p>        <!-- p thu 1 (STT DOM: 1) -->
  <span>Ghi chu 1</span> <!-- span thu 1 (STT DOM: 2) -->
  <p>Doan 2</p>        <!-- p thu 2 (STT DOM: 3) -->
  <p>Doan 3</p>        <!-- p thu 3 (STT DOM: 4) -->
  <span>Ghi chu 2</span> <!-- span thu 2 (STT DOM: 5) -->
</div>

<style>
  /* Select 3rd child overall in .container regardless of tag */
  .container :nth-child(3) {
    color: red;
  }

  /* Select 3rd <p> tag specifically among all <p> siblings */
  .container p:nth-of-type(3) {
    background-color: yellow;
  }
</style>
```

[Bảng] Bảng Thống Kê Thứ Tự DOM Chi Tiết Cho .container :nth-child(3)

STT DOM	Phần tử HTML con	Trạng thái chọn với :nth-child(3)
1	`<p>Doan 1</p>`	Bỏ qua (Vị trí 1)
2	`<span>Ghi chu 1</span>`	Bỏ qua (Vị trí 2)
3	`<p>Doan 2</p>`	KHỚP :nth-child(3) (← Tô màu chữ đỏ)
4	`<p>Doan 3</p>`	Bỏ qua (Vị trí 4)
5	`<span>Ghi chu 2</span>`	Bỏ qua (Vị trí 5)

[Bảng] Bảng Thống Kê Thứ Tự Nhóm Thẻ <p> Cho .container p:nth-of-type(3)

Thẻ <p> trong danh sách	Chỉ số :nth-of-type	Trạng thái chọn với p:nth-of-type(3)
`<p>Doan 1</p>`	`p:nth-of-type(1)`	Bỏ qua (Chỉ số 1 trong nhóm <p>)
`<p>Doan 2</p>`	`p:nth-of-type(2)`	Bỏ qua (Chỉ số 2 trong nhóm <p>)
`<p>Doan 3</p>`	`p:nth-of-type(3)`	KHỚP p:nth-of-type(3) (← Tô màu nền vàng)

BROWSER PREVIEW Kết Quả Hiển Thị Phân Biệt Chi Tiết Trong Trình Duyệt

Doan 1 (Thẻ <p> thứ 1 – STT DOM 1)
Ghi chu 1 (Thẻ thứ 1 – STT DOM 2)
Doan 2 (← .container :nth-child(3) KHỚP – Con thứ 3 tổng thể của .container)
Doan 3 (← .container p:nth-of-type(3) KHỚP – Thẻ <p> thứ 3 trong các <p> con của .container)
Ghi chu 2 (Thẻ thứ 2 – STT DOM 5)

[Bảng] Bảng Đánh Giá Kết Luận Chuẩn Xác Chọn Phần Tử

Selector	Tiêu chí chọn phần tử	Kết quả đúng chọn được
`:nth-child(3)`	Con thứ 3 tổng thể trong container	`<p>Doan 2</p>`
`p:nth-of-type(3)`	Thẻ <p> thứ 3 trong nhóm <p> con	`<p>Doan 3</p>`

[Cảnh Báo] Các Lưu Ý Quan Trọng Khi Sử Dụng :nth-of-type(n)

1. Đếm độc lập theo tag name: Mỗi loại thẻ trong container có một bộ đếm chỉ số độc lập.

2. An toàn hơn trong giao diện phức tạp: Khi làm việc với HTML động có thẻ tiêu đề, quảng cáo hoặc icon chen giữa các bài viết, `:nth-of-type(n)` là lựa chọn an toàn tuyệt đối.

[Mẹo] Mẹo Ghi Nhớ :nth-of-type(n)

Công thức tương đương: `p:nth-of-type(1)` ≡ `p:first-of-type` (đoạn văn đầu tiên).

[Khái Niệm] Ứng Dụng Thực Tế Của :nth-of-type(n)

- **Trang tin tức / Blog**: Định kiểu cho đoạn văn đầu tiên (**Drop cap**) hoặc các đoạn văn lẻ/chẵn khi có ảnh chen giữa.
- **Form phức tạp**: Chọn các ô `<input type="text">` chẵn/lẻ để tô màu nền khi có thẻ `<label>` chen giữa.
- **Gallery ảnh**: Style cho ảnh `<img>` thứ N trong bài viết.

e3. Bảng So Sánh Chi Tiết :nth-child(n) vs :nth-of-type(n)

Mã Nguồn Minh Họa 3 Trường Hợp Thất Bại & Cách Khắc Phục

```
<!-- Case 1: Header at position 1 -->
<div class="case1">
  <h2>Header Title</h2>
  <p>Paragraph 1</p> <!-- Child 2 -->
</div>
<style>
  /* FAILS: p is child 2, not child 1 */
  .case1 p:nth-child(1) { color: red; }
  /* FIX: Select 1st <p> specifically */
  .case1 p:nth-of-type(1) { color: green; }
</style>

<!-- Case 2: Insufficient items -->
<ul>
  <li>Item 1</li>
  <li>Item 2</li>
</ul>
<style>
  /* FAILS: List only has 2 items, 3n matches position 3, 6, 9 */
  li:nth-child(3n) { color: blue; }
</style>

<!-- Case 3: Mixed odd/even -->
<div class="case3">
  <span>Span 1</span>  <!-- Child 1 (odd) -->
  <p>Paragraph 2</p>  <!-- Child 2 (even) -->
  <span>Span 3</span>  <!-- Child 3 (odd) -->
  <p>Paragraph 4</p>  <!-- Child 4 (even) -->
</div>
<style>
  /* FAILS: <p> tags are at even positions 2 \& 4, no <p> at odd position */
  .case3 p:nth-child(odd) { color: red; }
  /* FIX: Select odd <p> within <p> elements only */
  .case3 p:nth-of-type(odd) { color: green; }
</style>
```

●●● **BROWSER PREVIEW** **Kết Quả Trực Quan 3 Trường Hợp Thất Bại & Cách Khắc Phục**

Trường hợp 1 (Vị trí 1 không phải <p>):
p:nth-child(1) → Thất bại (Không đỏ). p:nth-of-type(1) → **Thành công (Tô xanh)**

Trường hợp 2 (Không đủ 3 phần tử):
li:nth-child(3n) → Không có tác dụng vì danh sách chỉ có 2 items

Trường hợp 3 (Odd/Even trong DOM hỗn hợp):
p:nth-child(odd) → Thất bại (Vì <p> nằm ở vị trí chẵn 2 & 4). p:nth-of-type(odd) → **Thành công (Tô xanh Paragraph 2)**

[Bảng] Bảng So Sánh Toàn Diện Cơ Chế Đếm: :nth-child(n) vs :nth-of-type(n)

Tiêu chí so sánh	:nth-child(an + b)	:nth-of-type(an + b)
Cơ chế đếm	Đếm **TẤT CẢ** phần tử con (từ trên xuống)	Chỉ đếm phần tử con **CÙNG loại thẻ** (từ trên xuống)
Ảnh hưởng bởi thẻ khác loại?	**Có** (Thẻ khác loại chen vào sẽ làm tăng vị trí index)	**Không** (Thẻ khác loại chen vào bị bỏ qua hoàn toàn)
Tương đương logic với $n = 1$	`:first-child`	`:first-of-type`
Mức độ nghiêm ngặt	Nghiêm ngặt (Yêu cầu đúng vị trí tuyệt đối trong DOM)	Linh hoạt (Phù hợp với HTML lộn xộn nhiều loại thẻ)
Khi nào nên dùng?	Khi danh sách đồng nhất (ví dụ toàn `<li>` hoặc `<tr>`)	Khi container hỗn hợp nhiều loại thẻ (`<h2>`, `<p>`, `<span>`)

[Bảng] Bảng Tóm Tắt Các Trường Hợp Sai Phổ Biến & Cách Khắc Phục

Trường hợp sai	Nguyên nhân cốt lõi	Cách khắc phục chuẩn
nth-child(n) không áp dụng đúng	Không tính đúng thứ tự toàn bộ phần tử con trong DOM	Dùng :nth-of-type(n)
3n, 4n+1... không chọn gì	Không có phần tử ở vị trí khớp với công thức số học	Kiểm tra lại tổng số phần tử con
odd, even không chọn như ý	Đếm tính theo tất cả phần tử con (không phân biệt loại thẻ)	Dùng :nth-of-type(odd/even)
Kết hợp :nth-child() với selector thẻ	Chọn sai vì thẻ cụ thể không đứng ở đúng vị trí index trong DOM	Xem lại thứ tự DOM hoặc dùng :nth-of-type()

[Bảng] Bảng Tổng Hợp Các Biểu Thức n Và Cách Hoạt Động Chi Tiết

Biểu thức n	Ý nghĩa & Quy tắc chọn phần tử
`nth-child(3)`	Chọn chính xác phần tử ở vị trí thứ 3.
`nth-child(odd)`	Chọn các phần tử có vị trí lẻ (1, 3, 5, 7, 9...).
`nth-child(even)`	Chọn các phần tử có vị trí chẵn (2, 4, 6, 8, 10...).
`nth-child(3n)`	Chọn các phần tử là bội số của 3 (vị trí 3, 6, 9, 12...).
`nth-child(3n + 1)`	Chọn các phần tử ở vị trí 1, 4, 7, 10, 13... (bắt đầu từ 1, nhảy 3 bước).
`nth-child(-n + 3)`	Chọn 3 phần tử đầu tiên (vị trí 1, 2, 3).
`nth-child(n + 4)`	Chọn tất cả phần tử từ vị trí thứ 4 trở đi (4, 5, 6, 7...).

[Khái Niệm] Tóm Lại Về :nth-child(n) vs :nth-of-type(n)

- `:nth-child(n)` đếm tất cả phần tử con theo thứ tự tổng thể tuyệt đối.
- `:nth-of-type(n)` đếm theo loại thẻ riêng biệt trong nhóm phần tử cùng tên thẻ.
- Có thể dùng số nguyên cụ thể, từ khóa (`odd`, `even`), hoặc biểu thức số học (`3n + 1`).
- Rất linh hoạt và mạnh mẽ, cực kỳ hữu ích khi bạn muốn tạo kiểu theo quy luật mà không cần viết từng class riêng cho từng phần tử.

f. Chuyên Đề 4: Bộ Chọn Đếm Ngược Từ Đáy Lên (:nth-last-child() vs :nth-last-of-type())

Trong thiết kế giao diện động, việc định kiểu cho các phần tử đứng ở cuối hoặc gần cuối danh sách (ví dụ: nút cuối cùng, bài viết kế cuối, dòng tổng cộng ở đáy bảng) là nhu cầu rất phổ biến. CSS cung cấp hai bộ chọn đếm ngược từ đáy lên: `:nth-last-child()` và `:nth-last-of-type()`. Phần này sẽ phân tích chi tiết và tách biệt hai bộ chọn này.

[Khái Niệm] Tổng Quan Nhanh: 2 Bộ Chọn Đếm Ngược Trong CSS

- `:nth-last-child(an + b)`: Đếm thứ tự **từ dưới lên trên** (bắt đầu từ phần tử con cuối cùng), đếm **tất cả các phần tử con** không phân biệt loại thẻ.
- `:nth-last-of-type(an + b)`: Đếm thứ tự **từ dưới lên trên**, nhưng **chỉ đếm các phần tử cùng loại thẻ** (cùng tag name).

f1. Phân Tích Chuyên Sâu :nth-last-child() (Đếm Ngược Toàn Bộ Phần Tử Con) `:nth-last-child(an + b)` là pseudo-class selector chọn phần tử dựa trên vị trí thứ tự của nó khi **đếm từ cuối danh sách đếm ngược lên đầu** (từ phần tử con cuối cùng = 1).

[Khái Niệm] Định Nghĩa Chi Tiết :nth-last-child()

`:nth-last-child()` hoạt động hoàn toàn giống với `:nth-child()`, ngoại trừ chiều đếm bị đảo ngược:

- Phần tử con cuối cùng trong cha có chỉ số $index = 1$.
- Phần tử con kế cuối có chỉ số $index = 2$.
- Trình duyệt đếm **tất cả phần tử con** (element nodes) mà không phân biệt loại thẻ (`<div>`, `<p>`, `<span>`, `<li>`...).
- Biểu thức công thức $an + b$, từ khóa `odd`, `even`, hoặc số nguyên n đều áp dụng chính xác theo chiều ngược từ dưới lên.

Cú Pháp Cơ Bản Của :nth-last-child()

```
/* Select element at specific position from bottom */
selector:nth-last-child(n) {
    /* CSS properties apply here */
}

/* Select using formula from bottom */
selector:nth-last-child(an + b) {
    /* CSS properties apply here */
}
```

Ví Dụ 1: Chọn Phần Tử Kế Cuối Bằng li:nth-last-child(2)

```
<ul>
  <li>Item 1</li>
  <li>Item 2</li>
  <li>Item 3</li>
  <li>Item 4</li> <!-- Selected: 2nd from bottom -->
  <li>Item 5</li> <!-- Position 1 from bottom -->
</ul>

<style>
  /* Select 2nd element from bottom */
  li:nth-last-child(2) {
    color: red;
    font-weight: bold;
  }
</style>
```

BROWSER PREVIEW — **Kết Quả Ví Dụ 1: li:nth-last-child(2)**

- Item 1 (Vị trí 5 từ dưới)
- Item 2 (Vị trí 4 từ dưới)
- Item 3 (Vị trí 3 từ dưới)
- **Item 4 (Chữ đỏ, in đậm)** (← li:nth-last-child(2) khớp – vị trí thứ 2 từ dưới lên)
- Item 5 (Vị trí 1 từ dưới – phần tử cuối cùng)

Ví Dụ 2: Dùng Công Thức an + b Đếm Ngược Từ Đáy

```
<ul>
  <li>Item 1</li> <!-- Match n=2 (pos 5) -->
  <li>Item 2</li>
  <li>Item 3</li> <!-- Match n=1 (pos 3) -->
  <li>Item 4</li>
  <li>Item 5</li> <!-- Match n=0 (pos 1) -->
</ul>

<style>
  /* Select odd positions from bottom (1, 3, 5...) */
  li:nth-last-child(2n + 1) {
    background-color: #f0f0f0;
  }
</style>
```

BROWSER PREVIEW — **Kết Quả Ví Dụ 2: li:nth-last-child(2n + 1)**

- **Item 1** (← Nền xám #f0f0f0 – Vị trí 5 từ dưới lên)
- Item 2 (Vị trí 4 từ dưới – không đổi)
- **Item 3** (← Nền xám #f0f0f0 – Vị trí 3 từ dưới lên)
- Item 4 (Vị trí 2 từ dưới – không đổi)
- **Item 5** (← Nền xám #f0f0f0 – Vị trí 1 từ dưới lên)

Ví Dụ 3: DOM Hỗn Hợp – Lưu Ý Về Vị Trí Tuyệt Đối Đếm Ngược

```
<div class="container">
  <h2>Title</h2>      <!-- Pos 4 from bottom -->
  <p>Paragraph 1</p>  <!-- Pos 3 from bottom -->
  <p>Paragraph 2</p>  <!-- Pos 2 from bottom -->
  <span>Footer</span> <!-- Pos 1 from bottom -->
</div>

<style>
  /* FAILS to select Paragraph 2:
     Because at pos 1 from bottom is <span>, pos 2 is <p>Paragraph 2</p>,
     so p:nth-last-child(1) looks for <p> at position 1 from bottom (which is <
    span>) */
  p:nth-last-child(1) { color: red; }
</style>
```

BROWSER PREVIEW — **Kết Quả Ví Dụ 3: DOM Hỗn Hợp Đếm Ngược**

Title (Thẻ <h2> – Vị trí 4 từ dưới)
Paragraph 1 (Thẻ <p> – Vị trí 3 từ dưới)
Paragraph 2 (Thẻ <p> – Vị trí 2 từ dưới, KHÔNG đỏ)
Footer (Thẻ – Vị trí 1 từ dưới)

Giải thích: p:nth-last-child(1) thất bại vì phần tử vị trí 1 từ dưới lên là , không phải <p>.

[Cảnh Báo] Các Lưu Ý Quan Trọng Khi Sử Dụng :nth-last-child()

1. Đếm tất cả loại phần tử con: `:nth-last-child()` đếm mọi thẻ con trong container khi đếm ngược từ dưới lên.

2. Dễ nhầm lẫn vị trí trong DOM hỗn hợp: Nếu thẻ con cuối cùng là `<span>`, thì `p:nth-last-child(1)` sẽ không có tác dụng.

3. Giải pháp khi có nhiều thẻ khác loại: Dùng `:nth-last-of-type()` nếu chỉ muốn đếm ngược trên một loại thẻ cụ thể.

[Mẹo] Mẹo Ghi Nhớ :nth-last-child()

Công thức tương đương: `:nth-last-child(1)` ≡ `:last-child` (phần tử con cuối cùng tuyệt đối).

[Khái Niệm] Ứng Dụng Thực Tế Của :nth-last-child()

- **Ẩn đường phân cách (border) của N phần tử cuối**: Ví dụ xóa viền dưới của 2 item cuối trong danh sách.
- **Định dạng bảng dữ liệu**: Làm nổi bật dòng tổng cộng hoặc 2 dòng cuối bảng.
- **Timeline & Nhật ký**: Style khác biệt cho các bài viết cũ nhất (ở đáy timeline).

f2. Phân Tích Chuyên Sâu :nth-last-of-type() (Đếm Ngược Theo Loại Thẻ) `:nth-last-of-type(an + b)` là pseudo-class selector chọn phần tử dựa trên vị trí của nó khi **chỉ lọc đếm các phần tử cùng loại thẻ (tag name) từ dưới lên trên**.

[Khái Niệm] Định Nghĩa Chi Tiết :nth-last-of-type()

Khác với `:nth-last-child()`, `:nth-last-of-type()` **bỏ qua các thẻ khác loại** và chỉ đánh số thứ tự đếm ngược cho nhóm thẻ cùng loại:

- Trình duyệt lọc tất cả các thẻ cùng tag name (ví dụ tất cả các thẻ `<p>`).
- Thẻ `<p>` xuất hiện sau cùng trong DOM có chỉ số $index = 1$ (theo loại thẻ `<p>`).
- Thẻ `<p>` xuất hiện trước đó có chỉ số $index = 2$ (theo loại thẻ `<p>`).
- Thẻ `<h2>`, `<span>`, `<div>`... chen giữa hoàn toàn **không làm gián đoạn** danh sách đếm.

Cú Pháp Cơ Bản Của :nth-last-of-type()

```
/* Select specific tag type counting from bottom */
selector:nth-last-of-type(n) {
    /* CSS properties apply here */
}

/* Formula for specific tag type from bottom */
selector:nth-last-of-type(an + b) {
    /* CSS properties apply here */
}
```

Ví Dụ 1: Chọn Thẻ <p> Cuối Cùng Trong DOM Hỗn Hợp

```html
<div class="container">
  <h2>Header</h2>
  <p>Paragraph 1</p> <!-- Pos 2 of <p> from bottom -->
  <p>Paragraph 2</p> <!-- Pos 1 of <p> from bottom -->
  <span>Footer text</span>
</div>

<style>
  /* Select last <p> tag from bottom, regardless of <span> below it */
  p:nth-last-of-type(1) {
    color: green;
    font-weight: bold;
  }
</style>
```

BROWSER PREVIEW — Kết Quả Ví Dụ 1: p:nth-last-of-type(1)

Header (Thẻ <h2>)

Paragraph 1 (Thẻ <p> thứ 2 đếm từ dưới lên trong các thẻ <p>)

Paragraph 2 (← p:nth-last-of-type(1) khớp – Thẻ <p> cuối cùng, dù có đứng dưới)

Footer text (Thẻ – không bị đếm vào nhóm <p>)

Ví Dụ 2: Đếm Ngược Xen Kẽ Cho Nhóm Thẻ <p>

```html
<div>
  <h2>Title</h2>
  <p>Paragraph A</p> <!-- Pos 3 of <p> -> Matches odd (1st from top of <p>) -->
  <span>Divider</span>
  <p>Paragraph B</p> <!-- Pos 2 of <p> -> Even -->
  <p>Paragraph C</p> <!-- Pos 1 of <p> -> Matches odd -->
</div>

<style>
  /* Select odd <p> tags counting from bottom */
  p:nth-last-of-type(odd) {
    background-color: lightyellow;
  }
</style>
```

BROWSER PREVIEW — **Kết Quả Ví Dụ 2: p:nth-last-of-type(odd)**

Title (Thẻ <h2>)
Paragraph A (← Vị trí 3 đếm ngược trong các thẻ <p> – Khớp odd)
Divider (Thẻ – Không bị đếm)
Paragraph B (Vị trí 2 đếm ngược trong các thẻ <p> – Even)
Paragraph C (← Vị trí 1 đếm ngược trong các thẻ <p> – Khớp odd)

[Cảnh Báo] Các Lưu Ý Quan Trọng Khi Sử Dụng :nth-last-of-type()

1. Đếm độc lập cho từng loại thẻ: Nếu trong cùng container có thẻ `<h2>`, `<p>`, `<img>`, trình duyệt sẽ tạo 3 danh sách đếm ngược độc lập cho từng loại thẻ.

2. Linh hoạt hơn :nth-last-child(): Rất thích hợp cho mã HTML thực tế nơi các thẻ tiêu đề, đoạn văn, hình ảnh thường nằm xen kẽ nhau.

[Mẹo] Mẹo Ghi Nhớ :nth-last-of-type()

Công thức tương đương: `:nth-last-of-type(1)` ≡ `:last-of-type` (phần tử cuối cùng thuộc loại thẻ đó).

[Khái Niệm] Ứng Dụng Thực Tế Của :nth-last-of-type()

- **Đoạn văn cuối cùng trong bài viết**: Thêm chữ ký, nguồn tác giả, hoặc khoảng cách lề dưới cho thẻ `<p>` cuối cùng.
- **Hình ảnh cuối bài**: Tự động thêm caption hoặc bo góc cho ảnh `<img>` cuối cùng.
- **Form ô nhập liệu cuối**: Đổi style cho nút bấm hoặc input cuối cùng trong Form.

f3. Bảng So Sánh Chi Tiết :nth-last-child() vs :nth-last-of-type()

Ví Dụ Trực Quan So Sánh :nth-last-child(1) vs :nth-last-of-type(1)

```html
<div class="demo">
  <h2>Article Header</h2>
  <p>First paragraph</p>
  <p>Second paragraph</p>
  <span>Footnote reference</span>
</div>

<style>
  /* FAILS: Last child is <span>, not <p> */
  .demo p:nth-last-child(1) { color: red; }

  /* WORKS: Selects last <p> (Second paragraph), ignoring <span> */
  .demo p:nth-last-of-type(1) { color: green; font-weight: bold; }
</style>
```

BROWSER PREVIEW — Kết Quả So Sánh Trực Quan trong Trình Duyệt

Article Header (Thẻ <h2> – Vị trí 4 từ dưới)

First paragraph (Thẻ <p> – Vị trí 3 từ dưới)

Second paragraph (← p:nth-last-of-type(1) KHỚP – Thẻ <p> cuối cùng)

Footnote reference (Thẻ – Vị trí 1 từ dưới tuyệt đối)

Lưu ý: p:nth-last-child(1) KHÔNG khớp vì vị trí 1 từ dưới lên là , không phải <p>.

[Bảng] Bảng So Sánh Toàn Diện :nth-last-child() vs :nth-last-of-type()

Tiêu chí so sánh	:nth-last-child(an + b)	:nth-last-of-type(an + b)
Hướng đếm	Từ dưới lên trên (từ đáy lên)	Từ dưới lên trên (từ đáy lên)
Cơ chế đếm	Đếm **TẤT CẢ** phần tử con (không phân biệt loại thẻ)	Chỉ đếm phần tử con **CÙNG loại thẻ** (cùng tag name)
Ảnh hưởng bởi thẻ khác loại?	**Có** – Thẻ khác loại ở đáy sẽ làm thay đổi chỉ số vị trí	**Không** – Thẻ khác loại bị bỏ qua hoàn toàn
Tương đương logic với $n = 1$	`:last-child`	`:last-of-type`
Đối ứng với đếm từ trên xuống	`:nth-child(an + b)`	`:nth-of-type(an + b)`
Mức độ linh hoạt	Nghiêm ngặt (yêu cầu đúng vị trí tuyệt đối từ đáy)	Linh hoạt (phù hợp với HTML lộn xộn nhiều loại thẻ)
Khi nào dùng?	Khi cấu trúc danh sách đồng nhất (ví dụ toàn `<li>`)	Khi cấu trúc danh sách hỗn hợp nhiều loại thẻ (`<h2>`, `<p>`, `<span>`)

[Bảng] Bảng Tóm Tắt Quy Tắc Sử Dụng :nth-last-child() & :nth-last-of-type()

Quy tắc	Đúng / Sai	Ghi chú
:nth-last-child(1) luôn chọn phần tử cuối cùng tuyệt đối	**Đúng**	Tương đương :last-child
:nth-last-child() không đếm text node hay khoảng trắng	**Đúng**	Chỉ đếm element nodes HTML
Có thẻ khác loại ở đáy làm lệch đếm :nth-last-child()	**Đúng**	Vì đếm tất cả phần tử con
:nth-last-of-type() chỉ đếm thẻ cùng loại từ dưới lên	**Đúng**	Bỏ qua thẻ khác loại
Muốn chọn thẻ p cuối cùng trong container hỗn hợp dùng :nth-last-of-type(1)	**Đúng**	Chuẩn xác và an toàn nhất

[Mẹo] Tổng Kết Nhanh Mẹo Phân Biệt 4 Bộ Chọn Vị Trí

- `:nth-child(n)`: Đếm từ trên xuống → Tất cả loại thẻ.
- `:nth-of-type(n)`: Đếm từ trên xuống → Chỉ thẻ cùng loại.
- `:nth-last-child(n)`: Đếm từ dưới lên → Tất cả loại thẻ.
- `:nth-last-of-type(n)`: Đếm từ dưới lên → Chỉ thẻ cùng loại.

g. Chuyên Đề 5: Bộ Chọn Con Duy Nhất (:only-child vs :only-of-type)

Trong thiết kế giao diện Web động, có những trường hợp phần tử cha chỉ chứa duy nhất một phần tử con (ví dụ: một thông báo lỗi duy nhất trong hộp thoại, một nút bấm đơn lẻ trong card, hoặc bài viết chỉ chứa đúng một đoạn văn hay một hình ảnh). CSS3 cung cấp hai bộ chọn giả đặc biệt cho các tình huống này: `:only-child` và `:only-of-type`. Phần này sẽ phân tích chuyên sâu từng bộ chọn và bảng so sánh chi tiết.

[Khái Niệm] Tổng Quan Nhanh: 2 Cơ Chế Kiểm Tra Tính Duy Nhất

- `:only-child`: Phần tử là **phần tử con duy nhất tuyệt đối** của phần tử cha (không có bất kỳ anh chị em nào khác).
- `:only-of-type`: Phần tử là **duy nhất trong nhóm cùng loại thẻ** (cùng tag name) trong phần tử cha (có thể có các anh chị em khác loại thẻ).

g1. Phân Tích Chuyên Sâu :only-child (Con Duy Nhất Tuyệt Đối) `:only-child` là một pseudo-class selector dùng để chọn một phần tử khi nó là **phần tử con duy nhất tuyệt đối** của phần tử cha.

[Khái Niệm] Định Nghĩa Chi Tiết & Cơ Chế Hoạt Động Của :only-child

`:only-child` khớp với một phần tử khi và chỉ khi hai điều kiện sau thỏa mãn đồng thời:

- Phần tử là con duy nhất của phần tử cha (tức là cha chỉ có đúng 1 phần tử con HTML, không có anh chị em nào khác).
- Phần tử đó khớp với bộ chọn `selector` phía trước.
- **Lưu ý quan trọng:** Nếu trong phần tử cha xuất hiện thêm bất kỳ phần tử con nào khác (dù khác loại thẻ như `<span>`, `<div>`, `<i>`...), điều kiện `:only-child` sẽ ngay lập tức bị phá vỡ → **Không được chọn!**

Tương đương logic: `selector:only-child` ≡ `selector:first-child:last-child` (vừa là con đầu tiên vừa là con cuối cùng → phải là con duy nhất).

Cú Pháp Cơ Bản Của :only-child

```
selector:only-child {
    /* CSS properties apply here */
}

/* Equivalent to: */
selector:first-child:last-child {
    /* Same result */
}
```

**Ví Dụ 1: Đoạn Văn Là Con Duy Nhất Trong **

```
<div>
  <p>Day la doan duy nhat</p> <!-- Single child inside <div> -->
</div>

<style>
  p:only-child {
    color: red;
    font-weight: bold;
  }
</style>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Ví Dụ 1: p:only-child Thành Công**

Day la doan duy nhat (← p:only-child KHỚP – Được tô đỏ, in đậm vì là phần tử con duy nhất trong)

Ví Dụ 2: Trường Hợp THẤT BẠI Của :only-child Khi Có 2 Phần Tử

```html
<div>
  <p>Doan 1</p> <!-- Siblings present -->
  <p>Doan 2</p>
</div>

<style>
  /* FAILS: <div> has 2 children, not 1 */
  p:only-child {
    color: red;
  }
</style>
```

BROWSER PREVIEW — Kết Quả Hiển Thị Ví Dụ 2: :only-child Thất Bại

Doan 1 (Không tô đỏ – Có anh em cùng cấp)

Doan 2 (Không tô đỏ – Có anh em cùng cấp)

Giải thích: Dù viết p:only-child, cả hai <p> đều không bị áp dụng CSS vì chúng không phải là con duy nhất (có 2 phần tử con trong).

[Cảnh Báo] Các Lưu Ý Quan Trọng Khi Sử Dụng :only-child

1. Đếm tất cả các phần tử con HTML: `:only-child` đếm toàn bộ phần tử con element nodes (không phân biệt tag name).

2. Khoảng trắng / Text node không bị đếm: Các node văn bản thuần hoặc xuống dòng trong HTML không tính là phần tử con HTML.

3. Bị phá vỡ bởi thẻ khác loại: Nếu container chứa `<p>Paragraph</p>` và `<span>Note</span>`, `p:only-child` sẽ thất bại.

[Mẹo] Mẹo Ghi Nhớ :only-child

Hình dung: `:only-child` giống như ”con một” trong gia đình. Gia đình chỉ được phép có đúng 1 đứa con duy nhất.

[Khái Niệm] Ứng Dụng Thực Tế Của :only-child

- **Căn giữa nội dung đơn lẻ:** Khi một modal hoặc card chỉ có đúng 1 nút bấm hay 1 dòng thông báo, tự động căn giữa và tăng font-size.
- **Ẩn icon/separator:** Xóa đường gạch ngang hoặc icon phân cách khi danh sách chỉ có 1 item duy nhất.
- **Tự động căn chỉnh lề (Margin Reset):** Xóa khoảng cách lề thừa khi container chỉ có 1 phần tử.

g2. Phân Tích Chuyên Sâu :only-of-type (Duy Nhất Theo Loại Thẻ)

`:only-of-type` là pseudo-class selector dùng để chọn phần tử con duy nhất thuộc một loại (tag name) trong phần tử cha. Nếu trong phần tử cha chỉ có đúng 1 phần tử loại đó, thì nó sẽ được chọn.

[Khái Niệm] Định Nghĩa Chi Tiết & Cơ Chế Hoạt Động Của :only-of-type

`:only-of-type` lọc và hoạt động theo các bước:

- Trình duyệt lọc tất cả các phần tử con cùng tag name (ví dụ tất cả các thẻ `<p>`).
- Nếu nhóm phần tử cùng loại thẻ đó có **đúng 1 phần tử duy nhất**, phần tử đó sẽ được chọn.
- Các phần tử con khác loại thẻ (`<h2>`, `<span>`, `<div>`...) nằm trong container hoàn toàn **bị bỏ qua** và không ảnh hưởng đến điều kiện duy nhất.

Tương đương logic: `selector:only-of-type` ≡ `selector:first-of-type:last-of-type` (vừa là thẻ đầu vừa là thẻ cuối của loại đó → phải là thẻ duy nhất của loại đó).

Cú Pháp Cơ Bản Của :only-of-type

```
selector:only-of-type {
  /* style apply */
}
```

Ví Dụ 1: Thẻ <p> Duy Nhất Trong Container Có Thẻ <h2> Chen Giữa

```html
<div class="container">
  <h2>Tieu de bai viet</h2>
  <p>Day la doan van duy nhat</p> <!-- Only <p> tag in .container -->
</div>

<style>
  p:only-of-type {
    color: red;
    font-weight: bold;
  }
</style>
```

BROWSER PREVIEW — **Kết Quả Hiển Thị Ví Dụ 1: p:only-of-type Thành Công**

Tieu de bai viet (Thẻ <h2> – Bị bỏ qua)

Day la doan van duy nhat (← p:only-of-type KHỚP – Trong container chỉ có 1 phần tử <p>)

Ví Dụ 2: Trường Hợp THẤT BẠI Của :only-of-type Khi Có 2 Thẻ <p>

```html
<div class="container">
  <h2>Tieu de bai viet</h2>
  <p>Doan 1</p> <!-- 2 <p> tags present -->
  <p>Doan 2</p>
</div>

<style>
  /* FAILS: container has 2 <p> tags */
  p:only-of-type {
    color: red;
  }
</style>
```

BROWSER PREVIEW — **Kết Quả Hiển Thị Ví Dụ 2: :only-of-type Thất Bại**

Tieu de bai viet (Thẻ <h2>)

Doan 1 (Không chọn – Có 2 thẻ <p>)

Doan 2 (Không chọn – Có 2 thẻ <p>)

[Cảnh Báo] Các Lưu Ý Quan Trọng Khi Sử Dụng :only-of-type

1. Chỉ đếm các phần tử CÙNG loại thẻ: Trình duyệt lọc ra các phần tử cùng tag name, các thẻ khác loại hoàn toàn vô hình trong phép đếm này.

2. Linh hoạt hơn :only-child trong thực tế: Một container thường chứa nhiều loại thẻ khác nhau (`<h2>`, `<p>`, `<span>`...), nên `:only-of-type` dễ khớp và an toàn hơn.

[Bảng] Bảng So Sánh Chi Tiết: :only-child vs :only-of-type

Selector	Ý nghĩa cốt lõi
:only-child	Phần tử là phần tử con duy nhất tuyệt đối của cha (đếm mọi loại thẻ HTML).
:only-of-type	Phần tử là duy nhất trong nhóm cùng loại (cùng tag name) trong cha (bỏ qua thẻ khác).

h. Phân Tích Chuyên Sâu Bộ Chọn :empty (Chọn Phần Tử Rỗng 100%)

Bộ chọn lớp giả `:empty` trong CSS được dùng để chọn các phần tử **không có bất kỳ nội dung con nào cả**, kể cả văn bản, thẻ HTML, khoảng trắng (whitespace) hay ghi chú (comment HTML).

[Khái Niệm] Định Nghĩa & Cú Pháp Chuẩn Của :empty

Cú pháp khai báo:

Cú Pháp Định Kiểu :empty

```
selector:empty {
  /* Thuoc tinh CSS ap dung cho the rong */
}
```

Ý nghĩa cốt lõi: `:empty` chỉ chọn các phần tử không chứa bất kỳ thành phần nào bên trong — **không có text, không có thẻ con, không có khoảng trắng, không có ghi chú comment**.

Ví Dụ Thử Nghiệm 4 Phần Tử Box Với :empty

```
<div class="box box1"></div>
<div class="box box2">   </div>
<div class="box box3"><span></span></div>
<div class="box box4">Hello</div>

<style>
  /* Chi selector:empty moi co vien do va nen hong */
  div.box:empty {
    border: 2px solid red;
    width: 30px;
    height: 30px;
    background-color: #FFE6E6;
  }
</style>
```

BROWSER PREVIEW Kết Quả Trực Quan Hiển Thị Trên Trình Duyệt Cho 4 Thẻ Box

.box1	.box2	.box3	.box4
			Hello
:empty	(space)		Hello
[Có] KHỚP :empty	[Không] Bị loại	[Không] Bị loại	[Không] Bị loại
(Khung đỏ 30px, nền hồng)	(Chứa khoảng trắng)	(Chứa thẻ)	(Chứa chữ "Hello")

[Bảng] Bảng Kết Quả Kiểm Thử Thực Tế Của 4 Thẻ Box

Phần tử	Khớp :empty?	Giải thích nguyên nhân bản chất trong DOM
`.box1`	**[Có] PASS**	Không chứa bất kỳ nút con nào trong DOM (0 Child Nodes).
`.box2`	**[Không] FAIL**	Chứa ký tự khoảng trắng (Text Node khoảng trắng).
`.box3`	**[Không] FAIL**	Chứa phần tử `<span>` bên trong (Element Node).
`.box4`	**[Không] FAIL**	Chứa nội dung văn bản chữ `"Hello"` (Text Node).

[Khái Niệm] Cơ Chế Hoạt Động Theo Từng Bước Của :empty Trong DOM

Bộ chọn `:empty` hoạt động dựa trên cơ chế kiểm tra sự tồn tại của các nút con (**Child Nodes**) trong mô hình cây DOM (Document Object Model):

1. **Bước 1**: Trình duyệt duyệt qua danh sách các phần tử HTML trên trang.
2. **Bước 2**: Với mỗi phần tử, trình duyệt kiểm tra các nút con (*child nodes*) trực thuộc nó trong cây DOM (bao gồm Text node, Element node, Comment node).
3. **Bước 3**: Nếu **KHÔNG** có bất kỳ nút con nào (→ 0 child nodes), phần tử được chọn bởi `:empty`.
4. **Bước 4**: Nếu **CÓ** bất kỳ nút con nào (dù chỉ là 1 khoảng trắng, 1 câu ghi chú comment hay 1 thẻ rỗng), phần tử bị loại khỏi `:empty`.

Minh họa cơ chế chọn qua các dạng cấu trúc DOM:

- `<div></div>` → **[ĐƯỢC CHỌN]** Không có bất kỳ nút con nào trong DOM.
- `<div> </div>` → **[KHÔNG CHỌN]** Có nút văn bản (Text Node chứa khoảng trắng).
- `<div><span></span></div>` → **[KHÔNG CHỌN]** Có nút phần tử con (Element Node `<span>`).
- `<div><!-- comment --></div>` → **[KHÔNG CHỌN]** Có nút ghi chú (Comment Node).

[Cảnh Báo] 5 Quy Tắc Vàng & Lưu Ý Quan Trọng Khi Sử Dụng :empty

1. **:empty chỉ kiểm tra nút con trong DOM:** Không chứa văn bản, không chứa thẻ con, không chứa ghi chú comment HTML.
2. **Khoảng trắng và xuống dòng khiến phần tử KHÔNG rỗng:** Ví dụ `<div>\n</div>` hoặc `<div> </div>` không được coi là rỗng vì ký tự khoảng trắng/xuống dòng là một Text Node.
3. **Phần tử phải thật sự "trống" 100%:** Chỉ duy nhất cú pháp viết sát nhau không kẽ hở `<div></div>` mới thỏa mãn.
4. **:empty KHÔNG kiểm tra thuộc tính CSS hay kích thước hiển thị:** Dù phần tử có thuộc tính `display: none`, `visibility: hidden`, hay chiều cao bằng 0, nếu bên trong vẫn chứa nút con DOM thì vẫn **KHÔNG được coi là :empty**. Mặc dù nhìn bằng mắt thường có vẻ giống nhau, nhưng `:empty` chỉ quan tâm tới cây DOM chứ không quan tâm đến giao diện hiển thị!
5. **:empty giúp dọn rác và tối ưu giao diện HTML:** Tự động ẩn các đoạn văn rỗng hoặc các vùng chứa rỗng mà không cần JavaScript can thiệp.

[Mẹo] Mẹo Kiểm Tra Chuẩn Xác Để Phần Tử Đạt :empty

Muốn chắc chắn một phần tử được chọn bởi `:empty`, hãy đảm bảo 3 quy tắc:

- **KHÔNG** viết bất kỳ ký tự nào giữa thẻ mở và thẻ đóng.
- **KHÔNG** để khoảng trắng hay xuống dòng giữa cặp thẻ.
- **KHÔNG** chứa comment hay bất kỳ thẻ con nào.

[Bảng] Tóm Tắt Khả Năng Khớp Selector :empty Trong Các Trường Hợp DOM

Cấu trúc nội dung bên trong phần tử	Khớp :empty?	Ghi chú bản chất trong cây DOM
Không có gì (Sát nhau `<div></div>`)	**[Có] PASS**	Thỏa mãn điều kiện rỗng 100% (0 Child Nodes).
Khoảng trắng / Xuống dòng (`<div> </div>`)	**[Không] FAIL**	Chứa nút văn bản khoảng trắng (Text Node).
Comment HTML (`<div><!-- comment --></div>`)	**[Không] FAIL**	Chứa nút ghi chú (Comment Node trong DOM).
Thẻ con rỗng (`<div><span></span></div>`)	**[Không] FAIL**	Dù thẻ con rỗng vẫn tính là 1 Element Node.
Nội dung văn bản chữ (`<div>Hello</div>`)	**[Không] FAIL**	Chứa nút nội dung văn bản (Text Node).

[Khái Niệm] 3 Kịch Bản Ứng Dụng Thực Tế Phổ Biến Của :empty

Bộ chọn `:empty` cực kỳ hữu ích trong lập trình Frontend thực tế:

1. **Ẩn các phần tử rỗng tự động (Dọn dẹp layout):** Bạn có thể ẩn các thẻ đoạn văn `<p>` hoặc vùng chứa không có nội dung để giao diện gọn gàng hơn.

Ẩn Thẻ Đoạn Văn Rỗng Trong CSS

```
p:empty {
  display: none;
}
```

2. **Tạo placeholder hoặc thông báo cảnh báo:** Hiển thị khung viền hoặc tô nền cảnh báo cho các vùng chưa có dữ liệu (ví dụ: khung tải dữ liệu hoặc khung chứa ảnh bị lỗi).

Tạo Viền & Nền Hồng Cảnh Báo Cho Div Rỗng

```
div:empty {
  border: 1px dashed red;
  background-color: pink;
}
```

3. **Kiểm soát hiển thị động với JavaScript & Pseudo-element:** Khi nội dung được thêm hoặc xóa bằng JS, `:empty` giúp tự động cập nhật giao diện mà không cần thêm logic CSS phức tạp.

Tự Động Chèn Chữ "Rỗng" Cho Phần Tử Không Có Dữ Liệu

```
.data-box:empty::before {
  content: "Rong (Chua co du lieu)";
  color: gray;
  font-style: italic;
}
```

Mở Rộng Kiến Thức & Kinh Nghiệm Thực Tế Chuyên Sâu Về Selector :empty

[Khái Niệm] 1. Bẫy Định Dạng Trong React/JSX/Template Engine Với :empty

Trong lập trình Frontend hiện đại (React JSX, Vue, EJS, Blade Template), các kỹ sư rất dễ mắc lỗi làm cho `:empty` **thất bại im lặng** do khoảng trắng sinh ra từ code formatting:

Lỗi Phổ Biến Tạo Khoảng Trắng Im Lặng Trong React/JSX

```
<!-- Trong JSX: Viet xuong dong hoac co whitespace giua {} -->
<div class="user-badge">
  {user.title}
</div>

<!-- Ket qua HTML sinh ra DOM Tree: -->
<div class="user-badge"> </div>
<!-- -> CO NHO KHOANG TRANG -> THAT BAI :empty! -->
```

Giải pháp khắc phục Senior:

- Render có điều kiện: `{user.title ? <div class="user-badge">{user.title}</div> : null}`
- Đảm bảo thẻ tự đóng hoặc sát nhau 100% khi chuỗi rỗng: `<div class="user-badge">{user.title}</div>`

[Mẹo] 2. Kỹ Thuật CSS Skeleton Loading Tự Động Với :empty

Trong các ứng dụng Web gọi API bất đồng bộ, ta có thể kết hợp `:empty` với `@keyframes` để tạo hiệu ứng **Skeleton Loading** (mô phỏng khung xương tải dữ liệu) cực kỳ chuyên nghiệp mà không cần dùng đến thẻ HTML phụ:

Tạo Hiệu Ứng Skeleton Loading Bằng CSS :empty

```
/* Khi container chua co du lieu tu API nhan ve (dang empty) */
.profile-name:empty {
  width: 150px;
  height: 20px;
  background: linear-gradient(90deg, #eee 25%, #e0e0e0 50%, #eee 75%);
  background-size: 200% 100%;
  animation: skeleton-shimmer 1.5s infinite;
  border-radius: 4px;
}

@keyframes skeleton-shimmer {
  0% { background-position: 200% 0; }
  100% { background-position: -200% 0; }
}
```

Ưu điểm vượt trội: Khi API tải xong và chèn chữ vào `.profile-name`, thẻ lập tức mất tính chất `:empty`, hiệu ứng Skeleton tự động biến mất và hiển thị văn bản thật mượt mà!

[Cảnh Báo] 3. Bộ Câu Hỏi Phỏng Vấn Senior Frontend Về Selector :empty

1. **Câu hỏi 1: Thẻ tự đóng Void Elements (<input>, <img>,
) có được chọn bởi :empty không?**
   → **Trả lời: CÓ!** Vì các thẻ như `<input>`, `<img>`, `<hr>`, `<br>` là các phần tử rỗng (**Void Elements**), không có bất kỳ nút con nào trong cây DOM (0 Child Nodes). Do đó, `input:empty` hay `img:empty` luôn thỏa mãn điều kiện `:empty`!
2. **Câu hỏi 2: Thẻ có phần tử giả ::before hoặc ::after thì có khớp :empty không?**
   → **Trả lời: CÓ!** Bộ chọn phần tử giả `::before` và `::after` được sinh ra bởi CSS trên cây hiển thị (**Render Tree**), chúng **KHÔNG hề thêm nút con (Child Node) vào cây DOM**. Do đó, một thẻ `<div class="box"></div>` có định kiểu `.box::before { content: "Hi"; }` thì thẻ đó **vẫn khớp** `div:empty` **100%**!
3. **Câu hỏi 3: Bộ chọn modern :has() và :blank khác gì với :empty?**
   → **Trả lời:** Bộ chọn `:blank` (CSS Selectors Level 4) được thiết kế để chọn cả phần tử chứa khoảng trắng whitespace. Ngoài ra, ta có thể dùng bộ chọn modern `:not(:has(*))` để chọn phần tử không chứa bất kỳ thẻ HTML con nào dù cho có chứa khoảng trắng.

j. Bảng Master Cheatsheet Tra Cứu Toàn Diện 11 Pseudo-Classes Cấu Trúc

[Bảng] Bảng Tra Cứu Toàn Diện Tất Cả 11 Pseudo-Classes Vị Trí Cấu Trúc và Phủ Định

Selector	Nguyên lý & Phạm vi hoạt động
`:first-child`	Chọn phần tử nếu nó là con đầu tiên tuyệt đối của cha.
`:last-child`	Chọn phần tử nếu nó là con cuối cùng tuyệt đối của cha.
`:nth-child(n)`	Chọn phần tử đứng ở vị trí thứ n (từ trên xuống).
`:nth-last-child(n)`	Chọn phần tử đứng ở vị trí thứ n (từ dưới đếm lên).
`:only-child`	Chọn phần tử nếu nó là con duy nhất trong cha.
`:first-of-type`	Chọn phần tử cùng loại thẻ đầu tiên trong cha.
`:last-of-type`	Chọn phần tử cùng loại thẻ cuối cùng trong cha.
`:nth-of-type(n)`	Chọn phần tử cùng loại thẻ thứ n (từ trên xuống).
`:nth-last-of-type(n)`	Chọn phần tử cùng loại thẻ thứ n (từ dưới đếm lên).
`:only-of-type`	Chọn phần tử nếu nó là thẻ duy nhất thuộc loại đó trong cha.
`:not(selector)`	Chọn tất cả các phần tử loại trừ phần tử khớp với selector bên trong.

k. Đọc Thêm: Chuyên Đề Nâng Cao, Nguyên Lý Trình Duyệt & Kinh Nghiệm Thực Chiến

[Khái Niệm] 1. Cú Pháp Modern CSS Selectors Level 4: :nth-child(An+B of Selector)

Trình duyệt hiện đại (Chrome 111+, Firefox 113+, Safari 16.4+) hỗ trợ cú pháp mở rộng mạnh mẽ.

Sự khác nhau: Cú pháp cũ `.active:nth-child(2)` chỉ kiểm tra xem phần tử thứ 2 có class `.active` hay không. Cú pháp mới `li:nth-child(2 of .active)` sẽ **lọc tất cả thẻ li có class .active trước**, sau đó chọn thẻ thứ 2 trong tập hợp đó!

Ví Dụ Cú Pháp Modern CSS4 :nth-child(An+B of Selector)

```
/* Select 2nd element matching .active class specifically */
li:nth-child(2 of .active) { color: red; font-weight: bold; }
```

[Cảnh Báo] 2. Nguyên Lý Trình Duyệt & Tối Ưu Hiệu Năng Rendering (Reflow / Repaint)

- **Cơ chế tính toán DOM Tree**: Khi trình duyệt duyệt cây DOM để match các bộ chọn `:nth-child(an+b)`, trình duyệt phải đếm vị tríindex của từng phần tử con.
- **Cảnh báo Performance**: Trong các danh sách cực lớn (hàng nghìn dòng DOM), việc dùng các công thức phức tạp hoặc lồng ghép nhiều pseudo-classes cấu trúc có thể làm tăng thời gian tính toán Layout (**Reflow**) khi người dùng cuộn trang hoặc chèn thêm dữ liệu động.
- **Best Practice**: Với danh sách rất dài, nên kết hợp với kỹ thuật **Virtual Scrolling** hoặc dùng class cố định được render từ phía Server/JavaScript.

[Mẹo] 3. Bộ Câu Hỏi Phỏng Vấn Tuyển Dụng Frontend Thường Gặp (Interview Q&A)

1. **Q1: Phân biệt sự khác nhau giữa `:nth-child(2)` và `:nth-of-type(2)`?**
   → **Trả lời ghi điểm**: `:nth-child(2)` đếm vị trí thứ 2 tuyệt đối trong danh sách con của cha; nếu phần tử thứ 2 không đúng loại thẻ chỉ định thì rule bị hủy. Trong khi đó, `:nth-of-type(2)` chỉ đếm trong tập hợp các phần tử cùng kiểu thẻ.
2. **Q2: Tại sao bộ chọn `:empty` lại không hoạt động khi thẻ HTML có xuống dòng hoặc khoảng trắng?**
   → **Trả lời ghi điểm**: Vì trong mô hình DOM HTML, khoảng trắng (whitespace) và dấu xuống dòng (newline) được trình duyệt tính là một Text Node (`#text`). Do đó thẻ có khoảng trắng không còn rỗng hoàn toàn nữa. Muốn chọn thẻ rỗng kể cả có khoảng trắng trong CSS hiện đại, ta dùng bộ chọn `:blank`.

[Khái Niệm] 4. Lưu Ý Về Khả Năng Truy Cập (Accessibility - a11y)

Khi thay đổi thứ tự hiển thị trực quan hoặc màu sắc của các phần tử bằng `:first-child` hay `:last-child`, hãy đảm bảo thứ tự đọc của công cụ đọc màn hình (**Screen Reader**) và phím Tab (**Keyboard Navigation**) vẫn tuân theo đúng thứ tự ngữ nghĩa trong cây DOM HTML.

9.5.9 8. Chuyên Đề Phân Tích Chi Tiết & Thực Nghiệm Về Bộ Chọn Phần Tử Giả (Pseudo-element Selectors Masterclass)

Pseudo-element selector (bộ chọn phần tử giả) là các bộ chọn trong CSS cho phép bạn định kiểu một phần cụ thể của phần tử, mà không cần phải thêm thẻ HTML mới. Nó giống như tạo ra một phần tử ảo trong cấu trúc trang web!

Dưới đây là chi tiết về từng loại pseudo-element phổ biến và cách hoạt động của chúng:

[Bảng] Bảng Tra Cứu Tóm Tắt 6 Pseudo-Element Selectors Phổ Biến

Pseudo-element	Ý nghĩa	Ví dụ cú pháp	Mô tả chi tiết
`::before`	Thêm nội dung trước nội dung của phần tử.	`p::before { content: "[ICON] "; }`	Chèn icon hoặc văn bản vào đầu phần tử.
`::after`	Thêm nội dung sau nội dung của phần tử.	`p::after { content: " *"; }`	Chèn icon hoặc văn bản vào cuối phần tử.
`::first-letter`	Chọn ký tự đầu tiên của nội dung phần tử.	`p::first-letter { font-size: 2em; }`	Thường dùng để làm chữ cái đầu lớn hơn.
`::first-line`	Chọn dòng đầu tiên của nội dung phần tử.	`p::first-line { color: blue; }`	Định dạng dòng đầu tiên (thường gặp ở bài viết).
`::selection`	Chọn phần nội dung được bôi đen (highlight).	`p::selection { background: yellow; }`	Thay đổi màu nền khi người dùng chọn văn bản.
`::marker`	Định dạng ký hiệu của danh sách (bullet, số).	`li::marker { color: red; }`	Thay đổi màu hoặc kiểu ký hiệu danh sách.

[Cảnh Báo] Lưu Ý Quan Trọng Về Pseudo-Elements

- **Cú pháp chuẩn:** Sử dụng hai dấu hai chấm `::` để viết pseudo-element. (Đây là chuẩn CSS3 mới nhằm phân biệt với pseudo-class dùng 1 dấu chấm hai chấm `:`, nhưng viết `:` vẫn hoạt động trong một số trình duyệt cũ vì mục đích tương thích ngược).
- **Không tác động trực tiếp đến DOM:** Các phần tử giả này **không thật sự xuất hiện trong cây DOM** (Document Object Model), mà chỉ tồn tại trên giao diện hiển thị do trình duyệt dựng nên.

[Khái Niệm] Ví Dụ Thực Tế Minh Họa Pseudo-Elements

Dưới đây là ví dụ kết hợp bộ chọn `::before` và `::after` để chèn ký tự trang trí ở hai đầu văn bản:

Mã Nguồn HTML & CSS Minh Họa Chèn Ký Tự Trang Trí

```html
<div class="box">Chao ban!</div>

<style>
  .box::before {
    content: "[ICON] ";
    color: gold;
  }
  .box::after {
    content: " [ICON]";
    color: red;
  }
</style>
```

BROWSER PREVIEW — **Kết Quả Hiển Thị Trên Trình Duyệt**

Kết quả: [ICON] Chào bạn! [ICON]

[Khái Niệm] Ứng Dụng Thực Tế Của Pseudo-Elements

- **Tạo hiệu ứng trang trí:** Thêm icon, ký hiệu, hoặc hình ảnh nhỏ vào giao diện.
- **Trang trí văn bản:** Làm chữ cái đầu lớn (drop cap) như trên các bài báo in hoặc sách cổ.
- **Cải thiện UX/UI:** Highlight nội dung tùy chỉnh khi người dùng thao tác bôi đen văn bản.
- **Tối ưu mã HTML:** Giảm số lượng thẻ HTML thừa (như thẻ `<span>` rỗng) không cần thiết, giúp mã nguồn HTML sạch và chuẩn ngữ nghĩa.

Phân tích chi tiết từng pseudo-element selector để bạn hiểu rõ và áp dụng dễ dàng:

8.1. Bộ Chọn ::before (Chèn Nội Dung Vào Trước Phần Tử)

Cú Pháp Cơ Bản Của ::before

```css
selector::before {
  content: "Noi dung them vao";
  /* Cac thuoc tinh CSS khac */
}
```

[Khái Niệm] Giải Thích Chi Tiết Cấu Trúc ::before

Chèn nội dung vào **TRƯỚC** nội dung bên trong của phần tử, nhưng không làm thay đổi nội dung thực tế trong cấu trúc mã HTML.

- `selector`: Phần tử mà bạn muốn thêm nội dung phía trước.
- `::before`: Bộ chọn phần tử giả thêm nội dung ảo vào trước phần tử được chọn.
- `content`: **Bắt buộc phải có**, xác định nội dung sẽ hiển thị.

Lưu ý quan trọng: Nếu không có thuộc tính `content`, thì `::before` sẽ **không hiển thị bất kỳ thứ gì trên giao diện**!

Ví Dụ Minh Họa Sử Dụng ::before

```
<!-- VD 1: Khi khong dung ::before -->
<h2>Tieu de bai viet</h2>

<!-- Khi dung ::before ap dung vao CSS -->
<h2 class="title">Tieu de bai viet</h2>

<style>
  .title::before {
    content: "[ICON] ";
    color: orange;
    font-size: 24px;
  }
</style>
```

BROWSER PREVIEW — **Kết Quả Hiển Thị ::before**

[ICON] Tiêu đề bài viết

[Khái Niệm] VD 2 & Ứng Dụng Thực Tế Của ::before

Ứng dụng: Thêm biểu tượng, dấu hiệu nhấn mạnh, hoặc trang trí tiêu đề mà không cần thay đổi cấu trúc mã HTML gốc.

8.2. Bộ Chọn ::after (Chèn Nội Dung Vào Sau Phần Tử)

Cú Pháp Cơ Bản Của ::after

```
selector::after {
  content: "Noi dung them vao";
  /* Cac thuoc tinh CSS khac */
}
```

[Khái Niệm] Giải Thích Chi Tiết Cấu Trúc ::after

Chèn nội dung vào **SAU** nội dung bên trong của phần tử.

- `selector`: Phần tử mà bạn muốn thêm nội dung sau nó.
- `::after`: Thêm phần tử giả ngay sau nội dung bên trong phần tử.
- `content`: **Bắt buộc phải có**, nội dung muốn chèn thêm. Nếu không có `content`, `::after` không hiển thị gì cả!

Ví Dụ Minh Họa Sử Dụng ::after

```
/* VD khai bao CSS dang sau */
.after::after {
  content: "Noi dung dang sau";
}

<!-- Ap dung thuc te cho doan luu y -->
<p class="note">Luu y quan trong</p>

<style>
  .note::after {
    content: " [WARNING]";
    color: red;
    font-weight: bold;
  }
</style>
```

BROWSER PREVIEW — **Kết Quả Hiển Thị ::after**

Lưu ý quan trọng [WARNING]

[Khái Niệm] Ứng Dụng Thực Tế Của ::after

Ứng dụng: Thêm icon cảnh báo, tạo dấu kết thúc câu, tạo hiệu ứng đường kẻ gạch chân trang trí, hoặc tạo clearfix dọn dẹp float trong CSS.

8.3. Bộ Chọn ::first-letter (Định Dạng Ký Tự Đầu Tiên)

[Khái Niệm] Khái Niệm & Cấu Trúc Căn Bản Của ::first-letter

`::first-letter` là một pseudo-element selector trong CSS, dùng để định dạng **chữ cái đầu tiên** của phần tử văn bản dạng khối (block element).

Thường dùng để tạo **Drop Cap** (chữ cái lớn ở đầu đoạn văn như trong sách, báo in chuyên nghiệp).

Ví Dụ Đơn Giản Định Kiểu Chữ Cái Đầu

```
<p class="text">Hoc CSS rat thu vi va huu ich.</p>

<style>
  .text::first-letter {
    font-size: 36px;
    color: blue;
    font-weight: bold;
  }
</style>
```

BROWSER PREVIEW — **Kết Quả Hiển Thị**

Học CSS rất thú vị và hữu ích.

Kết quả: Chữ cái đầu "H" to hơn, màu xanh.

Cấu Trúc Cơ Bản Của ::first-letter

```
selector::first-letter {
  /* Cac thuoc tinh CSS ap dung cho chu cai dau tien */
}
```

[Khái Niệm] Giải Thích Chi Tiết ::first-letter

- `selector`: Chọn phần tử cần định dạng.
- `::first-letter`: Chọn chữ cái đầu tiên trong phần tử đó để áp dụng style.

Ví Dụ Minh Họa Chi Tiết Với Ký Tự Đầu

```
<p class="intro">Chao mung ban den voi the gioi CSS!</p>

<style>
  .intro::first-letter {
    font-size: 40px;
    color: red;
    font-weight: bold;
    margin-right: 5px;
  }
</style>
```

BROWSER PREVIEW — **Kết Quả Hiển Thị Trực Quan**

Chào mừng bạn đến với thế giới CSS!

[Khái Niệm] Phân Tích Các Thuộc Tính CSS Ảnh Hưởng Đến Ký Tự Đầu

Các thuộc tính áp dụng trong CSS sẽ ảnh hưởng trực tiếp đến chữ cái đầu tiên:

- `font-size: 40px` → Làm chữ cái đầu to lên.
- `color: red` → Đổi màu chữ thành đỏ.
- `font-weight: bold` → Chữ in đậm.
- `margin-right: 5px` → Tạo khoảng cách với ký tự tiếp theo.

[Cảnh Báo] Thuộc Tính CSS Phổ Biến Hỗ Trợ & Không Hỗ Trợ Cho ::first-letter

Không phải thuộc tính CSS nào cũng áp dụng được cho `::first-letter`! Đây là danh sách phân loại chính xác:

Các thuộc tính hợp lệ được hỗ trợ:

- **Kiểu chữ:** `font`, `font-size`, `font-weight`, `font-style`, `font-variant`
- **Màu sắc và nền:** `color`, `background`, `background-clip`, `background-origin`
- **Khoảng cách và viền:** `margin`, `padding`, `border`, `border-radius`
- **Text style:** `text-decoration`, `text-transform`, `letter-spacing`, `line-height`

Không hỗ trợ (hoặc bị hạn chế trong hầu hết ngữ cảnh tiêu chuẩn): `box-shadow`, `width`, `height`, `float`, `position`, `transform`...

●●● Ứng Dụng Thực Tế 1: Tạo Hiệu Ứng Chữ Cái Đầu Lớn Như Sách Cổ (Classic Drop Cap)

```
p::first-letter {
  font-size: 60px;
  color: #8B0000;
  float: left;
  margin-right: 10px;
  font-family: "Georgia", serif;
}
```

●●● Ứng Dụng Thực Tế 2: Trang Trí Tiêu Đề Bắt Mắt

```
h2::first-letter {
  font-size: 50px;
  text-transform: uppercase;
  color: #FF4500;
}
```

●●● Ứng Dụng Thực Tế 3: Tạo Điểm Nhấn Nền Vàng Cho Đoạn Văn

```
article::first-letter {
  font-size: 30px;
  background-color: #FFD700;
  padding: 5px;
  border-radius: 3px;
}
```

[Khái Niệm] Kết Luận Về ::first-letter

- `::first-letter` cực kỳ hữu ích để làm nổi bật chữ cái đầu tiên.
- Thường dùng trong bài viết, sách, hoặc blog để tạo điểm nhấn nghệ thuật.
- Kết hợp với các thuộc tính như `float`, `margin` có thể tạo ra hiệu ứng chữ cái lớn (drop cap) như trong sách in truyền thống!

8.4. Bộ Chọn ::first-line (Định Dạng Dòng Đầu Tiên Của Phần Tử Khối)

[Khái Niệm] Bản Chất & Cơ Chế Hoạt Động Của ::first-line

Bộ chọn `::first-line` nhắm đến **dòng đầu tiên** của phần tử văn bản dạng khối (block element).

Cấu trúc khai báo tổng quát:

Cấu Trúc Khai Báo ::first-line

```
selector::first-line {
  color: blue;
  font-weight: bold;
  font-size: 18px;
}
```

[Cảnh Báo] Phân Loại Thuộc Tính Hỗ Trợ & Không Áp Dụng Được Cho ::first-line

Do yếu tố tối ưu thuật toán dàn trang (Reflow / Repaint), trình duyệt **chỉ cho phép** áp dụng các thuộc tính liên quan đến văn bản và màu nền cho `::first-line`:

Các thuộc tính được phép hỗ trợ:

- **Màu sắc & Kiểu chữ:** `color`, `font`, `font-size`, `font-weight`, `font-family`, `font-style`, `font-variant`.
- **Định dạng văn bản:** `letter-spacing`, `word-spacing`, `text-transform`, `text-decoration`, `line-height`, `clear`.
- **Màu nền:** `background`, `background-color`.

Các thuộc tính KHÔNG áp dụng được: `margin`, `padding`, `border`, `width`, `height`, `position`, `float`...

Ví Dụ Minh Họa Định Kiểu Dòng Đầu Tiên

```
<p class="text">
  Day la noi dung vi du de minh hoa cach hoat dong cua pseudo-element ::
    first-line.
  Dong dau tien se co mau xanh va in dam.
</p>

<style>
  .text::first-line {
    color: blue;
    font-weight: bold;
    font-size: 18px;
  }
</style>
```

BROWSER PREVIEW — **Kết Quả Hiển Thị ::first-line Trực Quan**

ĐÂY LÀ NỘI DUNG VÍ DỤ ĐỂ MINH HỌA CÁCH HOẠT ĐỘNG CỦA PSEUDO-ELEMENT ::FIRST-LINE.

Dòng đầu tiên sẽ có màu xanh, in đậm và phóng to chữ, nhưng các dòng sau không bị ảnh hưởng.

[Mẹo] 2 Lưu Ý Quan Trọng Khi Sử Dụng ::first-line

1. **Ảnh hưởng bởi Responsive Viewport:** Số lượng từ nằm trong ”dòng đầu tiên” phụ thuộc hoàn toàn vào chiều rộng màn hình. Khi co giãn cửa sổ trình duyệt, dòng đầu tiên sẽ tự động thay đổi các từ thuộc về nó.
2. **Không hoạt động trên phần tử inline:** Chỉ hoạt động trên các phần tử dạng khối (`display: block`, `inline-block`, `list-item`) như `<p>`, `<div>`, `<article>`. KHÔNG hoạt động trên thẻ inline như `<span>`, `<a>`.

[Khái Niệm] Ứng Dụng Thực Tế Của ::first-line

- **Tạo điểm nhấn cho đoạn văn bản:** Làm nổi bật câu mở đầu trong bài viết, báo chí hoặc đoạn nội dung dài.
- **Thiết kế tiêu đề sáng tạo:** Làm dòng đầu tiên ấn tượng hơn các dòng theo sau.

8.5. Bộ Chọn ::selection (Tùy Chỉnh Văn Bản Khi Người Dùng Bôi Đen)

[Khái Niệm] Định Nghĩa & Cấu Trúc Căn Bản Của ::selection

`::selection` là một pseudo-element selector dùng để định dạng (**styling**) phần nội dung mà người dùng bôi đen (**highlight**) trên trang web bằng chuột hoặc thao tác vuốt cảm ứng!

Cú pháp chuẩn:

Cú Pháp Định Kiểu ::selection

```
::selection {
  background: #ffeb3b; /* Mau nen vang sang */
  color: #d50000;      /* Mau chu do dam */
}
```

Ví Dụ Minh Họa Nâng Cao Với ::selection

```
<p class="custom-select">Hay boi den doan van ban nay de xem hieu ung cua ::
    selection!</p>

<style>
  .custom-select::selection {
    background: #ffeb3b;
    color: #d50000;
    text-shadow: 1px 1px 2px #000;
  }
</style>
```

BROWSER PREVIEW — **Kết Quả Hiển Thị Khi Bôi Đen Nội Dung**

Khi bạn bôi đen đoạn văn bản, nó sẽ hiển thị: **Nền vàng sáng, chữ đỏ đậm, bóng chữ**.

[Bảng] Bảng Phân Loại Thuộc Tính Hỗ Trợ Cho ::selection

Nhóm thuộc tính	Danh sách thuộc tính được hỗ trợ
Được hỗ trợ [Có]	`color` (Màu chữ), `background` / `background-color` (Màu nền), `text-shadow` (Đổ bóng chữ), `background-clip` (Kiểm soát vùng nền).
Không hỗ trợ [Không]	`border`, `padding`, `margin`, `box-shadow`, `font-size`...

[Khái Niệm] 3 Ứng Dụng Thực Tế Phổ Biến Của ::selection

1. **Tạo trải nghiệm người dùng tốt hơn (UX):** Giúp nội dung bôi đen dễ nhìn, nổi bật và dịu mắt.
2. **Đồng bộ nhận diện thương hiệu:** Dùng màu sắc chủ đạo của thương hiệu khi người dùng bôi đen văn bản.
3. **Mẹo chống sao chép nội dung (tạm thời):** Đổi màu chữ thành trắng trên màu nền trắng khi bôi đen để gây khó khăn nhẹ cho việc xem nội dung bị copy.

8.6. Bộ Chọn ::marker (Tùy Chỉnh Ký Hiệu Đầu Dòng Danh Sách `<li>`)

[Khái Niệm] Định Nghĩa & Đối Tượng Nhắm Tới Của ::marker

`::marker` chọn phần ký hiệu đánh dấu của danh sách, bao gồm:

- Dấu chấm tròn (●), ô vuông trong danh sách không thứ tự (`<ul>`).
- Số thứ tự (1, 2, 3...) trong danh sách có thứ tự (`<ol>`).
- Ký hiệu tùy chỉnh tạo bởi `list-style-type` hoặc thuộc tính `content`.

[Bảng] Phân Tích 3 Trường Hợp Cú Pháp Bộ Chọn ::marker

Cú pháp bộ chọn	Cách đọc & Ý nghĩa phạm vi chọn
`li::marker`	**Trường hợp 1:** Chọn phần marker của tất cả thẻ `<li>` trong mọi danh sách trên trang web.
`ul li::marker`	**Trường hợp 2:** Chỉ chọn phần marker (dấu chấm •) của thẻ `<li>` nằm bên trong danh sách không có thứ tự (`<ul>`).
`ol li::marker`	**Trường hợp 3:** Chỉ chọn phần marker (số thứ tự 1, 2, 3) của thẻ `<li>` nằm bên trong danh sách có thứ tự (`<ol>`).

3 Ví dụ minh họa thực tế tùy chỉnh ::marker:

Ví Dụ 1: Tùy Chỉnh Màu Sắc Và Kích Thước Marker Mặc Định

```
<ul>
  <li>Muc 1</li>
  <li>Muc 2</li>
</ul>

<style>
  li::marker {
    color: blue;
    font-size: 24px;
  }
</style>
```

Ví Dụ 2: Thay Thế Marker Bằng Icon Hoặc Biểu Tượng Mũi Tên

```
<ul class="icon-list">
  <li>Hoc HTML5</li>
  <li>Hoc CSS3</li>
</ul>

<style>
  .icon-list li::marker {
    content: "-> ";
    color: orange;
    font-size: 18px;
  }
</style>
```

Ví Dụ 3: Kết Hợp Danh Sách Có Thứ Tự Với CSS Counter (Bước 1, Bước 2...)

```
<ol class="step-list">
  <li>Phan dau tien</li>
  <li>Phan thu hai</li>
  <li>Phan thu ba</li>
</ol>

<style>
  .step-list li::marker {
    content: "Buoc " counter(list-item) ": ";
    color: purple;
    font-weight: bold;
  }
</style>
```

BROWSER PREVIEW — **Kết Quả Hiển Thị Ví Dụ 3: Bước 1, Bước 2,...**

Bước 1: Phần đầu tiên
Bước 2: Phần thứ hai
Bước 3: Phần thứ ba

[Bảng] Phân Loại Thuộc Tính Hỗ Trợ & Không Hỗ Trợ Cho ::marker

Loại thuộc tính	Danh sách chi tiết
Được hỗ trợ [Có]	`color`, `font`, `font-size`, `font-weight`, `font-family`, `letter-spacing`, `text-transform`, `content` (thay thế marker).
Không hỗ trợ [Không]	`margin`, `padding`, `background`, `border`, `width`, `height`...

[Cảnh Báo] 3 Lưu Ý Vàng Khi Sử Dụng ::marker

1. **Phải là phần tử danh sách:** `::marker` chỉ hoạt động với các phần tử có thuộc tính `display: list-item`.
2. **Ảnh hưởng toàn bộ danh sách:** Quy tắc `li::marker` sẽ áp dụng cho tất cả các mục. Nếu muốn marker riêng cho từng mục, bạn cần thêm class riêng cho từng thẻ `<li>`.
3. **Hỗ trợ trình duyệt hiện đại:** Tất cả các trình duyệt hiện đại (Chrome, Firefox, Edge, Safari) đều đã hỗ trợ hoàn hảo `::marker`.

[Khái Niệm] Ứng Dụng Thực Tế Của ::marker

- **Tạo danh sách đẹp mắt:** Dùng icon, biểu tượng mũi tên hoặc ký hiệu đặc biệt làm dấu đầu dòng.
- **Hướng dẫn từng bước:** Thay số thứ tự thành ”Bước 1:”, ”Bước 2:”, v.v.
- **Điểm nhấn thiết kế:** Làm nổi bật danh sách sản phẩm/tính năng mà không cần thêm thẻ HTML phức tạp.

8.7. Bảng Tổng Hợp Cheatsheet & Nguyên Tắc Thiết Kế Thực Tế

[Bảng] Bảng Cheatsheet Tra Cứu Toàn Diện 6 Pseudo-Element Selectors

Pseudo-element	Chức năng chính	Thuộc tính hỗ trợ	Ứng dụng thực tế tiêu biểu
`::before`	Chèn nội dung trước phần tử	`content, color, font, layout`	Thêm icon, dấu hiệu, nhãn trang trí trước tiêu đề hoặc nút bấm.
`::after`	Chèn nội dung sau phần tử	`content, color, font, layout`	Thêm chú thích, hiệu ứng kết thúc, hoặc ký hiệu cảnh báo, clearfix.
`::first-letter`	Chọn chữ cái đầu tiên	`font-size, color, float, margin`	Tạo drop cap (chữ đầu lớn) nghệ thuật như trong sách, báo in.
`::first-line`	Chọn dòng đầu tiên	`font, color, text-transform`	Làm nổi bật dòng mở đầu bài viết blog, báo chí.
`::selection`	Chọn nội dung được bôi đen	`background, color, text-shadow`	Tạo hiệu ứng đặc biệt khi người dùng chọn văn bản.
`::marker`	Chọn ký hiệu danh sách	`color, content, font-size`	Tùy chỉnh bullet, số thứ tự trong danh sách `<li>`.

[Khái Niệm] Tổng Kết & Ứng Dụng Thực Tế Nâng Cao

- **Tối ưu HTML:** Giảm số lượng thẻ HTML không cần thiết, giữ mã nguồn sạch sẽ và chuẩn W3C semantic.
- **Nâng cao UX/UI:** Tạo hiệu ứng tương tác thị giác đẹp mắt, tăng trải nghiệm người dùng khi đọc và tương tác.
- **Dễ bảo trì mã nguồn:** Chỉ cần thay đổi CSS trong một file duy nhất mà không cần sửa cấu trúc mã HTML hàng loạt.

Ví dụ thực tiễn: Khi làm menu điều hướng hoặc card sản phẩm, bạn có thể dùng `::before` để thêm biểu tượng icon nhỏ, hoặc `::selection` để tạo trải nghiệm đọc thương hiệu thú vị hơn.

8.8. Đọc Thêm: Chuyên Đề Nâng Cao Về Pseudo-Elements (Mở Rộng Kiến Thức Thực Chiến)

[Khái Niệm] 1. Các Giá Trị Đa Dạng Của Thuộc Tính content Trong ::before / ::after

Thuộc tính `content` trong `::before` và `::after` không chỉ nhận chuỗi văn bản đơn thuần, mà còn hỗ trợ nhiều kiểu giá trị cực kỳ mạnh mẽ:

- `content: "Chuoi van ban";`: Chèn văn bản trực tiếp.
- `content: "";`: Chuỗi rỗng (Thường kết hợp với `display: block/inline-block`, `width`, `height`, `background` để vẽ các hình khối trang trí, đường kẻ, hoặc overlay).
- `content: attr(attribute-name);`: Lấy giá trị thuộc tính HTML động. Rất phổ biến để tạo Tooltip hoặc hiển thị đường dẫn URL khi in trang web.
- `content: url(image.png);`: Chèn hình ảnh trực tiếp.
- `content: counter(name);`: Tạo bộ đếm số tự động trong CSS (CSS Counters).

Ví Dụ Ứng Dụng content: attr(data-tooltip) Để Tạo Tooltip Thuần CSS

```
<button class="btn" data-tooltip="Nhan vao day de luu du lieu">Luu bai viet</button>

<style>
  .btn { position: relative; }
  .btn:hover::after {
    content: attr(data-tooltip);
    position: absolute;
    bottom: 100%;
    left: 50%;
    transform: translateX(-50%);
    background: #333;
    color: #fff;
    padding: 4px 8px;
    border-radius: 4px;
    white-space: nowrap;
    font-size: 12px;
  }
</style>
```

[Bảng] 2. Bảng So Sánh Chi Tiết: Pseudo-class (:) vs Pseudo-element (::)

Tiêu chí	Pseudo-class (:)	Pseudo-element (::)
Cú pháp tiêu chuẩn CSS3	Sử dụng 1 dấu hai chấm (`:hover`)	Sử dụng 2 dấu hai chấm (`::before`)
Bản chất tác động	Chọn phần tử dựa trên **trạng thái tương tác** hoặc **vị trí vị thế DOM**.	Định kiểu cho một **phần nội dung cụ thể** hoặc tạo **phần tử ảo**.
Sự tồn tại trong DOM	Nhắm trực tiếp vào thẻ HTML thực tế đã có trong cây DOM.	Tạo phần tử ảo hiển thị trên màn hình nhưng KHÔNG tồn tại trong cây DOM.
Ví dụ tiêu biểu	`:hover`, `:focus`, `:nth-child(2)`	`::before`, `::after`, `::first-letter`

[Cảnh Báo] 3. Lưu Ý Quan Trọng Về Accessibility (Khả Năng Truy Cập - a11y) & Hiệu Năng

- **Công cụ đọc màn hình (Screen Readers):** Một số công cụ đọc màn hình có thể đọc nội dung trong thuộc tính `content:"..."` của `::before` và `::after`. Tránh chèn các ký tự trang trí không có nghĩa ngữ văn bản (ví dụ: `content: ">>> ";`) vì có thể gây phiền cho người khiếm thị. Nếu nội dung chỉ mang tính trang trí, hãy cân nhắc áp dụng thuộc tính `aria-hidden="true"`.
- **Modern CSS Syntax for Alt Text:** Trong CSS Modern, bạn có thể truyền văn bản thay thế alt trực tiếp trong `content`: `content: "STAR" / "Bieu tuong ngoi sao";`.
- **Hiệu năng Rendering:** Các pseudo-elements được xử lý rất nhanh bởi GPU của trình duyệt. Tuy nhiên khi kết hợp animation biến đổi (`transform`, `opacity`) trên `::before`/`::after`, hãy luôn ưu tiên sử dụng thuộc tính `will-change` để tránh hiện tượng giật khung hình (**Jank**).

[Mẹo] 4. Bộ Câu Hỏi Phỏng Vấn Tuyển Dụng Frontend Chuyên Sâu (Interview Q&A)

1. **Q1: Phân biệt sự khác biệt cốt lõi giữa 1 dấu hai chấm (:) và 2 dấu hai chấm (::) trong CSS?**
   → **Trả lời ghi điểm**: Dấu single colon (`:`) dùng cho Pseudo-classes đại diện cho trạng thái tương tác của phần tử có sẵn trong DOM (như `:hover`, `:focus`, `:nth-child`). Dấu double colon (`::`) dùng cho Pseudo-elements đại diện cho phần tử ảo không có trong DOM hoặc một phần nội dung cụ thể (như `::before`, `::after`, `::first-letter`).
2. **Q2: Tại sao không thể sử dụng ::before hay ::after trên các thẻ <img>, <input>,
?**
   → **Trả lời ghi điểm**: Vì các thẻ như `<img>`, `<input>`, `<br>` là các phần tử thay thế (**Replaced Elements** / **Void Elements**). Chúng không chứa phần nội dung con bên trong (no child content), do đó trình duyệt không có vị trí con đầu tiên (first child) hay con cuối cùng (last child) để chèn phần tử giả `::before` hay `::after`.
3. **Q3: Điều gì xảy ra nếu bạn quên thuộc tính content khi khai báo ::before?**
   → **Trả lời ghi điểm**: Thuộc tính `content` là điều kiện tiên quyết bắt buộc. Nếu không có `content` (hoặc nếu set `content: none`), trình duyệt sẽ bỏ qua và không khởi tạo phần tử giả đó trên cây Render Tree.
4. **Q4: Kỹ thuật Micro Clearfix giải quyết vấn đề gì bằng ::after?**
   → **Trả lời ghi điểm**: Giải quyết hiện tượng sụp đổ chiều cao (**Height Collapse**) của thẻ cha khi các thẻ con dùng `float`. Bằng cách chèn `::after` có `content: ""; display: table; clear: both;`, thẻ cha tự động bao bọc trọn vẹn toàn bộ chiều cao của các thẻ con float mà không cần chèn thẻ HTML rỗng.
5. **Q5: Làm thế nào để đặt một phần tử giả ::before nằm phía sau background của thẻ cha?**
   → **Trả lời ghi điểm**: Thiết lập `position: relative` và `z-index: 1` (hoặc `isolation: isolate`) trên thẻ cha để tạo một Stacking Context độc lập, sau đó thiết lập `position: absolute` và `z-index: -1` trên `::before`.

8.9. Phân Tích & Xử Lý Xung Đột Khi Sử Dụng Pseudo-elements (CSS Conflicts Masterclass)

Khi phát triển giao diện Web phức tạp, việc khai báo nhiều quy tắc CSS cho cùng một bộ chọn phần tử giả có thể gây ra xung đột thị giác. Dưới đây là phân tích chi tiết 4 dạng xung đột phổ biến nhất cùng giải pháp xử lý triệt để:

[Khái Niệm] 1. Xung Đột Khi Khai Báo Nhiều Lần Cho Cùng Một Pseudo-element

Nguyên lý duy nhất (Single Instance Rule): Mỗi phần tử HTML chỉ sở hữu duy nhất **một instance (thể hiện ảo)** của từng loại phần tử giả (`::before` hoặc `::after`).

Cơ chế xung đột (Cascade Override): Nếu bạn khai báo nhiều khối CSS cùng nhắm vào `.class::before`, trình duyệt sẽ áp dụng quy tắc Cascading mặc định: **Quy tắc xuất hiện sau cùng trong mã nguồn sẽ ghi đè hoàn toàn quy tắc trước đó**!

Ví Dụ Xung Đột Khai Báo Nhiều Khối ::before

```html
<p class="text">Chao ban!</p>

<style>
  /* Quy tac 1: Bi ghi de */
  .text::before {
    content: "Xin chao! ";
    color: red;
  }
  /* Quy tac 2: Chien thang (viet sau) */
  .text::before {
    content: "Hello! ";
    color: blue;
  }
</style>
```

BROWSER PREVIEW **Kết Quả Hiển Thị: Quy Tắc Viết Sau Ghi Đè Quy Tắc Trước**

Kết quả: **Hello!** Chào bạn!

Giải thích: Quy tắc thứ hai (.text::before có content: "Hello! ") được viết sau nên ghi đè quy tắc thứ nhất.

[Mẹo] Giải Pháp Xử Lý Xung Đột Khai Báo Trùng

Giải pháp: Gộp tất cả thuộc tính định kiểu vào một khối CSS duy nhất hoặc sử dụng các selector có độ đặc hiệu (Specificity) cao hơn để phân biệt ngữ cảnh rõ ràng:

Mã Nguồn Giải Pháp Gộp Style Chuẩn

```css
.text::before {
  content: "Xin chao! ";
  color: red;
  font-weight: bold;
}
```

[Khái Niệm] 2. Xung Đột Khi Thẻ Cha Mang Thuộc Tính display: none

Hiện tượng: `::before` và `::after` tạo ra nội dung ảo nằm **bên trong** phần tử cha trên Render Tree.

Hệ quả: Nếu thẻ cha bị ẩn bằng thuộc tính `display: none`, toàn bộ phần tử giả con bên trong cũng sẽ **biến mất 100%** khỏi màn hình hiển thị, bất kể bạn có cố gắng đặt `display: block` hay `visibility: visible` riêng cho phần tử giả!

Ví Dụ Thẻ Cha display: none Làm Biến Mất Pseudo-element

```html
<p class="hidden-box">Noi dung bai viet</p>

<style>
  p.hidden-box {
    display: none; /* An toan bo the cha */
  }
  p.hidden-box::before {
    content: "Khong thay toi dau!";
    color: red;
  }
</style>
```

BROWSER PREVIEW — **Kết Quả Hiển Thị: Không Có Gì Hiển Thị**

(Không hiển thị bất kỳ thành phần nào trên màn hình vì thẻ cha p bị ẩn hoàn toàn bởi display: none)

[Mẹo] Giải Pháp Khi Cần Ẩn Nội Dung Thật Nhưng Giữ Phần Tử Giả

Giải pháp: Nếu muốn ẩn nội dung thật của thẻ cha mà vẫn duy trì phần tử giả hiển thị, bạn có thể thiết lập `font-size: 0` trên thẻ cha và đặt lại `font-size` chuẩn trên phần tử giả, hoặc dùng thuộc tính `visibility: hidden` kết hợp `visibility: visible` trên phần tử giả!

[Khái Niệm] 3. Xung Đột Tương Tác Giữa ::first-letter / ::first-line Với ::before / ::after

Nguyên lý tương tác: Bộ chọn `::first-letter` và `::first-line` được thiết kế để định dạng **văn bản thực tế** trong thẻ HTML. Chúng **KHÔNG tự động tác động** lên ký tự hoặc nội dung ảo được chèn từ thuộc tính `content` của `::before`!

Ví Dụ ::first-letter Không Ảnh Hưởng Đến Nội Dung Ảo Từ ::before

```
<p class="intro">Chao mung ban!</p>

<style>
  p.intro::before {
    content: "STAR "; /* Noi dung ao */
  }
  p.intro::first-letter {
    color: red;
    font-size: 30px;
  }
</style>
```

BROWSER PREVIEW — **Kết Quả Hiển Thị: ::first-letter Định Kiểu Cho Chữ C**

STAR **C**hào mừng bạn!

Giải thích: ::first-letter tác động vào chữ cái đầu "C" của nội dung thật, không phải chữ "S" trong nội dung giả "STAR".

[Mẹo] Giải Pháp Định Kiểu Riêng Cho Nội Dung Giả

Giải pháp: Nếu muốn định dạng kiểu chữ, màu sắc hay kích thước cho nội dung giả từ `::before`, hãy trực tiếp khai báo các thuộc tính đó bên trong khối `::before`:

Khai Báo Định Kiểu Trực Tiếp Trong ::before

```
p.intro::before {
  content: "STAR ";
  color: green;
  font-size: 30px;
  font-weight: bold;
}
```

[Khái Niệm] 4. Xung Đột Thứ Tự CSS Về Độ Ưu Tiên (Specificity Conflict)

Khi hai bộ chọn cùng nhắm tới phần tử giả của một thẻ (ví dụ: `h1::before` và `h1.special::before`), trình duyệt sẽ tính toán điểm **CSS Specificity** để quyết định quy tắc nào chiến thắng.

Ví Dụ Bộ Chọn Cụ Thể Hơn Chiến Thắng

```
<h1 class="special">Tieu de trang web</h1>

<style>
  /* Specificity: (0,0,0,2) - Class va Element */
  h1.special::before {
    content: "[SPECIAL] ";
    color: blue;
  }

  /* Specificity: (0,0,0,1) - Element */
  h1::before {
    content: "[DEFAULT] ";
    color: orange;
  }
</style>
```

BROWSER PREVIEW — **Kết Quả Hiển Thị: Specificity Cao Hơn Chiến Thắng**

[SPECIAL] Tiêu đề trang web

Giải thích: Thẻ h1 có class "special" nên bộ chọn h1.special::before (điểm cao hơn) chiến thắng.

[Bảng] Bảng Tra Cứu Toàn Diện Các Dạng Xung Đột, Nguyên Nhân & Giải Pháp Xử Lý

Pseudo-element	Bản chất đối tượng	Nguyên nhân xung đột thường gặp	Giải pháp xử lý chuẩn mực
`::before / ::after`	Nội dung ảo chèn vào đầu/cuối phần tử	Trùng lặp khai báo nhiều khối CSS; bị ẩn khi thẻ cha dùng `display: none`.	Dùng selector cụ thể hơn để phân tách; điều chỉnh logic hiển thị bằng `visibility`.
`::first-letter`	Chữ cái đầu tiên của văn bản thật	Không tự động tác động lên nội dung ảo của `::before`.	Khai báo định kiểu font/color trực tiếp trong khối `::before`.
`::first-line`	Dòng đầu tiên của văn bản khối	Không áp dụng được các thuộc tính layout như `margin`, `padding`, `width`.	Chỉ dùng các thuộc tính văn bản/màu sắc hợp lệ (`color`, `font`, `line-height`).
`Thứ tự CSS (Cascade)`	Quy tắc ghi nguồn từ trên xuống dưới	Khai báo sau ghi đè khai báo trước nếu cùng Specificity.	Kiểm tra thứ tự nạp file CSS, gộp thuộc tính trùng lặp.
`Độ ưu tiên (Specificity)`	Điểm số bộ chọn trình duyệt tính toán	Class mạnh hơn Element, ID mạnh hơn Class.	Nắm vững công thức Specificity; hạn chế tối đa việc lạm dụng `!important`.

[Mẹo] 3 Lời Khuyên Thực Chiến Tối Ưu Quản Lý Pseudo-elements

1. **Viết CSS rõ ràng, mô-đun hóa:** Đừng chồng chéo quá nhiều phần tử giả không cần thiết trên cùng một thẻ HTML.
2. **Kiểm tra điểm Specificity:** Khi thấy phần tử giả không hiển thị đúng màu hoặc content, hãy kiểm tra xem có selector nào mạnh hơn đang ghi đè hay không.
3. **Thành thạo công cụ Chrome DevTools:** Mở tab *Elements* trên trình duyệt, click mở rộng thẻ HTML target để soi trực tiếp phần tử ảo `::before` / `::after` và kiểm tra các dòng CSS đang bị gạch ngang (ghi đè).

9.6 Thứ Tự Ưu Tiên Trong CSS (CSS Specificity Masterclass)

Trong thiết kế giao diện Web thực tế, một phần tử HTML thường nằm trong phạm vi ảnh hưởng của nhiều quy tắc CSS khác nhau (ví dụ: vừa có style từ thẻ `p`, vừa có class `.text`, vừa có ID `#main`, lại vừa có thuộc tính `style="..."` trực tiếp). Khi xảy ra xung đột giữa các quy tắc này, trình duyệt sẽ dùng cơ chế **CSS Specificity** (Thứ tự ưu tiên / Tính đặc hiệu) để tính toán điểm số và quyết định quy tắc nào sẽ chiến thắng.

[Khái Niệm] Khái Niệm Specificity (Tính Đặc Hiệu) Là Gì?

CSS Specificity (Tính đặc hiệu) là thuật toán và hệ thống quy tắc mà trình duyệt sử dụng để quyết định quy tắc CSS nào sẽ được áp dụng cho một phần tử HTML khi có nhiều quy tắc cùng nhắm vào phần tử đó.

Vì sao cần Specificity? Khi trang web phát triển lớn, một phần tử (ví dụ: nút bấm `<button id="submit" class="btn">`) thường khớp với nhiều quy tắc CSS khác nhau. Specificity giúp trình duyệt tính toán điểm số công bằng để chọn ra quy tắc mạnh nhất dựa trên độ cụ thể của bộ chọn, **không phụ thuộc vào thứ tự viết code** (trừ khi hai bộ chọn có điểm số bằng nhau).

9.6.1 1. Bảng Xếp Hạng Chi Tiết Độ Ưu Tiên Specificity (6 Bậc Giá Trị)

[Bảng] Bảng Xếp Hạng Chi Tiết Độ Ưu Tiên Trong CSS

Cấp độ	Loại bộ chọn (Selector)	Cú pháp minh họa	Specificity	Đặc tính & Độ mạnh
(1)	**!important trong CSS**	`color: red !important;`	—	**[Cao nhất]** – Ghi đè tất cả (trừ inline style có !important)
(2)	**Style nội tuyến (Inline style)**	`<div style="color:red;">`	(1,0,0,0)	**Nội tuyến** – Rất mạnh, ghi đè mọi ID và Class
(3)	**Bộ chọn ID (#id)**	`#header { color: blue; }`	(0,1,0,0)	**ID Selector** – Rất mạnh, nhưng hạn chế lạm dụng
(4)	**Class, Attribute, Pseudo-class**	`.btn`, `[type="text"]`, `:hover`	(0,0,1,0)	**Chung nhóm** – Linh hoạt và được sử dụng phổ biến nhất
(5)	**Thẻ HTML & Pseudo-element**	`div`, `h1`, `p`, `::before`	(0,0,0,1)	**Thẻ Element** – Mức độ yếu nhất trong các bộ chọn
(6)	**Bộ chọn chung & Kế thừa**	`* { margin: 0; }`, `body { color }`	—	**Mặc định** – Yếu nhất, dễ dàng bị bất kỳ bộ chọn nào ghi đè

9.6.2 2. Quy Trình 4 Bước Duyệt DOM & Công Thức (A, B, C, D) / (a, b, c, d)

Trình duyệt thực hiện quá trình tính toán và áp dụng kiểu dáng theo 4 bước tiêu chuẩn:

[Khái Niệm] 4 Bước Xử Lý Thứ Tự Ưu Tiên Của Trình Duyệt

Bước 1: Thu thập tất cả các quy tắc CSS áp dụng cho phần tử.
Ví dụ: Với thẻ `<p id="main" class="text">`, trình duyệt gom tất cả các quy tắc khớp với phần tử này như `p`, `.text`, `#main`, `p.text`, `#main.text`...

Bước 2: Tính điểm specificity cho từng bộ chọn theo dạng 4 nhóm (A, B, C, D) hoặc (a, b, c, d).

- **Nhóm A / a (1,0,0,0):** Style nội tuyến (Inline styles) đặt trực tiếp trong thuộc tính `style="..."`.
- **Nhóm B / b (0,1,0,0):** Số lượng ID selectors (`#id`).
- **Nhóm C / c (0,0,1,0):** Số lượng Class (`.text`), Attribute (`[type="text"]`), Pseudo-class (`:hover`, `:focus`, `:not()`).
- **Nhóm D / d (0,0,0,1):** Số lượng Element (thẻ HTML như `div`, `p`) và Pseudo-element (`::before`, `::after`).

Bước 3: So sánh specificity theo thứ tự ưu tiên $A > B > C > D$ (hoặc $a > b > c > d$).
Trình duyệt so sánh từng chỉ số từ trái sang phải (giống như so sánh số nguyên nhiều chữ số):

- `p` → Specificity = `(0,0,0,1)`
- `.text` → Specificity = `(0,0,1,0)`
- `#main` → Specificity = `(0,1,0,0)`
- `style="..."` → Specificity = `(1,0,0,0)`
- `p.text:hover` → Specificity = `(0,0,2,1)`

Bước 4: Áp dụng quy tắc có specificity cao nhất.

- Nếu nhiều quy tắc có **cùng điểm specificity**, quy tắc nào **viết sau trong file CSS sẽ thắng** (theo thứ tự xuất hiện **Source Order**).
- Nếu có từ khóa `!important`, trình duyệt sẽ cấp quyền ưu tiên tuyệt đối để đè qua tất cả chỉ số specificity thông thường.

[Bảng] Bảng Định Nghĩa 4 Thành Phần Của Công Thức (A, B, C, D) / (a, b, c, d)

Ký hiệu	Thành phần quy đổi	Giá trị mẫu	Quy tắc cộng điểm
A / a	**Style nội tuyến (Inline style)**	(1,0,0,0)	**+1** nếu phần tử được viết trực tiếp trong thuộc tính `style="..."`
B / b	**Số lượng ID selectors (`#id`)**	(0,1,0,0)	**+1** cho mỗi ID xuất hiện trong bộ chọn
C / c	**Class, Attribute, Pseudo-classes**	(0,0,1,0)	**+1** cho mỗi `.class`, `[attr]`, `:hover`, `:nth-child()`
D / d	**Thẻ HTML (Elements) & Pseudo-elements**	(0,0,0,1)	**+1** cho mỗi thẻ HTML (`div`, `p`) và `::before`, `::after`

[Ghi Chú] Lưu Ý Quan Trọng Khi Tính Điểm Specificity

- **Cách so sánh**: Trình duyệt so sánh từ trái sang phải: A lớn hơn thì thắng; nếu A bằng nhau thì so sánh B; nếu B bằng nhau thì so sánh C; nếu C bằng nhau thì so sánh D.
- **Không quy đổi thập phân**: Điểm $(0, 0, 1, 0)$ đại diện cho 1 Class luôn luôn mạnh hơn bất kỳ số lượng thẻ HTML nào (ví dụ $(0, 0, 0, 11)$ gồm 11 thẻ HTML vẫn thua 1 Class).
- **Universal Selector (`*`) và Combinators (`>`, `+`, `~`)**: Có điểm số là $(0, 0, 0, 0)$, không cộng thêm điểm specificity.
- **File Internal (`<style>`) vs External (`.css`)**: Vị trí khai báo CSS không làm thay đổi specificity, chỉ bị ảnh hưởng bởi specificity và thứ tự xuất hiện.

9.6.3 3. Bảng Tra Cứu Specificity Của Các Bộ Chọn Thường Gặp (15 Ví Dụ)

[Bảng] Bảng Tra Cứu Specificity Của Các Bộ Chọn CSS Phổ Biến (Tự Động Sắp Xếp Từ Thấp Đến Cao)

Bộ chọn CSS (Selector)	Specificity (A,B,C,D)	Phân tích chi tiết thành phần
`h1`	(0,0,0,1)	1 thẻ HTML (`h1`)
`div`	(0,0,0,1)	1 thẻ HTML (`div`)
`.box / .btn`	(0,0,1,0)	1 class (`.box` hoặc `.btn`)
`#header / #main`	(0,1,0,0)	1 ID (`#header` hoặc `#main`)
`div p`	(0,0,0,2)	2 thẻ HTML (`div` + `p`)
`p::first-line`	(0,0,0,2)	1 thẻ HTML (`p`) + 1 pseudo-element (`::first-line`)
`a:hover`	(0,0,1,1)	1 thẻ HTML (`a`) + 1 pseudo-class (`:hover`)
`input[type="text"]`	(0,0,1,1)	1 thẻ HTML (`input`) + 1 attribute (`[type]`)
`div:not(.active)`	(0,0,1,1)	1 thẻ HTML (`div`) + 1 class bên trong `:not()` (`.active`)
`.nav ul li a`	(0,0,1,3)	1 class (`.nav`) + 3 thẻ HTML (`ul`, `li`, `a`)
`#main .button / div#main .btn:hover`	(0,1,2,1)	1 ID (`#main`) + 2 class (`.btn`, `:hover`) + 1 thẻ (`div`)
`a[href][data-id].active:hover`	(0,0,4,1)	1 thẻ (`a`) + 2 attributes (`[href]`, `[data-id]`) + 2 class (`.active`, `:hover`)
`section#hero .content p`	(0,1,1,2)	1 ID (`#hero`) + 1 class (`.content`) + 2 thẻ (`section`, `p`)
`html body .wrapper #main-content p`	(0,1,1,3)	1 ID (`#main-content`) + 1 class (`.wrapper`) + 3 thẻ (`html`, `body`, `p`)
`style="color:red" (inline style)`	(1,0,0,0)	1 thuộc tính nội tuyến `style="..."`

9.6.4 4. Những Quy Tắc Đặc Biệt Trong Specificity

4.1. Từ Khóa !important Không Được Tính Vào Specificity Nhưng Ghi Đè Tất Cả

[Khái Niệm] Tại Sao !important Không Tính Vào Specificity?

`!important` là một quy tắc ưu tiên tuyệt đối trong thuật toán Cascade của trình duyệt, nhưng nó **không được cộng điểm vào bộ số Specificity** (A, B, C, D). Điều này có nghĩa là, dù một quy tắc có specificity thấp, nếu có `!important`, nó vẫn được áp dụng thay vì các quy tắc có specificity cao hơn nhưng không có `!important`.

Ví Dụ 1: Quy Tắc Specificity Cao Hơn Nhưng Không Có !important

```
/* Specificity: (0,1,0,1) - Khong co !important */
#content p {
  color: red;
}

/* Specificity: (0,0,0,1) - Thap hon nhung CO !important -> WINS! */
p {
  color: blue !important;
}
```

BROWSER PREVIEW — **Kết Quả Ví Dụ 1: Văn Bản <p> Có Màu Xanh (Blue)**

Đoạn văn hiển thị màu blue

Giải thích: #content p có specificity (0,1,0,1), p có specificity (0,0,0,1) thấp hơn. Nhưng nhờ từ khóa !important, quy tắc p { color: blue !important; } ghi đè tất cả và màu xanh (blue) chiến thắng.

Ví Dụ 2: Nếu Cả Hai Quy Tắc Đều Có !important

```
/* Specificity: (0,1,0,1) + !important -> WINS! */
#content p {
  color: red !important;
}

/* Specificity: (0,0,0,1) + !important -> LOSES! */
p {
  color: blue !important;
}
```

BROWSER PREVIEW — **Kết Quả Ví Dụ 2: Văn Bản <p> Có Màu Đỏ (Red)**

Đoạn văn hiển thị màu red

Giải thích: Khi cả hai quy tắc đều có !important, quyền ưu tiên tuyệt đối triệt tiêu lẫn nhau. Trình duyệt sẽ quay lại so sánh specificity: #content p có (0,1,0,1) cao hơn p (0,0,0,1), nên màu đỏ (red) chiến thắng.

4.2. Universal Selector (*) Và Combinators (>, +, ~) Không Ảnh Hưởng Đến Specificity

[Khái Niệm] Tại Sao *, >, +, ~ Không Ảnh Hưởng Đến Specificity?

- **Universal selector (*):** Là bộ chọn chung áp dụng cho mọi phần tử, nhưng không có giá trị specificity $(0, 0, 0, 0)$ vì nó không giúp phân biệt một phần tử cụ thể.
- **Combinators (>, +, ~):** Chỉ định quan hệ giữa các phần tử (con trực tiếp, anh em liền kề, anh em chung cha), nhưng không làm tăng specificity.

Ví Dụ 3: Universal Selector (*)

```
/* Specificity: (0,0,0,0) */
* {
  color: green;
}

/* Specificity: (0,0,0,1) - WINS! */
p {
  color: red;
}
```

BROWSER PREVIEW — **Kết Quả Ví Dụ 3: Thẻ <p> Có Màu Đỏ (Red)**

Đoạn văn hiển thị màu red

*Giải thích: * { color: green; } có specificity (0,0,0,0), p { color: red; } có specificity (0,0,0,1) cao hơn nên màu đỏ chiến thắng.*

Ví Dụ 4: Combinator (>) Không Làm Tăng Specificity

```
/* Specificity: (0,0,0,2) - 2 elements (div, p), combinator > co diem = 0 */
div > p {
  color: blue;
}

/* Specificity: (0,0,0,1) - 1 element */
p {
  color: red;
}
```

BROWSER PREVIEW — **Kết Quả Ví Dụ 4: div > p (0,0,0,2) Thắng p (0,0,0,1)**

Đoạn văn trong div có màu blue

Giải thích: div > p gồm 2 thẻ element (div và p) nên có specificity (0,0,0,2) cao hơn p (0,0,0,1). Bản thân dấu > có điểm = 0.

4.3. CSS Internal (<style>) Và External (.css File) Không Ảnh Hưởng Độ Ưu Tiên

[Khái Niệm] Tại Sao Cách Viết CSS (Internal Hay External) Không Ảnh Hưởng Độ Ưu Tiên?

Tất cả các quy tắc CSS (dù nằm trong thẻ `<style>` hay file `.css` bên ngoài) đều có cùng mức độ ưu tiên dựa trên specificity và vị trí xuất hiện trong tài liệu HTML. Nếu hai quy tắc có cùng specificity, quy tắc nào xuất hiện sau sẽ ghi đè quy tắc trước.

Ví Dụ 5: Internal CSS vs External CSS

```
<!-- File index.html -->
<head>
  <!-- Internal CSS viet TRUOC -->
  <style>
    p { color: red; }
  </style>

  <!-- External CSS duoc link SAU -->
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <p>Doan van nay co mau gi?</p>
</body>
```

BROWSER PREVIEW **Kết Quả Ví Dụ 5: Thẻ <p> Có Màu Xanh (Blue) Vì style.css Được Gọi Sau**

Đoạn văn hiển thị màu blue

Giải thích: Cả hai quy tắc p đều có specificity (0,0,0,1). Vì style.css được chèn sau thẻ <style>, quy tắc p { color: blue; } trong style.css sẽ ghi đè quy tắc trước đó.

[Bảng] Bảng Tóm Tắt Quy Tắc Đặc Biệt Trong Specificity

Quy tắc	Giải thích nguyên lý
`!important` không tính vào specificity	Nhưng nó ghi đè tất cả, trừ khi một quy tắc khác cũng có `!important` và có specificity cao hơn.
Universal selector (*) không ảnh hưởng	Vì nó không chọn phần tử cụ thể nào, điểm specificity bằng $(0, 0, 0, 0)$.
Combinators (>, +, ~) không ảnh hưởng	Vì chúng chỉ định quan hệ giữa phần tử nhưng không làm tăng điểm specificity.
Internal và External CSS không ảnh hưởng	Độ ưu tiên được quyết định strictly bởi specificity và thứ tự xuất hiện trong tài liệu DOM.

9.6.5 5. Phân Tích Chi Tiết: Khi Nào Quy Tắc "Viết Sau Đè Viết Trước" Đúng Hoặc Sai?

5.1. Khi Nào Quy Tắc Này ĐÚNG?

[Khái Niệm] Điều Kiện Để Quy Tắc Source Order Áp Dụng

Nếu hai quy tắc CSS có **cùng điểm specificity** (và không có `!important`), quy tắc nào xuất hiện sau trong mã nguồn CSS sẽ được áp dụng.

Ví Dụ 1: Hai Quy Tắc Có Cùng Specificity (0,0,0,1)

```
/* Viet truoc */
p { color: red; }

/* Viet SAU -> WINS! */
p { color: blue; }
```

BROWSER PREVIEW **Kết Quả: Thẻ <p> Có Màu Xanh (Blue)**

Đoạn văn có màu blue

Giải thích: Cả hai quy tắc p đều có specificity (0,0,0,1). Quy tắc p { color: blue; } xuất hiện sau nên ghi đè lên quy tắc trước.

5.2. Khi Nào Quy Tắc Này KHÔNG ĐÚNG Nữa? (3 Trường Hợp)

Có 3 trường hợp khiến quy tắc ”viết sau đè viết trước” hoàn toàn không còn đúng:

[Cảnh Báo] Trường Hợp 1: Khi Một Quy Tắc Có !important

`!important` **luôn ghi đè tất cả**, trừ khi quy tắc khác cũng có `!important` và có specificity cao hơn. Dù quy tắc có `!important` viết trước, nó vẫn thắng quy tắc viết sau!

Ví Dụ 2: !important Ghi Đè Quy Tắc Viết Sau

```
/* Viet TRUOC nhung co !important -> WINS! */
p { color: red !important; }

/* Viet SAU nhung khong co !important -> LOSES! */
p { color: blue; }
```

BROWSER PREVIEW **Kết Quả: Thẻ <p> Có Màu Đỏ (Red) Dù Quy Tắc Viết Sau Đặt Màu Blue**

Đoạn văn có màu red

Giải thích: Quy tắc p { color: red !important; } viết trước nhưng có !important nên chiến thắng hoàn toàn quy tắc p { color: blue; } viết sau.

[Cảnh Báo] Trường Hợp 2: Khi Một Quy Tắc Có Specificity Cao Hơn

Nếu một quy tắc có specificity cao hơn, nó sẽ ghi đè lên quy tắc có specificity thấp hơn, **dù xuất hiện trước hay sau**!

Ví Dụ 3: Specificity Cao Hơn Ghi Đè Quy Tắc Viết Sau

```
/* Specificity: (0,0,0,1) - Viet TRUOC */
p { color: red; }

/* Specificity: (0,1,0,1) - Viết SAU nhưng specificity CAO HƠN -> WINS! */
#content p { color: blue; }
```

BROWSER PREVIEW **Kết Quả: #content p (0,1,0,1) Thắng p (0,0,0,1)**

Đoạn văn trong #content có màu blue

Giải thích: #content p có specificity (0,1,0,1) cao hơn p (0,0,0,1). Dù bạn có đảo vị trí viết p sau #content p thì #content p vẫn luôn chiến thắng!

[Cảnh Báo] Trường Hợp 3: Khi CSS Nội Tuyến (Inline Styles) Được Sử Dụng

Inline styles (`style="..."`) có specificity cao nhất $(1, 0, 0, 0)$, trừ khi bị `!important` ghi đè. Nó luôn ghi đè mọi quy tắc trong file CSS bên ngoài bất kể thứ tự.

Ví Dụ 4: Inline Styles Ghi Đè CSS Bên Ngoài

```
<!-- Inline Style có Specificity = (1,0,0,0) -->
<p style="color: green;">Đoạn văn này có màu gì?</p>

<style>
  /* Specificity: (0,0,0,1) - Viet SAU nhung thua Inline Style */
  p { color: red; }
</style>
```

BROWSER PREVIEW **Kết Quả: Thẻ <p> Có Màu Xanh Lá (Green) Nhờ Inline Style**

Đoạn văn hiển thị màu green

Giải thích: style="color: green;" có specificity (1,0,0,0) cao hơn p { color: red; } (0,0,0,1), nên đoạn văn có màu green dù CSS bên ngoài đặt màu red.

[Bảng] Bảng Tổng Kết: Khi Nào Quy Tắc "Cái Nào Viết Sau Sẽ Áp Dụng" Đúng Hoặc Sai?

Trường hợp xét duyệt	Quy tắc sau được áp dụng?	Phân tích nguyên do
Hai quy tắc có cùng specificity, không có `!important`	ĐÚNG	Áp dụng nguyên tắc Source Order chuẩn.
Một quy tắc có `!important`, quy tắc kia không có	SAI	Quy tắc có `!important` luôn thắng.
Một quy tắc có specificity cao hơn	SAI	Quy tắc có specificity cao hơn luôn thắng.
Một quy tắc là Inline styles (`style="..."`)	SAI	Inline styles có điểm $(1,0,0,0)$ cao hơn.

9.6.6 6. Bài Tập & Ví Dụ Phân Tích Chuyên Sâu Độ Ưu Tiên

Ví Dụ Chuyên Sâu 1: Phân Tích Độ Ưu Tiên Danh Sách <li> Trong

Xét 2 bộ chọn đối lập nhắm vào danh sách:

Ví Dụ Danh Sách: li vs ul li

```
/* Specificity: (0,0,0,1) - 1 element */
li { color: blue; }

/* Specificity: (0,0,0,2) - 2 elements (ul + li) -> WINS! */
ul li { color: red; }
```

BROWSER PREVIEW — **Kết Quả: Tất Cả Thẻ <li> Đều Có Màu Đỏ (Red)**

1. Mục danh sách thứ nhất (Red)
2. Mục danh sách thứ hai (Red)

Giải thích: ul li có specificity (0,0,0,2) cao hơn li (0,0,0,1). Do đó tất cả thẻ <li> bên trong <ul> sẽ hiển thị màu đỏ.

[Khái Niệm] Làm Thế Nào Để Đổi Sang Màu Xanh (Blue)? (3 Cách Xử Lý)

1. **Cách 1: Đổi thứ tự CSS?** Đảo `ul li { color: red; }` lên trước và `li { color: blue; }` xuống sau? **Vẫn có màu đỏ!** Vì `ul li` $(0, 0, 0, 2) > li(0, 0, 0, 1)$, specificity cao hơn luôn thắng bất kể vị trí!
2. **Cách 2: Tăng specificity cho bộ chọn muốn đổi?** Viết `ul > li { color: blue; }`? Specificity của `ul > li` vẫn là $(0, 0, 0, 2)$ bằng với `ul li`. Nếu viết `ul > li` sau `ul li`, màu xanh sẽ thắng nhờ Source Order.
3. **Cách 3: Sử dụng !important?** Đặt `li { color: blue !important; }` sẽ ngay lập tức ghi đè màu đỏ thành màu xanh!

Ví Dụ Chuyên Sâu 2: Chuỗi 10 Cấp Độ Leo Thang Specificity Tăng Dần

Để hiểu thấu đáo bản chất thuật toán so sánh Specificity của trình duyệt, chúng ta cùng phân tích một ví dụ kinh điển với 10 quy tắc CSS cùng nhắm vào thẻ `<h2>`. Điểm Specificity sẽ leo thang liên tục từ 1 điểm đến 213 điểm qua từng cấp độ.

1. Cấu Trúc HTML Mẫu Thử Nghiệm

```html
<div id="main">
  <div id="head" class="head">
    <h2 class="title">Specificity Test</h2>
  </div>
</div>
```

2. Chuỗi 10 Quy Tắc CSS Leo Thang Specificity Tăng Dần

```css
/* Cap 1: The don le (0,0,0,1) -> ~1 pt */
h2 { color: red; }

/* Cap 2: Class + The (0,0,1,1) -> ~11 pts */
h2.title { color: blue; }

/* Cap 3: Them div cha (0,0,1,2) -> ~12 pts */
div h2.title { color: orange; }

/* Cap 4: Them class .head (0,0,2,1) -> ~21 pts */
.head h2.title { color: green; }

/* Cap 5: Class cha + Tag div (0,0,2,2) -> ~22 pts */
div.head h2.title { color: gray; }

/* Cap 6: Them Pseudo-class :last-child (0,0,3,2) -> ~32 pts */
div.head h2.title:last-child { color: yellow; }

/* Cap 7: Them 1 ID #main (0,1,2,2) -> ~122 pts */
#main div.head h2.title { color: pink; }

/* Cap 8: Them ID thu 2 #head (0,2,1,2) -> ~212 pts */
#main div#head h2.title { color: purple; }

/* Cap 9: Pseudo-element chu dau (0,2,1,3) -> ~213 pts */
#main div#head h2.title::first-letter { color: red; }

/* Cap 10: Pseudo-element it class (0,2,0,3) -> ~203 pts */
#main div#head h2::first-letter { color: yellow; }
```

[Bảng] 3. Bảng Kết Quả Phân Tích 10 Cấp Độ Leo Thang Specificity

Cấp	Selector CSS	Specificity	Kết quả & Phân tích lý do
1	`h2`	(0,0,0,1)	**Red** – Bị Cấp 2 đè vì Cấp 2 có thêm Class `.title`.
2	`h2.title`	(0,0,1,1)	**Blue** – Bị Cấp 3 đè vì Cấp 3 có thêm thẻ `div` cha.
3	`div h2.title`	(0,0,1,2)	**Orange** – Bị Cấp 4 đè vì Cấp 4 có 2 Class: $C{=}2 > C{=}1$.
4	`.head h2.title`	(0,0,2,1)	**Green** – Bị Cấp 5 đè vì Cấp 5 có thêm thẻ `div`.
5	`div.head h2.title`	(0,0,2,2)	**Gray** – Bị Cấp 6 đè vì `:last-child` cộng thêm 1C.
6	`div.head h2.title:last-child`	(0,0,3,2)	**Yellow** – Bị Cấp 7 đè bẹp vì ID `#main`: $B{=}1 > B{=}0$.
7	`#main div.head h2.title`	(0,1,2,2)	**Pink** – Bị Cấp 8 đè vì thêm ID thứ 2 `#head`: $B{=}2$.
8	`#main div#head h2.title`	(0,2,1,2)	**Purple [VÔ ĐỊCH H2]** – Điểm cao nhất nhắm vào thân `h2`!
9	`#main div#head h2.title::first-letter`	(0,2,1,3)	**Red [VÔ ĐỊCH CHỮ 'S']** – Nhắm vào `::first-letter`, thắng Cấp 10.
10	`#main div#head h2::first-letter`	(0,2,0,3)	**Yellow** – Thua Cấp 9 vì thiếu Class `.title`: $C{=}0 < C{=}1$.

[Khái Niệm] 4. Giải Đáp 3 Thắc Mắc Cốt Lõi Về Thuật Toán So Sánh

1. **Tại sao Cấp 4 (0,0,2,1) màu Green lại thắng Cấp 3 (0,0,1,2) màu Orange?**
   Khi so sánh hai điểm số $(0, 0, 2, 1)$ và $(0, 0, 1, 2)$, trình duyệt so sánh từ trái sang phải: Nhóm A bằng nhau (0=0), Nhóm B bằng nhau (0=0), đến Nhóm C: Cấp 4 có 2 Class $(C = 2)$, Cấp 3 chỉ có 1 Class $(C = 1)$. Vì $2 > 1$, Cấp 4 chiến thắng **mặc dù Cấp 3 có nhiều thẻ HTML hơn**!
2. **Tại sao Cấp 7 (0,1,2,2) màu Pink lại đánh bại Cấp 6 (0,0,3,2) màu Yellow?**
   Cấp 6 dù gom tới 3 Class/Pseudo-class (điểm $C = 3$), nhưng Cấp 7 xuất hiện 1 bộ chọn ID `#main` (điểm $B = 1$). Vì nhóm ID (B) nằm ở vị trí ưu tiên cao hơn nhóm Class (C), nên $B = 1$ lập tức đè bẹp $B = 0$ bất kể Cấp 6 có bao nhiêu Class đi nữa!
3. **Tại sao chữ 'S' lại có màu Red còn toàn bộ chữ 'pecificity Test' lại màu Purple?**
   - Cấp 8 nhắm vào toàn bộ thẻ `<h2>` và đạt điểm Specificity $(0, 2, 1, 2)$ cao nhất $\rightarrow$ Tô màu **Purple** cho thân thẻ.
   - Cấp 9 dùng pseudo-element `::first-letter` nhắm riêng vào chữ cái đầu tiên `'S'` với điểm $(0, 2, 1, 3)$ thắng Cấp 10 $(0, 2, 0, 3)$ $\rightarrow$ Tô màu **Red** riêng cho chữ 'S'.

BROWSER PREVIEW 5. Giao Diện Mô Phỏng Kết Quả Trình Duyệt Thực Tế

Specificity Test

Chữ 'S' màu Đỏ (Red) [Quy tắc 9 thắng]
Chữ 'pecificity Test' màu Tím (Purple) [Quy tắc 8 thắng]

9.6.7 7. Pseudo-class :not(), :is() Và :where() Trong CSS Modern

7.1. Pseudo-class :not() – Bản Thân Không Tính Điểm

[Khái Niệm] Quy Tắc Đặc Biệt Của :not()

- **Bản thân `:not()` KHÔNG được tính điểm specificity.** Nó là một khung vô hình.
- **Chỉ có nội dung bên trong ngoặc đơn mới được tính.** Ví dụ: `:not(.active)` cộng 1 class (`.active`) vào nhóm C.
- **Kết quả**: `div:not(.active)` có specificity = `(0,0,1,1)` – gồm 1 thẻ `div` (D) + 1 class `.active` (C).

7.2. Pseudo-class :is() – Lấy Specificity Của Đối Số Cao Nhất

[Khái Niệm] Quy Tắc Của :is()

`:is()` gom nhóm selector và lấy specificity của **đối số có điểm CAO NHẤT** trong danh sách. Ví dụ: `:is(.btn, #submit)` lấy specificity của `#submit` $(0, 1, 0, 0)$.

7.3. Pseudo-class :where() – Specificity Luôn Bằng 0

[Khái Niệm] Quy Tắc Của :where() – Zero Specificity

`:where()` luôn có **specificity = (0,0,0,0)**, bất kể nội dung bên trong là gì. Rất hữu ích để viết CSS mặc định trong thư viện dễ ghi đè.

[Bảng] Bảng So Sánh Nhanh :not() vs :is() vs :where()

Pseudo-class	Cách tính Specificity	Ví dụ	Specificity kết quả
`:not(.active)`	Lấy specificity của đối số bên trong	`div:not(.active)`	(0,0,1,1)
`:is(.btn, #id)`	Lấy specificity của đối số **cao nhất**	`:is(.btn, #submit)`	(0,1,0,0)
`:where(.btn, #id)`	**Luôn = 0**, bất kể nội dung	`:where(#main, .box)`	(0,0,0,0)

9.6.8 8. Sơ Đồ Tính Điểm Specificity Từng Bước (Step-by-Step)

Ví dụ minh họa: Tính specificity cho `div#main .btn:hover`

[Bảng] Sơ Đồ Tính Điểm Từng Bước Cho Selector: div#main .btn:hover

Bước	Thành phần	Phân loại	Nhóm	Cộng điểm
1	`div`	Thẻ HTML (Element)	D	+1 vào D
2	`#main`	ID Selector	B	+1 vào B
3	`.btn`	Class Selector	C	+1 vào C
4	`:hover`	Pseudo-class	C	+1 vào C
		Tổng cộng:	**A=0, B=1, C=2, D=1**	
		Specificity:	`(0, 1, 2, 1)`	

9.6.9 9. Kinh Nghiệm Thực Tế & Tối Ưu Kiến Trúc CSS

[Bảng] Bảng Quy Tắc "Nên & Không Nên" Trong Thực Tế Lập Trình Frontend

[KHÔNG NÊN] làm trong CSS	[NÊN] thực hiện (Best Practices)
Dùng ID làm selector chính trong CSS (`#header { background: blue; }`) làm code bị cứng, khó ghi đè.	**Sử dụng Class cho tất cả các bộ chọn (`.header { background: blue; }`) để linh hoạt và tái sử dụng.**
Lạm dụng `!important` để giải quyết xung đột CSS tạm thời, gây rác mã nguồn.	**Chỉ dùng `!important` cho các utility class đặc biệt (như `.d-none`) hoặc override thư viện ngoài.**
Lồng selector quá sâu: `body div.container section.main article.post p` gây tăng điểm specificity không cần thiết.	**Viết selector ngắn gọn, phẳng (Flat Selectors): `.post p` hoặc áp dụng phương pháp luận đặt tên BEM (`.post__text`).**

[Bảng] Bảng Mẹo Nhớ Nhanh Sức Mạnh Của Các Loại Selector (Memory Tips)

Mức độ Selector	Mẹo ghi nhớ dễ thuộc lòng
!important	**Nói là làm** – Luôn luôn chiến thắng mọi bộ chọn khác.
Inline Style (1,0,0,0)	**Gần nhất phần tử** – Viết trực tiếp trong thuộc tính `style="..."` trên thẻ HTML.
ID Selector (0,1,0,0)	**Định danh duy nhất** – Rất mạnh mẽ, định danh duy nhất cho phần tử.
Class / Attr / Pseudo-class (0,0,1,0)	**Chung nhóm** – Ổn định, linh hoạt, được sử dụng phổ biến nhất trong thực tế.
Element / Pseudo-element (0,0,0,1)	**Thẻ HTML** – Mức độ yếu nhất trong các loại bộ chọn trực tiếp.
Inheritance / Kế thừa (0,0,0,0)	**Mặc định** – Dễ bị bất kỳ bộ chọn trực tiếp nào ghi đè.

[Khái Niệm] Tổng Kết Quy Tắc Vàng Về Thứ Tự Ưu Tiên CSS

Công thức tổng quát: `Specificity = (A, B, C, D)` hoặc `Specificity = (a, b, c, d)`

Chuỗi thứ tự ưu tiên tuyệt đối từ cao xuống thấp (1 → 6):

1. **!important trong CSS** – Cấp quyền ưu tiên tuyệt đối (ghi đè mọi bộ chọn).
2. **Style nội tuyến (Inline style)** `(1,0,0,0)` – Đặt trực tiếp trong thuộc tính `style="..."`.
3. **Bộ chọn ID (#id)** `(0,1,0,0)` – Mức ưu tiên cao cho định danh duy nhất.
4. **Class, Attribute, Pseudo-class** `(0,0,1,0)` – Nhóm bộ chọn phổ biến (`.class`, `[attr]`, `:hover`).
5. **Thẻ HTML & Pseudo-element** `(0,0,0,1)` – Nhóm thẻ phần tử (`div`, `p`, `::before`).
6. **Kế thừa & Universal Selector** `(0,0,0,0)` – Kế thừa mặc định và bộ chọn chung (`*`) – *Yếu nhất*.

9.7 Thuộc Tính Opacity (Độ Mờ & Độ Trong Suốt Trong CSS)

Thuộc tính `opacity` trong CSS cho phép lập trình viên kiểm soát mức độ hiển thị, độ mờ và độ trong suốt của bất kỳ phần tử HTML nào trên giao diện web.

1. Opacity Là Gì?

[Khái Niệm] Khái Niệm & Giá Trị Của Thuộc Tính Opacity

Opacity xác định độ trong suốt của một phần tử HTML. Nó quyết định xem ánh sáng/màu nền phía sau phần tử có thể nhìn xuyên qua được hay không.

Dải giá trị chấp nhận từ 0 đến 1:

- **0** → Hoàn toàn trong suốt (phần tử vô hình nhưng vẫn chiếm vị trí không gian trên luồng tài liệu).
- **1** → Không trong suốt (hiển thị rõ ràng 100% như bình thường).
- **Giữa 0 và 1 (ví dụ 0.1 – 0.9)** → Mờ dần theo tỷ lệ phần trăm (ví dụ: `0.5` mờ 50%, `0.2` mờ 80%).

2. Cú Pháp Khai Báo CSS:

Cú Pháp Định Mức Độ Mờ Opacity

```
selector {
    opacity: value; /* Gia tri so tu 0.0 den 1.0 */
}
```

- **0**: Hoàn toàn trong suốt (ẩn hoàn toàn).
- **1**: Hoàn toàn không trong suốt (hiển thị rõ ràng).
- **Giữa 0 và 1**: Mờ dần theo mức độ (ví dụ: `0.5` là mờ 50%).

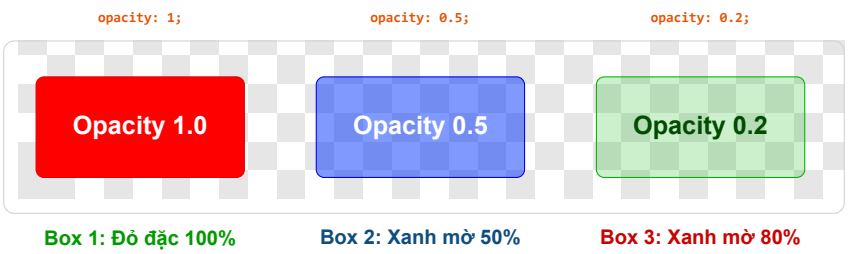
3. Ví Dụ Minh Họa & Kết Quả Trực Quan Trên Trình Duyệt:

Khai Báo 3 Mức Độ Mờ Cho Các Khối Div

```
<div class="box full-opacity">Opacity: 1</div>
<div class="box semi-opacity">Opacity: 0.5</div>
<div class="box low-opacity">Opacity: 0.2</div>

<style>
.box {
    width: 200px;
    height: 100px;
    margin: 10px;
    padding: 10px;
    color: white;
    text-align: center;
    line-height: 100px;
    font-size: 20px;
}
.full-opacity {
    background: #ff0000;
    opacity: 1; /* Khong mo */
}
.semi-opacity {
    background: #007bff;
    opacity: 0.5; /* Mo 50% */
}
.low-opacity {
    background: #28a745;
    opacity: 0.2; /* Mo 80% */
}
</style>
```

BROWSER PREVIEW **Kết Quả Trực Quan 3 Khối Box Trên Nền Ca-rô Trong Suốt Mờ Trình Duyệt**



4. Lưu Ý Quan Trọng Khi Sử Dụng Opacity:

[Cảnh Báo] 3 Cạm Bẫy Quan Trọng Cần Ghi Nhớ Với Opacity

1. **Opacity làm mờ CẢ phần tử lẫn tất cả nội dung con bên trong:** Khi bạn áp dụng `opacity` cho một thẻ cha, toàn bộ chữ, hình ảnh, nút bấm con bên trong thẻ cha đó **đều bị mờ theo**.

   Lỗi Tự Động Mờ Nội Dung Con

   ```
   .parent {
       background: purple;
       opacity: 0.5; /* Chu va nut bam con ben trong cung bi mo 50%! */
   }
   ```

2. **Phần tử vẫn chiếm không gian luồng tài liệu dù `opacity: 0`:** Phần tử không bị xóa khỏi bố cục trang mà chỉ bị biến thành vô hình. (Nếu muốn ẩn hoàn toàn và xóa khỏi luồng bố cục, hãy dùng `display: none;`).
3. **Không thể click tương tác khi phần tử vô hình (`opacity: 0`):** Mặc dù phần tử vẫn tồn tại nhưng người dùng không thể nhìn thấy để thao tác (tương tự như bị vô hình).

5. Thay Thế Bằng RGBA Để Chỉ Làm Mờ Nền (Giữ Chữ Rõ Nét):

Nếu bạn muốn **chỉ làm mờ màu nền** nhưng vẫn giữ nội dung văn bản và phần tử con hiển thị rõ nét 100%, hãy sử dụng giá trị màu `rgba()` thay vì thuộc tính `opacity`:

Sử Dụng RGBA Để Chỉ Mờ Màu Nền

```
.box {
    background: rgba(255, 0, 0, 0.5); /* Nen do mo 50%, van ban ben trong van ro
    100% */
}
```

[Khái Niệm] Cấu Trúc Giá Trị Màu RGBA (Red, Green, Blue, Alpha)

RGBA (Red, Green, Blue, Alpha):

- **R**: Giá trị kênh Đỏ (Red: $0 \rightarrow 255$).
- **G**: Giá trị kênh Xanh lá (Green: $0 \rightarrow 255$).
- **B**: Giá trị kênh Xanh dương (Blue: $0 \rightarrow 255$).
- **A**: Kênh Alpha độ trong suốt ($0 \rightarrow 1$).

Ưu điểm vượt trội của RGBA: Chỉ làm mờ riêng màu nền, nội dung văn bản và các phần tử con bên trong **hoàn toàn không bị ảnh hưởng**!

6. Các Ứng Dụng Thực Tế Phổ Biến Của Opacity:

[Khái Niệm] 4 Kịch Bản Ứng Dụng Opacity Trong Dự Án Web Thực Tế

1. **Hiệu ứng di chuột (Hover Effect):** Làm mờ nhẹ thẻ hoặc ảnh khi di chuột qua để báo hiệu khả năng nhấp chuột:

Làm Mờ Khi Di Chuột Di Vào Card

```
.card:hover {
    opacity: 0.7;
}
```

2. **Màn hình phủ tải trang (Loading Screen Overlay):** Làm mờ tối toàn bộ trang web khi hiển thị cửa sổ tải dữ liệu:

Tạo Màn Phủ Tối Overlay

```
.overlay {
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    background: rgba(0, 0, 0, 0.5);
}
```

3. **Hiệu ứng ảnh mờ dần / Hiện dần (CSS Fade Animation):**

Tạo Animation Hiện Dần Vừa Mờ Đến Rõ

```
@keyframes fade {
    0% { opacity: 0; }
    100% { opacity: 1; }
}
```

4. **Làm nổi bật & Vô hiệu hóa nút bấm (Disabled State):** Kết hợp `opacity` với `pointer-events: none` để vừa làm mờ vừa khóa tương tác nhấp chuột:

Vô Hiệu Hóa Thao Tác Click Vẫn Làm Mờ Nut

```
.disabled {
    opacity: 0.5;
    pointer-events: none; /* Vo hieu hoa moi tuong tac click */
}
```

7. Bảng Tổng Kết Trực Quan Kiến Thức Opacity:

[Bảng] Bảng Tổng Kết Tra Cứu Toàn Diện Về Thuộc Tính Opacity & Giải Pháp RGBA

Thuộc tính / Giá trị	Ý nghĩa & Hành vi hiển thị	Cú pháp ví dụ minh họa
`opacity: 1`	Phần tử hoàn toàn hiển thị (không mờ).	`opacity: 1;`
`opacity: 0.5`	Phần tử và mọi thẻ con mờ 50%.	`opacity: 0.5;`
`opacity: 0`	Phần tử hoàn toàn trong suốt (vẫn chiếm bố cục).	`opacity: 0;`
`pointer-events`	Kết hợp với opacity để vô hiệu hóa click chuột.	`pointer-events: none;`
`rgba(...)`	Chỉ làm mờ màu nền, giữ nguyên chữ con 100%.	`background: rgba(0, 0, 0, 0.5);`

[Khái Niệm] Tóm Lại Cốt Lõi Kiến Thức

- `opacity` là công cụ cực kỳ mạnh mẽ để tạo hiệu ứng mờ, lớp phủ overlay, và làm nổi bật nội dung giao diện.
- Thuộc tính này tác động đến **cả phần tử cha lẫn toàn bộ phần tử con**. Nếu chỉ muốn mờ màu nền, hãy sử dụng `rgba()` hoặc kết hợp `pointer-events` để kiểm soát trải nghiệm người dùng tốt nhất.

Mở Rộng Kiến Thức & Kinh Nghiệm Thực Tế Chuyên Sâu Về Opacity

[Khái Niệm] 1. Nguyên Lý Tạo Stacking Context Trực Tiếp Từ Opacity < 1

Trong cơ chế render của trình duyệt, khi một phần tử có thuộc tính `opacity < 1`, trình duyệt tự động tạo ra một **Ngữ cảnh xếp chồng mới (Stacking Context)** trên phần tử đó!

Hệ quả quan trọng: Thẻ con chứa `z-index: 9999` bên trong phần tử có `opacity: 0.99` sẽ **KHÔNG THỂ** ngoi lên trên các phần tử bên ngoài có z-index cao hơn thẻ cha! Đây là nguyên nhân khiến nhiều lập trình viên Junior loay hoay không hiểu tại sao z-index của thẻ con bị vô hiệu hóa.

[Bảng] So Sánh 3 Phương Pháp Ẩn Phần Tử HTML Trong CSS (opacity vs visibility vs display)

Tiêu chí so sánh	opacity: 0;	visibility: hidden;	display: none;
Chiếm không gian layout?	**CÓ** (Vẫn chiếm diện tích).	**CÓ** (Vẫn chiếm diện tích).	**KHÔNG** (Xóa khỏi luồng layout).
Click / Sự kiện DOM?	Vẫn nhận click ngoại trừ khi dùng `pointer-events: none`.	**KHÔNG** nhận sự kiện mouse/click.	**KHÔNG** nhận bất kỳ sự kiện nào.
Hỗ trợ CSS Transition / Animation?	**CÓ** (Mượt mà từ $0 \to 1$).	Hỗ trợ hạn chế (chỉ bật/tắt ở cuối).	**KHÔNG** (Biến mất tức thì, không transition).
Tăng tốc phần cứng GPU?	**CÓ** (Dùng GPU Composite Layer).	**KHÔNG**.	**KHÔNG**.

[Mẹo] 2. Tối Ưu Hiệu Năng Animation Bằng Tăng Tốc Phần Cứng GPU

`opacity` cùng với `transform` là hai thuộc tính CSS duy nhất được xử lý trực tiếp trên tầng **Composite Layer của GPU (Graphics Processing Unit)**. Khi thay đổi `opacity`, trình duyệt **KHÔNG cần phải tính toán lại Layout (Reflow) hay vẽ lại pixel (Repaint)**, giúp hiệu ứng đạt mượt mà chuẩn **60 FPS** ngay cả trên thiết bị di động cấu hình yếu.

[Cảnh Báo] 3. Bộ Câu Hỏi Phỏng Vấn Senior Frontend Về Opacity

1. **Câu hỏi 1: Làm thế nào để thẻ cha mờ nhưng thẻ con bên trong vẫn giữ nguyên opacity 100%?**
   → **Trả lời:** Không thể đặt `opacity` trên thẻ cha rồi chỉnh thẻ con `opacity: 1` vì opacity mang tính chất nhân dồn ($0.5 \times 1.0 = 0.5$). Giải pháp là dùng màu `rgba()` cho nền thẻ cha, hoặc tách thẻ con ra thành phần tử đồng cấp rồi xếp chồng bằng CSS `position: absolute`.
2. **Câu hỏi 2: Tại sao khi đổi opacity từ 1 về 0 lại không click được nếu dùng pointer-events: none?**
   → **Trả lời:** Thuộc tính `pointer-events: none` giúp phần tử hoàn toàn bỏ qua mọi sự kiện chuột, giúp các sự kiện click xuyên qua phần tử vô hình để tác động tới phần tử bên dưới.
3. **Câu hỏi 3: Sự khác biệt bản chất giữa opacity: 0 và display: none trong SEO và Accessibility (SEO & Màn hình đọc Screen Reader)?**
   → **Trả lời:** `display: none` ẩn hoàn toàn với cả người dùng lẫn Màn hình đọc (**Screen Reader**). Trong khi `opacity: 0` vẫn được trình đọc màn hình đọc dữ liệu văn bản bên trong cho người khiếm thị.

CHƯƠNG 10

Bố Cục Modern CSS: Flexbox & CSS Grid

10.1 Bố Cục Một Chiều Với CSS Flexbox

Các thuộc tính Container: `display: flex`, `flex-direction`, `justify-content`, `align-items`, `gap`. Thuộc tính Flex Items: `flex-grow`, `flex-shrink`, `flex-basis`.

10.2 Bố Cục Hai Chiều Với CSS Grid

Khai báo lưới `grid-template-columns`, `grid-template-rows`, kỹ thuật `repeat(auto-fit, minmax(250px, 1fr))` giúp responsive tự động không cần Media Query.

CHƯƠNG 11

Responsive Web Design Nâng Cao

11.1 Tư Duy Mobile-First

Xây dựng giao diện từ màn hình nhỏ nhất (Mobile), mở rộng dần sang Tablet và Desktop để tối ưu hiệu năng.

11.2 Fluid Typography & Container Queries

Sử dụng hàm CSS `clamp(1rem, 2.5vw, 2rem)` cho cỡ chữ linh hoạt và tính năng hiện đại **Container Queries**.

CHƯƠNG 12

CSS Variables & BEM Architecture

12.1 CSS Custom Properties (CSS Variables)

Khai báo biến toán cục trong `:root` để làm Dark Mode và Design System.

12.2 Phương Pháp Đặt Tên BEM (Block - Element - Modifier)

Tổ chức CSS sạch sẽ, tránh xung đột tên lớp bằng BEM (ví dụ: `.card__title--active`).

CHƯƠNG 13

CSS Animations & Transitions

13.1 Hiệu Ứng Chuyển Động Với CSS Transitions

Thuộc tính `transition`, `transition-timing-function` (`ease-in-out`, `cubic-bezier`).

13.2 Keyframe Animations

Viết animation phức tạp bằng `@keyframes` và biến đổi không gian 2D/3D `transform`.

PHẦN IV

JavaScript Modern – Ngôn Ngữ Xương Sống Frontend

CHƯƠNG 14

Cơ Cơ Hoạt Động Của JavaScript Engine

14.1 Execution Context & Call Stack

Bộ máy JavaScript (V8 Engine) xử lý mã nguồn qua hai pha: **Creation Phase** và **Execution Phase**.

14.2 Scope, Closure & Event Loop

Khái niệm Phạm vi (Lexical Scope), Closure trong JavaScript và mô hình xử lý bất đồng bộ đơn luồng với **Event Loop**, **Microtask Queue** (Promises) vs **Macrotask Queue** (`setTimeout`).

CHƯƠNG 15

Modern JavaScript ES6+

15.1 Khai Báo Biến: const, let vs var

So sánh pham vi block scope và hoisting.

15.2 Tính Năng ES6+ Đáng Giá

Destructuring, Spread/Rest Operator, Arrow Functions, Template Literals, ES Modules (`import/export`).

CHƯƠNG 16

Thao Tác DOM & Xử Lý Sự Kiện

Chương này đi sâu vào kiến trúc cây Document Object Model (DOM), các phương thức truy vấn phần tử, kỹ thuật thao tác giao diện động và cơ chế xử lý sự kiện nâng cao trong JavaScript hiện đại.

16.1 Tổng Quan Về DOM & Cấu Trúc Cây DOM Tree

Document Object Model (DOM) là giao diện lập trình ứng dụng (API) cho phép JavaScript truy cập, đọc, thay đổi cấu trúc, nội dung và định kiểu của tài liệu HTML.

[Khái Niệm] Khái Niệm Cốt Lõi Về Cây DOM Tree

Khi trình duyệt nạp một tệp HTML, nó phân tích (*parse*) mã nguồn và dựng thành một **cây nút (Node Tree)** trong bộ nhớ RAM:

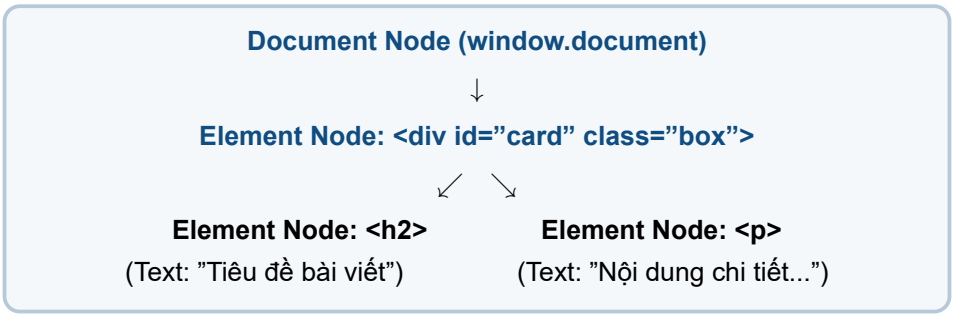
- **Document Node**: Nút gốc đại diện cho toàn bộ tài liệu web.
- **Element Node**: Các nút thẻ HTML (`<div>`, `<p>`, `<button>`).
- **Text Node**: Nút chứa nội dung văn bản bên trong thẻ.
- **Attribute Node**: Thuộc tính của phần tử (`id`, `class`, `src`).

16.1.1 Ví Dụ Mã Nguồn Cấu Trúc DOM

Mã Nguồn: Cấu Trúc Thẻ HTML Minh Họa DOM Tree

```
<div id="card" class="box">
  <h2>Tiêu đề bài viết</h2>
  <p>Nội dung chi tiết...</p>
</div>
```

BROWSER PREVIEW Kết Quả Trình Duyệt: Minh Họa Mô Hình Phân Cấp Cây DOM



16.2 DOM Manipulation API Masterclass

Để thao tác với bất kỳ phần tử nào trên trang web, trước hết trình duyệt cần xác định được vị trí phần tử đó thông qua các API truy vấn.

16.2.1 1. Các Phương Thức Truy Vấn Phần Tử (DOM Selection)

[Bảng] Bảng So Sánh Các API Truy Vấn DOM

Phương Thức Truy Vấn	Kiểu Dữ Liệu Trả Về	Đặc Điểm & Hiệu Năng
`document.getElementById()`	Single Element (hoặc `null`)	Truy vấn theo ID duy nhất. Tốc độ thực thi nhanh nhất.
`document.querySelector()`	Single Element (đầu tiên trùng khớp)	Dùng CSS Selector (`.class`, `#id`, `[attr]`). Rất linh hoạt.
`document.querySelectorAll`	`NodeList` tĩnh (*Static*)	Trả về danh sách tất cả phần tử khớp CSS Selector. Hỗ trợ `.forEach()`.
`getElementsByClassName()`	`HTMLCollection` động (*Live*)	Tự động cập nhật khi DOM thay đổi. Không hỗ trợ `.forEach()` trực tiếp.

Bảng 16.1: So sánh các API chọn phần tử DOM trong JavaScript

16.2.2 2. Ví Dụ Mã Nguồn Truy Vấn & Thao Tác Nội Dung

Mã Nguồn JavaScript: Truy Vấn Và Thay Đổi Nội Dung HTML

```
// 1. Truy van phan tu qua ID va CSS Selector
const mainTitle = document.querySelector('#main-title');
const statusBadge = document.querySelector('.badge');
const actionBtn = document.getElementById('btn-update');

// 2. Thao tac noi dung text va HTML
mainTitle.textContent = "Học Lập Trình DOM Masterclass 2026";
statusBadge.innerHTML = "<strong>Trạng Thái: Đã Cập Nhật</strong>";

// 3. Thay doi Style va Class bang JS
statusBadge.classList.remove('badge-pending');
statusBadge.classList.add('badge-success');
actionBtn.style.backgroundColor = '#16A34A';
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Giao Diện Sau Khi JS Thao Tác DOM**



→ **Kết quả trực quan**: Tiêu đề được đổi chữ mới, Badge đổi màu xanh lá lá nhờ class `badge-success`, và nút bấm đổi màu nền qua thuộc tính `style.backgroundColor`.

16.2.3 3. Tạo, Chèn Và Xóa Phần Tử Động (Dynamic DOM Node Building)

Trong các ứng dụng Web hiện đại, dữ liệu thường được tải từ Server (qua Fetch API/AJAX) và render động ra giao diện.

Mã Nguồn: Tạo Danh Sách Công Việc To-Do Động Với DocumentFragment

```
const todoData = ["Học HTML5 Semantics", "Thực hành CSS Grid", "Làm chủ DOM Events"];
const listContainer = document.getElementById('todo-list');

// Dùng DocumentFragment để tối ưu hiệu năng (Tránh reflow nhiều lần)
const fragment = document.createDocumentFragment();

todoData.forEach((taskText, index) => {
  const li = document.createElement('li');
  li.className = 'todo-item';
  li.innerHTML = `<span>${index + 1}. ${taskText}</span> <button class="btn-del">Xóa</button>`;
  fragment.appendChild(li);
});

listContainer.appendChild(fragment);
```

BROWSER PREVIEW Kết Quả Hiển Thị Trình Duyệt: Danh Sách Công Việc Render Từ JavaScript



16.3 Cơ Chế Sự Kiện: Bubbling, Capturing & Delegation

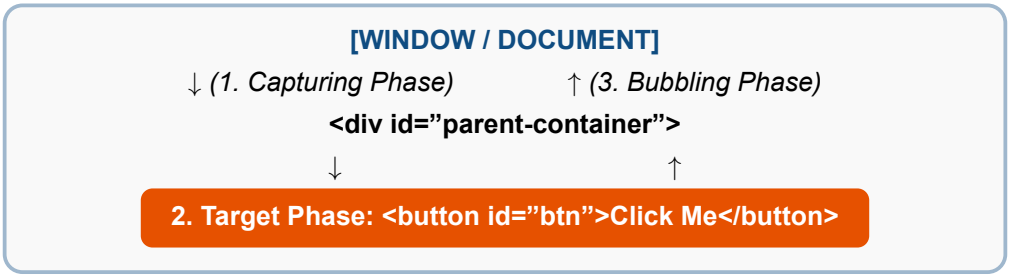
Sự kiện (**Events**) là các tín hiệu do trình duyệt hoặc người dùng kích hoạt (như nhấp chuột `click`, gõ phím `keydown`, cuộn trang `scroll`, gửi biểu mẫu `submit`).

16.3.1 1. Luồng Truyền Sự Kiện (Event Propagation Flow)

Khi một sự kiện xảy ra trên một phần tử con, sự kiện đó trải qua 3 giai đoạn:

1. **Giai Đoạn Bắt Sự Kiện (Event Capturing Phase)**: Sự kiện đi từ `window` down qua các phần tử cha tới phần tử đích.
2. **Giai Đoạn Tại Đích (Target Phase)**: Sự kiện kích hoạt ngay tại phần tử mục tiêu được nhấp chuột (`e.target`).
3. **Giai Đoạn Nổi Mọt (Event Bubbling Phase)**: Sự kiện ”nổi ngược” từ phần tử mục tiêu lên lại các phần tử cha đến tận `window`.

BROWSER PREVIEW **Kết Quả Trình Duyệt: Mô Phỏng Sơ Đồ Luồng Truyền Sự Kiện**



16.3.2 2. Kỹ Thuật Uỷ Quyền Sự Kiện (Event Delegation Masterclass)

[Mẹo] Tối Ưu Hiệu Năng Với Event Delegation

Thay vì gắn hàng trăm Event Listener lên từng nút bấm con trong một danh sách lớn, hãy gắn **duy nhất 1 Event Listener** lên phần tử cha chung và kiểm tra `e.target.matches()`!

Mã Nguồn JavaScript: Triển Khai Kỹ Thuật Event Delegation

```
const tableBody = document.querySelector('#user-table tbody');

// Gắn 1 listener duy nhất trên thẻ tbody cha
tableBody.addEventListener('click', function(e) {
  // Kiểm tra xem người dùng có nhấp vào nút Xoá hay không
  if (e.target.classList.contains('btn-delete-user')) {
    const userId = e.target.getAttribute('data-id');
    const row = e.target.closest('tr');

    row.remove(); // Xóa hàng khỏi DOM
    console.log(`Đã xóa người dùng có ID: ${userId}`);
  }
});
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Thao Tác Bảng Sử Dụng Event Delegation**

ID	Họ và Tên	Vai Trò	Hành Động
1	Nguyễn Văn A	Frontend Developer	Xóa User
2	Trần Thị B	UI/UX Designer	Xóa User

→ **Nhật ký Console**: `Đã xóa người dùng có ID: 1` (Hàng 1 tự động biến mất khỏi bảng mượt mà).

16.4 Tóm Tắt & Bộ Câu Hỏi Phỏng Vấn DOM & Events

- DOM là cấu trúc cây đối tượng biểu diễn toàn bộ tài liệu HTML trong bộ nhớ.
- Ưu tiên sử dụng `querySelector` / `querySelectorAll` cho sự linh hoạt và `getElementById` cho hiệu năng tối đa.
- Dùng `DocumentFragment` khi cần chèn nhiều phần tử liên tiếp để tối ưu số lần Render/Reflow.
- Hiểu rõ Event Bubbling và ứng dụng kỹ thuật Event Delegation để quản lý bộ nhớ hiệu quả.

Chương này đã hoàn thành việc trang bị toàn bộ kỹ thuật thao tác DOM và cơ chế sự kiện chuyên sâu cho ứng dụng Frontend.

CHƯƠNG 17

Lập Trình Bất Đồng Bộ: Promises & Async/Await

17.1 Từ Callback Hell Đến Promises

Khái niệm Promise states (*Pending, Fulfilled, Rejected*) và xử lý chuỗi `.then().catch()`.

17.2 Cú Pháp Hiện Đại Async/Await & Fetch API

Viết mã bất đồng bộ trông như đồng bộ với `async/await` và gọi HTTP API bằng `fetch()`.

CHƯƠNG 18

Lưu Trữ Trình Duyệt & Web APIs

18.1 Web Storage API: LocalStorage & SessionStorage

Lưu trữ key-value trên browser, sự khác biệt về vòng đời dữ liệu.

18.2 Cookies & IndexDB Overview

Quản lý trạng thái đăng nhập với HTTP Cookies và cơ sở dữ liệu NoSQL trình duyệt IndexedDB.

PHẦN V

Dự Án Thực Chiến & Hệ Sinh Thái Frontend

CHƯƠNG 19

Build Tools & Package Managers

19.1 Package Managers: npm, yarn & pnpm

Quản lý các thư viện ngoài và file `package.json`, `package-lock.json`.

19.2 Modern Build Tool: Vite

Tại sao Vite lại nhanh vượt trội so với Webpack nhờ sức mạnh của Native ES Modules và esbuild.

CHƯƠNG 20

Kiến Trúc Frontend Modern & Frameworks Overview

20.1 Single Page Application (SPA) vs Multi Page Application (MPA)

So sánh mô hình nạp trang tĩnh truyền thống và ứng dụng đơn trang hiện đại.

20.2 Kiến Trúc Dựa Trên Component (Component-Based Architecture)

Tổng quan hệ sinh thái React, Vue.js, Angular và tư duy chia nhỏ UI thành các Component độc lập.

CHƯƠNG 21

Dự Án Thực Chiến Frontend Từ A-Z

Chương này hướng dẫn xây dựng trọn vẹn một ứng dụng Web thực tế: **Dashboard Quản Lý Khóa Học Trực Tuyến (EduWeb Admin Dashboard)** từ khâu phân tích bố cục, dựng HTML5 ngữ nghĩa, thiết kế CSS3 nâng cao đến xử lý logic JavaScript tương tác và Dark Mode.

21.1 Phân Tích Yêu Cầu & Thiết Kế Bố Cục Dự Án

21.1.1 1. Cấu Trúc Thư Mục Dự Án Chuẩn Thực Tế

Cấu Trúc Thư Mục Dự Án Frontend Chuyên Nghiệp

```
eduweb-dashboard/├──
 index.html              # Trang chu giao dien Dashboard├──
 assets/│
   ├── css/│
   │   ├── main.css        # CSS Design System & Variables│
   │   └── dashboard.css   # Dynamic UI styles│
   ├── js/│
   │   ├── app.js          # Main Application Entry Point│
   │   └── courses.js      # Data Manager & Event Handlers│
   └── images/│
       └── logo.svg└──
 README.md
```

21.1.2 2. Bố Cục Giao Diện Kiến Trúc Grid & Flexbox

[Khái Niệm] Kiến Trúc Giao Diện 2 Cột (Sidebar + Main Area)

- **Thanh Bên (Sidebar <aside>)**: Chứa Logo thương hiệu, Menu điều hướng chính và Nút đổi chế độ Sáng/Tối (**Dark Mode Toggle**).
- **Khu Vực Chính (<main>)**: Chứa Header tìm kiếm, Bộ lọc khóa học theo danh mục và Lưới sản phẩm (**Course Card Grid**).

21.2 Triển Khai Mã Nguồn HTML, CSS & JavaScript

21.2.1 1. Mã Nguồn Cấu Trúc HTML5 Masterclass

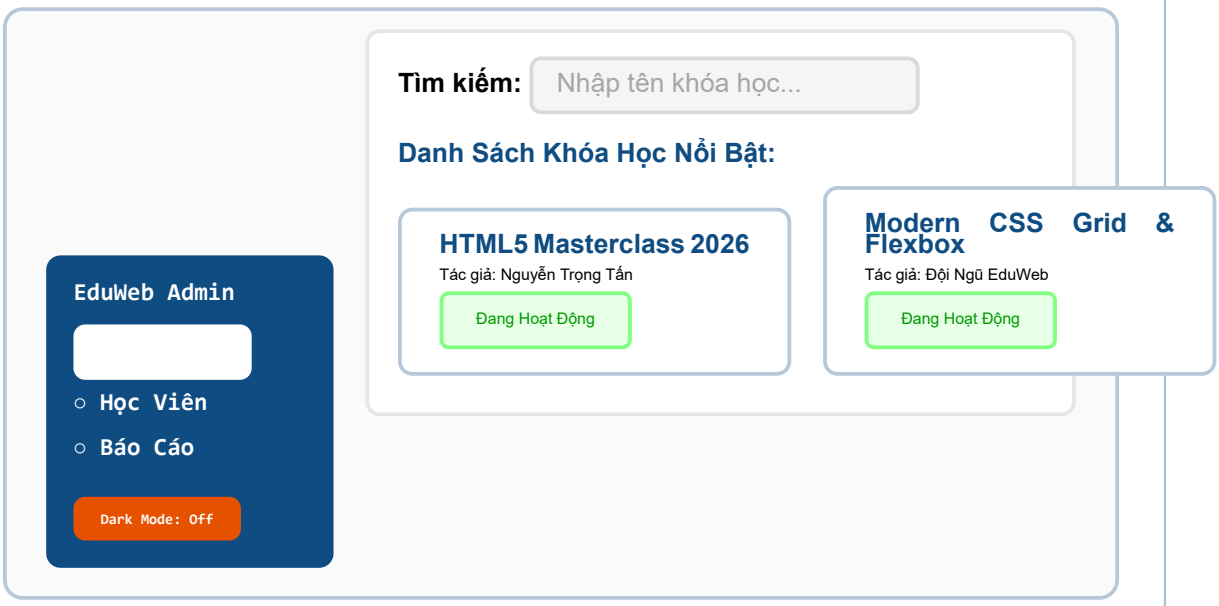
Mã Nguồn HTML5: Khung Xương Giao Diện Dashboard

```
<div class="dashboard-wrapper">
  <!-- Thanh bên Sidebar -->
  <aside class="sidebar">
    <div class="logo">EduWeb Admin</div>
    <nav class="sidebar-nav">
      <a href="#" class="active">Khóa Học</a>
      <a href="#">Học Viên</a>
      <a href="#">Báo Cáo</a>
    </nav>
    <button id="dark-mode-btn">Đổi Giao Diện</button>
  </aside>

  <!-- Khu vực nội dung chính -->
  <main class="main-content">
    <header class="top-bar">
      <search>
        <input type="search" id="search-input" placeholder="Tìm kiếm khóa
    học...">
      </search>
    </header>

    <section class="course-grid" id="course-list">
      <!-- Dynamic render by JS -->
    </section>
  </main>
</div>
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Mô Phỏng Giao Diện Dashboard Tổng Thể**



→ **Giải thích kết quả**: Bố cục 2 cột hoàn chỉnh với Sidebar bên trái chứa nút đổi giao diện và Main Area bên phải chứa thanh tìm kiếm cùng các Card khóa học render động.

21.2.2 2. Triển Khai Logic JavaScript Tương Tác (Search & Dark Mode)

Mã Nguồn JavaScript: Tìm Kiếm Thời Gian Thực & Chế Độ Dark Mode

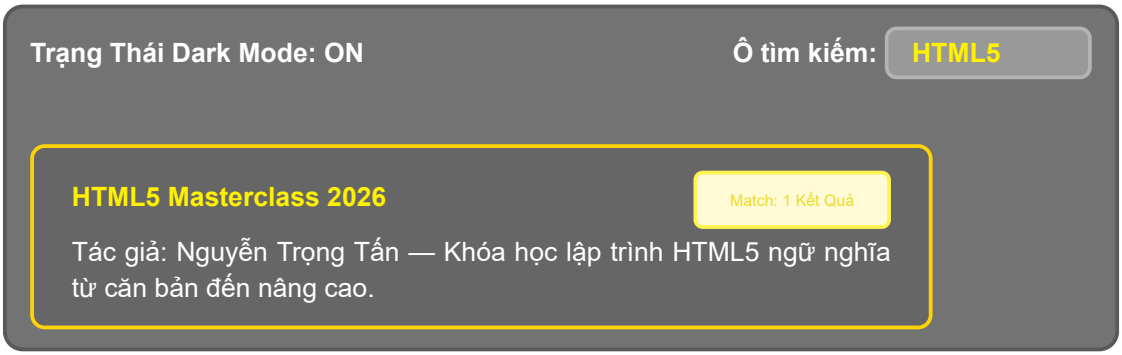
```javascript
// 1. Tinh nang tim kiem khoa hoc nhanh (Real-time Filter)
const searchInput = document.getElementById('search-input');
const courseCards = document.querySelectorAll('.course-card');

searchInput.addEventListener('input', function(e) {
  const searchTerm = e.target.value.toLowerCase().trim();

  courseCards.forEach(card => {
    const titleText = card.querySelector('.course-title').textContent.toLowerCase
    ();
    if (titleText.includes(searchTerm)) {
      card.style.display = 'block'; // Hiển thị card trùng khớp
    } else {
      card.style.display = 'none';  // Ẩn card không khớp
    }
  });
});

// 2. Tinh nang chuyen doi Dark Mode luu vao localStorage
const darkModeBtn = document.getElementById('dark-mode-btn');
darkModeBtn.addEventListener('click', () => {
  document.body.classList.toggle('dark-theme');
  const isDark = document.body.classList.contains('dark-theme');
  localStorage.setItem('theme', isDark ? 'dark' : 'light');
});
```

BROWSER PREVIEW **Kết Quả Hiển Thị Trình Duyệt: Mô Phỏng Khi Tìm Kiếm Từ Khóa ”HTML5”**



→ **Giải thích kết quả**: Khi gõ từ khóa ”HTML5”, JavaScript ngay lập tức lọc các card trùng khớp và giao diện tự động bật tone màu tối mượt mà nhờ Dark Mode!

21.3 Tóm Tắt & Checklist Đóng Gói Đưa Lên Production

- Đảm bảo mã HTML5 passing kiểm thử chuẩn W3C Validator không có lỗi đóng thẻ.
- Tối ưu dung lượng hình ảnh WebP/SVG và bật lazy loading cho các ảnh thumbnail.
- Kiểm thử giao diện Responsive trên màn hình di động 320px đến màn hình 4K.
- Kiểm tra điểm số Lighthouse đạt trên 90 điểm cho 4 tiêu chí: Performance, Accessibility, Best Practices và SEO.

Dự án thực chiến này đã tổng kết thành công toàn bộ hành trình học tập từ Nền tảng HTML5, CSS3 đến JavaScript Modern.